ชื่อหนังสือ MicroPython ภาษาระดับสูง ที่ง่ายสำหรับทุกคน

ผู้แต่ง/ผู้เขียน มานพ ปักษี

ออกแบบปก มานพ ปักษี

จัดรูปเล่ม มานพ ปักษี

สงวนลิขสิทธิ์ตามพระราชบัญญัติลิขสิทธิ์

ห้ามคัดลอกเนื้อหาของหนังสือเล่มนี้ไปเพื่อการค้าโดยเด็ดขาด

เว้นแต่จะได้รับอนุญาตเป็นลายลักษณ์อักษรจากผู้เขียนเท่านั้น

พิมพ์ครั้งที่ จัดพิมพ์ตามสั่ง หรือตามใจผู้เขียน

จำนวนหน้า 253 หน้า ไม่รวม สารบัญ และ คำนำ

เล่มนี้เป็นงานพิมพ์ด้วยเครื่องอิงค์เจท เข้าเล่มด้วยมือ (งานแฮนด์เมด)

ปกเคลือบสติ๊กเกอร์ใส ควรเก็บให้ห่างจากความร้อน และ ความชื้น

ข้อควรระวัง ไม่ควรให้โดนน้ำ

จัดพิมพ์โดย มานพ ปักษี

จัดจำหน่ายโดย มานพ ปักษี

ร้าน ManopLAB โทรศัพท์ 093-9749793

ที่อยู่ 179 หมู่ 13 ต.สารจิตร อ.ศรีสัชนาลัย จ.สุโขทัย รหัสไปรษณีย์ 64130

nopelectronic@gmail.com

ไฟล์โครงงานต่าง ๆ และเทคนิคพิเศษอีกมากมาย จะอัปเดตลงในเว็บไซต์

manoplab.com เร็ว ๆ นี้

เสมือนคำนำ (เรื่องเล่าก่อนเริ่มต้น)

หลายคนอาจเคยมีความฝันที่จะสร้างอุปกรณ์อิเล็กทรอนิกส์อัจฉริยะด้วยตนเอง ไม่ว่าจะเป็นระบบรดน้ำต้นไม้อัตโนมัติ, ไฟส่องสว่างที่เปิดปิดตามความเคลื่อนไหว, หรือสถานีตรวจวัดสภาพอากาศขนาดเล็กที่สามารถส่งข้อมูลขึ้นสู่อินเทอร์เน็ตได้ หากแนวคิดเหล่านี้สร้างความตื่นเต้น แต่ก็ตามมาด้วยความกังวลว่าการเริ่มต้นอาจเป็นเรื่องยากเกินไป หนังสือเล่มนี้ได้มอบแนวทางและคำตอบไว้ให้ทุกท่านแล้ว

ลองจินตนาการถึงการเขียนโปรแกรมควบคุมอุปกรณ์อิเล็กทรอนิกส์ต่าง ๆ ด้วยภาษา Python ซึ่งเป็นภาษาที่ขึ้นชื่อในด้านความเรียบง่าย โครงสร้างชัดเจน อ่านเข้าใจง่าย และได้รับความนิยมอย่างกว้างขวางในหมู่นักพัฒนาทั่วโลก นั่นคือสิ่งที่ MicroPython ได้เข้ามาสานต่อ โดยนำภาษา Python มาปรับแต่งให้สามารถทำงานได้บนไมโครคอนโทรลเลอร์ ซึ่งเป็นหัวใจสำคัญของระบบฝังตัว (Embedded Systems) มากมายในชีวิตประจำวันและเมื่อ MicroPython ถูกนำมาใช้งานร่วมกับ ESP32 ชิปไมโครคอนโทรลเลอร์ยอดนิยม ซึ่งมีทั้งประสิทธิภาพสูง มาพร้อมกับ Wi-Fi และ Bluetooth ที่อยู่ในชิพ ในราคาที่คนทั่วไปเข้าถึงได้ และมีผู้ใช้งานอย่างแพร่หลายทั่วโลก การผสมผสานนี้จึงก่อให้เกิดเป็นคู่หูที่ทรงพลังสำหรับการสร้างสรรค์โครงงานอิเล็กทรอนิกส์และ IoT (Internet of Things) ที่หลากหลาย จากผู้คนทั่วทุกมุมโลก

สำหรับผู้ที่คุ้นเคยกับการเขียนโปรแกรมบน Arduino ด้วยภาษา C/C++ มาก่อน อาจรู้สึกว่าการทดลองพัฒนาโค้ดแต่ละครั้งต้องผ่านกระบวนการคอมไพล์และอัปโหลดโปรแกรมไปยังบอร์ดใหม่ทุกครั้ง ซึ่งทำให้เสียเวลาในการทดสอบมาก โดยเฉพาะเมื่อทำงานกับโค้ดขนาดใหญ่หรือวงจรที่มีความซับซ้อน MicroPython เข้ามาเปลี่ยนประสบการณ์เหล่านี้ ด้วยความยืดหยุ่นของภาษา Python ผู้เรียนสามารถทดลองโค้ดได้ทันทีผ่านระบบ **REPL (Read-Evaluate-Print Loop)** ซึ่งช่วยให้มองเห็นผลลัพธ์ของการสั่งงานกับฮาร์ดแวร์แบบเรียลไทม์โดยไม่ต้องผ่านขั้นตอนการคอมไพล์และอัปโหลดซ้ำในทุกครั้ง โค้ดที่เขียนมีความกระชับ อ่านเข้าใจง่าย ลดภาระความซับซ้อนด้านไวยากรณ์ เหมาะอย่างยิ่งสำหรับผู้ที่ต้องการสำรวจเทคโนโลยีใหม่ ๆ หรือกำลังมองหาเครื่องมือที่ช่วยให้การพัฒนาโครงงานเป็นไปอย่างสนุกสนานและมีประสิทธิภาพมากยิ่งขึ้น

หนังสือเล่มนี้ไม่ได้มุ่งหวังเพียงการสอนวิธีทำให้ไฟ LED กะพริบ (แม้ว่านั่นจะเป็นจุดเริ่มต้นการเรียนรู้ที่ดีก็ตาม) แต่เนื้อหาจะนำพาผู้อ่านเดินทางไปไกลกว่านั้น โดยจะเริ่มปูพื้นฐานตั้งแต่การทำความเข้าใจ MicroPython และ ESP32, การติดตั้งเครื่องมือที่จำเป็น, การเรียนรู้คำสั่ง Python พื้นฐานในบริบทของการควบคุมฮาร์ดแวร์, ไปจนถึงการเชื่อมต่อ ESP32 เข้ากับเซ็นเซอร์ต่างๆ, การแสดงผล, การควบคุมมอเตอร์ และที่สำคัญคือการเชื่อมต่อกับเครือข่าย Wi-Fi เพื่อสร้างโครงงาน IoT ที่สามารถโต้ตอบกับโลกออนไลน์ได้จริง

เป้าหมายของหนังสือเล่มนี้

หนังสือเล่มนี้ถูกเรียบเรียงขึ้นด้วยความตั้งใจที่จะเป็น "คู่มือ" และ "แหล่งเรียนรู้" สำหรับ

- **ผู้เริ่มต้นใหม่ ที่อาจยังไม่มีประสบการณ์ด้านอิเล็กทรอนิกส์หรือการเขียนโปรแกรม**
- **ผู้ใช้งาน Arduino ที่ต้องการขยายขอบเขตความรู้และทดลองใช้ MicroPython**
- **นักเรียน นักศึกษา และผู้สนใจทั่วไป ที่มีความใฝ่ฝันจะเรียนรู้การสร้างโครงงาน IoT ด้วยตนเอง**
- **ครูอาจารย์ ที่กำลังมองหาแนวทางและสื่อการสอนสำหรับวิชาที่เกี่ยวข้อง**

เนื้อหาในหนังสือมุ่งเน้นการนำเสนอที่ เข้าใจง่าย เป็นขั้นตอน มีตัวอย่างที่ชัดเจน และ รวบรวมความรู้ที่จำเป็นทั้งภาคทฤษฎีและปฏิบัติ เพื่อให้ผู้เรียนสามารถศึกษาและลงมือทำตามได้จริง จนสามารถสร้างสรรค์โครงงานของตนเองได้อย่างมั่นใจ ทั้งหมดนี้ได้ถูกถ่ายทอดเป็นภาษาไทยที่กระชับและเข้าใจง่าย โดยคนไทย เพื่อคนไทย

การเดินทางสู่การสร้างสรรค์โครงงาน IoT ด้วย MicroPython และ ESP32:

ภายในหนังสือเล่มนี้ ผู้อ่านจะได้เรียนรู้ตามลำดับขั้นดังนี้ :

1. **การตั้งค่าสภาพแวดล้อม และเครื่องมือที่จำเป็นสำหรับการพัฒนา**
2. **พื้นฐานภาษา Python ที่สำคัญและประยุกต์ใช้บ่อยใน MicroPython**
3. **การควบคุมขา GPIO ของ ESP32 เพื่อสั่งงาน LED, อ่านค่าจากสวิตช์ และอื่นๆ**
4. **การทำงานกับ เซ็นเซอร์ ที่นิยมใช้ เช่น เซ็นเซอร์วัดอุณหภูมิ, ความชื้น, ระยะทาง**
5. **การ เชื่อมต่อ Wi-Fi และพื้นฐานการสื่อสารผ่านเครือข่าย**
6. **โครงงานตัวอย่าง ที่นำความรู้ทั้งหมดมาประยุกต์ใช้ เช่น สถานีตรวจอากาศส่วนบุคคล, ระบบควบคุมไฟอัจฉริยะ...**

ผู้เขียนเชื่อว่าเมื่อผู้อ่านศึกษาหนังสือเล่มนี้จบ จะไม่เพียงแต่เกิดความเข้าใจในหลักการทำงานของ MicroPython และ ESP32 แต่ยังจะได้รับแรงบันดาลใจและความพร้อมที่จะสร้างสรรค์สิ่งประดิษฐ์และนวัตกรรมใหม่ ๆ ออกมาอีกมากมาย ถึงเวลาแล้วที่จะปลดล็อกศักยภาพของไมโครคอนโทรลเลอร์ด้วยความมหัศจรรย์และความเรียบง่ายของ Python มาเริ่มต้นการเดินทางที่น่าตื่นเต้นนี้ไปด้วยกัน

มานพ ปักษี

สารบัญ

บทที่ 1: MicroPython และ ESP32 คืออะไร? ทำไมต้องใช้คู่นี้? 1

1.1 ยินดีต้อนรับสู่โลกแห่งไมโครคอนโทรลเลอร์ยุคใหม่ 1

1.2 MicroPython: Python ในร่างจิ๋วแต่ทรงพลัง 1

1.3 ESP32: ชิปมหัศจรรย์สำหรับโครงงาน IoT 2

1.4 MicroPython + ESP32: คู่หูที่ลงตัวที่สุดสำหรับการเริ่มต้น 3

1.5 เปรียบเทียบสำหรับผู้ใช้ Arduino: ก้าวใหม่ที่น่าลอง 4

1.6 MicroPython กับ ESP32 ในมุมมองของครูอาจารย์ 6

1.7 บทสรุป: เตรียมพร้อมสำหรับการเดินทาง 6

บทที่ 2: ติดตั้งและเริ่มต้นใช้งาน MicroPython บน ESP32 7

2.1 การเลือกบอร์ด ESP32 สำหรับมือใหม่ 7

2.2 เตรียมความพร้อมก่อนเริ่มต้น 8

2.3 การติดตั้งไดรเวอร์ USB to UART (ถ้าจำเป็น) 8

2.4 การติดตั้งเฟิร์มแวร์ MicroPython ลงบน ESP32 9

2.4.1 ดาวน์โหลดเฟิร์มแวร์ MicroPython 9

2.4.2 ติดตั้ง Python และ esptool.py 12

2.4.3 การเชื่อมต่อบอร์ดและค้นหาพอร์ต 15

2.4.4 การลบข้อมูลใน Flash (Erasing the flash) 15

การลบข้อมูลใน Flash ของ ESP32 ก่อนแฟลชเฟิร์มแวร์ MicroPython 22

2.4.5 การเขียนเฟิร์มแวร์ (Writing the firmware) 23

2.5 Thonny IDE: เพื่อนคู่ใจนักพัฒนา MicroPython 28

2.5.1 การติดตั้ง Thonny IDE 30

2.5.2 การตั้งค่า Thonny สำหรับ ESP32 31

2.6 โปรแกรมแรก: "สวัสดี ESP32!" และการควบคุม LED ออนบอร์ด....33
2.6.1 การใช้งาน REPL เบื้องต้น....33
2.6.2 การเขียนและบันทึกไฟล์สคริปต์....35
2.7 REPL: เครื่องมือทดลองโค้ดอันทรงพลัง....36
2.8 บทสรุปและสิ่งที่รออยู่ในบทถัดไป....37
บทที่ 3: กำเนิดและโครงสร้างของภาษา Python (สำหรับ MicroPython)....41
3.1 กำเนิดของภาษา Python....41
3.2 โครงสร้างของภาษา Python....41
3.2.1 ความแตกต่างระหว่าง Shell (REPL) และ Editor ใน Thonny....42
3.3 จุดเด่นของภาษา Python ที่ควรเข้าใจ....46
3.4 การใช้งาน Python กับฮาร์ดแวร์....51
บทที่ 4: เรียนรู้โครงสร้างภาษา Python เพื่อควบคุมอุปกรณ์จริง....52
4.1 ตัวแปร (Variables) และชนิดข้อมูลพื้นฐาน (Data Types)....52
กฎการตั้งชื่อตัวแปรใน Python (และ MicroPython)....53
ตัวดำเนินการทางคณิตศาสตร์ (Arithmetic Operators) ใน MicroPython บน ESP32....63
ตัวดำเนินการเปรียบเทียบ (Comparison Operators) ใน MicroPython บน ESP32....68
ตัวดำเนินการตรรกะ (Logical Operators) ใน MicroPython บน ESP32....71
ตัวดำเนินการกำหนดค่า (Assignment Operators) ใน MicroPython บน ESP32....75
ตัวดำเนินการกำหนดค่า (Assignment Operators) กับการควบคุมระบบฝังตัว....79
ตัวดำเนินการบิต (Bitwise Operators) ใน MicroPython บน ESP32....80
ตัวดำเนินการสมาชิก (Membership Operators) ใน MicroPython บน ESP32....83
นิพจน์ (Expressions) และลำดับการคำนวณ (Order of Evaluation) ใน MicroPython บน ESP32....86
คำสั่งควบคุมการทำงาน ใน MicroPython บน ESP32....90

คำสั่งเงื่อนไข (Conditional Statements) ใน MicroPython 91
การควบคุมลำดับการทำงานของลูป: break และ continue 102
คำสั่ง try/except – จัดการข้อผิดพลาด 107
คำสั่ง raise – การสร้างข้อผิดพลาดด้วยตนเอง 110
คำสั่ง finally – ทำงานเสมอไม่ว่าข้อผิดพลาดจะเกิดหรือไม่ 112
คำสั่ง with – การจัดการทรัพยากรอัตโนมัติอย่างปลอดภัย 114
3.5 ฟังก์ชัน (Functions) 117
ความสำคัญของฟังก์ชัน 117
การสร้างฟังก์ชัน (Defining Functions) 118
3.5.3 การคืนค่าด้วย return (Return Values) 123
3.5.4 ฟังก์ชันแบบไม่รับค่าหรือไม่คืนค่า 127
ฟังก์ชันแบบไม่รับค่า (No Parameter Function) 127
ฟังก์ชันแบบไม่คืนค่า (No Return Value Function) 128
ฟังก์ชันที่ไม่รับค่าและไม่คืนค่าเลย 129
3.5.5 Lambda Functions (ฟังก์ชันนิพจน์) 130
โครงสร้างและรูปแบบของ Lambda Function 131
ตัวอย่างการใช้งาน Lambda Function กับรายการข้อมูล 131
การประยุกต์ Lambda Function กับงานควบคุมฮาร์ดแวร์บน ESP32 132
3.5.6 ฟังก์ชันซ้อน (Nested Functions) 134
ความหมายและหลักการของฟังก์ชันซ้อน 134
โครงสร้างของฟังก์ชันซ้อน 134
3.6 โมดูล (Modules) 137
ความรู้เสริม ก่อนจะทำความเข้าใจเรื่องโมดูล 139

3.6.1 การนำเข้าโมดูล (Importing Modules)....140
3.6.2 การสร้างโมดูลของตนเอง (Creating Your Own Modules)....142
3.6.3 โครงสร้างโฟลเดอร์และโมดูลย่อย....144
3.6.4 โมดูลสำคัญใน MicroPython สำหรับ ESP32....148
3.6.5 เทคนิคมืออาชีพ....168
บทที่ 4: สั่งงานขา GPIO: จากไฟกะพริบสู่การอ่านค่าเซ็นเซอร์....171
4.1 บทนำสู่ GPIO (Introduction to GPIO)....172
4.2 การใช้งานโมดูล machine.Pin อย่างมืออาชีพ....173
4.3 Digital Output: การควบคุม LED และอุปกรณ์อื่นๆ....176
4.3.1 การทำให้ LED ออนบอร์ดกะพริบ (ทบทวนและขยายความ)....176
4.3.2 การต่อ LED ภายนอกและเขียนโปรแกรมควบคุม....177
4.3.3 ตัวอย่าง: ควบคุม LED 3 ดวงคล้ายไฟจราจรเบื้องต้น....178
4.4 Digital Input: การอ่านค่าจากสวิตช์และปุ่มกด....180
4.4.1 การต่อสวิตช์ (Push button) กับ ESP32....180
4.4.3 ปัญหาการสั่นของสวิตช์ (Switch Bouncing) และแนวทางการแก้ไขเบื้องต้น....181
4.5 Analog Input (ADC)....183
4.6 PWM (Pulse Width Modulation)....183
4.6.1 ตัวอย่าง: การหรี่ไฟ LED (Fading LED)....186
4.6.2 ตัวอย่าง: การควบคุมตำแหน่งของ Servo Motor (SG90) เบื้องต้น....189
4.7 สรุปและการประยุกต์ใช้งาน GPIO....193
บทที่ 5: การเชื่อมต่อ Wi-Fi และเครือข่ายเบื้องต้น....193
5.1 ทำไมต้องเชื่อมต่อ Wi-Fi?....194
5.2 โมดูล network: กุญแจสู่โลกออนไลน์....194

5.3 การเชื่อมต่อ ESP32 เข้ากับเครือข่าย Wi-Fi (โหมด Station) 195
5.3.1 ขั้นตอนการเชื่อมต่อ 195
5.3.2 โค้ดตัวอย่างฟังก์ชันสำหรับเชื่อมต่อ Wi-Fi 197
5.4 พื้นฐานเครือข่ายคอมพิวเตอร์เบื้องต้น (สำหรับ IoT) 198
5.5 การดึงข้อมูลจากอินเทอร์เน็ตด้วยโมดูล urequests 200
5.6 การสร้าง Web Server ง่ายๆ บน ESP32 (เบื้องต้น) 201
5.7 สรุปและก้าวต่อไปสู่โลก IoT 207
บทที่ 6: รู้จักกับเซ็นเซอร์และอุปกรณ์เสริมยอดนิยม 207
6.1 การรับรู้โลกภายนอกด้วยเซ็นเซอร์และปฏิสัมพันธ์กับอุปกรณ์ 207
เซ็นเซอร์คืออะไร? 207
6.1.1 วิธีการค้นหาและติดตั้งไลบรารีสำหรับ MicroPython 208
วิธีการติดตั้งไลบรารี 209
6.2 การอ่านค่าจากเซ็นเซอร์ดิจิทัล 212
6.3 การอ่านค่าจากเซ็นเซอร์แอนะล็อกด้วย ADC 214
6.4 การสื่อสารกับเซ็นเซอร์ผ่าน I2C 218
6.5 การสื่อสารกับเซ็นเซอร์ผ่าน SPI (ภาพรวมเบื้องต้น) 223
6.6 การแสดงผลข้อมูล 225
6.7 การควบคุมอุปกรณ์เอาต์พุตเพิ่มเติม 230
6.8 การเลือกใช้เซ็นเซอร์ อุปกรณ์แสดงผล และอุปกรณ์เอาต์พุต 232
บทที่ 7: โปรเจกต์ตัวอย่างสำหรับผู้เริ่มต้น 233
บทที่ 8: สิ่งที่เรียนรู้ต่อไปและแหล่งข้อมูลเพิ่มเติม 234
8.1 การเดินทางที่ยังไม่สิ้นสุด 234
8.2 ก้าวสู่หัวข้อขั้นสูง 234

8.2.1 การสื่อสารไร้สาย Bluetooth Low Energy (BLE) บน ESP32....234
8.2.2 การทำงานกับไฟล์บน ESP32 (File System)....239
8.2.3 การประหยัดพลังงานด้วยโหมด Deep Sleep....240
8.2.4 หัวข้ออื่นๆ ที่น่าสนใจ....241
8.3 เคล็ดลับในการแก้ไขปัญหาที่พบบ่อย (Troubleshooting Tips)....241
แผนผังขา (Pinout Diagram) ของ ESP32 รุ่นยอดนิยม....243
สรุปคำสั่งและโมดูล MicroPython ที่ใช้บ่อย....249

บทที่ 1: MicroPython และ ESP32 คืออะไร? ทำไมต้องใช้คู่นี้?

1.1 ยินดีต้อนรับสู่โลกแห่งไมโครคอนโทรลเลอร์ยุคใหม่

ในยุคปัจจุบัน เทคโนโลยีได้เข้ามาเป็นส่วนหนึ่งของชีวิตประจำวันอย่างแยกไม่ออก หนึ่งในเทคโนโลยีที่มีบทบาทสำคัญที่อยู่เบื้องหลังของนวัตกรรมและสิ่งอำนวยความสะดวกมากมายก็คือ **ไมโครคอนโทรลเลอร์ (Microcontroller)** หรือที่เราอาจคุ้นเคยในชื่อ **"สมองกลจิ๋ว","สมองกลฝังตัว"** อุปกรณ์ขนาดเล็กเหล่านี้เปรียบเสมือนหัวใจสำคัญที่ขับเคลื่อนอุปกรณ์อิเล็กทรอนิกส์หลากหลายชนิด ตั้งแต่ของเล่นเด็ก, เครื่องใช้ไฟฟ้าในบ้าน, ระบบควบคุมในรถยนต์, ไปจนถึงอุปกรณ์ทางการแพทย์ที่ซับซ้อน และที่กำลังได้รับความสนใจอย่างกว้างขวางคือการนำไปใช้ในโครงงาน **Internet of Things (IoT)** หรือ "อินเทอร์เน็ตของสรรพสิ่ง" ซึ่งเป็นการเชื่อมโยงอุปกรณ์ต่างๆ เข้ากับเครือข่ายอินเทอร์เน็ต ทำให้สามารถควบคุม สั่งการ หรือเก็บข้อมูลได้จากระยะไกล

การสร้างสรรค์สิ่งประดิษฐ์หรือระบบอัจฉริยะด้วยไมโครคอนโทรลเลอร์จึงไม่ใช่เรื่องไกลตัวอีกต่อไป เปิดโอกาสให้ทุกคน ไม่ว่าจะเป็นนักเรียน นักศึกษา นักประดิษฐ์ หรือผู้ที่สนใจทั่วไป สามารถเรียนรู้และพัฒนาโครงงานของตนเองขึ้นมาได้ อย่างไรก็ตาม ในอดีต การเริ่มต้นใช้งานไมโครคอนโทรลเลอร์อาจมีอุปสรรคอยู่บ้าง ไม่ว่าจะเป็นความซับซ้อนของภาษาโปรแกรมที่ใช้ควบคุม เครื่องมือพัฒนาที่อาจต้องตั้งค่าหลายขั้นตอน หรือข้อจำกัดทางด้านฮาร์ดแวร์ที่อาจไม่ยืดหยุ่นพอสำหรับโครงงานที่ต้องการความสามารถ หลากหลาย และรอบด้าน

แต่ในปัจจุบัน ด้วยความก้าวหน้าของเทคโนโลยี ทั้งด้านซอฟต์แวร์และฮาร์ดแวร์ ได้ทำให้การเข้าถึงและการเรียนรู้ไมโครคอนโทรลเลอร์เป็นเรื่องง่ายและสนุกสนานยิ่งขึ้นอย่างที่ไม่เคยเป็นมาก่อน หนังสือเล่มนี้จะนำพาผู้อ่านทุกท่านเข้าสู่โลกของไมโครคอนโทรลเลอร์ยุคใหม่ ผ่านการทำความรู้จักกับเครื่องมืออันทรงพลังสองสิ่งที่กำลังปฏิวัติวงการนี้ นั่นคือ **MicroPython** และชิป **ESP32** ซึ่งจะช่วยลดข้อจำกัดเดิม ๆ และเปิดประตูสู่ความเป็นไปได้ใหม่ ๆ ในการสร้างสรรค์โครงงานอิเล็กทรอนิกส์และ IoT ได้อย่างง่ายดายและมีประสิทธิภาพ

1.2 MicroPython: Python ในร่างจิ๋วแต่ทรงพลัง

MicroPython คืออะไร? MicroPython คือการนำภาษาโปรแกรม Python ที่ได้รับความนิยมอย่างสูงมาปรับปรุงและย่อส่วนเพื่อให้สามารถทำงานได้อย่างมีประสิทธิภาพบนไมโครคอนโทรลเลอร์ ซึ่งเป็นอุปกรณ์ที่มีทรัพยากรจำกัด เช่น หน่วยความจำ (RAM) และพื้นที่จัดเก็บข้อมูล (Flash memory) น้อยกว่าคอมพิวเตอร์ทั่วไป โดยยังคงรักษาจุดเด่นหลักของภาษา Python ไว้ นั่นคือไวยากรณ์ที่อ่านง่าย เข้าใจง่าย คล้ายคลึงกับภาษาอังกฤษ ทำให้ผู้เริ่มต้นสามารถเรียนรู้ได้อย่างรวดเร็ว ผู้สร้าง MicroPython คือ Damien George วิศวกรชาวออสเตรเลีย ผู้เริ่มต้นโครงการนี้ผ่านการระดมทุนบน Kickstarter ในปี 2013 โดยมีเป้าหมายเพื่อสร้างซอฟต์แวร์ที่ช่วยให้การ

เขียนโปรแกรมควบคุมฮาร์ดแวร์เป็นเรื่องง่ายขึ้น นับตั้งแต่นั้นมา MicroPython ก็ได้รับการพัฒนาอย่างต่อเนื่องและมีชุมชนผู้ใช้งานขยายใหญ่ขึ้นทั่วโลก รองรับไมโครคอนโทรลเลอร์หลากหลายรุ่น

ข้อดีของ MicroPython:

- **เรียนรู้ง่าย เขียนโค้ดสั้นกระชับ:** ด้วยไวยากรณ์ที่เรียบง่ายของ Python ผู้ที่ไม่มีพื้นฐานการเขียนโปรแกรมมาก่อนก็สามารถเริ่มต้นได้ไม่ยาก และสำหรับผู้ที่มีพื้นฐาน Python อยู่แล้ว ก็สามารถนำความรู้เดิมมาปรับใช้ได้ทันที นอกจากนี้ โค้ดที่เขียนด้วย MicroPython มักจะสั้นกว่าโค้ดที่เขียนด้วยภาษาโปรแกรมระดับต่ำอย่าง C/C++ สำหรับงานเดียวกัน ทำให้เข้าใจภาพรวมของโปรแกรมได้ง่ายขึ้น
- **พัฒนาและทดสอบได้รวดเร็ว (Rapid Prototyping):** MicroPython มาพร้อมกับเครื่องมือที่เรียกว่า REPL (Read-Evaluate-Print Loop) ซึ่งเป็น Command Line Interface ที่ช่วยให้ผู้ใช้สามารถพิมพ์คำสั่ง Python และเห็นผลลัพธ์การทำงานได้ทันทีบนตัวไมโครคอนโทรลเลอร์ ทำให้การทดลองโค้ดส่วนเล็กๆ หรือการตรวจสอบการทำงานของฮาร์ดแวร์เป็นไปอย่างรวดเร็ว ไม่จำเป็นต้องคอมไพล์และอัปโหลดโปรแกรมใหม่ทั้งหมดทุกครั้งที่แก้ไขโค้ดเพียงเล็กน้อย
- **มีไลบรารีพื้นฐานพร้อมใช้งาน:** MicroPython มีโมดูล (ไลบรารี) มาตรฐานที่จำเป็นสำหรับการควบคุมฮาร์ดแวร์ เช่น การจัดการขา GPIO (General Purpose Input/Output) การสื่อสารแบบ I2C, SPI, UART รวมถึงการเชื่อมต่อเครือข่าย ทำให้การเริ่มต้นพัฒนาโครงงานเป็นไปได้สะดวก
- **ขยายองค์ความรู้ Python:** การเรียนรู้ MicroPython เท่ากับเป็นการเรียนรู้ภาษา Python ไปในตัว ซึ่งเป็นภาษาที่มีความต้องการสูงในตลาดงานปัจจุบัน สามารถนำไปต่อยอดในการพัฒนาเว็บแอปพลิเคชัน, งานด้านข้อมูล (Data Science), ปัญญาประดิษฐ์ (AI) และอื่นๆ ได้อีกมากมาย

1.3 ESP32: ชิปมหัศจรรย์สำหรับโครงงาน IoT

ESP32 คืออะไร? ESP32 คือชื่อของซีรีส์ชิปไมโครคอนโทรลเลอร์ราคาประหยัดที่พัฒนาโดยบริษัท Espressif Systems จากประเทศจีน ซึ่งได้รับการออกแบบมาให้มีความสามารถรอบด้าน โดยเฉพาะอย่างยิ่งสำหรับงานที่เกี่ยวข้องกับ Internet of Things (IoT) และอุปกรณ์พกพา ESP32 เป็นรุ่นที่พัฒนาต่อยอดมาจาก ESP8266 ซึ่งเคยสร้างความนิยมอย่างสูงในหมู่นักพัฒนาและนักประดิษฐ์ ด้วยการเพิ่มคุณสมบัติและความสามารถที่ทรงพลังยิ่งขึ้น

โดยทั่วไป เมื่อพูดถึง ESP32 มักจะหมายถึงตัวชิป SoC (System on a Chip) แต่ในท้องตลาด บอร์ดพัฒนา (Development Board) ที่ใช้ชิป ESP32 ก็มักถูกเรียกว่า "บอร์ด ESP32" เช่นกัน บอร์ดเหล่านี้จะรวมเอา

ชิป ESP32 พร้อมวงจรสนับสนุนที่จำเป็น เช่น วงจรแปลงแรงดันไฟฟ้า, พอร์ต USB สำหรับการโปรแกรมและจ่ายไฟ, และขาต่อต่างๆ ทำให้ง่ายต่อการนำไปใช้งาน

จุดเด่นของ ESP32:

- **การเชื่อมต่อไร้สายครบครัน:** ESP32 มาพร้อมกับ Wi-Fi (มาตรฐาน 802.11 b/g/n) และ Bluetooth (ทั้ง Classic และ Bluetooth Low Energy - BLE) ในตัว ทำให้สามารถเชื่อมต่อกับเครือข่ายอินเทอร์เน็ตหรืออุปกรณ์อื่นๆ แบบไร้สายได้อย่างง่ายดาย ซึ่งเป็นหัวใจสำคัญของโครงงาน IoT
- **ประสิทธิภาพสูง:** ส่วนใหญ่มาพร้อมกับโปรเซสเซอร์ Xtensa LX6 แบบ Dual-core (สองแกนประมวลผล) ที่ความเร็วสัญญาณนาฬิกาสูงถึง 240 MHz (ขึ้นอยู่กับรุ่น) ทำให้สามารถประมวลผลงานที่ซับซ้อนได้ดีกว่าไมโครคอนโทรลเลอร์แกนเดียวหลายๆ รุ่น
- **หน่วยความจำเพียงพอ:** มีหน่วยความจำ SRAM และ Flash memory ภายในตัวในปริมาณที่ค่อนข้างมากสำหรับไมโครคอนโทรลเลอร์ (เช่น SRAM หลายร้อย KB และ Flash memory หลาย MB) เพียงพอสำหรับการรัน MicroPython และเก็บโปรแกรมขนาดกลางได้
- **ขาเชื่อมต่อ (GPIO) อเนกประสงค์:** มีขา General Purpose Input/Output (GPIO) จำนวนมาก ซึ่งสามารถกำหนดค่าให้ทำงานได้หลากหลายหน้าที่ เช่น เป็น Digital Input/Output, Analog Input (ADC), Digital to Analog Output (DAC), Pulse Width Modulation (PWM), รวมถึงรองรับโปรโตคอลการสื่อสารมาตรฐาน เช่น I2C, SPI, UART
- **เซ็นเซอร์ในตัว:** บางรุ่นของ ESP32 มีเซ็นเซอร์พื้นฐานในตัว เช่น เซ็นเซอร์สัมผัส (Touch Sensor) และเซ็นเซอร์อุณหภูมิ (Temperature Sensor)
- **ความปลอดภัย:** รองรับคุณสมบัติด้านความปลอดภัยของฮาร์ดแวร์ เช่น Secure Boot และ Flash Encryption
- **ราคาคุ้มค่า:** เมื่อเทียบกับคุณสมบัติและความสามารถที่ได้รับ บอร์ดพัฒนา ESP32 ถือว่ามีราคาที่เข้าถึงได้ง่าย ทำให้นักเรียน นักศึกษา และนักประดิษฐ์ทั่วไปสามารถเริ่มต้นได้โดยที่ไม่ต้องลงทุนสูงมากนัก

1.4 MicroPython + ESP32: คู่หูที่ลงตัวที่สุดสำหรับการเริ่มต้น

เมื่อความเรียบง่ายและยืดหยุ่นของ MicroPython ผสานเข้ากับพลังและความสามารถรอบด้านของชิป ESP32 จึงเกิดเป็นแพลตฟอร์มที่ลงตัวอย่างยิ่งสำหรับการเรียนรู้และพัฒนาโครงงานอิเล็กทรอนิกส์และ IoT โดยเฉพาะสำหรับผู้เริ่มต้นและผู้ที่ต้องการสร้างต้นแบบ (Prototype) อย่างรวดเร็ว

เหตุผลที่ MicroPython และ ESP32 เป็นคู่หูที่ยอดเยี่ยม:

- **ลดกำแพงการเริ่มต้น:** ผู้เริ่มต้นไม่จำเป็นต้องกังวลกับความซับซ้อนของภาษา C/C++ หรือการตั้งค่าเครื่องมือที่ยุ่งยาก เพียงแค่ติดตั้งเฟิร์มแวร์ MicroPython ลงบนบอร์ด ESP32 และใช้โปรแกรม Text Editor หรือ IDE ง่ายๆ อย่าง Thonny ก็สามารถเริ่มเขียนโค้ดควบคุมฮาร์ดแวร์ได้ทันที
- **ใช้ประโยชน์จากฮาร์ดแวร์ ESP32 ได้เต็มที่ (อย่างง่ายดาย):** MicroPython สำหรับ ESP32 มีโมดูลที่ช่วยให้การเข้าถึงคุณสมบัติหลักของ ESP32 เช่น Wi-Fi, Bluetooth, GPIO, ADC, PWM เป็นเรื่องง่าย เพียงเขียนโค้ด Python ไม่กี่บรรทัดก็สามารถสั่งงานฟังก์ชันเหล่านี้ได้
- **เหมาะสำหรับโครงงาน IoT:** การเชื่อมต่อ Wi-Fi ของ ESP32 เมื่อรวมกับความสามารถในการจัดการข้อความและข้อมูลของ Python ทำให้การพัฒนาอุปกรณ์ IoT ที่ส่งข้อมูลไปยังเซิร์ฟเวอร์ หรือรับคำสั่งผ่านเครือข่าย ทำได้ไม่ซับซ้อน
- **ชุมชนผู้ใช้งานขนาดใหญ่:** ทั้ง MicroPython และ ESP32 ต่างก็มีชุมชนผู้ใช้งานที่กระตือรือร้นและพร้อมให้ความช่วยเหลือ เมื่อพบปัญหาหรือมีข้อสงสัย สามารถค้นหาข้อมูล แนวทางการแก้ไข หรือตัวอย่างโค้ดได้ไม่ยาก ด้วยการผสมผสานนี้ ผู้อ่านสามารถจินตนาการถึงการสร้างสรรค์โครงงานที่หลากหลายได้อย่างง่ายดาย เช่น:
    - ระบบควบคุมไฟส่องสว่างในบ้านผ่านสมาร์ตโฟน
    - สถานีตรวจวัดสภาพอากาศขนาดเล็กที่ส่งข้อมูลอุณหภูมิและความชื้นขึ้นเว็บไซต์
    - หุ่นยนต์ขนาดเล็กที่ควบคุมการเคลื่อนที่ด้วยคำสั่ง Python
    - ระบบแจ้งเตือนเมื่อมีผู้บุกรุกผ่านเซ็นเซอร์ตรวจจับความเคลื่อนไหว
    - อุปกรณ์สวมใส่ที่เก็บข้อมูลสุขภาพเบื้องต้นและส่งไปยังแอปพลิเคชัน

1.5 เปรียบเทียบสำหรับผู้ใช้ Arduino: ก้าวใหม่ที่น่าลอง

สำหรับผู้ที่คุ้นเคยกับแพลตฟอร์ม Arduino และภาษา C/C++ การมาถึงของ MicroPython บน ESP32 อาจเป็นทั้งโอกาสและความท้าทายใหม่ที่น่าสนใจ Arduino ได้สร้างคุณูปการอย่างใหญ่หลวงในการทำให้การเรียนรู้ไมโครคอนโทรลเลอร์เป็นเรื่องง่ายสำหรับคนจำนวนมาก และ MicroPython ก็มีเป้าหมายเดียวกัน แต่มาพร้อมกับแนวทางที่แตกต่างออกไป

ความแตกต่างหลักระหว่าง Arduino (C/C++) และ MicroPython บน ESP32

คุณสมบัติ	Arduino (C/C++)	MicroPython บน ESP32
ภาษาโปรแกรม	C/C++ (Static Typing)	Python (Dynamic Typing)
การทำงาน	คอมไพล์เป็น Machine Code แล้วอัปโหลด	Interpreter ทำงานบนเฟิร์มแวร์ MicroPython
การพัฒนา	เขียนโค้ด -> คอมไพล์ -> อัปโหลด -> ทดสอบ	เขียนโค้ด -> (อัปโหลดไฟล์ .py) -> ทดสอบผ่าน REPL/รันไฟล์
ความเร็วในการทดลอง	ช้ากว่า เนื่องจากต้องคอมไพล์ใหม่ทุกครั้ง	เร็วกว่ามาก สามารถทดลองโค้ดทีละบรรทัดได้
ความซับซ้อนของภาษา	สูงกว่า โดยเฉพาะเรื่อง Pointer, Memory Management	ต่ำกว่า ไวยากรณ์อ่านง่าย จัดการหน่วยความจำอัตโนมัติ
การจัดการทรัพยากร	ผู้เขียนโค้ดต้องจัดการอย่างระมัดระวัง	มี Garbage Collector ช่วยจัดการหน่วยความจำ

ข้อดีที่ผู้ใช้ Arduino อาจได้รับจาก MicroPython:

- **ลดเวลาในการพัฒนา:** วงจรการแก้ไขโค้ดและทดสอบที่สั้นลงอย่างเห็นได้ชัด
- **ภาษาที่ยืดหยุ่นกว่า:** การจัดการข้อความ (String), รายการข้อมูล (List), และโครงสร้างข้อมูลแบบ Dictionary ใน Python ทำได้ง่ายและมีประสิทธิภาพกว่าใน C/C++ พื้นฐาน
- **การเขียนโปรแกรมเชิงวัตถุ (OOP):** แม้ C++ จะรองรับ OOP แต่ Python ก็มีโครงสร้าง OOP ที่เข้าใจง่ายและเป็นธรรมชาติ
- **ลดความกังวลเรื่อง Memory Management พื้นฐาน:** ช่วยให้ผู้เริ่มต้นโฟกัสไปที่ตรรกะของโปรแกรมได้มากขึ้น

อย่างไรก็ตาม ไม่ได้หมายความว่า MicroPython ดีกว่า Arduino C/C++ ในทุกกรณี การเขียนโปรแกรมด้วย C/C++ โดยตรงยังคงให้ประสิทธิภาพการทำงานที่เร็วกว่าและใช้ทรัพยากรน้อยกว่า เหมาะสำหรับงานที่ต้องการความเร็วสูงมากๆ หรือมีข้อจำกัดด้านหน่วยความจำที่เข้มงวดสุดๆ MicroPython จึงเป็นอีก **ทางเลือกหนึ่ง** ที่น่าสนใจสำหรับโครงงานหลายประเภท โดยเฉพาะโครงงาน IoT ที่เน้นการเชื่อมต่อเครือข่ายและการจัดการข้อมูล ซึ่งความเร็วในการพัฒนาอาจมีความสำคัญมากกว่าประสิทธิภาพระดับไมโครวินาที

1.6 MicroPython กับ ESP32 ในมุมมองของครูอาจารย์

สำหรับครูอาจารย์ผู้สอนวิชาวิทยาการคำนวณ, การออกแบบและเทคโนโลยี, หรือวิชาที่เกี่ยวข้องกับ STEM และ IoT นั้น MicroPython และ ESP32 ถือเป็นเครื่องมือการสอนที่มีศักยภาพสูงมาก

- **ลดอุปสรรคการเรียนรู้โค้ดดิ้ง:** ไวยากรณ์ที่ง่ายของ Python ช่วยให้นักเรียนที่ไม่เคยมีประสบการณ์มาก่อน สามารถเข้าใจแนวคิดการเขียนโปรแกรมได้เร็วขึ้น ลดความกลัวและความรู้สึกว่าโค้ดดิ้งเป็นเรื่องยาก
- **ส่งเสริมการเรียนรู้แบบโครงงานเป็นฐาน (Project-Based Learning):** นักเรียนสามารถนำความรู้ไปสร้างสรรค์โครงงานที่จับต้องได้จริงอย่างรวดเร็ว ตั้งแต่การควบคุม LED ง่ายๆ ไปจนถึงการสร้างอุปกรณ์ IoT ที่เชื่อมต่ออินเทอร์เน็ต ซึ่งช่วยเพิ่มแรงจูงใจและความสนุกสนานในการเรียนรู้
- **เชื่อมโยงกับชีวิตจริง:** โครงงาน IoT ที่สร้างจาก ESP32 และ MicroPython สามารถออกแบบให้แก้ปัญหาในชีวิตประจำวันหรือในโรงเรียนได้ เช่น ระบบตรวจวัดคุณภาพอากาศ, ระบบรดน้ำต้นไม้อัตโนมัติสำหรับแปลงผักของโรงเรียน ทำให้นักเรียนเห็นคุณค่าของสิ่งที่เรียน
- **พัฒนาทักษะการแก้ปัญหาและคิดเชิงคำนวณ:** การออกแบบและแก้ไขโปรแกรมสำหรับควบคุมฮาร์ดแวร์ เป็นการฝึกฝนทักษะการคิดวิเคราะห์ การวางแผน และการแก้ปัญหาอย่างเป็นระบบ
- **เครื่องมือที่เข้าถึงง่ายและราคาไม่แพง:** บอร์ด ESP32 มีราคาที่ไม่สูงนัก ทำให้โรงเรียนหรือสถาบันการศึกษาสามารถจัดหาให้นักเรียนใช้งานได้อย่างทั่วถึง

การใช้ MicroPython และ ESP32 ในห้องเรียน จึงไม่เพียงแต่สอนทักษะการเขียนโปรแกรม แต่ยังเป็นการเปิดโลกทัศน์ให้นักเรียนเห็นถึงพลังของเทคโนโลยีในการสร้างสรรค์นวัตกรรมและแก้ปัญหาต่างๆ รอบตัว

1.7 บทสรุป: เตรียมพร้อมสำหรับการเดินทาง

จากที่กล่าวมาทั้งหมด จะเห็นได้ว่า MicroPython และ ESP32 คือการผสมผสานที่ลงตัวระหว่างความง่ายในการใช้งานของซอฟต์แวร์และความสามารถอันทรงพลังของฮาร์ดแวร์ นับเป็นเครื่องมือที่เปิดโอกาสให้ทุกคนสามารถก้าวเข้าสู่โลกแห่งการสร้างสรรค์โครงงานอิเล็กทรอนิกส์และ Internet of Things ได้อย่างมั่นใจ ไม่ว่าจะมีพื้นฐานมาก่อนหรือไม่ก็ตาม

เสน่ห์ของ MicroPython อยู่ที่ความสามารถในการปลดปล่อยจินตนาการของผู้พัฒนาให้เป็นอิสระจากความซับซ้อนของโค้ด ในขณะที่ ESP32 ก็มอบขุมพลังและช่องทางการเชื่อมต่อที่หลากหลายรอให้ถูกนำไปใช้งาน หนังสือเล่มนี้จะเป็นเสมือนแผนที่นำทาง พาผู้อ่านไปสำรวจดินแดนอันน่าตื่นเต้นนี้ ตั้งแต่การทำความรู้จัก การติดตั้งเครื่องมือ จนถึงการลงมือสร้างโครงงานจริง

ในบทต่อไป เราจะเริ่มต้นการเดินทางภาคปฏิบัติ ด้วยการเตรียมความพร้อมเครื่องมือและอุปกรณ์ที่จำเป็น รวมถึงการติดตั้ง MicroPython ลงบนบอร์ด ESP32 เพื่อให้พร้อมสำหรับการเขียนโปรแกรมแรก ขอเชิญผู้อ่านทุกท่านเตรียมตัวให้พร้อม แล้วมาเริ่มต้นสร้างสรรค์สิ่งประดิษฐ์ด้วยกัน!

บทที่ 2: ติดตั้งและเริ่มต้นใช้งาน MicroPython บน ESP32

หลังจากที่เราได้ทำความรู้จักกับ MicroPython และ ESP32 รวมถึงเหตุผลที่ทำให้ทั้งสองเป็นคู่หูที่ยอดเยี่ยมสำหรับการเริ่มต้นในบทที่ 1 แล้ว ในบทนี้ จะเข้าสู่ภาคปฏิบัติกันอย่างเต็มตัว โดยจะแนะนำขั้นตอนที่จำเป็นทั้งหมด ตั้งแต่การเลือกบอร์ด ESP32, การเตรียมอุปกรณ์, การติดตั้งไดรเวอร์, การติดตั้งเฟิร์มแวร์ MicroPython ลงบนบอร์ด ESP32, ไปจนถึงการตั้งค่าโปรแกรม Thonny IDE และการทดลองเขียนโปรแกรมแรกเพื่อควบคุมฮาร์ดแวร์เบื้องต้น การทำตามขั้นตอนในบทนี้อย่างละเอียดจะช่วยให้ผู้อ่านมีพื้นฐานที่มั่นคงและพร้อมสำหรับการเรียนรู้การเขียนโปรแกรม MicroPython เพื่อสร้างสรรค์โครงงานต่างๆ ในบทต่อๆ ไป

2.1 การเลือกบอร์ด ESP32 สำหรับมือใหม่

ชิป ESP32 ถูกนำไปใช้งานในบอร์ดพัฒนา (Development Board) หลากหลายรูปแบบและหลายผู้ผลิต สำหรับผู้เริ่มต้น การเลือกบอร์ดที่เหมาะสมจะช่วยให้การเรียนรู้ง่ายขึ้น บอร์ดที่แนะนำควรมีลักษณะดังนี้:

- **มีพอร์ต Micro-USB หรือ USB-C:** สำหรับการจ่ายไฟและอัปโหลดโปรแกรมได้สะดวก
- **มีวงจร USB to UART ในตัว:** บอร์ดส่วนใหญ่มักใช้ชิป เช่น CP2102, CP2104 หรือ CH340/CH9102F เพื่อทำหน้าที่นี้ ทำให้สามารถเชื่อมต่อกับคอมพิวเตอร์ผ่านสาย USB ได้โดยตรง
- **มีปุ่ม BOOT (หรือ FLASH) และ EN (หรือ RESET/RST):** ปุ่มเหล่านี้จำเป็นสำหรับการเข้าโหมดดาวน์โหลดเฟิร์มแวร์
- **มี LED ออนบอร์ด (Onboard LED):** อย่างน้อยหนึ่งดวง เพื่อใช้ทดสอบการเขียนโปรแกรมเบื้องต้นได้ง่าย
- **ขา GPIO ถูกต่อออกมาให้ใช้งานง่าย:** ควรมี Pinout Diagram (แผนผังขา) ที่ชัดเจน และขาต่างๆ ควรถูกพิมพ์ชื่อกำกับไว้บนบอร์ดหรือหาข้อมูลได้ง่าย

บอร์ด ESP32 ที่เป็นที่นิยมและเหมาะสำหรับผู้เริ่มต้น ได้แก่:

- **ESP32-DevKitC:** เป็นบอร์ดมาตรฐานจาก Espressif เอง มีหลายเวอร์ชันย่อย (เช่น ใช้ชิป ESP32-WROOM-32, ESP32-WROVER) หาซื้อง่าย มีข้อมูลสนับสนุนเยอะ
- **NodeMCU-32S:** เป็นอีกหนึ่งบอร์ดที่ได้รับความนิยมสูง มีลักษณะคล้ายกับ NodeMCU ที่ใช้ ESP8266 ทำให้ผู้ที่เคยใช้ ESP8266 มาก่อนคุ้นเคยได้ง่าย

- **บอร์ดจากผู้ผลิตอื่นๆ:** เช่น Wemos Lolin D32, Adafruit HUZZAH32, SparkFun ESP32 Thing ซึ่งมักจะมีคุณสมบัติเพิ่มเติมหรือการออกแบบที่เป็นเอกลักษณ์

คำแนะนำ: ก่อนซื้อ ควรตรวจสอบข้อมูลจำเพาะของบอร์ด รีวิวจากผู้ใช้งาน และแหล่งข้อมูลสนับสนุน เพื่อให้ได้บอร์ดที่ตรงกับความต้องการและง่ายต่อการเริ่มต้น

2.2 เตรียมความพร้อมก่อนเริ่มต้น

ก่อนจะลงมือติดตั้งและเขียนโปรแกรม มีสิ่งจำเป็นที่ต้องเตรียมให้พร้อมดังนี้:

1. **บอร์ด ESP32:** ตามที่ได้เลือกไว้ในหัวข้อ 2.1
2. **สาย Micro-USB หรือ USB-C Data Cable:**
    - *ข้อควรระวัง:* ตรวจสอบให้แน่ใจว่าเป็นสายข้อมูล (Data Cable) ไม่ใช่สายชาร์จไฟอย่างเดียว (Charge-only Cable) สายข้อมูลจะสามารถทั้งจ่ายไฟและรับส่งข้อมูลได้ สังเกตได้จากเมื่อเสียบกับคอมพิวเตอร์แล้วควรมีอุปกรณ์ใหม่ถูกตรวจพบ (แม้จะยังไม่ได้ติดตั้งไดรเวอร์ก็ตาม)
3. **คอมพิวเตอร์:** ระบบปฏิบัติการ Windows, macOS, หรือ Linux ก็ได้
4. **การเชื่อมต่ออินเทอร์เน็ต:** สำหรับดาวน์โหลดซอฟต์แวร์และไดรเวอร์ที่จำเป็น

2.3 การติดตั้งไดรเวอร์ USB to UART (ถ้าจำเป็น)

บอร์ด ESP32 ส่วนใหญ่จะใช้ชิปแปลงสัญญาณ USB เป็น UART (Universal Asynchronous Receiver/Transmitter) เพื่อให้คอมพิวเตอร์สามารถสื่อสารกับชิป ESP32 ผ่านทางพอร์ต USB ได้ ชิปที่นิยมใช้คือ CP210x (จาก Silicon Labs) หรือ CH340/CH9102F (จาก WCH)

- **สำหรับ Windows:**
    - โดยทั่วไป Windows 10/11 อาจรู้จักชิปเหล่านี้และติดตั้งไดรเวอร์ให้โดยอัตโนมัติเมื่อเสียบบอร์ด ESP32 ครั้งแรก
    - หากไม่รู้จัก (เช่น ใน Device Manager แสดงเป็น Unknown Device หรือมีเครื่องหมายตกใจสีเหลือง) จำเป็นต้องดาวน์โหลดและติดตั้งไดรเวอร์ด้วยตนเอง:
        - **CP210x:** ดาวน์โหลดจากเว็บไซต์ Silicon Labs (ค้นหา "CP210x USB to UART Bridge VCP Drivers")
        - **CH340/CH9102F:** ดาวน์โหลดจากเว็บไซต์ผู้ผลิต WCH (ค้นหา "CH341SER.EXE" หรือ "CH340 driver")
    - หลังติดตั้งไดรเวอร์และเสียบบอร์ด ESP32 ควรจะเห็นพอร์ต COM ใหม่ปรากฏขึ้นใน Device Manager (อยู่ใต้ Ports (COM & LPT)) เช่น Silicon Labs CP210x USB to UART Bridge

(COM3) หรือ USB-SERIAL CH340 (COM4) หรือ COM เลขอื่น ๆ แล้วแต่เครื่องนั้น ๆ ให้จดจำหมายเลขพอร์ต COM นี้ไว้

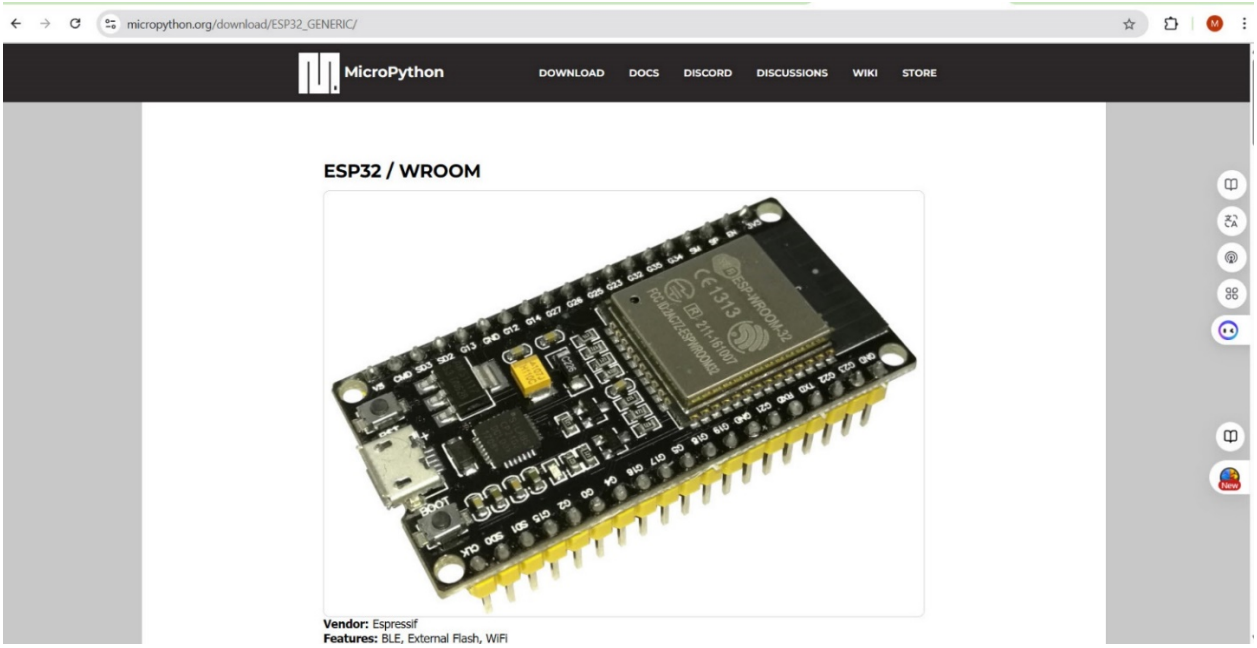
- **สำหรับ macOS:**
    - macOS ส่วนใหญ่มักจะรู้จักชิป CP210x โดยไม่ต้องติดตั้งไดรเวอร์เพิ่มเติม
    - สำหรับชิป CH340/CH9102F อาจจำเป็นต้องติดตั้งไดรเวอร์จากผู้ผลิต WCH (ค้นหา "CH34x_Install_V1.x.pkg" หรือไดรเวอร์สำหรับ Mac รุ่นล่าสุด)
    - หลังจากเสียบบอร์ด ESP32 พอร์ตจะปรากฏในรูปแบบ /dev/cu.usbserial-xxxx หรือ /dev/cu.SLAB_USBtoUART (สำหรับ CP210x) หรือ /dev/cu.wchusbserialxxxx (สำหรับ CH340) สามารถตรวจสอบได้โดยเปิด Terminal แล้วพิมพ์คำสั่ง ls /dev/cu.*
- **สำหรับ Linux:**
    - Linux Kernel ส่วนใหญ่รองรับชิปทั้ง CP210x และ CH340/CH9102F โดยไม่ต้องติดตั้งไดรเวอร์เพิ่มเติม
    - เมื่อเสียบบอร์ด ESP32 พอร์ตมักจะปรากฏในรูปแบบ /dev/ttyUSB0, /dev/ttyUSB1 หรือคล้ายกัน สามารถตรวจสอบได้โดยเปิด Terminal แล้วพิมพ์คำสั่ง ls /dev/ttyUSB* หรือ dmesg | grep tty หลังจากเสียบบอร์ด
    - *ข้อควรทราบสำหรับ Linux:* ผู้ใช้อาจต้องเพิ่มสิทธิ์ในการเข้าถึงพอร์ตอนุกรม โดยการเพิ่ม user เข้าไปในกลุ่ม dialout (หรือ tty ในบาง Distribution) ด้วยคำสั่ง sudo usermod -a -G dialout $USER (แล้ว logout และ login ใหม่ หรือ reboot)

2.4 การติดตั้งเฟิร์มแวร์ MicroPython ลงบน ESP32

เฟิร์มแวร์ (Firmware) คือซอฟต์แวร์ที่ทำงานอยู่บนตัวฮาร์ดแวร์โดยตรง ในที่นี้คือระบบ MicroPython ที่จะทำให้ ESP32 สามารถเข้าใจและรันโค้ด Python ได้ โดยปกติบอร์ด ESP32 ที่ซื้อมาใหม่จะยังไม่มีเฟิร์มแวร์ MicroPython ติดตั้งอยู่ หรืออาจจะมีเฟิร์มแวร์อื่น (เช่น AT Command firmware หรือ Arduino bootloader) ดังนั้น จึงจำเป็นต้องติดตั้งเฟิร์มแวร์ MicroPython ก่อน

2.4.1 ดาวน์โหลดเฟิร์มแวร์ MicroPython

1. ไปที่เว็บไซต์ทางการของ MicroPython: https://micropython.org/download/esp32/



2. มองหาส่วน "Firmware for ESP32 boards"
3. ดาวน์โหลดไฟล์เฟิร์มแวร์เวอร์ชันล่าสุดที่เป็น **Stable** (แนะนำ) หรือ Daily builds (หากต้องการลองฟีเจอร์ใหม่ล่าสุด แต่อาจไม่เสถียร) ไฟล์เฟิร์มแวร์สำหรับ ESP32 ทั่วไปจะมีนามสกุล .bin (เช่น ESP32_GENERIC-YYYYMMDD-vx.xx.x.bin)

คำแนะนำ: สำหรับบอร์ด ESP32 ทั่วไปที่ใช้ชิป ESP32-WROOM หรือ ESP32-SOLO ให้เลือกเฟิร์มแวร์ GENERIC หากใช้บอร์ดที่มีคุณสมบัติพิเศษ เช่น SPIRAM (PSRAM) อาจมีเฟิร์มแวร์เฉพาะ เช่น GENERIC_SPIRAM ควรตรวจสอบสเปกของบอร์ดที่ใช้

- On Windows, the port name is usually similar to `COM4`.

Flashing

Then deploy the firmware to the board, starting at address 0x1000:

```
esptool.py --baud 460800 write_flash 0x1000 ESP32_BOARD_NAME-DATE-VE
RSION.bin
```

Replace `ESP32_BOARD_NAME-DATE-VERSION.bin` with the `.bin` file downloaded from this page.

As above, if `esptool.py` can't automatically detect the serial port then you can pass it explicitly on the command line instead. For example:

```
esptool.py --port PORTNAME --baud 460800 write_flash 0x1000 ESP32_BO
ARD_NAME-DATE-VERSION.bin
```

Troubleshooting

If flashing starts and then fails partway through, try removing the `--baud 460800` option to flash at the slower default speed.

If these steps don't work, consult the MicroPython ESP32 Troubleshooting steps and the esptool documentation.

Important: From the options below, download the `.bin` file for your board.

Firmware

Releases

v1.25.0 (2025-04-15) .bin / [.app-bin] / [.elf] / [.map] / [Release notes] (latest)
v1.24.1 (2024-11-29) .bin / [.app-bin] / [.elf] / [.map] / [Release notes]
v1.24.0 (2024-10-25) .bin / [.app-bin] / [.elf] / [.map] / [Release notes]
v1.23.0 (2024-06-02) .bin / [.app-bin] / [.elf] / [.map] / [Release notes]
v1.22.2 (2024-02-22) .bin / [.app-bin] / [.elf] / [.map] / [Release notes]
v1.22.1 (2024-01-05) .bin / [.app-bin] / [.elf] / [.map] / [Release notes]
v1.22.0 (2023-12-27) .bin / [.app-bin] / [.elf] / [.map] / [Release notes]
v1.21.0 (2023-10-05) .bin / [.app-bin] / [.elf] / [.map] / [Release notes]
v1.20.0 (2023-04-26) .bin / [.elf] / [.map] / [Release notes]
v1.19.1 (2022-06-18) .bin / [.elf] / [.map] / [Release notes]
v1.18 (2022-01-17) .bin / [.elf] / [.map] / [Release notes]
v1.17 (2021-09-02) .bin / [.elf] / [.map] / [Release notes]
v1.16 (2021-06-23) .bin / [.elf] / [.map] / [Release notes]
v1.15 (2021-04-18) .bin / [.elf] / [.map] / [Release notes]
v1.14 (2021-02-02) .bin / [.elf] / [.map] / [Release notes]
v1.13 (2020-09-02) .bin / [.elf] / [.map] / [Release notes]
v1.12 (2019-12-20) .bin / [.elf] / [.map] / [Release notes]

Preview builds

v1.26.0-preview.158.g5cfafb73d (2025-05-28) .bin / [.app-bin] / [.elf] / [.map]
v1.26.0-preview.148.g49f81d504 (2025-05-22) .bin / [.app-bin] / [.elf] / [.map]
v1.26.0-preview.146.g7f6fedef2 (2025-05-21) .bin / [.app-bin] / [.elf] / [.map]
v1.26.0-preview.142.ga1ee42cd3 (2025-05-21) .bin / [.app-bin] / [.elf] / [.map]
(These are automatic builds of the development branch for the next release)

Firmware (Support for OTA)

Releases

v1.25.0 (2025-04-15) .bin / [.app-bin] / [.elf] / [.map] / [Release notes] (latest)
v1.24.1 (2024-11-29) .bin / [.app-bin] / [.elf] / [.map] / [Release notes]
v1.24.0 (2024-10-25) .bin / [.app-bin] / [.elf] / [.map] / [Release notes]
v1.23.0 (2024-06-02) .bin / [.app-bin] / [.elf] / [.map] / [Release notes]
v1.22.2 (2024-02-22) .bin / [.app-bin] / [.elf] / [.map] / [Release notes]
v1.22.1 (2024-01-05) .bin / [.app-bin] / [.elf] / [.map] / [Release notes]
v1.22.0 (2023-12-27) .bin / [.app-bin] / [.elf] / [.map] / [Release notes]
v1.21.0 (2023-10-05) .bin / [.app-bin] / [.elf] / [.map] / [Release notes]
v1.20.0 (2023-04-26) .bin / [.elf] / [.map] / [Release notes]
v1.19.1 (2022-06-18) .bin / [.elf] / [.map] / [Release notes]
v1.18 (2022-01-17) .bin / [.elf] / [.map] / [Release notes]

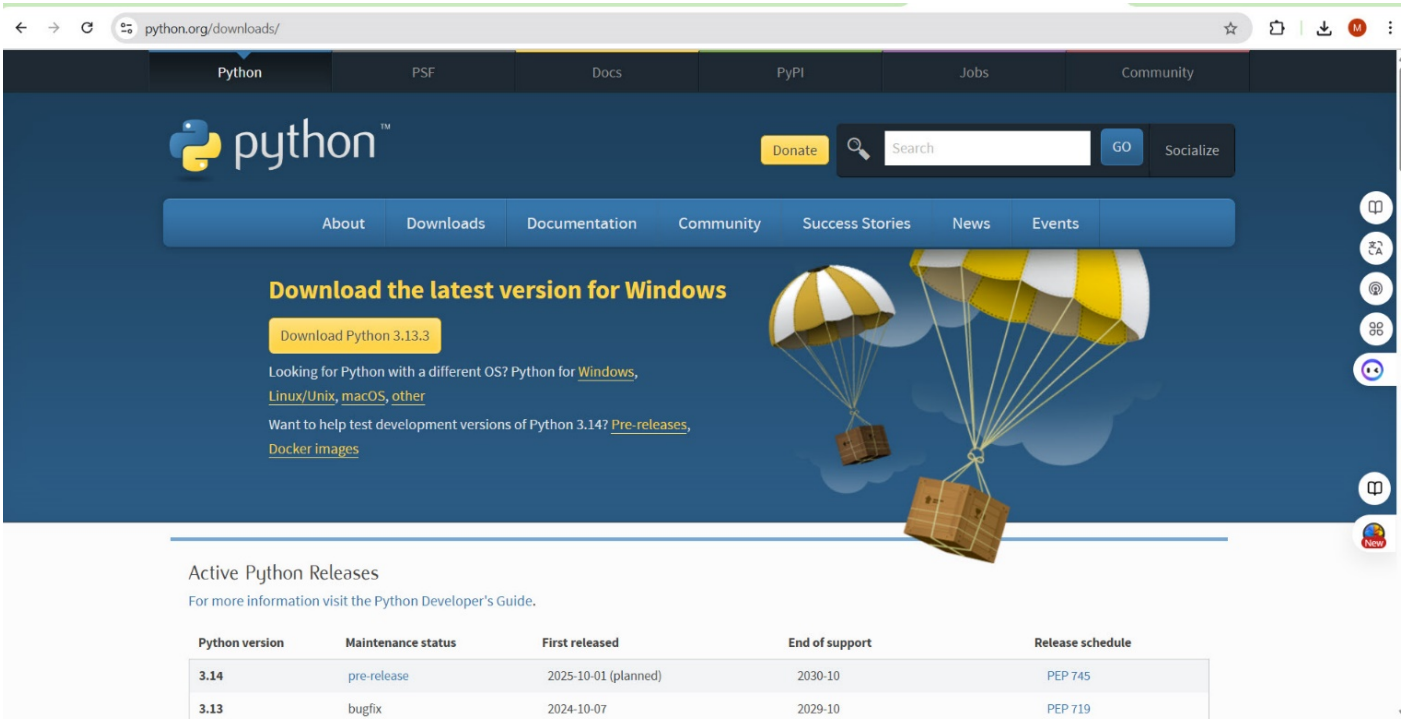
เครื่องมือที่ใช้ในการอัปโหลดเฟิร์มแวร์ มีทั้งแบบพิมพ์คำสั่ง Command-Line และ GUI เลือกใช้งานได้ตามถนัดหากถนัดแบบพิมพ์คำสั่งก็ต้องติดตั้ง esptool.py แต่ถ้าหากว่าการพิมพ์คำสั่งมันดูยุ่งยากซับซ้อนเกินไป ก็ให้ข้ามขั้นตอนนี้ไป ดาวน์โหลดโปรแกรมแบบ GUI มาติดตั้งซึ่งใช้งานง่ายกว่า

แต่อย่างไรก็ดี ต้องติดตั้ง python ตามข้อ1 ก่อน ส่วน esptool.py ไม่จำเป็นต้องติดตั้งก็ได้

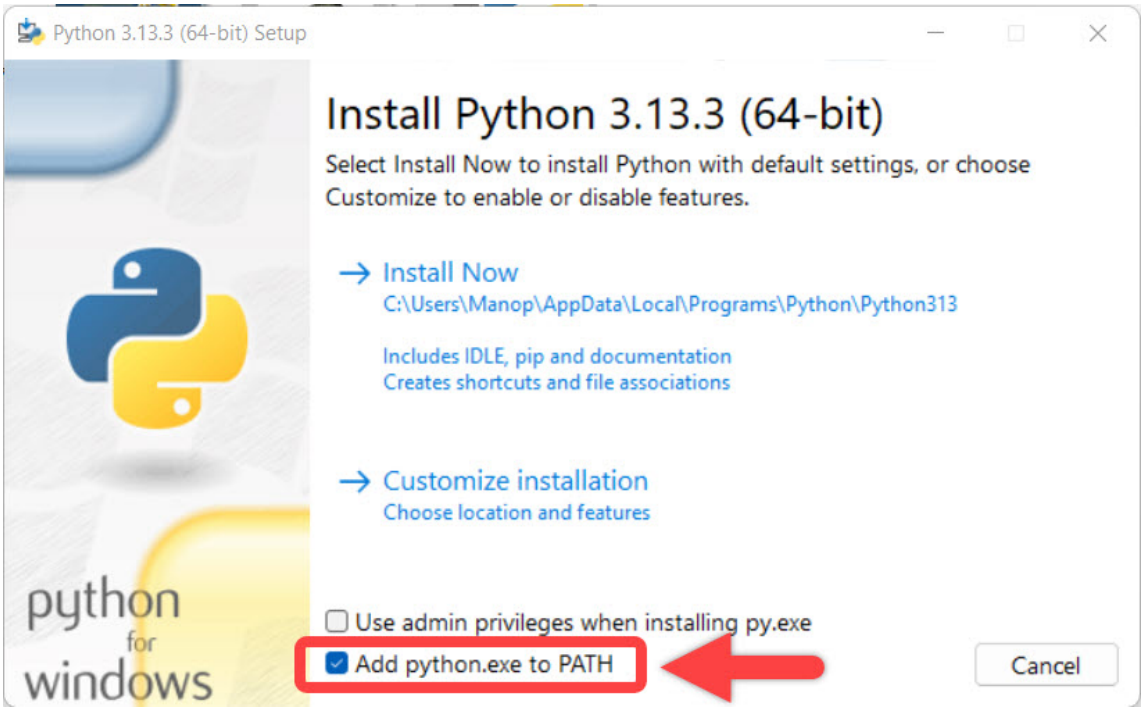
2.4.2 ติดตั้ง Python และ esptool.py

esptool.py เป็นเครื่องมือที่พัฒนาด้วย Python สำหรับสื่อสารกับ Bootloader ของชิป Espressif (ESP8266, ESP32, ESP32-S2, etc.) เพื่ออัปโหลดเฟิร์มแวร์, อ่านข้อมูลจากชิป และอื่นๆ

1. **ต้องมี Python ติดตั้งบนคอมพิวเตอร์:** หากยังไม่มี ให้ดาวน์โหลดและติดตั้ง Python 3.x จาก https://www.python.org/downloads/

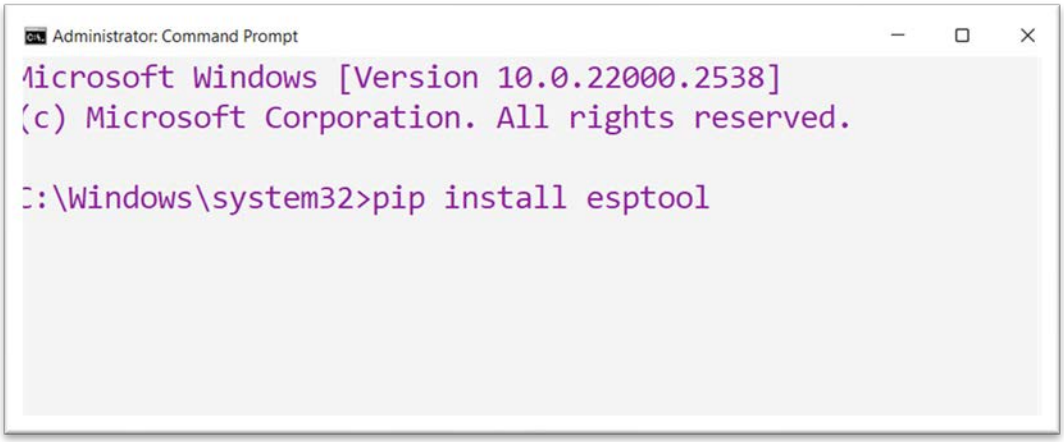


ในระหว่างการติดตั้งบน Windows แนะนำให้เลือก "Add Python to PATH"

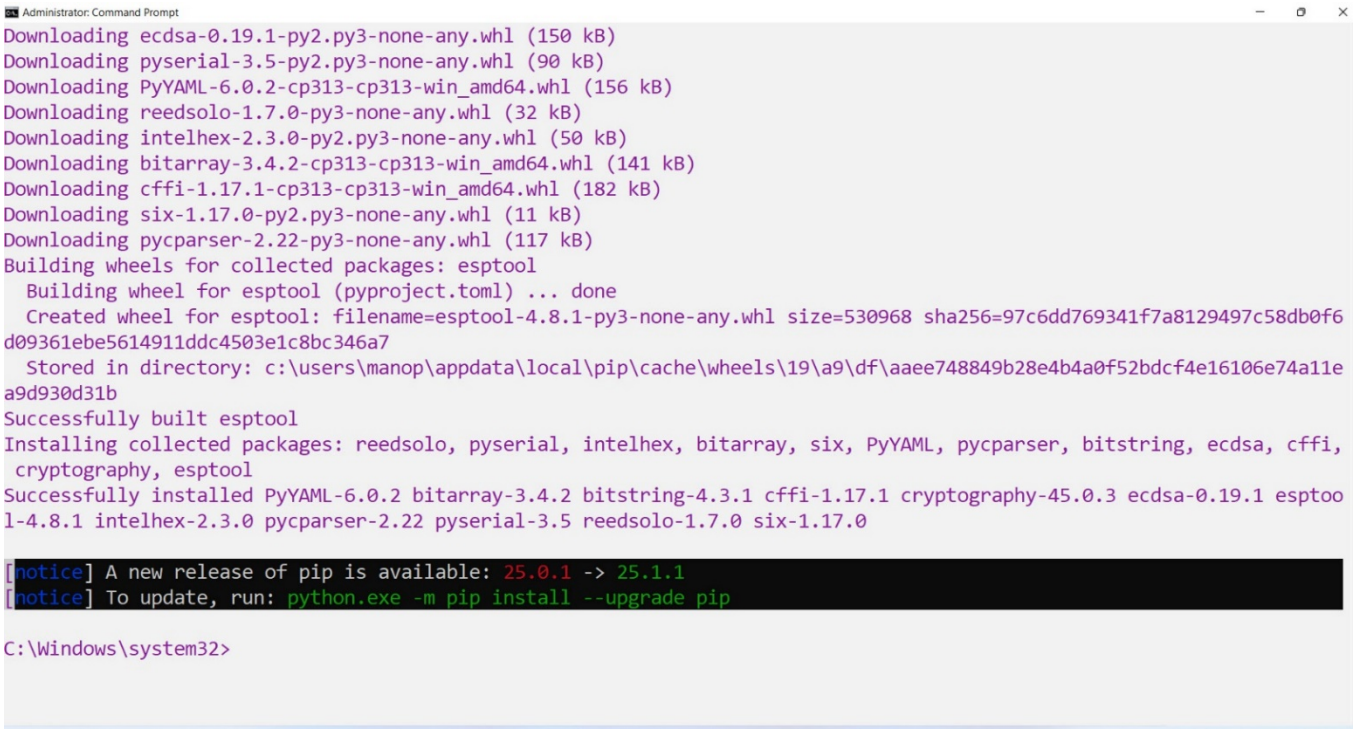


2. **ติดตั้ง esptool.py ผ่าน pip:** เปิด Command Prompt (Windows) หรือ Terminal (macOS/Linux) แล้วพิมพ์คำสั่ง:

pip install esptool แล้วกดปุ่ม **Enter**

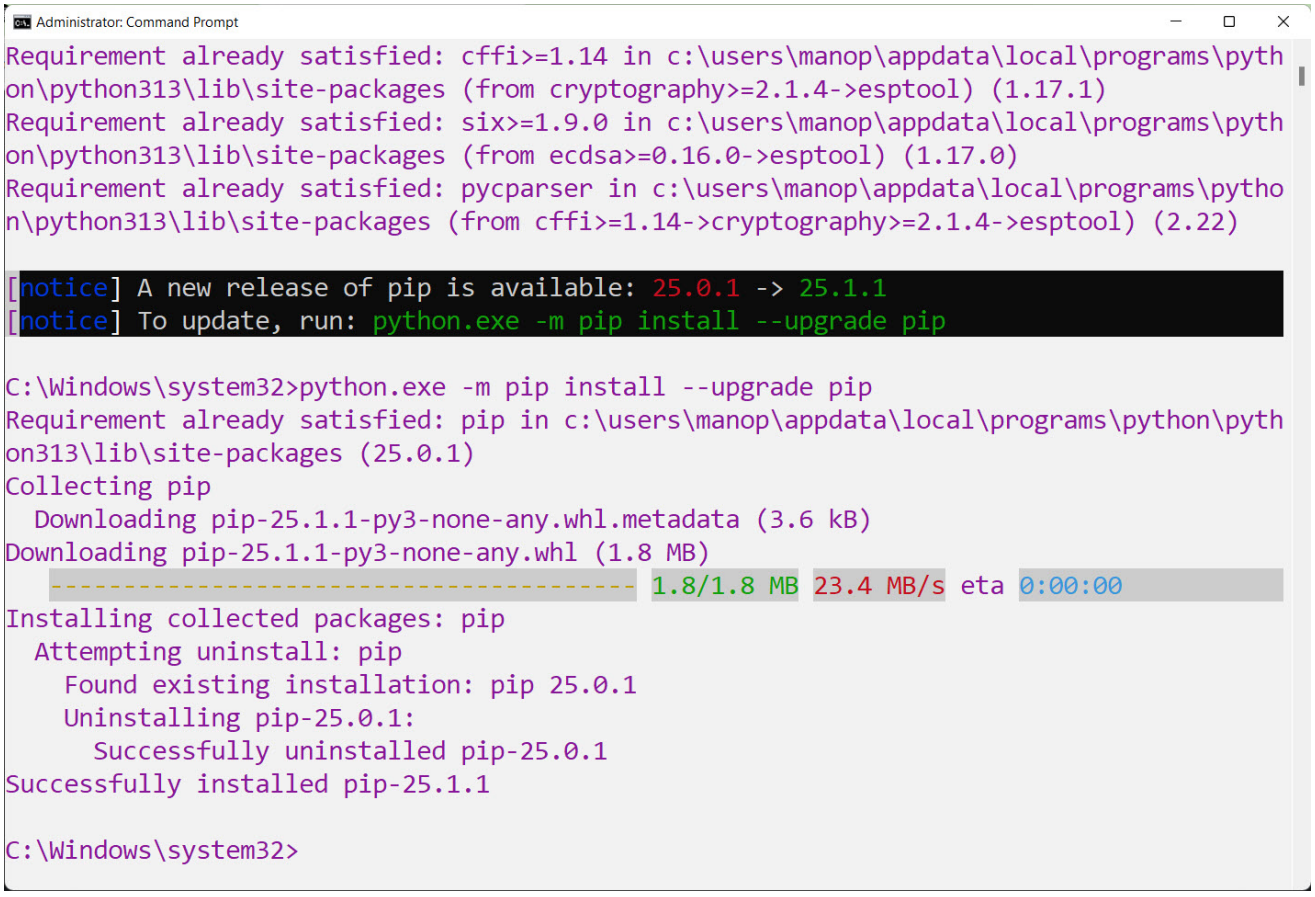


เมื่อติดตั้งเสร็จแล้วจะมีข้อความแจ้งในหน้าต่าง พร้อมกับแสดงเวอร์ชันที่ติดตั้ง



หากต้องการหากต้องการอัปเกรดเป็นเวอร์ชันล่าสุด พิมพ์ **python.exe -m pip install –upgrade pip**

เสร็จแล้วกดปุ่ม Enter อีกครั้งแล้วรอจนกว่าจะมีข้อความแจ้ง



3. **หากต้องการตรวจสอบเวอร์ชันให้พิมพ์ esptool version แล้วกด Enter**

2.4.3 การเชื่อมต่อบอร์ดและค้นหาพอร์ต

1. เสียบบอร์ด ESP32 เข้ากับคอมพิวเตอร์ผ่านสาย USB
2. ค้นหาหมายเลขพอร์ต (Port) ที่บอร์ด ESP32 เชื่อมต่ออยู่ (ตามวิธีในหัวข้อ 2.3)
   - Windows: เช่น COM3, COM4
   - macOS: เช่น /dev/cu.usbserial-xxxx หรือ /dev/cu.SLAB_USBtoUART
   - Linux: เช่น /dev/ttyUSB0

2.4.4 การลบข้อมูลใน Flash (Erasing the flash)

ก่อนติดตั้งเฟิร์มแวร์ใหม่ แนะนำให้ลบข้อมูลทั้งหมดในหน่วยความจำ Flash ของ ESP32 ก่อน เพื่อป้องกันปัญหาที่อาจเกิดจากข้อมูลเก่าที่หลงเหลืออยู่ ในส่วนของการ ลบเฟิร์มแวร์นั้น สามารถทำได้ทั้งวิธี Command Line หรือ ใช้โปรแกรมสำเร็จรูปแบบ GUI ก็ได้เหมือนกัน ส่วนนนี้แล้วแต่คนถนัดการใช้โปรแกรมแบบ GUI ต้องดาวน์โหลดโปรแกรมมาก่อน แต่ถ้าจะใช้แบบ command line ก็เปิด terminal พิมพ์คำสั่งได้เลย แต่ต้องดูหมายเลข port ที่เชื่อมต่อด้วย

1. เปิด Command Prompt หรือ Terminal
2. พิมพ์คำสั่ง esptool.py โดยแทนที่ [PORT] ด้วยหมายเลขพอร์ตที่ถูกต้องด้วย

esptool.py --chip esp32 --port [PORT] erase_flash

ตัวอย่างสำหรับ Windows: esptool.py --chip esp32 --port COM3 erase_flash

ตัวอย่างสำหรับ macOS/Linux: esptool.py --chip esp32 --port /dev/ttyUSB0 erase_flash

นี่คือการแฟลชเฟิร์มแวร์ด้วยการพิมพ์คำสั่ง Command-Line

หากไม่ถนัดการพิมพ์คำสั่งแบบ **command line** ก็มีทางเลือก:

🔧 **ทางเลือกการใช้เครื่องมือแบบ GUI ที่ใช้งานง่ายกว่า**

หากไม่ต้องการพิมพ์คำสั่ง ก็ใช้โปรแกรม GUI เหล่านี้แทนได้เลย จะปลอดภัย และ ใช้งานง่ายกว่า

✅ **1. ESPHome-Flasher**

- มีปุ่ม "Erase Flash" โดยตรง
- ใช้งานง่าย เหมาะกับผู้ไม่ถนัด Command Line

📌 ใช้ได้กับ: ESP32 / ESP8266

🔗 https://github.com/esphome/esphome-flasher/releases

✅ **2. ESP32 Flash Download Tool (Espressif Official)**

- มี checkbox ให้เลือก **“Erase Flash”**
- รองรับพาร์ติชันแบบละเอียด

📌 ใช้ได้กับ: ESP32 โดยตรง

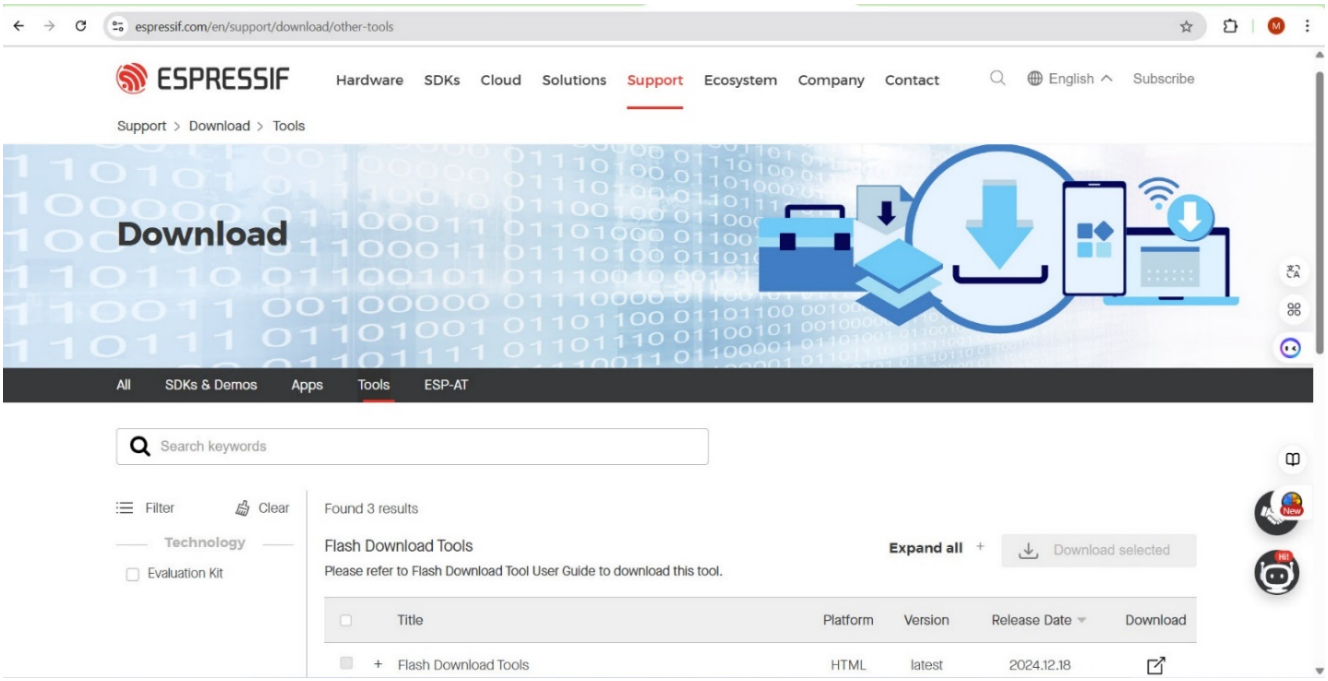
🔗 https://www.espressif.com/en/support/download/other-tools

✅ **3. NodeMCU PyFlasher (สำหรับ ESP8266)**

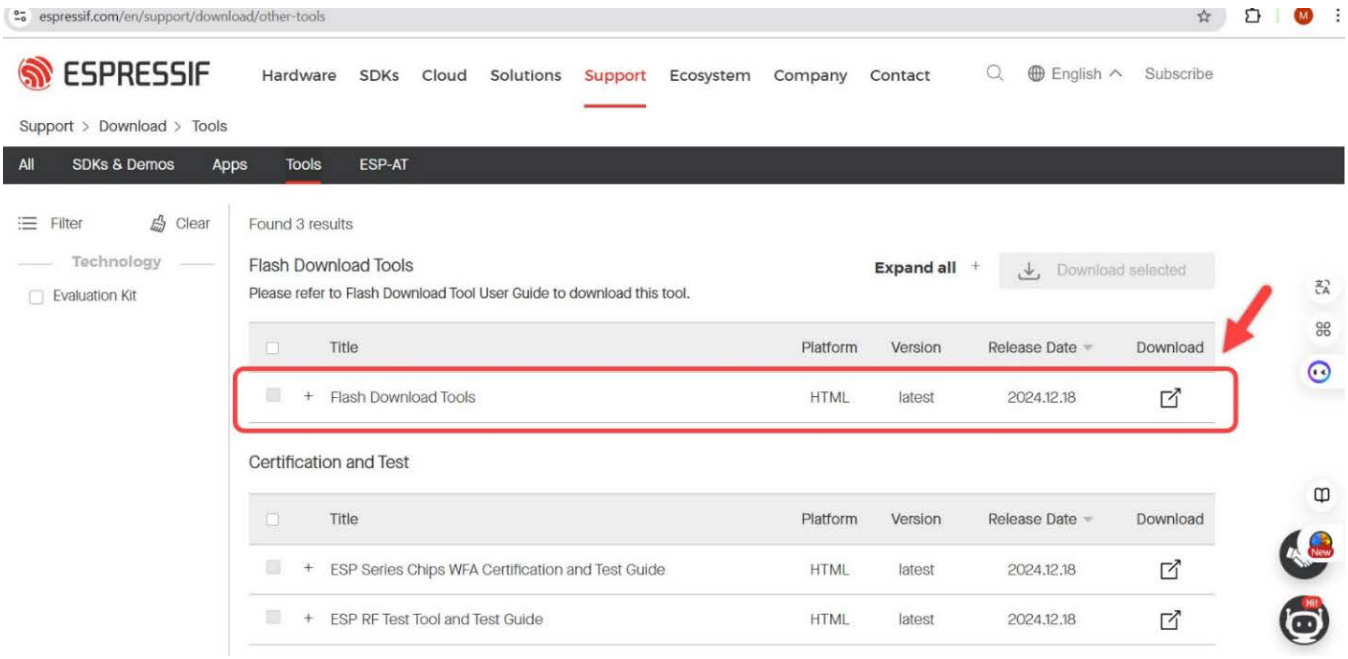
- มีปุ่ม **“Erase Flash before flashing”**
- ใช้งานง่ายมากในสาย MicroPython

แต่ในหนังสือเล่มนี้ผู้เขียนขอเลือกใช้ **ESP32 Flash Download Tool** มาเป็นตัวอย่าง เพราะเป็นโปรแกรมของ **Espressif** เอง

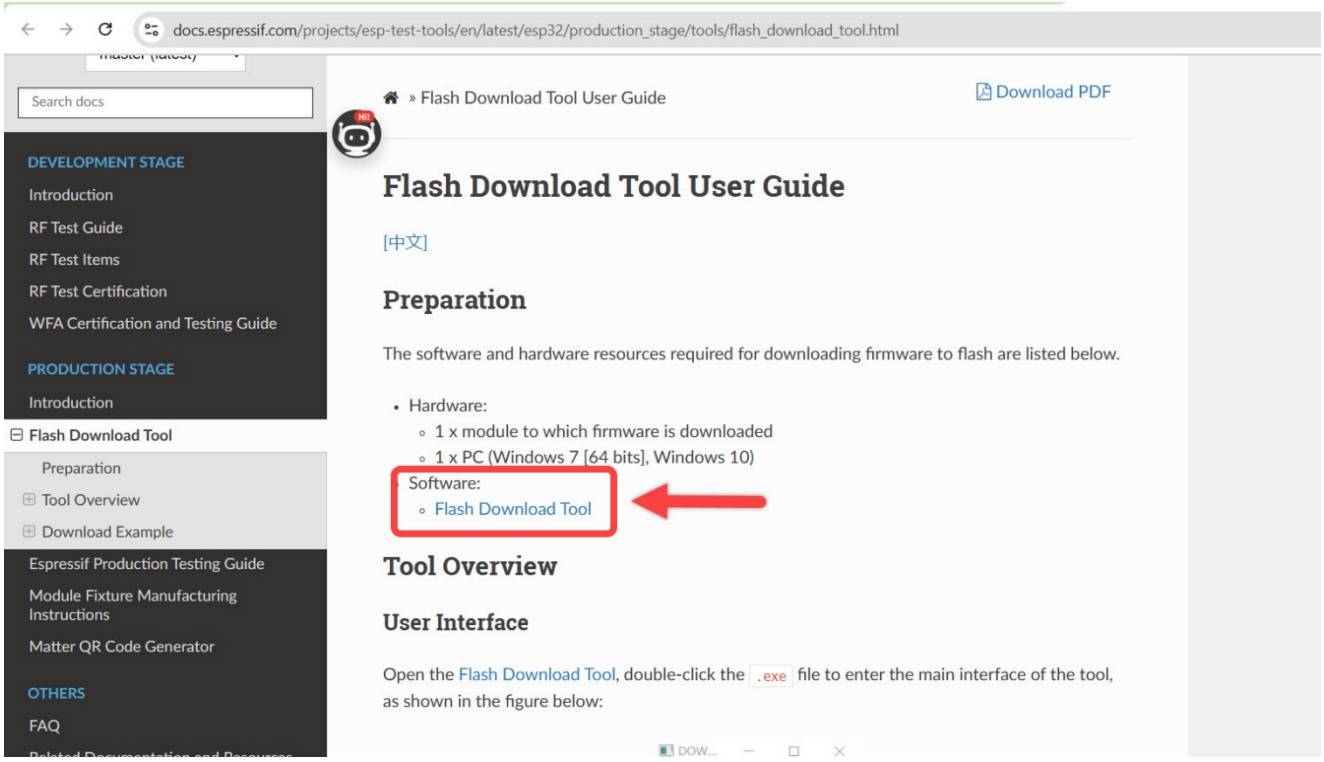
ให้ผู้อ่านเข้าไปในเว็บไซต์ https://www.espressif.com/en/support/download/other-tools



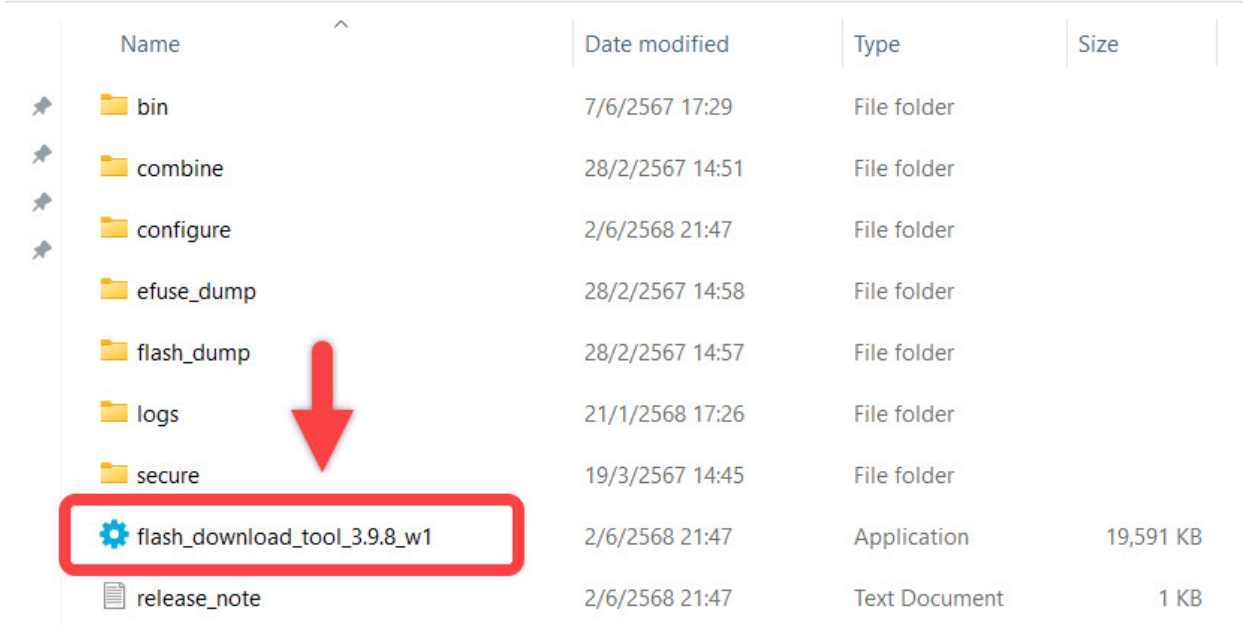
เมื่อเข้าไปที่หน้าเว็บไซต์จะพบกับหน้าตาแบบนี้



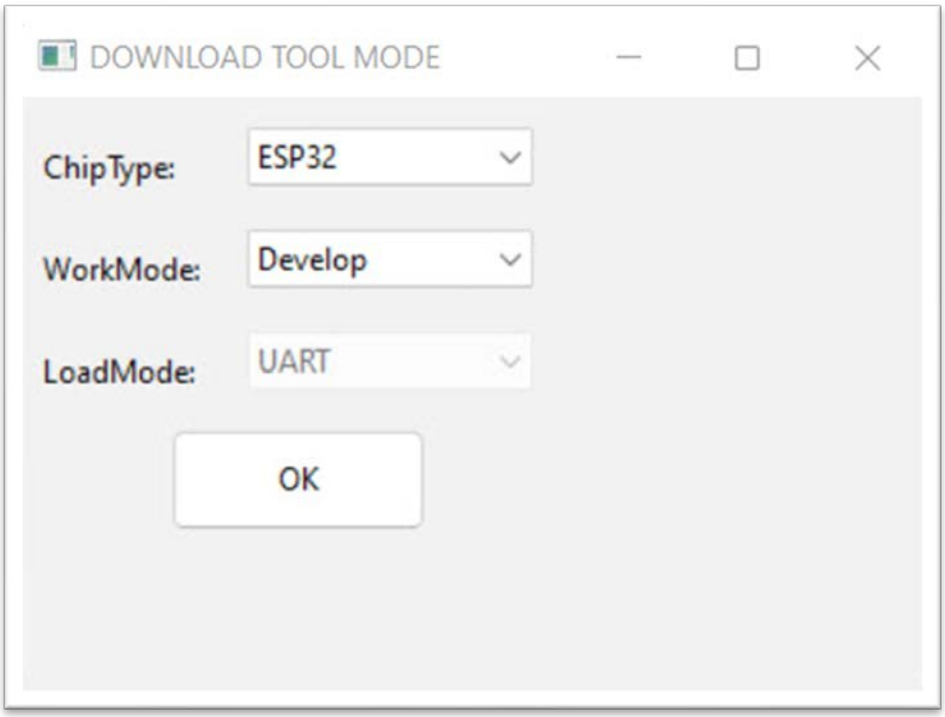
ให้คลิกที่ flash Download Tools



คลิกที่ Flash Download Tool เพื่อดาวน์โหลดโปรแกรม



เมื่อแตกไฟล์แล้วจะได้ไฟล์ตามภาพ เปิดไฟล์โปรแกรมที่ลูกศรชี้ด้วยการดับเบิ้ลคลิก



หน้าตาโปรแกรมเริ่มแรกจะเป็นแบบนี้ ให้เลือก Chipset ให้ตรงกับที่เราจะใช้งาน

และในส่วนของ **WorkMode** นั้นมี 2 โหมด คือ **Develop** และ **Factory** ขออธิบายเพิ่มเติมว่า

ความแตกต่างระหว่างโหมด Factory และ Develop ในโปรแกรม ESP32 Flash Download Tool

เมื่อเปิดใช้งานโปรแกรม **ESP32 Flash Download Tool** ซึ่งเป็นเครื่องมืออย่างเป็นทางการจากบริษัท **Espressif** ผู้พัฒนาไมโครคอนโทรลเลอร์ตระกูล ESP32 ผู้ใช้จะพบเมนูให้เลือก **Work Mode** ซึ่งประกอบด้วยสองตัวเลือกหลัก คือ **Factory Mode** และ **Develop Mode** โดยแต่ละโหมดมีวัตถุประสงค์ในการใช้งานแตกต่างกันอย่างชัดเจน ดังนี้

1. Develop Mode (โหมดนักพัฒนา)

Develop Mode เป็นโหมดที่ออกแบบมาสำหรับผู้พัฒนาเฟิร์มแวร์ทั่วไปหรือผู้ใช้งาน ESP32 ในระดับบุคคลหรือองค์กรขนาดเล็ก โดยมีคุณสมบัติที่เปิดให้ผู้ใช้สามารถกำหนดรายละเอียดต่าง ๆ ได้อย่างยืดหยุ่น เช่น:

- การเลือกพอร์ต (COM port) และความเร็ว Baud Rate
- การกำหนดไฟล์เฟิร์มแวร์ .bin หลายไฟล์ เช่น bootloader, partition table, firmware
- การเลือกตำแหน่งหน่วยความจำ (offset) ของแต่ละไฟล์
- การลบข้อมูลในหน่วยความจำ Flash ทั้งหมด (Erase Flash) ก่อนแฟลช
- การแสดง Log และสถานะการแฟลชแบบเรียลไทม์

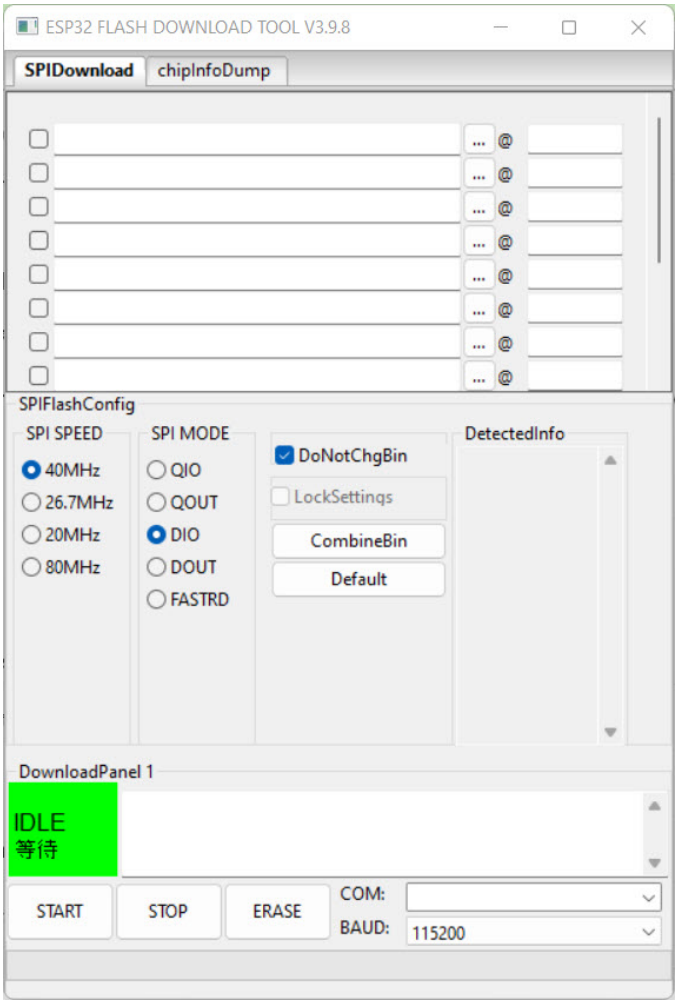
โหมดนี้เหมาะกับงานพัฒนา ทดลอง แก้ไขโค้ด หรือแฟลชเฟิร์มแวร์ด้วยตนเองในสถานการณ์ทั่วไป เช่น การอัปโหลดเฟิร์มแวร์ MicroPython, Arduino หรือเฟิร์มแวร์ที่คอมไพล์เองจาก ESP-IDF

2. Factory Mode (โหมดโรงงาน)

Factory Mode เป็นโหมดเฉพาะทางที่ออกแบบมาสำหรับกระบวนการผลิตในระดับอุตสาหกรรม โดยรองรับการเขียนเฟิร์มแวร์ลงในอุปกรณ์หลายตัวแบบต่อเนื่องอัตโนมัติ (Mass Production) โหมดนี้สามารถเชื่อมต่อกับระบบสายพานการผลิต หรือเครื่องเขียนเฟิร์มแวร์ได้ และมักใช้งานร่วมกับสคริปต์หรือไฟล์ควบคุมเฉพาะ เช่น:

- การกำหนดหมายเลข MAC Address หรือ Serial Number ที่ไม่ซ้ำกันในแต่ละอุปกรณ์
- การบันทึกหรืออ่านข้อมูลเฉพาะทางจาก Flash หลังแฟลชเสร็จ
- การควบคุมการเขียนแฟลชแบบไม่มีการแสดง GUI (Headless)

โหมดนี้เหมาะสำหรับโรงงานหรือผู้ผลิตสินค้าจำนวนมากที่ต้องการกระบวนการผลิตอัตโนมัติ รวดเร็ว และควบคุมคุณภาพอย่างเข้มงวด เมื่อเลือกแล้วกดปุ่ม OK จะได้หน้าต่างใหม่ตามภาพ



การตั้งค่าพารามิเตอร์สำคัญใน ESP32 Flash Download Tool

ในการใช้งานโปรแกรม ESP32 Flash Download Tool ผู้ใช้จำเป็นต้องกำหนดค่าพารามิเตอร์ต่าง ๆ ให้เหมาะสมกับเฟิร์มแวร์และบอร์ด ESP32 ที่ใช้งาน โดยเฉพาะในส่วนของ **SPIFlashConfig** ซึ่งมีผลโดยตรงต่อความเร็วและความเข้ากันได้ของการแฟลชเฟิร์มแวร์

1. SPI SPEED (ความเร็วในการส่งข้อมูลแฟลช)

ตัวเลือกนี้กำหนดความเร็วของสัญญาณ SPI ที่ใช้ในการเขียนข้อมูลไปยัง Flash

ค่า	ความหมาย	ข้อแนะนำ
20 MHz	ความเร็วต่ำสุด	ใช้ในกรณีที่แฟลชไม่เสถียร
26.7 MHz	ความเร็วกลาง	ใช้งานได้ทั่วไป
40 MHz	ความเร็วมาตรฐาน **(ค่าเริ่มต้น)**	แนะนำสำหรับการใช้งานทั่วไป
80 MHz	ความเร็วสูงสุด	ใช้ในกรณีบอร์ดหรือ Flash รองรับความเร็วสูง

แนะนำ: ให้เลือก **40 MHz** เป็นค่าเริ่มต้น เพราะเหมาะกับบอร์ด ESP32 ทั่วไป และมีความเสถียรในการแฟลช

---

2. SPI MODE (โหมดการสื่อสารของแฟลช)

ตัวเลือกนี้กำหนดรูปแบบการเชื่อมต่อระหว่างชิป ESP32 กับ Flash memory

โหมด	คำอธิบาย	ความเข้ากันได้
QIO	Quad I/O (4 บิต)	ใช้กับ Flash ที่รองรับ QIO
DIO	Dual I/O (2 บิต)	**ใช้กันทั่วไป**, ปลอดภัย, เสถียร
DOUT	Dual Output	ใช้ในบางกรณี
QOUT	Quad Output	ใช้กับ Flash บางรุ่น
FASTRD	Fast Read	เร็ว แต่ไม่รองรับทุกบอร์ด

แนะนำ: เลือกเป็น **DIO** สำหรับการใช้งานทั่วไป เพราะเป็นโหมดที่ใช้ได้กับบอร์ดส่วนใหญ่ และให้ความเสถียรสูง

---

3. ตัวเลือกอื่น ๆ ที่สำคัญ

- **DoNotChgBin**

หากติ๊กถูกไว้ (ค่าเริ่มต้น) โปรแกรมจะไม่เปลี่ยนแปลงไฟล์ .bin ที่เลือกไว้

แนะนำให้เปิดไว้เสมอ เพื่อป้องกันการเปลี่ยนแปลงโดยไม่ตั้งใจ

- **CombineBin**

  ใช้รวมไฟล์ .bin หลายไฟล์เป็นไฟล์เดียว (สำหรับ advanced user เท่านั้น)

- **Default**

  ปุ่มนี้จะรีเซ็ตการตั้งค่าทั้งหมดกลับเป็นค่าเริ่มต้นของโปรแกรม

สรุปการตั้งค่าที่แนะนำสำหรับผู้เริ่มต้น

ตัวเลือก	**ค่าที่แนะนำ**
SPI SPEED	40 MHz
SPI MODE	DIO
DoNotChgBin	✓ เปิดไว้
CombineBin	ไม่ต้องใช้
COM Port	ตรวจสอบจาก Device Manager
Baudrate	115200 หรือ 921600 /460800(flash)

การตั้งค่าทั้งหมดนี้สามารถใช้ร่วมกับเฟิร์มแวร์ MicroPython, Arduino, หรือ ESP-IDF ได้อย่างมีเสถียรภาพ โดยไม่ต้องแก้ไขเพิ่มเติมในกรณีทั่วไป หากใช้งานกับบอร์ดที่มีพฤติกรรมพิเศษ หรือแฟลชประเภทพิเศษ อาจต้องปรับค่าบางส่วนตามคู่มือของผู้ผลิตบอร์ดนั้น ๆ

การลบข้อมูลใน Flash ของ ESP32 ก่อนแฟลชเฟิร์มแวร์ MicroPython

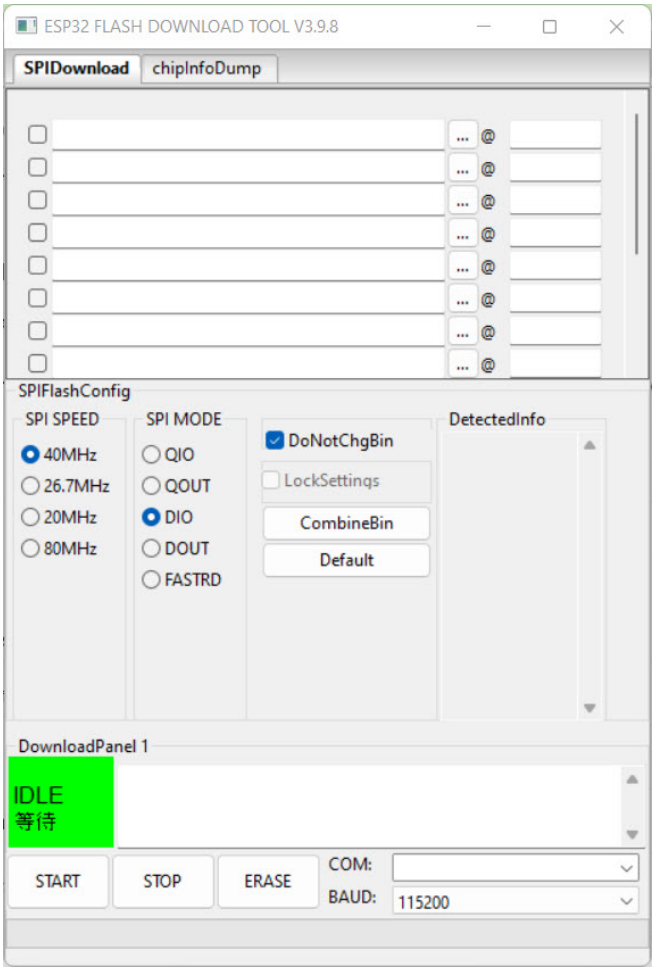
ก่อนการติดตั้งเฟิร์มแวร์ MicroPython หรือเฟิร์มแวร์ใด ๆ ลงใน ESP32 แนะนำให้ลบข้อมูลทั้งหมดในหน่วยความจำ Flash ของบอร์ดก่อน เพื่อป้องกันปัญหาที่อาจเกิดจากข้อมูลเก่าหลงเหลืออยู่ เช่น โค้ดจากเฟิร์มแวร์เดิม หรือการจัดวางพาร์ติชันที่ไม่สอดคล้องกัน ซึ่งอาจทำให้เฟิร์มแวร์ใหม่ทำงานไม่สมบูรณ์

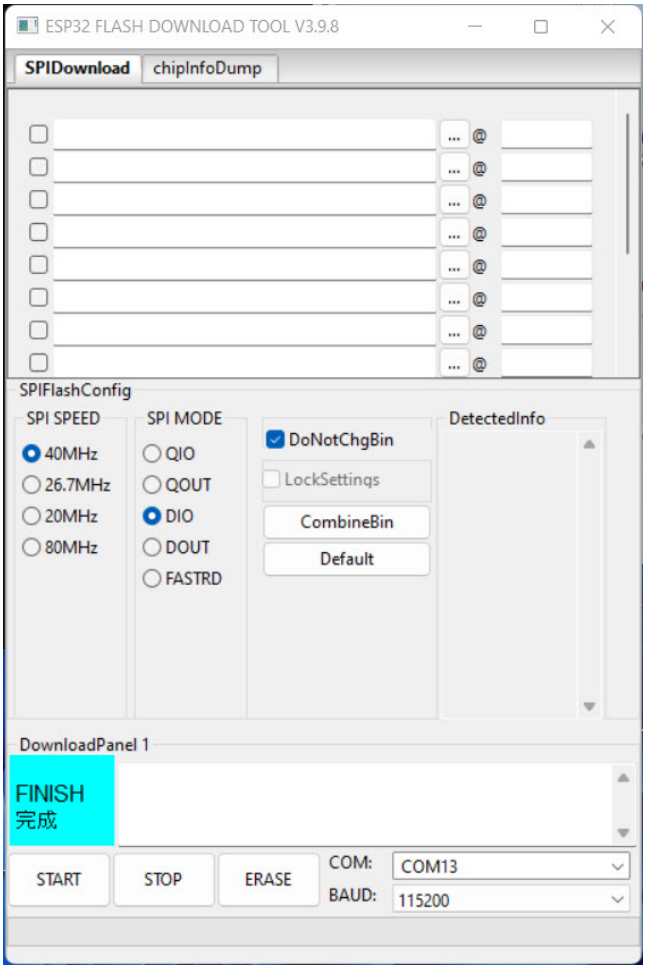
หากไม่ต้องการใช้คำสั่งผ่าน Terminal การลบ Flash สามารถทำได้ผ่านโปรแกรม **ESP32 Flash Download Tool** ซึ่งเป็นซอฟต์แวร์แบบกราฟิกที่พัฒนาโดย Espressif ผู้ผลิตชิป ESP โดยตรง

ขั้นตอนการลบ Flash ด้วย ESP32 Flash Download Tool

1. เปิดโปรแกรม **ESP32 Flash Download Tool** และเลือกแท็บ SPIDownload
2. ปล่อยช่องใส่ไฟล์ทั้งหมดว่างไว้ เนื่องจากต้องการลบ Flash เท่านั้น ไม่ได้แฟลชไฟล์ใด
3. กำหนดพอร์ตเชื่อมต่อของบอร์ดในช่อง COM: (เช่น COM3, COM4 เป็นต้น)
4. กำหนดความเร็ว Baudrate ที่ใช้ในการเชื่อมต่อ เช่น 115200
5. คลิกปุ่ม **ERASE** เพื่อเริ่มต้นลบข้อมูลใน Flash
6. รอจนกระทั่งสถานะด้านล่างแสดงข้อความว่า "FINISH" หรือ "ERASE SUCCESS"

เมื่อกระบวนการลบเสร็จสมบูรณ์ หน่วยความจำ Flash ของ ESP32 จะอยู่ในสถานะพร้อมสำหรับการติดตั้งเฟิร์มแวร์ใหม่ได้อย่างปลอดภัย





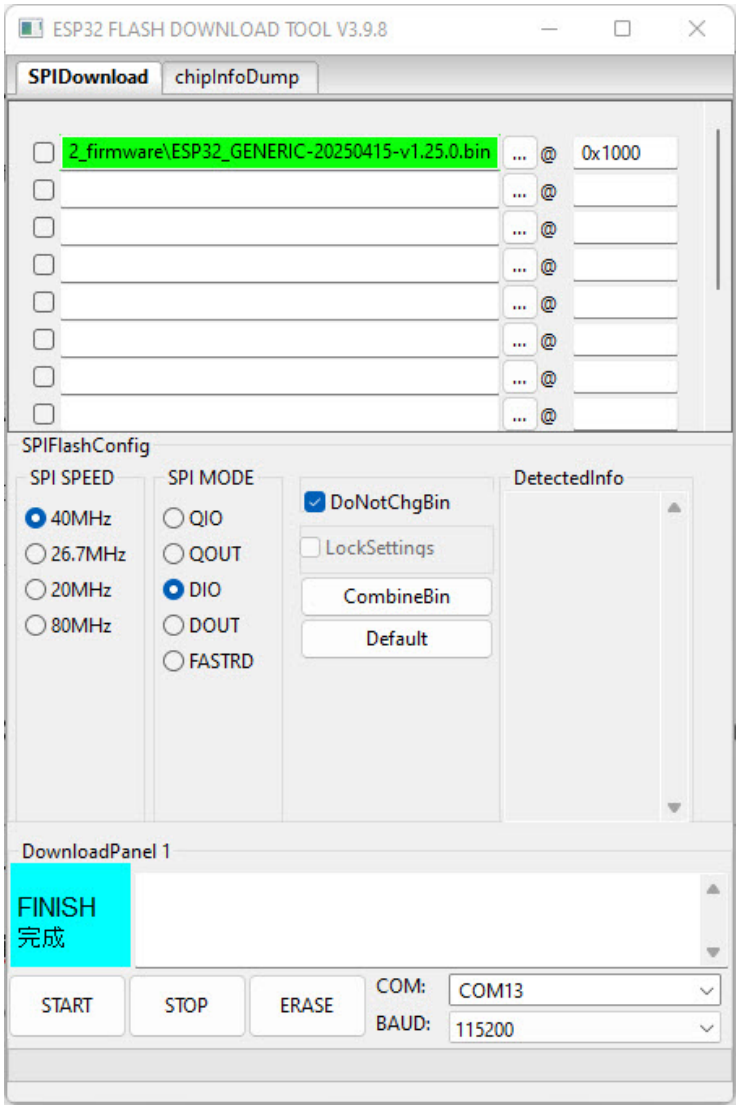
2.4.5 การเขียนเฟิร์มแวร์ (Writing the firmware)

การเตรียมแฟลชเฟิร์มแวร์ MicroPython

หลังจากลบ Flash แล้ว สามารถดำเนินการแฟลชเฟิร์มแวร์ MicroPython ได้โดยใช้เครื่องมือเดียวกัน โดยมีขั้นตอนเพิ่มเติมดังนี้:

1. ใช้เฟิร์มแวร์ .bin ของ MicroPython ที่ดาวน์โหลดมาจากในหัวข้อ 2.4.1

2. ในหน้าหลักของโปรแกรม ESP32 Flash Download Tool:
   - ในบรรทัดแรกของช่องใส่ไฟล์ คลิกปุ่ม ... เพื่อเลือกไฟล์ .bin ที่ดาวน์โหลดมา



   - ระบุตำแหน่งการแฟลช (offset) เป็น 0x1000 ซึ่งเป็นตำแหน่งมาตรฐานของเฟิร์มแวร์ MicroPython

ทำไมต้องระบุตำแหน่ง 0x1000 เมื่อแฟลชเฟิร์มแวร์ MicroPython?

การระบุตำแหน่ง 0x1000 ในขั้นตอนการแฟลชเฟิร์มแวร์ MicroPython ลงบนบอร์ด ESP32 ไม่ใช่การกำหนดตำแหน่งแบบสุ่ม แต่เป็นการอ้างอิงถึง **โครงสร้างหน่วยความจำแฟลช (Flash memory map)** ที่ถูกกำหนดมาโดย MicroPython และ ESP-IDF (ESP32 SDK)

◆ ความเข้าใจเบื้องต้นเกี่ยวกับ Flash Memory ของ ESP32

ESP32 มีหน่วยความจำแฟลชภายนอก (external flash) ซึ่งแบ่งออกเป็นหลายส่วน และถูกโหลดเข้าตาม **offset address** ที่เฉพาะเจาะจง เช่น:

- 0x1000 ⟶ Bootloader
- 0x8000 ⟶ Partition Table
- 0x10000 ⟶ Application หรือ Firmware หลัก

แต่สำหรับ MicroPython นั้น ได้จัดเตรียม **ไฟล์ .bin เดียว** ซึ่งรวมทุกส่วนเอาไว้แล้ว ไม่จำเป็นต้องแยกแฟลชไฟล์ bootloader หรือ partition table เหมือนกับบางเฟิร์มแวร์ของ ESP-IDF

◆ เหตุผลที่แฟลช MicroPython ต้องใช้ตำแหน่ง 0x1000

แม้ว่าไฟล์ .bin ที่ดาวน์โหลดจาก MicroPython.org จะเป็นแบบรวมไฟล์ (monolithic binary) แต่ก็ออกแบบมาให้ถูกโหลดที่ตำแหน่ง 0x1000 โดยเฉพาะ ซึ่งเป็น **ตำแหน่งมาตรฐานของ Bootloader** ในระบบ ESP32 โดย Bootloader นี้มีหน้าที่เริ่มระบบและโหลดโค้ดหลักเข้า RAM เพื่อนำไปประมวลผล

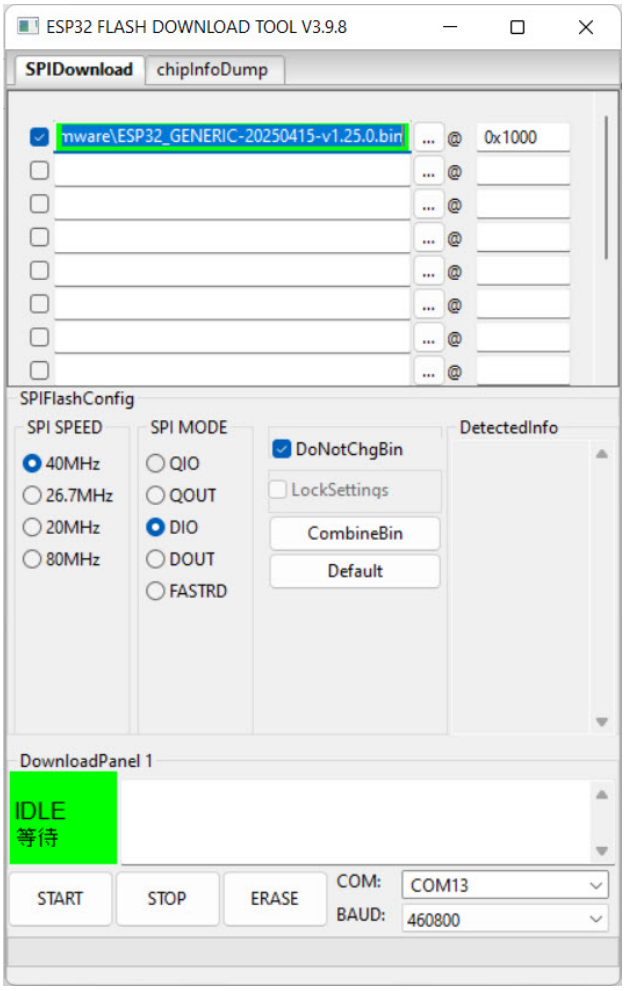
หากแฟลชไปที่ตำแหน่งผิด เช่น 0x0000 หรือ 0x2000 อาจทำให้บอร์ดไม่สามารถบู๊ตเข้าสู่ MicroPython ได้เลย เพราะ Bootloader จะไม่อยู่ในตำแหน่งที่สามารถเรียกบู๊ตได้

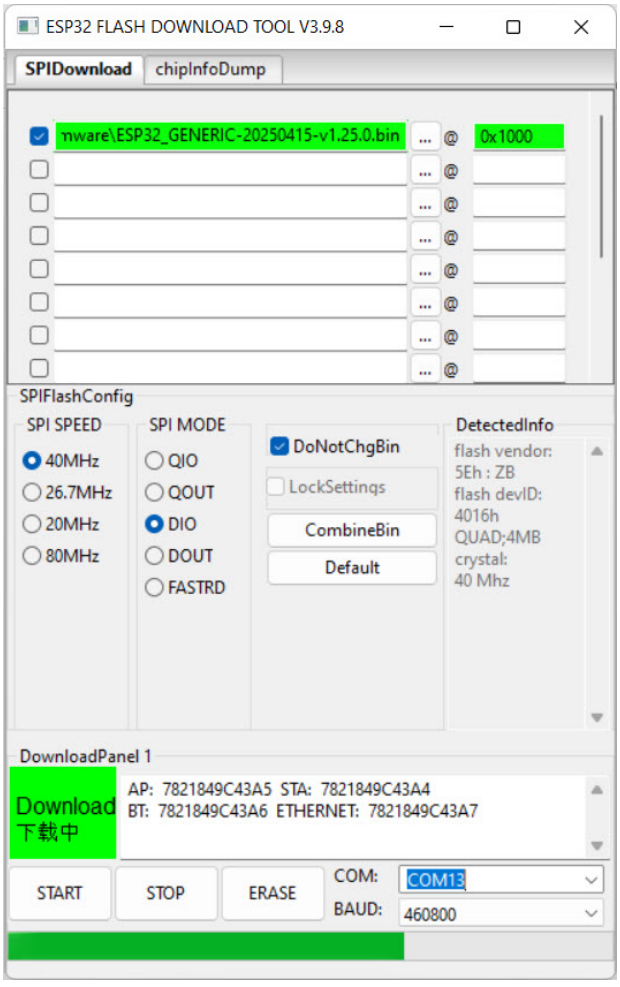
◆ เปรียบเทียบให้เห็นภาพ

ตำแหน่งในแฟลช (Offset)	เนื้อหาที่ควรอยู่	หมายเหตุ
0x1000	Bootloader (MicroPython รวมไว้แล้ว)	ตำแหน่งที่ต้องระบุ
0x8000	Partition Table	รวมใน .bin แล้ว
0x10000	Firmware หลัก (MicroPython REPL)	รวมใน .bin แล้ว

การระบุตำแหน่งแฟลชที่ 0x1000 เป็นการอ้างอิงตามโครงสร้างที่ MicroPython กำหนดไว้ในเฟิร์มแวร์แบบ monolithic ซึ่งรวม Bootloader, Partition Table และ Firmware ไว้ในไฟล์เดียว และถูกออกแบบมาให้ทำงานถูกต้องเมื่อลงที่ตำแหน่ง 0x1000 หากระบุตำแหน่งไม่ถูกต้อง ESP32 จะไม่สามารถบูตเข้าสู่ MicroPython ได้ เพราะจะหา Bootloader ไม่เจอหรือโหลดผิดที่

3. **ตรวจสอบพอร์ต และ Baudrate ให้ถูกต้อง ในการแฟลชนั้นแนะนำให้ตั้งค่าที่ 460800**
4. คลิกปุ่ม **START** เพื่อเริ่มแฟลชเฟิร์มแวร์ลงบนบอร์ด จะมีแถบแสดงสีเขียวด้านล่าง

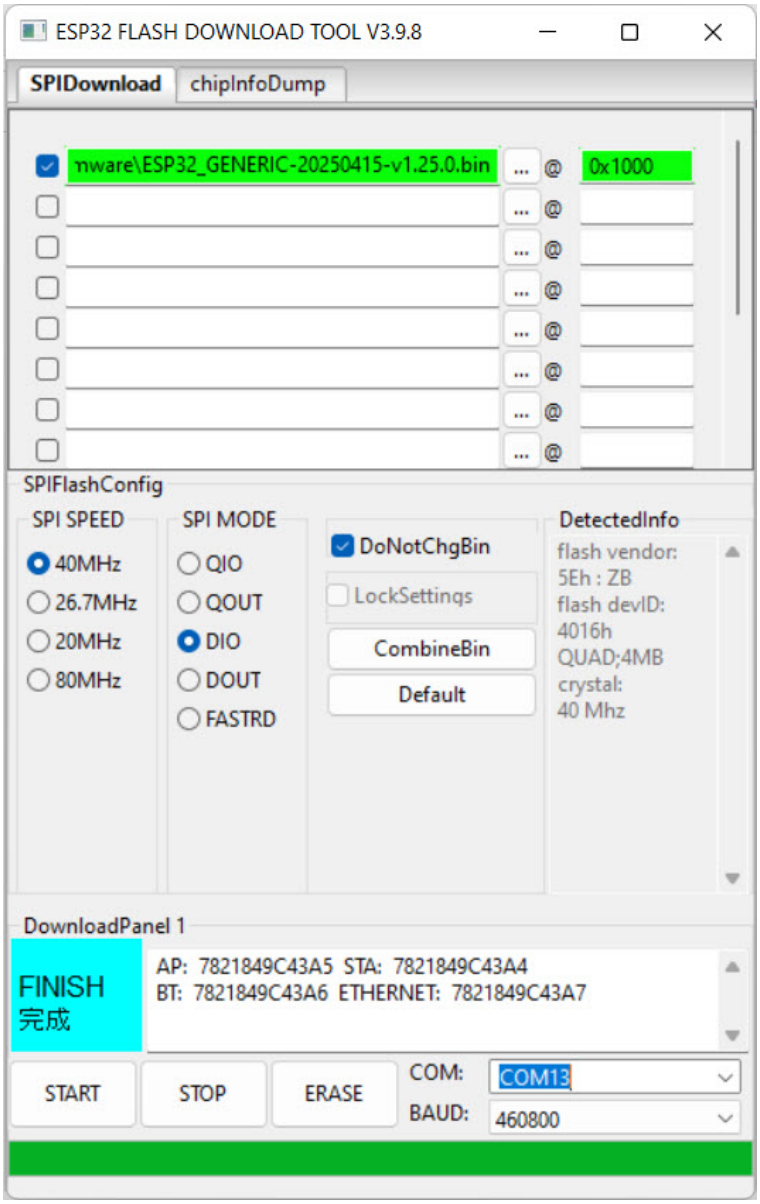




เมื่อแฟลชเสร็จสมบูรณ์ บอร์ด ESP32 จะพร้อมใช้งานกับ MicroPython ทันที โดยสามารถเชื่อมต่อผ่าน Serial Terminal หรือโปรแกรม Thonny IDE เพื่อเริ่มต้นการเขียนโปรแกรมต่อไป

การยืนยันการแฟลชเฟิร์มแวร์ และการแสดงรหัสประจำตัวของบอร์ด ESP32

หลังจากดำเนินการแฟลชเฟิร์มแวร์ MicroPython ลงบนบอร์ด ESP32 โดยใช้โปรแกรม ESP32 Flash Download Tool แล้ว ผู้ใช้จะได้รับข้อความแจ้งเตือน และโปรแกรมจะดึงข้อมูลรหัสประจำตัวของบอร์ดมาแสดงในช่องด้านล่างของหน้าต่าง เมื่อการแฟลชสำเร็จ โปรแกรมจะแสดงข้อความ **"FINISH 完成"** ที่มุมล่างซ้าย ซึ่งเป็นภาษาจีน แปลว่า "เสร็จสมบูรณ์" แสดงว่าเฟิร์มแวร์ถูกเขียนลงในหน่วยความจำของ **ESP32** เรียบร้อยแล้ว โดยไม่มีข้อผิดพลาด ขั้นตอนต่อไปคือการรีสตาร์ทบอร์ดด้วยการกดปุ่ม **EN** (หรือ **RESET**) บนบอร์ด หรือถอดสาย USB แล้วเสียบใหม่ เพื่อให้บอร์ดเริ่มต้นทำงานด้วยเฟิร์มแวร์ใหม่นี้



การแสดง MAC Address ของ ESP32

หลังจากแฟลชเสร็จ โปรแกรมจะอ่านข้อมูล MAC Address ของระบบสื่อสารต่าง ๆ ที่อยู่ภายใน ESP32 และแสดงออกมาดังตัวอย่าง แต่ละบอร์ดจะมีค่าที่แตกต่าง และไม่ซ้ำกัน

AP: 7821849C43A5 STA: 7821849C43A4

BT: 7821849C43A6 ETHERNET: 7821849C43A7

MAC Address แต่ละค่าคือหมายเลขประจำตัวเฉพาะของอุปกรณ์เครือข่ายภายในบอร์ด เลขเหล่านี้เปรียบได้กับ "เลขบัตรประชาชนของ ESP32" ที่ใช้ระบุตัวตนในระบบเครือข่าย และที่สำคัญคือ **ไม่สามารถเปลี่ยนแปลงได้**

ความหมายของแต่ละค่า

รหัสตัวย่อ	ชื่อเต็ม	การใช้งาน
AP	Wi-Fi Access Point	ใช้เมื่อบอร์ดเป็นตัวปล่อยสัญญาณ Wi-Fi
STA	Wi-Fi Station	ใช้เมื่อบอร์ดเชื่อมต่อ Wi-Fi ภายนอก
BT	Bluetooth	ใช้ในการเชื่อมต่อ Bluetooth กับอุปกรณ์อื่น
ETHERNET	Ethernet	ใช้เมื่อเชื่อมต่อกับโมดูล LAN PHY ภายนอก

ความสำคัญของ MAC Address

MAC Address ใน ESP32 ถูกฝังถาวรไว้ในหน่วยความจำชนิดพิเศษชื่อ eFuse ซึ่งเป็นหน่วยความจำแบบเขียนได้ครั้งเดียว ดังนั้นค่าที่แสดงออกมาเหล่านี้จะไม่สามารถเปลี่ยนแปลงได้ และแตกต่างกันในแต่ละบอร์ดอย่างแน่นอน 100% แม้ว่าโปรแกรมเมอร์จะสามารถตั้งค่า MAC Address ปลอมได้ในซอฟต์แวร์ (spoofing) แต่เมื่อบอร์ดถูกรีเซตหรือเปิดใหม่ MAC Address จะกลับคืนเป็นค่าต้นฉบับที่ถูกฝังไว้ในโรงงานเสมอ

- การแสดงข้อความ **FINISH 完成** หมายถึงการแฟลชเฟิร์มแวร์สำเร็จแล้ว
- ข้อมูล AP, STA, BT, ETHERNET คือ MAC Address ถาวรที่แตกต่างกันในแต่ละบอร์ด
- MAC Address จะไม่มีทางซ้ำกัน และไม่สามารถเปลี่ยนได้
- ใช้ในการระบุตัวตนของบอร์ดในระบบ Wi-Fi, Bluetooth หรือ Ethernet ได้อย่างแม่นยำ

หากนำ ESP32 หลายตัวมาใช้งานร่วมกันในระบบเดียว เช่น ระบบบ้านอัจฉริยะ MAC Address เหล่านี้จะช่วยให้แยกบอร์ดแต่ละตัวได้อย่างชัดเจนแม้มีหน้าตาเหมือนกันทุกประการ

2.5 Thonny IDE: เพื่อนคู่ใจนักพัฒนา MicroPython

การเริ่มต้นเรียนรู้ MicroPython จะง่ายขึ้นมาก หากมีเครื่องมือที่ออกแบบมาอย่างเข้าใจผู้ใช้งาน โดยเฉพาะมือใหม่ที่อาจยังไม่ถนัดการเขียนโค้ดหรือใช้งานเครื่องมือซับซ้อน Thonny IDE จึงเป็นทางเลือกยอดนิยมที่นักพัฒนาทั่วโลกแนะนำ เพราะมีจุดเด่นคือความเรียบง่าย ใช้งานง่าย แต่ยังคงทรงพลังและรองรับการทำงานกับบอร์ดไมโครคอนโทรลเลอร์ยอดนิยมอย่างครบถ้วน

Thonny เป็นโปรแกรมประเภท Integrated Development Environment (IDE) ที่พัฒนาขึ้นโดยมหาวิทยาลัย Tartu จากประเทศเอสโตเนีย โดยเน้นการใช้งานสำหรับภาษา Python เป็นหลัก แต่สิ่งที่ทำให้ Thonny แตกต่างจาก IDE ทั่วไปคือความสามารถในการรองรับ MicroPython ได้โดยตรง ซึ่งทำให้ผู้ใช้งานสามารถเชื่อมต่อกับบอร์ดเช่น ESP32, ESP8266 หรือแม้แต่ Raspberry Pi Pico ได้ทันทีโดยไม่ต้องติดตั้งปลั๊กอินเพิ่มเติม

จุดเด่นข้อแรกของ Thonny IDE คือสามารถเชื่อมต่อกับบอร์ด ESP32 หรือ ESP8266 ได้ทันที หลังจากที่ผู้ใช้แฟลชเฟิร์มแวร์ MicroPython ลงในบอร์ดเรียบร้อยแล้ว เพียงเสียบสาย USB เข้ากับคอมพิวเตอร์ Thonny จะตรวจจับพอร์ตอนุกรม (Serial) ได้อัตโนมัติ ช่วยลดขั้นตอนที่ยุ่งยากและเหมาะอย่างยิ่งสำหรับผู้เริ่มต้น
อีกฟีเจอร์หนึ่งที่โดดเด่นคือการมีหน้าต่าง REPL Console ในตัว ผู้ใช้สามารถพิมพ์คำสั่งแบบสด (Interactive) เพื่อทดสอบการทำงานของบอร์ดได้ทันที เช่น การสั่งพิมพ์ข้อความ การอ่านค่าพิน หรือการทดลองฟังก์ชัน Python ต่าง ๆ โดยไม่ต้องเขียนโค้ดยาวแล้วกดรันเสมอไป ซึ่งช่วยให้การเรียนรู้และแก้ไขโค้ดเป็นไปได้อย่างรวดเร็วและแม่นยำ

Thonny ยังมีระบบจัดการไฟล์ที่รองรับการอัปโหลดไฟล์ .py ไปยังบอร์ด MicroPython ได้โดยตรงผ่านหน้าต่างฝั่งซ้ายของโปรแกรม ซึ่งเปรียบเสมือนตัวจัดการไฟล์ในระบบปฏิบัติการ ทำให้สามารถลากวางไฟล์ หรือบันทึกโค้ดลงในบอร์ดได้ทันทีโดยไม่ต้องพึ่งพาเครื่องมือเสริม เช่น ampy หรือ mpremote

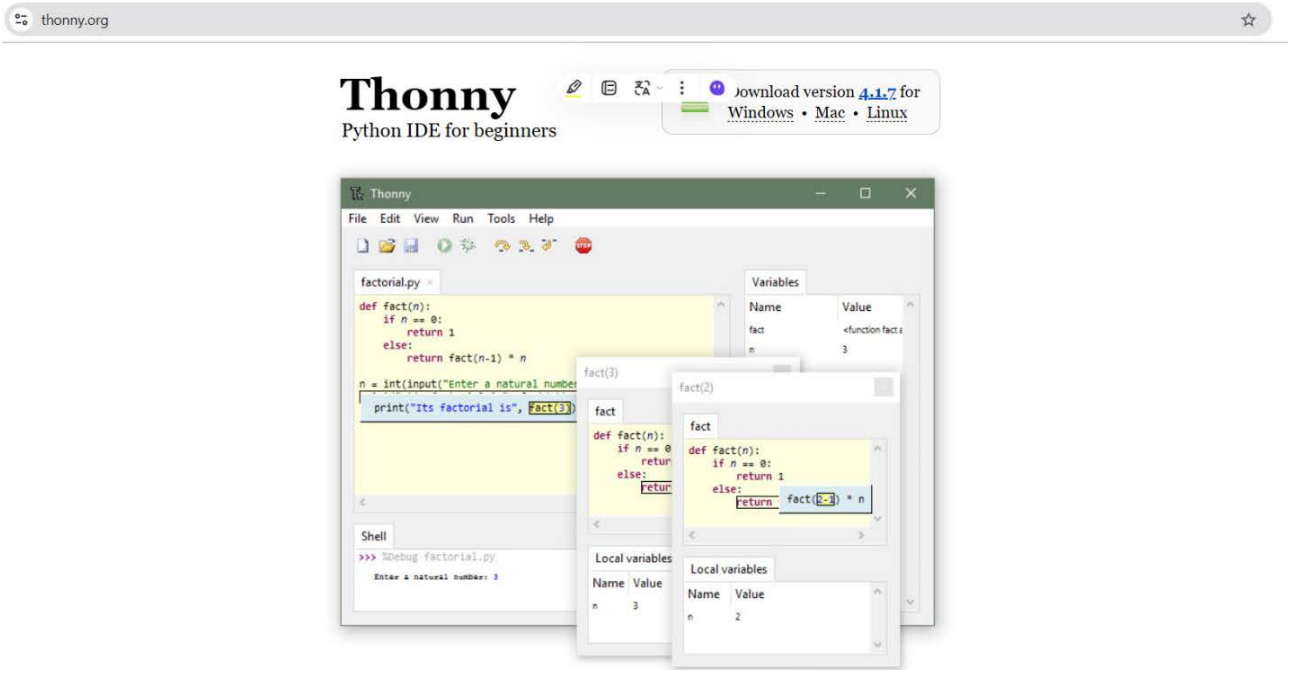
อินเทอร์เฟซของ Thonny ได้รับการออกแบบมาให้เหมาะสำหรับผู้เริ่มต้นอย่างแท้จริง โดยแสดงเฉพาะเมนูและปุ่มที่จำเป็น เช่น Run, Save, Stop, และ Debug ไม่รกสายตาเหมือน IDE ระดับสูงทั่วไป ทำให้ผู้ใช้ไม่รู้สึกสับสนและสามารถจดจ่อกับการเขียนโปรแกรมได้เต็มที่

อีกหนึ่งคุณสมบัติที่น่าประทับใจคือความสามารถในการแสดงค่าตัวแปรแบบภาพ (Visual Variable Inspector) ซึ่งช่วยให้ผู้เรียนสามารถติดตามการเปลี่ยนแปลงค่าของตัวแปรในแต่ละช่วงของโปรแกรมได้อย่างเป็นระบบ เหมาะสำหรับการสอนและการเรียนรู้แนวคิดของโค้ดเชิงลึก โดยเฉพาะผู้ที่ยังไม่เข้าใจการ Debug แบบข้อความ

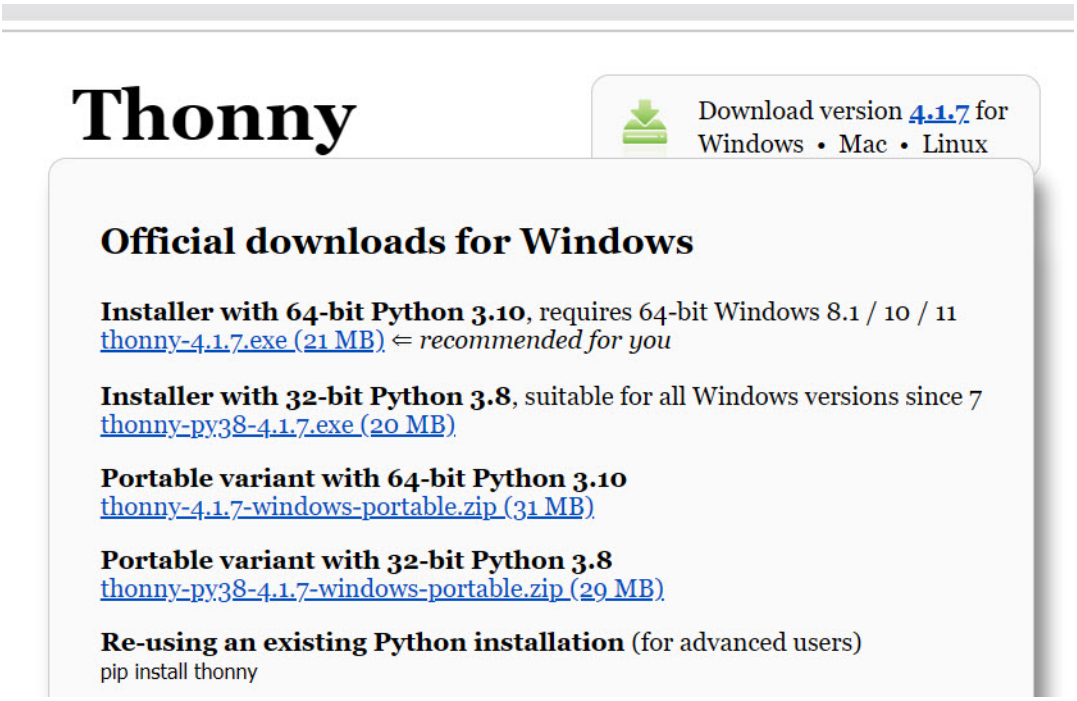
Thonny IDE เหมาะสำหรับนักเรียน นักศึกษา ผู้สอน และผู้เริ่มต้นใช้งาน MicroPython ที่ต้องการเครื่องมือที่ใช้งานง่าย ติดตั้งไม่ยุ่งยาก และสามารถเริ่มต้นได้ทันที เหมาะสำหรับใช้งานทั้งในห้องเรียน เวิร์กช็อป หรือการเรียนรู้ด้วยตนเองที่บ้าน โปรแกรมนี้สามารถใช้งานได้ฟรี และรองรับทั้ง Windows, macOS และ Linux ด้วยคุณสมบัติทั้งหมดนี้ Thonny IDE จึงถือเป็นเครื่องมือหลักที่ควรติดตั้งเป็นอันดับแรก สำหรับผู้ที่ต้องการเข้าสู่โลกของ MicroPython อย่างมั่นใจและมีประสิทธิภาพ

2.5.1 การติดตั้ง Thonny IDE

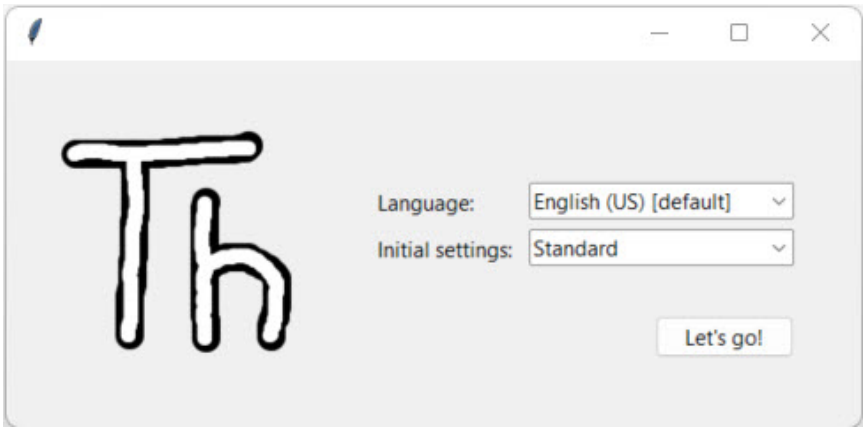
1. ไปที่เว็บไซต์ทางการของ Thonny: https://thonny.org/



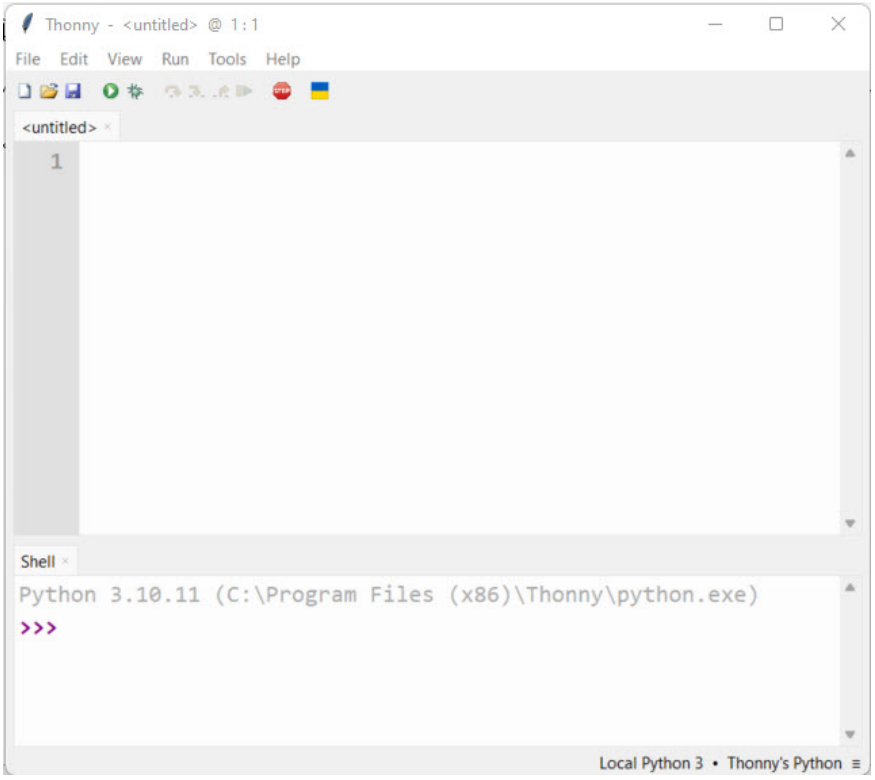
2. ดาวน์โหลดโปรแกรมติดตั้ง (Installer) ที่ตรงกับระบบปฏิบัติการของคอมพิวเตอร์ (Windows, macOS, Linux) แต่สำหรับผู้เขียนขอเลือกเวอร์ชันที่ทำงานบนระบบปฏิบัติการ wondows 8.1/10/11 ให้ผู้อ่านเลือกตามระบบปฏิบัติการที่ท่านใช้งานจริง



3. ทำการติดตั้งตามขั้นตอนปกติของแต่ละระบบปฏิบัติการ เช่นเดียวกับโปรแกรมทั่วไป



เมื่อติดตั้งเสร็จการเปิดใช้งานครั้งแรกโปรแกรมจะถามให้เลือกภาษา แนะนำให้เลือกภาษาอังกฤษ เพื่อความเป็นสากลในการเขียนโปรแกรม และการแนะนำการตั้งค่าต่างๆในโปรแกรมจะได้เป็นไปในทิศทางเดียวกัน หรือใครอยากลองภาษาไทยก็ลองดูครับ



2.5.2 การตั้งค่า Thonny สำหรับ ESP32

1. เปิดโปรแกรม **Thonny IDE**
2. ไปที่เมนู **Tools > Options...**

   (หรือ **Thonny > Preferences...** สำหรับผู้ใช้ macOS)
3. เลือกแท็บ **Interpreter**

4. ในช่อง

   “Which interpreter or device should Thonny use for running your code?”

   ให้เลือกเป็น:

   ⟶ **MicroPython (ESP32)**

5. ในช่อง *“Port or WebREPL”* ให้เลือกพอร์ตที่บอร์ด ESP32 เชื่อมต่ออยู่

   เช่น COM13, /dev/cu.usbserial-xxxx, หรือ /dev/ttyUSB0

   - หากไม่พบพอร์ตที่ถูกต้อง ให้คลิกปุ่ม **Rescan**

     หรือ **ตรวจสอบการเชื่อมต่อบอร์ด** และ **การติดตั้งไดรเวอร์** ว่าถูกต้องหรือไม่
   - หากเพิ่งติดตั้งเฟิร์มแวร์ MicroPython และรีเซ็ตบอร์ดแล้ว

     ESP32 ควรจะพร้อมให้ Thonny เชื่อมต่อทันที

6. คลิกปุ่ม **OK** เพื่อบันทึกการตั้งค่า
7. ที่ด้านล่างของหน้าต่าง Thonny จะมีส่วนที่เรียกว่า **Shell** หรือ **REPL**

   หากการเชื่อมต่อสำเร็จ จะเห็นข้อความต้อนรับจาก MicroPython ปรากฏขึ้น พร้อมเครื่องหมาย >>>

   เช่น:

```
MicroPython v1.25.0 on 2025-04-15; Generic ESP32 module with
ESP32
Type "help()" for more information.
>>>
```

8. หากไม่มีเครื่องหมาย >>> ปรากฏขึ้น ให้ลองกดปุ่ม

   STOP / Restart backend (ปุ่มสีแดงมีไอคอน Stop)

   หรือกลับไปตรวจสอบการตั้งค่าพอร์ตอีกครั้ง

✅ คำแนะนำเพิ่มเติม

💡 **หมายเหตุ:** การเลือก Interpreter และพอร์ตที่ถูกต้องเป็นสิ่งสำคัญ หากเลือกผิด อาจไม่สามารถเชื่อมต่อกับบอร์ด ESP32 ได้ และจะไม่เห็นข้อความ REPL ที่พร้อมใช้งาน

2.6 โปรแกรมแรก: "สวัสดี ESP32!" และการควบคุม LED ออนบอร์ด

เมื่อ Thonny IDE เชื่อมต่อกับ ESP32 ได้แล้ว ก็ถึงเวลาทดลองเขียนโปรแกรมแรก

2.6.1 การใช้งาน REPL เบื้องต้น

REPL (Read-Evaluate-Print Loop) ช่วยให้สามารถส่งคำสั่ง Python ไปยัง ESP32 และรับผลลัพธ์กลับมาได้ทันที

1. ที่หน้าต่าง Shell ของ Thonny (ตรงที่มี >>>) ลองพิมพ์คำสั่ง Python ง่ายๆ

   print("สวัสดี ESP32 จาก MicroPython!") ตามรูปภาพข้างล่าง แล้วกดปุ่ม Enter

จะแสดงผลตอบกลับมาทันที หรืออาจลองพิมพ์บวกเลขง่าย ๆ เช่น 250+250 จะแสดงผลลัพธ์ออกมาดังภาพ

2. **การควบคุม LED ออนบอร์ดผ่าน REPL:** บอร์ด ESP32 ส่วนใหญ่จะมี LED ติดตั้งอยู่บนบอร์ด (Onboard LED) ซึ่งมักจะเชื่อมต่อกับขา GPIO หมายเลข 2 (GPIO2) แต่บางบอร์ดอาจเป็นขาอื่น ให้ตรวจสอบจาก Pinout Diagram ของบอร์ดที่ใช้

ลองพิมพ์คำสั่งต่อไปนี้ทีละบรรทัดใน REPL เพื่อทำให้ LED ติดและดับ (สมมติว่า LED ต่อกับ GPIO2):

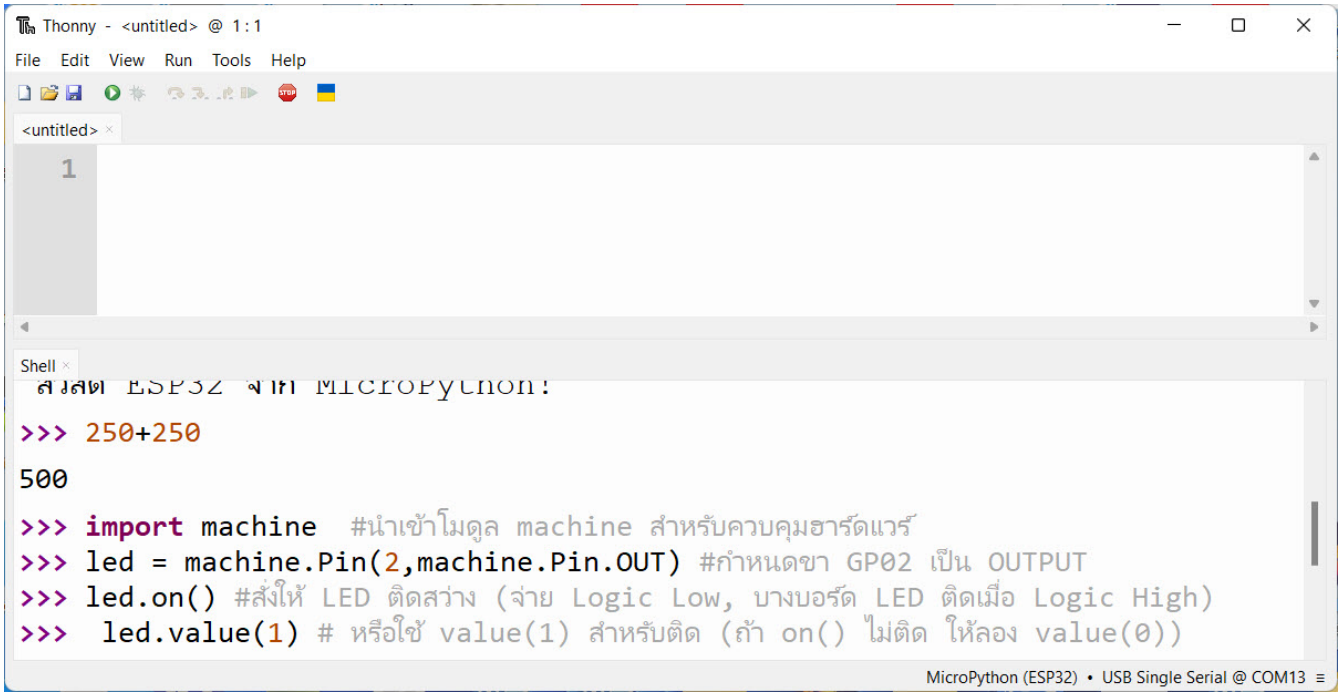
```
>>> import machine # นำเข้าโมดูล machine สำหรับควบคุมฮาร์ดแวร์
>>> led = machine.Pin(2, machine.Pin.OUT) # กำหนดขา GPIO2 เป็น Output
>>> led.on() # สั่งให้ LED ติด (จ่าย Logic Low, บางบอร์ด LED ติดเมื่อ Logic High)
>>> led.value(1) # หรือใช้ value(1) สำหรับติด (ถ้า on() ไม่ติด ให้ลอง value(0))
```

สังเกตว่า LED บนบอร์ดควรติดขึ้นมา

```
>>> led.off() # สั่งให้ LED ดับ (จ่าย Logic High, บางบอร์ด LED ดับเมื่อ Logic Low)
>>> led.value(0) # หรือใช้ value(0) สำหรับดับ (ถ้า off() ไม่ดับ ให้ลอง value(1))
```

LED บนบอร์ดควรจะดับลง

หมายเหตุ: ลักษณะการทำงานของ led.on(), led.off(), led.value(1), led.value(0) อาจแตกต่างกันไปในแต่ละบอร์ด บางบอร์ด LED จะติดเมื่อได้รับสัญญาณ LOW (Active Low) บางบอร์ดจะติดเมื่อได้รับ HIGH (Active High) หากคำสั่งหนึ่งไม่ทำงานตามคาด ให้ลองใช้คำสั่งตรงข้ามดู



2.6.2 การเขียนและบันทึกไฟล์สคริปต์

หมายเหตุสำคัญ: ควรทำความเข้าใจก่อนว่า คำสั่งใน REPL เป็นเพียงการควบคุมชั่วคราวเท่านั้น เหมาะสำหรับการทดลองสั้นๆ แต่ถ้าต้องการให้โปรแกรมทำงานซ้ำๆ หรือทำงานอัตโนมัติเมื่อเปิดเครื่อง ควรบันทึกโค้ดเป็นไฟล์

ตัวอย่างเช่น เมื่อเราพิมพ์คำสั่งในหน้าต่าง REPL ของ Thonny เหมือนที่เราทำกันไปเมื่อครู่

```
import machine
led = machine.Pin(2, machine.Pin.OUT)
led.on()
```

บอร์ดจะตอบสนองทันที เช่น ไฟ LED ติด แต่การทำงานเหล่านี้เป็นเพียงการทำงาน ชั่วคราว และเกิดขึ้นใน หน่วยความจำแรม (RAM) เท่านั้น เมื่อรีสตาร์ทบอร์ด, ปิดไฟ, หรือถอดสาย USB – คำสั่งที่พิมพ์ไว้จะหายไปทันที

ถ้าอยากให้บอร์ดจำคำสั่งและทำงานอัตโนมัติทุกครั้งที่เปิดเครื่อง

ให้เขียนคำสั่งเหล่านั้นลงในไฟล์ชื่อว่า main.py

แล้วอัปโหลดไฟล์นี้ไปยังบอร์ด ESP32 ผ่านโปรแกรม Thonny หรือเครื่องมืออื่น ๆ เช่น mpremote

1. ใน Thonny IDE คลิกที่ File > New เพื่อเปิดหน้าต่าง Editor ใหม่สำหรับเขียนโค้ด
2. พิมพ์โค้ดต่อไปนี้ใน Editor (โค้ดไฟกะพริบ):

main.py *

```
import machine
import time

# กำหนดขา LED (ตรวจสอบหมายเลขขาของบอร์ดที่ใช้)
led_pin = 2
led = machine.Pin(led_pin, machine.Pin.OUT)

print("โปรแกรมไฟกะพริบเริ่มต้นแล้ว!")

while True:
    led.on()  # LED ติด
    time.sleep_ms(500)  # หน่วงเวลา 500 มิลลิวินาที (0.5 วินาที)
    led.off()  # LED ดับ
    time.sleep_ms(500)  # หน่วงเวลา 500 มิลลิวินาที
```

3. คลิกที่ File > Save as...
4. Thonny จะถามว่าจะบันทึกไฟล์ไว้ที่ไหน ให้เลือก MicroPython device
5. ตั้งชื่อไฟล์เป็น main.py แล้วคลิก OK
    - **ความสำคัญของ boot.py และ main.py:**
        - boot.py: เป็นไฟล์สคริปต์ที่จะทำงานเป็นไฟล์แรกสุดเมื่อ ESP32 เปิดเครื่อง (หลังจากรีเซ็ต) มักใช้สำหรับตั้งค่าระบบเบื้องต้น (เช่น การเชื่อมต่อ Wi-Fi)
        - main.py: เป็นไฟล์สคริปต์หลักของโปรแกรม จะทำงานหลังจาก boot.py ทำงานเสร็จ (หากมี boot.py)
6. หลังจากบันทึกไฟล์ main.py ลงใน ESP32 แล้ว ให้ลองกดปุ่ม STOP/Restart backend ใน Thonny หรือกดปุ่ม EN/RESET บนบอร์ด ESP32 โปรแกรมไฟกะพริบควรจะเริ่มทำงานโดยอัตโนมัติ และ LED ออนบอร์ดจะกะพริบ สามารถดูข้อความ print ได้ในหน้าต่าง Shell ของ Thonny
7. หากต้องการหยุดโปรแกรมที่กำลังรันอยู่ใน Loop (เช่น while True) ให้กด Ctrl+C ในหน้าต่าง Shell ของ Thonny

2.7 REPL: เครื่องมือทดลองโค้ดอันทรงพลัง

ดังที่ได้ทดลองไปแล้ว REPL (Read-Evaluate-Print Loop) เป็นเครื่องมือที่มีประโยชน์อย่างยิ่งในการพัฒนา MicroPython:

- **ทดสอบแนวคิดอย่างรวดเร็ว:** สามารถทดลองคำสั่งหรือฟังก์ชันเล็กๆ ได้ทันทีโดยไม่ต้องเขียนเป็นไฟล์เต็ม
- **ตรวจสอบสถานะฮาร์ดแวร์:** ใช้ REPL เพื่ออ่านค่าจากเซ็นเซอร์ หรือตรวจสอบสถานะของขา GPIO ได้โดยตรง
- **ดีบักโปรแกรม:** เมื่อโปรแกรมที่รันจากไฟล์เกิดปัญหา สามารถใช้ REPL เพื่อตรวจสอบค่าของตัวแปร หรือทดลองเรียกใช้ฟังก์ชันต่างๆ เพื่อหาสาเหตุ
- **เรียนรู้โมดูลและคำสั่งใหม่ๆ:** สามารถ import โมดูลแล้วใช้คำสั่ง help(module_name) หรือ dir(module_name) เพื่อดูว่ามีฟังก์ชันอะไรให้ใช้งานบ้าง

การฝึกใช้งาน REPL ให้คล่องแคล่วจะช่วยให้กระบวนการเรียนรู้และพัฒนาโครงงาน MicroPython เป็นไปอย่างราบรื่นและมีประสิทธิภาพมากขึ้น

2.8 บทสรุปและสิ่งที่รออยู่ในบทถัดไป

ในบทนี้ ผู้อ่านได้เรียนรู้ขั้นตอนที่สำคัญทั้งหมดในการเตรียมความพร้อมสำหรับการพัฒนา MicroPython บน ESP32 ตั้งแต่การเลือกบอร์ด, การติดตั้งไดรเวอร์, การติดตั้งเฟิร์มแวร์ MicroPython ด้วย esptool.py, ไปจนถึงการตั้งค่า Thonny IDE และการทดลองเขียนโปรแกรมแรกเพื่อควบคุม LED ออนบอร์ด ทั้งผ่าน REPL และการบันทึกเป็นไฟล์ main.py

การที่สามารถทำให้ LED ดวงเล็กๆ บนบอร์ดกะพริบได้ด้วยโค้ด Python ที่เขียนขึ้นเองนั้น ถือเป็นก้าวแรกที่ยิ่งใหญ่ และเป็นพื้นฐานสำคัญสู่การสร้างสรรค์โครงงานที่ซับซ้อนยิ่งขึ้น

💡 **ทำไมต้องใช้ชื่อ main.py กับ MicroPython?**

เมื่อใช้งาน MicroPython บนบอร์ด ESP32 หรือ ESP8266 เรามักจะต้องการให้บอร์ด "เริ่มทำงานอัตโนมัติ" ทันทีหลังจากได้รับพลังงาน ไม่ว่าจะเป็นการจ่ายไฟผ่านพอร์ต USB, การเสียบเข้ากับแบตเตอรี่, หรือการใช้แหล่งจ่ายไฟภายนอกอื่น ๆ

กล่าวคือ **หลังจากจ่ายไฟเลี้ยงให้กับบอร์ด** MicroPython จะเริ่มกระบวนการทำงานตามลำดับดังนี้:

1. **รัน boot.py (ถ้ามี):** มักใช้สำหรับตั้งค่าระบบ เช่น Wi-Fi, UART
2. **รัน main.py (ถ้ามี):** เป็นจุดเริ่มต้นของโปรแกรมที่เขียนโดยผู้ใช้งาน

ดังนั้น หากต้องการให้บอร์ด **เริ่มทำงานอัตโนมัติทันทีหลังจากได้รับไฟเลี้ยง** ไม่ว่าจะต่อผ่าน USB หรือจ่ายไฟจากภายนอก — จะต้องมีไฟล์ main.py อยู่ในระบบไฟล์ของบอร์ด

🔸 **คำว่า main.py จึงเปรียบเสมือนกับ int main() ในภาษา C:**

เป็นตำแหน่งที่ระบบจะเริ่ม "รัน" โปรแกรมของเราโดยอัตโนมัติ และเป็นมาตรฐานของ MicroPython ที่ฝังอยู่ในเฟิร์มแวร์อยู่แล้ว หากไม่มีไฟล์ main.py อยู่เลย บอร์ดจะเข้าสู่โหมด REPL (พร้อมพิมพ์คำสั่งแบบสด) แต่จะไม่ทำงานใด ๆ อัตโนมัติ — ซึ่งอาจไม่เหมาะกับโปรเจกต์ที่ต้องติดตั้งจริง เช่น วัดอุณหภูมิ, สั่งเปิดปั๊มน้ำ, หรือควบคุมอุปกรณ์ภาคสนาม

⚠️ **แล้วถ้ามี "หลายโปรเจกต์"... จะเกิดอะไรขึ้น?**

ลองจินตนาการว่า:

- เรามีโปรเจกต์ควบคุมไฟ LED อัตโนมัติ
- มีอีกโปรเจกต์วัดอุณหภูมิ DHT22
- และโปรเจกต์แสดงผลบน OLED

แน่นอนว่า **แต่ละโปรเจกต์ก็ต้องมี main.py ของตัวเอง** แล้วถ้าเราเก็บโปรเจกต์ทั้งหมดนี้ไว้ในคอมพิวเตอร์ และทุกโปรเจกต์ดันใช้ชื่อไฟล์ว่า main.py เหมือนกันหมด... สิ่งที่จะเกิดขึ้น คือ: เราจะไม่รู้ว่าไฟล์ main.py ไฟล์ไหนเป็นของโปรเจกต์ไหน และโดยปกติทั่วไปแล้วคอมพิวเตอร์จะไม่อนุญาตให้เราตั้งชื่อไฟล์ซ้ำกันไว้ในที่เดียวกันอยู่แล้ว ดังนั้นการบันทึกไฟล์ควรสร้างโฟล์เดอร์ขึ้นมาแยกเก็บแต่ละโปนเจกต์ให้ชัดเจน เพราะทุกครั้งที่อัปโหลด main.py ลงบอร์ด ถ้าเราเผลอเลือกผิดไฟล์ แฟลชผิดโปรเจกต์ หรือเขียนทับโดยไม่ได้ตั้งใจ — โค้ดเดิมจะหายไปทันที และเมื่อรีสตาร์ทบอร์ดอีกครั้ง มันก็จะรันโค้ดผิดโปรเจกต์แบบไม่รู้ตัว

วิธีจัดการหลายโปรเจกต์อย่างมืออาชีพ

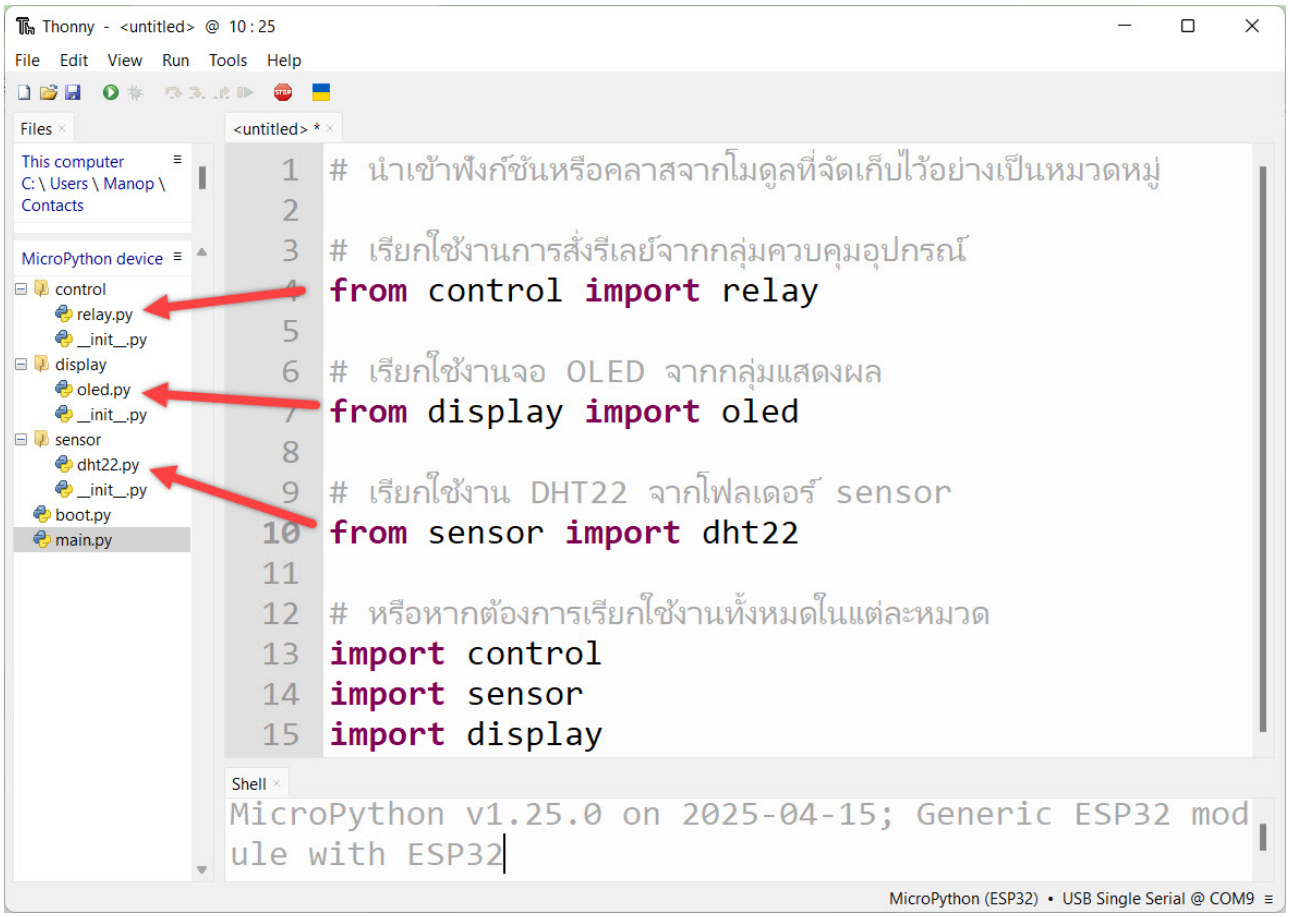
เพื่อหลีกเลี่ยงความโกลาหลของ main.py ที่ซ้ำซ้อน อาจเลือกวิธีใดวิธีหนึ่งต่อไปนี้:

1. **จัดเก็บแยกตามโฟลเดอร์บนคอมพิวเตอร์**

คือการสร้างโฟลเดอร์และตั้งชื่อไว้ให้ชัดเจนแยกเอาไว้ เมื่อจะใช้งานโปรเจกต์ใด ก็ค่อยอัปโหลด main.py จากโฟลเดอร์นั้นไปยังบอร์ด

---

2. **รวมโมดูลการทำงานแต่ละส่วนไว้ในบอร์ด แล้วใช้ main.py เป็นตัวเลือก**



---

3. ใช้ config.txt เลือกโปรเจกต์แบบยืดหยุ่น

วิธีนี้เหมาะสำหรับนักพัฒนาที่ต้องการสลับโปรเจกต์หลายชุดบนบอร์ดเดียวโดยไม่ต้องแฟลชใหม่ทุกครั้ง โดยจะใช้ไฟล์ config.txt เป็นตัวกำหนดชื่อโปรเจกต์ที่ต้องการให้รัน เช่น: เขียนในไฟล์ config.txt ว่า sensor.dht22

แล้วที่ไฟล์ main.py ให้เพิ่มโค้ดเรียกใช้เข้าไป

```
with open("config.txt") as f:
module = f.read().strip()
__import__(module)
```

แต่ผู้เขียน **ไม่แนะนำสำหรับผู้เริ่มต้น** เนื่องจากต้องพิมพ์ชื่อโมดูลให้ถูกต้อง 100% ตามโครงสร้างโฟลเดอร์ (เช่น sensor.dht22, display.oled, control.relay) หากพิมพ์ผิดเพียงตัวเดียว บอร์ดจะไม่สามารถรันโปรเจกต์ได้ และไม่มีข้อความแจ้งเตือน

ดังนั้น วิธีนี้จึงเหมาะกับนักพัฒนาระดับสูงหรือผู้ที่ต้องการระบบที่ปรับแต่งอัตโนมัติโดยเขียนโค้ดควบคุมเพิ่มเติมเอง

สรุปอีกครั้ง

- บอร์ด MicroPython จะรัน main.py ทุกครั้งหลังจากจ่ายไฟเลี้ยง
- ถ้าเรามีหลายโปรเจกต์ ควรแยกโฟลเดอร์ไว้ หรือเขียน main.py ให้เลือกงานเอง
- หลีกเลี่ยงการใช้ชื่อ main.py ซ้ำกันโดยไม่จัดระเบียบ

เพราะการมีหลาย main.py โดยไม่วางแผน จะทำให้เกิดความสับสนหากมีหลายโปรเจกต์

ถึงตอนนี้... เรากำลังทำอะไรอยู่?

ในบทที่ผ่านมา เราได้เริ่มต้นทดลองเขียนคำสั่งควบคุมบอร์ด ESP32 ผ่านภาษา MicroPython ไม่ว่าจะเป็นการเปิด-ปิดไฟ LED, การพิมพ์ข้อความ, หรือการทดลองสั่งงานผ่าน REPL ซึ่งทั้งหมดนี้ถือเป็นจุดเริ่มต้นที่ช่วยเปิดประตูสู่โลกของการเขียนโปรแกรมฝังตัว (Embedded Programming) อย่างเรียบง่ายและน่าตื่นเต้น เพราะทุกคำสั่งที่เราพิมพ์ลงไป บอร์ดก็ตอบสนองกลับมาทันที ไม่ว่าจะเป็นแสงไฟที่กระพริบขึ้น การแสดงผลข้อความ หรือค่าที่ส่งกลับจากเซ็นเซอร์ต่าง ๆ ความสำเร็จในระดับพื้นฐานนี้สร้างแรงจูงใจให้ผู้เรียนรู้สึกว่า “เขียนโปรแกรมได้แล้ว” อย่างเป็นรูปธรรม

แต่สิ่งสำคัญที่อยากให้ทุกคนตระหนักไว้ตั้งแต่เนิ่น ๆ ก็คือ ✨ สิ่งที่เราทำมาจนถึงตอนนี้ ยังเป็นเพียงแค่ “การทดลองใช้งานเบื้องต้น” เท่านั้น เพราะเราแค่พิมพ์คำสั่งแล้วดูผลลัพธ์ทันที ซึ่งก็คือการสั่งให้บอร์ดทำงานแบบสำเร็จรูปด้วยคำสั่งง่าย ๆ โดยยังไม่ได้สัมผัสกับ "โครงสร้างของภาษา Python" ที่อยู่เบื้องหลังเลย — ทั้งการนิยามตัวแปร, การกำหนดเงื่อนไข, การใช้ลูป, หรือแม้แต่ฟังก์ชันที่ทำให้โค้ดสามารถจัดระเบียบและนำกลับมาใช้ซ้ำได้อย่างเป็นระบบ

ลองจินตนาการว่าเราเดินทางไปยังประเทศที่ใช้ภาษาอังกฤษเป็นหลัก แล้วเริ่มสื่อสารได้จากประโยคพื้นฐานอย่าง “Hello”, “How are you?”, “Thank you.” ฟังดูเหมือนเราพูดภาษาอังกฤษได้แล้วใช่ไหม? แต่ในความเป็นจริง หากเรายังไม่เข้าใจไวยากรณ์, คำกริยา, การเรียงลำดับคำ หรือกฎของภาษา เราก็ยังไม่สามารถสื่อสารในสถานการณ์ที่ซับซ้อนได้อย่างมั่นใจ เช่นเดียวกันกับการเขียน MicroPython — หากเราไม่เข้าใจพื้นฐานของภาษา Python อย่างแท้จริง การพัฒนาโค้ดที่ซับซ้อน หรือการควบคุมฮาร์ดแวร์ที่มีหลายองค์ประกอบพร้อมกัน ก็จะกลายเป็นเรื่องที่ยุ่งยากและสับสนในที่สุด

💡 **แล้ว “พื้นฐานของภาษา Python” คืออะไร?** มันคือเครื่องมือชุดสำคัญที่จะทำให้เราเปลี่ยนจากผู้ใช้คำสั่งเป็นครั้งคราว มาเป็นนักพัฒนาที่ควบคุมตรรกะของโปรแกรมได้อย่างเต็มรูปแบบ ไม่ว่าจะเป็นตัวแปร (Variables), ชนิดข้อมูล (Data Types), ตัวดำเนินการ (Operators), นิพจน์ (Expressions), คำสั่งควบคุมการทำงาน (Control Statements) เช่น if, while, for, ฟังก์ชัน (Functions), โมดูล (Modules), การจัดการข้อผิดพลาด (Error Handling) ตลอดจนแนวคิดเชิงวัตถุ (OOP) ที่เป็นรากฐานของการพัฒนาระบบในระดับมืออาชีพ

🧠 **ทำไมเราจึงต้องเข้าใจสิ่งเหล่านี้?** เพราะการเขียนโปรแกรมควบคุมอุปกรณ์จริง ๆ ไม่ได้จบแค่การทำให้ไฟติดหรือดับ แต่คือการสั่งงานแบบมีเงื่อนไข ควบคุมตามเวลา จัดการหลายอุปกรณ์พร้อมกัน รอรับค่าจากเซ็นเซอร์หลายตัว หรือแม้แต่ทำงานร่วมกับเครือข่ายอินเทอร์เน็ต หากเราไม่มีความเข้าใจในภาษา Python การพัฒนาโปรเจกต์แบบนี้จะกลายเป็นการลองผิดลองถูกที่เต็มไปด้วยความเครียด แต่หากเรารู้จักภาษาอย่างแท้จริง แม้โค้ดจะยากขึ้น เราก็จะสามารถเขียน วิเคราะห์ และแก้ไขได้อย่างเป็นระบบด้วยตนเอง

📘 สรุปก่อนเข้าสู่บทถัดไป สิ่งที่เราได้เรียนรู้คือการใช้งาน MicroPython อย่างง่าย ซึ่งเป็นจุดเริ่มต้นที่ยอดเยี่ยม แต่ยังไม่เพียงพอหากเราต้องการสร้างระบบจริงที่มีตรรกะซับซ้อน หรือสามารถนำไปประยุกต์ใช้ในงานต่าง ๆ ได้อย่างมั่นใจ ต่อจากนี้ เราจะเริ่มต้นเรียนรู้ "ภาษา Python" อย่างจริงจัง เพื่อให้เราสามารถพัฒนาโปรแกรมของตนเองได้อย่างมั่นคง เป็นระบบ และพร้อมต่อยอดสู่โปรเจกต์ที่ใช้งานได้จริงในอนาคต

ในบทถัดไป (บทที่ 3) เราจะทบทวนพื้นฐานภาษา Python ที่จำเป็นและมักใช้งานบ่อยในการเขียนโปรแกรมสำหรับไมโครคอนโทรลเลอร์ เพื่อให้ผู้อ่านมีความเข้าใจในโครงสร้างภาษาและคำสั่งต่างๆ เพียงพอที่จะนำไปประยุกต์ใช้ในการควบคุมฮาร์ดแวร์ได้อย่างมั่นใจ เตรียมพบกับความสนุกในการเรียนรู้ภาษา Python ในบริบทของ MicroPython กันได้เลย!

ส่วนที่ 2: พื้นฐาน MicroPython สำหรับควบคุมฮาร์ดแวร์

บทที่ 3: กำเนิดและโครงสร้างของภาษา Python (สำหรับ MicroPython)

ก่อนที่เราจะเริ่มเรียนรู้การเขียนโปรแกรมเพื่อควบคุมฮาร์ดแวร์ด้วย MicroPython อย่างเป็นระบบ เราควรทำความเข้าใจรากฐานของภาษา Python เสียก่อน เพราะ MicroPython แม้จะเป็นเวอร์ชันย่อของ Python แต่ก็ยังคงใช้โครงสร้างหลักเดียวกันเกือบทั้งหมด หากเข้าใจพื้นฐานของ Python อย่างถ่องแท้แล้ว การต่อยอดไปสู่การควบคุมเซ็นเซอร์ การสื่อสารข้อมูล หรือการจัดการเงื่อนไขซับซ้อนต่าง ๆ ก็จะง่ายขึ้นอย่างมาก

3.1 กำเนิดของภาษา Python

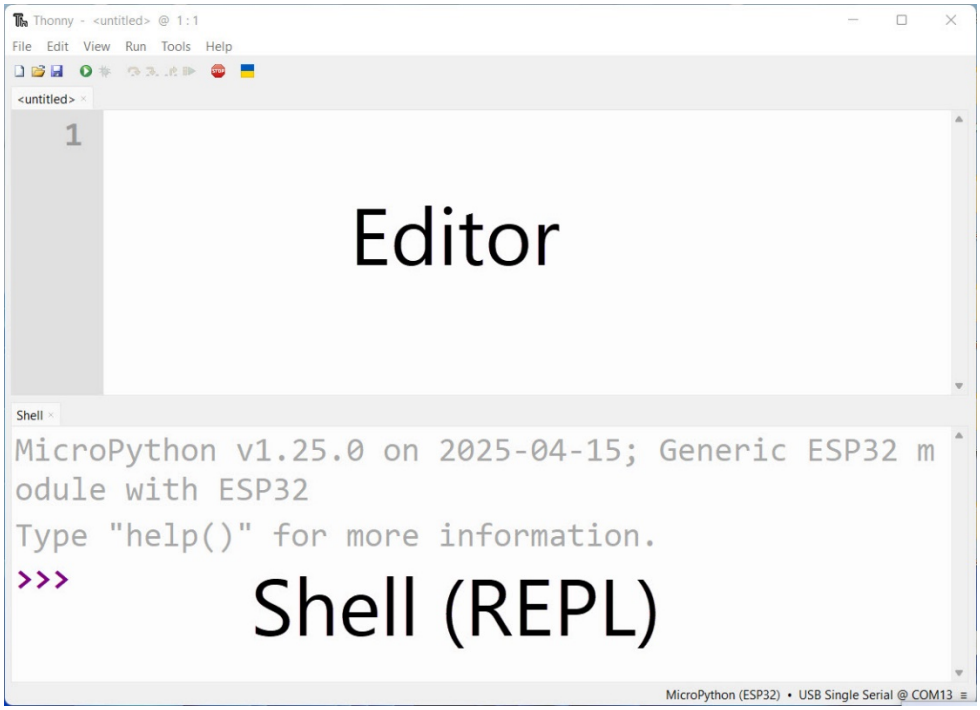
ภาษา Python ถูกออกแบบโดย Guido van Rossum ในช่วงปลายทศวรรษ 1980 และเปิดตัวอย่างเป็นทางการในปี 1991 โดยมีเป้าหมายเพื่อให้เป็นภาษาที่อ่านง่าย เข้าใจง่าย และมีไวยากรณ์ที่ชัดเจน ไม่ซับซ้อนเหมือนภาษา C หรือ Java ซึ่งเป็นที่นิยมในยุคนั้น Python ได้รับแรงบันดาลใจจากภาษาการเขียนโปรแกรม ABC (ที่พัฒนาโดย CWI – ศูนย์วิจัยของเนเธอร์แลนด์) และมีเป้าหมายที่จะใช้งานได้ทั้งเพื่อการเรียนการสอนและพัฒนาแอปพลิเคชันจริงจัง ปัจจุบัน Python ถูกนำไปใช้ทั้งในงานวิทยาศาสตร์ข้อมูล, ปัญญาประดิษฐ์, ระบบเว็บ, งานอัตโนมัติ, รวมถึงงาน Embedded ที่เราใช้กับไมโครคอนโทรลเลอร์เช่น ESP32 ด้วย

3.2 โครงสร้างของภาษา Python

Python เป็นภาษาระดับสูง (High-level Language) ที่ถูกออกแบบมาให้มีลักษณะ:

- **อ่านง่าย**: ใช้การจัดย่อหน้า (Indentation) เพื่อกำหนดขอบเขตของโค้ดแทนการใช้วงเล็บปีกกา
- **ไดนามิก**: ไม่จำเป็นต้องกำหนดชนิดของตัวแปรล่วงหน้า (Dynamic Typing)
- **กระชับ**: เขียนโค้ดให้น้อยที่สุด แต่สามารถสื่อความหมายได้ชัดเจน
- **เป็นเชิงวัตถุ (OOP)**: สนับสนุนการเขียนแบบวัตถุเป็นพื้นฐาน พร้อมด้วยแนวคิดคลาส (Class) และอ็อบเจกต์ (Object)

ก่อนจะทำความเข้าใจเรื่องของโครงสร้างภาษา เราจะมาดูเครื่องมือกันก่อน เพราะเราจะต้อง ทำตามตัวอย่างในหนังสือเพื่อเพิ่มความเข้าใจจริง ใน Thonny มีอยู่ 2 ส่วนคือ ช่อง Editor ข้างบน กับ ช่อง Shell (REPL) ด้านล่าง



3.2.1 ความแตกต่างระหว่าง Shell (REPL) และ Editor ใน Thonny

เมื่อเริ่มต้นใช้งาน Thonny IDE เพื่อเขียนโปรแกรมด้วย MicroPython บนบอร์ดไมโครคอนโทรลเลอร์ เช่น ESP32 ผู้เรียนจะพบว่าหน้าต่างโปรแกรมมีสองส่วนหลัก ได้แก่ “Shell” (หรือ REPL) และ “Editor” ซึ่งทั้งสองส่วนนี้มีวัตถุประสงค์และลักษณะการใช้งานที่แตกต่างกันอย่างชัดเจน **การใช้งานเมนูต่างๆในโปรแกรมจะอยู่ท้ายเล่ม**

Shell (REPL):

REPL ย่อมาจาก Read-Eval-Print Loop ซึ่งหมายถึงการรับคำสั่งจากผู้ใช้ ประมวลผล แสดงผล และรอรับคำสั่งใหม่วนไปเรื่อย ๆ ส่วนนี้อยู่ที่ด้านล่างของหน้าต่าง Thonny โดยผู้ใช้สามารถพิมพ์คำสั่ง Python ลงไปโดยตรง เช่น print("Hello"), led.on() หรือ 2 + 3 แล้วกด Enter ระบบจะประมวลผลและแสดงผลทันทีในบรรทัดถัดไป

REPL เหมาะสำหรับการทดลองคำสั่งสั้น ๆ การทดสอบแนวคิดเบื้องต้น หรือการตรวจสอบค่าของตัวแปรในขณะทำงาน แต่ข้อจำกัดของ REPL คือ คำสั่งต่าง ๆ ที่พิมพ์ลงไปจะไม่ถูกบันทึกไว้เป็นโปรแกรม และจะหายไปเมื่อปิดบอร์ดหรือปิดโปรแกรม หรือให้เข้าใจง่ายๆ ก็คือโค้ดที่เขียนในช่องนี้จะไม่ได้ถูกบันทึกไว้ในบอร์ดเลย ไม่ใช่การอัปโหลดเหมือนใน Arduino

Editor:

ส่วนของ Editor อยู่ที่ด้านบนของ Thonny เป็นพื้นที่สำหรับเขียนโปรแกรมเต็มรูปแบบ ซึ่งประกอบด้วยคำสั่งหลายบรรทัด ฟังก์ชัน หรือโครงสร้างควบคุมแบบซับซ้อน โปรแกรมที่เขียนใน **Editor** จะต้องสั่งให้ทำงานด้วยการกดปุ่ม “**Run**” หรือกดปุ่มลัด **F5** เพื่อส่งโค้ดทั้งหมดไปยังบอร์ด ESP32 คล้ายกับการการกดอัปโหลดใน Arduino นั่นเอง

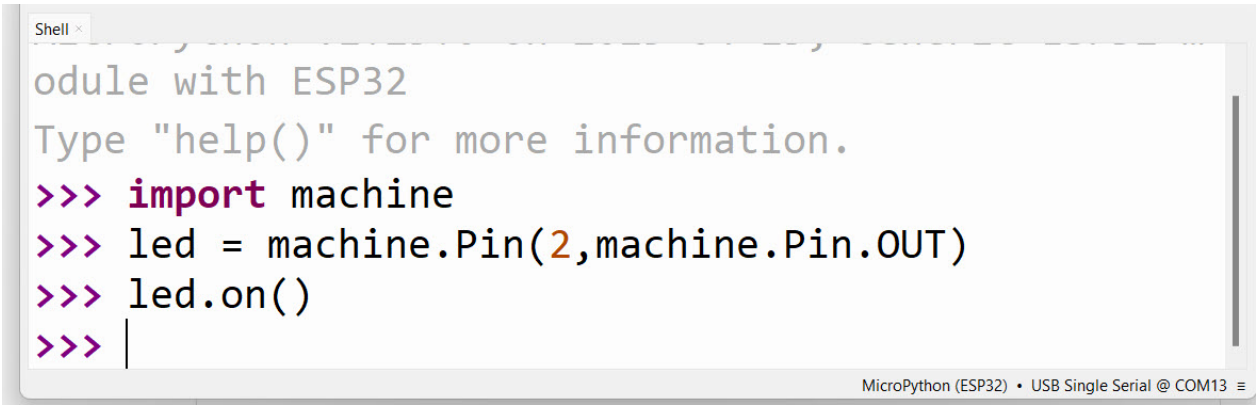
โค้ดใน Editor สามารถบันทึกไว้ในรูปแบบไฟล์ .py และนำกลับมาใช้งานใหม่ได้ในภายหลัง จึงเหมาะสำหรับการเขียนโปรเจกต์จริงที่มีความซับซ้อนหรือโค้ดที่ต้องใช้ซ้ำหลายครั้ง

เปรียบเทียบการใช้งาน:

- ถ้าเปรียบ REPL เป็น “การพิมพ์เขียนโค้ด เพื่อสื่อสารกับบอร์ดแบบสด ๆ”
  Editor ก็เหมือน “เขียนจดหมายยาว ๆ ให้บอร์ดทำงานตามลำดับ”
- REPL ใช้ได้เร็ว เห็นผลทันที
  Editor ต้องสั่งรันก่อนถึงจะเห็นผล
- REPL เหมาะสำหรับการทดลองเฉพาะหน้า
  Editor เหมาะสำหรับเขียนงานจริงที่ต้องการความต่อเนื่อง

ตัวอย่างการใช้งาน:

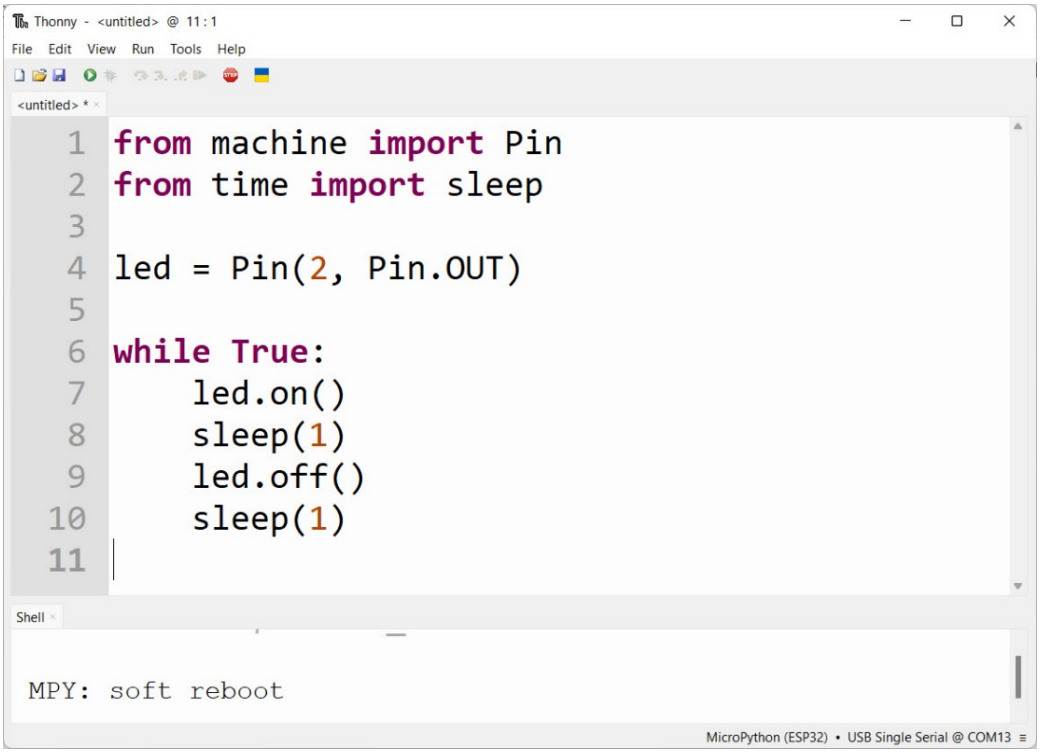
ทดลองสั่งให้ LED บนบอร์ดติดทันที: แต่หากปิดโปรแกรม led จะไม่ติดอีกต่อไป



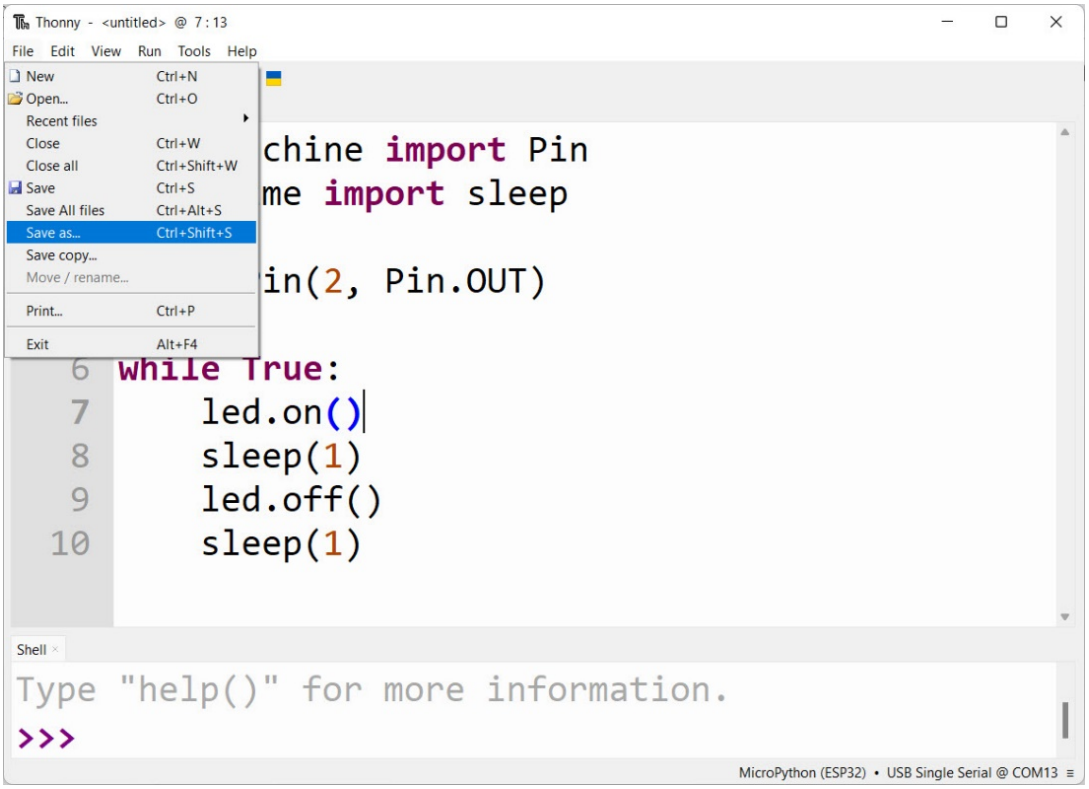
เขียนโค้ดกระพริบไฟแบบวนซ้ำ: จากนั้นต้อง **กดปุ่ม Run** เพื่อให้โค้ดเริ่มทำงาน แต่หากปิดโปรแกรมและถอดสายออกแล้วเอามาเสียบใหม่บอร์ดก็จะยังไม่ทำงานเหมือนกัน การจะให้บอร์ดทำงานได้เองจะต้องบันทึกไฟล์เอาไว้ในหน่วยความจำของบอร์ด

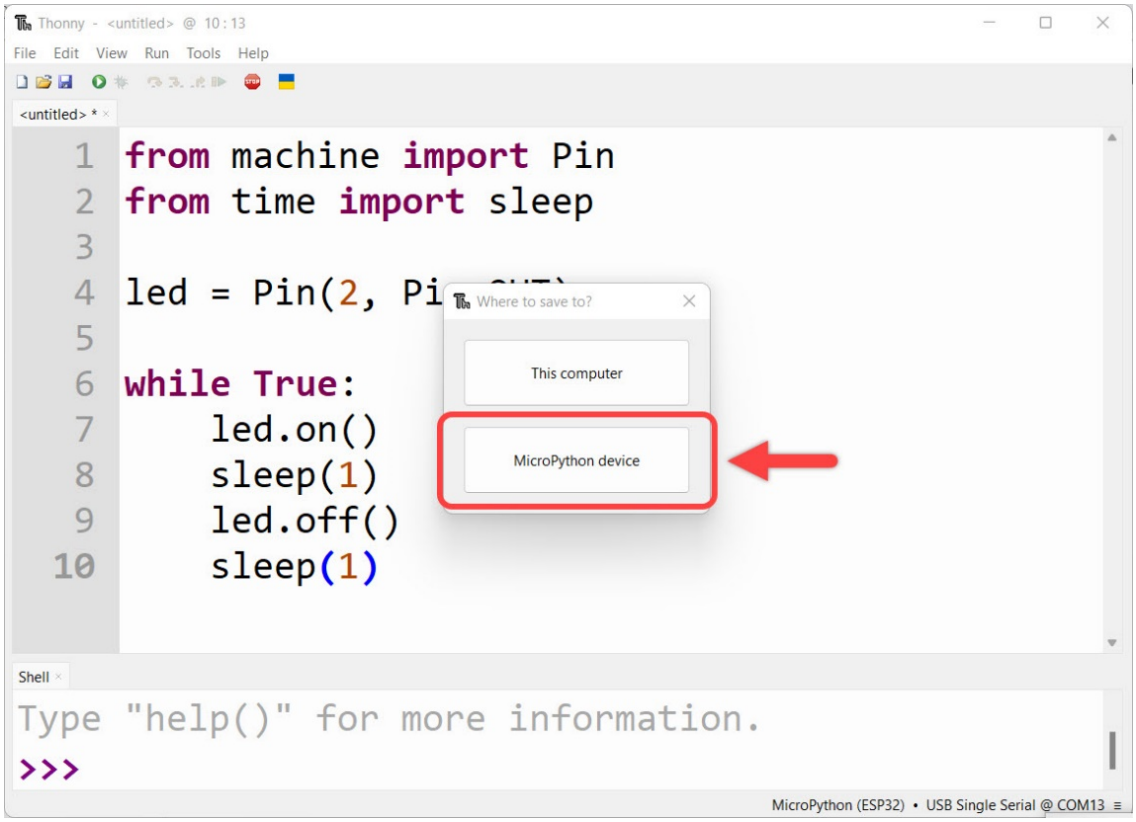
หมายเหตุ:

บอร์ด ESP32 บางรุ่น โดยเฉพาะรุ่น 38 ขา มักไม่มี LED ที่ต่อไว้กับขา GPIO ใด ๆ หากทดลองโปรแกรมแล้วไม่พบไฟ LED บนบอร์ดติดขึ้น นั่นหมายถึงบอร์ดของท่านไม่มี LED On-board เหมือนบางรุ่น ให้เตรียม LED และตัวต้านทานไว้ใช้งานเอง เมื่อถึงเนื้อหาส่วนการทดลองวงจรจริง ผู้อ่านจะสามารถนำอุปกรณ์เหล่านี้มาต่อกับขา GPIO ที่เหมาะสม เพื่อทดลองควบคุม LED ได้ตามขั้นตอนในบทเรียนถัดไป

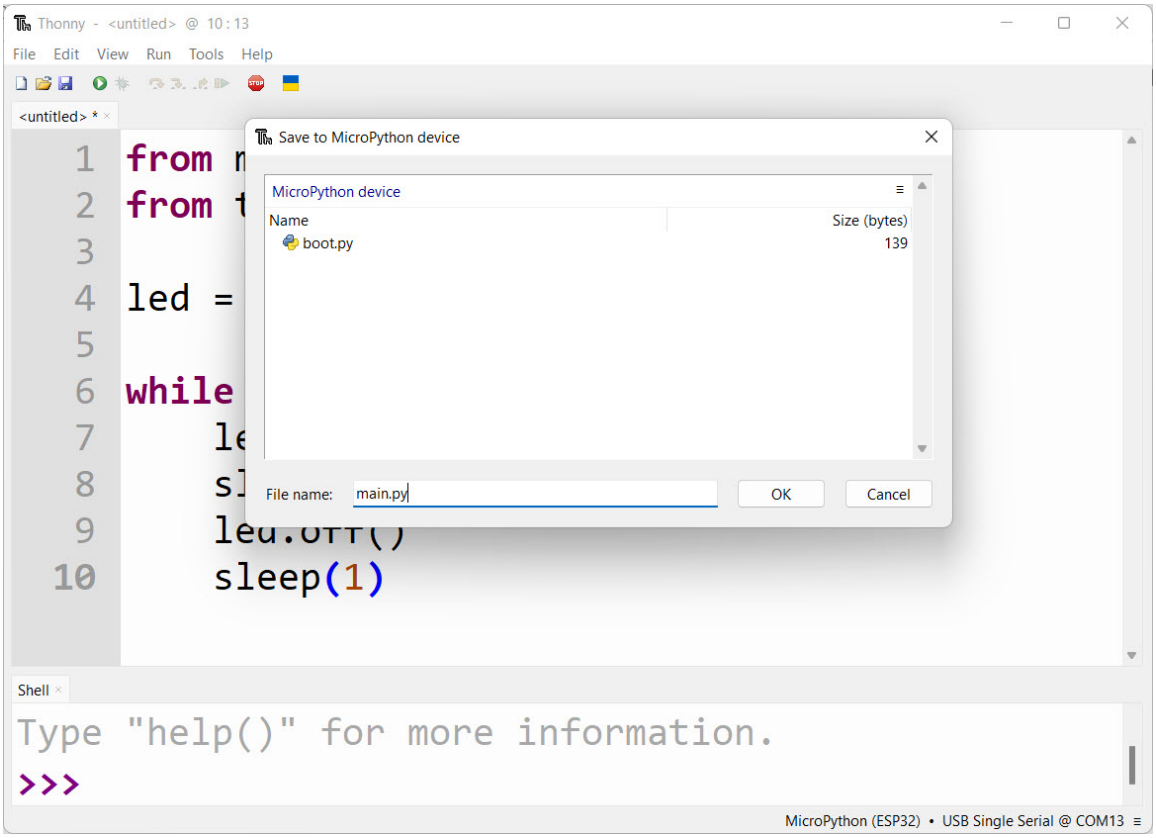


การบันทึกที่เหมือนกับการอัปโหลดโปรแกรมลงบอร์ดให้ไปที่เมนู File เลือก save หรือ save As จะมีหน้าต่างเล็ก ๆ เด้งขึ้นมาให้เราเลือกว่าจะบันทึกไว้ที่ไหน หากต้องการบันทึกไว้ในบอร์ดให้เลือก MicroPython device ดังรูป





และเมื่อกดแล้วจะมีหน้าต่างใหม่ขึ้นมาให้ตั้งชื่อไฟล์ว่า **main.py** แล้วกด OK



หากบันทึกแล้วไฟไม่กระพริบให้กดปุ่ม Reset หรือถอดสาย USB ออกแล้วเสียบใหม่

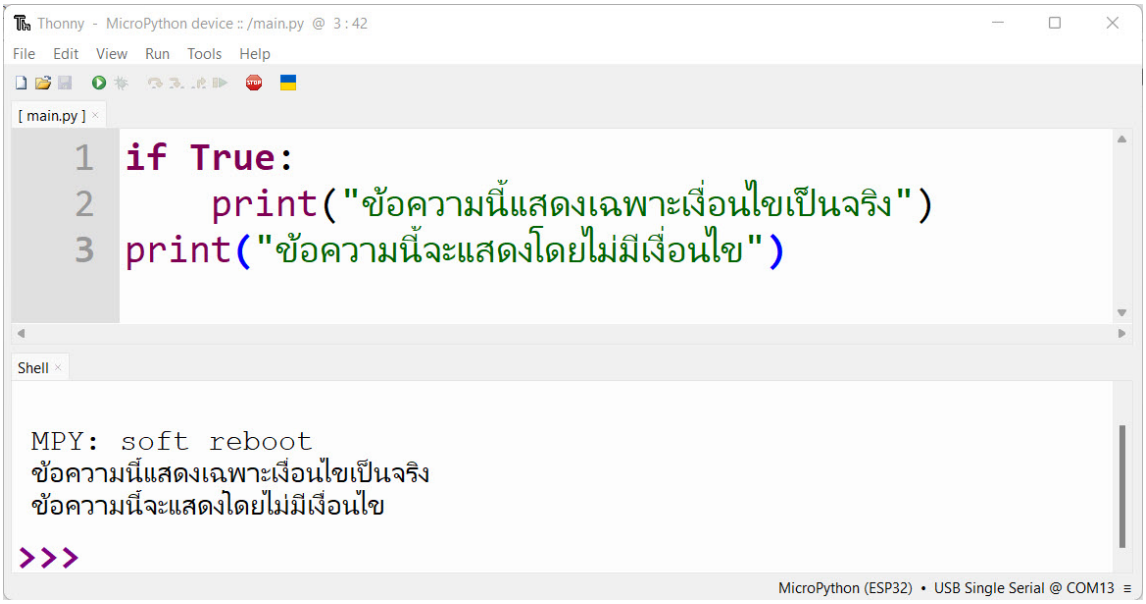
การเข้าใจความแตกต่างระหว่าง REPL และ Editor เป็นพื้นฐานสำคัญที่ผู้เริ่มต้นควรเรียนรู้ เพื่อสามารถเลือกใช้ให้เหมาะสมกับจุดประสงค์แต่ละช่วงในการพัฒนาโปรแกรมด้วย MicroPython บน ESP32 ได้อย่างมีประสิทธิภาพ

3.3 จุดเด่นของภาษา Python ที่ควรเข้าใจ

เพื่อให้เข้าใจและใช้ภาษา Python ได้อย่างมั่นใจในการเขียน MicroPython ควรเข้าใจจุดเด่นสำคัญของภาษา ซึ่งแตกต่างจากภาษาคอมพิวเตอร์อื่น ๆ โดยเฉพาะผู้ที่มีพื้นฐานจากภาษา C หรือ C++ จะพบว่ามีความแตกต่างหลายจุด เช่น:

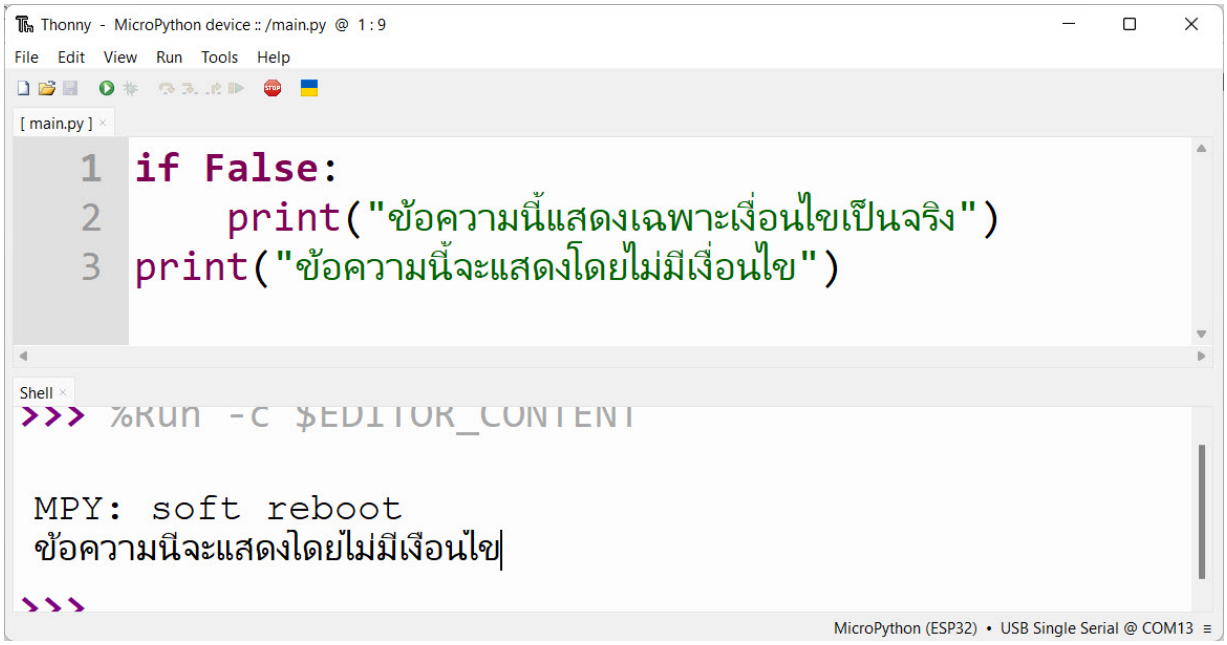
- **อ่านง่าย (Readable)**

Python ใช้ "การจัดย่อหน้า" หรือ **Indentation** เพื่อกำหนดขอบเขตของโค้ด แทนการใช้เครื่องหมายวงเล็บปีกกา {} เหมือนในภาษา C/C++

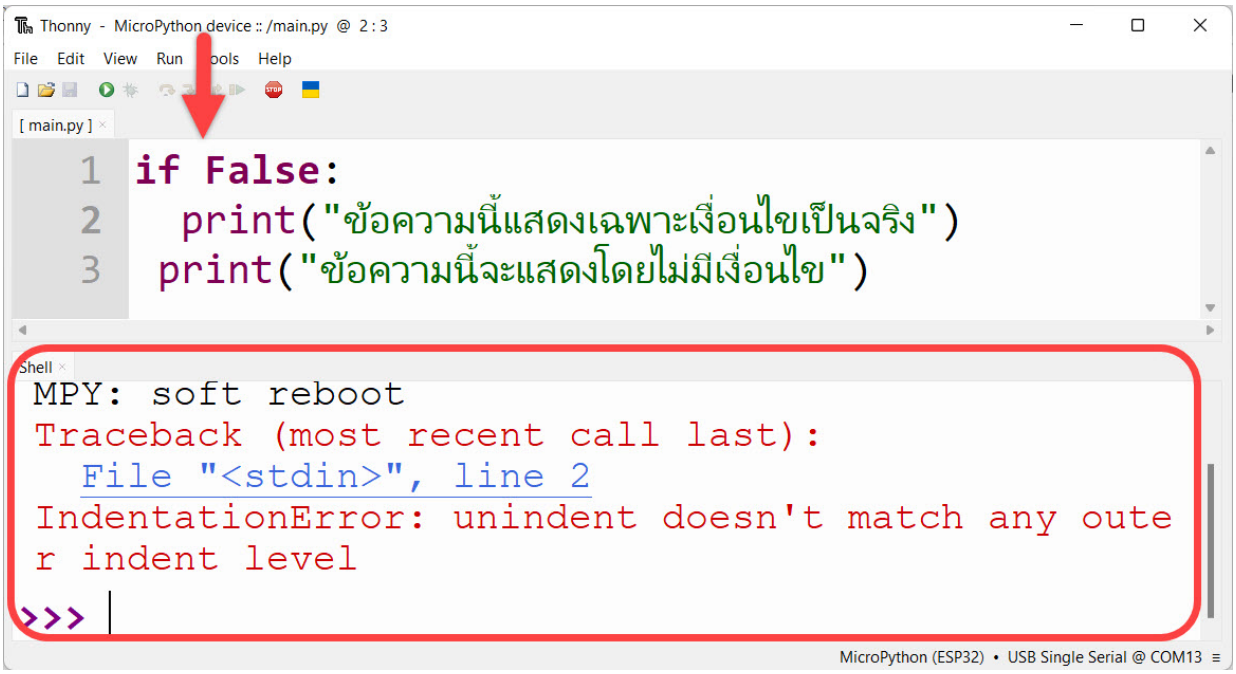


จากภาพจะเห็นว่าบรรทัดที่ 2 ถูกย่อหน้า หากเทียบกับภาษา C คืออยู่ในวงเล็บปีกกานั่นเอง

ในตัวอย่างนี้ print("ข้อความนี้แสดงเฉพาะเงื่อนไขเป็นจริง") จะถูกดำเนินการเมื่อเงื่อนไข **if True:** เป็นจริง เพราะมันอยู่ภายใต้การย่อหน้า **(Tab หรือ Space 4 ช่อง)** ส่วนบรรทัดที่ 3 ก็จะแสดงไม่ว่าเงื่อนไขจะเป็นจริงหรือเท็จ เพราะไม่มีย่อหน้า python จะถือว่าไม่ได้อยู่ในเงื่อนไข ถ้าเปรียบเทียบกับภาษา C ก็คืออยู่นอกวงเล็บปีกกาของ if นั่นเอง และเมื่อเราทดลองเปลี่ยนเงื่อนไขให้เป็นเท็จ หรือ False จะเห็นว่าข้อความบรรทัดที่ 2 จะไม่แสดงออกมา จะมีเพียงข้อความบรรทัดที่ 3 เท่านั้น



และถ้าเราทดลองเขียนแบบไม่มีการย่อหน้า หรือย่อหน้าไม่ถูกต้อง โปรแกรมจะไม่สามารถรันได้เลย จะแสดง error ทันที ถือเป็นจุดเด่นที่ช่วยให้โค้ดดูสะอาด อ่านง่าย และลดโอกาสเกิดข้อผิดพลาดจากการลืมปิดวงเล็บ



จะรู้ได้อย่างไรว่าต้องย่อหน้า (Indentation) กี่ที?

ทุกครั้งที่เขียนคำสั่งภายใต้ if, for, while, def, class, หรือ try — ต้องย่อหน้าเข้าไป 1 ระดับ และทุกคำสั่งที่อยู่ในกลุ่มเดียวกันต้องย่อหน้าให้อยู่ระดับเดียวกันทั้งหมด

- **ย่อหน้าแบบมาตรฐาน**: ใช้ Space 4 ครั้ง (หรือ 1 Tab หากตั้งค่าตัวแก้ไขให้ใช้แทนกันได้)
- **ห้ามผสม Tab และ Space ในโปรแกรมเดียวกันเด็ดขาด**

ตัวอย่างเช่น

เคล็ดลับ: ใช้โปรแกรม Thonny IDE ซึ่งช่วยย่อหน้าอัตโนมัติเมื่อพิมพ์คำสั่ง เช่น if, for, while แล้วกด Enter

โดยเมื่อพิมพ์เสร็จแล้วกดปุ่ม Enter โปรแกรม จะย่อหน้าให้อัตโนมัติอยู่แล้ว แต่ความผิดพลาดอาจเกิดขึ้นได้หากมีการเคาะเพิ่มเติมหรือแก้ไขแทรกโค้ดระหว่างบรรทัดด้วยตัวเอง

- **ไดนามิก (Dynamic Typing)**

การเขียนโปรแกรมภาษา Python ไม่จำเป็นต้องระบุชนิดของตัวแปรก่อนใช้งาน เช่น ไม่ต้องระบุว่าชนิดของตัว

แปรนั้นว่าจะใช้เก็บข้อมูลชนิดไหน เป็น int, float, หรือ string เหมือนภาษา C/C++ หรือ Java

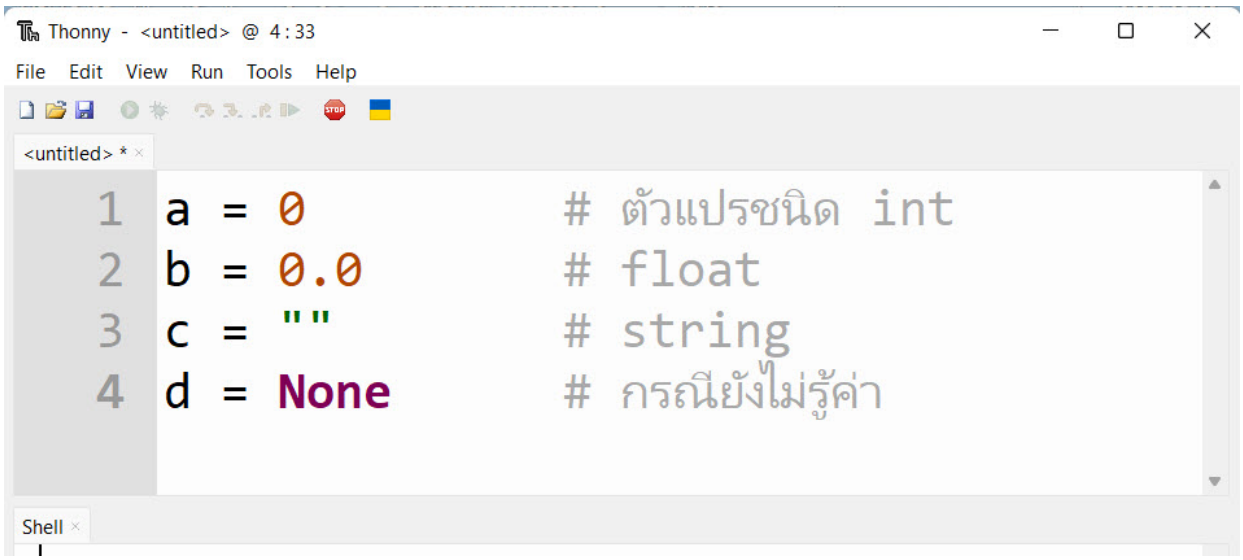
ในภาษา Python ผู้ใช้เพียงกำหนดค่าลงไป ตัวแปรจะได้รับชนิดตามค่าที่กำหนดให้โดยอัตโนมัติ

ข้อควรรู้: ใน Python ถ้ายังไม่กำหนดค่าให้ตัวแปร ก็ไม่สามารถประกาศได้ล่วงหน้าแบบเปล่า ๆ

การประกาศตัวแปรในภาษา Python ไม่สามารถประกาศตัวแปรขึ้นมาลอย ๆ ได้ ยกตัวอย่างในภาษา C หรือ JAVA หากประกาศว่า int x; โดยไม่กำหนดค่า x มันจะมีค่าเริ่มต้นเป็น 0 เอง แต่ในภาษา Python นั้นจะต้องกำหนดค่าขึ้นมาก่อนเสมอ

เช่น: **a เฉย ๆ แบบนี้ ❌ ผิด ไม่สามารถประกาศลอย ๆ แบบนี้ได้ ต้องกำหนดค่าก่อน เช่น a = 0**

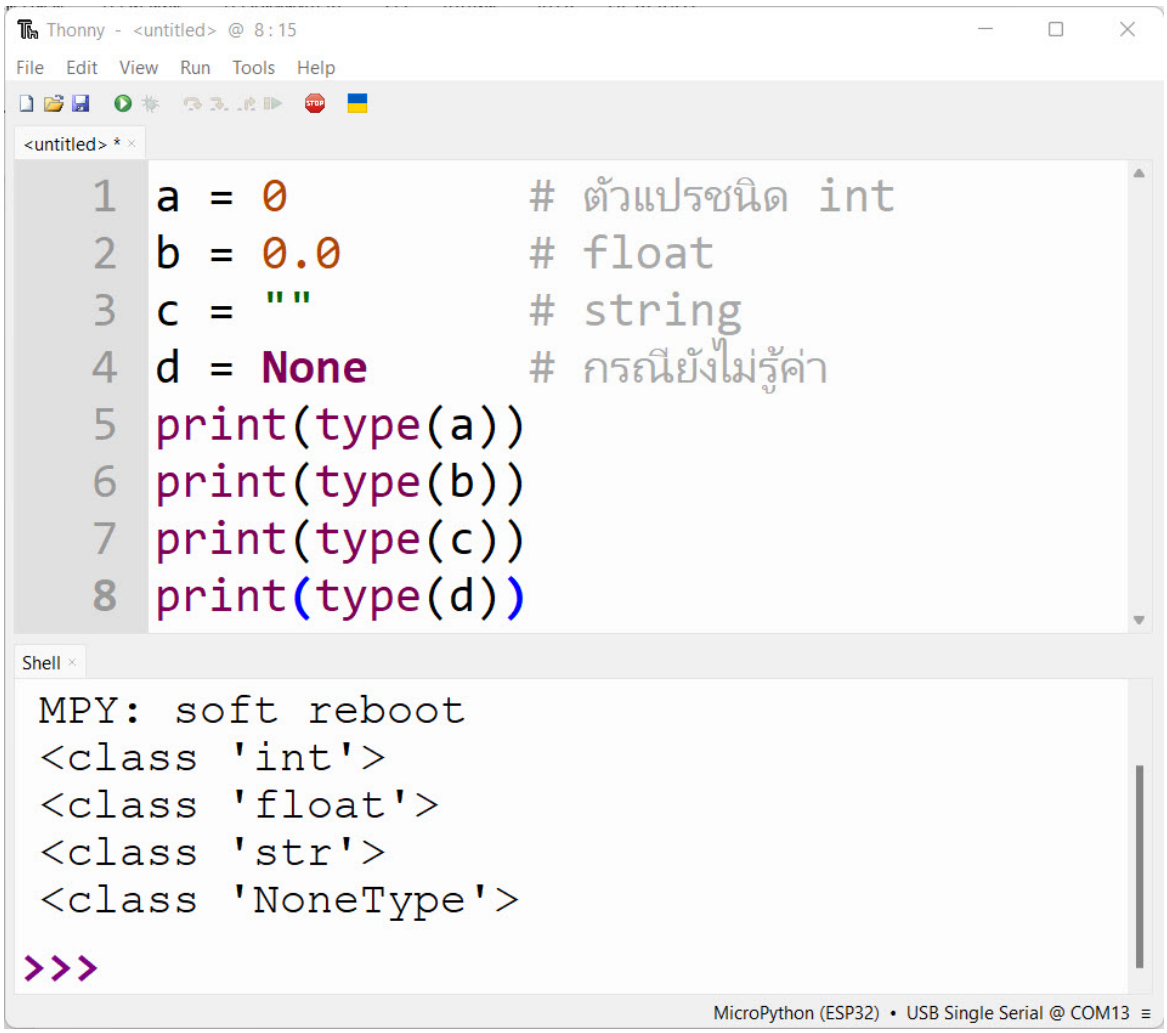
หากต้องการประกาศไว้ก่อนว่าตัวแปรนี้จะใช้ในภายหลัง กำหนดค่าเริ่มต้น ไว้ก่อน



การ คอมเมนต์ ในภาษา Python ใช้เครื่องหมาย #

ข้อความหลังเครื่องหมาย # เขียนอธิบายการทำงาน เอาไว้ให้คนอ่านเท่านั้น ไม่ใช่โค้ดโปรแกรม

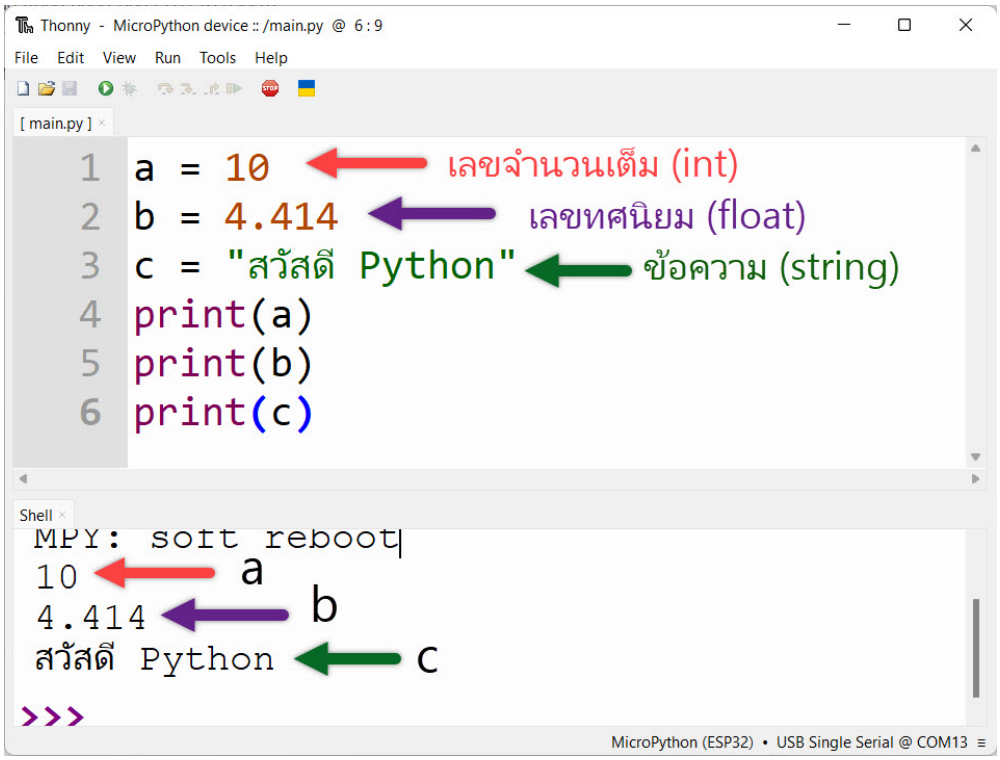
ถ้าต้องการตรวจสอบว่าตัวแปรเป็นชนิดข้อมูลอะไร ใช้คำสั่ง type() ได้ เช่น:



ตัวอย่าง:

- a = 10 # เป็นตัวเลขจำนวนเต็ม (int)
- b = 4.414 # เป็นทศนิยม (float)
- c = "hello" # เป็นข้อความ (string)

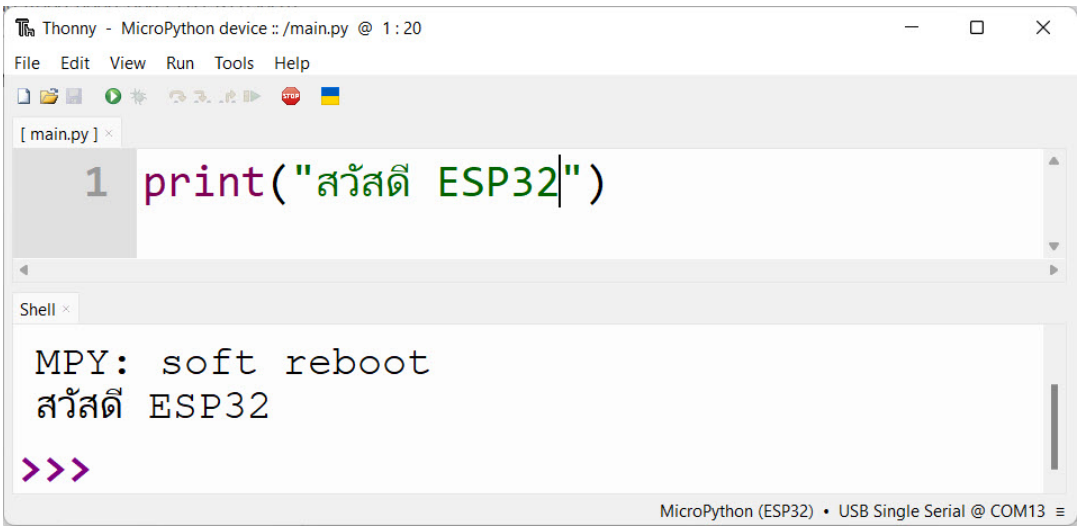
เมื่อตัวแปรถูกนำไปใช้เช่นสั่ง print ค่าออกมาแสดงผล



✨ จุดเด่นนี้ช่วยให้มือใหม่ไม่ต้องจำหรือประกาศชนิดของตัวแปรล่วงหน้า ลดความสับสนในการเริ่มต้นเขียนโปรแกรมได้อย่างมาก

- **เขียนสั้นกระชับ (Concise)**

โค้ดใน Python มักมีจำนวนบรรทัดน้อยกว่าภาษาอื่นสำหรับงานเดียวกัน ตัวอย่างเช่น การแสดงข้อความใน ภาษา **C/C++** ต้องเขียนหลายบรรทัด แต่ใน **Python** สามารถเขียนได้เพียงบรรทัดเดียว:



ในขณะที่ภาษา C/C++ จะต้องเขียนดังนี้:

ความสั้นกระชับนี้ช่วยลดเวลาในการเขียนโปรแกรม และทำให้ผู้เริ่มต้นสามารถเข้าใจการทำงานของโปรแกรมได้เร็วขึ้น ด้วยจุดเด่นเหล่านี้ Python จึงเป็นภาษาที่เหมาะสำหรับผู้เริ่มต้นเขียนโปรแกรม และเหมาะอย่างยิ่งกับการเรียนรู้ควบคู่กับการทดลองใช้งานบอร์ดไมโครคอนโทรลเลอร์ เช่น **ESP32** ผ่าน **MicroPython**

3.4 การใช้งาน Python กับฮาร์ดแวร์

Python ปกติไม่สามารถควบคุมฮาร์ดแวร์ได้โดยตรง แต่ MicroPython สามารถทำได้ผ่านโมดูลเฉพาะ เช่น:

- machine — สำหรับควบคุม GPIO, PWM, ADC, ฯลฯ
- utime หรือ time — ใช้หน่วงเวลา (delay)
- network — ใช้เชื่อมต่อ Wi-Fi หรือ Ethernet

ดังนั้น การเรียนรู้ **Python** ในบริบทของ **MicroPython** จึงจำเป็นต้องเน้นเฉพาะส่วนที่สามารถนำมาใช้ควบคุมฮาร์ดแวร์ได้จริง

ต่อจากนี้ เราจะเริ่มเข้าสู่พื้นฐานสำคัญที่ควรรู้ทุกประการ ตั้งแต่การสร้างตัวแปร การใช้ชนิดข้อมูลพื้นฐาน การใช้คำสั่งควบคุม การสร้างฟังก์ชัน ไปจนถึงการเขียนโค้ดที่ซับซ้อนยิ่งขึ้น โดยทุกหัวข้อจะมีตัวอย่างที่ **เชื่อมโยงกับฮาร์ดแวร์จริง** เพื่อให้เห็นภาพและเข้าใจได้ง่ายที่สุด

เราจะไม่เรียน **Python** เพื่อ **print("Hello")** แต่จะเรียนเพื่อ "ควบคุมอุปกรณ์จริงด้วยความเข้าใจอย่างมืออาชีพ"!

บทที่ 4: เรียนรู้โครงสร้างภาษา Python เพื่อควบคุมอุปกรณ์จริง

หลังจากที่เราได้ทดลองใช้งาน MicroPython เบื้องต้นและเข้าใจแนวคิดการควบคุมฮาร์ดแวร์ด้วยคำสั่งแบบง่าย ๆ กันมาแล้ว บทนี้จะเป็นจุดเริ่มต้นของการเรียนรู้ "โครงสร้างภาษา Python" อย่างเป็นระบบ เพื่อให้สามารถเขียนโปรแกรมที่ซับซ้อนและมีประสิทธิภาพได้จริงในการควบคุมอุปกรณ์ต่าง ๆ

เราจะเริ่มจากรากฐานสำคัญที่สุดของการเขียนโปรแกรม นั่นคือ "ตัวแปรและชนิดข้อมูลพื้นฐาน" ซึ่งถือเป็นรากของทุกโค้ดที่มีอยู่ในระบบคอมพิวเตอร์ และทุกโครงงานควบคุมฮาร์ดแวร์ด้วย Python หรือ MicroPython

ทำความเข้าใจเนื้อหาก่อนจะเข้าสู่เรื่องของตัวแปร

ในบทนี้จะเริ่มต้นทำความรู้จักกับ “ตัวแปร” (Variables) ซึ่งเป็นพื้นฐานสำคัญของทุกภาษาโปรแกรม รวมถึง MicroPython สำหรับบอร์ด ESP32 อาจมีตัวอย่างโค้ดหรือคำสั่งใหม่ ๆ ที่ดูยังไม่คุ้นเคยปรากฏขึ้นระหว่างการอธิบาย ขอให้ผู้อ่านไม่ต้องกังวลหรือจดจำทุกรายละเอียดในทันที เป้าหมายสำคัญของบทนี้คือการปูความเข้าใจพื้นฐาน เพื่อให้สามารถนำไปใช้ในการเขียนโปรแกรมควบคุมอุปกรณ์จริงในบทต่อ ๆ ไป หากพบว่ามีบางประเด็นยังไม่เข้าใจเต็มที่ ขอให้ศึกษาต่อเนื่องก่อน แล้วค่อยย้อนกลับมาทบทวนเนื้อหาในบทนี้อีกครั้ง เมื่อเริ่มทดลองใช้งานจริงและเห็นการนำตัวแปรไปใช้ในโปรเจกต์ จะเข้าใจและเห็นความสำคัญของแนวคิดเหล่านี้อย่างเป็นธรรมชาติ

4.1 ตัวแปร (Variables) และชนิดข้อมูลพื้นฐาน (Data Types)

📌 ตัวแปรคืออะไร?

ตัวแปร (Variable) เปรียบเสมือน "ป้ายชื่อ" หรือ "ฉลาก" ที่เราสร้างขึ้นเพื่อใช้อ้างอิงไปยัง "กล่องเก็บของ" ในหน่วยความจำของไมโครคอนโทรลเลอร์ (เช่น ESP32) "กล่อง" นี้สามารถเก็บข้อมูลได้หลากหลายประเภท เพื่อให้โปรแกรมของเรานำข้อมูลนั้นไปใช้งานต่อได้ ไม่ว่าจะเป็นการคำนวณ การตัดสินใจ หรือการควบคุมอุปกรณ์ฮาร์ดแวร์

ลองจินตนาการว่าหน่วยความจำของ ESP32 คือโกดังขนาดใหญ่ที่มีช่องเก็บของมากมาย การที่เราจะจำตำแหน่งของแต่ละช่อง (ซึ่งเป็นตัวเลขฐานสิบหกที่ซับซ้อน) นั้นเป็นเรื่องยากมาก ตัวแปรจึงเข้ามาช่วยให้เราสามารถตั้งชื่อที่สื่อความหมายให้กับช่องเก็บของเหล่านั้นได้ เช่น แทนที่จะจำว่าข้อมูลอุณหภูมิถูกเก็บไว้ที่ตำแหน่ง 0x3FF0001A เราสามารถสร้างตัวแปรชื่อ current_temperature เพื่ออ้างอิงถึงข้อมูลนั้นได้ง่ายกว่ามาก ตัวอย่างในส่วนของการนำตัวแปรไปใช้เก็บข้อมูลในงานจริงกับโปรเจกต์ต่าง ๆ ของเรา เช่น ตัวเลข, ข้อความ, ค่าจากเซ็นเซอร์, หรือสถานะของอุปกรณ์ต่าง ๆ เช่น LED, ปุ่มกด เป็นต้น

✨ **คุณสมบัติสำคัญของตัวแปรใน MicroPython**

- **ความสะดวกในการอ่านและเขียนโค้ด:** การใช้ชื่อที่สื่อความหมายทำให้โค้ดเข้าใจง่ายขึ้นมาก
- **สามารถเปลี่ยนค่าได้ตลอดเวลา**: ช่วยให้โปรแกรมมีความยืดหยุ่นและปรับตามสถานการณ์
- **การนำข้อมูลมาใช้ซ้ำ:** เก็บค่าที่คำนวณได้หรือค่าที่รับมาจากเซ็นเซอร์ไว้ในตัวแปร เพื่อนำไปใช้ในส่วนอื่น ๆ ของโปรแกรมได้โดยไม่ต้องคำนวณใหม่หรืออ่านค่าซ้ำ
- **ไม่ต้องประกาศชนิดล่วงหน้า** (Dynamic Typing): ชนิดข้อมูลจะถูกกำหนดตามค่าที่กำหนดให้ในขณะรันโปรแกรม
- **ไม่ต้องจองพื้นที่หน่วยความจำเอง**: Python จะจัดการให้โดยอัตโนมัติ ต่างจากภาษา C/C++

กฎการตั้งชื่อตัวแปรใน Python (และ MicroPython)

1. ต้องขึ้นต้นด้วยตัวอักษร (a-z, A-Z) หรือเครื่องหมายขีดล่าง (_) เท่านั้น

✅ ถูกต้อง: my_var , _internal_var , Temperature

❌ ไม่ถูกต้อง: 1var (ขึ้นต้นด้วยตัวเลข), !error (ขึ้นต้นด้วยสัญลักษณ์อื่น)

2. ตัวถัดไปใช้ตัวอักษร, ตัวเลข หรือขีดล่าง (_) ได้

✅ ถูกต้อง: sensor_reading_1 , pin_A0 , user_input_value

❌ ไม่ถูกต้อง: sensor-reading (ใช้ขีดกลางไม่ได้), sensor reading (มีเว้นวรรคไม่ได้)

3. ตัวพิมพ์เล็กและตัวพิมพ์ใหญ่ถือเป็นคนละตัวแปร (Case-Sensitive)

✅ ถูกต้อง: myVariable กับ myvariable เป็นคนละตัวแปร

❌ ไม่ถูกต้อง: ใช้ Temperature กับ temperature แทนกัน

4. ห้ามใช้คำสงวน (Reserved Keywords) ของ Python เป็นชื่อตัวแปร

❌ ไม่ถูกต้อง: ตั้งชื่อตัวแปรเป็น if , while , for , def , class , True , False , None

คำสงวนดูได้จากภาคผนวกท้ายเล่ม

5. ควรตั้งชื่อให้สื่อความหมาย (Descriptive Names) เพื่อให้ง่ายต่อการเข้าใจ

✅ ถูกต้อง: temperature , user_name , sensor_value (รู้ทันทีว่าตัวแปรเก็บข้อมูลอะไร)

❌ ไม่ถูกต้อง: x , a1 , temp (ไม่สื่อความหมาย)

6. เคล็ดลับ: ใช้ snake_case (ขีดล่างคั่นแต่ละคำ) เพื่อให้อ่านง่าย

✅ ถูกต้อง: sensor_reading , motor_direction , user_input

❌ ไม่ถูกต้อง: SensorReading (camelCase), sensor reading (มีช่องว่าง

⚙️ ชนิดข้อมูลพื้นฐานใน Python

1. **int (Integer - จำนวนเต็ม)**

ชนิดข้อมูล **int** หรือจำนวนเต็มใน MicroPython คือชนิดข้อมูลที่ใช้แทนตัวเลขที่ไม่มีจุดทศนิยม ไม่ว่าจะเป็นจำนวนเต็มบวก จำนวนเต็มลบ หรือศูนย์ เช่น 0, 100, -5, 2048 หรือ -9999 โดยลักษณะเด่นของ int ใน MicroPython ที่ทำงานบน ESP32 คือมีความยืดหยุ่นสูงมาก เนื่องจาก MicroPython ใช้ระบบตัวแปรแบบ **arbitrary-precision integers** ซึ่งหมายความว่าตัวแปรจำนวนเต็มจะสามารถเก็บค่าที่มีขนาดใหญ่ได้เท่ากับพื้นที่หน่วยความจำที่มีอยู่ ไม่ได้จำกัดขนาดสูงสุดเหมือนภาษา C หรือ C++ ที่ int มักจะมีขนาดตายตัว เช่น 16 บิตหรือ 32 บิต ตัวอย่างเช่น ถ้าเขียนโค้ดให้บวกเลขทีละล้านจนถึงหลักหมื่นล้าน โปรแกรมก็ยังสามารถเก็บค่าจำนวนเต็มเหล่านั้นไว้ได้ (ตราบเท่าที่หน่วยความจำยังเพียงพอ) โดยไม่เกิดปัญหา overflow แบบที่พบในภาษา C/C++ จุดนี้ทำให้ผู้เขียนโปรแกรมบน ESP32 ด้วย MicroPython ไม่ต้องกังวลเรื่องขนาดสูงสุดของตัวแปร int เหมือนแต่ก่อน

ในทางปฏิบัติ int ถูกนำไปใช้ในหลากหลายสถานการณ์ เช่น

- **การนับรอบของลูป**
- **การเก็บค่าดิจิทัลจากเซ็นเซอร์ที่ให้ผลลัพธ์เป็นจำนวนเต็ม**
- **การกำหนดสถานะต่าง ๆ ที่ใช้เลขจำนวนเต็มแทนเงื่อนไข เช่น เปิด/ปิด**
- **การนับจำนวนผู้ใช้**
- **การคำนวณคะแนน**
- **การจัดการกับค่าเวลา เช่น จำนวนมิลลิวินาทีที่ผ่านไปแล้วในระบบ**

ตัวอย่างโค้ดง่าย ๆ ใน MicroPython

<untitled> *

```python
# ตัวอย่างการใช้งานชนิดข้อมูล int ใน MicroPython
user_count = 10                 # จำนวนผู้ใช้งาน
device_status = 1               # สถานะอุปกรณ์ (1 = เปิด, 0 = ปิด)
elapsed_time_ms = 5000          # เวลาที่ผ่านไปในระบบ (มิลลิวินาที)
score = 0                       # คะแนนสะสม

# วนลูปนับรอบ 5 ครั้ง และเพิ่มคะแนนสะสม
for i in range(5):
    score = score + 20          # เพิ่มคะแนนครั้งละ 20

print("จำนวนผู้ใช้งานทั้งหมด:", user_count)
print("สถานะอุปกรณ์:", device_status)
print("เวลาที่ผ่านไป:", elapsed_time_ms, "มิลลิวินาที")
print("คะแนนสะสมรวม:", score)
```

คำอธิบายตัวอย่างโค้ดนี้:

- ตัวแปร user_count ใช้แทนจำนวนผู้ใช้งาน (จำนวนเต็ม)
- ตัวแปร device_status ใช้แทนสถานะ เปิด(1)/ปิด(0)
- ตัวแปร elapsed_time_ms ใช้เก็บค่าจำนวนเต็มแทนเวลาที่ผ่านไปในระบบ
- ตัวแปร score และตัวแปรควบคุมลูป i เป็นจำนวนเต็มที่แสดงให้เห็นถึงการใช้งานทางคณิตศาสตร์ เช่น บวก ลบ คูณ หารได้โดยตรง

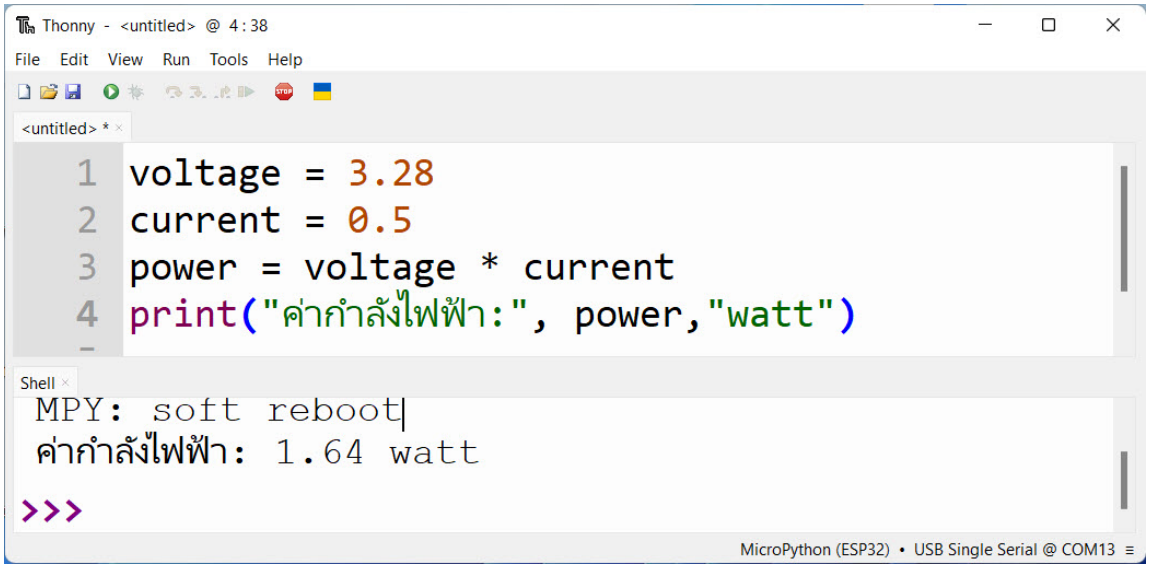
การดำเนินการทางคณิตศาสตร์ด้วย int ใน MicroPython จะดำเนินการได้อย่างรวดเร็วและตรงไปตรงมา นอกจากนี้ เรายังสามารถใช้ int เพื่อควบคุมลอจิกในโปรแกรม เช่น ใช้เป็นตัวแปรเงื่อนไขใน if หรือใช้วนซ้ำในลูป for ได้อย่างมีประสิทธิภาพ

จุดที่ควรระวังสำหรับการใช้ int ใน MicroPython คือ แม้จะสามารถเก็บค่าขนาดใหญ่ได้ แต่หากเก็บค่าที่ใหญ่จนเกินไปจนหน่วยความจำไม่พอ โปรแกรมอาจหยุดทำงานหรือเกิดข้อผิดพลาด (MemoryError) ดังนั้นควรใช้ขนาดข้อมูลให้เหมาะสมกับงานที่ต้องการ และหมั่นตรวจสอบสถานะของหน่วยความจำอยู่เสมอ โดยรวมแล้ว int เป็นชนิดข้อมูลหลักที่ใช้งานได้ง่าย ยืดหยุ่น และเป็นหัวใจสำคัญของทุกโปรแกรมที่เกี่ยวข้องกับตัวเลขใน MicroPython

2. **float (Floating-point - จำนวนจริง/ทศนิยม):**

ชนิดข้อมูล float หรือจำนวนจริงทศนิยมใน MicroPython คือชนิดข้อมูลที่ใช้แทนตัวเลขที่มีจุดทศนิยม เช่น 3.14, -0.5 หรือ 1.414 ตัวแปรชนิดนี้มีความสำคัญอย่างมากเมื่อเขียนโปรแกรมที่ต้องคำนวณค่าที่ละเอียด เช่น การอ่านค่าจากเซ็นเซอร์วัดอุณหภูมิที่อาจได้ค่าเป็น 28.35 องศาเซลเซียส หรือการคำนวณแรงดันไฟฟ้าที่วัดได้ในวงจรจริง สำหรับ MicroPython บน ESP32 ข้อมูลแบบ float จะใช้ระบบ 32 บิต หรือที่เรียกว่า single precision ซึ่งสามารถเก็บค่าทศนิยมได้ประมาณ 6–7 หลักสำคัญ และมีช่วงค่ากว้างเพียงพอสำหรับงานไมโครคอนโทรลเลอร์และการประมวลผลสัญญาณแบบแอนาล็อกทั่วไป

ข้อดีของ float คือสามารถดำเนินการทางคณิตศาสตร์กับตัวเลขที่มีจุดทศนิยมได้โดยตรง เช่น การบวก ลบ คูณ หาร และการคำนวณทางฟิสิกส์ที่ต้องการความละเอียดของผลลัพธ์ เช่น เมื่อวัดแรงดันไฟฟ้าแล้วต้องนำไปคำนวณหาค่ากำลังไฟฟ้าในวงจร หรือประเมินผลการเปลี่ยนแปลงของอุณหภูมิที่ต้องใช้ความแม่นยำมากกว่าจำนวนเต็ม ตัวอย่างเช่น ถ้าวัดแรงดันไฟฟ้าได้ 3.28 โวลต์ และวัดกระแสได้ 0.5 แอมแปร์ ต้องการคำนวณค่ากำลังไฟฟ้าในหน่วยวัตต์ก็สามารถเขียนโปรแกรมดังนี้



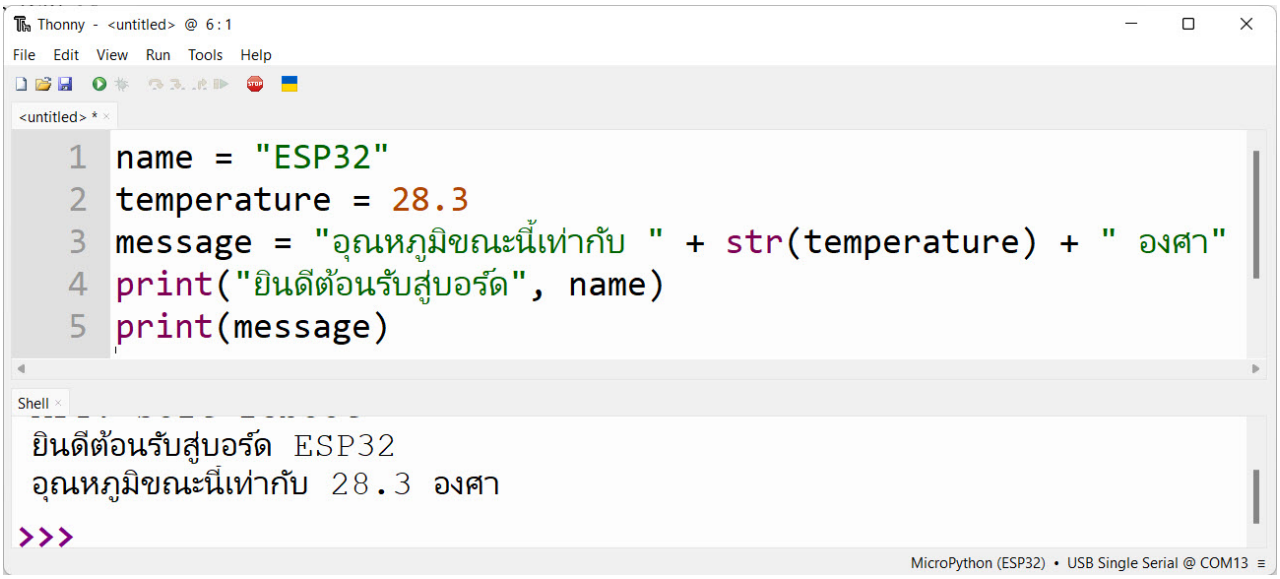
ในตัวอย่างนี้ ตัวแปร voltage, current และ power ต่างก็เป็น float ทั้งหมด สามารถนำไปใช้คำนวณค่าที่เกี่ยวข้องกับสัญญาณแอนาล็อกหรือค่าทางวิทยาศาสตร์ได้อย่างถูกต้องและแม่นยำ อย่างไรก็ตามควรทราบว่าการเปรียบเทียบค่าทศนิยมด้วยเครื่องหมายเท่ากัน (==) อาจไม่เหมาะสมเสมอไป เพราะผลลัพธ์ที่ได้จากการคำนวณ float อาจมีความคลาดเคลื่อนเล็กน้อยด้วยข้อจำกัดของการจัดเก็บข้อมูลภายในระบบคอมพิวเตอร์ หากต้องการเปรียบเทียบค่าทศนิยมว่ามีค่าใกล้เคียงกันมากหรือไม่ ควรใช้วิธีเปรียบเทียบแบบช่วง หรือใช้ฟังก์ชัน math.isclose

นอกจากนี้การคำนวณด้วย float อาจใช้เวลามากกว่าการคำนวณจำนวนเต็มเล็กน้อยบนไมโครคอนโทรลเลอร์ จึงควรเลือกใช้ให้เหมาะสมกับลักษณะงาน ถ้าข้อมูลเป็นค่าทศนิยมที่สำคัญต่อผลลัพธ์หรือจำเป็นต้องใช้ความละเอียด เช่น ในการอ่านเซ็นเซอร์แบบแอนาล็อก การคำนวณอัตราส่วน การประมวลผลผลรวมเฉลี่ย หรือการวัดปริมาณทางวิทยาศาสตร์ float จะเป็นชนิดข้อมูลที่ขาดไม่ได้สำหรับการพัฒนาโปรแกรม MicroPython บน ESP32

3. **str (String - ข้อความ/สายอักขระ):**

ชนิดข้อมูล str หรือที่เรียกว่าสตริง (String) ใน MicroPython คือชนิดข้อมูลที่ใช้สำหรับเก็บข้อมูลในรูปแบบข้อความหรือสายอักขระ เช่น คำว่า “Hello, world!”, “อุณหภูมิ = 28.3 องศา” หรือ “ESP32” สตริงประกอบขึ้นจากอักขระแต่ละตัวที่เรียงต่อกัน และสามารถใช้เครื่องหมายคำพูดเดี่ยว (') หรือคำพูดคู่ (") ครอบข้อความไว้ก็ได้ เช่น 'sensor', "temperature" หรือ "ค่าแรงดันไฟฟ้า"

สตริงเป็นข้อมูลพื้นฐานที่มีบทบาทสำคัญในโปรแกรม MicroPython โดยเฉพาะอย่างยิ่งในงานที่ต้องรับค่าจากผู้ใช้ แสดงผลบนหน้าจอ ส่งข้อความผ่านพอร์ตอนุกรม หรือบันทึกข้อมูลลงในไฟล์ สตริงสามารถนำมารวมต่อกันหรือดัดแปลงข้อความได้ง่าย เช่น การเชื่อมข้อความสองชุดด้วยเครื่องหมายบวก (+) หรือแปลงค่าตัวเลขเป็นข้อความก่อนนำไปแสดงผล ตัวอย่างการใช้งานเช่น



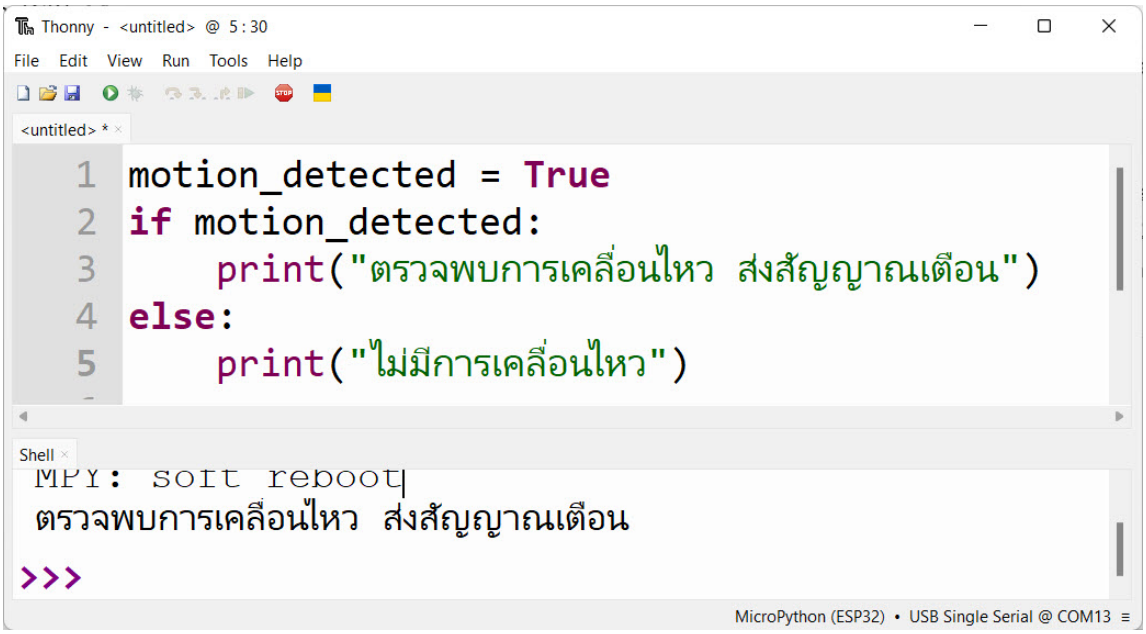
ในตัวอย่างนี้ name เป็นสตริงที่เก็บชื่อบอร์ด ESP32 ตัวแปร message เป็นสตริงที่ประกอบขึ้นใหม่โดยเชื่อมข้อความและค่าตัวแปร temperature ที่ถูกแปลงเป็นสตริงด้วยฟังก์ชัน str() ก่อนนำมาเชื่อมกัน การแปลงข้อมูลชนิดอื่นเป็นสตริงแบบนี้จำเป็นมากเมื่อเราต้องการแสดงผลค่าตัวเลขหรือค่าตัวแปรประเภทอื่นร่วมกับข้อความในประโยคเดียว

นอกจากนี้ สตริงใน MicroPython ยังสามารถนำไปตัดแบ่ง นับจำนวนอักขระ ดึงข้อมูลบางส่วน หรือค้นหาข้อความย่อยได้โดยใช้ฟังก์ชันหรือเมธอดต่าง ๆ เช่น len() เพื่อดูความยาวของข้อความ หรือ .split() เพื่อแยกคำในประโยค โดยรูปแบบและความสามารถของสตริงใน MicroPython จะคล้ายกับ Python ทั่วไป ทำให้เขียนโปรแกรมที่จัดการข้อมูลข้อความได้ง่าย สะดวก และยืดหยุ่นมาก เหมาะสำหรับงานที่เกี่ยวข้องกับการรับส่งข้อมูล การบันทึกข้อมูล หรือการประมวลผลข้อความในระบบ IoT และงานอิเล็กทรอนิกส์แบบฝังตัวทุกชนิด

4. **bool (Boolean - ค่าตรรกะ):**

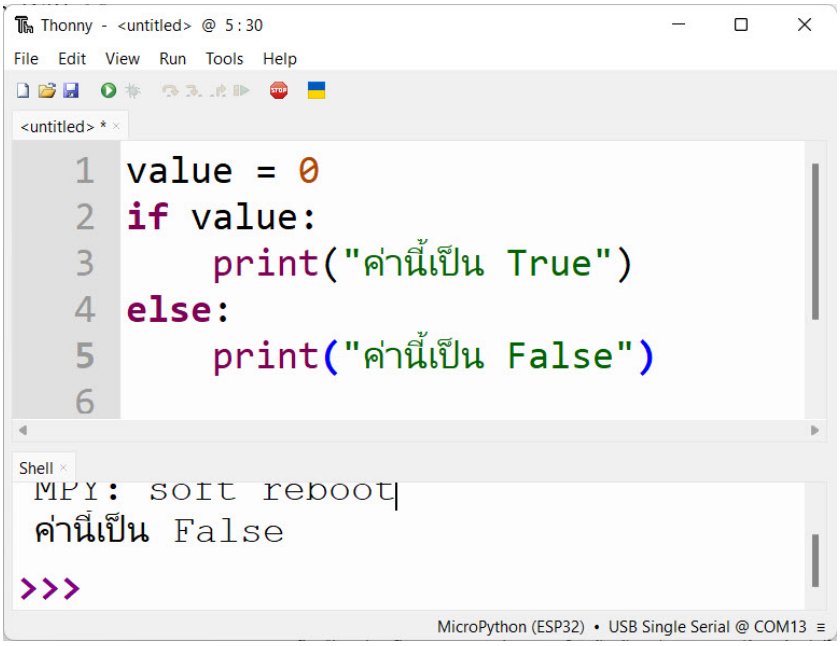
ชนิดข้อมูล bool หรือ Boolean ใน MicroPython คือชนิดข้อมูลที่ใช้แทนค่าตรรกะเพียงสองค่า ได้แก่ True (จริง) และ False (เท็จ) ค่านี้มีความสำคัญอย่างยิ่งในการเขียนโปรแกรมเพื่อควบคุมเงื่อนไขต่าง ๆ ไม่ว่าจะเป็นการตัดสินใจผ่านคำสั่ง if, การวนซ้ำในลูป while, หรือการตรวจสอบสถานะของอุปกรณ์และเซ็นเซอร์ในระบบฝังตัวบน ESP32 ตัวแปรชนิด bool มักจะเกิดขึ้นจากผลลัพธ์ของนิพจน์เปรียบเทียบ เช่น 5 > 3 จะได้ค่า True หรือจากการอ่านสถานะที่มีเพียงสองทางเลือก เช่น สวิตช์เปิด (True) หรือปิด (False)

การใช้งานค่าตรรกะใน MicroPython เป็นไปอย่างตรงไปตรงมา เช่น การตรวจสอบเงื่อนไขและการสั่งงานต่อเนื่องตามค่าที่ได้ ตัวอย่างเช่น เมื่ออ่านค่าจากเซ็นเซอร์ตรวจจับความเคลื่อนไหวแล้วต้องการให้ระบบแจ้งเตือนเฉพาะเมื่อเซ็นเซอร์ตรวจพบการเคลื่อนไหวเท่านั้น ก็สามารถเขียนโปรแกรมโดยใช้ตัวแปร Boolean เพื่อควบคุมการทำงานได้ทันที ตัวอย่างโค้ดต่อไปนี้แสดงให้เห็นการใช้ตัวแปร Boolean อย่างชัดเจน



นอกจากนี้ Boolean ยังสามารถนำไปใช้ร่วมกับการเปรียบเทียบผลลัพธ์ที่ได้จากเซ็นเซอร์หรือพอร์ตอินพุตของ ESP32 ได้โดยตรง เช่น การเปรียบเทียบค่าความชื้นว่าเกินระดับที่กำหนดหรือไม่ หรือเช็กสถานะของปุ่มกดว่าถูกกดอยู่หรือไม่ โดยค่าตรรกะนี้จะช่วยให้โปรแกรมสามารถตัดสินใจและควบคุมการทำงานของอุปกรณ์ได้อย่างแม่นยำ

อีกคุณสมบัติหนึ่งที่ควรรู้คือ MicroPython (และ Python) จะถือว่าค่าตัวเลข 0, อักขระว่าง "" หรือค่า None เป็น False ขณะที่ค่าตัวเลขที่ไม่ใช่ศูนย์ หรือสตริงที่ไม่ว่างจะถูกมองเป็น True โดยอัตโนมัติในบริบทของการเปรียบเทียบ เช่น



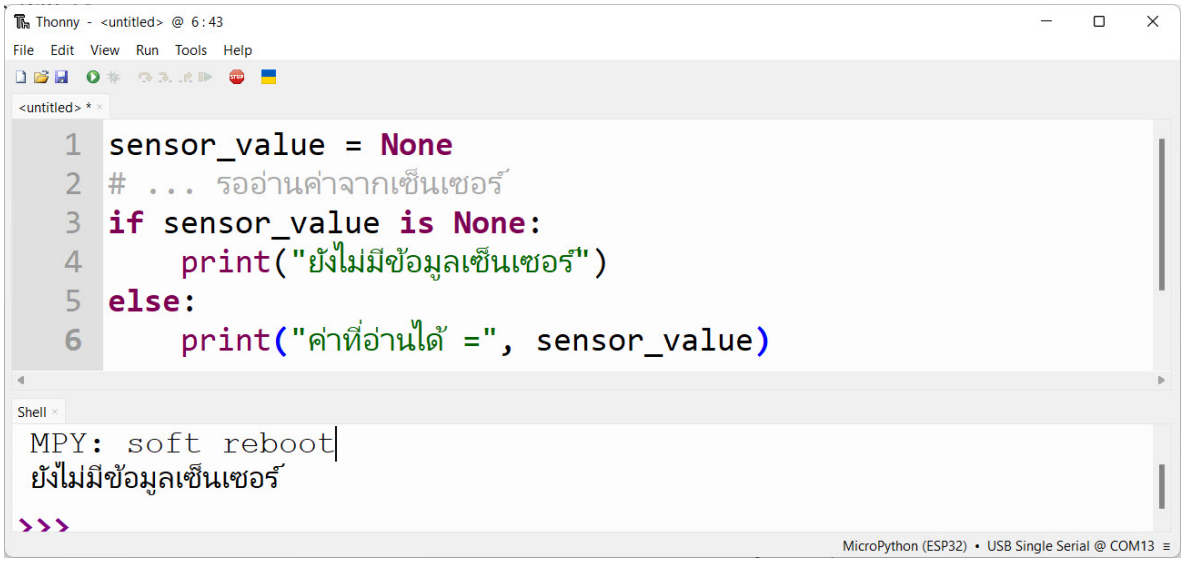
ในตัวอย่างนี้ value มีค่า 0 ดังนั้นเงื่อนไขจะเป็น False โปรแกรมจะพิมพ์ข้อความ "ค่านี้เป็น False" ออกมา จุดนี้มีประโยชน์อย่างมากในการเขียนโค้ดที่ต้องตรวจสอบค่าสถานะในลักษณะเงื่อนไขที่หลากหลาย

ชนิดข้อมูล Boolean จึงเป็นหัวใจสำคัญในการสร้างตรรกะให้กับโปรแกรม MicroPython ไม่ว่าจะเป็นการควบคุมการไหลของโปรแกรม การตรวจสอบสถานะอุปกรณ์ การตัดสินใจ และการสั่งงานต่าง ๆ ในระบบอิเล็กทรอนิกส์แบบฝังตัวทุกชนิด

5. **NoneType (ชนิดข้อมูลของ None):**

ชนิดข้อมูล NoneType ใน MicroPython คือชนิดข้อมูลที่ใช้แทนค่า "ไม่มีค่า" หรือ "ว่างเปล่า" โดยจะมีเพียงค่าคงที่เดียวเท่านั้นคือ None การมี NoneType มีความสำคัญในการเขียนโปรแกรมที่ต้องการระบุว่ายังไม่มีข้อมูลเกิดขึ้น ยังไม่ได้รับค่ากลับมาจากฟังก์ชัน หรือใช้เป็นค่าสถานะเริ่มต้นในกรณีที่ยังไม่ต้องการกำหนดค่าให้กับตัวแปร

ในภาษา MicroPython บน ESP32 เช่นเดียวกับใน Python มาตรฐาน None จะถูกใช้เป็นค่าคืนกลับโดยอัตโนมัติเมื่อฟังก์ชันไม่ได้มีคำสั่ง return ผลลัพธ์ใด ๆ หรือเมื่อสร้างตัวแปรขึ้นมาเพื่อจองพื้นที่ก่อนที่จะนำไปใช้งานจริง ตัวอย่างเช่น เมื่อต้องการเตรียมตัวแปรสำหรับเก็บค่าที่จะได้รับจากการอ่านเซ็นเซอร์ในภายหลัง แต่ยังไม่ได้อ่านค่าในขณะนั้น ก็สามารถกำหนดให้ตัวแปรเริ่มต้นเป็น None ได้ก่อน



ในตัวอย่างนี้ sensor_value จะเริ่มต้นด้วยค่า None ซึ่งแปลว่าไม่มีข้อมูล เมื่อถึงจุดที่อ่านค่าเซ็นเซอร์และนำค่ามาเก็บไว้ใน sensor_value เงื่อนไข if จะช่วยให้โปรแกรมสามารถแยกกรณีที่ยังไม่มีข้อมูลกับกรณีที่มีข้อมูลแล้วได้อย่างชัดเจน จุดนี้มีประโยชน์มากสำหรับโปรแกรมที่ต้องจัดการกับกระบวนการที่อาจเกิดข้อผิดพลาดหรือยังไม่มีข้อมูลพร้อมใช้ในบางช่วงเวลา

นอกจากนี้ NoneType ยังถูกใช้ในการตรวจสอบว่าฟังก์ชันใดคืนค่ากลับมาหรือไม่ และยังเป็นค่าเริ่มต้นที่ใช้แทนความว่างเปล่าในระบบอิเล็กทรอนิกส์หรือ IoT ที่ต้องการความชัดเจนในการจัดการข้อมูล ดังนั้น NoneType จึงเป็นเครื่องมือสำคัญที่ช่วยให้การเขียนโปรแกรมด้วย MicroPython มีความยืดหยุ่นและตรวจสอบสถานะต่าง ๆ ได้อย่างมีประสิทธิภาพ

ชนิดข้อมูลพื้นฐานที่ใช้งานบ่อย

ชนิดข้อมูลพื้นฐานที่ใช้งานบ่อยใน MicroPython คือหัวใจสำคัญของการเขียนโปรแกรมบนบอร์ด ESP32 และไมโครคอนโทรลเลอร์อื่น ๆ ทุกระบบ เพราะเป็นรากฐานที่กำหนดลักษณะและวิธีจัดเก็บข้อมูลที่จะใช้ตลอดทั้งโปรเจกต์ โดยชนิดข้อมูลหลักที่พบเห็นและใช้งานอยู่เสมอ ได้แก่ int สำหรับเก็บจำนวนเต็มที่ไม่มีจุดทศนิยม เช่น ค่าที่ได้จากการนับรอบ การวัดพัลส์ดิจิทัล หรือใช้ในลูปควบคุมต่าง ๆ, float สำหรับตัวเลขที่มีจุดทศนิยม เช่น ผลการวัดแรงดันไฟฟ้าหรืออุณหภูมิที่ต้องการความละเอียดสูง, str ซึ่งใช้เก็บข้อความหรือสายอักขระทุกชนิด ไม่ว่าจะเป็นชื่ออุปกรณ์ ข้อมูลที่ต้องการแสดงผลหรือส่งผ่านระบบสื่อสาร และ bool สำหรับข้อมูลที่มีค่าตรรกะจริงหรือเท็จ ซึ่งมักนำมาใช้ควบคุมการตัดสินใจของโปรแกรม ตรวจสอบสถานะอุปกรณ์ หรือใช้กับคำสั่งเงื่อนไข if และลูป while

นอกจากนี้ยังมีชนิดข้อมูล NoneType ซึ่งใช้สำหรับแทนค่าที่ว่างเปล่าหรือยังไม่มีข้อมูล โดยเหมาะกับสถานการณ์ที่ต้องการเช็กสถานะก่อนนำข้อมูลไปใช้งาน หรือใช้เป็นค่าเริ่มต้นในโปรแกรมที่มีการรอรับข้อมูลจากอุปกรณ์ภายนอก สิ่งเหล่านี้นับว่าเป็นพื้นฐานสำคัญที่ต้องเข้าใจอย่างลึกซึ้ง เพราะหากสามารถเลือกและใช้งานชนิดข้อมูลให้เหมาะสม จะช่วยให้โปรแกรมมีประสิทธิภาพและปลอดภัยต่อการใช้งานจริงบนอุปกรณ์ฝังตัวทุกชนิด

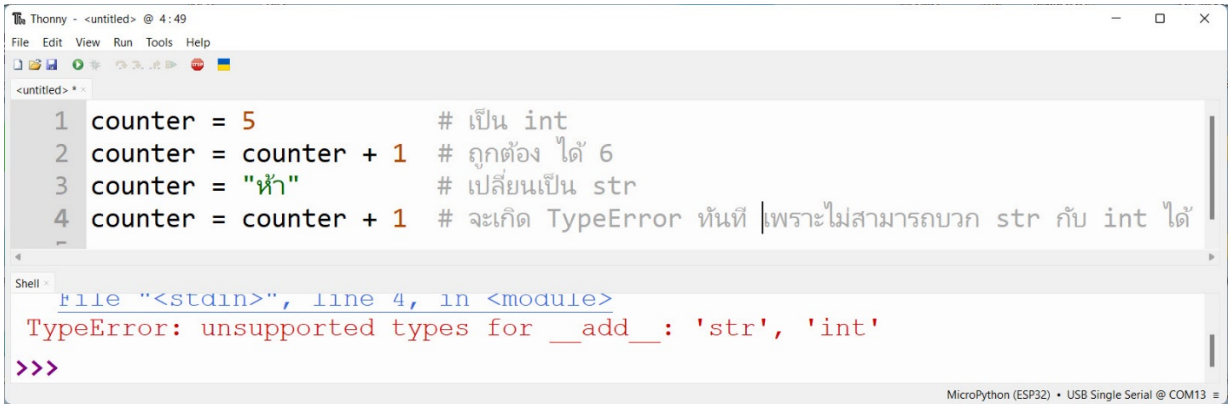
❗ หมายเหตุสำคัญ: ในการเขียนโปรแกรมด้วย Python และ MicroPython ไม่จำเป็นต้องประกาศชนิดของตัวแปรล่วงหน้าเหมือนภาษา C หรือภาษาอื่น ๆ ที่เป็น static type เพราะ Python เป็นภาษาแบบ dynamic type ตัวแปรทุกตัวจะถูกกำหนดชนิดข้อมูลโดยอัตโนมัติจากค่าที่กำหนดให้ครั้งแรก เช่น หากกำหนด score = 5 ตัวแปร score จะกลายเป็น int ทันที หรือถ้ากำหนด temperature = 27.5 ก็จะกลายเป็น float อัตโนมัติ และสามารถเปลี่ยนชนิดข้อมูลในภายหลังได้ด้วย เช่น จากจำนวนเต็มกลายเป็นข้อความได้ทันทีหากมีการกำหนดค่าใหม่ เช่น score = "ห้า" โค้ดทั้งหมดจึงกระชับและเรียบง่ายมาก ทำให้เหมาะกับการทดลองและพัฒนาโปรแกรมบนไมโครคอนโทรลเลอร์โดยไม่ต้องกังวลกับข้อจำกัดเรื่องชนิดข้อมูลแบบในภาษาอื่น

แต่ถึงแม้ว่า Python และ MicroPython เป็นภาษาที่จัดการตัวแปรแบบ dynamic type หมายความว่าตัวแปรแต่ละตัวไม่ถูกผูกมัดกับชนิดข้อมูลใด ๆ ตั้งแต่เริ่มต้น ตัวแปรเดียวกันสามารถเปลี่ยนชนิดข้อมูลได้ทันทีที่กำหนดค่าชนิดใหม่เข้าไป เช่น ในตัวอย่างนี้

```
x = 10        # ขณะนี้ x เป็น int
x = "สิบ"     # ขณะนี้ x เปลี่ยนเป็น str โดยอัตโนมัติ
```

ข้อดีอย่างหนึ่งของแนวคิดนี้คือช่วยให้การเขียนโค้ดมีความยืดหยุ่นสูงมาก โปรแกรมเมอร์สามารถทดลองเปลี่ยนชนิดข้อมูล หรือรับค่าที่แตกต่างกันจากอุปกรณ์ต่าง ๆ ได้โดยไม่ต้องแก้ไขโครงสร้างหรือประกาศชนิดล่วงหน้า ทำให้การพัฒนาโปรแกรมในช่วงแรกหรือการทดลองกับเซ็นเซอร์และอุปกรณ์ต่าง ๆ เป็นไปได้ง่ายและรวดเร็ว นอกจากนี้ยังช่วยลดจำนวนบรรทัดโค้ดและความซับซ้อนที่ต้องดูแลเมื่อเปลี่ยนชนิดข้อมูลระหว่างการทำงาน เช่น ในการอ่านค่าจาก Serial หรือรับข้อมูลจากผู้ใช้ ซึ่งอาจต้องแปลงค่าหลายครั้งระหว่างจำนวนเต็ม จำนวนทศนิยม หรือข้อความ

แต่ในทางกลับกัน การที่ตัวแปรสามารถเปลี่ยนชนิดข้อมูลได้ตลอดเวลาโดยไม่มีการแจ้งเตือนใด ๆ ก็มีข้อเสียเช่นกัน โดยเฉพาะในโปรเจกต์ที่มีความซับซ้อน หรือทำงานกับข้อมูลขนาดใหญ่และต้องการความถูกต้องแน่นอน เช่น หากตัวแปรที่ใช้เก็บจำนวนรอบการทำงานของมอเตอร์เปลี่ยนจาก int กลายเป็น str โดยไม่ได้ตั้งใจ โปรแกรมอาจเกิดบั๊กที่ยากต่อการตรวจสอบ หรือเกิดข้อผิดพลาดในขณะรันโปรแกรมโดยไม่ทราบสาเหตุ เช่น



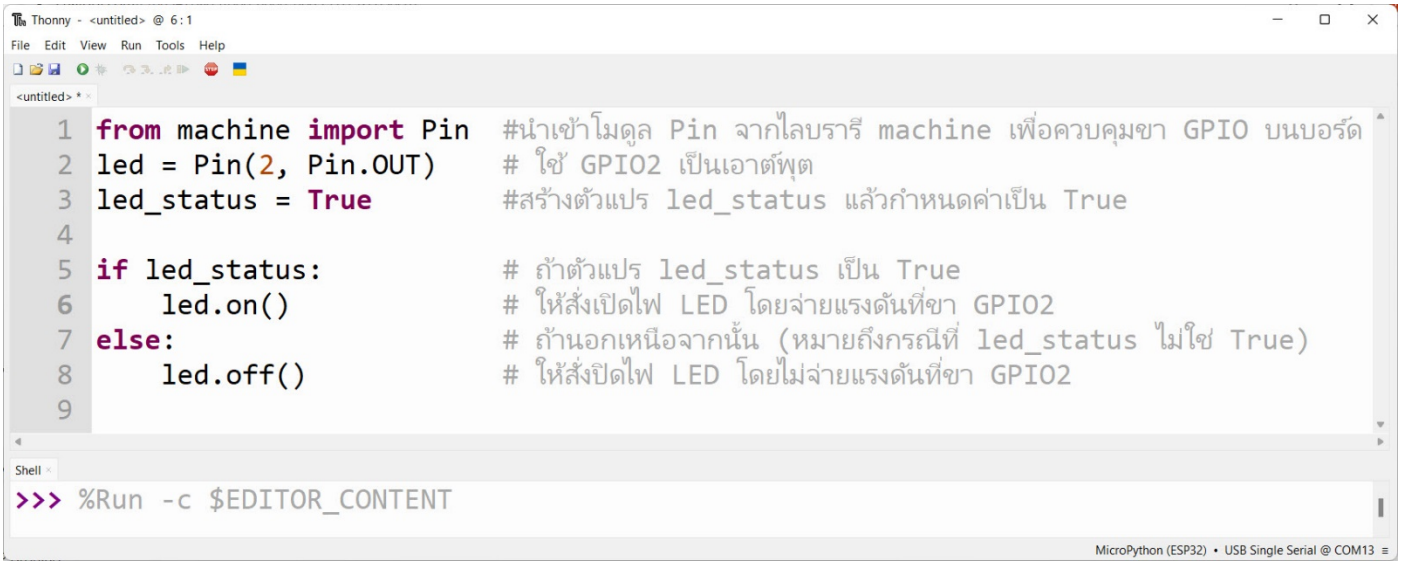
counter = counter + 1 # จะเกิด TypeError ทันที เพราะไม่สามารถบวก str กับ int ได้

นอกจากนี้ Python และ MicroPython จะไม่เตือนหรือป้องกันกรณีแบบนี้ให้เลย โปรแกรมเมอร์จึงต้องมีวินัยในการตั้งชื่อ และตรวจสอบว่าตัวแปรแต่ละตัวควรใช้กับชนิดข้อมูลอะไร ควรหลีกเลี่ยงการนำตัวแปรเดียวกันไปใช้ข้ามประเภทบ่อย ๆ ในส่วนที่สำคัญของโปรแกรม และถ้าเป็นไปได้ควรแปลงค่าหรือสร้างตัวแปรใหม่แทนการ reuse ตัวแปรเดิม

แนวทางที่แนะนำสำหรับการเขียนโปรแกรมบนไมโครคอนโทรลเลอร์คือ ควรใช้ตัวแปรหนึ่งตัวต่อหนึ่งชนิดข้อมูลตลอดทั้งโปรแกรม เว้นแต่เป็นกรณีพิเศษหรือเพื่อการแปลงค่าอย่างชัดเจนเท่านั้น เพื่อป้องกันข้อผิดพลาดที่ตรวจสอบได้ยากในภายหลัง การใช้แนวทางนี้จะช่วยให้โค้ดมีความน่าเชื่อถือ และแก้ไขหรือดูแลต่อยอดได้ง่ายมากขึ้นในระยะยาว

ดังนั้น แม้ dynamic type จะเป็นจุดเด่นของ Python และ MicroPython ในแง่ของความยืดหยุ่นและความสะดวกสบายในการทดลองและพัฒนาโค้ด แต่ในงานระบบฝังตัวหรือ IoT ที่ต้องการความแม่นยำและปลอดภัยสูง ควรเลือกใช้อย่างมีสติและระมัดระวังเสมอ

ตัวอย่างใช้งานตัวแปรจริงกับอุปกรณ์



ในตัวอย่างนี้ ตัวแปร led_status คือการเก็บสถานะของไฟ หากเป็น True ก็สั่งให้ไฟติดทันที ซึ่งในบรรทัดที่ 3 เราได้กำหนดค่าให้ตัวแปร led_status = True ดังนั้นเมื่อโปรแกรมทำงานและตรวจสอบพบว่าค่าเป็น True ก็จะสั่งให้ LED ที่อยู่บนบอร์ด ESP32 ติดสว่างขึ้นมา หากต้องการให้ LED ดับ ก็เปลี่ยนค่า led_status = False เมื่อกดรันอีกครั้ง LED ก็จะดับทันที (ทุกการทดลองเขียนโปรแกรม ต้องต่อบอร์ด ESP เอาไว้ด้วย และเลือก PORT ให้ถูกต้อง)

ในหัวข้อต่อไป เราจะเจาะลึกเรื่อง **ตัวดำเนินการ (Operators)** และ **นิพจน์ (Expressions)** ซึ่งเป็นกลไกในการคำนวณ เปรียบเทียบ และตัดสินใจ — เป็นสิ่งที่จำเป็นอย่างยิ่งสำหรับการควบคุมอุปกรณ์แบบมีเงื่อนไขต่อจากนี้ไป เราจะไม่เขียนโค้ดเพื่อแค่ให้รันได้ แต่จะเขียนให้ควบคุมอุปกรณ์จริงได้อย่างเข้าใจ!

ก่อนที่จะเข้าสู่เรื่องของตัวดำเนินการ (Operators)

เนื้อหาบทนี้จะปูพื้นฐานเกี่ยวกับ “ตัวดำเนินการ” (Operators) ซึ่งเป็นเครื่องมือสำคัญสำหรับการประมวลผลข้อมูลและควบคุมตรรกะในภาษา MicroPython อาจมีคำศัพท์หรือแนวคิดบางส่วนที่ยังดูยากหรือยังไม่เห็นภาพชัดเจนในครั้งแรกที่อ่าน ขอให้ผู้อ่านไม่ต้องกังวล หากรู้สึกว่ายังไม่เข้าใจทุกจุด ประสบการณ์จากผู้เรียนส่วนใหญ่พบว่า เมื่อได้เรียนรู้บทเกี่ยวกับ “คำสั่งควบคุมการทำงาน” แล้ว จะเห็นภาพการใช้งานตัวดำเนินการได้ชัดเจนมากขึ้น หากรู้สึกไม่มั่นใจ ให้ตั้งใจศึกษาต่อไปก่อน แล้วค่อยย้อนกลับมาทบทวนเนื้อหาในบทนี้ใหม่ รับรองว่าจะเข้าใจลึกซึ้งกว่าเดิมอย่างแน่นอน

ตัวดำเนินการทางคณิตศาสตร์ (Arithmetic Operators) ใน MicroPython บน ESP32

ตัวดำเนินการทางคณิตศาสตร์ (Arithmetic Operators) ใน MicroPython บน ESP32 เป็นพื้นฐานสำคัญสำหรับการพัฒนาโปรแกรมในระบบฝังตัว เนื่องจากทุกระบบอิเล็กทรอนิกส์ต้องอาศัยการคำนวณตัวเลขในรูปแบบต่าง ๆ อยู่เสมอ ไม่ว่าจะเป็นการนับจำนวนครั้งของเหตุการณ์ เช่น การกดปุ่ม, การคำนวณเวลาเปิดปิดของอุปกรณ์, หรือแม้แต่การคำนวณค่าทางฟิสิกส์และวิศวกรรมที่ได้จากการอ่านข้อมูลจากเซ็นเซอร์ พื้นฐานทั้งหมดนี้เกิดขึ้นจากการใช้งานตัวดำเนินการทางคณิตศาสตร์ที่มีให้เลือกใช้อย่างครบถ้วนใน MicroPython

ในสถาปัตยกรรมของ ESP32 การดำเนินการคำนวณทั้งหมดจะถูกส่งต่อไปยัง Arithmetic Logic Unit (ALU) ซึ่งเป็นหน่วยประมวลผลทางคณิตศาสตร์ที่ทำงานได้อย่างรวดเร็วและแม่นยำ ตัวดำเนินการกลุ่มนี้จึงมีความสำคัญต่อทั้งประสิทธิภาพของระบบและความถูกต้องในการประมวลผลข้อมูล

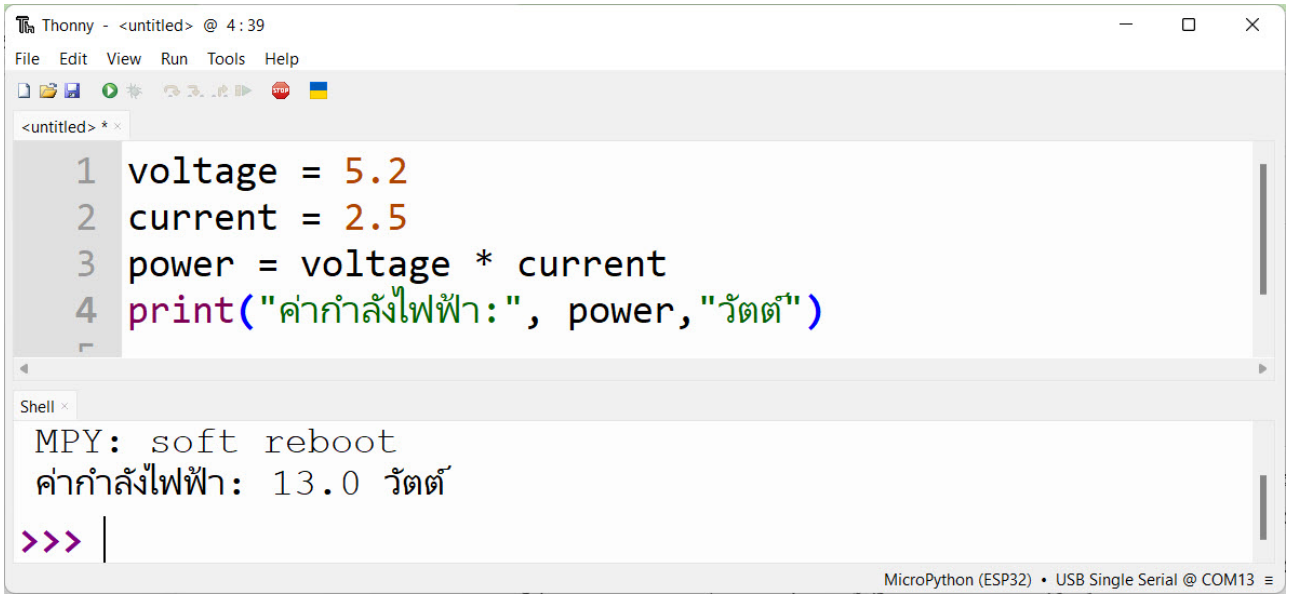
ตัวดำเนินการทางคณิตศาสตร์ใน MicroPython ประกอบไปด้วยเครื่องหมายที่คุ้นเคย ได้แก่ + สำหรับการบวก, - สำหรับการลบ, * สำหรับการคูณ, / สำหรับการหารแบบมีทศนิยม, // สำหรับการหารแบบจำนวนเต็ม, % สำหรับการหารเอาเศษ และ ** สำหรับการยกกำลัง ตัวดำเนินการเหล่านี้สามารถใช้ได้กับทั้งข้อมูลชนิดจำนวนเต็ม (int) และจำนวนทศนิยม (float) โดย MicroPython จะเลือกวิธีคำนวณและส่งคืนค่าที่ถูกต้องโดยอัตโนมัติ

ตารางต่อไปนี้สรุปความหมายและตัวอย่างของตัวดำเนินการทางคณิตศาสตร์ใน MicroPython ที่ใช้งานจริง

สัญลักษณ์	ความหมาย	ตัวอย่าง	ผลลัพธ์
+	บวก	7 + 3	10
-	ลบ	7 - 3	4
*	คูณ	7 * 3	21
/	หารทศนิยม	7 / 3	2.333...
//	หารจำนวนเต็ม	7 // 3	2
%	หารเอาเศษ	7 % 3	1
**	ยกกำลัง	2 ** 3	8

การเลือกใช้ตัวดำเนินการที่เหมาะสมกับงานแต่ละประเภทมีความสำคัญอย่างมาก เช่น การใช้ / จะให้ผลลัพธ์เป็นทศนิยม แม้ว่าตัวเลขต้นทางจะเป็นจำนวนเต็ม ในขณะที่ // จะปัดเศษลงให้เหลือจำนวนเต็มทันที ซึ่งมีประโยชน์มากสำหรับการแบ่งกลุ่มข้อมูล การกำหนดรอบของอุปกรณ์ หรือการควบคุมลอจิกในระบบฝังตัว ตัวอย่างต่อไปนี้แสดงการใช้งานตัวดำเนินการทางคณิตศาสตร์ในโปรแกรม MicroPython ที่สามารถนำไปทดลองบน ESP32 ได้ทันที

สำหรับการประยุกต์ใช้งานกับข้อมูลจากเซ็นเซอร์หรืออุปกรณ์จริง ตัวอย่างต่อไปนี้แสดงการคำนวณค่ากำลังไฟฟ้าจากแรงดันและกระแสที่วัดได้



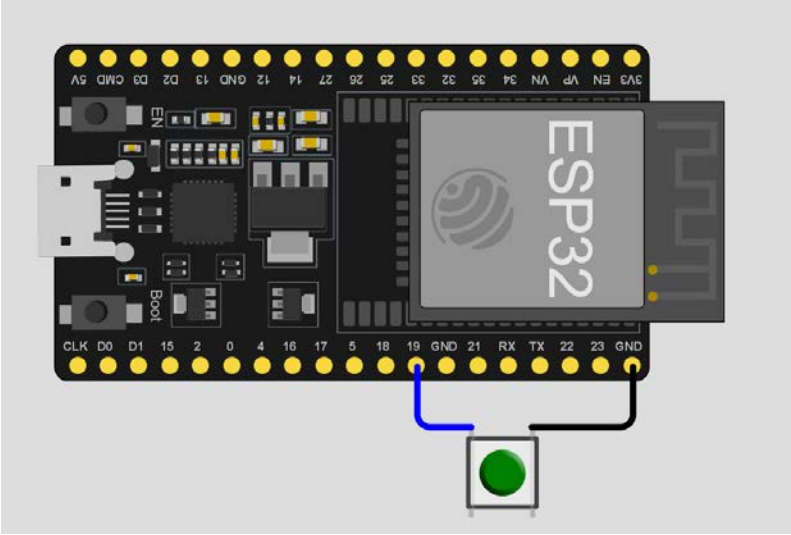
ตัวแปร voltage, current และ power ในตัวอย่างนี้เป็นชนิดข้อมูล float ทั้งหมด ซึ่งสามารถคำนวณค่าทางฟิสิกส์ได้อย่างถูกต้อง และสามารถนำแนวคิดเดียวกันไปใช้ในงานควบคุมและตรวจสอบอุปกรณ์อิเล็กทรอนิกส์อื่น ๆ ได้อย่างยืดหยุ่น

สรุปได้ว่าการเข้าใจและใช้งานตัวดำเนินการทางคณิตศาสตร์ใน MicroPython อย่างถูกต้อง เป็นรากฐานของการเขียนโปรแกรมบน ESP32 ที่สามารถประยุกต์ใช้ในงานคำนวณทางฟิสิกส์ วิศวกรรม และการควบคุมฮาร์ดแวร์ทุกประเภทในโลกของระบบฝังตัว

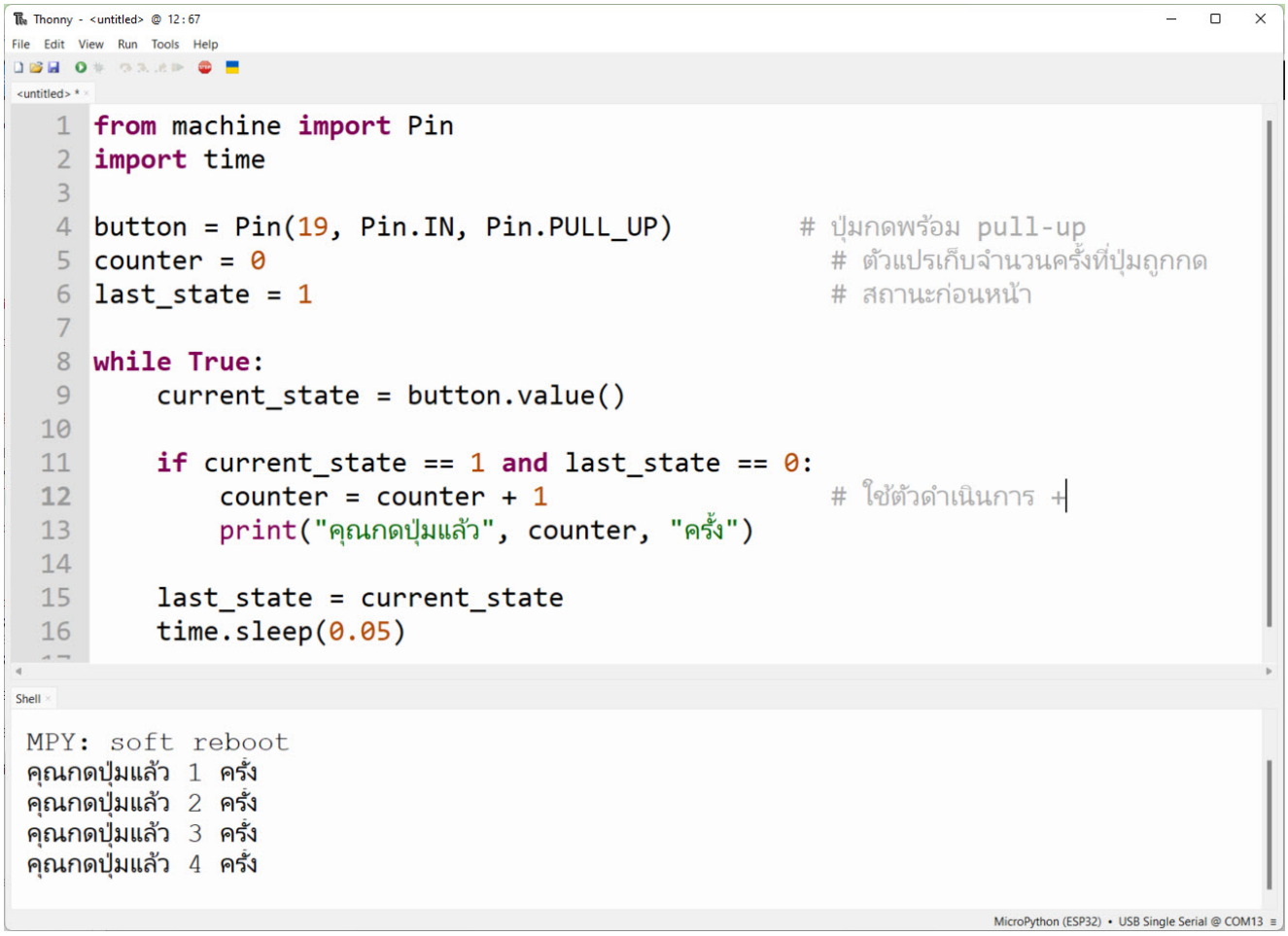
ตัวอย่างโปรเจกต์ เพิ่มเลขนับทุกครั้งที่ปุ่มถูกกด และแสดงผลบนหน้าจอ Serial

อุปกรณ์ที่ใช้: บอร์ด ESP32 ,ปุ่มกด 1 ตัว , สาย USB ต้องเสียบบอร์ดพร้อมทดลองบน Thonny

วิธีต่อวงจร: เราจะใช้ขา 19 เป็น INPUT เปิดใช้งาน R PullUp ภายใน



โค้ดที่ใช้ได้จริงกับ Thonny IDE:



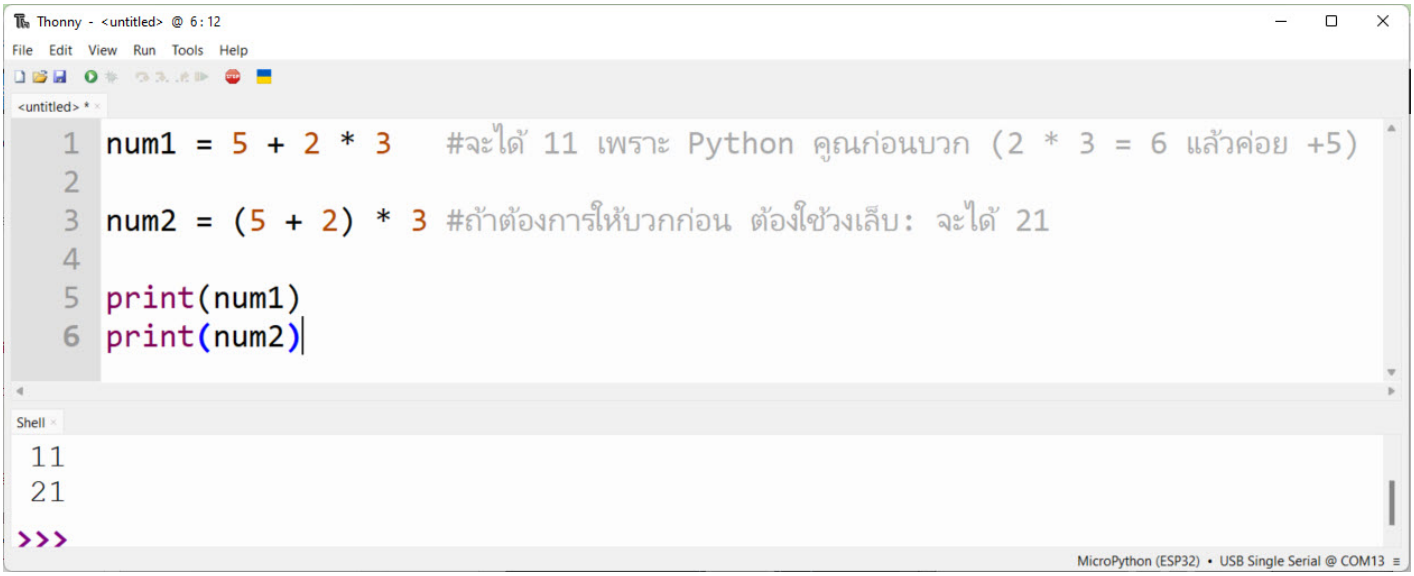
ในบรรทัด counter = counter + 1 มีการใช้ตัวดำเนินการ + ซึ่งแม้จะดูธรรมดา แต่ในบริบทของ MicroPython และ ESP32 มันคือการ “สร้างระบบที่นับได้” ว่ามีเหตุการณ์เกิดขึ้นกี่ครั้งแล้ว และหากศึกษาต่อไปเรื่อย ๆ ตัวเลขนี้อาจกลายเป็นพื้นฐานของโปรเจกต์อื่น ๆ ได้อีกมากมาย เช่น:

- เปิดไฟเมื่อกดครบ 5 ครั้ง
- เปิดเสียงเตือนเมื่อเกิน 10 ครั้ง
- บันทึกจำนวนคนเดินผ่านจุดตรวจจับ (ถ้าเปลี่ยนจากปุ่มเป็นเซ็นเซอร์)

นี่คือตัวอย่างเบื้องต้นของการใช้งานตัวดำเนินการคณิตศาสตร์ในงานจริง

📘 เสริมความเข้าใจ: ลำดับความสำคัญของการคำนวณ

ตัวดำเนินการในภาษา MicroPython มีลำดับความสำคัญ (Operator Precedence) เช่นเดียวกับในวิชาคณิตศาสตร์ ตัวอย่างเช่น:



การเข้าใจลำดับเหล่านี้สำคัญมากเมื่อเราต้องเขียนนิพจน์ซับซ้อนที่เกี่ยวข้องกับเวลา, ค่าแรงดัน, การปรับระดับ PWM

ตัวดำเนินการทางคณิตศาสตร์ไม่ใช่แค่พื้นฐาน แต่คือหัวใจสำคัญของการเขียนโปรแกรมสำหรับระบบสมองกลฝังตัว ถ้าเข้าใจเรื่องตัวแปร และตัวดำเนินการ รวมไปถึงคำสั่งควบคุมการทำงาน เราก็จะสามารถควบคุมข้อมูลและสั่งงานอุปกรณ์ต่าง ๆ ได้อย่างถูกต้องและมีประสิทธิภาพ

ทุกครั้งที่เราต้องควบคุมพฤติกรรมของฮาร์ดแวร์จากข้อมูล เช่น “ปุ่มถูกกดกี่ครั้งแล้ว”, “ดินมีความชื้นไหม”, “มอเตอร์หมุนเร็วแค่ไหน” ต่าง ๆ เหล่านี้ เราต้องใช้การคำนวณด้วยตัวดำเนินการทางคณิตศาสตร์ทั้งสิ้น

MicroPython ช่วยให้เราสามารถเขียนนิพจน์แบบเดียวกับที่เราใช้ในคณิตศาสตร์ได้ทันทีบน ESP32 โดยไม่ต้องเรียนรู้ภาษาแปลกใหม่ และสามารถทดสอบผลได้จริงผ่าน Serial Monitor ของ Thonny จึงเหมาะอย่างยิ่งสำหรับผู้เริ่มต้นที่อยากเรียนรู้วิธีควบคุมอุปกรณ์เครื่องใช้ไฟฟ้าต่าง ๆ ผ่านการโปรแกรมด้วยตัวเราเอง

ตัวดำเนินการเปรียบเทียบ (Comparison Operators) ใน MicroPython บน ESP32

ตัวดำเนินการเปรียบเทียบ (Comparison Operators) ใน MicroPython บน ESP32 เป็นกลุ่มของเครื่องมือพื้นฐานที่ช่วยให้โปรแกรมสามารถเปรียบเทียบค่าระหว่างตัวแปรหรือค่าคงที่ได้โดยตรง ซึ่งถือเป็นหัวใจสำคัญของการตัดสินใจ (Decision Making) และการสร้างเงื่อนไขในโปรแกรมทุกประเภท ไม่ว่าจะเป็นการตรวจสอบสถานะของเซ็นเซอร์ การควบคุมอุปกรณ์ การวนลูป หรือการตรวจเช็กข้อมูลที่ได้รับจากภายนอก

ตัวดำเนินการเปรียบเทียบจะประเมินนิพจน์ทางตรรกะและคืนค่าผลลัพธ์เป็น Boolean คือ True (จริง) หรือ False (เท็จ) เท่านั้น ซึ่งเหมาะกับการนำไปใช้ในคำสั่ง if, while หรือใช้ในการกำหนดเงื่อนไขของระบบอัตโนมัติต่าง ๆ เช่น การสั่งเปิดพัดลมเมื่ออุณหภูมิสูงกว่าเกณฑ์ หรือการแจ้งเตือนเมื่อค่าความชื้นน้อยกว่ากำหนด

ตัวดำเนินการเปรียบเทียบที่ MicroPython บน ESP32 รองรับ มีดังนี้:

ตัวดำเนินการ	ความหมาย	ตัวอย่าง	ผลลัพธ์
==	เท่ากับ	x == y	True หรือ False
!=	ไม่เท่ากับ	x != y	True หรือ False
>	มากกว่า	x > y	True หรือ False
<	น้อยกว่า	x < y	True หรือ False
>=	มากกว่าหรือเท่ากับ	x >= y	True หรือ False
<=	น้อยกว่าหรือเท่ากับ	x <= y	True หรือ False

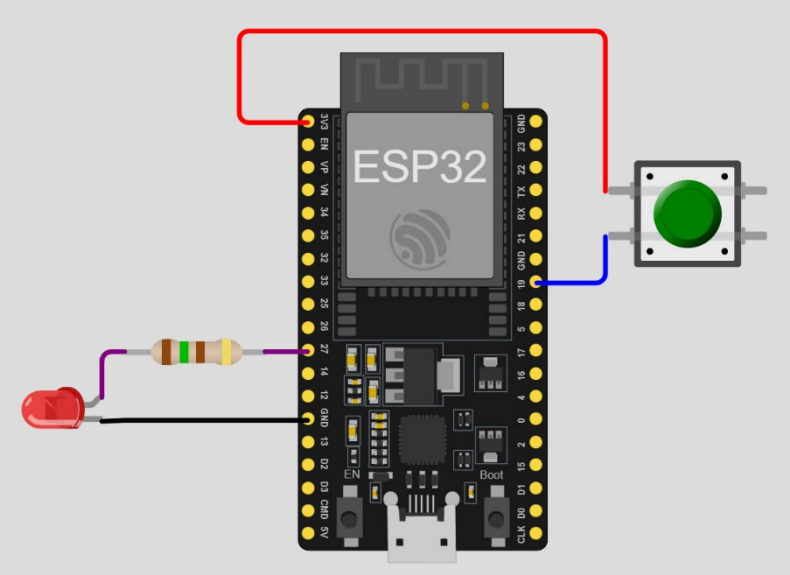
การเปรียบเทียบนี้ใช้งานได้กับทั้งตัวเลขจำนวนเต็ม (int), จำนวนทศนิยม (float), ข้อความ (str) และแม้แต่ข้อมูลชนิดอื่น ๆ ที่อยู่ในตัวแปรก็สามารถนำมาเปรียบเทียบได้ทั้งหมด เช่น การตรวจสอบค่าจากเซ็นเซอร์ หรือสถานะของอุปกรณ์ในระบบ โดยเมื่อเปรียบได้ผลลัพธ์แล้ว ทุกตัวดำเนินการข้างต้นจะคืนค่าเป็น Boolean เท่านั้น ซึ่งมีแค่ True (จริง) หรือ False (เท็จ)

ซึ่งค่าทั้งสองนี้จะเป็นตัวกำหนดว่า ESP32 จะ “ทำ” หรือ “ไม่ทำ” บางอย่างตามคำสั่ง if, while, หรือ for

ตัวอย่างโปรเจกต์: เปิด LED เมื่อกดปุ่ม และปิด LED เมื่อปล่อยปุ่ม

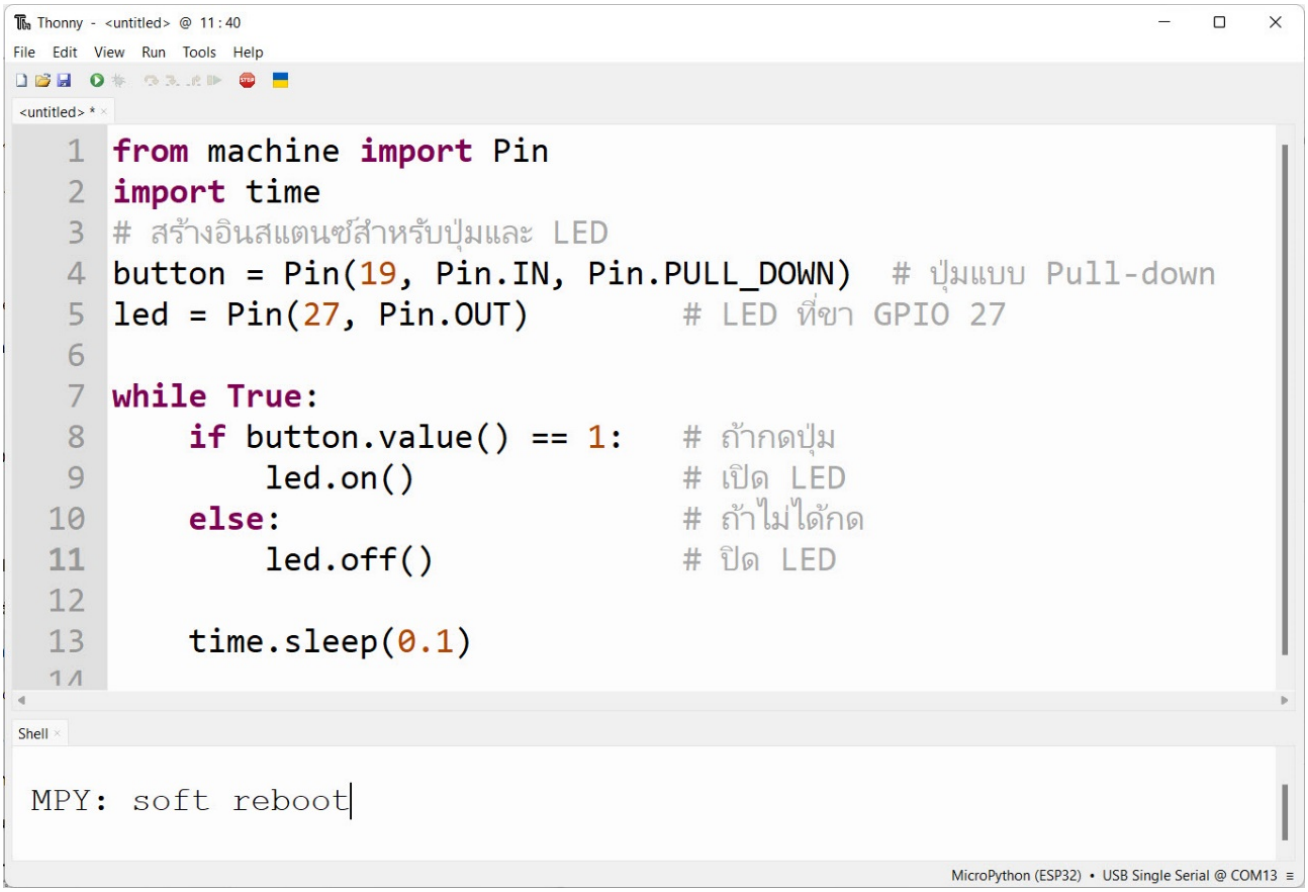
อุปกรณ์ที่ใช้:

- ESP32
- ปุ่มกด 1 ตัว (GPIO 19)
- LED พร้อม R ต่ออนุกรม (GPIO 27)
- สาย USB
- Thonny IDE



การต่อวงจร:

โค้ดที่รันได้จริงใน Thonny:



ขออธิบายเพิ่มเติมเล็กน้อย เพื่อเพิ่มความเข้าใจ

คำสั่ง **if button.value() == 1** เป็นนิพจน์เปรียบเทียบที่ตรวจสอบว่า

"ค่าที่อ่านจากปุ่ม มีค่าเท่ากับ 1 หรือไม่?"

- ถ้าเป็นจริง **(True)** แสดงว่า **ผู้ใช้ได้กดปุ่มลงไป** หรือ **ปุ่มถูกกด** นั่นเอง
- ถ้าเป็นเท็จ **(False)** แสดงว่า **ผู้ใช้ยังไม่ได้กดปุ่ม+**

นี่คือตัวอย่างการใช้ตัวดำเนินการเปรียบเทียบ **==** (เท่ากับ) ซึ่งเป็นตัวดำเนินการที่พบได้บ่อยที่สุด และมีความสำคัญอย่างยิ่งในการควบคุมพฤติกรรมของอุปกรณ์ต่าง ๆ

ในขณะเดียวกัน ถ้าเราอยากให้ LED ติดในขณะที่ยังไม่ได้กดปุ่ม พอกดปุ่มแล้วให้ LED ดับ ก็สามารถทำได้โดยการเปลี่ยนโค้ดบรรทัดที่ 8 ซึ่งเป็นการดำเนินการเปรียบเทียบ ใช้ตัวดำเนินการ **!=** (ไม่เท่ากับ)

if button.value() != 1:

จริง ๆ แล้วการสั่งให้กลับสถานะการทำงานหรือกลับสถานะการตรวจสอบนั้นสามารถทำได้หลายวิธี เขียนโค้ดได้หลากหลายรูปแบบขึ้นอยู่กับความรู้ความเข้าใจของผู้เขียนโปรแกรมนั้น แต่ตัวอย่างที่แสดงให้ดูนี้ เป็นตัวอย่างการใช้งานตัวดำเนินการเปรียบเทียบ จึงแสดงให้ดูว่าหากเราเปลี่ยนการเปรียบเทียบ ผลลัพธ์ย่อมแตกต่างออกไป และในที่นี้ เราเปลี่ยนจาก **==** เป็น **!=** ผลที่ออกมาจึงต่างกันแบบตรงกันข้าม

เสริมความเข้าใจสำหรับผู้เริ่มต้น:

ตัวดำเนินการเปรียบเทียบเหล่านี้ ไม่ได้เป็นเพียงเครื่องมือสำหรับคณิตศาสตร์เท่านั้น แต่เป็น หัวใจของการตัดสินใจในระบบอิเล็กทรอนิกส์อัตโนมัติ ยกตัวอย่างเช่น:

- ตรวจสอบว่าปุ่มถูกกดหรือปล่อยอยู่? (if button.value() == 1)
- ตรวจสอบว่าสวิตช์วงจรอยู่ในตำแหน่งเปิดหรือปิด?
- เช็คว่าระดับน้ำในถังถึงจุดสูงสุดหรือยัง (เช่น if water_level >= max_level)
- ตรวจสอบว่ามีสัญญาณอินพุตจากอุปกรณ์เซ็นเซอร์หรือไม่?

ทั้งหมดนี้ล้วนเกิดจากการดำเนินการเปรียบเทียบด้วยคำสั่ง **if** ซึ่งช่วยให้ระบบสามารถประมวลผลและตอบสนองต่อสภาวะแวดล้อมได้อย่างเหมาะสม แม้แต่การสร้างระบบเปิด-ปิดประตูอัตโนมัติ ก็ยังต้องอาศัยการเปรียบเทียบสถานะของเซ็นเซอร์และปุ่มกดร่วมกันเสมอ และอาจกล่าวได้ว่า :

"ทุกระบบสมองกลฝังตัว ต่างต้องพึ่งพาตัวดำเนินการเปรียบเทียบในการตัดสินใจทำงานเสมอ"

สรุปเรื่องตัวดำเนินการเปรียบเทียบ

ตัวดำเนินการเปรียบเทียบใน MicroPython คือเครื่องมือสำคัญที่ทำให้ ESP32 ดูเหมือนว่ามี "สมอง" ในการประเมินสถานการณ์ต่าง ๆ รอบตัว เช่น เมื่อเขียนคำสั่ง **x > 100** หรือ **button.value() == 1** แท้จริงแล้วเรากำลังเขียนเงื่อนไขกำกับให้ ESP32 ตัดสินใจทำงานตอบสนองอย่างมีตรรกะต่อสิ่งแวดล้อม ซึ่งสิ่งเหล่านี้คือรากฐานสำคัญของการพัฒนาระบบอัตโนมัติอัจฉริยะในโลกของไมโครคอนโทรลเลอร์

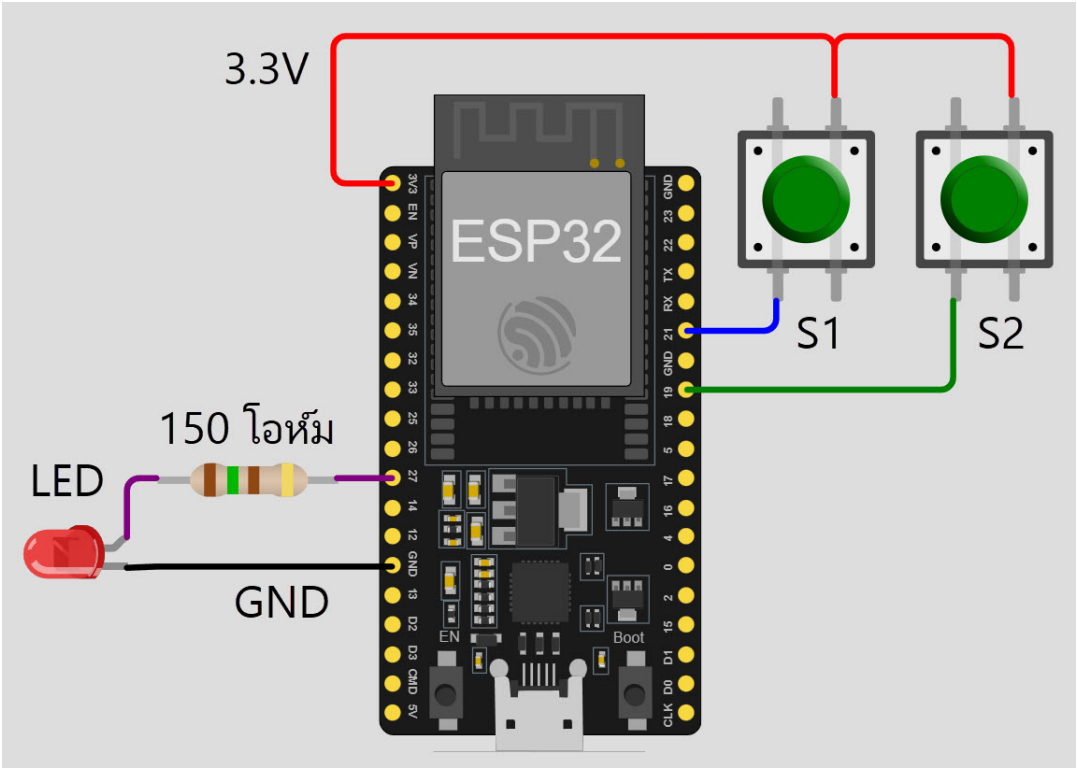
ตัวดำเนินการตรรกะ (Logical Operators) ใน MicroPython บน ESP32

ในการเขียนโปรแกรมสำหรับระบบสมองกลฝังตัว เช่น ESP32 ด้วยภาษา MicroPython “ตัวดำเนินการตรรกะ” หรือ Logical Operators ถือเป็นหัวใจสำคัญที่ช่วยให้โปรแกรมสามารถตัดสินใจจากสถานการณ์ที่มีหลายเงื่อนไขพร้อมกัน ตัวดำเนินการตรรกะเป็นเครื่องมือที่ใช้กำกับตรรกะหรือความสัมพันธ์ระหว่างเงื่อนไขสองเงื่อนไขขึ้นไป เพื่อควบคุมการทำงานของอุปกรณ์หรือระบบอัตโนมัติในชีวิตจริง

โดยตัวดำเนินการตรรกะที่ใช้บ่อยมีอยู่ 3 ประเภทหลัก ได้แก่

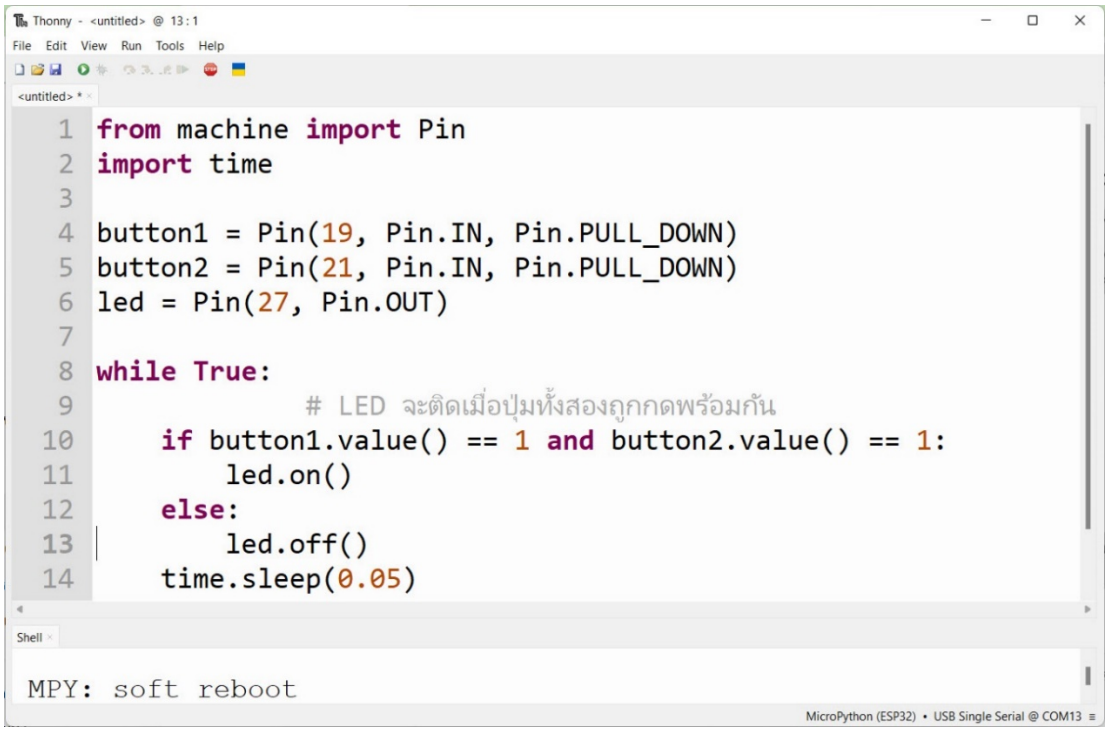
1. and (และ): ให้ค่าผลลัพธ์เป็นจริง (True) เมื่อทุกเงื่อนไขเป็นจริง
2. or (หรือ): ให้ค่าผลลัพธ์เป็นจริง เมื่อมีเงื่อนไขอย่างน้อยหนึ่งข้อเป็นจริง
3. not (ไม่): เปลี่ยนค่าความจริงของเงื่อนไข จากจริงเป็นเท็จ หรือจากเท็จเป็นจริง

ตัวอย่างการใช้งานกับปุ่มกด 2 ปุ่มและ LED



สมมติว่าต้องการให้ระบบ ESP32 ตรวจสอบการกดปุ่มกด 2 ปุ่ม (เชื่อมต่อที่ขา GPIO 19 และ GPIO 21) และให้ LED (เชื่อมต่อที่ขา GPIO 27) ติดขึ้น **ก็ต่อเมื่อ** ผู้ใช้กดปุ่มทั้งสองปุ่มพร้อมกันเท่านั้น หากกดเพียงปุ่มเดียวหรือไม่กดเลย LED จะดับทันที

ตัวอย่างนี้สะท้อนให้เห็นการใช้ตัวดำเนินการตรรกะ **and** เพื่อควบคุมเงื่อนไขสองเงื่อนไขพร้อมกัน

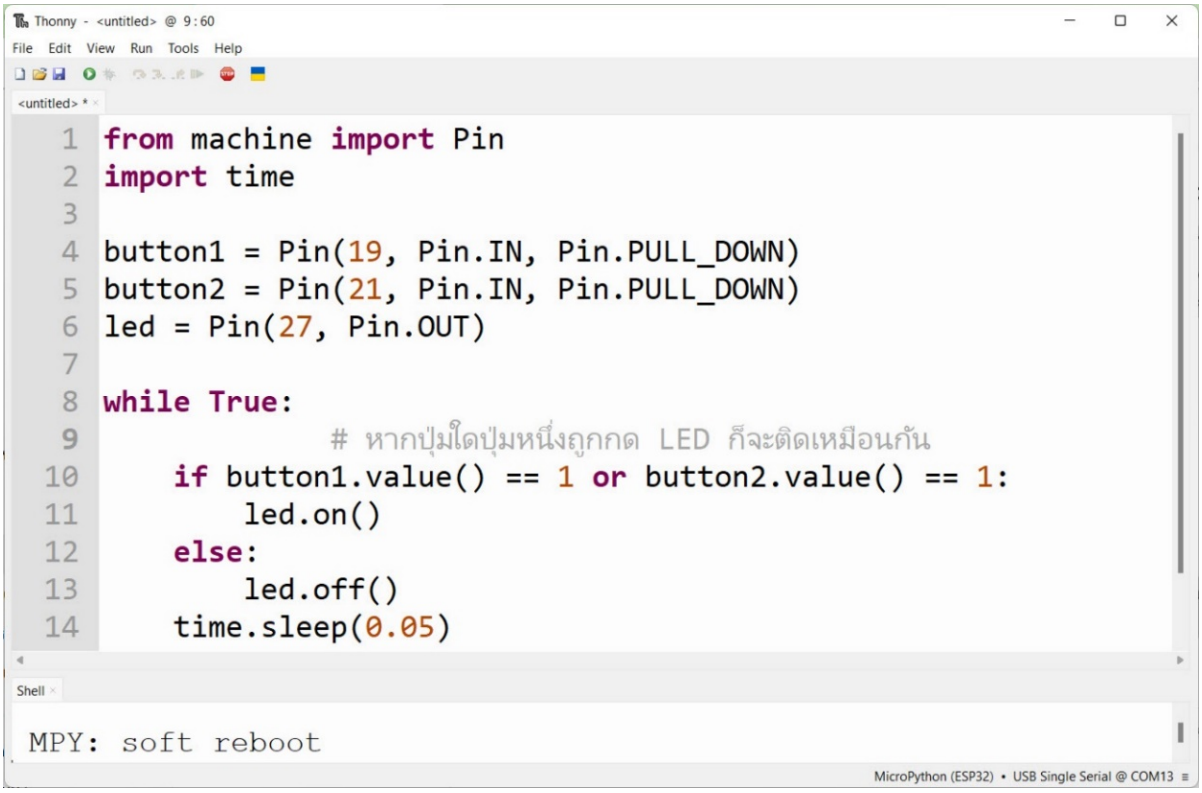


LED จะติดขึ้นทันทีหากผู้ใช้กดทั้งสองปุ่ม หากปล่อยปุ่มใดปุ่มหนึ่ง LED จะดับทันที

หลักการนี้ใช้ตัวดำเนินการตรรกะ **and** เพื่อให้ทั้งสองเงื่อนไขต้องเป็นจริงพร้อมกัน จึงจะมีผลต่อการตัดสินใจ

การขยายแนวคิด: เปลี่ยนเป็นการใช้ or หรือ not

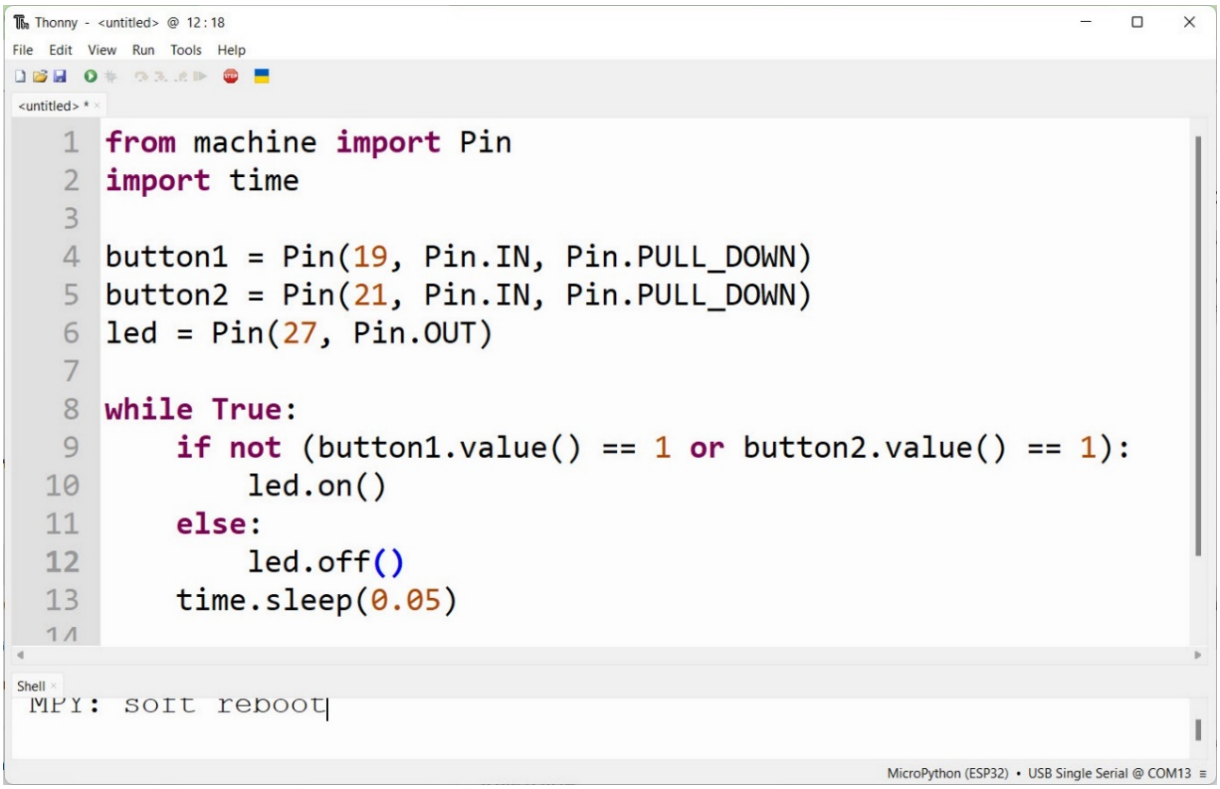
ถ้าต้องการให้ **LED ติดขึ้นเมื่อปุ่มใดปุ่มหนึ่งถูกกด** (ไม่จำเป็นต้องกดพร้อมกัน) สามารถใช้ตัวดำเนินการตรรกะ or เราสามารถทำได้โดยการเปลี่ยนโค้ดบรรทัดที่ 10 เพียงบรรทัดเดียว ดังตัวอย่าง



ในกรณีนี้ LED จะติดเมื่อกดปุ่มอย่างน้อยหนึ่งปุ่ม หากไม่กดปุ่มใดเลย LED จะดับทันที

นี่คือตัวอย่างการใช้ **or** เพื่อสร้างเงื่อนไข “อย่างน้อยหนึ่งข้อเป็นจริง”

หากต้องการให้ **LED ติดในขณะที่ไม่มีปุ่มใดถูกกด** และ LED ดับทันทีที่มีการกดปุ่มใดปุ่มหนึ่ง สามารถใช้ not ประกอบกับ or ดังนี้

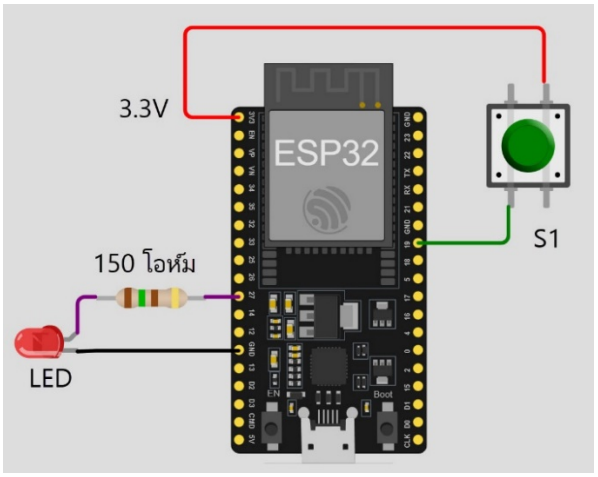


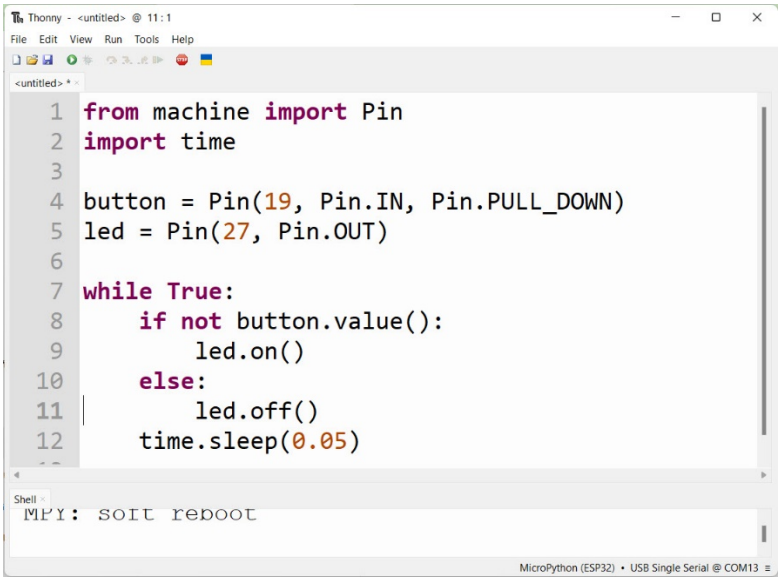
ในตัวอย่างนี้จะเห็นว่า LED จะติดอยู่ตลอด หากไม่กดปุ่มใดเลย เมื่อกดปุ่มใดปุ่มหนึ่ง (หรือทั้งสองปุ่มพร้อมกัน) LED จะดับทันที

ตัวดำเนินการ not (ไม่)

ตัวดำเนินการ not ใช้สำหรับ “กลับค่าความจริง” ของเงื่อนไข เช่น ถ้าเงื่อนไขเดิมเป็นจริง (True) เมื่อใช้ not จะกลายเป็นเท็จ (False) และถ้าเดิมเป็นเท็จ (False) จะกลายเป็นจริง (True) การใช้ not มีประโยชน์ในกรณีที่ต้องการตรวจสอบสถานะ “ตรงข้าม” กับที่อ่านได้จากอินพุต

ตัวอย่าง: ต้องการให้ LED ติดเมื่อยังไม่ได้กดปุ่ม





ในตัวอย่างนี้ LED จะติดเมื่อ “ไม่ได้กดปุ่ม” และจะดับเมื่อ “กดปุ่ม” ซึ่งแสดงถึงการนำ not มาใช้กับงานควบคุมที่ต้องการเงื่อนไขกลับทาง ตัวดำเนินการ not ไม่ได้ใช้บ่อยเท่ากับ and และ or แต่ถ้าเข้าใจหลักการใช้งาน ก็จะช่วยให้โปรแกรมอ่านง่ายและยืดหยุ่นมากขึ้น

การประยุกต์ใช้งานจริง

ตัวดำเนินการตรรกะเช่นนี้ไม่เพียงใช้กับปุ่มกดและ LED เท่านั้น แต่ยังเป็นแกนหลักของระบบควบคุมอัตโนมัติแทบทุกแบบในชีวิตประจำวัน เช่น ระบบเตือนภัยบ้านอัจฉริยะที่ต้องตรวจสอบหลายเซ็นเซอร์ร่วมกันก่อนแจ้งเตือน ระบบสมาร์ทโฮมที่ต้องตรวจเช็กเงื่อนไขซ้อนกันหลายชั้นก่อนเปิดหรือปิดอุปกรณ์อัตโนมัติ เมื่อเข้าใจการใช้ตัวดำเนินการตรรกะ ผู้อ่านจะสามารถออกแบบโปรแกรมควบคุมอุปกรณ์หรือระบบที่ต้อง “คิดเป็นขั้นเป็นตอน” และตอบสนองได้อย่างแม่นยำตามสถานการณ์จริง

ข้อควรระวังในการทดลองจริง

หากผู้อ่านต้องการการแสดงผลที่เป็นการ print ค่าข้อมูลออกมาแสดงผ่าน Serial ควรคำนึงถึงโปรแกรมที่ออกแบบมาอย่างเหมาะสมไม่ควรสั่ง print ข้อมูลทุก ๆ รอบใน loop โดยไม่จำเป็น เพราะจะทำให้ข้อความใน Serial Monitor ไหลเร็วมากจนอ่านไม่ออก หรืออาจไม่เกิดประโยชน์ การแสดงผลควรทำเฉพาะเมื่อมีเหตุการณ์สำคัญ เช่น มีการเปลี่ยนสถานะของปุ่ม หรือจำนวนครั้งที่กดเปลี่ยนไปจริง ๆ แต่หากเป็นการทดลองเบื้องต้นเพื่อดูการทำงานของ LED และปุ่มกด สามารถใช้โค้ดข้างต้นได้โดยไม่ต้องสั่ง print เลย ให้ผู้ใช้สังเกตผลลัพธ์จากการติดดับของ LED ได้โดยตรง

การเรียนรู้และเข้าใจตัวดำเนินการตรรกะใน MicroPython คือรากฐานของการออกแบบโปรแกรมระบบสมองกลฝังตัวทุกรูปแบบ หากสามารถนำไปประยุกต์ใช้กับสถานการณ์จริง ไม่ว่าจะเป็นการควบคุมอุปกรณ์ง่าย ๆ อย่าง LED หรือระบบซับซ้อนอย่าง Smart Home และ IoT ผู้อ่านก็จะสามารถต่อยอดความรู้สู่การพัฒนานวัตกรรมอัตโนมัติที่ตอบสนองต่อสภาพแวดล้อมได้อย่างชาญฉลาดและแม่นยำ

ตัวดำเนินการกำหนดค่า (Assignment Operators) ใน MicroPython บน ESP32

เมื่อเริ่มต้นเขียนโปรแกรมสำหรับระบบสมองกลฝังตัว เช่น ESP32 ด้วยภาษา MicroPython หนึ่งในหัวใจสำคัญที่ขาดไม่ได้เลยก็คือ “ตัวดำเนินการกำหนดค่า” หรือ Assignment Operators กลุ่มตัวดำเนินการเหล่านี้เป็นกลไกพื้นฐานที่ทำให้ไมโครคอนโทรลเลอร์สามารถจดจำ บันทึก และปรับเปลี่ยนข้อมูลต่าง ๆ ได้อย่างต่อเนื่อง ไม่ว่าจะเป็นการตั้งค่าเริ่มต้น การเก็บผลลัพธ์ที่ได้จากการประมวลผล การสะสมข้อมูลเหตุการณ์ หรือการคำนวณระหว่างทาง ทุกกระบวนการล้วนเริ่มต้นและจบลงด้วยการ “ให้ค่า” หรือ “ปรับค่า” ในตัวแปรทั้งสิ้น

ลองนึกภาพว่าหากไม่มีตัวดำเนินการกำหนดค่า โปรแกรมจะไม่สามารถบันทึกสถานะปัจจุบันหรือเหตุการณ์ที่เกิดขึ้นได้เลย ทุกครั้งที่ไมโครคอนโทรลเลอร์อ่านค่าจากเซ็นเซอร์ กดปุ่มสั่งงาน หรือประมวลผลสัญญาณอินพุตใด ๆ จะต้องมีการ “อัปเดตค่า” ในตัวแปรที่ใช้ติดตามและควบคุมการทำงาน หากไม่มีการกำหนดค่าหรืออัปเดตข้อมูล ระบบก็จะไร้ความสามารถในการคิด วิเคราะห์ ตัดสินใจ จดจำสถานะ หรือข้อมูลที่เกิดขึ้นระหว่างการทำงาน

ตัวดำเนินการกลุ่มนี้มีหน้าที่รับผิดชอบทั้งในแง่การมอบค่าใหม่ การปรับปรุงค่าเดิม และการคำนวณข้อมูลสะสมในระบบอัตโนมัติ เช่น การนับจำนวนครั้งที่เกิดเหตุการณ์ การสะสมผลรวมค่าจากเซ็นเซอร์ การรีเซ็ตค่าสถานะ หรือแม้แต่การลบ คูณ หาร และยกกำลังค่าต่าง ๆ เพื่อประมวลผลที่ซับซ้อนยิ่งขึ้น

โดยสรุปแล้ว Assignment Operators คือเครื่องมือที่ทำให้โปรแกรมบน ESP32 “คิดเป็นขั้นเป็นตอน” และ “จดจำสิ่งที่เกิดขึ้นในอดีต” ได้อย่างต่อเนื่อง เป็นรากฐานที่สำคัญต่อการสร้างระบบอัตโนมัติที่ฉลาดและ

ยืดหยุ่นในยุค IoT ทุกกระบวนการ ตั้งแต่โปรเจกต์ทดลองพื้นฐาน ไปจนถึงระบบควบคุมอัตโนมัติในอุตสาหกรรมล้วนต้องอาศัยการกำหนดค่าให้กับตัวแปรทั้งสิ้น

ประเภทของตัวดำเนินการกำหนดค่าที่ใช้บ่อยใน MicroPython

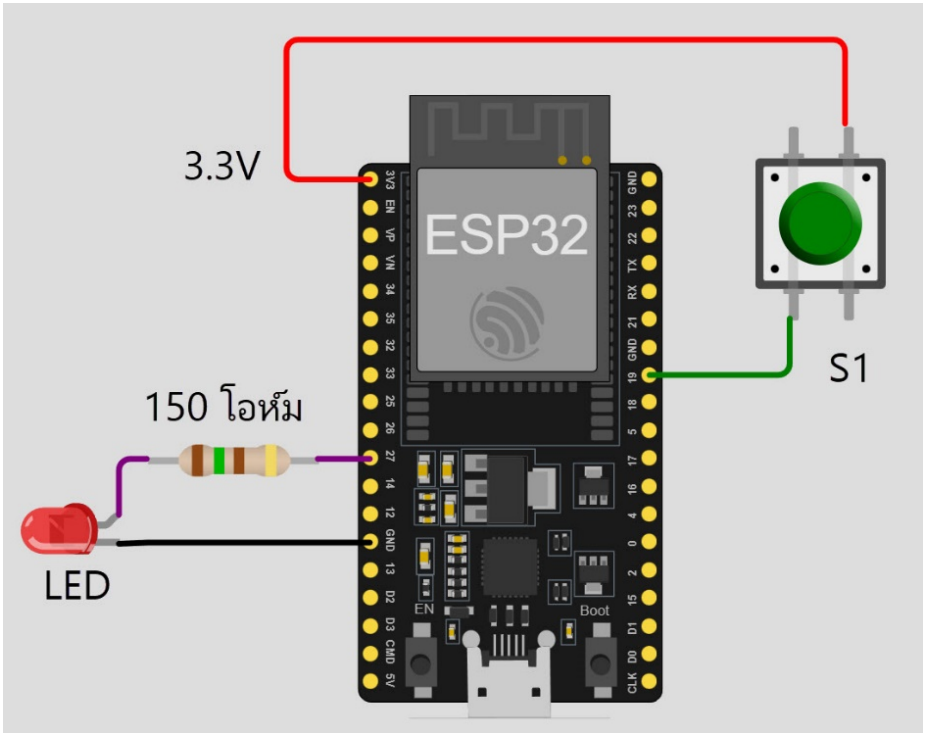
ตัวดำเนินการกำหนดค่ามีหลากหลายแบบ ซึ่งแต่ละแบบจะช่วยให้การอัปเดตค่าในตัวแปรมีความยืดหยุ่นและสะดวกมากขึ้น ตัวอย่างเช่น

ตารางสรุป Assignment Operators ที่ใช้ใน MicroPython

ตัวดำเนินการ	ตัวอย่างการใช้	ความหมายแบบเข้าใจง่าย
=	X = 5	กำหนดค่า 5 ให้กับตัวแปร x
+=	X += 2	บวก 2 เข้าไปกับค่าเดิมในตัวแปร x แล้วเก็บผลใหม่กลับเข้า x
-=	X -= 3	ลบ 3 ออกจากค่าเดิมในตัวแปร x แล้วเก็บผลใหม่กลับเข้า x
*=	X *= 4	คูณ 4 กับค่าเดิมในตัวแปร x แล้วเก็บผลใหม่กลับเข้า x
/=	X /= 5	หารค่าเดิมในตัวแปร x ด้วย 5 แล้วเก็บผลใหม่กลับเข้า x
%=	X %= 2	หารเอาเศษของค่าเดิมในตัวแปร x ด้วย 2 แล้วเก็บผลใหม่กลับเข้า x
**=	X **= 3	ยกกำลังสามกับค่าเดิมในตัวแปร x แล้วเก็บผลใหม่กลับเข้า x
//=	X //= 2	หารปัดเศษของค่าเดิมในตัวแปร x ด้วย 2 แล้วเก็บผลใหม่กลับเข้า x

ตัวอย่างการนำไปใช้ในโปรแกรม MicroPython จริง

ตัวอย่างที่ 1: นับจำนวนการกดปุ่มและบันทึกผล



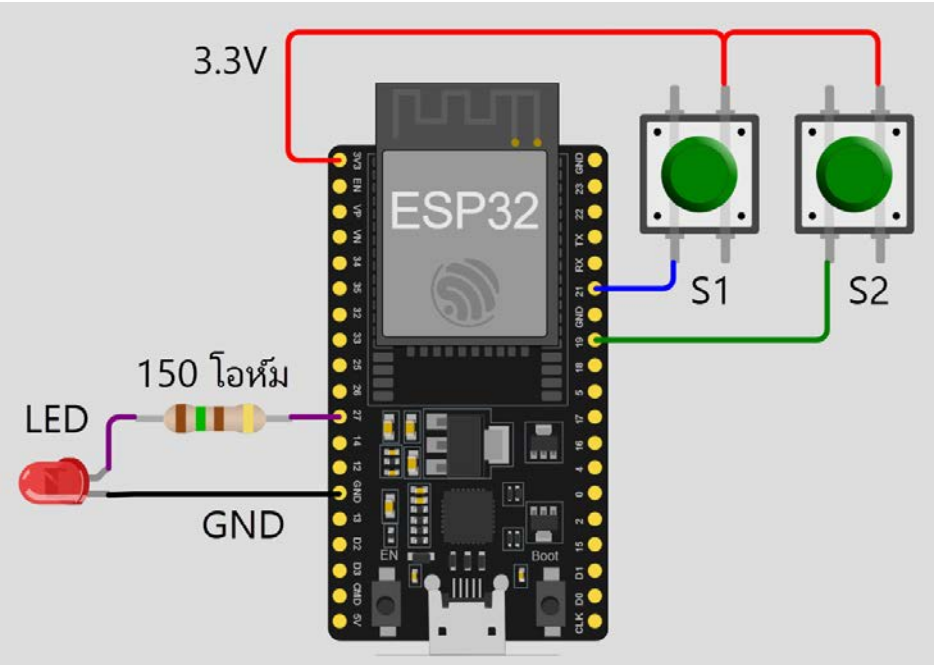
ต่อปุ่มกดเข้าที่ขา 19 และต่อ LED เข้าที่ขา 27

```
from machine import Pin; import time

button = Pin(19, Pin.IN, Pin.PULL_DOWN)  # ปุ่มกด GPIO19
led = Pin(27, Pin.OUT)                   # LED GPIO27
counter = 0; last_state = 0              # ตัวนับและสถานะก่อนหน้า

while True:
    current_state = button.value()       # อ่านสถานะปุ่ม
    if current_state == 1 and last_state == 0:  # ตรวจจับการกดใหม่
        counter += 1                     # บวกค่าเข้าไปกับค่าเดิมในตัวแปร
        print("กดปุ่มไปแล้ว", counter, "ครั้ง")  # แสดงจำนวนครั้ง
        for i in range(counter):         # กระพริบ LED ตามจำนวนที่กด
            led.on(); time.sleep(0.2)
            led.off(); time.sleep(0.2)
        while button.value() == 1: time.sleep(0.05)  # รอปล่อยปุ่ม
    last_state = current_state           # อัปเดตสถานะก่อนหน้า
    time.sleep(0.05)                     # หน่วงเวลาเล็กน้อย
```

```
MPY: soft reboot
กดปุ่มไปแล้ว 1 ครั้ง
กดปุ่มไปแล้ว 2 ครั้ง
กดปุ่มไปแล้ว 3 ครั้ง
กดปุ่มไปแล้ว 4 ครั้ง
กดปุ่มไปแล้ว 5 ครั้ง
กดปุ่มไปแล้ว 6 ครั้ง
```

ตัวอย่างนี้โปรแกรมจะนับการกดและเก็บค่าไว้ว่ากดไปกี่ครั้งแล้ว

ตัวอย่างที่ 2: ลบค่าหรือรีเซ็ตค่าในตัวแปร



หากต้องการให้มีปุ่มอีกปุ่มสำหรับลบค่าจากตัวแปร counter (เช่นใช้ -=) ก็สามารถเพิ่มเข้าไปได้ในลูป

Thonny - <untitled> @ 18 : 58

File Edit View Run Tools Help

<untitled> *

```
from machine import Pin; import time

button_plus = Pin(19, Pin.IN, Pin.PULL_DOWN)     # ปุ่มเพิ่ม (GPIO19)
button_minus = Pin(21, Pin.IN, Pin.PULL_DOWN)    # ปุ่มลด (GPIO21)
led = Pin(27, Pin.OUT)                           # LED (GPIO27)

counter = 0                                      # ตัวแปรนับจำนวน
last_plus = 0                                    # สถานะก่อนหน้าของปุ่มเพิ่ม
last_minus = 0                                   # สถานะก่อนหน้าของปุ่มลด

while True:
    plus_state = button_plus.value()             # อ่านสถานะปุ่มเพิ่ม
    minus_state = button_minus.value()           # อ่านสถานะปุ่มลด

    if plus_state == 1 and last_plus == 0:       # ตรวจจับการกดปุ่มเพิ่มใหม่
        counter += 1                             # บวกค่าเข้าไปกับค่าเดิมในตัวแปร
        print("เพิ่ม:", counter)                # แสดงค่า counter
        for i in range(counter):                 # กระพริบ LED ตามค่าปัจจุบัน
            led.on(); time.sleep(0.2)
            led.off(); time.sleep(0.2)
        while button_plus.value() == 1: time.sleep(0.05)  # รอปล่อยปุ่ม

    if minus_state == 1 and last_minus == 0:     # ตรวจจับการกดปุ่มลดใหม่
        counter -= 1                             # ลบค่าออกจากค่าเดิมในตัวแปร
        if counter < 0: counter = 0              # ไม่ให้ค่าติดลบ
        print("ลด:", counter)                    # แสดงค่า counter
        for i in range(counter):                 # กระพริบ LED ตามค่าปัจจุบัน
            led.on(); time.sleep(0.2)
            led.off(); time.sleep(0.2)
        while button_minus.value() == 1: time.sleep(0.05)  # รอปล่อยปุ่ม

    last_plus = plus_state                       # อัปเดตสถานะปุ่มเพิ่ม
    last_minus = minus_state                     # อัปเดตสถานะปุ่มลด
    time.sleep(0.05)                             # หน่วงเวลาเล็กน้อย

```

Shell

```
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
เพิ่ม: 1
เพิ่ม: 2
เพิ่ม: 3
เพิ่ม: 4
เพิ่ม: 5
เพิ่ม: 6
ลด: 5
ลด: 4
ลด: 3
ลด: 2
ลด: 1
```

LED จะกระพริบตามค่าตัวแปรในขณะนั้น และในตอนที่กดปุ่มเพิ่มหรือลดแต่ละครั้ง เท่านั้น หลังจากกระพริบเสร็จ จะรอจนกว่าผู้ใช้ปล่อยปุ่มก่อนรับ event ใหม่ ไม่มีการกระพริบซ้ำ ๆ ในทุก loop และเมื่อกดลดลงไปจนถึง 0 แล้ว LED จะไม่ติดเลย และตัวแปร counter จะไม่สามารถกดให้ลดลงต่ำกว่า 0 ได้ เนื่องจากโค้ดบรรทัดที่ 25 ได้เขียนดักเอาไว้แล้วว่า **if counter < 0: counter = 0**

หมายถึง ถ้า counter น้อยกว่า 0 ให้ counter = 0

ความสำคัญในงานระบบสมองกลฝังตัว

ไม่ว่าจะเป็นงานนับเหตุการณ์ (Event Counting) สะสมค่าจากเซ็นเซอร์ (Sensor Data Accumulation) จัดการสถานะอุปกรณ์ หรือแม้แต่การคำนวณที่ซับซ้อนยิ่งขึ้น ทุกอย่างล้วนต้องใช้ Assignment Operators ทั้งสิ้น

ยิ่งโปรแกรมมีความซับซ้อนมากขึ้นเท่าใด การใช้ตัวดำเนินการกลุ่มนี้อย่างถูกต้อง จะช่วยให้โค้ดอ่านง่าย บำรุงรักษาง่าย และลดความผิดพลาดจากการเขียนโปรแกรมได้อย่างมีประสิทธิภาพ

ตัวดำเนินการกำหนดค่า (Assignment Operators) กับการควบคุมระบบฝังตัว

Assignment Operators ไม่ได้จำกัดอยู่แค่การเก็บค่าเลขในตัวแปรเท่านั้น แต่สามารถนำไปประยุกต์ใช้กับงานควบคุมในระบบฝังตัวและ IoT ได้หลากหลายอย่างน่าทึ่ง ตัวอย่างต่อไปนี้แสดงให้เห็นว่าเพียงเข้าใจและประยุกต์ใช้ตัวดำเนินการกำหนดค่าได้ถูกต้อง จะช่วยให้การออกแบบระบบซับซ้อน กลายเป็นเรื่องง่ายและประหยัดโค้ด

- **นับรอบการหมุนของล้อ:**
  เมื่อต่อเซ็นเซอร์แม่เหล็กหรือออปติคัลไว้ที่ล้อ เพียงเขียน rounds += 1 ทุกครั้งที่ตรวจจับสัญญาณ ก็สามารถนับจำนวนรอบการหมุนได้แบบเรียลไทม์ ซึ่งเป็นพื้นฐานของมาตรวัดความเร็วและระยะทางในโปรเจกต์รถอัตโนมัติและหุ่นยนต์
- **จำกัดค่าตัวแปรแบบวนรอบ (Modulo):**
  หากต้องการสร้างเอฟเฟกต์ที่ค่าตัวแปร “วนรอบ” เช่น ต้องการแสดงเลข 0–4 วนไปเรื่อย ๆ ทุกครั้งที่กดปุ่ม สามารถใช้ index += 1 แล้วกำหนด index %= 5 ค่าจะวนกลับไปเริ่มที่ 0 อัตโนมัติ เหมือนกับเข็มนาฬิกาที่หมุนครบแล้วกลับไปเริ่มต้นใหม่ เหมาะกับงานเช่นการเลือกเมนู หรือหมุน stepper motor
- **สร้างตัวหน่วงเวลาควบคุมเอง:**
  ในระบบควบคุมที่ต้องการหน่วงเวลาแบบละเอียด เช่นต้องการให้คำสั่งบางอย่างทำงานหลังผ่านไปจำนวน

รอบหนึ่ง สามารถสร้างตัวแปร timer แล้วใช้ timer -= 1 ในแต่ละรอบของ loop เมื่อ timer ลดถึง 0 ก็สั่งงานที่ต้องการ แล้วรีเซ็ต timer ใหม่ วิธีนี้ไม่ต้องพึ่งไลบรารี timer ให้โค้ดกระชับและควบคุมเองได้ 100%

Assignment Operators ใน MicroPython ไม่ใช่แค่ทางลัดในการเขียนโค้ดคำนวณ แต่คือหัวใจที่ทำให้ ESP32 “จำ” และ “เปลี่ยนแปลง” ข้อมูลในระบบได้อย่างมีประสิทธิภาพ เมื่อนำไปประยุกต์ใช้กับการนับรอบ, ควบคุมสถานะ, หรือปรับค่าเงื่อนไขต่าง ๆ — ตัวดำเนินการเหล่านี้จะเปลี่ยนงานที่ซับซ้อนให้เป็นเรื่องง่าย เพียงเข้าใจและใช้ =, +=, -=, *=, /= ได้ถูกต้อง ก็สามารถสร้างระบบที่นับจำนวน, สร้างลูกเล่นวนรอบ, หรือควบคุมสถานะในระบบสมองกลฝังตัวได้โดยตรง ไม่จำเป็นต้องพึ่งพาไลบรารีเสริม หรือโค้ดยืดยาวให้วุ่นวาย **การเข้าใจ Assignment Operators อย่างลึกซึ้ง จึงเป็นหนึ่งในกุญแจสำคัญของนักพัฒนาระบบอัตโนมัติและ IoT ทุกรูปแบบ**

ตัวดำเนินการบิต (Bitwise Operators) ใน MicroPython บน ESP32

หรือบางตำราอาจเรียกว่าตัวดำเนินการระดับบิต การทำงานกับระบบสมองกลฝังตัว เช่น ESP32 นอกจากการประมวลผลข้อมูลแบบตัวเลขทั่วไปแล้ว ยังมีความจำเป็นต้องควบคุมหรืออ่านค่าระดับ “บิต” (Bit) อยู่เสมอ ไม่ว่าจะเป็นการสั่งงานอุปกรณ์ที่ต่อหลายชิ้นพร้อมกัน หรือการอ่านสถานะอินพุตจำนวนมากในคราวเดียว การจัดการข้อมูลระดับบิตนี้เองที่ทำให้โปรแกรมมีประสิทธิภาพสูง และทำให้โค้ดของเราสั้นลง ซึ่งเป็นข้อได้เปรียบสำคัญของไมโครคอนโทรลเลอร์ทุกยุค

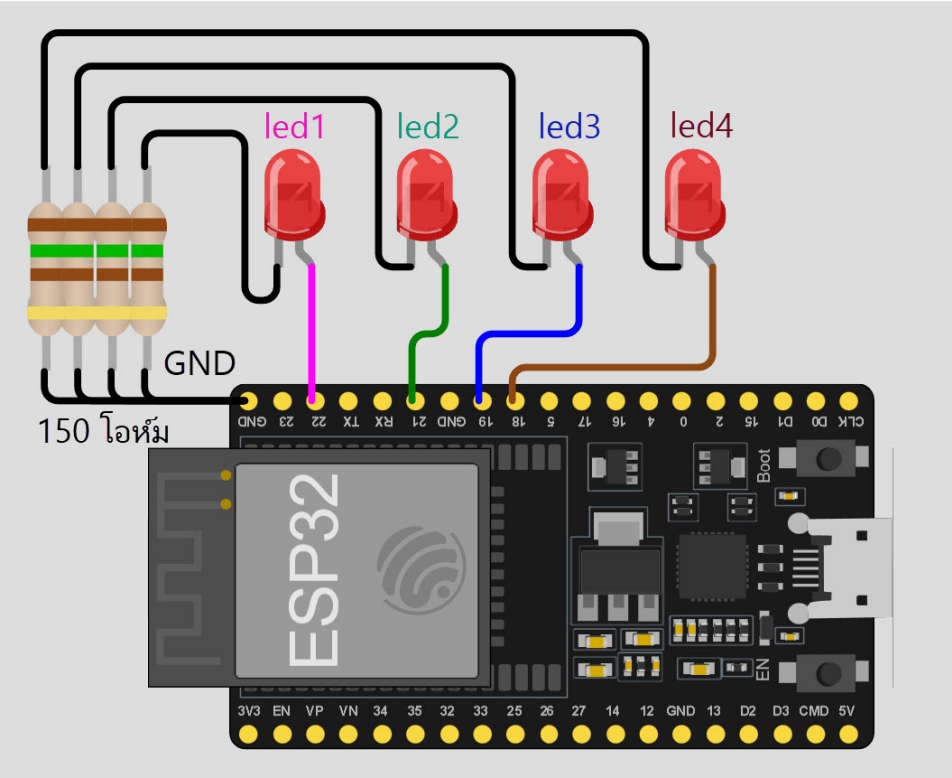
ประเภทของตัวดำเนินการบิตใน MicroPython

สัญลักษณ์	ตัวอย่าง	ความหมาย
&	a & b	AND (และ): บิตทั้งสองเป็น 1 ผลลัพธ์เป็น 1
|	`a | b	OR (หรือ): บิตใดบิตหนึ่งเป็น 1 ผลลัพธ์เป็น 1
^	a ^ b	XOR (Exclusive OR): บิตต่างกัน ผลลัพธ์เป็น 1
~	~a	NOT (กลับค่าบิต: 0 เป็น 1, 1 เป็น 0)
<<	a << 1	Shift Left: เลื่อนบิตไปทางซ้าย 1 ตำแหน่ง
>>	a >> 2	Shift Right: เลื่อนบิตไปทางขวา 2 ตำแหน่ง

หลักการและการประยุกต์ใช้งานกับวงจรจริง

ตัวอย่าง 1: ควบคุม LED 4 ดวงด้วย Bitwise Operators

สมมติว่าเรามี LED 4 ดวงต่อกับ ESP32 ที่ขา GPIO 18, 19, 21, 22 เราต้องการควบคุมให้ LED แต่ละดวงเปิดหรือปิดโดยใช้ “เลขฐานสอง” ร่วมกับ Bitwise Operator แนวคิดนี้คือการ “เก็บสถานะของทุกดวงไว้ในตัวแปรเดียว” แล้วใช้บิตแต่ละบิตแทนแต่ละดวง




```
from machine import Pin; import time

led_pins = [18, 19, 21, 22]                    # ขา GPIO สำหรับ LED
leds = [Pin(pin, Pin.OUT) for pin in led_pins]   # สร้างออบเจกต์ LED

value = 0b1110  # เลขฐานสอง 1110: LED4 คือขา 18 จะดับ เพราะอยู่ตำแหน่ง 0
for i in range(4):
    leds[i].value((value >> i) & 1)  # เช็กแต่ละบิตด้วย shift และ AND แล้วสั่งงาน LED
```

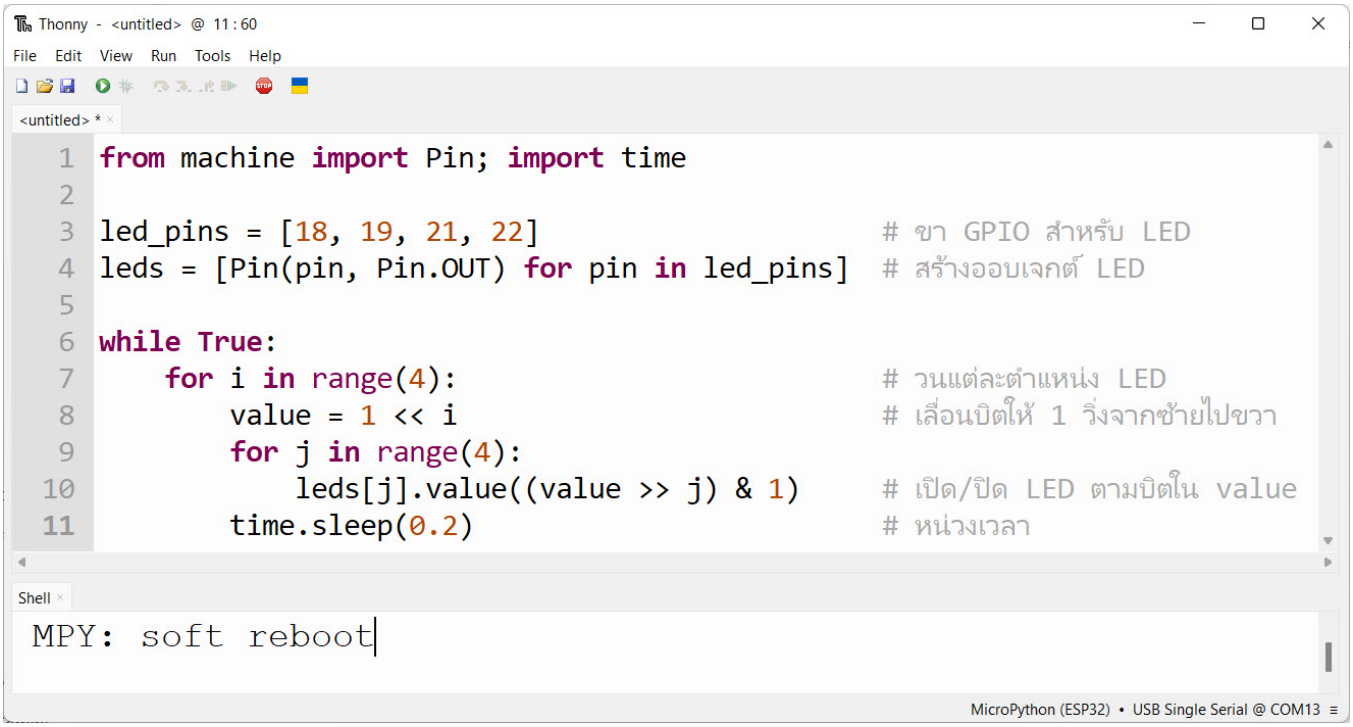
```
MPY: soft reboot
>>>
```

อธิบาย:

- value >> i เลื่อนบิตของ value มาตำแหน่งที่ต้องการ
- & 1 เช็กว่าเป็น 1 หรือ 0 แล้วสั่งงาน LED ดวงนั้น
- วิธีนี้ทำให้ควบคุม LED ทุกดวงพร้อมกันได้ในบรรทัดเดียว

ตัวอย่าง 2: การใช้ Shift Operators เพื่อสร้างเอฟเฟกต์ไฟวิ่ง 4 ดวง

เราสามารถสร้างไฟวิ่งโดยใช้ << หรือ >> เพื่อขยับตำแหน่งบิต แล้วเปิด/ปิด LED ตามตำแหน่งนั้นได้ง่าย ๆ



อธิบายการใช้งาน ตัวดำเนินการบิต:

- สร้าง list led_pins สำหรับขา 18, 19, 21, 22 ตามลำดับที่ต่อ LED
- ใช้ลูป for เพื่อไล่ตำแหน่ง bit โดยใช้ 1 << i ให้เลข 1 วิ่งทีละบิต
- ลูปอีกชั้นเช็กแต่ละบิตของ value ถ้าเป็น 1 ให้ LED ดวงนั้นติด ถ้า 0 ให้ดับ
- สร้างเอฟเฟกต์ “ไฟวิ่ง” จากซ้ายไปขวาตามตำแหน่ง LED จริง

ประโยชน์ของ Bitwise Operators กับระบบฝังตัว

- **ควบคุมอุปกรณ์หลายชิ้นผ่านตัวแปรเดียว:** เช่น LED, Relay, Buzzer, Motor Driver
- **อ่านอินพุตหลายตัวพร้อมกัน:** ประหยัดเวลาและโค้ดสั้นลง
- **ตั้งหรือล้างบิตบางตำแหน่งในรีจิสเตอร์:** งาน low-level เช่นสื่อสาร SPI, I2C
- **สร้างเอฟเฟกต์หรือฟังก์ชันซับซ้อน:** เช่น Binary Counter, Toggle, Pattern Generator

Bitwise Operators คือเครื่องมือที่เปลี่ยนให้ ESP32 ควบคุมและจัดการข้อมูลระดับบิตได้อย่างมีประสิทธิภาพ

ไม่ว่าจะเปิด/ปิด LED หลายดวง อ่านปุ่มหลายปุ่ม หรือประยุกต์กับการตั้งค่าฮาร์ดแวร์ภายใน หากเข้าใจการใช้ &, |, ^, ~, <<, >> อย่างถูกต้อง จะช่วยให้โค้ดของคุณทั้งกระชับ ยืดหยุ่น และพร้อมต่อยอดงานระบบสมองกลฝังตัวระดับสูงได้ทุกรูปแบบ

ตัวดำเนินการสมาชิก (Membership Operators) ใน MicroPython บน ESP32

การพัฒนาโปรแกรมสำหรับระบบสมองกลฝังตัว เช่น ESP32 ด้วยภาษา MicroPython นอกจากต้องควบคุมอุปกรณ์ฮาร์ดแวร์แล้ว ยังจำเป็นต้องตรวจสอบข้อมูลต่าง ๆ ที่อยู่ในรูปแบบ **ลิสต์** (List), **ทูเพิล** (Tuple), **สตริง** (String) หรือแม้แต่ชุดข้อมูลจากเซ็นเซอร์

ในสถานการณ์เหล่านี้ “ตัวดำเนินการสมาชิก” หรือ **Membership Operators** จะเป็นเครื่องมือสำคัญที่ช่วยให้โปรแกรมสามารถตรวจสอบได้ว่า “ข้อมูลที่ต้องการ” อยู่ในกลุ่มข้อมูลหรือไม่ สิ่งนี้มีประโยชน์มากทั้งการตรวจสอบคำสั่ง, การแยกข้อมูลจากเซ็นเซอร์, และการสร้างเงื่อนไขที่ยืดหยุ่นในระบบอัตโนมัติ

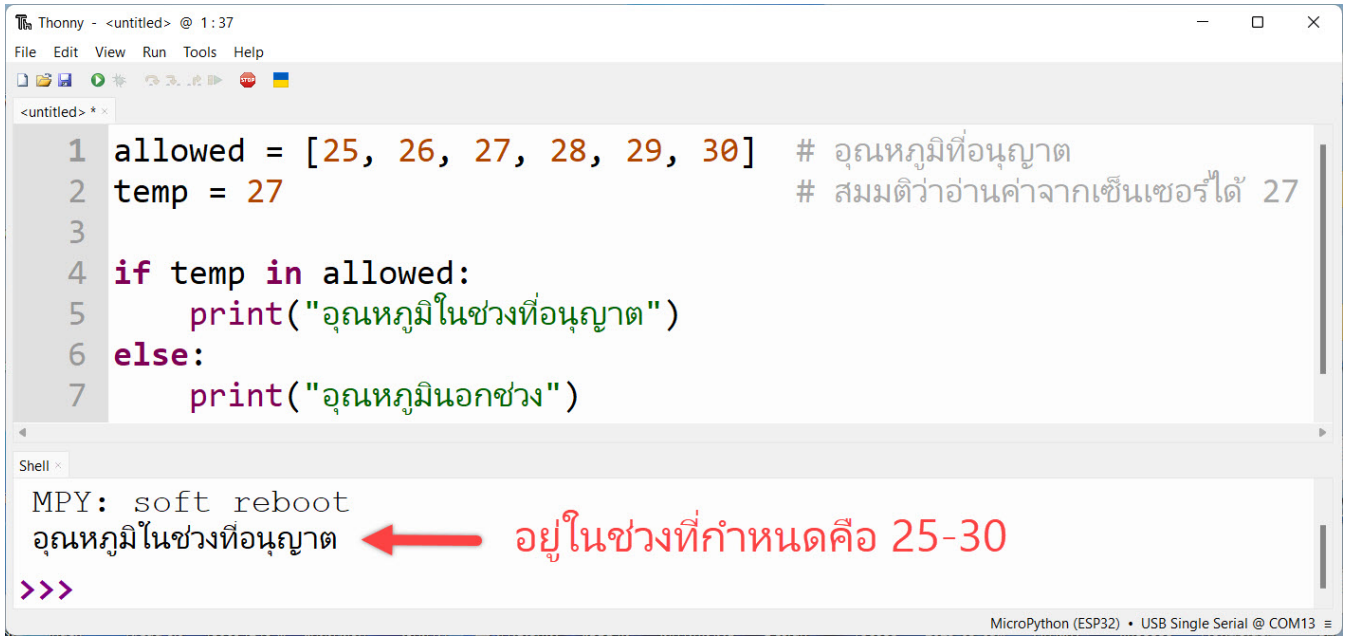
ประเภทของตัวดำเนินการสมาชิก

สัญลักษณ์	ความหมาย	ตัวอย่าง
in	อยู่ใน/เป็นสมาชิกของ	x in data
not in	ไม่อยู่ใน/ไม่เป็นสมาชิกของ	x not in data

ตัวอย่างการใช้งาน Membership Operators กับ ESP32

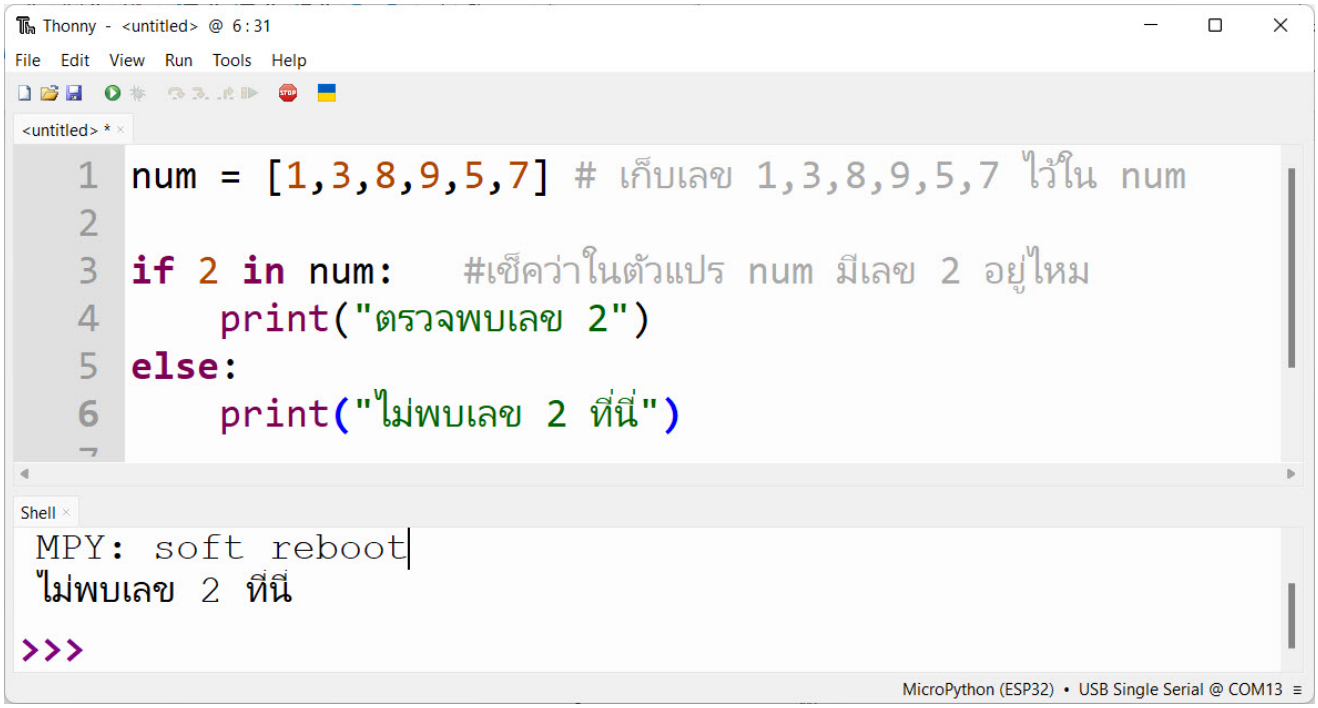
1. ตรวจสอบค่าที่รับมาจากเซ็นเซอร์ว่า “อยู่ใน” ช่วงที่ต้องการหรือไม่

สมมติว่าอ่านค่าจากเซ็นเซอร์อุณหภูมิ แล้วต้องการตรวจสอบว่าค่านั้นอยู่ใน “ค่าที่อนุญาต” หรือไม่

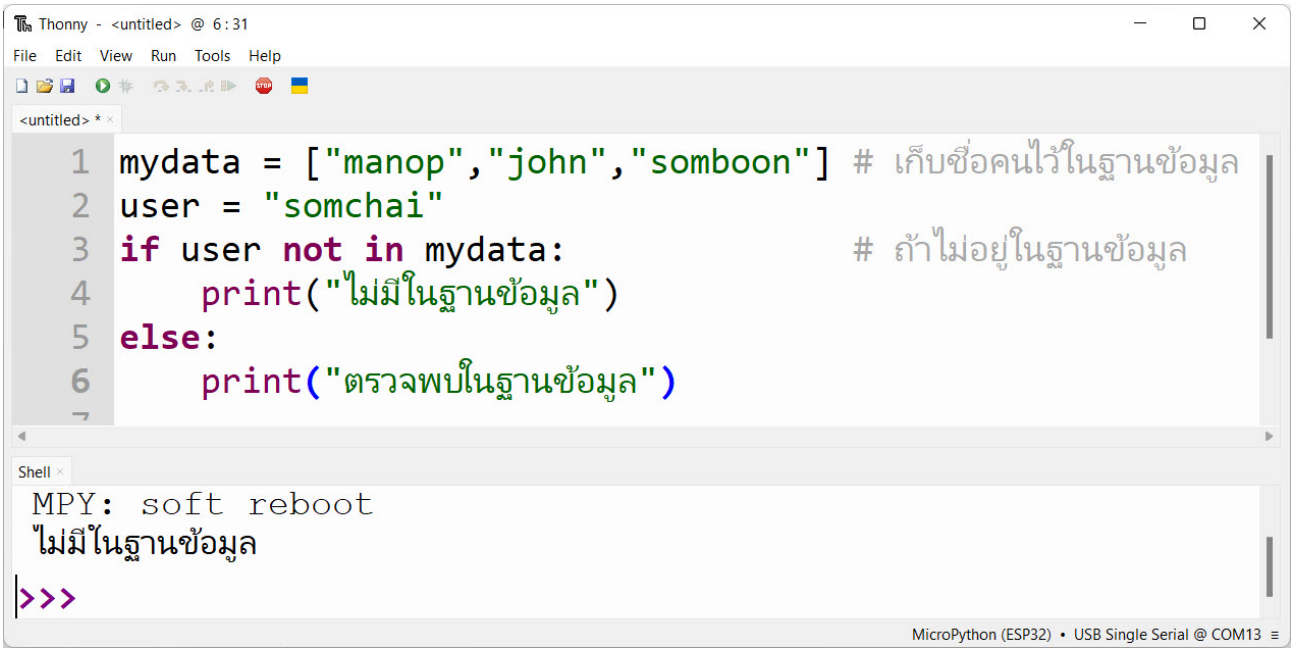


จากตัวอย่างนี้หากทดลองเปลี่ยนค่าของตัวแปร temp เป็น 24 หรือ 31 ระบบจะ print “อุณหภูมินอกช่วง”

3. อีกตัวอย่างง่าย ๆ ตรวจสอบข้อมูลในตัวแปร num ว่ามีข้อมูลที่เราต้องการหรือไม่



4. ตัวอย่างการใช้งาน not in ตรวจเช็คว่ามีรายชื่อบุคคลอยู่ในฐานข้อมูลหรือไม่



จากตัวอย่างจะสังเกตได้ว่าการใช้ in หรือ not in นั้นมีผลการทำงานเหมือนกัน เพราะถึงอย่างไรเราก็ตรวจด้วยคำสั่ง if ซึ่งเป็นไปได้แค่ 2 ทางเลือก คือ มี หรือไม่มี ตรงกันหรือไม่ตรงกัน แค่นั้น แล้วทำไมจะต้องมี in กับ not in ด้วย เพราะการตรวจสอบลักษณะนี้เราใช้แค่ in ก็ตรวจสอบได้เหมือนกันทุกประการ หรืออาจจะใช้เพียง not in ก็ได้ แต่ดูก่อน ทุกอย่างที่เขาสร้างมาย่อมมีเหตุผล

ความแตกต่างและเหตุผลที่ต้องมีทั้ง in และ not in ในภาษา Python/MicroPython

ในการเขียนโปรแกรม โดยเฉพาะกับระบบสมองกลฝังตัว เช่น ESP32 ด้วยภาษา Python หรือ MicroPython การตรวจสอบว่าสิ่งหนึ่ง “อยู่” หรือ “ไม่อยู่” ในชุดข้อมูล เป็นสิ่งที่เกิดขึ้นบ่อยมาก ไม่ว่าจะเป็นการเช็กว่าค่าที่อ่านจากเซ็นเซอร์อยู่ในช่วงที่ยอมรับได้หรือไม่ ตรวจสอบว่าคำสั่งที่รับเข้ามาเป็นหนึ่งในชุดคำสั่งที่อนุญาต หรือแม้แต่การดูว่าอุปกรณ์ชิ้นใดได้รับอนุญาตให้เชื่อมต่อกับระบบหรือยัง

ตัวดำเนินการสมาชิก (Membership Operators) ที่สำคัญในภาษา Python/MicroPython มีสองตัว ได้แก่ in และ not in ซึ่งถูกออกแบบมาเพื่อรองรับการตรวจสอบทั้งในแง่บวกและแง่ลบโดยตรง แม้ในทางเทคนิคจะสามารถใช้ in คู่กับ not เพื่อสื่อความหมายเดียวกับ not in ได้ เช่น if not (x in data): แต่รูปแบบนี้จะดูซับซ้อนมากขึ้นและอาจสร้างความสับสนในบางกรณี

ประโยชน์ของการมีทั้ง in และ not in ก็คือ “ความชัดเจนของเจตนา”

เวลาที่เขียนว่า if x in data: จะอ่านว่า “ถ้า x อยู่ใน data ให้ทำ ...” ตรงไปตรงมา ส่วน if x not in data: อ่านว่า “ถ้า x ไม่อยู่ใน data ให้ทำ ...” ซึ่งชัดเจนและเข้าใจง่ายมากกว่าใช้ not ครอบเงื่อนไขเดิม

ความแตกต่างนี้ไม่ได้เป็นเพียงเรื่องของความสะดวก แต่ยังมีผลอย่างมากต่อความถูกต้องของตรรกะในโปรแกรมด้วย เพราะการสลับเงื่อนไขหรือเข้าใจผิดระหว่างการ “มีอยู่” กับ “ไม่มีอยู่” สามารถทำให้โปรแกรมทำงานผิดพลาดได้โดยไม่รู้ตัว ยิ่งถ้าเป็นโปรแกรมที่มีเงื่อนไขซ้อนกันหลายชั้น หรือโค้ดยาว การใช้ not in แทน not (x in data) จะช่วยลดโอกาสเกิดบั๊กได้ดีขึ้นมาก

ยิ่งไปกว่านั้น รูปแบบ not in ยังทำให้ภาษา Python/MicroPython มีความเป็นธรรมชาติคล้ายกับภาษาอังกฤษ ผู้เรียนหรือผู้อ่านโค้ดจึงเข้าใจตรรกะของโปรแกรมได้ทันทีโดยไม่ต้องแปลความหมายซ้ำสอง เหมาะกับทั้งงานเช็กข้อมูล, การเขียนโปรแกรมควบคุม, การทำระบบรักษาความปลอดภัย หรือแม้แต่การกรองข้อมูลที่ต้องการหรือไม่ต้องการในระบบอัตโนมัติ

กล่าวโดยสรุป การมีทั้ง in และ not in เป็นสิ่งที่ช่วยให้ภาษา Python/MicroPython ทรงพลังมากขึ้น ทั้งในด้านความชัดเจนของตรรกะ ความถูกต้องในการสื่อความหมาย และความสะดวกในการอ่านและดูแลโค้ดในระยะยาว แม้ในสถานการณ์ที่สามารถใช้ not คู่กับ in ได้เหมือนกัน แต่การเลือกใช้ not in โดยตรงจะทำให้โปรแกรมดูสะอาดและปลอดภัยกว่ามาก และนี่เองคือเหตุผลว่าทำไมโปรแกรมเมอร์มืออาชีพจึงนิยมใช้ not in เมื่อมีเงื่อนไขที่ต้องตรวจสอบว่าสิ่งใด “ไม่มีอยู่” ในชุดข้อมูลเสมอ

การประยุกต์ใช้งานกับโปรเจกต์จริง

- ตรวจสอบค่าที่อ่านจากเซ็นเซอร์ว่าตรงกับเงื่อนไขที่ต้องการ เช่น “อุณหภูมิสูงเกิน/ต่ำเกิน”
- เช็กรหัสผ่านหรือ Token ว่าตรงกับที่กำหนดไว้หรือไม่
- ตรวจสอบรายชื่ออุปกรณ์ที่เชื่อมต่อผ่าน WiFi/Bluetooth ว่าเป็นอุปกรณ์ที่ได้รับอนุญาตหรือไม่
- กรองข้อมูลอินพุตจากผู้ใช้, Serial, หรือ MQTT ด้วยคำสั่งเดียว

ตัวดำเนินการสมาชิก (Membership Operators) อย่าง in และ not in คือเครื่องมือที่ทำให้โปรแกรม MicroPython บน ESP32 มีความยืดหยุ่นในการตรวจสอบข้อมูล ไม่ว่าจะเป็นชุดค่าคงที่, ชุดคำสั่ง, ผลจากเซ็นเซอร์ หรือแม้แต่ข้อมูลจากผู้ใช้ เมื่อเข้าใจหลักการนี้ จะสามารถออกแบบระบบที่ตรวจสอบเงื่อนไขต่าง ๆ ได้ง่าย รัดกุม และขยายโปรแกรมให้รองรับงานที่ซับซ้อนขึ้นได้อย่างมืออาชีพ

นิพจน์ (Expressions) และลำดับการคำนวณ (Order of Evaluation) ใน MicroPython บน ESP32

การเขียนโปรแกรมบน ESP32 ด้วย MicroPython นั้น สิ่งหนึ่งที่ขาดไม่ได้และมักปรากฏอยู่ในแทบทุกบรรทัดของโค้ด คือ "นิพจน์" (Expression) ซึ่งก็คือการรวมกันของค่าคงที่ ตัวแปร และตัวดำเนินการ (Operators) เพื่อให้เกิดการประเมินผลเป็นค่าบางอย่าง เช่น ตัวเลขที่ได้จากการคำนวณ หรือค่าบูลีนที่ใช้ตัดสินใจในคำสั่ง if การเข้าใจวิธีการประเมินนิพจน์และลำดับการทำงานของตัวดำเนินการ (Operator Precedence) จึงเป็นสิ่งจำเป็นอย่างยิ่งต่อการเขียนโปรแกรมให้ทำงานได้อย่างถูกต้อง มีประสิทธิภาพ และปราศจากบั๊กที่เกิดจากการตีความผิด

นิพจน์คืออะไร?

นิพจน์ในโปรแกรมมิ่ง หมายถึง ส่วนหนึ่งของโค้ดหรือกลุ่มของโค้ดที่ประกอบด้วยตัวแปร ค่าคงที่ ตัวดำเนินการ

(Operators) และอาจรวมถึงฟังก์ชันหรือการเรียกใช้งานอ็อบเจกต์ต่าง ๆ ด้วยเมื่อประมวลผลแล้วจะให้ค่าผลลัพธ์เป็นค่าหนึ่ง ซึ่งอาจเป็นค่าตัวเลข ตัวอักษร ลิสต์ หรือค่าบูลีน (True/False) ก็ได้

ตัวอย่างนิพจน์ใน MicroPython:

- `x + y * 2`
- `temperature > 30`
- `button.value() == 1 and led_state == False`
- `analog_reading / 1024.0`
- `"ON" if status == 1 else "OFF"`

ในแต่ละบรรทัดข้างต้น เมื่อรันแล้ว MicroPython จะ “คำนวณ” หรือ “ประเมินค่า” เพื่อให้ผลลัพธ์ที่นำไปใช้ต่อได้ เช่น ตัดสินใจใน if, ส่งออกผ่าน serial, หรือควบคุมอุปกรณ์ต่อพ่วง

ตัวอย่าง: `x = 3 + 4`

ในบรรทัดนี้ 3 + 4 คือ "นิพจน์" ที่ประกอบด้วยค่าคงที่ 3 และ 4 ร่วมกับตัวดำเนินการ + ผลลัพธ์ที่ได้คือ 7 ซึ่งจะถูกเก็บไว้ในตัวแปร x

หรืออีกตัวอย่างหนึ่ง:

`is_ready = (temp > 30) and fan_enabled`

นิพจน์นี้ประกอบด้วยเงื่อนไขเปรียบเทียบ **(temp > 30)** และตัวแปร **fan_enabled** ซึ่งเชื่อมกันด้วยตัวดำเนินการตรรกะ **and** และผลลัพธ์ที่ได้จะเป็นค่าบูลีน **True** หรือ **False**

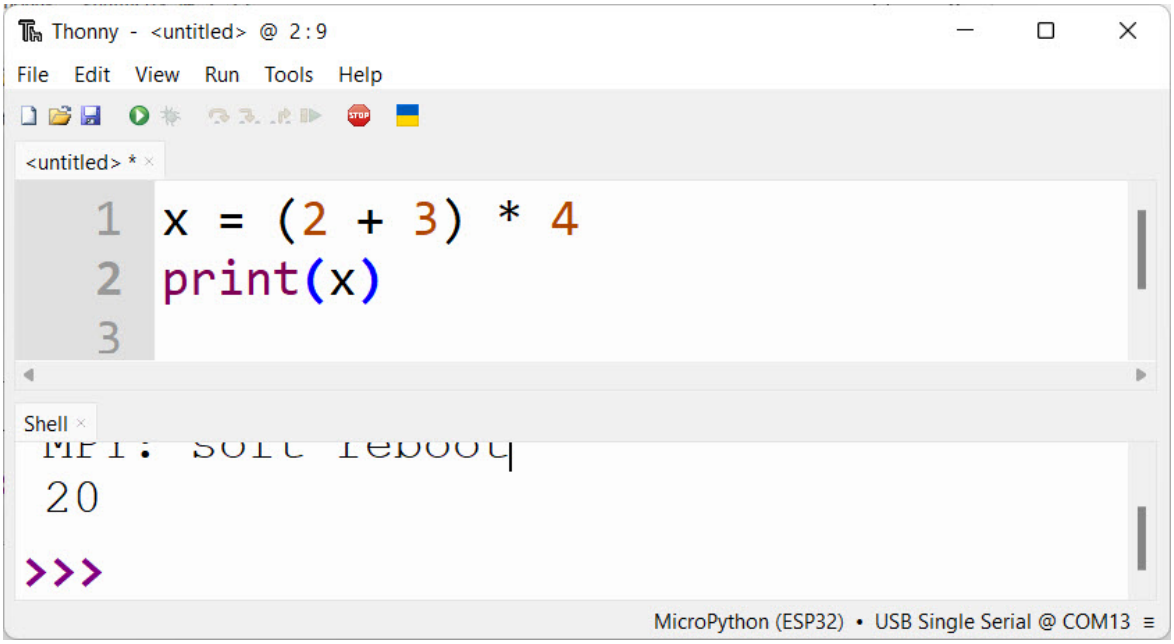
ความสำคัญของลำดับการคำนวณ

ลำดับการคำนวณ เมื่อเขียนนิพจน์ที่มีตัวดำเนินการหลายตัวในบรรทัดเดียว ลำดับที่ตัวดำเนินการเหล่านั้นถูกประเมิน (หรือเรียกว่าความสำคัญของตัวดำเนินการ - Operator Precedence) จะส่งผลโดยตรงต่อผลลัพธ์

เช่นเดียวกับการคำนวณคณิตศาสตร์ที่ต้องมีการจัดลำดับวงเล็บ, การคูณ/หาร มาก่อนการบวก/ลบ ลำดับของตัวดำเนินการมีผลอย่างมากต่อผลลัพธ์ของนิพจน์ เพราะตัวแปลภาษา (Interpreter) จะประเมินค่าตามลำดับความสำคัญของตัวดำเนินการ ไม่ใช่จากซ้ายไปขวาเสมอไป เช่น: `x = 2 + 3 * 4`

ผลลัพธ์คือ 14 เพราะ * มีลำดับความสำคัญสูงกว่า + ดังนั้นจะคำนวณ 3 * 4 ก่อน แล้วจึงนำผลลัพธ์ไปบวกกับ 2

ถ้าต้องการบวกก่อน ให้ใช้วงเล็บ: x = (2 + 3) * 4 **ผลลัพธ์จึงจะได้ 20**



2 + 3 = 5

5 * 4 = 20

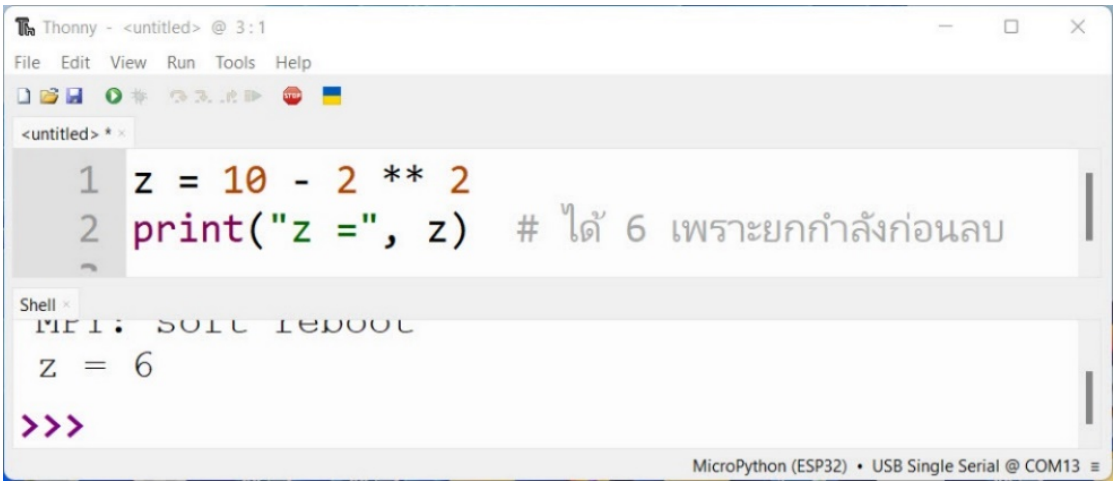
ตารางลำดับความสำคัญของตัวดำเนินการใน MicroPython

ลำดับ	ตัวดำเนินการ	ตัวอย่าง
1	วงเล็บ ()	(a + b) * c
2	ยกกำลัง **	a ** b
3	+, -, ไม่ตรงข้าม	-a, +a
4	*, /, //, %	a * b, a / b
5	+, -	a + b, a - b
6	<<, >>	a << 2, b >> 1
7	&, |, ^	a & b, a | b
8	ตัวดำเนินการเปรียบเทียบ	a > b, a == b
9	not	not a
10	and	a and b
11	or	a or b

ควรใช้วงเล็บ () ช่วยจัดลำดับเพื่อให้โค้ดอ่านง่ายและปลอดภัยจากตรรกะผิดพลาด

🧪 ทดลองจริงบน ESP32

ทดลองต่อบอร์ด ESP32 แล้วเปิดโปรแกรม Thonny จากนั้นพิมพ์โค้ดต่อไปนี้:



🎯 ประยุกต์ใช้ในโปรเจกต์จริง

กรณีอ่านค่าจากตัวแปรที่คำนวณจากการอ่านเซ็นเซอร์:

หากอุณหภูมิสูงเกิน 35 จะเปิดพัดลมทันที หรือหากทั้งอุณหภูมิ > 30 และความชื้น > 80 ก็เปิดเช่นกัน

🧩 ปัญหาที่มักเจอเมื่อไม่เข้าใจลำดับ

สมมุติว่ามีเงื่อนไขนี้:

```
if not sensor_triggered and active:
    print("เริ่มทำงาน")
```

หากไม่เข้าใจว่าคำสั่ง not จะมีลำดับก่อน and อาจตีความว่ากำลังกลับค่าของ sensor_triggered and active ทั้งชุด ทั้งที่จริงเป็นการกลับแค่ sensor_triggered เท่านั้น แล้วจึงนำไป and กับ active

เทคนิคที่ผู้เขียนแนะนำ และควรนำไปใช้

- ใช้วงเล็บเพื่อเน้นลำดับที่ต้องการเสมอ แม้จะไม่จำเป็น เพื่อป้องกันข้อผิดพลาดจากการตีความผิด
- อย่าเขียนนิพจน์ยาวเกินไปจนอ่านไม่ออก แยกเป็นหลายบรรทัดดีกว่า:

```
temp_valid = temperature > 30
humid_valid = humidity > 80
if temp_valid and humid_valid:
print("ควรเปิดพัดลม")
```

นิพจน์คือพื้นฐานสำคัญที่สุดของทุกคำสั่งใน MicroPython และลำดับการคำนวณคือกลไกเบื้องหลังที่ตัดสินว่า

โค้ดจะได้ผลลัพธ์ถูกหรือผิด การเข้าใจสิ่งเหล่านี้ไม่ใช่แค่รู้ตามทฤษฎี แต่ต้องฝึกเขียนบ่อย ๆ และพยายามวิเคราะห์ผลลัพธ์ให้ลึกถึงระดับลอจิกของ MCU เพื่อให้สามารถควบคุมการทำงานของ ESP32 ได้อย่างมั่นใจ

ข้อควรทราบสำหรับผู้อ่าน

หากอ่านจบบทนี้แล้วยังรู้สึกว่า “ตัวดำเนินการ” (Operators) บางประเภทหรือแนวคิดบางอย่างยังเข้าใจไม่ลึก หรือยังไม่แน่ใจว่าจะนำไปใช้จริงอย่างไร ขอให้ไม่ต้องกังวล ในบทถัดไป เมื่อได้ศึกษา “คำสั่งควบคุมการทำงาน” (Control Flow Statements) และทดลองใช้งานจริงกับโปรเจกต์ จะพบว่าตัวดำเนินการที่ได้เรียนไปแล้วจะถูกนำมาใช้อย่างต่อเนื่องและชัดเจน **แนะนำให้ก้าวเข้าสู่บทถัดไปก่อน** แล้วค่อยย้อนกลับมาทบทวนเนื้อหานี้อีกครั้งจะช่วยให้เข้าใจการทำงานของแต่ละตัวดำเนินการได้ลึกซึ้งและเห็นภาพรวมของโปรแกรมควบคุม ESP32 ด้วย MicroPython อย่างแท้จริง

คำสั่งควบคุมการทำงาน ใน MicroPython บน ESP32

การพัฒนาโปรแกรมสำหรับไมโครคอนโทรลเลอร์ เช่น ESP32 ด้วย MicroPython ไม่ได้มีเพียงแค่การสั่งให้เปิดหรือปิดอุปกรณ์เท่านั้น แต่ยังต้องให้โปรแกรม “ตัดสินใจ เลือกลำดับการทำงาน และตอบสนองต่อสถานการณ์ต่าง ๆ ได้อย่างยืดหยุ่น” ซึ่งทั้งหมดนี้เกิดจากการใช้ “คำสั่งควบคุมการทำงาน” (Control Statements) คำสั่งเหล่านี้เป็นพื้นฐานสำคัญที่ทำให้โปรแกรมสามารถกำหนดลำดับ กระบวนการตัดสินใจ และการทำงานซ้ำได้อย่างเป็นระบบ โดยสามารถแบ่งออกได้เป็น 4 ประเภทหลัก ได้แก่

1. **คำสั่งเงื่อนไข (Conditional Statements)**

   ใช้สำหรับตรวจสอบเงื่อนไขหรือตัดสินใจเลือกทางเดินของโปรแกรม เช่น if, elif, else

2. **คำสั่งวนซ้ำ (Loop Statements)**

   ใช้สำหรับการทำงานซ้ำ ๆ เช่น for, while และการควบคุมการวนซ้ำด้วย break, continue, pass

3. **คำสั่งควบคุมลำดับการทำงาน (Flow Control Statements)**

   เช่น return (ใช้ในฟังก์ชัน), pass (ใช้เว้นว่างคำสั่ง), break, continue

4. **คำสั่งจัดการข้อผิดพลาด (Exception Handling Statements)**

   เช่น try, except, finally, raise สำหรับจัดการเหตุการณ์ผิดปกติระหว่างรันโปรแกรม

คำสั่งควบคุมเหล่านี้เป็นหัวใจของการออกแบบตรรกะการทำงานในทุกโปรแกรมควบคุมไมโครคอนโทรลเลอร์ หากเข้าใจและใช้งานได้อย่างถูกต้อง จะสามารถสร้างระบบที่ตอบสนองต่อสภาพแวดล้อมหรือเหตุการณ์ต่าง ๆ ได้อย่างชาญฉลาด เหมาะกับทั้งงานอัตโนมัติ งาน IoT และโปรเจกต์ที่ต้องการความน่าเชื่อถือสูง

คำสั่งเงื่อนไข (Conditional Statements) ใน MicroPython

การเขียนโปรแกรมเพื่อให้สามารถตัดสินใจและตอบสนองต่อสถานการณ์ต่าง ๆ ได้อย่างยืดหยุ่น ถือเป็นหัวใจสำคัญของการพัฒนาโปรแกรมควบคุมไมโครคอนโทรลเลอร์ หนึ่งในโครงสร้างที่สำคัญที่สุดสำหรับงานลักษณะนี้คือ **คำสั่งเงื่อนไข** (Conditional Statement) ซึ่งเป็นโครงสร้างที่ใช้ในการตรวจสอบเงื่อนไขบางอย่าง หากเงื่อนไขนั้นเป็นจริงก็จะดำเนินการทำตามคำสั่งที่กำหนดไว้ ถ้าไม่เป็นจริงก็อาจเลือกดำเนินการในอีกชุดคำสั่งหนึ่งแทน

ใน MicroPython การใช้คำสั่งเงื่อนไขจะเหมือนกับภาษา Python ทั่วไป คือใช้คำสั่ง if, elif (else if), และ else ซึ่งช่วยให้โปรแกรมสามารถเลือกเส้นทางการทำงานได้อย่างหลากหลาย ตัวอย่างเช่น การตรวจสอบสถานะของปุ่มกดและควบคุมการเปิด–ปิด LED ตามสถานะของปุ่มกด การพัฒนาโปรแกรมสำหรับไมโครคอนโทรลเลอร์หรือระบบสมองกลฝังตัว เช่น ESP32 ด้วย MicroPython คำสั่งเงื่อนไข (Conditional Statements) เป็นหนึ่งในโครงสร้างพื้นฐานที่สำคัญที่สุด ช่วยให้โปรแกรมสามารถ “ตรวจสอบสถานะ” หรือ “เงื่อนไข” ที่เกิดขึ้นในระบบ แล้วดำเนินการตามที่กำหนดไว้ ซึ่งหลักการนี้ถูกใช้งานอย่างแพร่หลายในงานอัตโนมัติ งานควบคุม และอุปกรณ์อิเล็กทรอนิกส์เกือบทุกชนิด ในชีวิตประจำวัน มนุษย์เราต้องเผชิญกับสถานการณ์ที่ต้องใช้การ “ตรวจสอบเงื่อนไข” เพื่อประกอบการตัดสินใจอยู่เสมอ เช่น

- ถ้า **ฝนตก** (เงื่อนไข) ⟶ เราจะ **กางร่ม** (การกระทำ)
- ถ้า **รู้สึกหิว** (เงื่อนไข) ⟶ จะ **ออกไปหาอาหาร** (การกระทำ)

- ถ้า **ได้รับข้อความจากเพื่อน** (เงื่อนไข) ⟶ จะ **หยิบโทรศัพท์ขึ้นมาอ่าน**
- ถ้า **ไฟจราจรเปลี่ยนเป็นเขียว** (เงื่อนไข) ⟶ จะ **เดินข้ามถนน**
- ถ้า **นาฬิกาปลุกดัง** (เงื่อนไข) ⟶ จะ **ลุกขึ้นจากที่นอน**

ในแต่ละตัวอย่าง “เงื่อนไข” คือสิ่งที่ต้องตรวจสอบก่อน เมื่อเป็นจริงแล้วจึงเกิด “การกระทำ” ตามมา

การเขียนโปรแกรมหรือควบคุมไมโครคอนโทรลเลอร์ก็ใช้หลักการเดียวกัน เช่น

- ถ้า **กดปุ่มบนบอร์ด** (เงื่อนไข) ⟶ ให้ **เปิดไฟ LED**
- ถ้า **มีสัญญาณจากเซ็นเซอร์** (เงื่อนไข) ⟶ **ส่งเสียงแจ้งเตือน**
- ถ้า **ค่าเซ็นเซอร์เกินระดับที่กำหนด** (เงื่อนไข) ⟶ **สั่งหยุดเครื่องจักร**

การตรวจสอบเงื่อนไข (Condition Checking) ใน MicroPython หมายถึงการประเมินผลการเปรียบเทียบหรือการตรวจสอบค่าของตัวแปรหรือสถานะต่าง ๆ ว่าตรงตาม “เงื่อนไข” ที่กำหนดไว้หรือไม่

- ถ้าเป็น “จริง” (True) โปรแกรมจะเข้าสู่บล็อกคำสั่งที่กำหนด
- ถ้าเป็น “เท็จ” (False) โปรแกรมจะข้ามไปดำเนินการส่วนถัดไป

ตัวอย่างสถานการณ์ที่ตรวจสอบเงื่อนไขในงานจริง เช่น

- ตรวจสอบค่าปุ่มกด (button.value() == 1)
- ตรวจสอบค่าสัญญาณไฟ (led.value() == 0)
- ตรวจสอบค่าจากเซ็นเซอร์ (temp > 30) ฯลฯ

โครงสร้างของคำสั่งเงื่อนไขใน MicroPython

ใน MicroPython จะใช้โครงสร้าง if–elif–else เหมือนภาษา Python ทั่วไป ตัวอย่างเช่น

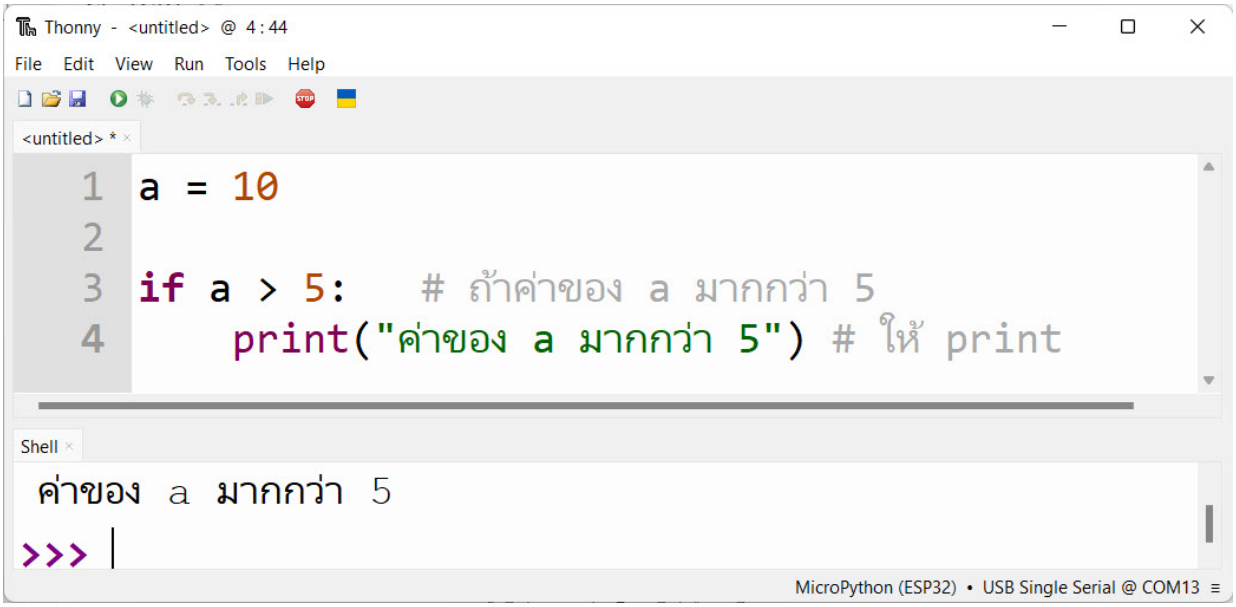
```
if เงื่อนไข:
    คำสั่งเมื่อเงื่อนไขเป็นจริง
elif เงื่อนไขอื่น:
    คำสั่งเมื่อเงื่อนไขอื่นเป็นจริง
else:
    คำสั่งเมื่อไม่มีเงื่อนไขใดเป็นจริงเลย
```

สามารถมีเพียง if เดียว หรือเพิ่ม elif และ else ตามลำดับความซับซ้อนได้ โดยทั่วไป โครงสร้างของคำสั่งเงื่อนไขจะถูกแบ่งเป็น 4 กรณีหลักดังนี้

1. **คำสั่ง if (กรณีเงื่อนไขเดียว)**

```
if เงื่อนไข:
    คำสั่งเมื่อเงื่อนไขเป็นจริง
```

ตัวอย่าง: ใช้คำสั่งเงื่อนไข if โดยมีเงื่อนไข ว่า ถ้า a มากกว่า 5 ให้ print ข้อความออกมา



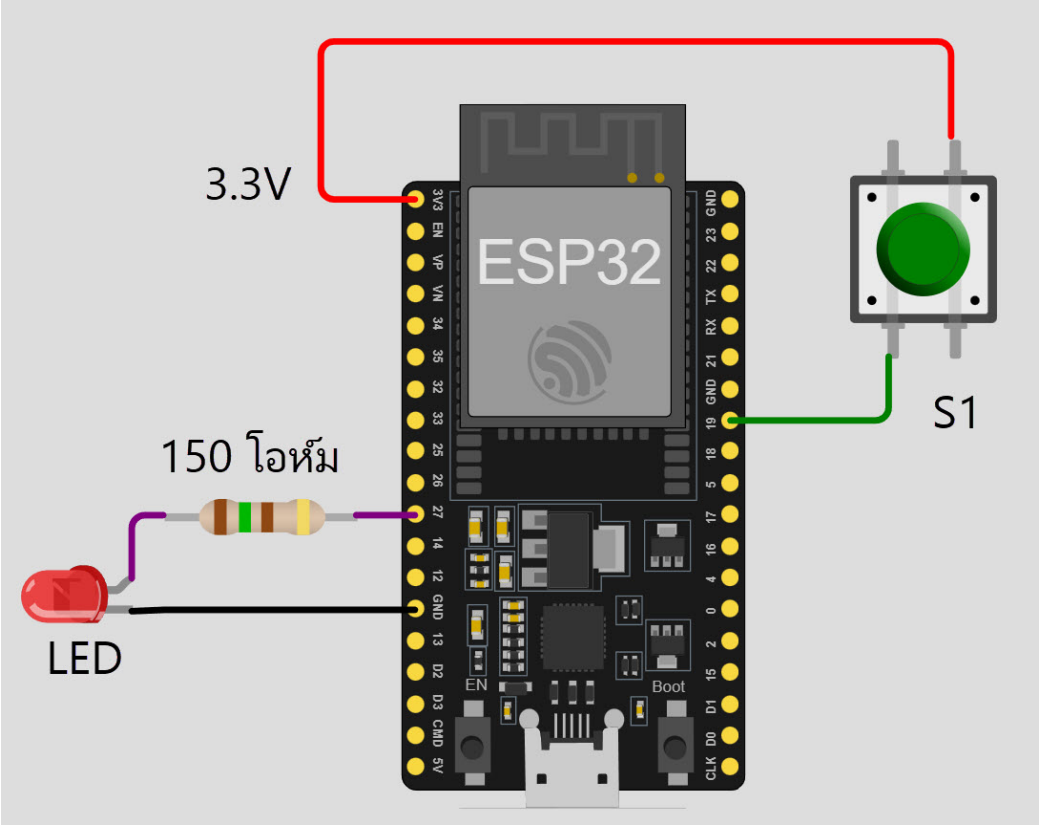
เงื่อนไขแบบนี้คือมีทางเลือกเดียวให้ทำ ไม่มีทางเลือกอื่น

หากต้องการให้ทำงานมากกว่าทางเลือกเดียวก็ต้องใช้คำสั่ง if แบบสองเงื่อนไข **คำสั่ง if–else**

2. **คำสั่ง if–else (สองเงื่อนไข) หรืออาจเรียกได้ว่า สองทางเลือก**

```
if เงื่อนไข:
    คำสั่งเมื่อเงื่อนไขเป็นจริง
else:
    คำสั่งเมื่อเงื่อนไขเป็นเท็จ
```

ตัวอย่าง: การใช้งานกดปุ่มให้สั่งให้ไฟ LED ติด ไม่กดปุ่ม LED ดับ ต่อวงจรตามรูป



Thonny - <untitled> @ 12 : 20

File Edit View Run Tools Help

<untitled> *

```
1  from machine import Pin
2  import time
3
4  button = Pin(19, Pin.IN, Pin.PULL_UP)    # ใช้ PULL_UP
5  led = Pin(27, Pin.OUT)                   # ใช้ขา 27 เป็นเอาต์พุต
6
7  while True:
8      if button.value() == 0:              # ถ้ากดปุ่ม
9          led.value(1)                     # ให้ไฟติด
10     else:                                # นอกจากนั้น คือปล่อยปุ่ม
11         led.value(0)                     # ให้ไฟดับ
12     time.sleep(0.1)
```

Shell

```
MPY: soft reboot
```

MicroPython (ESP32) • USB Single Serial @ COM13

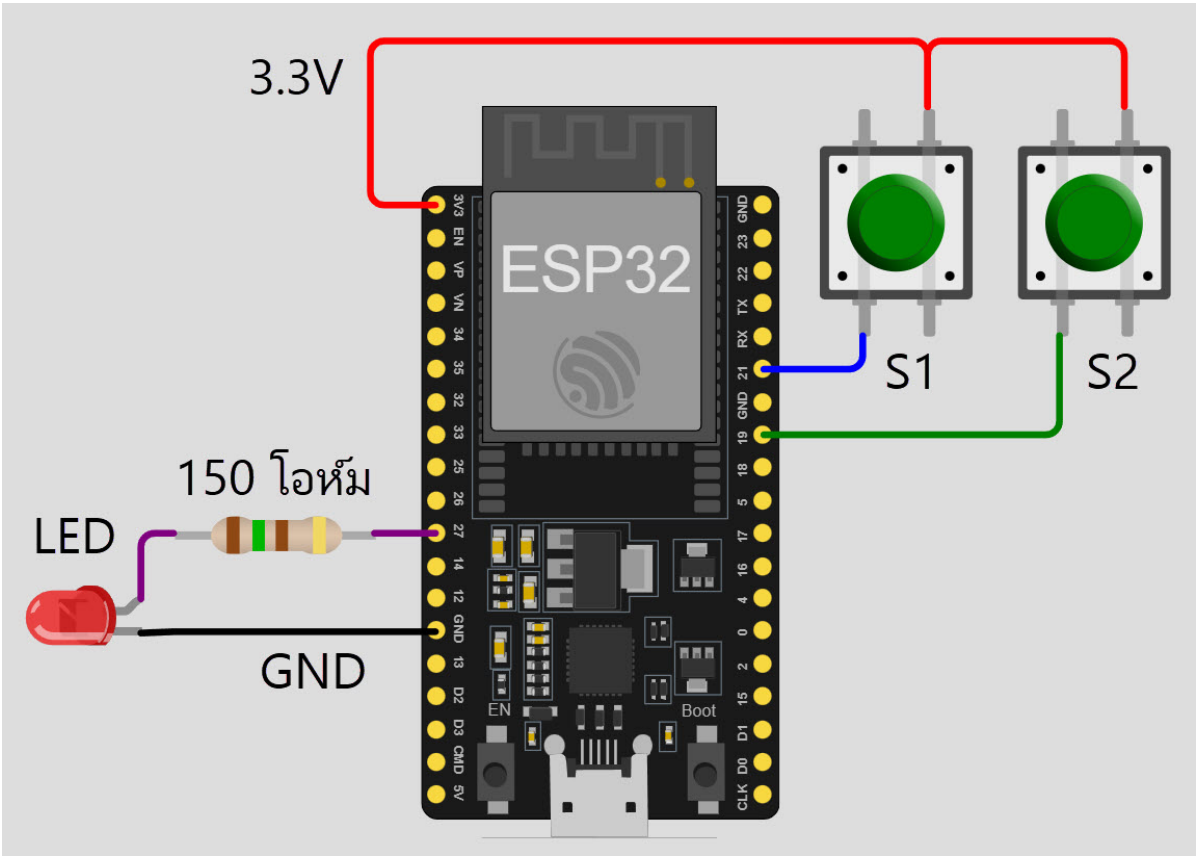
ทุกครั้งที่เขียนโค้ดเสร็จแล้วอย่าลืมกดปุ่ม Run หรือ F5 เพื่อดูผลการทำงานของโปรแกรม

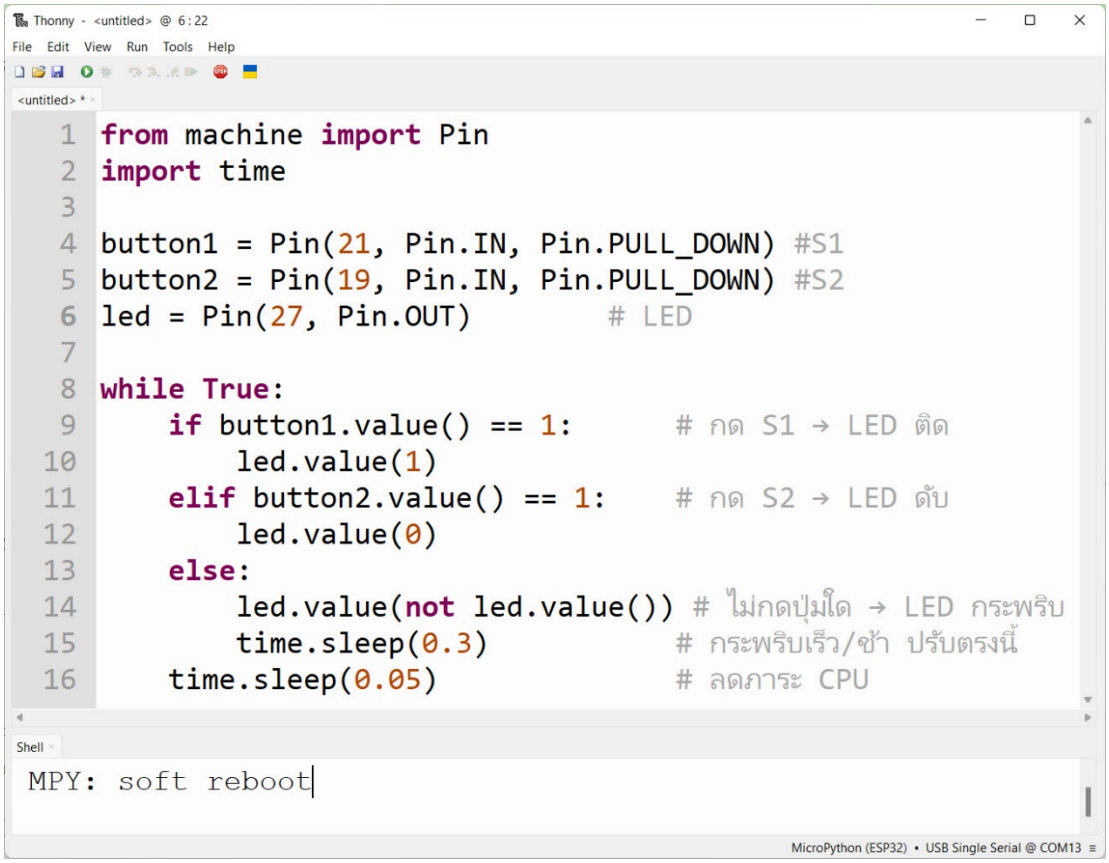
3. คำสั่ง if–elif–else (หลายเงื่อนไข) หากมีหลายเงื่อนไขใช้ elif ไปเรื่อย ๆ

```
if เงื่อนไข1:
    คำสั่งเมื่อเงื่อนไข1 เป็นจริง
elif เงื่อนไข2:
    คำสั่งเมื่อเงื่อนไข2 เป็นจริง
else:
    คำสั่งเมื่อไม่มีเงื่อนไขใดเป็นจริงเลย
```

ตัวอย่าง:

กดปุ่ม 1 ให้ LED ติด กดปุ่ม 2 ให้ LED ดับ ไม่กดปุ่มใด LED กระพริบ

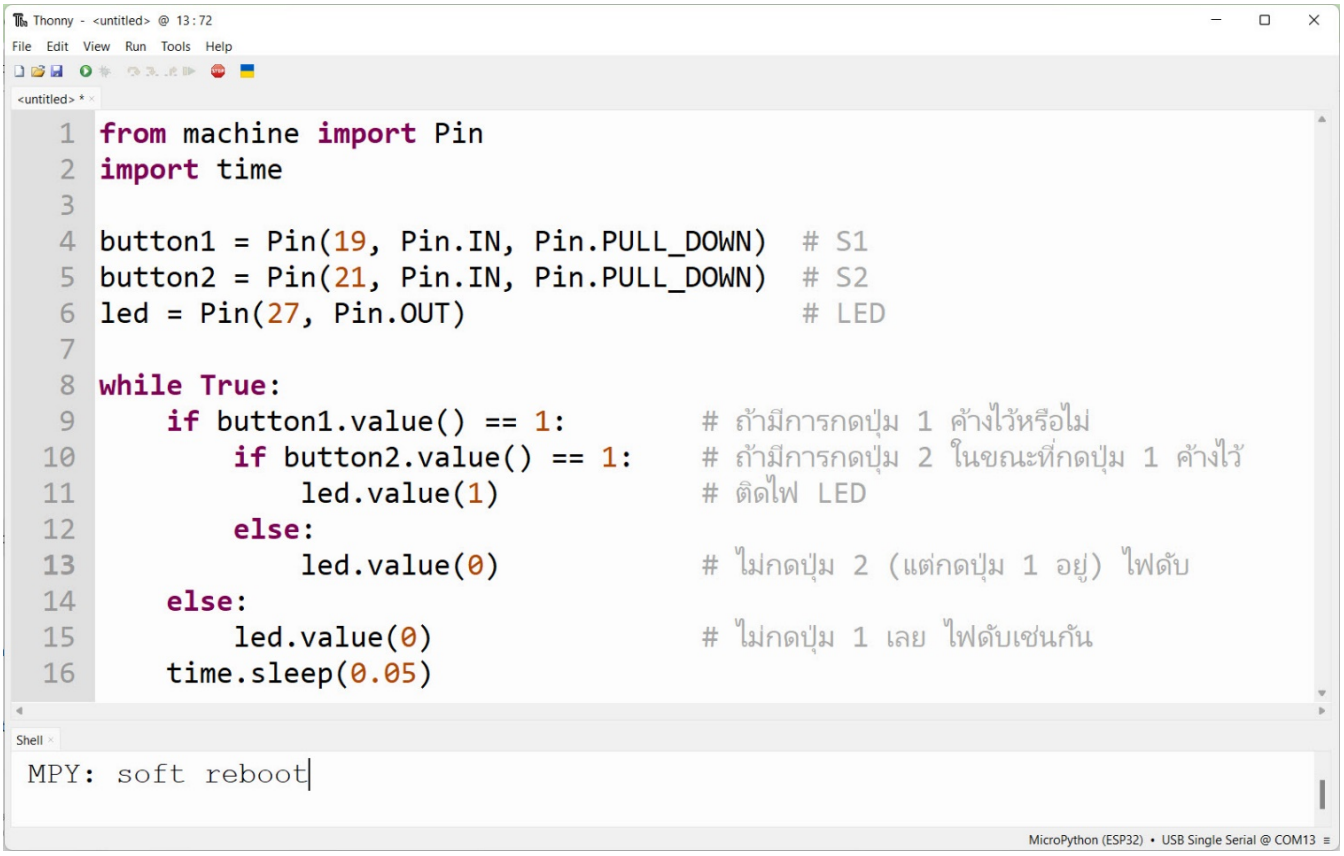




4. การซ้อนคำสั่ง if (Nested if) ให้ใช้วงจรเดิมในการทดลอง

```
if เงื่อนไข1:
    if เงื่อนไข2:
        คำสั่งเมื่อทั้งสองเงื่อนไขเป็นจริง
```

ตัวอย่าง: ต้องกดปุ่ม 1 ค้างไว้ แล้วกดปุ่ม 2 พร้อมกัน LED จึงจะติด



คำสั่งเงื่อนไข (if, elif, else) คือรากฐานของการเขียนโปรแกรมที่ “ตอบสนองต่อเหตุการณ์” อย่างอัตโนมัติในงานควบคุมจริง

คำสั่งทำซ้ำแบบไม่จำกัดจำนวนรอบ: while

ในโลกของการเขียนโปรแกรมสำหรับงานควบคุมไมโครคอนโทรลเลอร์และระบบสมองกลฝังตัว (Embedded Systems) นั้น “โครงสร้างการทำซ้ำ” หรือ Loop ถือเป็นหัวใจสำคัญที่ขาดไม่ได้ โดยเฉพาะอย่างยิ่งในงานที่ต้องการให้โปรแกรมตรวจสอบเหตุการณ์ต่าง ๆ อย่างต่อเนื่อง เช่น การอ่านค่าจากเซ็นเซอร์ การเฝ้ารอการกดปุ่ม การควบคุมอุปกรณ์ หรือแม้กระทั่งการเชื่อมต่อกับเครือข่าย IoT จุดเด่นของไมโครคอนโทรลเลอร์ก็คือความสามารถในการทำงานซ้ำเดิม ๆ ได้อย่างไม่รู้จบ โดยไม่ต้องมีผู้ใช้มากดสั่งงานซ้ำเองทุกครั้ง ซึ่งความสามารถนี้เกิดขึ้นได้ก็เพราะ “ลูป” ที่อยู่ในโปรแกรมนั่นเอง

โครงสร้างลูปที่พบมากที่สุดในภาษา MicroPython คือคำสั่ง while โดยเฉพาะโครงสร้าง while True: ซึ่งกลายเป็นมาตรฐานของงานควบคุมระบบฝังตัวทุกระดับ เพราะ True เป็นค่าคงที่ที่ไม่เปลี่ยนแปลง โปรแกรมจึงเข้าสู่บล็อกของ while ซ้ำไปเรื่อย ๆ อย่างไม่รู้จบ จนกว่าจะถูกสั่งให้หยุดโดยเงื่อนไขบางอย่าง หรือจนกว่าผู้ใช้จะรีเซ็ตบอร์ดหรือหยุดโปรแกรมเอง หลักการนี้ต่างจากการเขียนโปรแกรมบนคอมพิวเตอร์ทั่วไป ที่มักมีจุดเริ่มต้นและจุดจบที่ชัดเจน แต่ในระบบควบคุมจริง โปรแกรมจำเป็นต้องทำงาน 24 ชั่วโมงเพื่อรับมือกับเหตุการณ์ต่าง ๆ ที่เกิดขึ้นแบบ Real-Time ตลอดเวลา

โครงสร้างพื้นฐานของ while ใน MicroPython มีลักษณะดังนี้

while เงื่อนไข:

คำสั่งที่ต้องการทำซ้ำ

ก่อนเข้ารอบใหม่ทุกครั้ง เงื่อนไขจะถูกตรวจสอบ หากเงื่อนไขยังเป็นจริง โปรแกรมจะวนเข้าบล็อกใน while ทุกครั้ง แต่ถ้าเป็นเท็จจะออกจากลูปไปทำงานหลัง while ต่อทันที ดังนั้นถ้าเขียนว่า while True: ก็จะหมายถึง โปรแกรมจะทำซ้ำไม่รู้จบจนกว่าจะมีคำสั่งให้หยุด (เช่น break) หรือจนกว่าไมโครคอนโทรลเลอร์จะดับหรือถูกรีเซ็ต

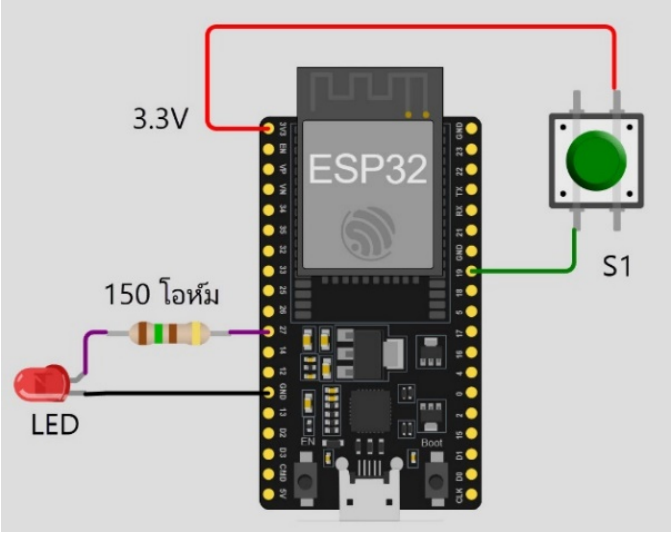
หัวใจสำคัญของลูป while True: ก็คือ “การเฝ้าตรวจสอบ” หรือ Polling Input/Output อย่างต่อเนื่อง เช่น ตรวจสอบสถานะปุ่มกด ตรวจค่าจากเซ็นเซอร์ ตรวจสอบว่ามีข้อมูลใหม่เข้ามาหรือไม่ หรือสั่งงานอุปกรณ์เอาต์พุตแบบทันที ถ้าไม่มีลูปนี้ โปรแกรมจะทำงานเพียงครั้งเดียวแล้วหยุด ส่งผลให้ระบบไม่ตอบสนองต่อเหตุการณ์ใหม่ ๆ เลย ลูป while True: จึงถูกใช้เป็นโครงสร้างหลักในทุกโปรแกรมของไมโครคอนโทรลเลอร์ โดยเฉพาะกับบอร์ด ESP32, ESP8266, Arduino (ในรูปแบบอื่น) และระบบควบคุมอัตโนมัติทุกประเภท

ตัวอย่างการนำคำสั่งวนซ้ำ while ไปใช้ในงานจริง

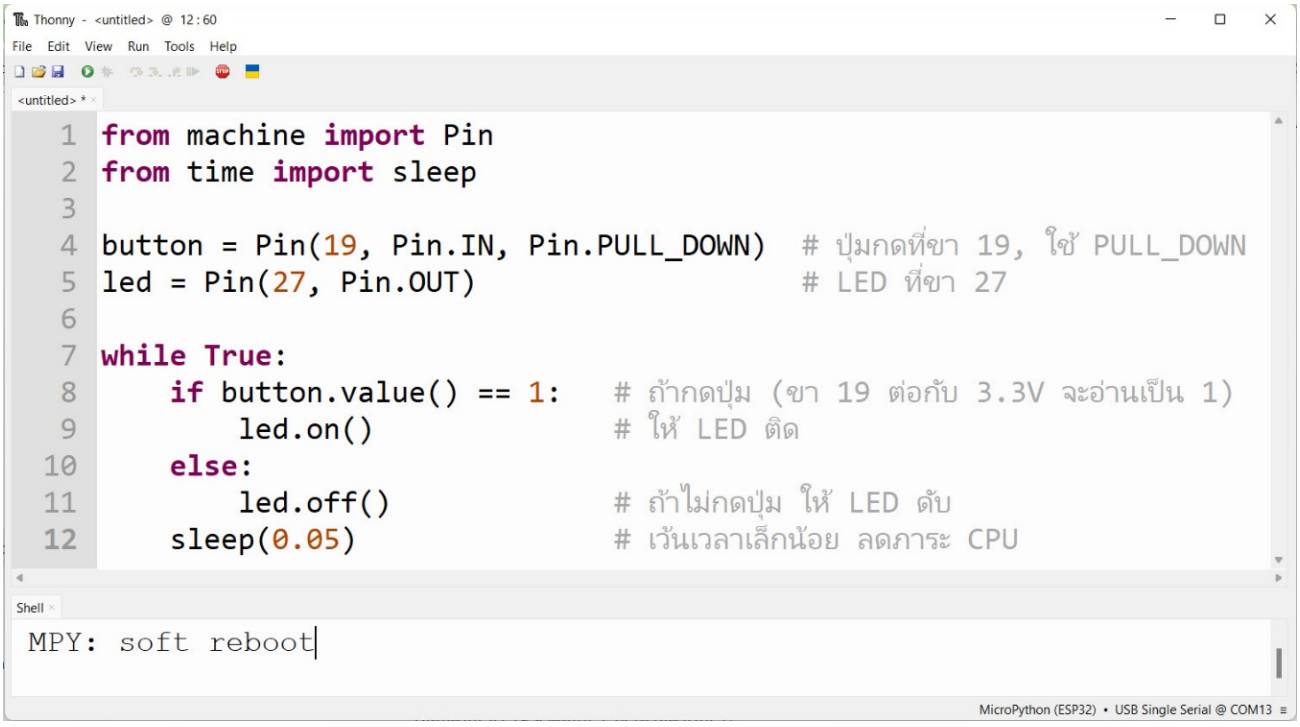
ยกตัวอย่างสถานการณ์จริง เช่น ต้องการตรวจสอบสถานะปุ่มกดหรือสวิตช์อยู่ตลอดเวลา หนึ่งในงานที่ใช้บ่อยมากในระบบฝังตัวก็คือ การสั่งให้ LED ติดเมื่อกดปุ่ม และดับเมื่อปล่อยปุ่ม หากไม่มีลูป โปรแกรมจะตรวจสอบสถานะปุ่มเพียงครั้งเดียวตั้งแต่เปิดเครื่อง ถ้าในเวลานั้นไม่ได้กดปุ่มเลย โปรแกรมก็จะไม่ตอบสนองอีกต่อไป แต่ถ้าใช้ while True: โปรแกรมจะคอยอ่านสถานะปุ่มและควบคุม LED ตามต้องการตลอดเวลา

- **กดปุ่ม ⟶ ให้ LED ติด**
- **ปล่อยปุ่ม ⟶ ให้ LED ดับ**

การทดลองเราจะใช้วงจรเดิมที่ใช้กับ if เพราะคำสั่งวนซ้ำนั้นมักถูกใช้งานควบคู่กับ if...else



การเขียนโปรแกรมให้ตรวจสอบสถานะของปุ่มตลอดเวลาจะต้องใช้ “ลูปไม่รู้จบ” เช่น while True: เพื่อวนตรวจสอบอินพุตนี้ซ้ำ ๆ ตลอดเวลา โดยการอ่านค่าอินพุตและตัดสินใจว่าจะสั่งงานอุปกรณ์ขาออกหรือไม่



โค้ดข้างต้นจะแสดงการตรวจสอบปุ่มและควบคุม LED ตลอดเวลา ถ้ากดปุ่ม LED จะติด ปล่อยปุ่ม LED จะดับ โปรแกรมจะวนตรวจสอบสถานะนี้ไปเรื่อย ๆ โดยไม่หยุดจนกว่าจะปิดบอร์ดหรือรีเซ็ต ในตัวอย่างนี้จะเห็นได้ว่าคำสั่ง while True: ทำให้ระบบตอบสนองต่อเหตุการณ์ภายนอกได้อย่าง Real-Time ไม่พลาดแม้แต่จังหวะเดียว และสามารถปรับเปลี่ยนไปควบคุมอุปกรณ์หรืออ่านค่าเซ็นเซอร์อื่น ๆ ได้ทันทีในบล็อกของลูปเดียวกัน

นอกจากการตรวจสอบปุ่มแล้ว โครงสร้าง while True: ยังถูกนำไปใช้กับโปรเจกต์จริงในหลายรูปแบบ เช่น การอ่านค่าเซ็นเซอร์ตลอดเวลา ไม่ว่าจะเป็นอุณหภูมิ ความชื้น หรือการเคลื่อนไหว เพื่อนำข้อมูลไปประมวลผลหรือส่งขึ้น Cloud หรือใช้ควบคุมอุปกรณ์เอาต์พุต เช่น มอเตอร์ รีเลย์ หรือหลอดไฟฟ้า ตามเงื่อนไขที่ผู้ใช้ต้องการ ในระบบ IoT จริง โปรแกรมจะต้องรอเชื่อมต่อ WiFi หรือรอคำสั่งจาก Server ตลอดเวลา โดยทุกอย่างต้องอยู่ในลูป while True: ทั้งหมด

อย่างไรก็ตาม การเขียนโปรแกรมให้มีประสิทธิภาพสูงในขณะที่ใช้ while True: ควรระวังไม่ให้ใส่คำสั่งที่กินเวลานานหรือมีการหน่วงที่ไม่จำเป็นในลูป เพราะจะทำให้ระบบตอบสนองช้า หรือลดประสิทธิภาพของ CPU ควรออกแบบให้ลูปสั้น กระชับ ตรวจสอบอินพุตได้เร็ว และเพิ่มคำสั่ง sleep() ที่เหมาะสมเพื่อลดภาระของหน่วยประมวลผล ในกรณีที่ต้องการหยุดลูปอย่างถูกต้องควรใส่เงื่อนไขพิเศษ (เช่น มีปุ่มหยุด หรือค่าเซ็นเซอร์ถึงจุดอันตราย) แล้วออกจากลูปด้วย break

จุดแข็งของ while True: คือทำให้โปรแกรมควบคุมสามารถตรวจสอบและตอบสนองต่อเหตุการณ์จากปุ่ม เซ็นเซอร์ หรืออุปกรณ์ใด ๆ ได้ตลอดเวลา เหมาะกับการออกแบบระบบอัตโนมัติทุกประเภทตั้งแต่ของเล่นจนถึงอุตสาหกรรมจริง และยังง่ายต่อการขยายฟังก์ชันหรือเพิ่มความสามารถใหม่ ๆ ในโปรเจกต์โดยตรงในบล็อก while เดียวกัน

คำสั่งทำซ้ำแบบจำนวนรอบที่แน่นอน: for

นอกจากการทำซ้ำแบบไม่จำกัดรอบแล้ว ในการควบคุมไมโครคอนโทรลเลอร์ ยังมีหลายสถานการณ์ที่ต้องการสั่งให้โปรแกรม “ทำซ้ำ” คำสั่งเดิมจำนวนรอบที่แน่นอน เช่น สั่งให้ LED กระพริบ 10 ครั้ง อ่านค่าจากเซ็นเซอร์ 5 รอบแล้วนำมาคำนวณเฉลี่ย หรือวนแสดงข้อมูลในรายการต่าง ๆ อย่างมีขอบเขตชัดเจน

คำสั่ง for คือโครงสร้างที่ออกแบบมาสำหรับงานลักษณะนี้โดยเฉพาะ ในภาษา MicroPython โครงสร้าง for จะวนซ้ำตามจำนวนข้อมูลหรือรอบที่กำหนดในแต่ละรอบอย่างแม่นยำ เหมาะกับงานที่เราทราบจำนวนครั้งที่ต้องการวนซ้ำล่วงหน้า ไม่ว่าจะเป็นการทำซ้ำด้วย range() เพื่อระบุจำนวนรอบ หรือการวนผ่านรายการ (list) เพื่อประมวลผลข้อมูลทีละตัว รูปแบบคำสั่ง for ที่นิยมใช้ใน MicroPython คือ

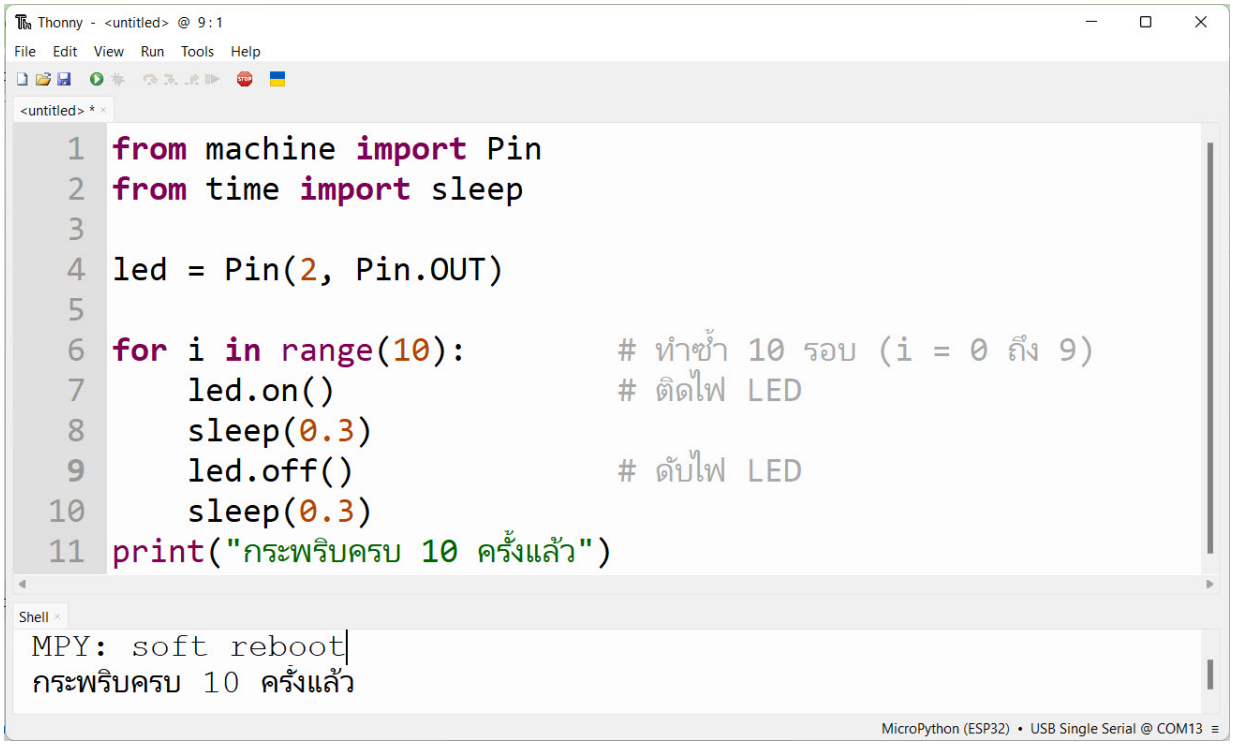
```
for ตัวแปร in ชุดข้อมูล:
    คำสั่งที่ต้องการให้ทำซ้ำ
```

โดย “ชุดข้อมูล” ที่ใช้งานบ่อยที่สุดก็คือ **range()** ซึ่งเป็นฟังก์ชันสร้างชุดตัวเลขจำนวนรอบตามที่ต้องการ

ตัวอย่างการใช้ **range** ในงานควบคุม

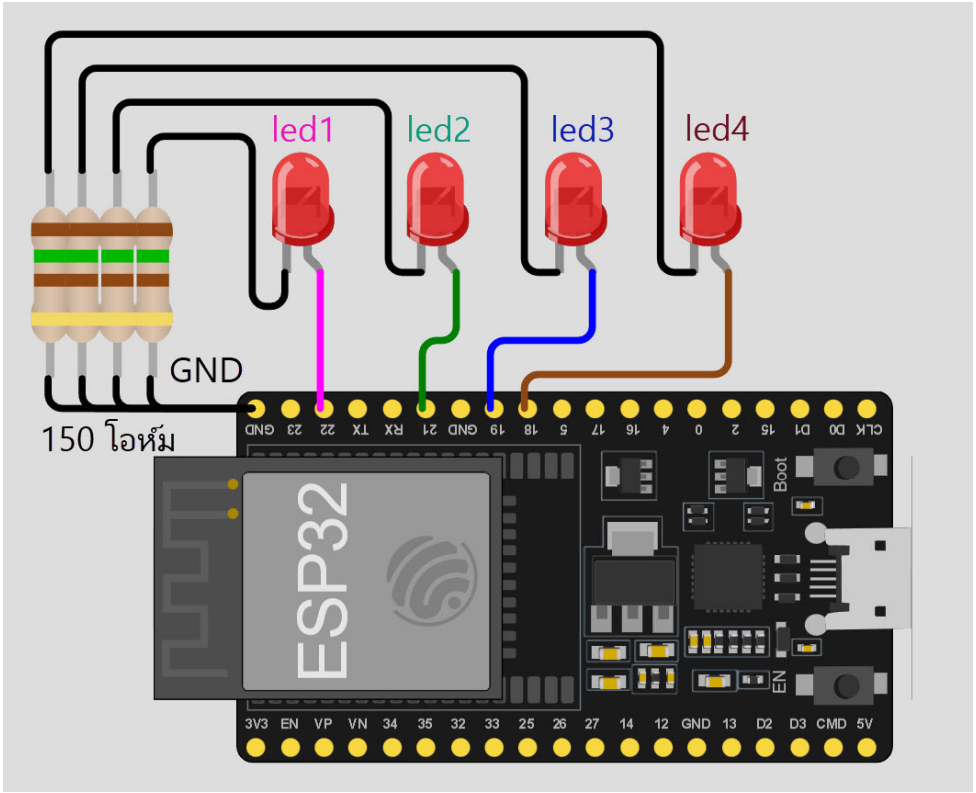
- **range(stop):** สร้างลิสต์ตัวเลขตั้งแต่ 0 ถึง (stop - 1)
- **range(start, stop):** สร้างลิสต์ตัวเลขตั้งแต่ start ถึง (stop - 1)
- **range(start, stop, step):** สร้างลิสต์ตัวเลขโดยเพิ่มทีละ step

📌 ตัวอย่าง 1: กระพริบ LED จำนวน 10 ครั้ง

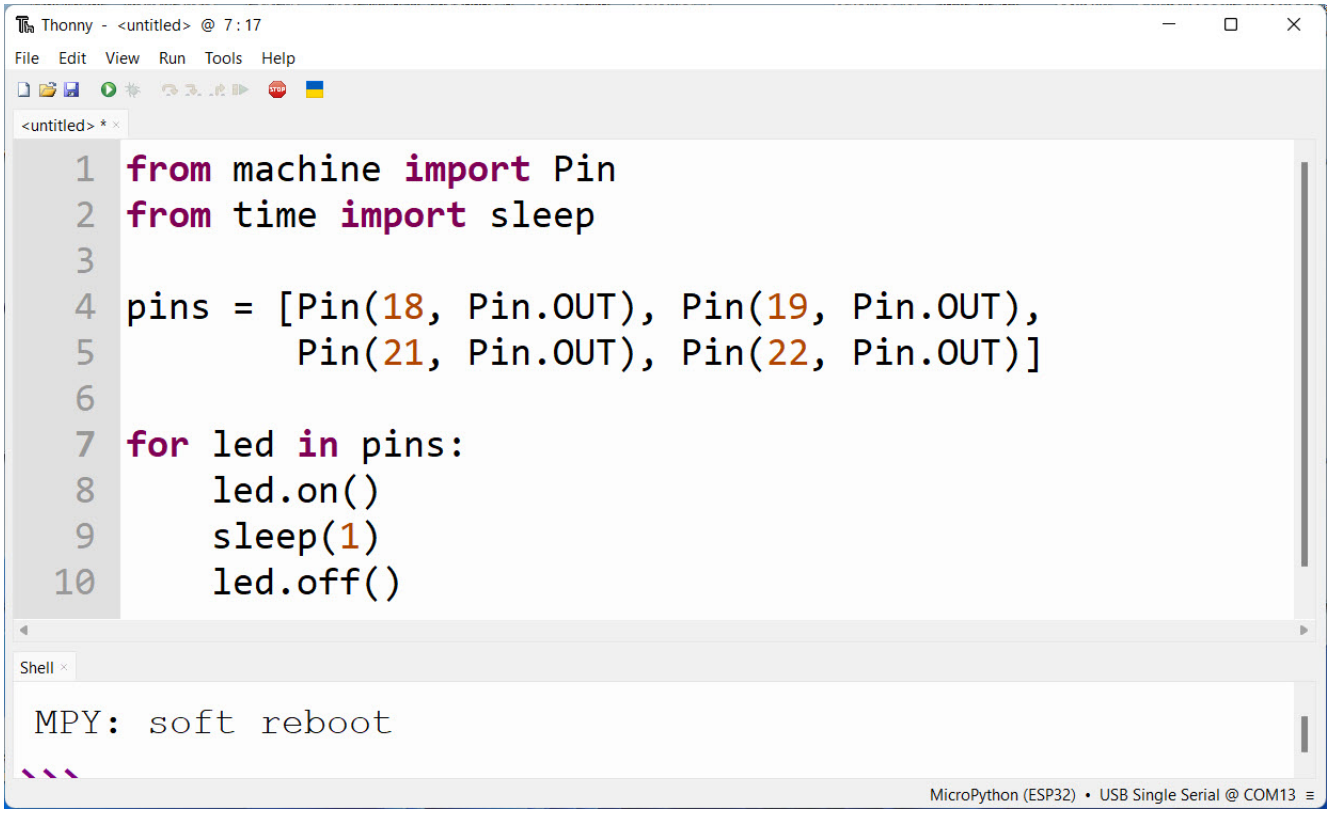


ตัวอย่างนี้ใช้ for ควบคุมให้ไฟ LED กระพริบ 10 ครั้งพอดี ซึ่งสะดวกและอ่านง่ายกว่า while สำหรับงานที่ต้องกำหนดจำนวนรอบแน่นอน

📌 ตัวอย่าง 2: วนค่าจากรายการเพื่อควบคุม LED หลายขา



อีกตัวอย่างหนึ่งที่ใช้บ่อยในงานควบคุมอุปกรณ์คือ การวนผ่านลิสต์ขา GPIO หลายขาเพื่อสั่งให้ LED แต่ละดวงติดเรียงกัน (เช่นโปรเจกต์ไฟวิ่ง 4 ดวง) สามารถเขียนแบบนี้ได้



โปรเจกต์ที่ควบคุมไฟวิ่งแบบเรียงลำดับตัวอย่างนี้ ไฟจะติด และดับ เรียงกันไปทีละดวง แต่ทำงานรอบเดียว

ข้อดีของ for คือ สามารถวนซ้ำผ่านลิสต์หรือข้อมูลหลายตัวได้อย่างยืดหยุ่นและปลอดภัย โดยที่ไม่ต้องสร้างตัวนับหรือจัดการกับเงื่อนไขเองเหมือนกับ while โครงสร้างนี้ยังเหมาะกับงานที่ต้องประมวลผลข้อมูลจำนวนมาก หรืองานที่ต้องการอ่านค่าจากหลายเซ็นเซอร์ในแต่ละรอบ และยังสามารถประยุกต์ใช้งานร่วมกับ enumerate() เพื่อระบุตำแหน่งหรือจัดการข้อมูลหลายรายการในโปรเจกต์ขนาดใหญ่ได้อย่างมีประสิทธิภาพ

การฝึกใช้ for-loop ร่วมกับ range() และ enumerate() จะช่วยให้โค้ดในงานควบคุม ESP32, IoT หรือไมโครคอนโทรลเลอร์รุ่นอื่น ๆ มีความกระชับ อ่านง่าย และพร้อมรองรับการขยายงานในอนาคตได้อย่างมืออาชีพ

การควบคุมลำดับการทำงานของลูป: break และ continue

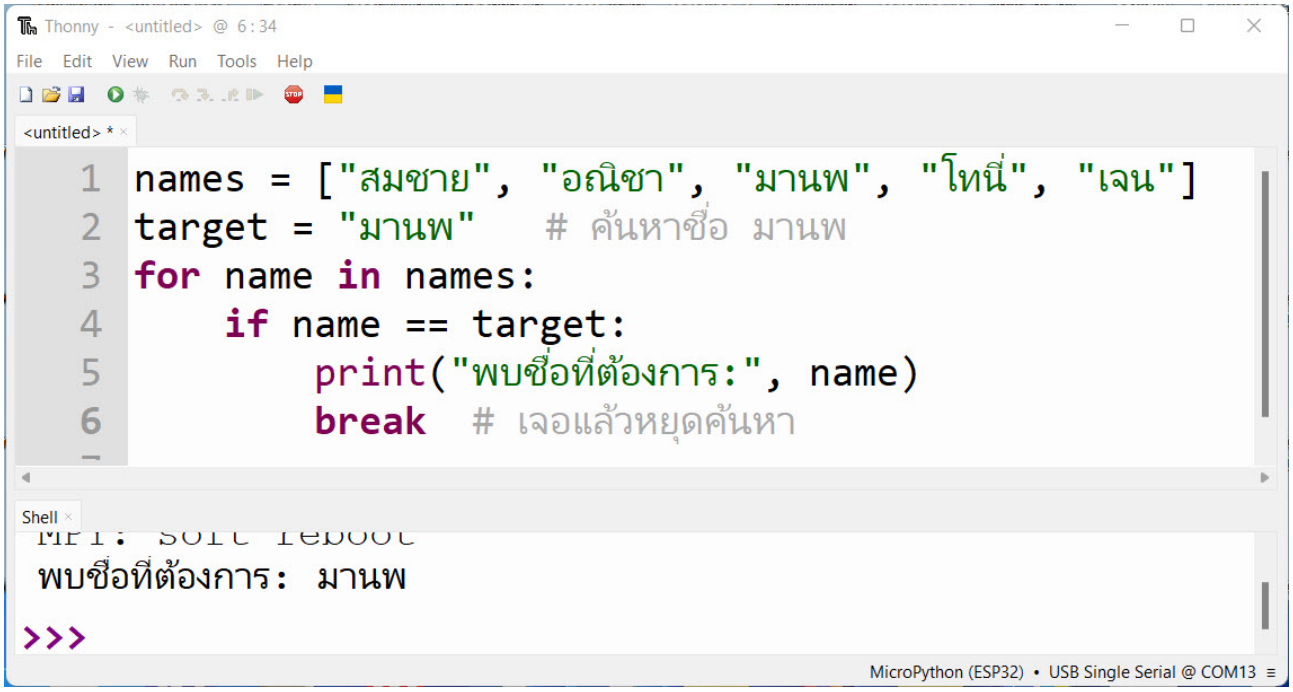
การเขียนโปรแกรมควบคุมอุปกรณ์หรือระบบ IoT ด้วย MicroPython จำเป็นต้องเข้าใจกลไกการทำซ้ำ หรือที่เรียกว่าลูป (Loop) อย่างลึกซึ้ง เพราะโครงสร้างลูปถือเป็นหัวใจสำคัญที่ทำให้โปรแกรมสามารถทำงานซ้ำ ๆ ได้โดยอัตโนมัติ ไม่ว่าจะเป็นการตรวจสอบสถานะเซ็นเซอร์ การรับค่าจากผู้ใช้ หรือการสั่งงานอุปกรณ์ในระบบควบคุมต่าง ๆ ลูปในโปรแกรมทำหน้าที่เหมือนกับวงจรชีวิตที่ดำเนินไปอย่างต่อเนื่อง หากไม่มีลูป โปรแกรมก็จะไม่สามารถตอบสนองต่อการเปลี่ยนแปลงต่าง ๆ ในระบบจริงได้

แต่การที่ลูปจะตอบสนองต่อเหตุการณ์ต่าง ๆ ได้อย่างถูกต้องและมีประสิทธิภาพนั้น ไม่ใช่แค่การทำซ้ำคำสั่งเดิมไปเรื่อย ๆ เท่านั้น เพราะในโลกแห่งความจริง สถานการณ์ที่เกิดขึ้นอาจต้องการให้โปรแกรมหยุดการทำงานทันที หรือข้ามการทำงานบางส่วนไป โดยไม่ต้องรอให้จบลูปแต่ละรอบเสมอไป นี่จึงเป็นที่มาของคำสั่ง break และ continue ซึ่งถือเป็นเครื่องมือสำคัญในการควบคุมลำดับการทำงานของลูปให้มีความยืดหยุ่นและตอบสนองต่อเหตุการณ์ต่าง ๆ ได้อย่างชาญฉลาด

หากเปรียบเทียบกับการทำงานในชีวิตประจำวัน เช่น การค้นหารายชื่อเพื่อนในห้องเรียน สมมติครูมีรายชื่ออยู่ 30 คน และต้องการค้นหาเพื่อนชื่อ “มานพ” ในรายการชื่อเหล่านั้น หากใช้วิธีไล่ตรวจทีละชื่อโดยไม่หยุดจนกว่าจะถึงชื่อสุดท้าย ก็คงจะเสียเวลาโดยใช่เหตุ ทั้งที่จริง ๆ แล้ว หากพบชื่อ “มานพ” เมื่อไร ก็ควรหยุดค้นหาทันที เพราะได้ผลลัพธ์ที่ต้องการแล้ว นี่คือหลักการเดียวกับการใช้คำสั่ง break ในโปรแกรม เมื่อลูปทำงานถึงจุดที่ต้องการ ก็สามารถสั่งให้หยุดลูปทันทีโดยไม่ต้องรอให้จบรอบทั้งหมด

ในทางตรงกันข้าม ถ้าระหว่างตรวจสอบรายชื่อในห้องเรียนแล้วพบว่านักเรียนบางคนไม่มาเรียน หรือข้อมูลไม่สมบูรณ์ ครูอาจจะข้ามชื่อนั้นไปก่อน แล้วไปตรวจชื่อถัดไปโดยไม่ต้องเสียเวลากับข้อมูลที่ไม่สมบูรณ์ หลักการนี้ตรงกับการใช้คำสั่ง continue ในโปรแกรม กล่าวคือ เมื่อโปรแกรมเจอข้อมูลที่ไม่ต้องการประมวลผล ก็สามารถข้ามรอบนั้น แล้วเริ่มรอบถัดไปทันที ช่วยให้โปรแกรมทำงานได้อย่างมีประสิทธิภาพ และไม่หยุดชะงักเพราะข้อมูลผิดปกติ

โครงสร้างของคำสั่ง break และ continue ใน MicroPython จะใช้ร่วมกับลูปทั้ง for และ while โดย break จะหยุดการทำงานของลูปในทันทีเมื่อเข้าเงื่อนไขที่กำหนด ส่วน continue จะข้ามคำสั่งที่เหลือในลูปรอบนั้น แล้วไปเริ่มต้นรอบใหม่ต่อไป ตัวอย่างเช่น ในการค้นหาชื่อ “มานพ” ใน list รายชื่อ หากใช้โค้ดดังนี้

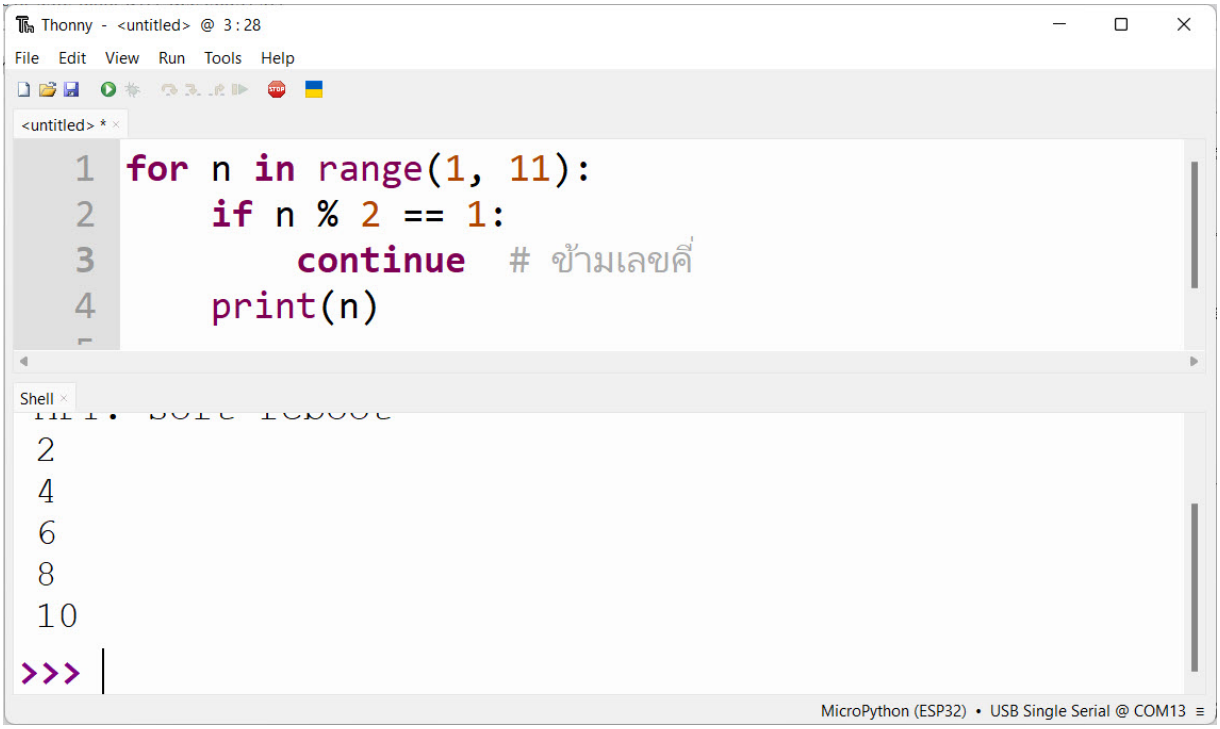


โปรแกรมจะวนลูปตรวจสอบชื่อทีละคน เมื่อเจอชื่อ “มานพ” จะหยุดการค้นหาโดยทันที ไม่ต้องเช็คชื่ออื่นที่เหลือ ซึ่งสะท้อนแนวคิดของการเพิ่มประสิทธิภาพให้กับการทำงานของโปรแกรมอย่างชัดเจน ตัวอย่างนี้หากนำไปประยุกต์กับระบบ IoT ก็เช่น การอ่านค่าจากเซ็นเซอร์หลายตัว เมื่อพบค่าที่ผิดปกติหรือถึงจุดที่ต้องการแล้ว ก็หยุดการทำงานของลูปทันทีเพื่อประหยัดเวลาและพลังงานของระบบ

ในตัวอย่างโค้ดต่อไปนี้จะแสดงถึงการนำโครงสร้าง for-loop มาใช้ร่วมกับคำสั่ง continue เพื่อเลือกทำงานเฉพาะบางกรณีเท่านั้น ภายใต้สถานการณ์ที่ต้องการแสดงผลเฉพาะเลขคู่จากกลุ่มตัวเลข 1 ถึง 10 โปรแกรมจะเริ่มวนซ้ำค่าตัวแปร n ตั้งแต่ 1 ไปจนถึง 10 ทีละรอบ โดยอัตโนมัติ ซึ่งแต่ละรอบของลูป จะมีการนำค่าของ n ในรอบนั้นมาพิจารณาว่าเป็นเลขคู่หรือเลขคี่ก่อนจะดำเนินการใด ๆ ต่อไป หลักการตัดสินใจที่ใช้งานในที่นี้ คือการแบ่งเลขคู่เลขคี่โดยอาศัย “การหารเอาเศษ” (modulo operation) ซึ่งเป็นแนวคิดทางคณิตศาสตร์พื้นฐานในเกือบทุกภาษาคอมพิวเตอร์

เครื่องหมายร้อยละ (%) ในภาษา Python และ MicroPython ใช้สำหรับคำนวณ “เศษที่เหลือ” จากการหารค่าทางซ้ายด้วยค่าทางขวา เช่น n % 2 จะเป็นการนำ n ไปหาร 2 แล้วดูว่าเหลือเศษเท่าไร หากเศษที่เหลือคือ 0 หมายถึง n เป็นเลขคู่ เพราะเลขคู่เมื่อหารด้วย 2 จะไม่มีเศษ แต่ถ้าเหลือเศษ 1 แสดงว่า n เป็นเลขคี่ เพราะเลขคี่เมื่อหารด้วย 2 จะต้องเหลือเศษ 1 เสมอ ตัวอย่างเช่น เมื่อ n = 4 จะได้ 4 % 2 เท่ากับ 0 แสดงว่า 4 เป็นเลขคู่ แต่ถ้า n = 5 จะได้ 5 % 2 เท่ากับ 1 แสดงว่า 5 เป็นเลขคี่

โค้ดตัวอย่างที่นำมาใช้จะมีลักษณะดังนี้



ในโปรแกรมนี้ลูป for จะรันรอบแรกด้วย n = 1 แล้วตรวจสอบว่า n % 2 == 1 หรือไม่ ซึ่งสำหรับ n = 1 จะพบว่ามีเศษ 1 จึงเข้าเงื่อนไข if และเจอคำสั่ง continue ทำให้โปรแกรมข้ามการแสดงผล n ในรอบนี้ไปเลย

และจะวนลูปรอบต่อไปทันที เมื่อ n เป็นเลขคู่ เช่น n = 2 เมื่อนำ 2 % 2 จะได้เศษ 0 ซึ่งไม่เข้าเงื่อนไข if โปรแกรมจึงข้ามคำสั่ง continue ไปทำงาน print(n) ทำให้เลข 2 ถูกแสดงออกมาทางหน้าจอ จากนั้นโปรแกรมจะดำเนินการในลูป for ต่อไปกับ n = 3, 4, ... จนครบทุกค่า

หากวิเคราะห์ลำดับการทำงานแต่ละรอบจะเห็นว่า โปรแกรมจะข้ามการทำงานในรอบที่ n เป็นเลขคี่ทั้งหมด ได้แก่ 1, 3, 5, 7, 9 โดยจะไม่มีการแสดงผลในรอบเหล่านี้ ส่วนรอบที่ n เป็นเลขคู่ เช่น 2, 4, 6, 8, 10 โปรแกรมจะสั่ง print(n) ให้แสดงตัวเลขเหล่านี้ออกมาในแต่ละรอบตามลำดับ จึงสรุปได้ว่า ผลลัพธ์ของโปรแกรมนี้จะได้ตัวเลข 2, 4, 6, 8 และ 10 ปรากฏบนหน้าจอทีละบรรทัดโดยไม่ปรากฏเลขคี่ใด ๆ เลย

เพื่อให้เห็นภาพชัดเจนขึ้น หากเปรียบเทียบกับสถานการณ์ในชีวิตจริง สามารถอธิบายได้เหมือนกับครูที่ต้องการแจกขนมให้กับนักเรียนที่นั่งอยู่ในห้อง โดยครูเดินแจกขนมทีละคน ไล่ตามหมายเลขประจำตัวตั้งแต่ 1 ถึง 10 ในแต่ละรอบก่อนแจกขนมจะตรวจสอบว่า หมายเลขที่นั่งนั้นเป็นเลขคู่หรือเลขคี่ หากเป็นเลขคี่ครูก็ข้ามนักเรียนคนนั้นไปโดยไม่แจกขนม แล้วเดินไปหาเด็กคนถัดไปทันที แต่หากเป็นเลขคู่จึงจะยื่นขนมให้เด็กคนนั้น ผลที่ได้คือ นักเรียนที่มีหมายเลขที่นั่งเป็นเลขคี่จะไม่ได้รับขนม ส่วนเด็กที่หมายเลขคู่จะได้รับขนมครบถ้วนตามที่ต้องการ นี่คือการเลือกปฏิบัติเฉพาะบางกรณีด้วยการ “ข้ามรอบ” เหมือนกับแนวคิดของ continue ในลูป for-loop ในโปรแกรมข้างต้น

การใช้เครื่องหมาย % ในการตรวจสอบเลขคู่เลขคี่เช่นนี้เป็นเทคนิคที่ใช้กันอย่างแพร่หลายในวงการเขียนโปรแกรม เพราะสามารถประยุกต์ใช้กับการตรวจสอบคุณสมบัติต่าง ๆ ของตัวเลข เช่น หารด้วย 3 แล้วเหลือเศษ 0 เพื่อหาตัวเลขที่หารด้วย 3 ลงตัว หรือใช้กับการคัดกรองข้อมูลในเซ็นเซอร์หรือการประมวลผลข้อมูลชุดใหญ่ในโปรแกรมควบคุมอัตโนมัติ ตัวอย่างที่นำเสนอจึงไม่ได้จำกัดแค่การเลือกเลขคู่เลขคี่เท่านั้น แต่สามารถต่อยอดไปสู่สถานการณ์ควบคุมที่ซับซ้อนขึ้นในระบบจริงได้อีกมาก

🚧 คำสั่ง pass ใน MicroPython

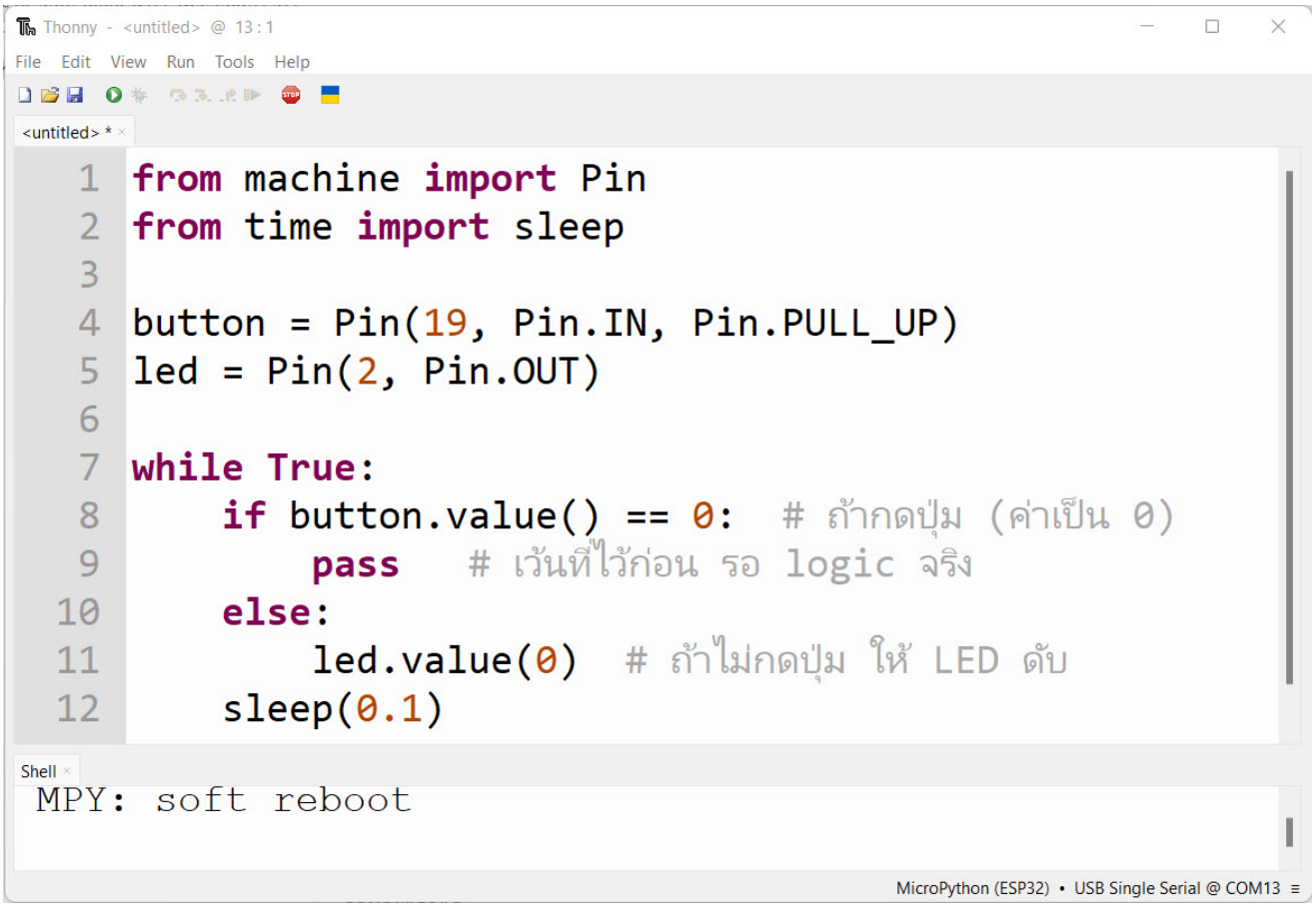
ในการเขียนโปรแกรมด้วย MicroPython โดยเฉพาะงานควบคุมไมโครคอนโทรลเลอร์ เช่น ESP32 จะพบว่างานแต่ละชิ้นมักถูกแยกออกเป็น “โครงสร้างควบคุม” ย่อย ๆ ไม่ว่าจะเป็นบล็อก if-else, while, for หรือแม้แต่การประกาศฟังก์ชันเฉย ๆ โดยยังไม่ได้เติมคำสั่งลงไปในช่วงแรก ๆ ของการออกแบบโปรแกรม หลายครั้งที่ผู้พัฒนาอาจต้องการเว้นบล็อกว่างเอาไว้ก่อน เพราะยังคิด logic ไม่จบ หรือยังรอข้อมูลเพิ่มเติม หรือกำลังเขียนโครงสร้างภาพรวมของโปรแกรมทีละส่วน ในภาษา Python/MicroPython ถ้าเว้นว่างบล็อกใด ๆ หลัง if, else, while, for หรือ def โดยไม่มีคำสั่งเลย ระบบจะฟ้อง error ในทันที เพราะถือว่าโครงสร้างไม่สมบูรณ์

ที่นี่เองที่คำสั่ง **pass** มีบทบาทสำคัญ **pass** ถูกใช้เพื่อ “จองที่” หรือ “เว้นว่าง” ในบล็อกควบคุมเหล่านี้ ทำให้โครงสร้างโปรแกรมยังคงสมบูรณ์ สามารถรันและทดลองได้โดยไม่เกิด error แม้จะยังไม่ได้กำหนดคำสั่งจริง ๆ ในบล็อกนั้นก็ตาม การใช้ pass ในงานจริงจึงเหมือนกับการวางโครงร่างของหนังสือที่ใส่หัวข้อไว้แต่ยังไม่ได้ลงรายละเอียด หรือเหมือนวาดแผนที่โรงงานโดยมีห้องบางห้องที่ยังไม่แน่ใจว่าจะใช้ทำอะไร จึงขีดกรอบไว้ก่อนเพื่อรอเติมฟังก์ชันในภายหลัง หัวใจของการใช้ pass คือ **ช่วยให้โปรแกรมเมอร์สามารถสร้างโครงสร้างโปรแกรมได้ล่วงหน้า โดยยังไม่ต้องใส่รายละเอียดของคำสั่งจริง** ทั้งหมดนี้สอดคล้องกับลักษณะการทำงานในโครงการใหญ่หรือโปรเจกต์ที่แบ่งงานเป็นหลายฟังก์ชัน หลายเงื่อนไข

ตัวอย่างจริงที่สอดคล้องกับวงจรทดลอง: ใช้ pass กับปุ่มกดและ LED

สมมติเรามีวงจรทดลองพื้นฐาน ใช้ **ปุ่มกด** ที่ขา 19 แบบ pull-up และ **LED** ที่ขา 2 ในระหว่างการพัฒนาโปรแกรม อาจต้องการออกแบบลอจิกว่า “ถ้ากดปุ่มแล้วจะให้ LED ติด” แต่ยังไม่แน่ใจว่าต้องเขียนคำสั่งอะไรดี หรืออาจต้องรอทดลองจริงก่อนจึงจะตัดสินใจใส่โค้ดที่เหมาะสม ในกรณีนี้โปรแกรมเมอร์จะประกาศโครงสร้าง if-else เพื่อเตรียมตรวจสอบปุ่มกดไว้ล่วงหน้า แล้วใส่ **pass** ไว้ในบล็อก if เพื่อเว้นที่ไว้ก่อนจนกว่าจะคิด logic ได้แน่ชัด

ตัวอย่างโค้ดที่ทดลองได้จริงบนบอร์ด ESP32 จะเป็นดังนี้



ในตัวอย่างนี้ หากผู้ใช้กดปุ่ม (button.value() == 0) โปรแกรมจะเข้าสู่บล็อก if แต่ยังไม่มีการสั่งงานใด ๆ จริง ๆ (เนื่องจาก pass ถูกใช้เว้นที่ไว้) ส่วนกรณีไม่ได้กดปุ่ม else จะสั่งให้ LED ดับชัดเจน จุดนี้เองผู้เขียนโปรแกรมสามารถกลับมาเติมคำสั่งจริงภายหลังได้ เช่น จะให้ LED ติด, จะให้ส่งสัญญาณ, หรือสั่งงานเพิ่มเติมอื่น ๆ ก็ได้ นี่แสดงให้เห็นว่า pass มีประโยชน์มากในกระบวนการพัฒนาโค้ดจริง ช่วยให้ไม่ต้องลบโครงสร้าง if-else หรือสลับไปปิด comment คำสั่งบ่อย ๆ

การใช้ pass ยังสอดคล้องกับแนวคิด “ค่อย ๆ เติมรายละเอียดของโปรแกรมในแต่ละขั้นตอน” ซึ่งมักพบได้ในงานเขียนโปรแกรมขนาดใหญ่ หรือการสอนพื้นฐานให้ผู้เรียนลองวางโครงร่างก่อนเติม logic ทีหลัง pass ช่วยให้โค้ดสะอาด อ่านง่าย ป้องกัน error เรื่องโครงสร้างผิดโดยไม่จำเป็น ในทางปฏิบัติ เมื่อผู้เขียนโปรแกรมพร้อมจะเพิ่ม logic จริง ๆ ในบล็อกที่เว้นไว้ด้วย pass ก็สามารถลบ pass ออกแล้วแทนที่ด้วยคำสั่งที่ต้องการได้ทันที โดยที่โครงสร้างของโปรแกรมยังคงสมบูรณ์และสื่อความหมายชัดเจน

กล่าวได้ว่าคำสั่ง pass จึงไม่ได้ “ทำอะไร” กับวงจรจริงโดยตรงในขณะทดลอง แต่ช่วยสร้างความเป็นระเบียบในกระบวนการพัฒนาโค้ด ทำให้สามารถทดลองโครงสร้าง logic ทุกส่วนได้ก่อนที่จะพร้อมใส่คำสั่งจริงในบล็อกนั้น และที่สำคัญคือ สามารถนำไปทดลองได้จริงทุกบรรทัดกับบอร์ดและวงจรทดลองมาตรฐาน

คำสั่ง try/except – จัดการข้อผิดพลาด

ในภาษา MicroPython และ Python โครงสร้าง try/except คือเครื่องมือสำคัญในการจัดการข้อผิดพลาด หรือที่ในเชิงวิชาการเรียกว่า Exception Handling หลักการของ try/except คือให้ผู้เขียนโปรแกรมนำคำสั่งหรือกระบวนการที่คาดว่าอาจเกิด error ได้มาไว้ในบล็อก try แล้ว MicroPython จะคอยเฝ้าดูให้ตลอดว่าในบล็อกนั้นมีอะไรผิดพลาดหรือไม่ ถ้าทุกอย่างดำเนินไปได้ดี โปรแกรมจะทำงานตามปกติและข้ามบล็อก except ไปโดยไม่สนใจอะไร แต่ถ้าระหว่างรันคำสั่งในบล็อก try เกิดข้อผิดพลาดขึ้นมา เช่น ข้อมูลบางอย่างไม่พร้อมหรืออุปกรณ์เชื่อมต่อไม่ได้ โปรแกรมจะไม่หยุดหรือค้างอยู่ตรงนั้นอย่างกระทันหันเหมือนในภาษาอื่น ๆ ที่ไม่มี Exception Handling ทันทีที่พบข้อผิดพลาด MicroPython จะข้ามไปทำงานในบล็อก except ต่ออัตโนมัติ ซึ่งเราสามารถออกแบบให้แจ้งเตือนผู้ใช้ ตั้งค่ากลับเป็นค่าปลอดภัย หรือแม้แต่สั่งรีเซ็ตการทำงานบางส่วนของระบบ เพื่อให้โปรแกรมโดยรวมยังเดินหน้าต่อได้อย่างปลอดภัย

หลักการนี้เปรียบเหมือนการวางแผนสำรองเมื่อเดินทางไกล สมมติว่ามีการวางเส้นทางไว้แต่แรกแต่เมื่อเจออุบัติเหตุหรือถนนปิดกลางทาง ถ้าไม่มีแผนสำรองหรือไม่คิดถึงความเสี่ยง เราอาจหยุดเดินทางและเสียเวลาทั้งหมด ในทางกลับกัน ถ้ามีแผนเผื่อไว้ เช่น เบี่ยงเส้นทาง แจ้งปลายทาง หรือหาวิธีไปต่อได้โดยไม่หยุดนิ่ง ระบบก็ยังเดินหน้าได้ ไม่เกิดความเสียหายมากกว่าเดิม การใช้ try/except ในโปรแกรมก็เหมือนการรับมือกับเหตุการณ์

ที่ไม่คาดคิด ไม่ว่าปัญหาจะเกิดขึ้นที่ไหนในกระบวนการ ระบบจะไม่หยุดลงทันทีแต่จะเดินหน้าต่อด้วยแผนสำรองที่เราวางไว้ใน except

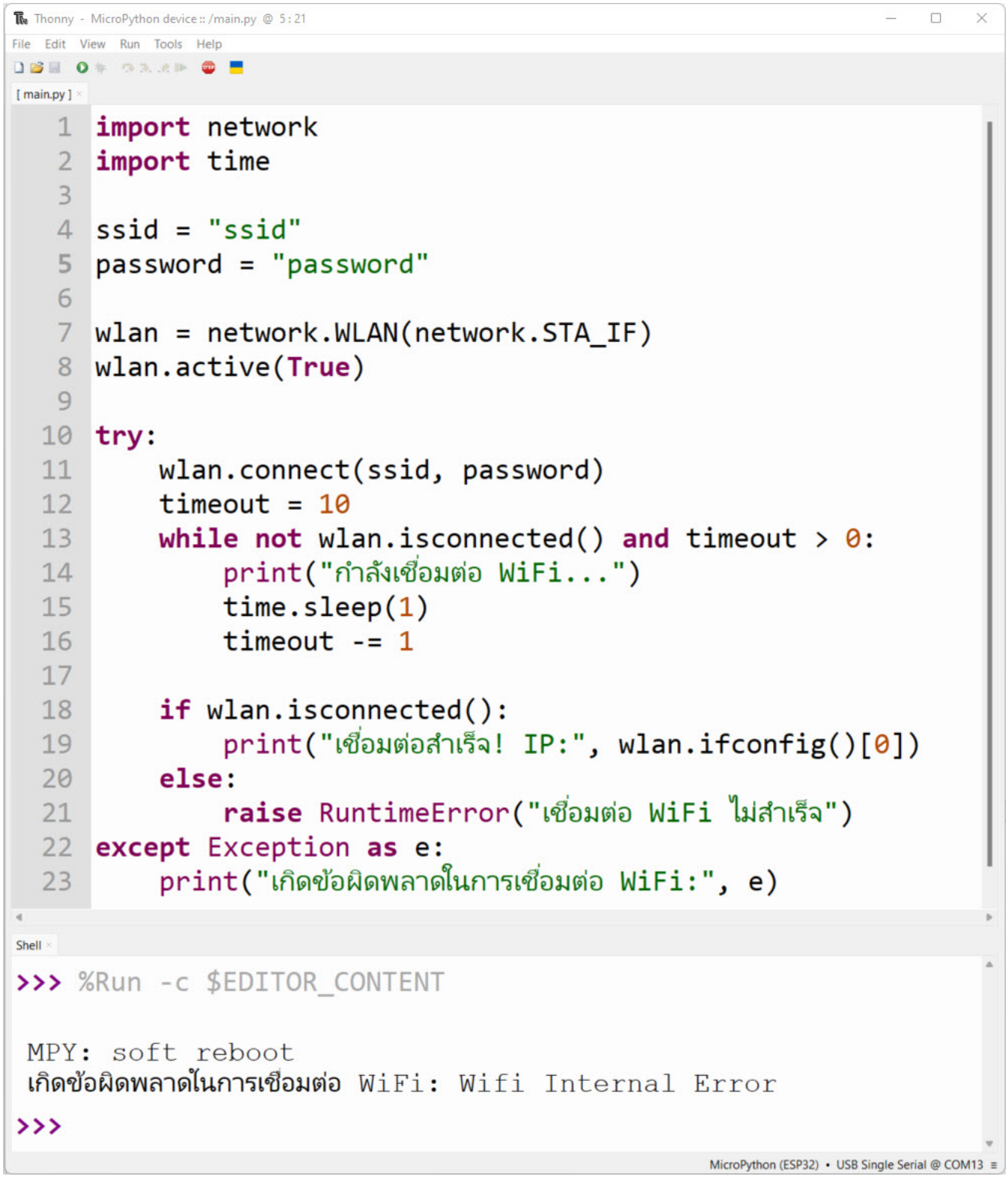
สำหรับรูปแบบโครงสร้างมาตรฐานของ try/except ใน MicroPython จะเขียนได้ดังนี้

```
try:
    # คำสั่งที่อาจเกิดข้อผิดพลาด
except:
    # คำสั่งที่ใช้จัดการข้อผิดพลาด
```

หรือจะขยายไปถึงการระบุชนิดข้อผิดพลาดที่เฉพาะเจาะจงได้ เช่น OSError หรือ RuntimeError เพื่อให้โปรแกรมตอบสนองแต่ละสถานการณ์อย่างถูกต้อง ในโลกของ IoT จริง โดยเฉพาะบน ESP32 งานที่พบข้อผิดพลาดได้บ่อยมากที่สุดคือการเชื่อมต่อ WiFi เช่น ใส่รหัสผิด เราเตอร์ปิด สัญญาณอ่อน หรือสภาพแวดล้อมไม่พร้อมรับการเชื่อมต่อ ถ้าไม่ใช้ try/except โปรแกรมจะหยุดนิ่งและต้องรีสตาร์ทใหม่หรือรอให้ใครซ่อมเครือข่ายก่อน ระบบจึงจะกลับมาทำงาน แต่ถ้าออกแบบโค้ดโดยใช้ try/except โปรแกรมจะสามารถตรวจสอบเฝ้าระวัง และแจ้งเตือนได้อัตโนมัติ เมื่อเกิดปัญหา โปรแกรมสามารถแจ้งให้ผู้ใช้ทราบหรือรอแก้ปัญหาชั่วคราวจนกลับมาเชื่อมต่อได้อีกครั้ง ทั้งหมดนี้ช่วยให้ระบบมีความยืดหยุ่นและทนทานต่อปัญหาในโลกจริง

ตัวอย่างการใช้ try/except กับ WiFi ของ ESP32 ที่ทุกคนสามารถทดลองได้จริงกับบอร์ดทุกรุ่น คือการสั่งเชื่อมต่อ WiFi แล้วรอดูว่าในเวลาที่กำหนดสามารถเชื่อมต่อสำเร็จหรือไม่ ถ้าสำเร็จจะแสดง IP Address ให้เห็นทาง Serial Monitor ถ้าไม่สำเร็จ เช่น ใส่รหัสผิดหรือไม่มีเครือข่าย โปรแกรมจะเข้าสู่บล็อก except ทันที แล้วแสดงข้อความข้อผิดพลาดให้รู้โดยไม่หยุดหรือค้างอยู่กับปัญหานั้น

และจากรูปต่อไปนี้ จะแสดงการใช้งานเชื่อมต่อ WIFI โดย จะทดลองใส่ ssid และ password ที่ไม่มีอยู่จริง แล้วทดลองกด Run เพื่อทดสอบการทำงาน หรือถ้าใครอยากจะทดลองเชื่อมต่อจริงเพื่อดูผลการทำงานก็ให้ใส่ SSID และ password ของตัวเองลงไปเพื่อทดสอบ เพราะมีแต่ผู้อ่านเองเท่านั้นที่จะรู้ว่า ssid และ password ของท่านคืออะไร



ตัวอย่างนี้สะท้อนแนวคิดว่าระบบควบคุมที่ดีต้องเตรียมพร้อมรับมือกับข้อผิดพลาดทุกสถานการณ์ โดยไม่หยุดเดินหน้าเมื่อเกิดปัญหาเล็กน้อยแต่ต้องแจ้งเตือนและพร้อมปรับตัวตลอดเวลา การฝึกใช้ try/except ให้ชำนาญจึงเป็นหัวใจสำคัญของการพัฒนาโปรแกรมควบคุมไมโครคอนโทรลเลอร์และระบบ IoT ที่ปลอดภัยน่าเชื่อถือ และตอบสนองโลกความจริงได้ดีที่สุด

คำสั่ง raise – การสร้างข้อผิดพลาดด้วยตนเอง

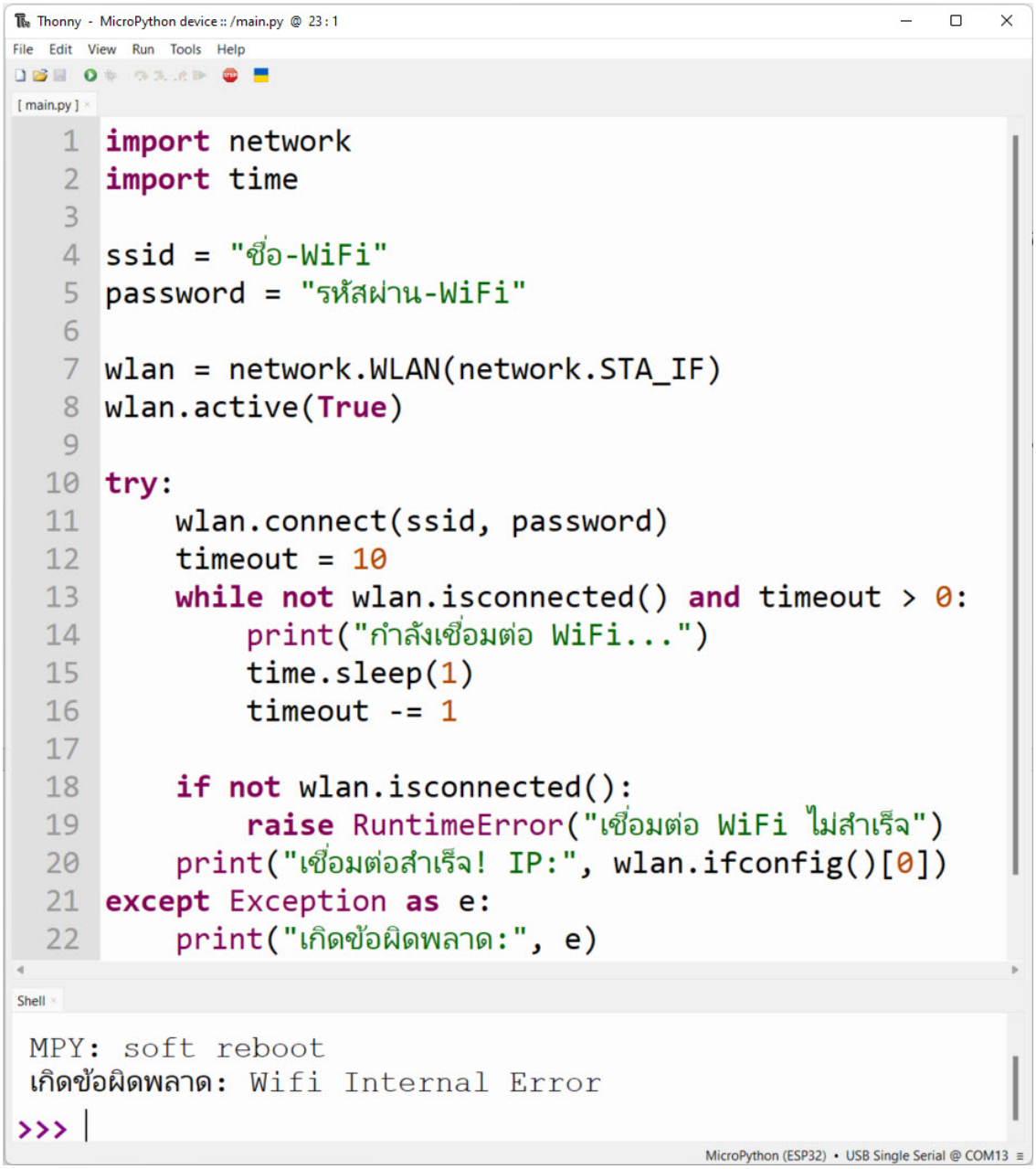
ในงานเขียนโปรแกรม MicroPython โดยเฉพาะสำหรับไมโครคอนโทรลเลอร์และระบบ IoT บน ESP32 ไม่เพียงแต่ต้องรู้จักการรับมือกับข้อผิดพลาดที่อาจเกิดขึ้นจากปัจจัยภายนอก เช่น การเชื่อมต่อ WiFi ไม่สำเร็จหรือไฟล์ข้อมูลขาดหาย แต่ยังต้องเข้าใจและรู้จัก “สร้างข้อผิดพลาด (Exception) ขึ้นมาเอง” เพื่อควบคุมลำดับเหตุการณ์หรือป้องกันสถานการณ์ที่อาจก่อให้เกิดปัญหาในอนาคตให้ชัดเจน คำสั่งสำคัญที่ใช้ในงานลักษณะนี้คือ **raise** ซึ่งเป็นเครื่องมือในภาษา MicroPython (รวมถึง Python มาตรฐาน) สำหรับ “สร้างข้อผิดพลาดด้วยตนเอง” ตามเงื่อนไขที่โปรแกรมเมอร์กำหนดอย่างเจาะจง

การใช้ raise ในโปรแกรมจริงนั้นมีประโยชน์หลายประการ โดยเฉพาะในงานที่ต้องการ “หยุด” กระบวนการเมื่อพบเหตุการณ์ที่ไม่ควรปล่อยผ่าน เช่น พบข้อมูลที่ผิดปกติ, ค่าเซ็นเซอร์ออกนอกช่วง, การตั้งค่าหรือพารามิเตอร์ที่ผู้ใช้ป้อนมาผิด หรือแม้แต่ในการสื่อสารกับอุปกรณ์ภายนอกแล้วเจอความผิดปกติที่เสี่ยงจะทำให้ระบบเสียหาย หากปล่อยให้โปรแกรมเดินหน้าต่อไปโดยไม่หยุดเพื่อแจ้งเตือน จะเสี่ยงให้เกิดปัญหาใหญ่ตามมาได้ ดังนั้นผู้พัฒนาจึงสามารถ “ยกข้อผิดพลาดขึ้นมา” ด้วย raise เพื่อบอกโปรแกรมและผู้ใช้ให้รู้ทันทีว่าสถานการณ์นี้ผิดปกติ ต้องหยุด หรือเข้าสู่โหมดจัดการปัญหาโดยทันที หลักการของ raise คือ ให้ใช้คู่กับชนิดข้อผิดพลาดที่เหมาะสม เช่น RuntimeError, ValueError, OSError ฯลฯ หรือจะสร้างข้อความอธิบายเหตุผลประกอบก็ได้ เมื่อคำสั่ง raise ถูกเรียกในโปรแกรม ข้อผิดพลาดนั้นจะเกิดขึ้นทันที โปรแกรมจะข้ามไปยังบล็อก except ที่เตรียมไว้ และดำเนินการตามแผนจัดการข้อผิดพลาดที่กำหนดไว้ต่อไป

ยกตัวอย่างสถานการณ์ในชีวิตจริง สมมติว่าโรงงานผลิตน้ำดื่มมีเซ็นเซอร์ตรวจจับคุณภาพน้ำอยู่ หากวันหนึ่งค่าความขุ่นของน้ำสูงเกินกว่ามาตรฐานที่ปลอดภัย หากไม่มีระบบเตือนหรือสั่งหยุดการผลิตทันที ผลลัพธ์อาจร้ายแรงและสินค้าเสียหายได้ แต่หากมีระบบที่คอยเฝ้าตรวจสอบและ “ยกข้อผิดพลาดขึ้นมา” เมื่อเจอสถานการณ์เสี่ยง ระบบจะหยุดสายการผลิต แจ้งเตือนผู้ควบคุม และเข้าสู่โหมดตรวจสอบได้ทันที ระบบซอฟต์แวร์ก็เช่นกัน การใช้ raise จะช่วยให้โค้ดมีความยืดหยุ่นและปลอดภัยขึ้นในทุกสถานการณ์

ในงานควบคุม ESP32 จริง การใช้ raise มักพบในกระบวนการ “ตรวจสอบเงื่อนไขก่อนดำเนินการสำคัญ” เช่น เมื่อต้องการเชื่อมต่อ WiFi และพบว่าไม่สามารถเชื่อมต่อได้ในเวลาที่กำหนด โปรแกรมเมอร์อาจใช้ raise เพื่อหยุดลูปและแจ้งข้อผิดพลาด หรือเมื่อตรวจสอบค่าจากเซ็นเซอร์แล้วพบว่าอยู่นอกขอบเขตที่กำหนด เช่น อ่านอุณหภูมิจากเซ็นเซอร์ได้ 999°C ซึ่งเป็นไปไม่ได้ในโลกจริง โค้ดสามารถ raise Exception ทันทีเพื่อเข้าสู่โหมดความปลอดภัยโดยไม่รอให้โปรแกรมเดินหน้าต่อและอาจก่อปัญหาใหญ่

ตัวอย่างโค้ดต่อไปนี้เป็นการใช้ raise คู่กับ try/except ในงานเชื่อมต่อ WiFi บน ESP32 ซึ่งผู้เรียนสามารถนำไปทดลองได้จริง เปลี่ยนค่า ssid หรือ password ให้ผิด หรือปิด WiFi Access Point แล้วสั่งรัน จะเห็นข้อผิดพลาดแสดงทันที



ในโปรแกรมนี้จะเห็นว่า เมื่อพยายามเชื่อมต่อ WiFi หากไม่สำเร็จภายในเวลาที่กำหนด โปรแกรมจะยกข้อผิดพลาด (raise RuntimeError) พร้อมระบุเหตุผล ทำให้ except จับเหตุการณ์นี้ได้ทันทีและแจ้งรายละเอียดข้อผิดพลาดออกมา โค้ดลักษณะนี้เหมาะอย่างยิ่งสำหรับงานที่ต้องการควบคุมความเสี่ยงและป้องกันความเสียหายของระบบจริง

กล่าวโดยสรุป คำสั่ง raise ใน MicroPython เป็นเครื่องมือสำคัญที่ช่วยให้ผู้พัฒนาสามารถสร้างข้อผิดพลาดเองในจุดที่ต้องการเพื่อป้องกันปัญหาที่อาจเกิดขึ้นจากข้อมูลหรือสถานการณ์ผิดปกติ พร้อมกำหนดแผนรับมือและแก้ไขปัญหาอย่างชาญฉลาด เป็นแนวคิดที่ผู้เขียนโปรแกรมควบคุมและงาน IoT ทุกระดับควรฝึกฝนให้เชี่ยวชาญ เพราะช่วยยกระดับความปลอดภัย ความยืดหยุ่น และความน่าเชื่อถือของระบบให้ใกล้เคียงกับมาตรฐานงานวิศวกรรมอุตสาหกรรมจริง

คำสั่ง finally – ทำงานเสมอไม่ว่าข้อผิดพลาดจะเกิดหรือไม่

ในกระบวนการเขียนโปรแกรมควบคุมไมโครคอนโทรลเลอร์และระบบ IoT โดยเฉพาะในภาษา MicroPython และ Python นอกจากคำสั่ง try/except ที่ใช้จัดการข้อผิดพลาดระหว่างรันโปรแกรมแล้ว ยังมีอีกหนึ่งโครงสร้างสำคัญที่ช่วยให้โค้ดของเรามีความปลอดภัยและเป็นระบบมากขึ้น นั่นคือ **finally** โครงสร้างนี้ถูกออกแบบมาเพื่อ “รับประกัน” ว่าชุดคำสั่งภายในบล็อก finally จะถูกดำเนินการทุกครั้ง ไม่ว่าในระหว่างรันโค้ดจะเกิดข้อผิดพลาด (Exception) หรือไม่ก็ตาม

หลักการสำคัญของคำสั่ง finally คือ หลังจากที่โปรแกรมพยายามทำงานในบล็อก try แล้วตรวจสอบว่ามีข้อผิดพลาดหรือไม่ และดำเนินการตามบล็อก except หากพบ error แต่ไม่ว่าจะเกิดอะไรขึ้นก็ตาม สุดท้ายโค้ดใน finally จะต้องถูกรันเสมอ เช่น การปิดการเชื่อมต่อกับอุปกรณ์ การบันทึกข้อมูล การคืนค่าทรัพยากร หรือแม้แต่การส่งสัญญาณเตือนความล้มเหลว ทั้งหมดนี้เป็นสิ่งที่ควรทำไม่ว่ากระบวนการก่อนหน้าจะสำเร็จหรือล้มเหลว เปรียบเทียบกับชีวิตจริง ในการทำงานจริงหรือในระบบอัตโนมัติ สมมติว่ามีการเปิดใช้งานเครื่องจักรอัตโนมัติเพื่อบรรจุสินค้า หลังจากที่ดำเนินการใส่สินค้าในกล่องเสร็จแล้ว ไม่ว่ากระบวนการจะสมบูรณ์หรือเกิดปัญหาขัดข้อง สิ่งที่ต้องทำเสมอก็คือการหยุดสายพาน ปิดวาล์ว หรือรีเซ็ตเครื่องให้กลับสู่สภาพพร้อมเริ่มใหม่ในรอบถัดไป เช่นเดียวกับบล็อก finally ที่จะทำหน้าที่จัดการกระบวนการสุดท้ายก่อนที่โปรแกรมจะออกจากโครงสร้างการทำงานนั้น เพื่อให้แน่ใจว่าระบบจะไม่ค้าง ไม่เสียทรัพยากร หรือทิ้งสถานะที่ไม่ถูกต้องไว้เบื้องหลัง

ในทางปฏิบัติการใช้ finally พบได้บ่อยในสถานการณ์ที่ต้องมีการเปิดและปิดทรัพยากรต่าง ๆ เช่น การเปิดไฟล์, การเชื่อมต่อกับฐานข้อมูล, การติดต่อกับอุปกรณ์ภายนอก, หรือแม้แต่การจัดการกับระบบเครือข่าย WiFi บน ESP32 ตัวอย่างเช่น หากมีการเปิดไฟล์เพื่ออ่านหรือเขียนข้อมูล ในกรณีที่ทุกอย่างปกติดี โปรแกรมจะปิดไฟล์ตามขั้นตอนปกติ แต่หากเกิดข้อผิดพลาดขึ้นในระหว่างกระบวนการ เช่น เขียนข้อมูลไม่สำเร็จหรือไฟล์หาย โปรแกรมก็ยังต้องปิดไฟล์เสมอเพื่อไม่ให้ระบบค้างหรือล็อกไฟล์นั้นไว้ต่อไป

โครงสร้างคำสั่ง try/except/finally ใน MicroPython จะเขียนได้ในรูปแบบนี้

```
try:
    # คำสั่งที่อาจเกิดข้อผิดพลาด
except Exception as e:
    # คำสั่งที่ใช้จัดการข้อผิดพลาด
finally:
    # คำสั่งที่ต้องการให้ทำเสมอ ไม่ว่าข้อผิดพลาดจะเกิดหรือไม่
```

การใช้ finally มีความสำคัญมากขึ้นในงานระบบสมองกลฝังตัว (Embedded Systems) หรือ IoT เพราะอุปกรณ์เหล่านี้มักรันโปรแกรมต่อเนื่องเป็นเวลานาน การละเลยกระบวนการปิดทรัพยากรหรือคืนสถานะอุปกรณ์เมื่อจบการทำงานอาจทำให้ระบบขัดข้องในระยะยาว เช่น หน่วยความจำรั่ว การสื่อสารผิดพลาด หรืออุปกรณ์ไม่ตอบสนองต่อไป ตัวอย่างโค้ดต่อไปนี้แสดงการใช้ finally ในการเชื่อมต่อ WiFi บน ESP32 ซึ่งสามารถทดลองได้จริง เมื่อโปรแกรมพยายามเชื่อมต่อ WiFi แล้วไม่ว่าจะสำเร็จหรือไม่ ในบล็อก finally จะสั่งปิด WiFi (deactivate) ทุกครั้งเพื่อให้ระบบพร้อมสำหรับการเชื่อมต่อใหม่ในอนาคต และเพื่อประหยัดพลังงาน

ในตัวอย่างนี้ไม่ว่าโปรแกรมจะเชื่อมต่อ WiFi สำเร็จหรือไม่ ในบรรทัดสุดท้ายของ finally จะสั่งปิดการทำงานของ WiFi STA ทุกครั้ง เป็นการคืนสถานะบอร์ดให้พร้อมสำหรับงานอื่นหรือรอรับคำสั่งใหม่ ถือเป็นการ “ล้างระบบ” เพื่อให้ไม่เหลือสถานะค้างคาใด ๆ ไว้เบื้องหลัง

กล่าวโดยสรุป การใช้ finally จึงเป็นหัวใจในการออกแบบโปรแกรมที่ปลอดภัย มีความรับผิดชอบต่อทรัพยากรของระบบ และป้องกันปัญหาที่อาจเกิดขึ้นในระยะยาว โดยเฉพาะในงาน IoT, ระบบฝังตัว, และการพัฒนาอุปกรณ์อิเล็กทรอนิกส์จริงทุกประเภท การฝึกใช้งาน finally คู่กับ try/except จึงเป็นทักษะสำคัญที่นักพัฒนาและผู้เรียนควรเข้าใจอย่างลึกซึ้ง เพื่อสร้างระบบที่มั่นคงพร้อมใช้งานในทุกสถานการณ์

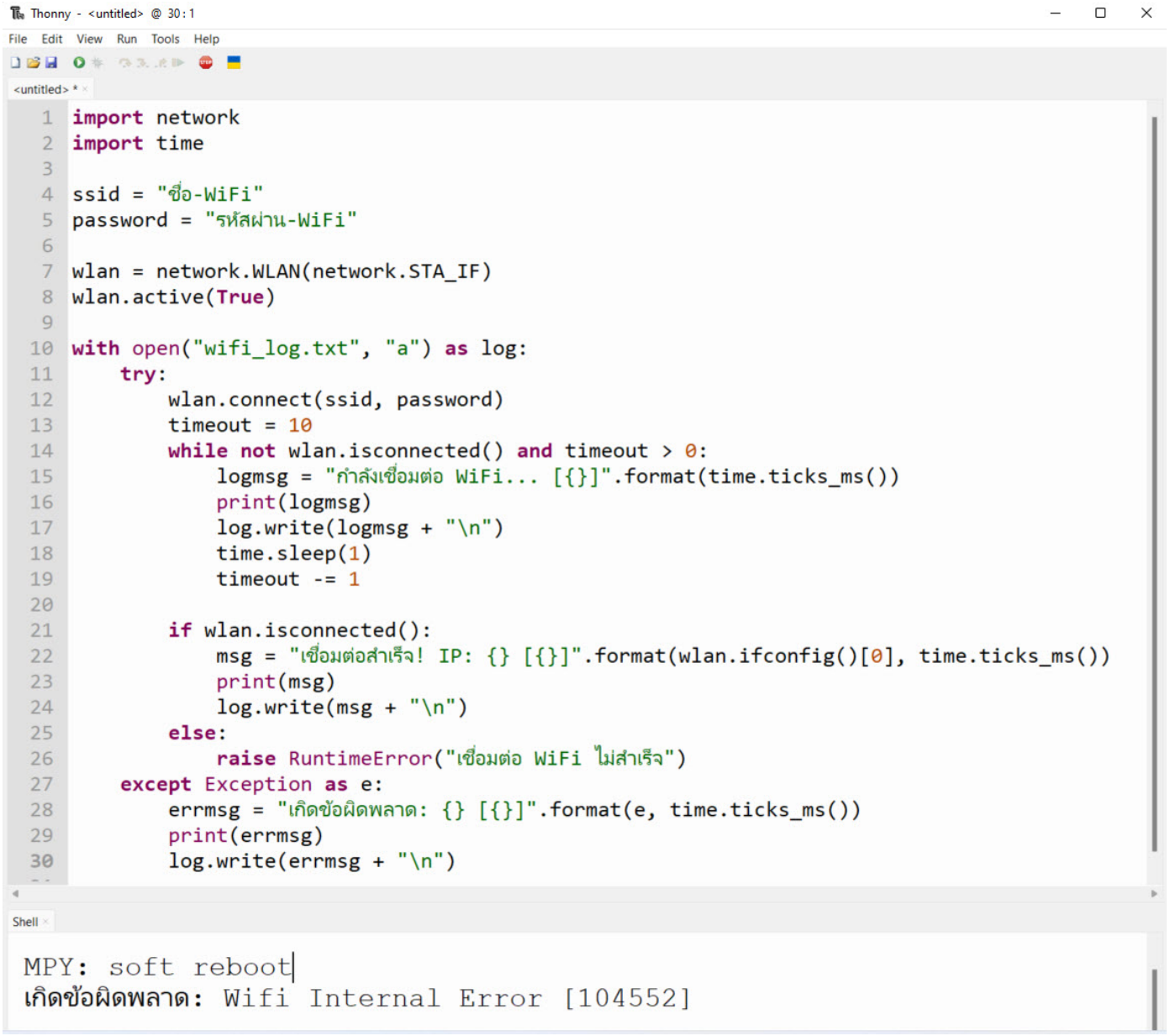
คำสั่ง with – การจัดการทรัพยากรอัตโนมัติอย่างปลอดภัย

การพัฒนาโปรแกรมบนระบบสมองกลฝังตัวและ IoT ด้วย MicroPython หนึ่งในหัวใจสำคัญที่ไม่ควรมองข้ามคือ “การจัดการทรัพยากร” โดยเฉพาะกับไฟล์และอุปกรณ์เก็บข้อมูล เพราะหากเปิดไฟล์แล้วลืมปิด หรือเกิดข้อผิดพลาดระหว่างการอ่าน/เขียนไฟล์ อาจทำให้หน่วยความจำรั่ว ไฟล์เสียหาย หรือแม้แต่ระบบค้างในระยะยาวได้ เพื่อป้องกันปัญหาเหล่านี้ MicroPython จึงรับแนวคิด *Context Manager* ของ Python มาตรฐานมาใช้กับคำสั่ง **with** ซึ่งช่วยให้การจัดการทรัพยากรเป็นไปโดยอัตโนมัติ ปลอดภัย และลดโอกาสข้อผิดพลาดจากมนุษย์อย่างสิ้นเชิง

หลักการของ with คือ เมื่อต้องใช้งานทรัพยากรสำคัญ เช่น การเปิดไฟล์ คำสั่ง with จะรับประกันว่าทรัพยากรนั้นจะถูกคืนกลับระบบโดยอัตโนมัติ ไม่ว่าจะสิ้นสุดกระบวนการด้วยความสำเร็จหรือเกิดข้อผิดพลาดระหว่างทางก็ตาม ต่างจากการเปิดไฟล์แบบเดิมที่ต้องปิดไฟล์เอง ซึ่งหากลืมหรือโปรแกรมหยุดกลางคันก็จะสร้างปัญหาทันที เปรียบเทียบกับชีวิตจริงก็เหมือนกับการใช้ห้องประชุมหรือเช่าสถานที่ ทุกครั้งที่เลิกใช้งานจะต้องคืนห้องให้เจ้าหน้าที่ ไม่ว่าจะเกิดเหตุฉุกเฉินหรือใช้งานครบตามกำหนด เพื่อให้คนต่อไปสามารถใช้สถานที่นั้นได้อย่างปลอดภัย

สำหรับระบบ MicroPython บน ESP32 ตัวอย่างที่เหมาะสมที่สุดคือการจัดการไฟล์บน Flash ของบอร์ด การใช้คำสั่ง with เปิดไฟล์เพื่อบันทึก log เหตุการณ์ต่าง ๆ เช่น การเชื่อมต่อ WiFi หรือการวิเคราะห์สถานะอุปกรณ์ จะช่วยป้องกันปัญหาที่อาจเกิดจากการลืมปิดไฟล์ หรือเกิดข้อผิดพลาดระหว่างเขียนข้อมูล ตัวอย่างต่อไปนี้จะแสดงการใช้ with บันทึก log การเชื่อมต่อ WiFi ซึ่งเป็นงานจริงในระบบ IoT ทั่วไป

เมื่อโปรแกรมเริ่มทำงาน จะเปิดไฟล์ wifi_log.txt ในโหมดต่อท้าย (append) ด้วยคำสั่ง with จากนั้นจะพยายามเชื่อมต่อ WiFi ตามค่าที่ตั้งไว้ ทุกขั้นตอนของการเชื่อมต่อ เช่น กำลังเชื่อมต่อ สำเร็จ หรือเกิดข้อผิดพลาดจะถูกเขียนลงไฟล์ log ทันที เมื่อสิ้นสุดกระบวนการ ไม่ว่าจะสำเร็จหรือไม่ ระบบจะปิดไฟล์ให้อัตโนมัติ ไม่ว่าบล็อกจะจบปกติหรือเกิดข้อผิดพลาดระหว่างทางก็ตาม

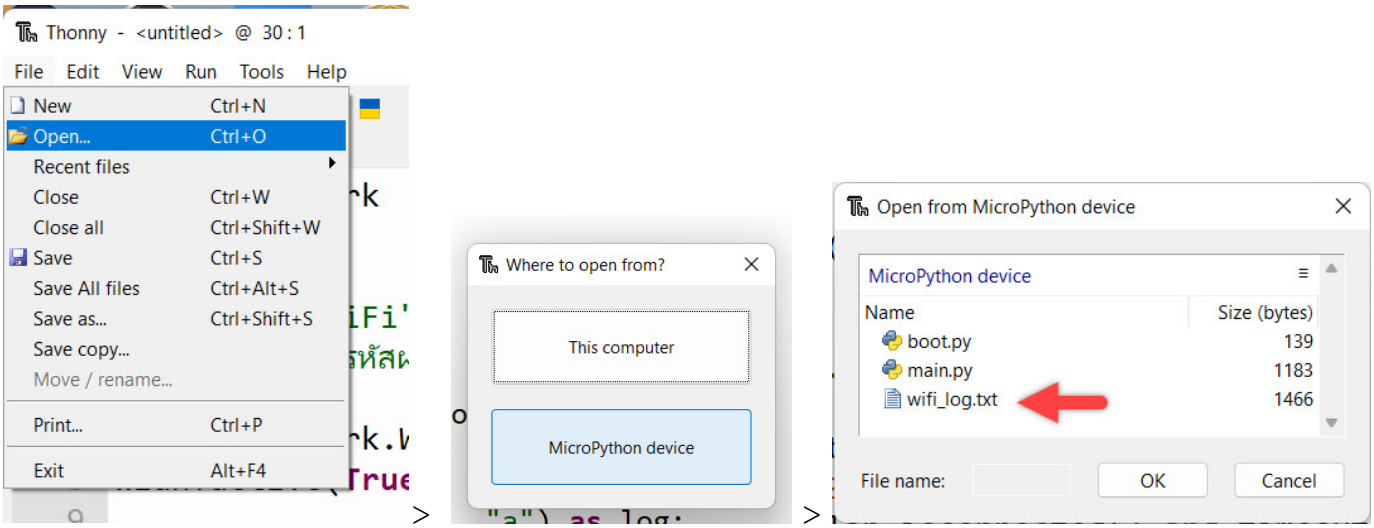


หลังจากรันโปรแกรมนี้ ทุกเหตุการณ์ที่เกิดขึ้นระหว่างการเชื่อมต่อ WiFi จะถูกบันทึกลงในไฟล์ wifi_log.txt บน Flash ของบอร์ด ESP32 เสมอ ไม่ว่าจะสำเร็จหรือเกิดข้อผิดพลาด ข้อมูลใน log นี้จะไม่สูญหายหากไม่ได้สั่งลบเอง ช่วยให้สามารถวิเคราะห์ ตรวจสอบ หรือย้อนดูปัญหาที่เกิดขึ้นได้อย่างละเอียด

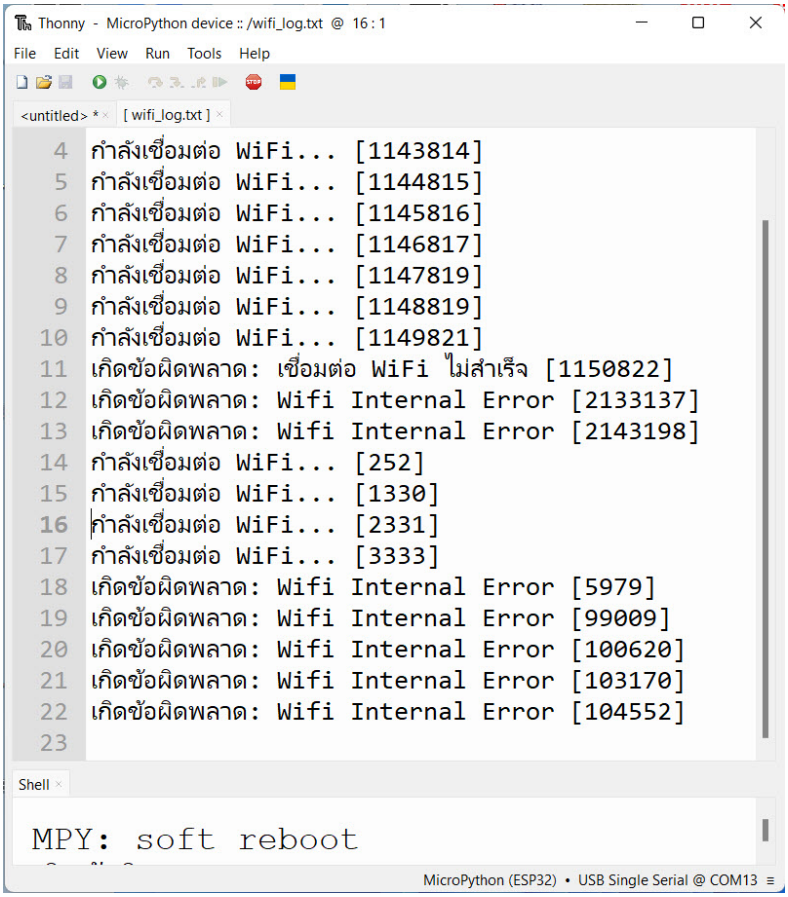
สำหรับการตรวจสอบข้อมูล log ที่เก็บไว้ ผู้ใช้สามารถเปิดดูไฟล์ wifi_log.txt ได้ง่าย ๆ ผ่านโปรแกรม Thonny ซึ่งเป็นเครื่องมือมาตรฐานสำหรับงาน MicroPython ทุกระดับ เมื่อเชื่อมต่อบอร์ด ESP32 กับคอมพิวเตอร์และเปิดโปรแกรม Thonny ให้เลือก MicroPython device ในแถบ Files จะเห็นไฟล์ wifi_log.txt อยู่ในรายการไฟล์บนบอร์ด สามารถดับเบิลคลิกเพื่อเปิดอ่าน หรือคลิกขวาเลือก “Download to this computer” เพื่อเก็บไฟล์ log ลงคอมพิวเตอร์สำหรับวิเคราะห์ต่อไปก็ได้ การดู log ผ่าน Thonny จะเห็นประวัติทุกเหตุการณ์ย้อนหลัง ทั้งช่วงเวลาที่เชื่อมต่อสำเร็จ ข้อความ error ต่าง ๆ รวมถึง timestamp ของแต่ละเหตุการณ์ ช่วยให้ผู้พัฒนาระบบสามารถวิเคราะห์ปัญหา แก้ไข หรือปรับปรุงประสิทธิภาพได้ตรงจุด ถือเป็นแนวทางมาตรฐานในอุตสาหกรรม IoT

อย่างไรก็ตาม พื้นที่ Flash ของบอร์ด ESP32 มีขนาดจำกัด เหมาะสำหรับเก็บ log หรือข้อมูลสำคัญขนาดเล็กหรือระยะสั้นเท่านั้น หากต้องการบันทึกข้อมูลปริมาณมาก หรือบันทึก log ต่อเนื่องในระยะยาว เช่น เก็บ log หลายวันหรือหลายเดือน ควรเลือกใช้ SD card แทนพื้นที่ Flash

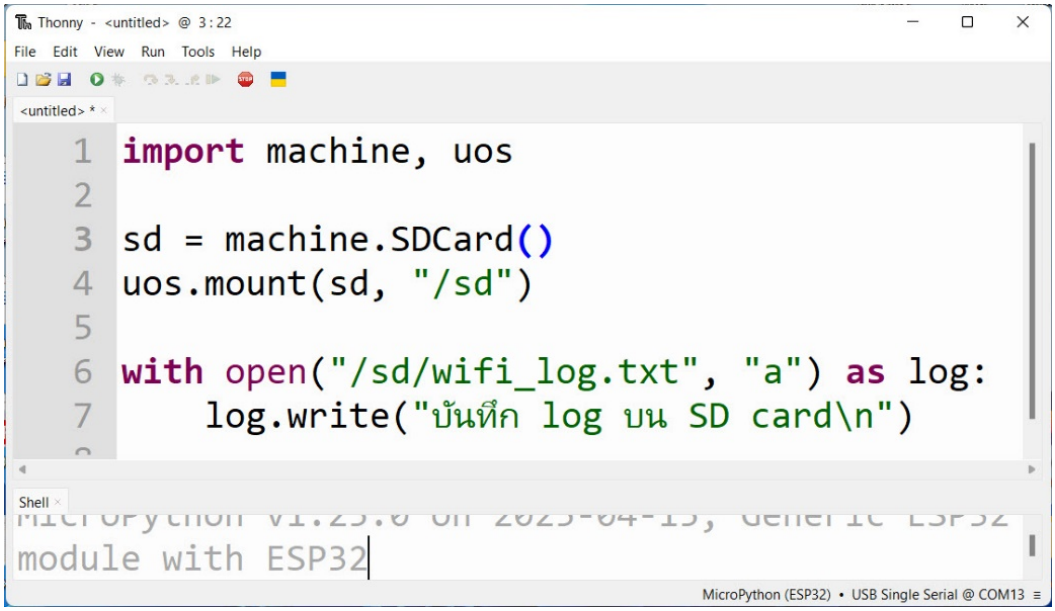
การเปิดดูไฟล์ คลิกที่เมนู File > open เลือก MicroPython device จะพบไฟล์ที่สร้างไว้



สามารถเปิดดู logfile ได้เลย



การใช้กับ SD card สำหรับผู้ใช้ที่บอร์ดมี SD card slot หรือได้ติดตั้ง SD card เพิ่มเติม สามารถนำหลักการเดียวกันนี้ไปใช้ได้ทันที เพียงแต่ต้องเมานต์ SD card เข้าระบบก่อน ด้วยคำสั่ง uos.mount() หรือ machine.SDCard() แล้วกำหนด path ไฟล์เป็น /sd/ชื่อไฟล์.txt ตามตำแหน่งที่เมานต์ ตัวอย่างเช่น



ในกรณีนี้ ระบบจะเก็บข้อมูล log บน SD card แทน Flash ของบอร์ด เหมาะสำหรับงานที่ต้องการพื้นที่เก็บข้อมูลขนาดใหญ่ หรือเก็บประวัติย้อนหลังในระยะยาว เพราะ SD card รองรับไฟล์ขนาดใหญ่กว่ามาก และสามารถนำไป backup หรือวิเคราะห์ข้อมูลย้อนหลังนอกบอร์ดได้สะดวกยิ่งขึ้น การบันทึกข้อมูลลง SD card ควรปิดไฟล์ทุกครั้งหลังใช้งาน (ซึ่ง with ช่วยรับประกันให้) และควรมีฟีเจอร์ตรวจสอบว่า SD card พร้อมใช้งานก่อนเปิดไฟล์เสมอ เพื่อลดปัญหาการเขียนไฟล์ล้มเหลวหรือ SD card หลุดระหว่างการใช้งาน เช่น ในบางโปรเจกต์อาจต้อง try/except รอบการเมานต์ SD card เพิ่มเติม และควรออกแบบให้ระบบแจ้งเตือนผู้ใช้เมื่อเกิดปัญหาการเมานต์หรือการเขียนไฟล์ผิดพลาด

กล่าวโดยสรุป คำสั่ง with ใน MicroPython เป็นแนวปฏิบัติที่สำคัญในการจัดการไฟล์หรือทรัพยากรอัตโนมัติอย่างปลอดภัย ทำให้ระบบของเราทำงานต่อเนื่องในระยะยาวได้อย่างมั่นใจ ลดโอกาสข้อผิดพลาดจากมนุษย์ และช่วยให้ทุกขั้นตอนของการบันทึก วิเคราะห์ หรือแก้ไขปัญหาในระบบฝังตัวเป็นไปอย่างโปร่งใสและมืออาชีพ

3.5 ฟังก์ชัน (Functions)

ความสำคัญของฟังก์ชัน

การเขียนโปรแกรมสำหรับงานสมองกลฝังตัวหรือไมโครคอนโทรลเลอร์ เช่น ESP32 ด้วยภาษา MicroPython นั้น หนึ่งในแนวคิดพื้นฐานที่ขาดไม่ได้คือ “ฟังก์ชัน” (Function) เพราะฟังก์ชันเป็นกลไกสำคัญที่ช่วยให้โค้ดของเราถูกจัดระเบียบอย่างเป็นระบบมากขึ้น ลดความซ้ำซ้อน และทำให้โปรแกรมสามารถขยายต่อยอดในอนาคตได้ง่ายขึ้นอย่างมีประสิทธิภาพ ฟังก์ชันคือกลุ่มคำสั่งย่อยที่ถูกรวบรวมไว้ภายใต้ชื่อเดียว เมื่อต้องการสั่งงานหรือทำซ้ำ สามารถเรียกชื่อฟังก์ชันนั้น ๆ แทนการเขียนคำสั่งเดิมซ้ำ ๆ ได้ทันที เปรียบเทียบกับชีวิตประจำวัน เหมือนกับการใช้เครื่องใช้ไฟฟ้าหรืออุปกรณ์ต่าง ๆ ที่มีปุ่มควบคุมเฉพาะ เช่น สวิตช์เปิด–ปิดไฟใน

บ้าน หรือการกดปุ่มลิฟต์ขึ้นลง การทำงานที่ซ่อนอยู่ข้างในอาจมีหลายขั้นตอนซับซ้อน แต่ผู้ใช้ไม่จำเป็นต้องรู้รายละเอียด เพียงแค่กดปุ่ม ระบบก็จะทำงานให้ครบถ้วนตามหน้าที่โดยอัตโนมัติ

ในแง่การเขียนโปรแกรม หากไม่มีฟังก์ชัน โปรแกรมที่ซับซ้อนหรือทำงานซ้ำ ๆ หลายจุดจะเต็มไปด้วยคำสั่งที่เหมือนกันอยู่ทั่วทั้งไฟล์ ส่งผลให้แก้ไขโค้ดได้ยากและเกิดข้อผิดพลาดได้ง่าย เมื่อใดที่มีความจำเป็นต้องปรับเปลี่ยนพฤติกรรมของโปรแกรม นักพัฒนาต้องแก้ไขโค้ดในทุกจุดที่มีการทำงานซ้ำซ้อนเหล่านั้น ซึ่งเสี่ยงต่อการลืมหรือเกิดความคลาดเคลื่อนได้ตลอดเวลา การมีฟังก์ชันจึงเปรียบเสมือนการวางระบบควบคุมกลาง หากต้องเปลี่ยนแปลงหรือปรับปรุงการทำงาน สามารถแก้ไขที่ฟังก์ชันเดียวแล้วคำสั่งอื่น ๆ ที่เรียกใช้ฟังก์ชันนั้นจะเปลี่ยนตามโดยอัตโนมัติ

นอกจากนี้ ฟังก์ชันยังเป็นพื้นฐานของแนวคิดการพัฒนาโปรแกรมสมัยใหม่ ไม่ว่าจะเป็นแนวทางแบบโครงสร้าง (Structured Programming) หรือแนวคิดเชิงวัตถุ (Object-Oriented Programming) ในงานที่ต้องควบคุมอุปกรณ์จำนวนมาก อ่านค่าจากเซ็นเซอร์หรือสั่งงานอุปกรณ์เอาต์พุตซ้ำ ๆ การใช้ฟังก์ชันจะช่วยลดความซับซ้อนของโค้ดหลัก เพิ่มความปลอดภัย ลดข้อผิดพลาดจากการแก้ไขผิดจุด และทำให้โค้ดสามารถอ่านหรือสื่อสารกับทีมงานคนอื่นได้ง่ายขึ้น ฟังก์ชันที่ออกแบบดีจะช่วยให้ผู้อื่นสามารถนำไปใช้ซ้ำในโปรแกรมอื่น ๆ ได้ หรือแม้แต่แยกโค้ดเป็นโมดูลที่นำไปพัฒนาเป็นไลบรารีต่อยอดได้ในอนาคต

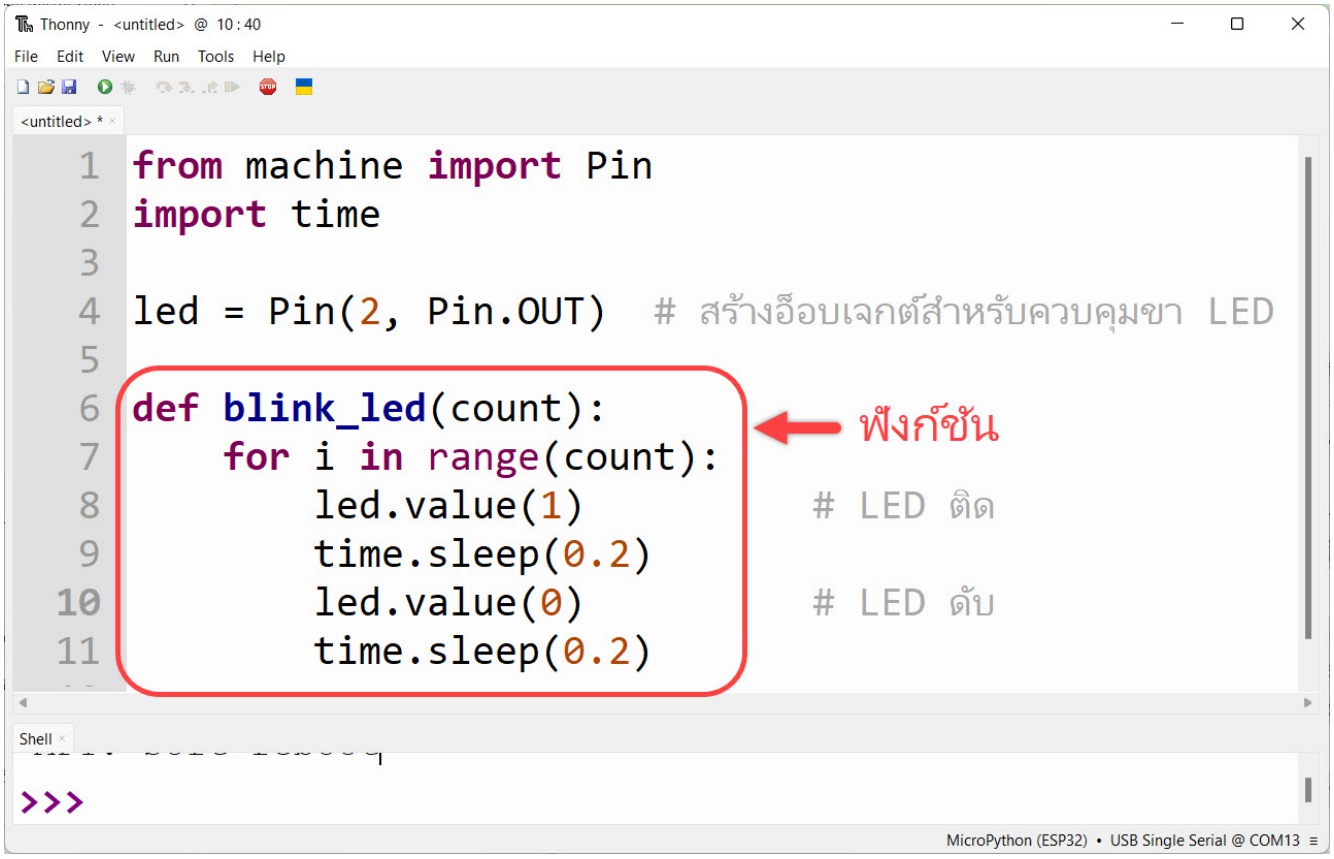
กล่าวได้ว่าฟังก์ชันคือรากฐานของการเขียนโปรแกรมทุกภาษา โดยเฉพาะกับ MicroPython บน ESP32 ซึ่งต้องการความชัดเจนของโค้ด ความง่ายในการแก้ไข และความปลอดภัยในการใช้งานระยะยาว ฟังก์ชันจึงเป็นเครื่องมือหลักที่จะช่วยให้ผู้เขียนโปรแกรมควบคุมอุปกรณ์ได้อย่างมีประสิทธิภาพ เป็นระบบ และพร้อมสำหรับการขยายตัวในทุกระดับของงาน ตั้งแต่งานทดลองในห้องเรียนจนถึงระบบอุตสาหกรรมจริง

การสร้างฟังก์ชัน (Defining Functions)

การสร้างฟังก์ชัน (Defining Functions) คือกระบวนการสำคัญที่ช่วยจัดระเบียบคำสั่งหรือแนวคิดในโปรแกรมให้กลายเป็นหน่วยย่อยที่นำกลับมาใช้ซ้ำได้ง่ายในภายหลัง โดยหลักการของการสร้างฟังก์ชันใน MicroPython นั้นจะเหมือนกับภาษา Python มาตรฐานทุกประการ ฟังก์ชันประกอบด้วยชื่อเฉพาะ กลุ่มคำสั่งภายใน และอาจมีพารามิเตอร์สำหรับรับค่าจากภายนอก โดยโครงสร้างของฟังก์ชันใน MicroPython จะเริ่มต้นด้วยคำสั่ง def ตามด้วยชื่อฟังก์ชันที่ต้องการ ตั้งชื่ออย่างมีความหมาย สื่อถึงหน้าที่หรือผลลัพธ์ของฟังก์ชันนั้น ๆ ในวงเล็บสามารถกำหนดตัวแปรรับค่าจากภายนอก (parameter) ถ้ายังไม่มีค่ารับเข้า สามารถเว้นวงเล็บให้ว่างไว้ได้ หลังจากวงเล็บให้ใส่เครื่องหมายโคลอน (:) จากนั้นย่อหน้าขึ้นบรรทัดใหม่แล้วจึงเขียนกลุ่มคำสั่งที่ต้องการให้อยู่ภายในฟังก์ชัน

การจัดย่อหน้า (indentation) มีความสำคัญมากในภาษา Python และ MicroPython เพราะใช้เพื่อแยกขอบเขตของฟังก์ชันให้ชัดเจน โค้ดทุกบรรทัดภายในฟังก์ชันจะต้องอยู่ในระดับย่อหน้าเดียวกัน มิฉะนั้นจะเกิดข้อผิดพลาดขณะรันโปรแกรม นอกจากนี้ การตั้งชื่อฟังก์ชันควรใช้ตัวอักษรเล็กและขีดล่าง (_) เพื่อความอ่านง่าย และควรหลีกเลี่ยงชื่อตรงกับคำสั่งหรือชื่อสำรองในภาษา

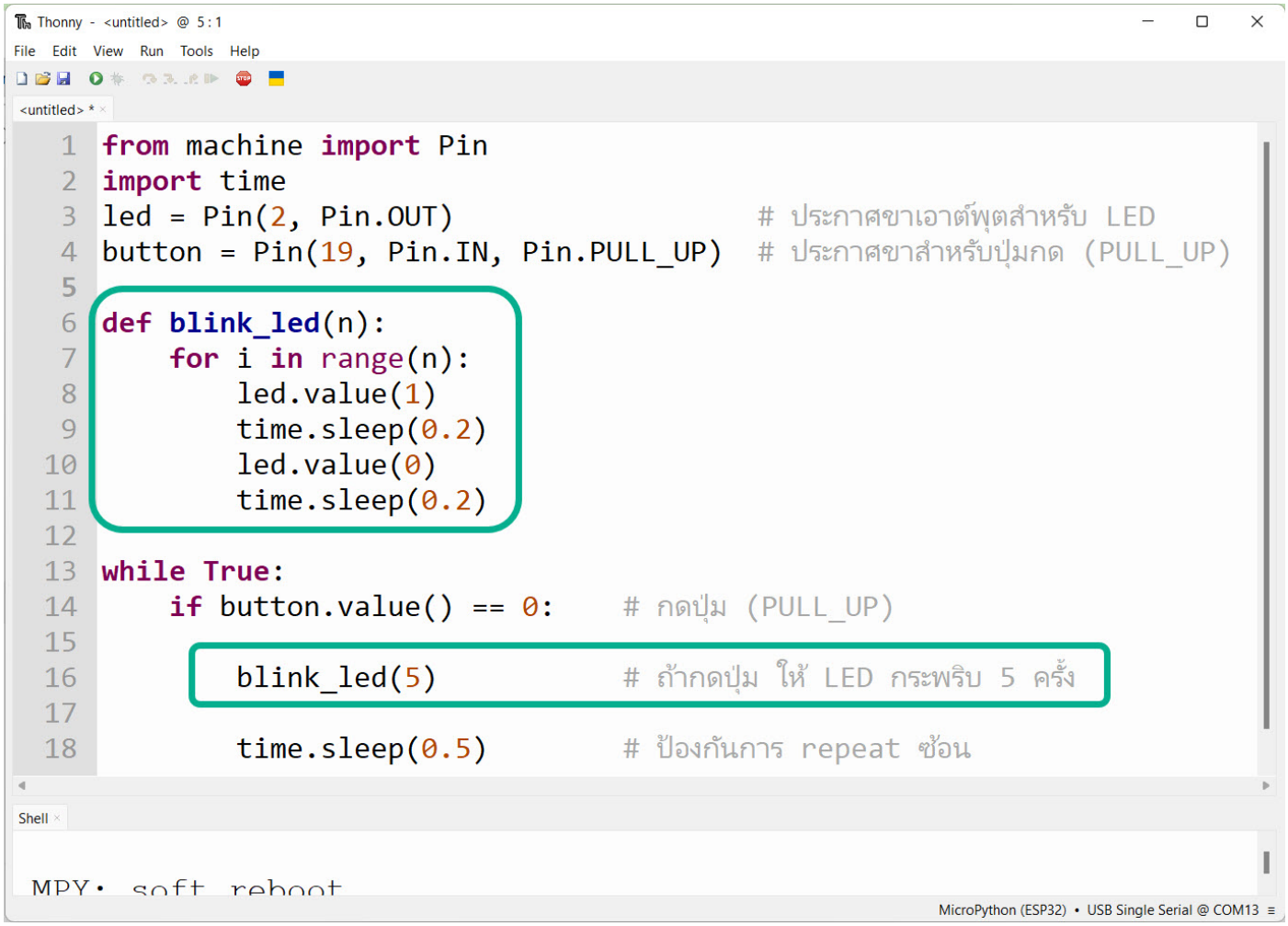
ยกตัวอย่างเช่น ถ้าต้องการสร้างฟังก์ชันสำหรับสั่งให้ LED กระพริบจำนวนครั้งที่กำหนด สามารถเขียนฟังก์ชันชื่อว่า blink_led ได้ดังนี้



ในตัวอย่างนี้ ฟังก์ชัน blink_led รับพารามิเตอร์ชื่อ count เพื่อระบุจำนวนรอบการกระพริบ LED จากนั้นใช้ลูป for เพื่อสั่งให้ LED ติดและดับสลับกันไปตามจำนวนรอบที่ต้องการ โค้ดทุกบรรทัดภายในฟังก์ชันจะต้องย่อหน้าเข้าไปเสมอ และเมื่อจบฟังก์ชันแล้วบรรทัดต่อไปจะต้องถอยกลับมาอยู่ในระดับเดียวกับ def เพื่อแสดงว่าจบขอบเขตของฟังก์ชันแล้ว และหลังจากประกาศฟังก์ชันนี้ไว้ในโปรแกรม สามารถเรียกใช้งานได้ทันที ตามตัวอย่าง

```
blink_led(10)  # เรียกฟังก์ชันให้ LED กระพริบ 10 ครั้ง
```

เพื่อทดลองให้เห็นผลลัพธ์แบบวนซ้ำจริงในงานฝังตัว สามารถนำตัวอย่างโค้ดต่อไปนี้ไปทดลองได้ทันที โดยบรรจุฟังก์ชันไว้ในลูป while เพื่อให้สามารถทดลองกับเงื่อนไขต่าง ๆ เช่น กดปุ่มแล้ว LED กระพริบ 5 ครั้ง



สำหรับการใช้งานจริง ในโปรแกรม MicroPython เมื่อสร้างฟังก์ชันไว้แล้วสามารถเรียกใช้งานฟังก์ชันนี้ได้ทุกที่หลังจากที่ประกาศ def เรียบร้อย เช่น ถ้าต้องการให้ LED กระพริบ 5 ครั้ง สามารถสั่งได้ทันทีด้วย blink_led(5) โดยไม่ต้องเขียนชุดคำสั่งยาว ๆ ซ้ำ ๆ หลายจุดในโปรแกรม

การสร้างฟังก์ชันที่ดีควรตั้งชื่อสื่อความหมาย เลือก parameter ที่เหมาะสม และระบุหน้าที่ของฟังก์ชันอย่างชัดเจน โค้ดที่เขียนไว้ในฟังก์ชันควรเป็นงานที่ต้องทำซ้ำ ๆ หรือมีความเป็นโมดูลในตัวเอง เช่น การอ่านค่าจากเซ็นเซอร์ การคำนวณข้อมูล การควบคุมเอาต์พุตต่าง ๆ หรือแม้แต่การจัดรูปแบบข้อมูลสำหรับแสดงผล

ใน MicroPython สามารถประกาศฟังก์ชันใหม่ได้ตลอดเวลา ไม่จำเป็นต้องประกาศไว้ต้นไฟล์เท่านั้น แต่ต้องแน่ใจว่าฟังก์ชันถูกประกาศก่อนจะถูกเรียกใช้งานจริง มิฉะนั้นโปรแกรมจะเกิดข้อผิดพลาดระหว่างรัน

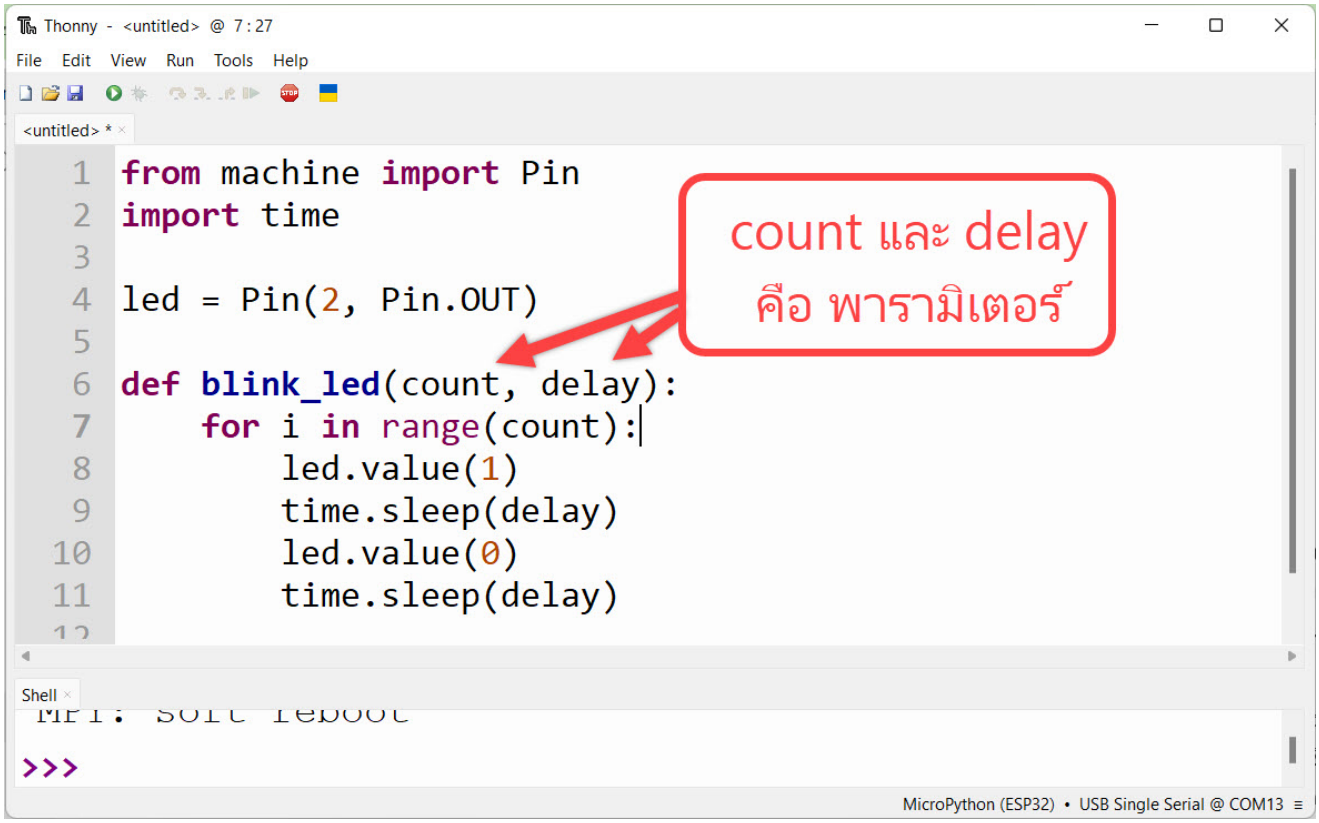
กล่าวโดยสรุป การสร้างฟังก์ชันใน MicroPython เป็นทักษะพื้นฐานที่ช่วยให้โปรแกรมมีความเป็นระบบมากขึ้น จัดระเบียบคำสั่งได้อย่างมืออาชีพ ลดข้อผิดพลาดและเพิ่มประสิทธิภาพในการพัฒนาโค้ด ไม่ว่าจะเป็นงานควบคุมฮาร์ดแวร์ การประมวลผลข้อมูล หรือสร้างฟังก์ชันช่วยเหลือย่อย ๆ สำหรับงานเฉพาะทาง ฟังก์ชันจึงเป็นเสาหลักของการพัฒนาโปรแกรมที่ดีในโลกของสมองกลฝังตัว

4.5.1 การรับพารามิเตอร์ (Function Parameters)

ในการเขียนโปรแกรมเชิงโครงสร้างหรือเชิงวัตถุ หนึ่งในคุณสมบัติสำคัญที่ทำให้โค้ดมีความยืดหยุ่น สามารถนำกลับมาใช้ซ้ำ และรองรับสถานการณ์ที่หลากหลายได้อย่างแท้จริง ก็คือ “ฟังก์ชัน” (Function) และฟังก์ชันจะมีพลังอย่างเต็มที่ก็ต่อเมื่อสามารถรับข้อมูลจากภายนอกเข้ามาประมวลผลได้ โดยข้อมูลเหล่านี้ถูกเรียกว่า “พารามิเตอร์” (parameter) ในขณะที่ข้อมูลที่ถูกส่งเข้ามาในแต่ละครั้งเรียกว่า “อาร์กิวเมนต์” (argument) ซึ่งเป็นประเด็นสำคัญที่ผู้เรียนทุกคนควรทำความเข้าใจให้ถูกต้อง

หากเปรียบเทียบกับสถานการณ์ในชีวิตประจำวัน การทำงานของพารามิเตอร์และอาร์กิวเมนต์อาจคล้ายกับการมี “ยาม” เฝ้าหน้าห้องประชุม เมื่อมีใครต้องการนำของบางอย่างเข้าไปในห้อง จะต้องส่งของนั้นให้กับยามก่อนเสมอ ซึ่งในที่นี้ “ยาม” ก็คือ parameter ที่ถูกเตรียมไว้ให้รับของ ส่วน “ของจริง” ที่นำมาส่งให้ยามในแต่ละครั้งนั้นก็คือ argument และทุกครั้งที่ยามได้รับของจริง ยามก็จะนำของเหล่านั้นเข้าไปในห้องประชุมเพื่อให้คนในห้อง (ซึ่งเปรียบได้กับคำสั่งที่อยู่ในฟังก์ชัน) นำไปใช้ประโยชน์ในการดำเนินการตามหน้าที่ต่อไป ดังนั้น parameter จึงทำหน้าที่เป็น “ประตูรับข้อมูล” จากภายนอก ขณะที่ argument คือข้อมูลจริงที่เดินทางผ่านประตูนั้นเข้าไปข้างใน เพื่อให้โปรแกรมสามารถปรับเปลี่ยนการทำงานได้ตามข้อมูลที่ได้รับ

ตัวอย่างเช่น ใน MicroPython หากต้องการสั่งให้ LED กระพริบจำนวนรอบที่ไม่แน่นอน และสามารถเปลี่ยนแปลงความเร็วในการกระพริบได้ทุกครั้งที่เรียกใช้งาน สามารถสร้างฟังก์ชันขึ้นมาโดยกำหนด parameter สำหรับรับค่าจำนวนรอบ (count) และเวลาหน่วง (delay) ตามนี้

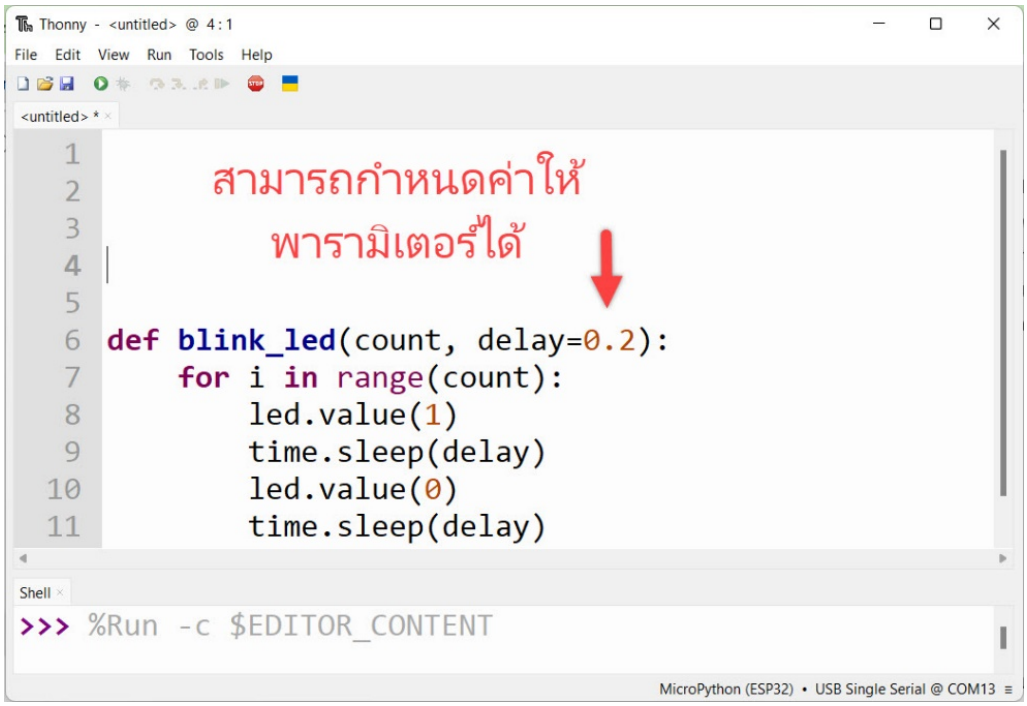


ในการประกาศฟังก์ชันข้างต้น count และ delay คือ parameter ที่ฟังก์ชันนี้เตรียมไว้รอรับข้อมูลจริงจากภายนอก ส่วนในขณะที่มีการเรียกใช้งาน เช่น blink_led(5, 0.2) ค่าจริงที่ส่งเข้าไปในแต่ละครั้ง (5 กับ 0.2) คือ argument ซึ่งเดินทางผ่านยาม (parameter) เข้าไปในห้องประชุม (ฟังก์ชัน) เพื่อให้คำสั่งที่อยู่ข้างในนำไปใช้งานจริง กล่าวอีกนัยหนึ่ง parameter คือชื่อที่ฟังก์ชันประกาศไว้เป็น “ที่รับข้อมูล” ส่วน argument คือค่าจริงที่ผู้ใช้โปรแกรมจะส่งเข้ามาให้ทุกครั้งที่ต้องการใช้งานฟังก์ชันนี้

ความแตกต่างสำคัญระหว่าง parameter กับ argument คือ parameter จะมีอยู่ในขณะประกาศฟังก์ชันเท่านั้น โดยเปรียบได้กับช่องที่เตรียมไว้ในใบรับของที่ยามต้องถือรอ ส่วน argument จะมีอยู่จริงในขณะเรียกใช้ฟังก์ชัน โดยเป็นของจริงที่ถูกส่งให้ยามในแต่ละครั้ง ดังนั้น parameter จึงเปรียบเสมือน “ช่องทางผ่าน” ที่รอรับข้อมูล ในขณะที่ argument คือข้อมูลที่เติมเข้ามาในช่องนั้นเพื่อถูกนำไปใช้งานจริง

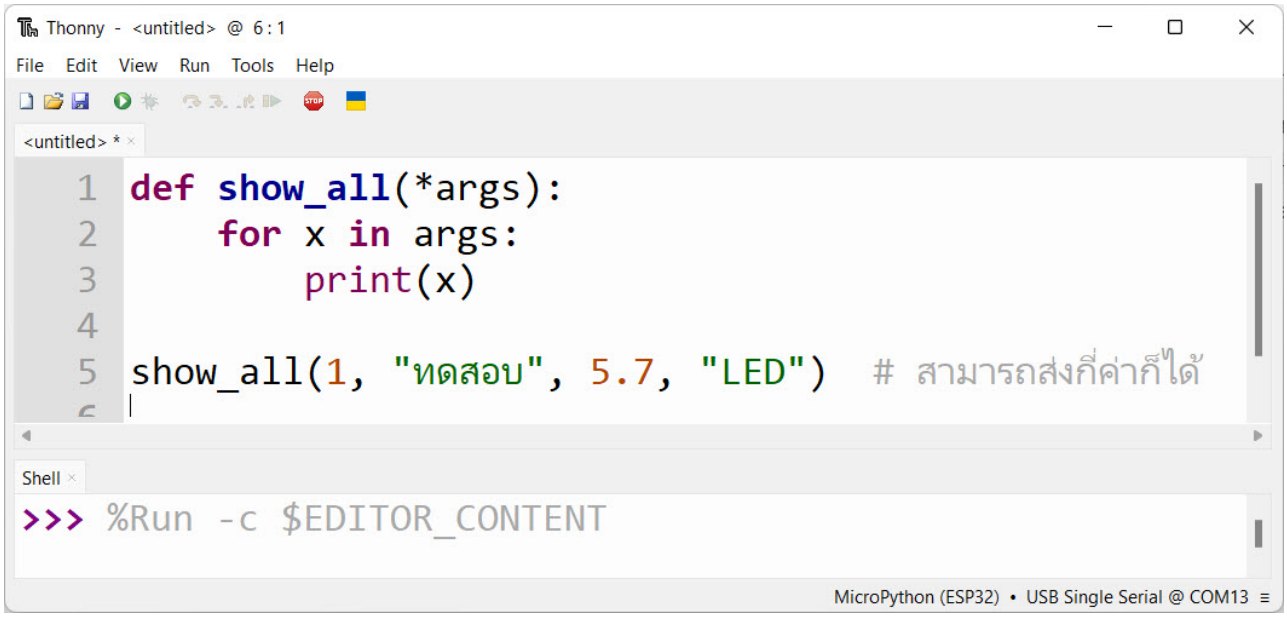
ข้อดีของการใช้ parameter คือ ทำให้ฟังก์ชันหนึ่ง ๆ มีความยืดหยุ่นสูง สามารถนำกลับมาใช้ใหม่กับข้อมูลที่หลากหลายได้ในโปรแกรมเดียวกัน ไม่ต้องเขียนฟังก์ชันใหม่สำหรับแต่ละกรณี เช่น ต้องการให้ LED กระพริบ 3 ครั้ง หรือ 10 ครั้ง เพียงแค่เรียกฟังก์ชัน blink_led ด้วย argument ที่ต่างกัน ทุกอย่างก็ทำงานได้โดยไม่ต้องแก้ไขโค้ดภายในฟังก์ชันแต่อย่างใด และหากต้องการเปลี่ยนแปลงความเร็วในการกระพริบ ก็สามารถส่ง argument ของเวลาหน่วงที่ต่างกันได้ทันที ตัวอย่างเช่น blink_led(10, 0.1) จะสั่งให้ LED กระพริบ 10 ครั้งด้วยความเร็วสูง ในขณะที่ blink_led(3, 0.5) จะกระพริบ 3 ครั้งช้ากว่ามาก ทุกอย่างนี้เกิดขึ้นเพราะ parameter เปิดทางให้โปรแกรมรองรับ argument ที่หลากหลาย

นอกจากนี้ MicroPython ยังอนุญาตให้กำหนดค่าเริ่มต้นให้ parameter ได้ เช่น



ในกรณีนี้ หากเรียก blink_led(4) โดยไม่ระบุ delay ฟังก์ชันก็จะใช้ค่า delay เท่ากับ 0.2 วินาทีโดยอัตโนมัติ แต่หากต้องการเปลี่ยนค่า delay ในบางกรณีก็สามารถใส่ argument ใหม่ในขณะที่เรียกใช้งานได้ทันที

อีกหนึ่งความยืดหยุ่นคือการใช้ *args สำหรับรับ argument จำนวนไม่จำกัด และรับข้อมูลต่างชนิดกันได้



ข้อควรระวังในการใช้งาน parameter และ argument คือ ควรตั้งชื่อ parameter ให้ชัดเจน สื่อความหมายตรงกับสิ่งที่จะรับข้อมูลมาใช้งานจริง และต้องตรวจสอบว่าในขณะที่เรียกใช้ฟังก์ชัน ได้ส่ง argument ตามที่ parameter รอรับไว้ครบถ้วน หากขาดหรือส่งผิดประเภท จะทำให้ฟังก์ชันทำงานผิด หรือเกิดข้อผิดพลาดได้ทันที การวางโครงสร้างของ parameter และการเลือกส่ง argument ที่เหมาะสมจึงเป็นหัวใจสำคัญในการเขียนโปรแกรมที่ดี การทำความเข้าใจเรื่อง parameter และ argument อย่างลึกซึ้ง ช่วยให้สามารถออกแบบฟังก์ชันที่ใช้งานได้จริงในทุกโปรเจกต์ MicroPython ไม่ว่าจะเป็นงานควบคุมอุปกรณ์ หรือการประมวลผลข้อมูล ฟังก์ชันที่ดีจะยืดหยุ่น รองรับข้อมูลหลากหลาย ทำงานได้อัตโนมัติ และลดความซ้ำซ้อนในการเขียนโค้ด ทั้งหมดนี้คือแก่นแท้ของ “การรับพารามิเตอร์” ที่นักเขียนโปรแกรมมืออาชีพต้องเข้าใจและใช้ประโยชน์ให้เป็น

3.5.3 การคืนค่าด้วย return (Return Values)

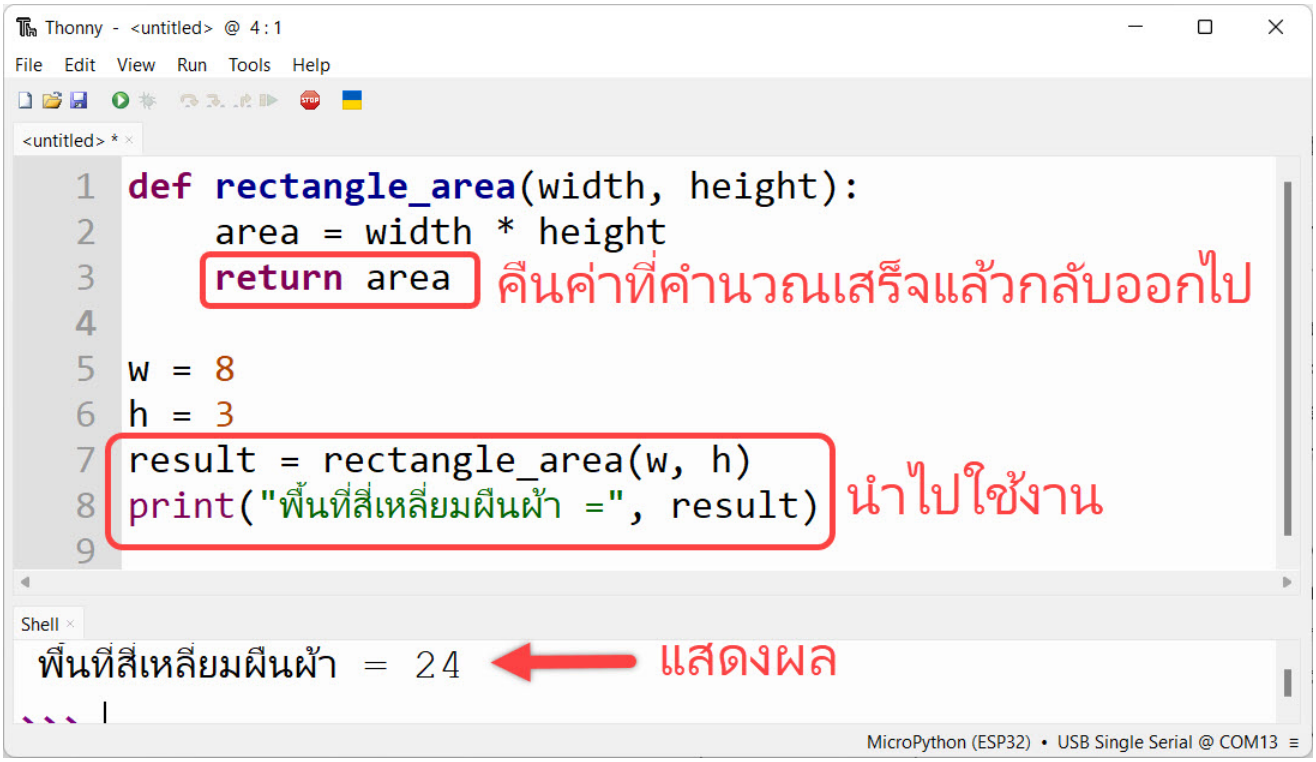
การเขียนฟังก์ชันใน MicroPython ไม่ได้มีเป้าหมายเพียงเพื่อแบ่งกลุ่มคำสั่งให้ใช้งานซ้ำเท่านั้น แต่หัวใจสำคัญอย่างหนึ่งก็คือ “การสื่อสารข้อมูลระหว่างฟังก์ชันกับโปรแกรมหลัก” ฟังก์ชันที่ดีจะไม่ใช่แค่รับข้อมูลจากภายนอก (ผ่าน parameter/argument) แล้วทำงานเงียบ ๆ แต่ยังสามารถ “ส่งผลลัพธ์กลับไป” เพื่อให้โปรแกรมหลักนำไปใช้ต่อยอดได้อย่างยืดหยุ่น องค์ประกอบนี้เองที่แยกโปรแกรมระดับเริ่มต้นออกจากโปรแกรมเชิงวิชาชีพและระบบอัตโนมัติทุกแขนง

ในชีวิตประจำวัน เรามักพบกระบวนการ “รับ–ส่ง–รับคืน” อยู่ตลอดเวลา ตัวอย่างเช่น หากเราเอาเสื้อผ้าไปซักที่ร้านซักรีด เมื่อฝากเสื้อผ้า (argument) ผ่านยามหน้าร้าน (parameter) เข้าสู่กระบวนการภายในร้าน (ฟังก์ชัน) ร้านจะทำการซักให้สะอาด ตรวจสอบเรียบร้อย แล้วเมื่อถึงเวลารับกลับ ร้านจะคืนเสื้อผ้าสะอาด (ผลลัพธ์) มาให้เจ้าของเสมอ หากไม่มีการ “คืนค่า” ออกมา การฝากซักนี้ก็ไร้ประโยชน์ทันที เช่นเดียวกัน ใน

โปรแกรม ถ้าเขียนฟังก์ชันขึ้นมาเพื่อประมวลผลหรือคำนวณสิ่งใดแล้วไม่ส่งข้อมูลกลับคืน โปรแกรมหลักก็จะไม่สามารถนำผลลัพธ์นั้นไปใช้ประโยชน์ต่อได้

ภาษา MicroPython และ Python ทั่วไปจึงออกแบบให้มีคำสั่ง **return** ซึ่งทำหน้าที่เป็น “ประตูส่งผลลัพธ์กลับออกไป” หลังจากฟังก์ชันทำงานเสร็จ เปรียบได้กับการที่ร้านซักรีดนำผ้าที่ซักเสร็จแล้วมาคืนเจ้าของ หรือในบริบทของสายงานอิเล็กทรอนิกส์ก็อาจเปรียบได้กับกระบวนการทดสอบชิ้นส่วน เมื่อลูกค้านำอุปกรณ์มาตรวจสอบ วิศวกร (ฟังก์ชัน) จะวัดค่า ทดสอบประสิทธิภาพ เมื่อเสร็จสิ้นก็จะคืนผลลัพธ์กลับไปให้เจ้าของ ไม่ว่าจะเป็น “ผ่าน/ไม่ผ่าน” หรือข้อมูลเชิงตัวเลขใด ๆ

โครงสร้างของการใช้ **return** ในฟังก์ชัน MicroPython มีความชัดเจน ตรงไปตรงมา เช่น เมื่อต้องการให้ฟังก์ชันประมวลผลข้อมูลแล้วส่งค่ากลับมา ก็ให้ใช้คำสั่ง return ตามด้วยค่าที่ต้องการคืนออกมา ตัวอย่างเช่น ฟังก์ชันคำนวณพื้นที่สี่เหลี่ยมผืนผ้า

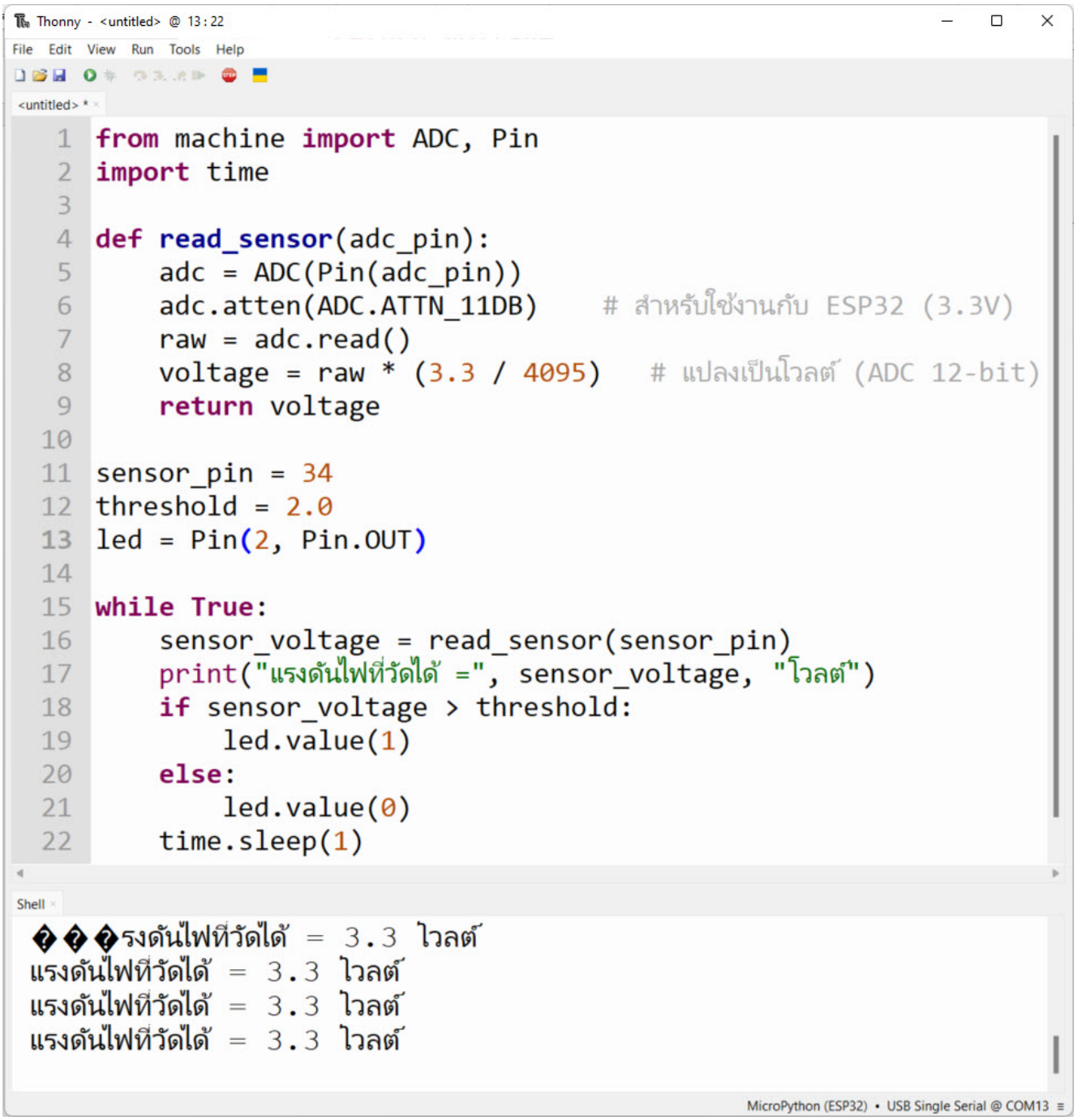


เมื่อโปรแกรมหลักเรียกใช้ฟังก์ชัน rectangle_area(w, h) ตัวแปร result จะได้รับค่าผลลัพธ์ที่ฟังก์ชันคำนวณแล้วส่งกลับมาให้โดยตรง ไม่ใช่เพียงแค่รันคำสั่งภายในแล้วจบไปเท่านั้น แต่ข้อมูลจะ “เดินทางกลับ” ออกจากฟังก์ชันและพร้อมใช้งานต่อทันทีในส่วนอื่นของโปรแกรม

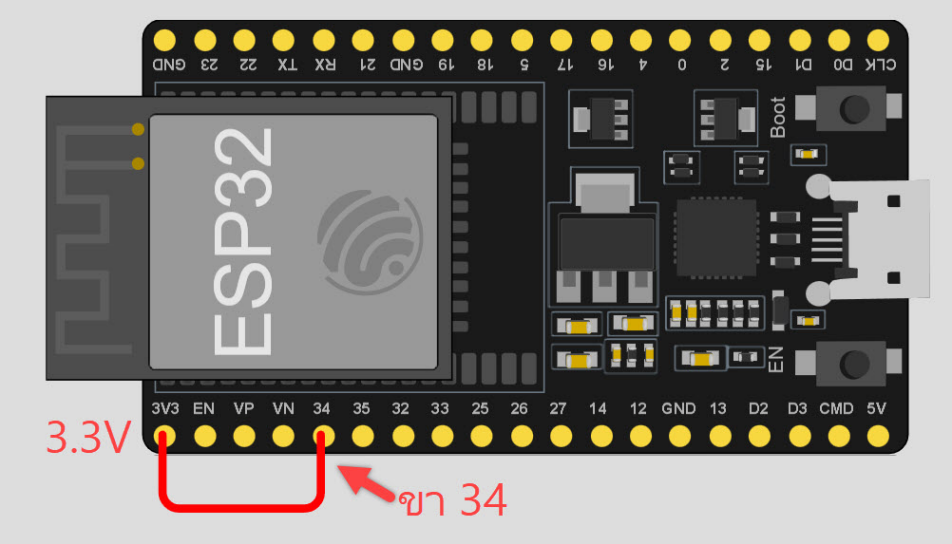
การคืนค่า (return value) เป็นหัวใจสำคัญของโปรแกรมที่เน้น “การวิเคราะห์และตัดสินใจอัตโนมัติ” เช่น หากเขียนระบบควบคุมหุ่นยนต์ หรือระบบวัดค่าสัญญาณในโรงงาน จะไม่มีทางรู้สถานะปัจจุบันได้เลยหากไม่มีการส่งค่ากลับหลังประมวลผลทุกขั้นตอน ฟังก์ชันที่ไม่มี return ก็เหมือน “กล่องดำ” ที่ข้อมูลไหลเข้าไปแล้วไม่มีทางรู้ว่าเกิดอะไรขึ้นหรือได้ผลลัพธ์อะไรกลับมา

ตัวอย่างการประยุกต์กับ ESP32 ที่ทดลองได้จริง

ในงานควบคุมหรืออ่านค่าจากเซ็นเซอร์ เรามักต้องการนำค่าที่อ่านได้จากฟังก์ชันไปตัดสินใจต่อ เช่น สั่งให้ LED ติดเมื่อเซ็นเซอร์มีแรงดันไฟสูงกว่าเกณฑ์ที่กำหนด ฟังก์ชันควรคืนค่าให้โปรแกรมหลักนำไปประเมินหรือสั่งงานอุปกรณ์อื่นต่อได้ทันที

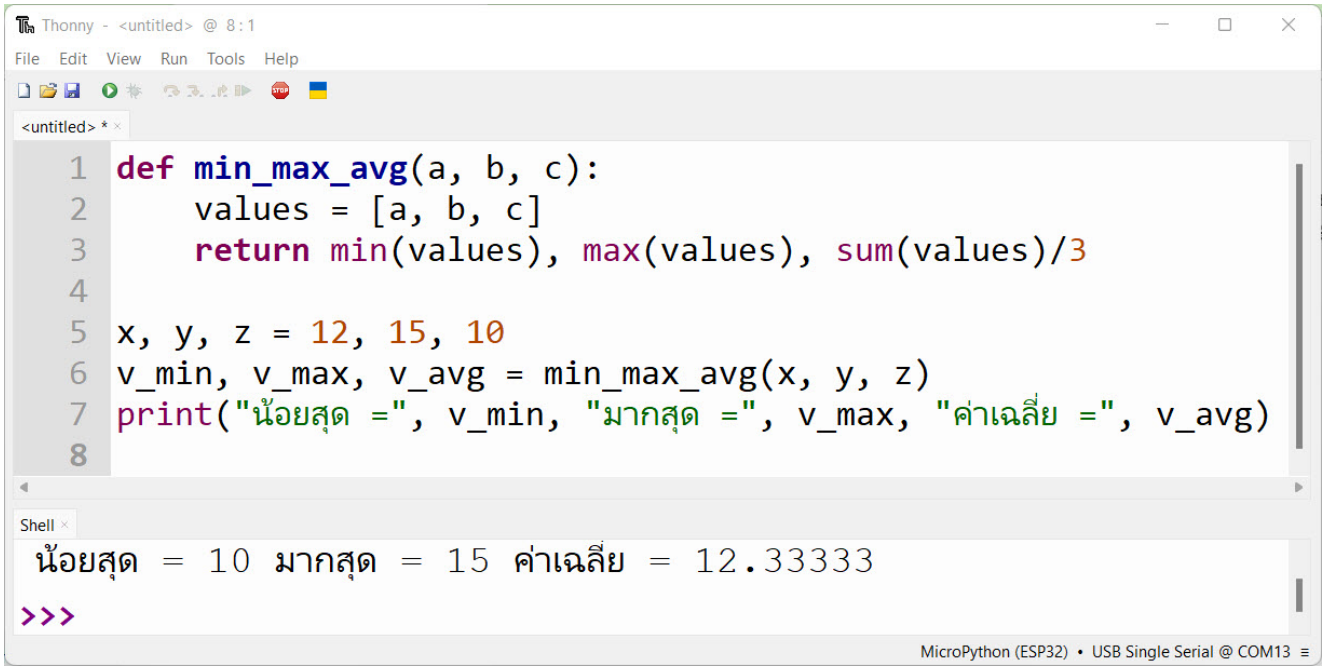


โค้ดตัวอย่างนี้สามารถทดลองได้จริงบนบอร์ด ESP32 หากต่อเซ็นเซอร์ที่ขา 34 ผลลัพธ์จากการอ่านแรงดันจะถูกคืนค่ากลับจากฟังก์ชัน read_sensor() โปรแกรมหลักจะนำไปใช้เปรียบเทียบกับค่า threshold และควบคุม LED ให้ติดหรือดับได้แบบเรียลไทม์ หรือหากไม่มีเซ็นเซอร์ ให้ทดลอง ต่อแรงดันไฟ 3.3V เข้าไปที่ขา 34 ได้เลย



ข้อควรเข้าใจอย่างยิ่งคือ เมื่อโปรแกรมพบคำสั่ง return ภายในฟังก์ชัน จะหยุดทำงานส่วนที่เหลือของฟังก์ชันทันที ไม่ว่าคำสั่งหลัง return จะเป็นอะไร ก็จะถูกข้ามไป ฟังก์ชันจึงจบการทำงานโดยสมบูรณ์ ณ จุดนั้น สิ่งนี้มีความสำคัญต่อการออกแบบโปรแกรมที่ต้องควบคุมลำดับขั้นตอนหรือเลือกหยุดการทำงานบางกรณีโดยเฉพาะ

อีกประเด็นที่เป็นจุดเด่นของ MicroPython คือการคืนค่ามากกว่าหนึ่งค่าในฟังก์ชันเดียวได้ทันที เพียงเขียนค่าทุกค่าคั่นด้วยเครื่องหมายจุลภาค (comma) โปรแกรมหลักก็สามารถแยกนำไปใช้ตามต้องการ เช่น กรณีวัดค่าข้อมูลหลายอย่างจากเซ็นเซอร์



ฟังก์ชันนี้คืนค่า “น้อยสุด มากสุด และค่าเฉลี่ย” ในครั้งเดียว ช่วยให้โปรแกรมนำไปประมวลผลต่อได้สะดวกและแม่นยำมากขึ้น นอกจากนี้ ในการเขียนฟังก์ชันสำหรับงานควบคุม เช่น สั่งงานอุปกรณ์ต่าง ๆ เราสามารถใช้ return เป็นตัวเชื่อมลำดับการทำงาน เช่น เมื่อสั่งวัดค่าเซ็นเซอร์ก่อน ถ้าได้ค่าอยู่ในช่วงปลอดภัยจึง return True

เพื่อบอกให้ระบบไปดำเนินการส่วนอื่นต่อ แต่ถ้าพบความผิดปกติก็ return False แล้วโปรแกรมหลักจะหยุดหรือตัดสินใจตามผลลัพธ์นั้นได้ทันที

ข้อควรระวัง คือ ฟังก์ชันที่ไม่มี return หรือไม่มีค่าใน return จะคืนค่า None เสมอ (None คือค่าว่างในภาษา Python/MicroPython) ดังนั้น ถ้าต้องการนำค่าผลลัพธ์ไปใช้ต่อเสมอ ควรตรวจสอบว่าฟังก์ชันได้คืนค่าที่คาดหวังจริงหรือไม่ และในการออกแบบฟังก์ชันที่คืนค่ามากกว่าหนึ่งค่า ควรระวังเรื่องลำดับการรับค่ากลับในโปรแกรมหลักด้วย เพื่อป้องกันการสลับข้อมูลโดยไม่ตั้งใจ

ดังนั้นจึงสรุปได้ว่า “การคืนค่าด้วย return” คือหัวใจที่ทำให้ฟังก์ชันใน MicroPython ไม่เพียงแต่รับข้อมูลเข้าไปประมวลผล แต่ยังตอบกลับและเชื่อมโยงการทำงานให้โปรแกรมหลักสามารถตัดสินใจ ประมวลผล หรือควบคุมอุปกรณ์ต่อไปได้อย่างมีประสิทธิภาพ เป็นหลักการสำคัญที่ทำให้โปรแกรมมีความฉลาดและยืดหยุ่น พร้อมรับมือกับงานจริงในสายอิเล็กทรอนิกส์และ IoT ได้อย่างเต็มประสิทธิภาพ

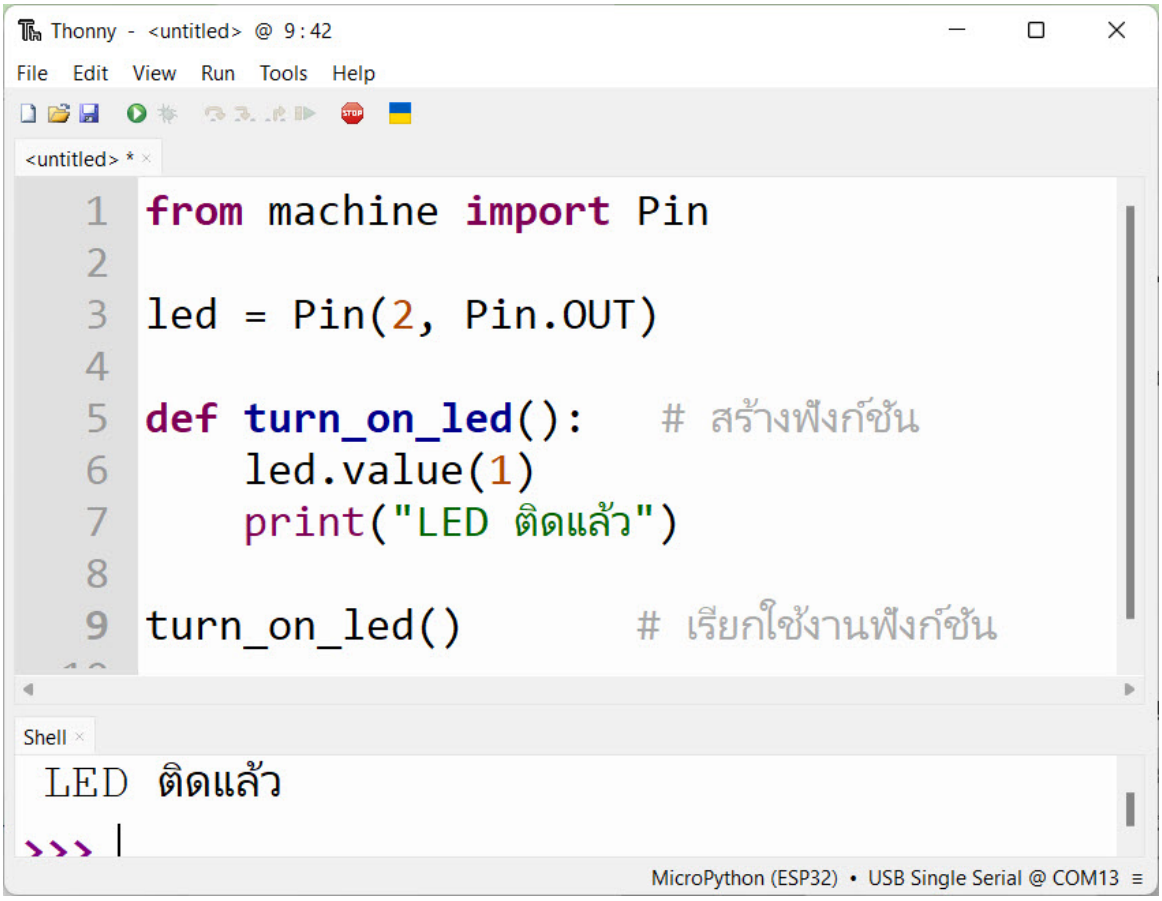
3.5.4 ฟังก์ชันแบบไม่รับค่าหรือไม่คืนค่า

เมื่อศึกษาการประกาศฟังก์ชันใน MicroPython จะพบว่าฟังก์ชันทุกชนิด “ไม่จำเป็นต้องรับค่าพารามิเตอร์เสมอไป” และ “ไม่จำเป็นต้องคืนค่ากลับออกมาเสมอไป” ก็ได้ ซึ่งลักษณะนี้พบได้บ่อยในการพัฒนาโปรแกรมควบคุมอุปกรณ์หรือระบบฝังตัวที่ต้องการสั่งให้ดำเนินการบางอย่างแบบตรงไปตรงมา เช่น สั่งให้ LED ติด/ดับ สั่งแสดงข้อความ หรือตั้งค่าพอร์ตต่าง ๆ โดยไม่จำเป็นต้องส่งค่าผลลัพธ์กลับมา หรือไม่ต้องการรับข้อมูลจากโปรแกรมหลักเข้าไปใช้ในฟังก์ชันเลย

ฟังก์ชันแบบไม่รับค่า (No Parameter Function)

ฟังก์ชันประเภทนี้คือฟังก์ชันที่ไม่ต้องรับข้อมูลจากภายนอก ไม่ต้องมีพารามิเตอร์ในวงเล็บขณะประกาศ โดยจะทำหน้าที่เหมือนกับ “คนที่ทำงานประจำหน้าที่เดิมซ้ำ ๆ” ทุกครั้งที่ถูกเรียกใช้ ตัวอย่างในชีวิตประจำวัน เช่น “แม่บ้านเปิดไฟในบ้านทุกเช้า” โดยไม่ต้องถามข้อมูลจากใคร หรือ “ระบบนาฬิกาปลุกที่ตั้งเวลาคงที่ไว้แล้ว” จะทำงานของมันเองโดยอัตโนมัติ

ตัวอย่างใน MicroPython ที่ทดลองได้จริง เช่น การสร้างฟังก์ชันสำหรับสั่งเปิด LED ทันทีที่เรียกใช้ โดยไม่รับข้อมูลใด ๆ

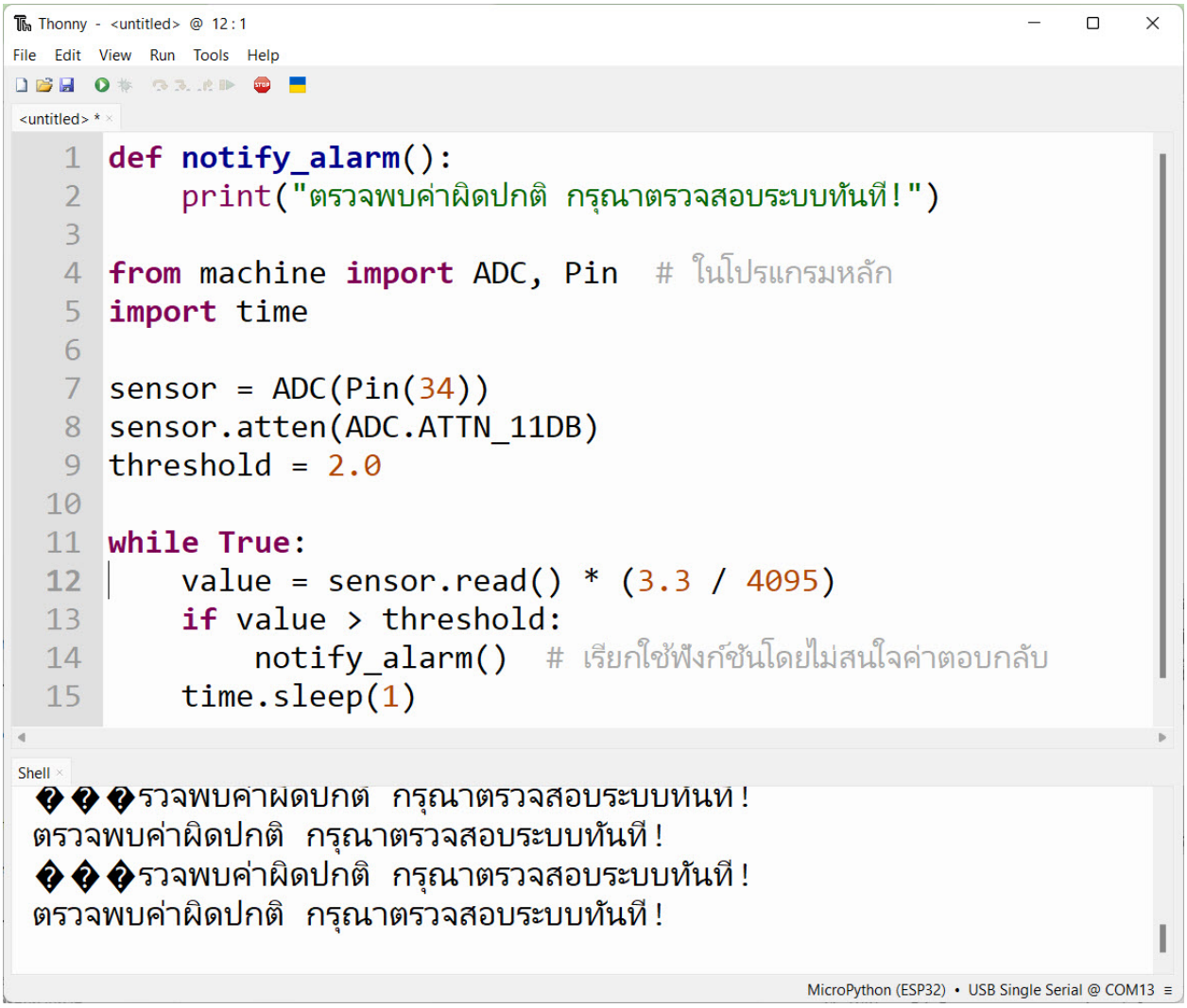


ในตัวอย่างนี้ ฟังก์ชัน turn_on_led() ไม่ต้องรับค่าใด ๆ ทั้งสิ้น เพียงแค่เรียกใช้ก็จะสั่งให้ LED ติด และพิมพ์ข้อความออกมาได้ทันที การออกแบบฟังก์ชันลักษณะนี้เหมาะกับงานที่ต้องการความเรียบง่าย ไม่มีความซับซ้อนเรื่องข้อมูลที่เปลี่ยนแปลงบ่อย หรือใช้สำหรับสั่งงานบางอย่างที่มีขั้นตอนตายตัว

ฟังก์ชันแบบไม่คืนค่า (No Return Value Function)

ฟังก์ชันที่ “ไม่คืนค่า” คือฟังก์ชันที่ไม่มีคำสั่ง return อยู่ในตัว หรือมีแต่ไม่มีค่าตามหลัง return ซึ่งในกรณีนี้ฟังก์ชันจะทำงานตามคำสั่งที่อยู่ข้างในเท่านั้น และเมื่อจบแล้วจะไม่ส่งผลลัพธ์ใด ๆ กลับไปยังโปรแกรมหลัก เปรียบเสมือน “คนที่ทำงานให้เราเสร็จแล้วก็เดินกลับ ไม่รายงานผล ไม่ส่งอะไรคืน” เช่น การสั่งเปิดปิดไฟบ้าน หรือแจ้งเตือนผู้ใช้ผ่านเสียงหรือแสง แต่ไม่ต้องการทราบผลตอบกลับ

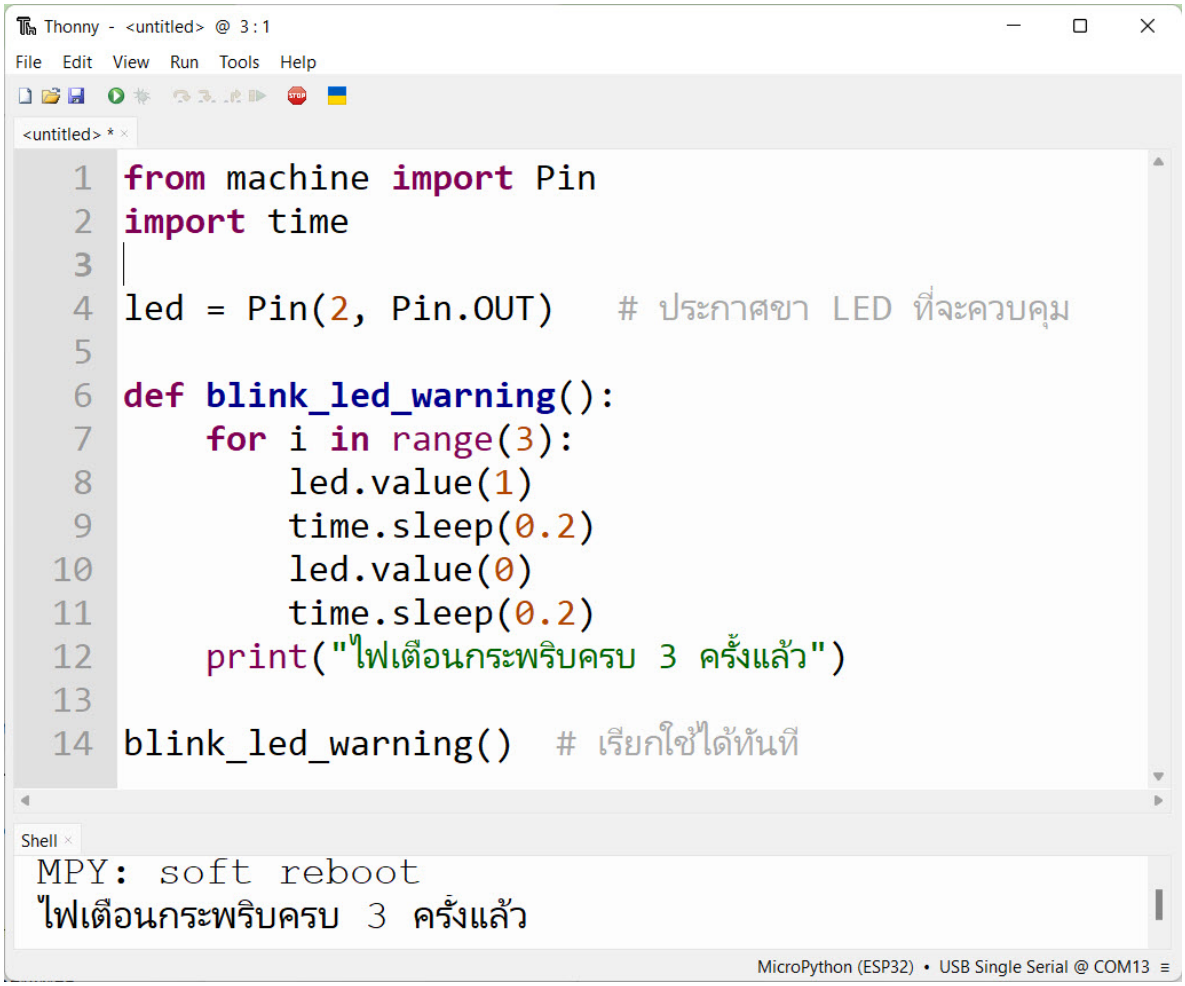
ตัวอย่างที่ทดลองได้จริง เช่น ฟังก์ชันที่ใช้สำหรับแสดงข้อความบน Serial Monitor ทุกครั้งที่เซ็นเซอร์วัดค่าเกินกำหนด



ในตัวอย่างนี้ ฟังก์ชัน notify_alarm() ไม่มี return ค่ากลับมา เมื่อถูกเรียกจะเพียงแค่แสดงข้อความเตือนออกทาง Serial Monitor เท่านั้น โปรแกรมหลักไม่ต้องสนใจว่าฟังก์ชันส่งข้อมูลอะไรกลับมา (จริง ๆ แล้วค่าที่ได้จะเป็น None เสมอหากเขียน result = notify_alarm() แล้ว print(result) จะได้ None) จุดนี้เหมาะกับการแจ้งเตือนหรือดำเนินการที่ “รู้ผลแล้วจบ” ไม่ต้องนำไปประมวลผลต่อ

ฟังก์ชันที่ไม่รับค่าและไม่คืนค่าเลย

ในบางสถานการณ์ อาจมีฟังก์ชันที่ทั้ง “ไม่รับค่า” และ “ไม่คืนค่า” เช่น การสั่งเปิดเสียงเตือน, สั่งรีเซ็ตระบบ หรือสั่งกระพริบไฟเตือนตามค่าที่กำหนดไว้ในโค้ด โดยไม่ต้องส่งค่ากลับมาหรือรับค่าใด ๆ จากภายนอก ตัวอย่างนี้เหมาะกับคำสั่งควบคุมอุปกรณ์หรือโปรเซสที่ต้องทำงานเหมือนกันทุกครั้งที่เรียกใช้



ฟังก์ชันที่ทำงานแบบ (ไม่รับค่า / ไม่คืนค่า) มีบทบาทสำคัญในการออกแบบโปรแกรมควบคุมที่ต้องการความเรียบง่าย ประหยัดบรรทัด ไม่ซับซ้อนกับการจัดการข้อมูลมากเกินจำเป็น โดยควรเลือกใช้ให้เหมาะกับวัตถุประสงค์ เช่น งานควบคุมพื้นฐาน งานแจ้งเตือน หรือสั่งงานที่มีขั้นตอนแน่นอนทุกครั้ง ฟังก์ชันแบบนี้จะช่วยให้โปรแกรมเป็นระบบ ระเบียบ และดูแลรักษาง่ายในระยะยาว

3.5.5 Lambda Functions (ฟังก์ชันนิพจน์)

ในการเขียนโปรแกรมด้วยภาษา MicroPython หรือ Python ทั่วไป ฟังก์ชันนิพจน์ หรือ Lambda Function คือฟังก์ชันชนิดหนึ่งที่ไม่มีชื่ออย่างเป็นทางการ (anonymous function) ถูกออกแบบมาเพื่อใช้งานชั่วคราว งานเฉพาะกิจ หรือเมื่อต้องการนิยามฟังก์ชันขนาดเล็กที่จบในนิพจน์เดียว จุดเด่นของ Lambda Function คือสามารถสร้างได้ “ทันทีในจุดที่ต้องการใช้” โดยไม่ต้องประกาศแยกไว้ล่วงหน้าแบบฟังก์ชันปกติ (def) เหมาะกับการประมวลผลข้อมูลที่เปลี่ยนแปลงบ่อย หรือเมื่อต้องการส่งฟังก์ชันเข้าไปในฟังก์ชันอื่น เช่น map, filter, sorted ฯลฯ

Lambda Function เปรียบเสมือน “คนงานรับจ้างชั่วคราว” หรือ “แม่บ้านรับจ้างรายวัน” ที่มาทำงานเฉพาะกิจเพียงครั้งเดียว เสร็จงานแล้วก็จบ ไม่ต้องบันทึกชื่อในระบบหรือจัดเก็บไว้ใช้งานระยะยาว ต่างจากฟังก์ชันปกติที่เปรียบเหมือนพนักงานประจำบริษัท ต้องมีชื่อ มีหน้าที่ชัดเจน และอยู่ประจำในโค้ดตลอดไป Lambda Function จึงเหมาะกับงานเล็ก ๆ ที่ไม่ซับซ้อน ใช้แล้วจบในที่เดียว

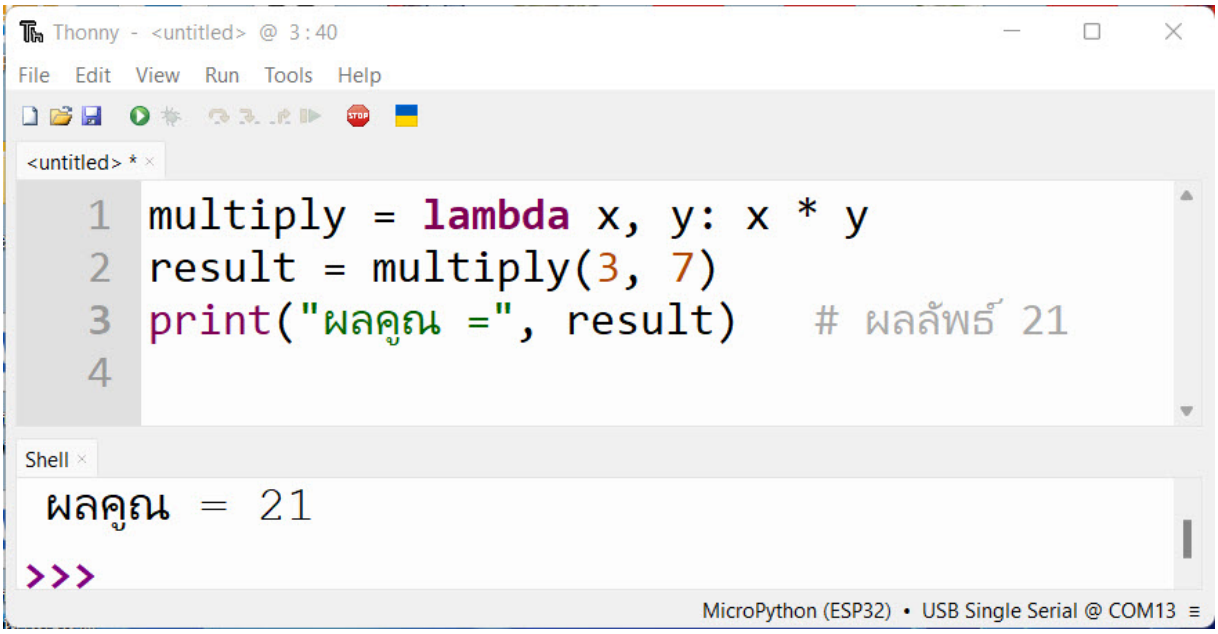
โครงสร้างและรูปแบบของ Lambda Function

การสร้าง Lambda Function ใน MicroPython หรือ Python ใช้โครงสร้างดังนี้

lambda parameter_list : expression

โดย parameter_list คือรายชื่อพารามิเตอร์ (สามารถมีหรือไม่มีเลยก็ได้) และ expression คือ “นิพจน์” (expression) หรือการคำนวณใด ๆ ที่ต้องการให้ฟังก์ชันส่งค่ากลับ โดย Lambda Function สามารถคืนค่าได้เพียงหนึ่งนิพจน์เท่านั้น ไม่สามารถเขียนหลายบรรทัดหรือหลายคำสั่งได้เหมือนฟังก์ชันปกติ

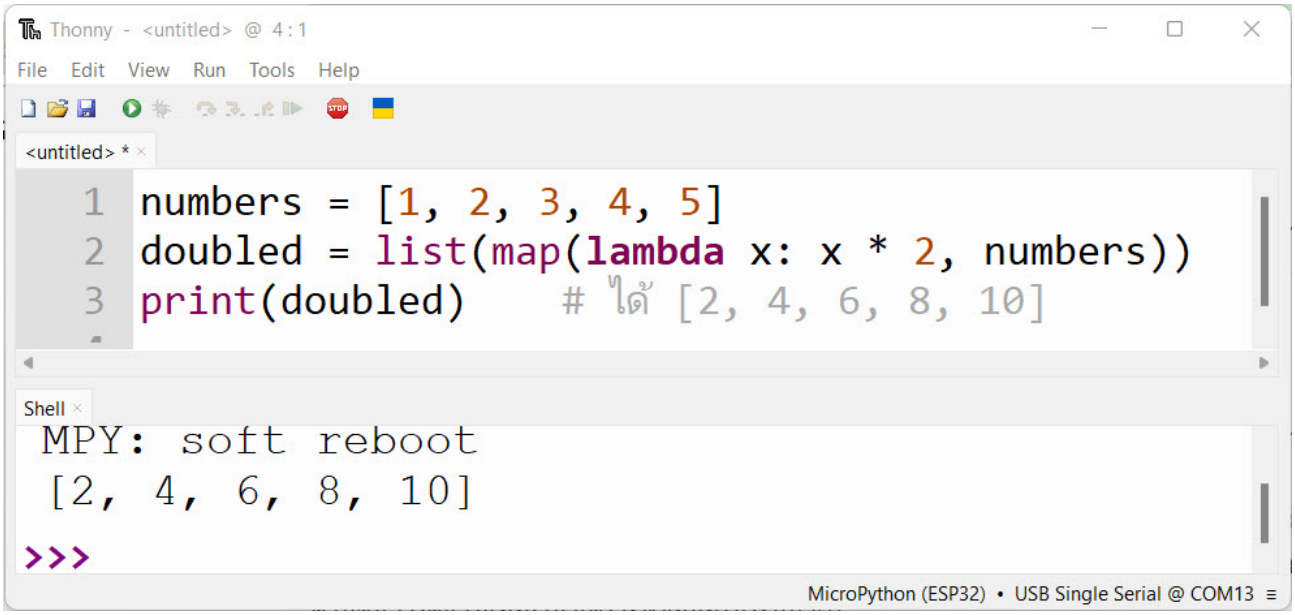
ตัวอย่าง



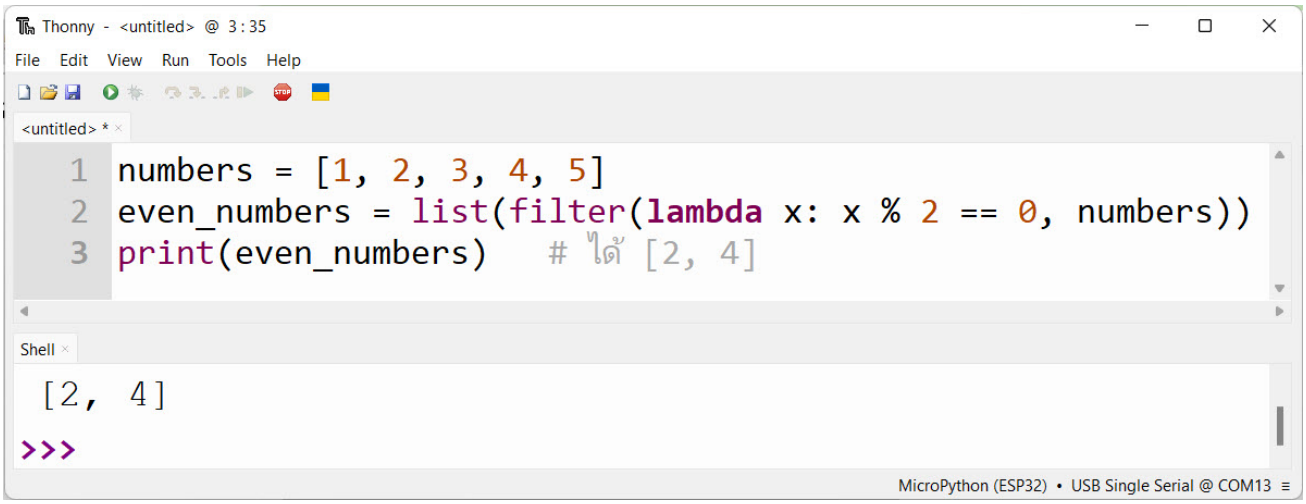
การประกาศ Lambda Function จะไม่ต้องใช้ชื่อฟังก์ชันเหมือน def แต่เรามักกำหนดตัวแปรเพื่อเก็บ Lambda Function ไว้ใช้งานต่อ (ในที่นี้คือ multiply) การเรียกใช้งานก็เหมือนกับฟังก์ชันปกติทุกประการ

ตัวอย่างการใช้งาน Lambda Function กับรายการข้อมูล

จุดแข็งสำคัญของ Lambda Function คือการนำไปใช้งานคู่กับฟังก์ชันประมวลผลข้อมูล เช่น map, filter, sorted ฯลฯ โดยที่ไม่ต้องสร้างฟังก์ชัน def แยกไว้ในโค้ด ตัวอย่างเช่น หากต้องการคูณค่าทุกตัวในลิสต์ numbers ด้วย 2 สามารถเขียนได้ว่า



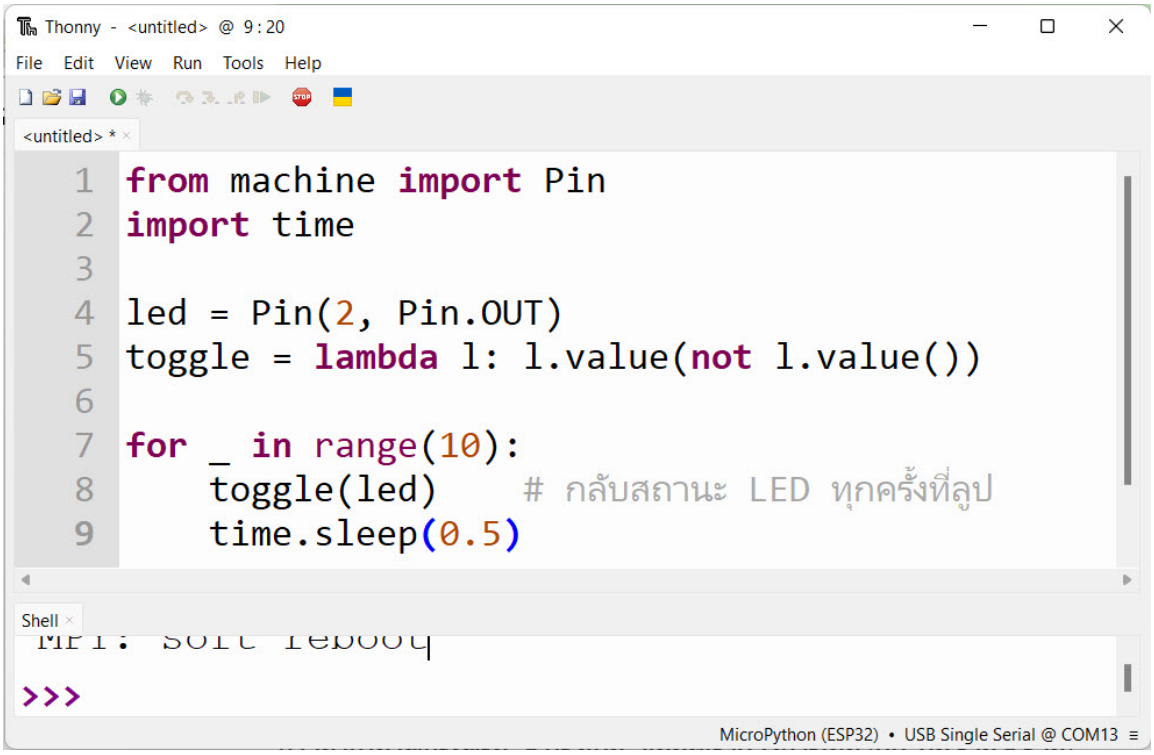
Lambda Function จะถูกส่งเข้าไปใน map เพื่อใช้ประมวลผลข้อมูลแต่ละตัวทันที หรือหากต้องการกรองเฉพาะเลขคู่ในลิสต์ สามารถใช้ filter ได้ดังนี้



ตัวอย่างเหล่านี้พบได้บ่อยในงานประมวลผลข้อมูล, รายการอุปกรณ์, หรือการคำนวณซ้ำ ๆ ในงานควบคุมที่ต้องการความรวดเร็ว

การประยุกต์ Lambda Function กับงานควบคุมฮาร์ดแวร์บน ESP32

Lambda Function ไม่ได้จำกัดเฉพาะงานประมวลผลข้อมูลเท่านั้น ในงานควบคุมอุปกรณ์หรือระบบสมองกลฝังตัว Lambda Function ยังช่วยให้โค้ดควบคุมดูสะอาด กระชับ และง่ายต่อการทดลอง เช่น หากต้องการนิยามฟังก์ชันสำหรับ “กลับสถานะ LED” ที่ขา 2 ของ ESP32 สามารถเขียนแบบ lambda ได้ทันทีโดยไม่ต้องประกาศ def แยก



เมื่อรันโค้ดนี้ LED จะกระพริบ 5 ครั้งแบบรวบรัดด้วย Lambda Function เพียงบรรทัดเดียว เหมาะกับงานเฉพาะกิจ เช่น ทดสอบวงจร, เช็คสถานะ หรือ callback ที่ไม่ต้องการใช้ def ให้เปลืองพื้นที่โค้ด

ข้อดี ข้อจำกัด และข้อควรระวัง

Lambda Function มีจุดเด่นด้าน “ความรวดเร็วและกระชับ” เหมาะกับงานที่ต้องการฟังก์ชันสั้น ๆ ใช้ครั้งเดียวจบ หรือใช้เป็น argument ส่งเข้า map/filter/sorted แต่ Lambda Function ไม่เหมาะกับงานที่มีความซับซ้อนหรือมีหลายขั้นตอน เพราะอนุญาตให้มีเพียงนิพจน์เดียวเท่านั้น หากต้องการฟังก์ชันที่มีคำสั่งมากกว่า 1 บรรทัด หรือมีการควบคุมเงื่อนไขซับซ้อน ควรเลือกใช้ def ตามปกติ ซึ่งจะช่วยให้โค้ดอ่านง่าย แก้ไขหรือบำรุงรักษาง่ายในระยะยาว

ข้อควรระวัง คือ Lambda Function ไม่มีชื่อ (anonymous) และอาจทำให้ debug หรือค้นหาความผิดพลาดในโค้ดได้ยากกว่าฟังก์ชันปกติ อีกทั้งการใช้ Lambda Function ซ้อนกันหลายระดับ อาจทำให้ผู้อ่านใหม่หรือมือใหม่สับสนได้ง่าย

Lambda Function เป็นเครื่องมือสำคัญของนักพัฒนา MicroPython ที่ต้องการ “ฟังก์ชันแบบเฉพาะกิจ” เพื่อให้โค้ดดูโปร่ง โล่ง และง่ายต่อการประมวลผลข้อมูลหรือควบคุมอุปกรณ์ในงานจริง Lambda Function ช่วยประหยัดบรรทัด ลดความซ้ำซ้อน และเหมาะกับงานที่ไม่ต้องการเก็บฟังก์ชันระยะยาว หรือไม่ต้องการสร้าง def แยกให้เปลืองพื้นที่โค้ด หากเลือกใช้อย่างเหมาะสม Lambda Function จะเป็นผู้ช่วยสำคัญของงานพัฒนา IoT และระบบสมองกลฝังตัวที่เน้นความเร็วและกระชับสูงสุด

3.5.6 ฟังก์ชันซ้อน (Nested Functions)

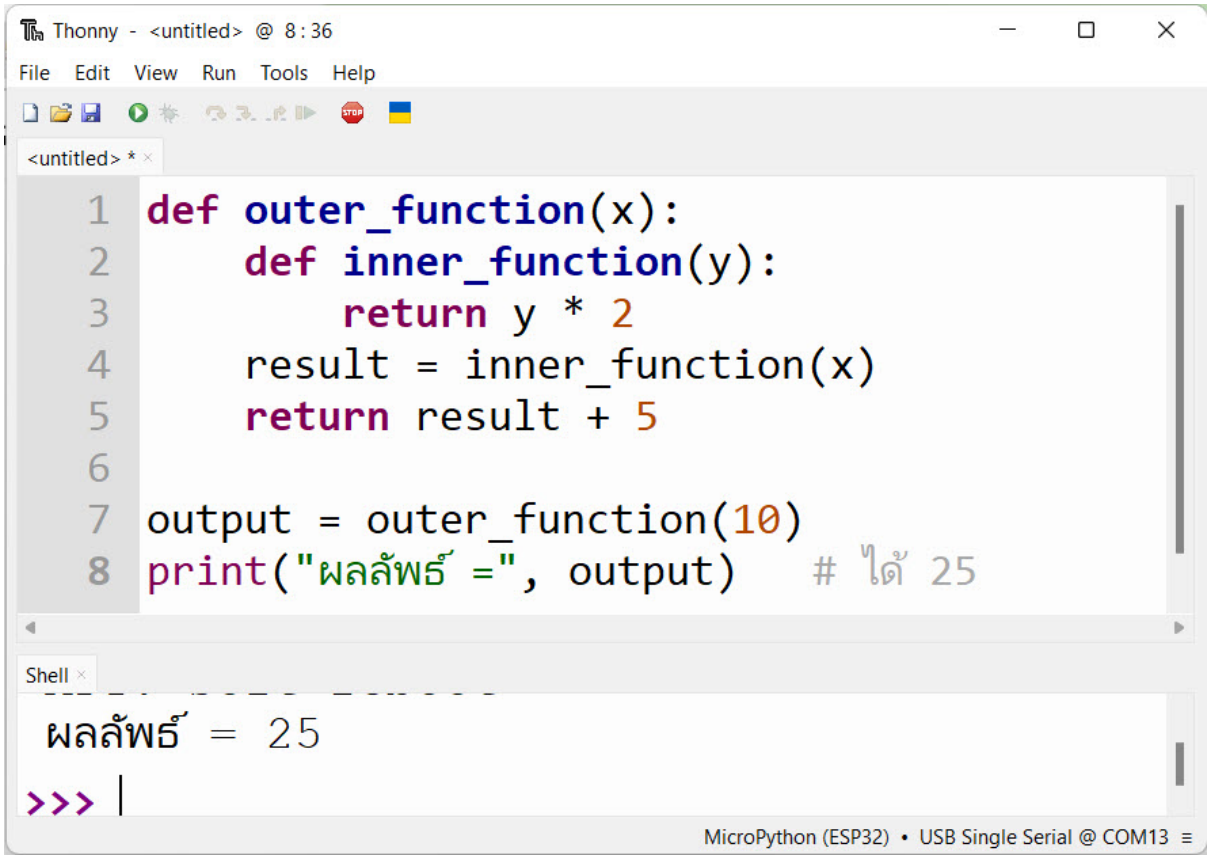
ความหมายและหลักการของฟังก์ชันซ้อน

การเขียนโปรแกรมด้วย MicroPython และ Python ทั่วไป นอกจากจะสามารถประกาศฟังก์ชันแบบปกติและฟังก์ชันนิพจน์ (lambda) แล้ว ยังสามารถ “ซ้อนฟังก์ชันไว้ภายในฟังก์ชันอีกชั้นหนึ่ง” ได้ด้วย เทคนิคนี้เรียกว่า **ฟังก์ชันซ้อน** (Nested Functions) ซึ่งหมายถึงการนิยามฟังก์ชันใหม่ไว้ “ภายในบล็อกของฟังก์ชันหลัก” ไม่ใช่ในระดับโปรแกรมหลักโดยตรง แนวคิดนี้ช่วยให้เราสามารถสร้างฟังก์ชันย่อยสำหรับงานเฉพาะทางภายในกระบวนการหลัก โดยฟังก์ชันที่อยู่ข้างในจะถูกจำกัดขอบเขต (scope) ให้ใช้ได้เฉพาะภายในฟังก์ชันหลักเท่านั้น ไม่สามารถเรียกใช้งานจากภายนอกได้โดยตรง

หากเปรียบเทียบกับชีวิตจริง ฟังก์ชันซ้อนเหมือนกับ “ทีมย่อยเฉพาะกิจ” ที่ตั้งขึ้นมาในระหว่างประชุมใหญ่ ทีมนี้มีอำนาจเฉพาะภารกิจเฉพาะหน้าและจะเลิกไปทันทีเมื่อภารกิจหลักจบ ต่างจากทีมงานหลักที่อยู่ถาวรและสามารถเรียกใช้งานจากภายนอกได้ตลอดเวลา การใช้ฟังก์ชันซ้อนจึงเป็นการควบคุมขอบเขตการใช้งาน ลดความซับซ้อนของโปรแกรมในระดับโครงสร้างหลัก

โครงสร้างของฟังก์ชันซ้อน

โครงสร้างของฟังก์ชันซ้อนใน MicroPython จะใช้การประกาศฟังก์ชันหนึ่ง (inner function) ไว้ในบล็อกของฟังก์ชันหลัก (outer function) ตัวอย่างที่เห็นได้ชัด เช่น



ลำดับการทำงาน:

1. มีการเรียก outer_function(10) จึงกำหนดให้ x = 10
2. ภายใน outer_function มีการประกาศฟังก์ชันย่อยชื่อ inner_function ซึ่งรอรับอาร์กิวเมนต์ในชื่อ y
3. ฟังก์ชัน inner_function นี้ถูกเรียกใช้งานด้วย inner_function(x) ซึ่งเท่ากับการส่งค่า 10 (ค่าของ x) เข้าไปแทนที่ y
4. inner_function ทำการคำนวณ return y * 2 คือ 10 * 2 = 20
5. ค่าที่ได้จาก inner_function ถูกเก็บในตัวแปร result
6. คำสั่งสุดท้ายใน outer_function คืนค่า result + 5 หรือ 20 + 5 = 25 ออกไป
7. สุดท้าย print("ผลลัพธ์ =", output) จึงแสดงผลลัพธ์ 25

การอธิบายด้วยสถานการณ์เปรียบเทียบ

ลองนึกภาพว่า outer_function เปรียบเสมือนเจ้าหน้าที่รับของจากผู้มาติดต่อ (เช่น ยามรับพัสดุ) ค่า x คือของที่ลูกค้านำมาฝาก เจ้าหน้าที่ (outer) จะส่งของนั้นต่อไปยังเจ้าหน้าที่อีกคน (inner) เพื่อดำเนินการพิเศษ เช่น ตรวจสอบและชั่งน้ำหนัก โดยระหว่างส่งงานนี้ จะนำค่า x (ของจริง) ส่งให้ inner ในชื่อ y (ไม่จำเป็นต้องเหมือนชื่อ x) เมื่อเจ้าหน้าที่ข้างใน (inner) คำนวณเสร็จ จะคืนผลลัพธ์ให้เจ้าหน้าที่ข้างนอก (outer) เพื่อนำไปบวกเพิ่มค่าอื่นหรือดำเนินการต่อ แล้วค่อยส่งผลลัพธ์สุดท้ายคืนให้ลูกค้า

สิ่งที่ต้องเข้าใจคือ แม้ชื่อของ parameter ภายนอก (x) กับภายใน (y) จะไม่เหมือนกัน แต่สามารถส่งค่าจากภายนอกเข้าสู่ภายในได้โดยตรง เมื่อเรียกใช้ inner_function(x) จะเท่ากับการส่งค่า x ไปให้ y ในฟังก์ชันย่อย ฟังก์ชันย่อยนี้จะคำนวณและคืนค่ากลับมาเพื่อให้ฟังก์ชันหลักนำไปใช้ประโยชน์ต่อได้ทันที

จุดสำคัญ

- ฟังก์ชันซ้อน (nested function) จะ “มองไม่เห็น” ตัวแปร parameter ของฟังก์ชันหลักโดยตรง (ถ้าไม่ส่งเข้าไปเอง)
- การส่งค่าเข้า inner_function จึงต้องระบุค่าให้ชัดเจน เช่น ส่ง x เข้า y หรือจะส่งค่าอื่นก็ได้
- โครงสร้างนี้ช่วยให้การทำงานเป็นระบบ ป้องกันการชนกันของตัวแปร และช่วยแบ่งหน้าที่การคำนวณให้ชัดเจน

การส่งค่าระหว่างฟังก์ชันซ้อนใน Python/MicroPython คือการถ่ายทอดข้อมูลจาก parameter ภายนอก (เช่น x) ไปยัง parameter ภายใน (เช่น y) ผ่านการเรียกใช้ฟังก์ชันย่อย เมื่อคำนวณเสร็จ จะคืนผลลัพธ์

กลับออกมาให้ฟังก์ชันหลักดำเนินการต่อ เปรียบเหมือนกระบวนการส่งงานจากเจ้าหน้าที่คนหนึ่งไปยังอีกคน ก่อนจะกลับสู่เจ้าของเรื่องเดิมอีกครั้ง ในตัวอย่างนี้ inner_function จะใช้ได้เฉพาะภายใน outer_function เท่านั้น เมื่อ outer_function ถูกเรียก ฟังก์ชันย่อยนี้จะทำงานและส่งผลลัพธ์กลับไปสู่กระบวนการหลักก่อนจะจบภารกิจ

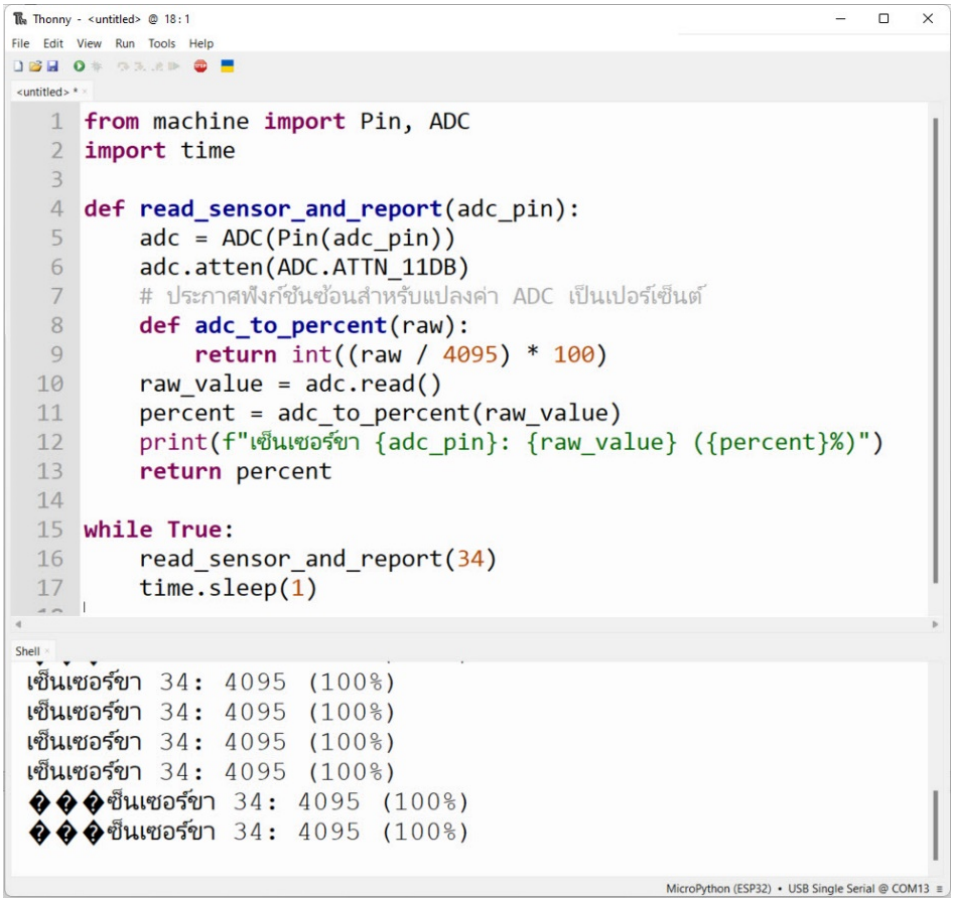
ประโยชน์และข้อดีของฟังก์ชันซ้อน

การใช้ฟังก์ชันซ้อนเหมาะกับสถานการณ์ที่ต้องการ “ลดขอบเขตของฟังก์ชันย่อย” ให้ใช้เฉพาะจุด เช่น งานที่ต้องมีขั้นตอนคำนวณซ้ำ ๆ ภายในกระบวนการหลักโดยไม่ต้องการให้มีผลกระทบต่อส่วนอื่นของโปรแกรม หรือกรณีที่ต้องการ “ซ่อนกระบวนการ” ไม่ให้ใช้งานโดยตรงนอกฟังก์ชันหลัก เช่น ฟังก์ชันสำหรับตรวจสอบค่าภายใน , ฟังก์ชันช่วยประมวลผลในแต่ละรอบของลูป หรือฟังก์ชันสำหรับเตรียมข้อมูลให้เหมาะสมก่อนส่งต่อไปยังงานหลักในกระบวนการเดียวกัน

ข้อดีของฟังก์ชันซ้อนคือช่วย “ลดปัญหาความซับซ้อนของโค้ดระดับสูง” (namespace pollution) ป้องกันการชนกันของชื่อฟังก์ชันหรือการเรียกใช้ผิดจุด ช่วยเพิ่มความเป็นระเบียบและ modularity ในโค้ด สามารถแบ่งความรับผิดชอบของแต่ละส่วนอย่างชัดเจน

ตัวอย่างฟังก์ชันซ้อนที่ทดลองได้จริงบน ESP32

ในงานควบคุมอุปกรณ์หรือวัดค่าบน ESP32 อาจพบว่ามีขั้นตอนที่ต้องคำนวณซ้ำ ๆ เฉพาะในกระบวนการหลัก เช่น ต้องการฟังก์ชันแปลงค่าแรงดัน ADC ให้เป็นเปอร์เซ็นต์เฉพาะเมื่ออ่านค่าเซ็นเซอร์ ที่ขา 34



ในตัวอย่างนี้ ฟังก์ชันซ้อน adc_to_percent จะถูกใช้เฉพาะใน read_sensor_and_report เท่านั้น ช่วยให้โค้ดหลักไม่ต้องประกาศฟังก์ชันแปลงค่าซ้ำซ้อนในทุกส่วนของโปรแกรม ช่วยให้โค้ดอ่านง่ายและควบคุมขอบเขตของฟังก์ชันได้ดีขึ้น

ข้อควรระวังและข้อจำกัด

แม้ว่าฟังก์ชันซ้อนจะช่วยลดความซับซ้อนและเพิ่มความปลอดภัยให้กับโครงสร้างโปรแกรม แต่ควรใช้อย่างเหมาะสม โดยเฉพาะกับงานที่มีลำดับการเรียกใช้งานซับซ้อน หรือมีการซ้อนกันหลายชั้นมากเกินไป อาจทำให้โค้ดอ่านยากและ debug ยากขึ้น ฟังก์ชันซ้อนเหมาะกับงานที่มีขอบเขตชัดเจน งานที่ต้องการ encapsulation หรือซ่อนกระบวนการช่วยเฉพาะจุด แต่หากงานใดต้องการนำฟังก์ชันย่อยไปใช้ซ้ำหลายจุดในโปรแกรมควรแยกประกาศเป็นฟังก์ชันหลักดีกว่า

สรุปแนวทางการเลือกใช้งาน

ฟังก์ชันซ้อน (Nested Functions) ใน MicroPython เป็นเครื่องมือสำคัญในการออกแบบโปรแกรมที่เน้นการควบคุมขอบเขตของตัวแปรและการทำงานภายใน ช่วยลดความซับซ้อนของโค้ดโดยการแยกงานย่อยเฉพาะกิจออกจากฟังก์ชันหลักอย่างเป็นระบบ ลดปัญหาการชนกันของชื่อ เพิ่มความปลอดภัยจากการเรียกใช้งานผิดจุด และช่วยให้โค้ดมีความกระชับ อ่านง่าย และมีความเป็นโมดูล (Modular) ที่ชัดเจน การใช้ฟังก์ชันซ้อนอย่างเหมาะสมจึงเป็นแนวทางที่ส่งเสริมทั้งความสามารถในการดูแลรักษา และการนำไปประยุกต์ใช้งานจริง ไม่ว่าจะเป็นในระบบควบคุมฮาร์ดแวร์ หรือระบบวิเคราะห์ข้อมูลในงาน IoT

3.6 โมดูล (Modules)

โมดูลใน MicroPython หมายถึงไฟล์โปรแกรมที่เก็บฟังก์ชัน คลาส หรือแม้แต่ตัวแปรซึ่งมีความเกี่ยวข้องกันไว้ในที่เดียว เช่น sensor_utils.py, display_driver.py หรือแม้แต่ time.py ที่มากับ MicroPython ก็ถือเป็นโมดูลเช่นกัน การแบ่งโปรแกรมออกเป็นโมดูลช่วยให้จัดระเบียบโค้ดได้ง่ายขึ้นมาก โดยเฉพาะเมื่อโครงการเริ่มซับซ้อน มีหลายหน้าที่ และต้องเปลี่ยนแปลงหรือดูแลในระยะยาว หลายคนที่เพิ่งเริ่มเขียนโปรแกรมมักเขียนทุกอย่างรวมกันในไฟล์เดียว เพราะงานเล็ก ๆ มักจะจัดการได้ แต่เมื่อโปรเจกต์เติบโต การรวมโค้ดทั้งหมดไว้ด้วยกันจะกลายเป็นปัญหาใหญ่ เช่น ถ้าโค้ดควบคุมเซ็นเซอร์กับโค้ดแสดงผลและโค้ดเชื่อมต่ออินเทอร์เน็ตปะปนกันหมด เวลาต้องแก้ไขหรือขยายฟังก์ชันใหม่ จะสับสนจนหาไม่เจอ หรือเผลอแก้ผิดจนโค้ดพังทั้งโปรแกรม

การใช้โมดูลคือวิธีแบ่งแยกความรับผิดชอบแต่ละส่วนอย่างเป็นระบบ เช่น ในโปรเจกต์ Smart Farm ที่มีทั้งส่วนวัดความชื้นดิน, ควบคุมปั๊มน้ำ, เชื่อมต่อ Wi-Fi, และแสดงผลบนจอ OLED โค้ดแต่ละกลุ่มควรแยกเป็นโมดูลของตัวเอง sensor_soil.py สำหรับวัดความชื้น, pump_control.py สำหรับควบคุมปั๊ม, wifi_utils.py สำหรับเชื่อมต่อ Wi-Fi, oled_display.py สำหรับจัดการจอแสดงผล โปรแกรมหลักจะ import โมดูลเหล่านี้ตาม

ต้องการ โค้ดแต่ละไฟล์จะไม่ก้าวก่ายกัน ถ้าต้องเปลี่ยนเซ็นเซอร์หรือเปลี่ยนจอ ก็แค่แก้ไขหรือสลับโมดูลที่เกี่ยวข้องเท่านั้น ไม่ต้องไล่แก้โค้ดทั้งโปรแกรม

MicroPython ยังคงใช้แนวคิดการแบ่งส่วนโปรแกรมเป็นโมดูลแบบเดียวกับภาษา Python เพื่อช่วยให้สามารถแยกส่วนการทำงานออกจากกันอย่างเป็นระบบ แม้ว่าในระบบฝังตัวเช่น ESP32 จะมีข้อจำกัดด้านหน่วยความจำและทรัพยากรฮาร์ดแวร์ แต่การใช้โมดูลช่วยให้โปรแกรมสามารถโหลดเฉพาะส่วนที่จำเป็นในแต่ละงาน ลดภาระการใช้งานหน่วยความจำลง และทำให้โค้ดแต่ละส่วนสามารถพัฒนา ทดสอบ และปรับปรุงได้อย่างยืดหยุ่นโดยไม่ต้องกระทบต่อระบบทั้งหมดโมดูลมาตรฐานอย่าง machine, time, os, network จะเก็บฟังก์ชันสำคัญไว้แยกต่างหาก ผู้ใช้แค่ import มาใช้งาน ไม่ต้องเขียนเองทั้งหมด ตัวอย่างเช่น ถ้าต้องควบคุมขา GPIO ก็ **import machine** แล้วเรียกใช้ **machine.Pin(...)** ได้เลย ถ้าต้องการเชื่อมต่อ Wi-Fi ก็ **import network** แล้วใช้ **network.WLAN(...)**

ประโยชน์ของการใช้โมดูลมีมากกว่าแค่จัดระเบียบหรือแก้ไขโค้ดง่าย การแบ่งโปรแกรมเป็นโมดูลยังช่วยให้ทำงานร่วมกันในทีมได้สะดวกขึ้น คนหนึ่งอาจพัฒนาโมดูลควบคุมอุปกรณ์ อีกคนพัฒนาโมดูลสำหรับเชื่อมต่อระบบคลาวด์ ทุกคนทำงานแยกไฟล์กันได้ จากนั้นนำโค้ดมารวมกันที่โปรแกรมหลักก็พร้อมใช้งานทันที หรือถ้าทำโปรเจกต์ใหม่ที่ต้องใช้ฟังก์ชันเหมือนเดิม เช่น การอ่านเซ็นเซอร์เดิม ก็แค่นำโมดูลที่เคยเขียนไว้มาวางในโฟลเดอร์แล้ว import ไปใช้ ไม่ต้องเขียนซ้ำ

ในทางปฏิบัติ ทุกครั้งที่ต้องการแยกโค้ดส่วนใดที่ใช้ซ้ำหรือมีหน้าที่เฉพาะออกจากโปรแกรมหลัก ให้สร้างไฟล์ใหม่ .py แล้วเขียนฟังก์ชัน คลาส หรือโค้ดส่วนที่เกี่ยวข้องไว้ในนั้น ตัวอย่างเช่น ถ้าต้องการจัดการกับเซ็นเซอร์วัดอุณหภูมิและความชื้น ให้สร้าง temp_humi_sensor.py แล้วเก็บฟังก์ชันอ่านค่าสัญญาณต่าง ๆ ไว้ในไฟล์นี้ ต่อไปจะเขียนโปรแกรมใหม่ก็นำเข้ามาใช้งานได้ตลอด โค้ดจะดูสะอาด ทดสอบง่าย และถ้ามีปัญหาก็แก้ไขแค่ไฟล์เดียวโดยไม่กระทบส่วนอื่น

โมดูลจึงเป็น “รากฐาน” ของโปรแกรมที่เขียนอย่างเป็นระบบใน MicroPython ไม่ว่าคุณจะพัฒนาโปรเจกต์เล็กหรือใหญ่ หากฝึกใช้โมดูลให้คล่องตั้งแต่แรก โปรแกรมจะขยายได้ง่าย ย้ายชิ้นส่วนโค้ดไปมาได้อย่างอิสระ และพร้อมสำหรับการนำไปใช้ซ้ำหรือทำงานร่วมกับคนอื่นเสมอ

ความรู้เสริม ก่อนจะทำความเข้าใจเรื่องโมดูล

ความแตกต่างระหว่าง Module และ Library ใน MicroPython เทียบกับภาษา C

ในการเขียนโปรแกรมไมโครคอนโทรลเลอร์ คนที่เคยเขียนภาษา C มาก่อนมักจะสับสนเมื่อเริ่มต้นกับ MicroPython เพราะคำว่า "ไลบรารี" ที่คุ้นเคยใน C กับ "โมดูล" ใน MicroPython นั้นดูเหมือนใกล้เคียงกัน

แต่จริง ๆ แล้ว เป็นคนละระบบการทำงาน

ในภาษา C (ระบบ Compile-Time):

- ไลบรารี (Library) หมายถึงชุดของโค้ดฟังก์ชันที่ถูกคอมไพล์และจัดเก็บไว้ล่วงหน้า
- ทุกครั้งที่เขียน #include <xxx.h> จะเป็นการนำเข้าหัวไฟล์ (Header File) ซึ่งบอกโปรแกรมว่าโค้ดมีฟังก์ชันอะไรบ้าง
- ตัวโค้ดจริงของไลบรารีจะถูกนำมารวมกับโปรแกรมในขั้นตอนการคอมไพล์ (ลิงก์เข้ามาตอน Compile-Time)
- สรุปง่าย ๆ:

ไลบรารีใน C = ทุกอย่างถูกรวมเสร็จก่อนโปรแกรมเริ่มทำงาน

ใน MicroPython (ระบบ Run-Time):

- โค้ดแต่ละไฟล์ .py จะถือเป็น **โมดูล (Module)** ทันที
- เมื่อเขียน import xxx จะเป็นการโหลดโมดูลเข้ามาทำงานขณะโปรแกรมกำลังทำงานจริง (Run-Time Import)
- หากมีหลายโมดูลถูกออกแบบให้ทำงานร่วมกันเป็นชุด เช่น urequests, umqtt ⟶ ทั้งชุดนี้เรามักเรียกรวมว่าเป็น **ไลบรารี (Library)**
- สรุปง่าย ๆ:

โมดูลใน MicroPython = 1 ไฟล์ .py

ไลบรารีใน MicroPython = ชุดของหลายโมดูลรวมกัน

สรุปเปรียบเทียบให้เห็นภาพชัด ๆ

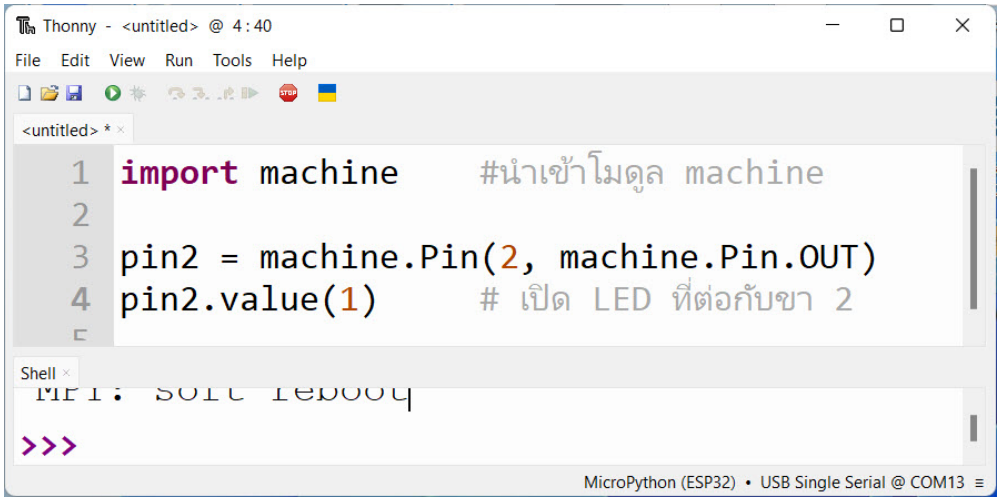
ประเด็น	ภาษา C	MicroPython
หน่วยย่อย	ไลบรารี (Library) + Header File	โมดูล (Module)
ไฟล์จริง	.h, .c, .a, .so	.py
เวลาโหลด	Compile-Time	Run-Time
เวลาเรียกใช้	#include	import
ไลบรารีคืออะไร	ชุดของฟังก์ชันที่ลิงก์ไว้ก่อนรัน	ชุดของหลายโมดูลรวมกัน

3.6.1 การนำเข้าโมดูล (Importing Modules)

ใน MicroPython และ Python การนำเข้าโมดูล (import module) คือหัวใจสำคัญที่ทำให้โปรแกรมมีความยืดหยุ่นและขยายความสามารถได้อย่างไร้ขีดจำกัด แต่เดิม การเขียนโปรแกรมที่ไม่มีแนวคิดเรื่องโมดูล โค้ดทุกอย่างจะถูกรวมอยู่ในไฟล์เดียวกันทั้งหมด เมื่อโปรเจกต์เติบโตหรือมีฟังก์ชันเพิ่มขึ้นเรื่อย ๆ โค้ดจะเริ่มรกซ้ำซ้อน แก้ไขยาก และเสี่ยงต่อข้อผิดพลาด การออกแบบภาษา Python และ MicroPython ให้รองรับ “โมดูล” จึงตอบโจทย์การพัฒนาซอฟต์แวร์ยุคใหม่ที่เน้นการแยกหน้าที่ (modularity) ช่วยให้แต่ละส่วนของโค้ดเป็นอิสระต่อกัน นำกลับมาใช้ซ้ำในโปรเจกต์อื่นได้ และแบ่งงานร่วมกันในทีมได้สะดวก

สำหรับงานบนบอร์ด ESP32 หรือฮาร์ดแวร์ฝังตัว การนำเข้าโมดูลสำคัญยิ่งกว่าการเขียนโปรแกรมทั่วไป เพราะงานแต่ละโปรเจกต์มักต้องควบคุมอุปกรณ์หรือใช้ฟังก์ชันเฉพาะทางที่ไม่อยู่ในตัวภาษา เช่น การควบคุมขา GPIO, อ่านค่าอนาล็อก, สื่อสารผ่าน I2C, SPI, UART, หรือเชื่อมต่อ Wi-Fi ถ้าไม่สามารถ import โมดูล เช่น machine, time, หรือ network ได้ โปรแกรมจะกลายเป็นแค่โค้ด print ข้อมูลธรรมดา จะไม่สามารถควบคุมฮาร์ดแวร์ได้จริง

โมดูลมาตรฐานของ MicroPython จะถูกรวมอยู่ในเฟิร์มแวร์บอร์ด ESP32 อยู่แล้ว เช่น machine สำหรับจัดการฮาร์ดแวร์, time สำหรับหน่วงเวลาหรือจับเวลา, network สำหรับเชื่อมต่อเครือข่าย, os สำหรับจัดการไฟล์บนระบบ และ sys สำหรับดูสถานะระบบ ซึ่งทั้งหมดนี้ต้อง import ก่อนใช้งานเสมอ ถ้าไม่ import แล้วเรียกใช้ฟังก์ชันในโมดูลนั้น โปรแกรมจะเกิดข้อผิดพลาด (NameError) ทันที ตัวอย่างเช่น

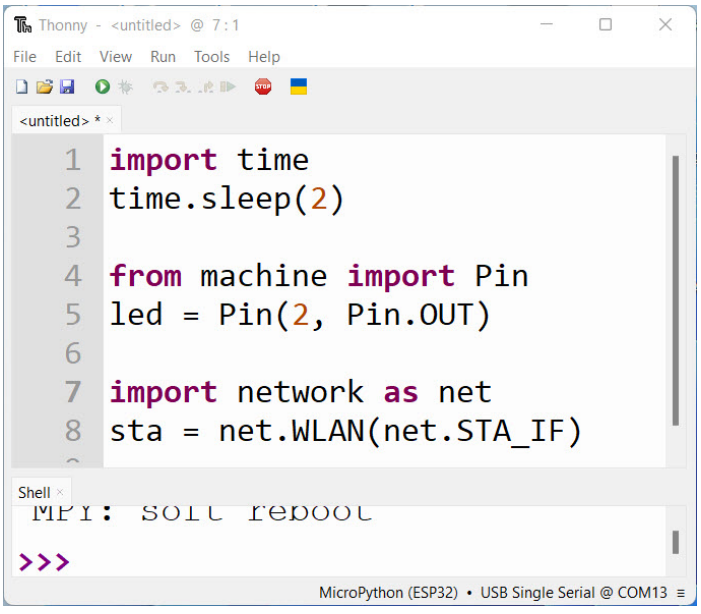


จากตัวอย่างโค้ดนี้ หากไม่มีการ **import machine** (บรรทัดที่ 1) แล้วไปสั่ง machine.Pin(2, ...) โปรแกรมจะไม่ทำงาน และแจ้งว่าไม่รู้จักชื่อ machine เพราะ machine ไม่ใช่คำสั่งพื้นฐานของภาษา python แต่เป็น โมดูลที่มากับเฟิร์มแวร์ MicroPython

นอกจากโมดูลมาตรฐาน MicroPython ยังเปิดโอกาสให้ผู้เขียนสร้างโมดูลของตนเอง แยกโค้ดที่ใช้งานบ่อย หรือโค้ดซับซ้อนออกจากโปรแกรมหลัก แล้วนำเข้าใช้งานในหลายโปรเจกต์ได้ทันที การ import โมดูลจึงไม่ใช่แค่การเพิ่มความสามารถ แต่ยังทำให้การดูแลและเขียนโค้ดเพิ่มได้ง่ายขึ้น ตัวอย่างเช่น ถ้าสร้าง sensor_utils.py เพื่อรวมฟังก์ชันอ่านค่าจากเซ็นเซอร์ทุกชนิดไว้ สามารถ import sensor_utils แล้วเรียกใช้ฟังก์ชันเหล่านั้นได้ในทุกโปรเจกต์ที่ต้องการ

การ import ใน MicroPython มีหลายรูปแบบ

- import <module> : นำเข้าโมดูลทั้งไฟล์ เรียกใช้ด้วยชื่อโมดูลนำหน้า
- from <module> import <function/class> : นำเข้าเฉพาะฟังก์ชันหรือคลาสที่ต้องการ
- import <module> as <alias> : นำเข้าโมดูลพร้อมเปลี่ยนชื่อให้สั้นลง เหมาะกับโปรแกรมที่ต้องใช้โมดูลบ่อยหรือชื่อโมดูลยาวเกินไป เช่น network แต่เราอยากเขียนสั้น ๆ แค่ net ดูตัวอย่างในภาพ

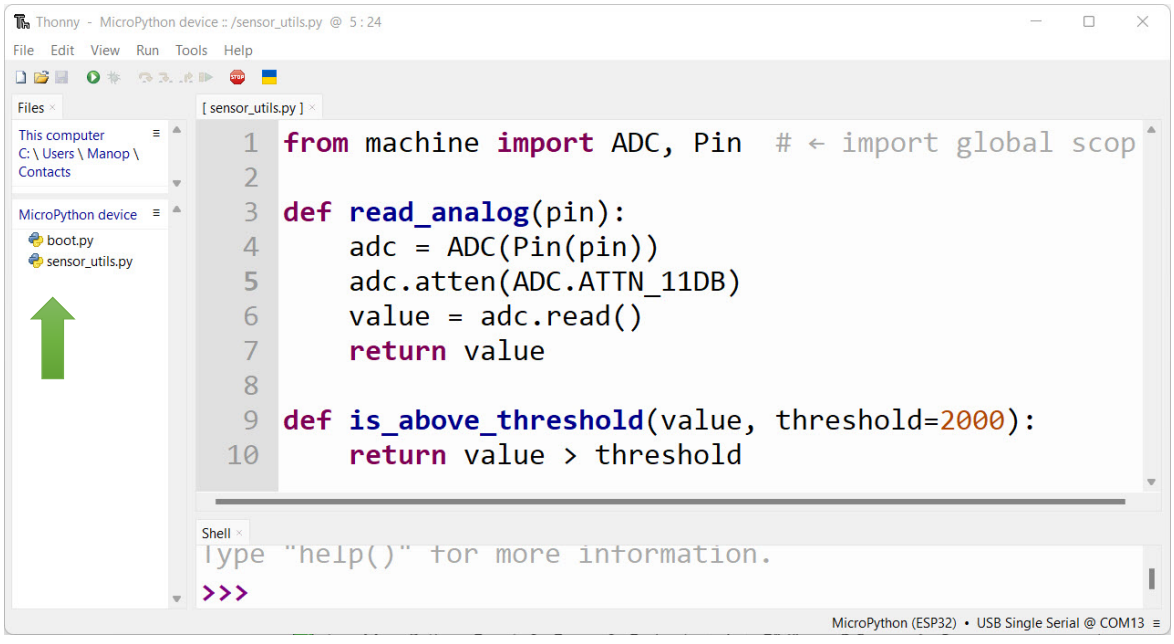


ข้อควรระวัง คือ MicroPython จะค้นหาโมดูลที่นำเข้าในโฟลเดอร์เดียวกับโปรแกรมหลักก่อน แล้วจึงค้นหาในโมดูลมาตรฐาน ถ้าไฟล์โมดูลไม่มีอยู่ หรือใช้ชื่อซ้ำกับโมดูลสำเร็จรูป โปรแกรมจะเกิดข้อผิดพลาดทันที สำหรับคนที่เพิ่งเริ่มต้น ให้ตรวจสอบเสมอว่าไฟล์โมดูลที่ import นั้นอยู่ในไดเรกทอรีเดียวกับไฟล์หลัก หรืออยู่ใน path ที่ MicroPython มองเห็น หรือไม่ ถ้าไม่ import โมดูลที่ต้องใช้ โค้ดจะไม่สามารถสั่งงานกับฮาร์ดแวร์ หรือใช้ฟังก์ชันเฉพาะทางใด ๆ ได้เลย เช่น สั่ง time.sleep(1) โดยไม่ import time โปรแกรมจะ error ว่าไม่รู้จักชื่อ t i m e งานฝังตัวทุกประเภทจึงเริ่มต้นจากการ **import** โมดูลเสมอ

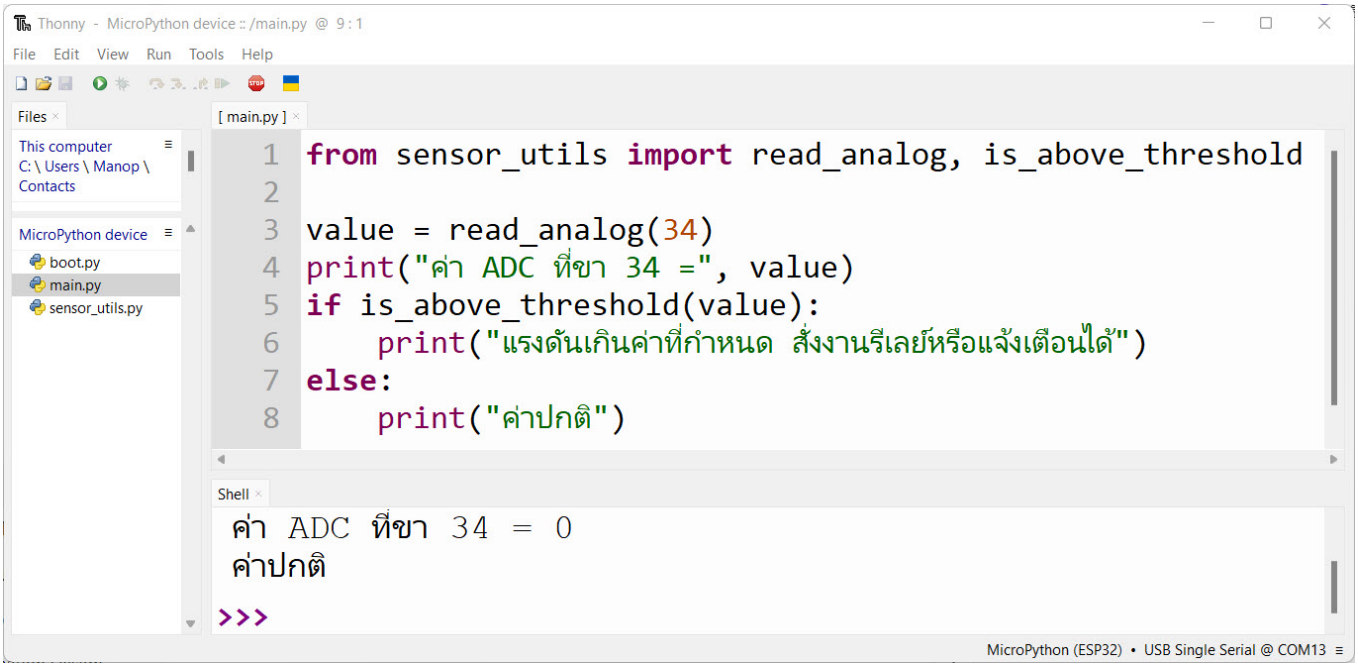
การเข้าใจการนำเข้าโมดูลอย่างถูกต้อง ถือเป็นพื้นฐานสำคัญที่ช่วยให้เขียนโปรแกรมบน MicroPython ได้อย่างเป็นระบบ และสามารถขยายขีดความสามารถของภาษาเพื่อควบคุมทั้งฟังก์ชันภายในและฮาร์ดแวร์ภายนอกได้อย่างเต็มประสิทธิภาพ ในทุกโปรเจกต์ระดับมืออาชีพ การจัดวางคำสั่ง import อย่างเป็นระเบียบตั้งแต่ต้นของโครงสร้างโปรเจกต์ จะช่วยให้การดูแลรักษาโค้ดเป็นไปได้อย่างง่ายดาย และสามารถแก้ไขเพิ่มเติมหรือขยายความสามารถในอนาคตได้อย่างมีประสิทธิภาพ

3.6.2 การสร้างโมดูลของตนเอง (Creating Your Own Modules)

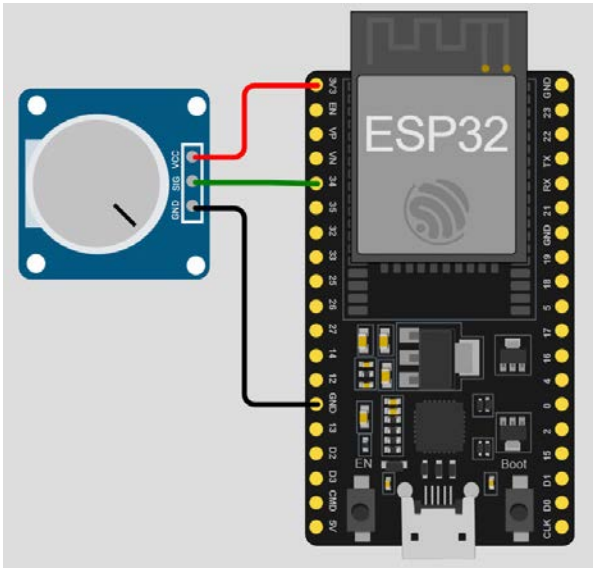
การสร้างโมดูลของตนเองใน MicroPython เป็นกระบวนการที่ช่วยให้โค้ดเป็นระบบมากขึ้น เหมาะกับการเขียนโปรแกรมที่ต้องการนำฟังก์ชันหรือคลาสกลับมาใช้ซ้ำในหลายโปรเจกต์ หรือโปรแกรมที่ขยายขนาดขึ้นจนต้องแยกความรับผิดชอบแต่ละส่วนออกจากกัน เริ่มต้นจากการวิเคราะห์ว่าโค้ดส่วนไหนในโปรเจกต์ควรแยกออกมาเป็นโมดูล มักจะเลือกฟังก์ชันที่ใช้งานซ้ำบ่อยหรือกลุ่มฟังก์ชันที่เกี่ยวข้องกัน จากนั้นสร้างไฟล์ใหม่ด้วยนามสกุล .py เช่น sensor_utils.py, led_driver.py หรือ relay_control.py ภายในไฟล์นี้จะรวบรวมฟังก์ชันหรือคลาสที่ต้องการให้อยู่ด้วยกัน ตัวอย่างเช่น ต้องการสร้างโมดูลสำหรับอ่านค่าอนาล็อกจากขา ADC ของ ESP32 และตรวจสอบค่ากับ threshold สามารถเขียนใน sensor_utils.py ดังนี้

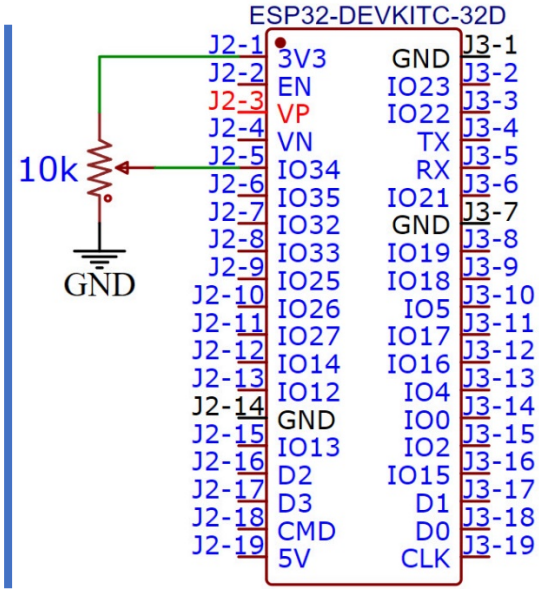


เมื่อบันทึกไฟล์ sensor_utils.py แล้ว ให้วางไฟล์นี้ไว้ในโฟลเดอร์เดียวกับโปรแกรมหลัก จากนั้นในโปรแกรมหลัก เช่น main.py สามารถนำเข้าและเรียกใช้งานฟังก์ชันในโมดูลได้ทันที



ขั้นตอนปฏิบัติสำหรับทดลองใช้งานบนบอร์ดจริง ให้เชื่อมต่อบอร์ด ESP32 กับคอมพิวเตอร์ผ่านสาย USB เปิดโปรแกรม Thonny หรือเครื่องมือที่ใช้ แล้วสร้างไฟล์ sensor_utils.py พร้อมคัดลอกโค้ดตัวอย่างลงไป จากนั้นสร้างไฟล์ main.py วางโค้ดโปรแกรมหลักลงไปอีกไฟล์ ต่อ VR หรือเซ็นเซอร์อนาล็อกเข้าขา 34 (ADC1_CH6) ของ ESP32 หากไม่มีเซ็นเซอร์ สามารถใช้แรงดัน 3.3V ต่อเข้าขา 34 เพื่อทดสอบ เมื่อรันโปรแกรมจะเห็นค่าที่อ่านได้แสดงบน Serial Monitor หากค่ามากกว่า threshold ที่กำหนด โปรแกรมจะแจ้งเตือนทันที





ข้อควรระวังในการสร้างโมดูลคือ ชื่อไฟล์ไม่ควรซ้ำกับโมดูลมาตรฐานของ MicroPython เช่น machine.py, time.py, network.py เพื่อหลีกเลี่ยงปัญหาโปรแกรมนำเข้าโมดูลผิดหรือเกิด error นอกจากนี้ควรตรวจสอบเสมอว่าไฟล์ .py ของโมดูลอยู่ในโฟลเดอร์เดียวกับโปรแกรมหลัก ในกรณีที่ต้อง import module

ภายนอกฟังก์ชันที่ใช้บ่อย ให้นำคำสั่ง import ไปไว้ด้านบนสุดของไฟล์โมดูลเพื่อประหยัดหน่วยความจำและให้โค้ดอ่านง่ายขึ้น

โมดูลที่สร้างเองนี้จะสามารถนำกลับมาใช้ในโปรเจกต์อื่น ๆ ได้ง่าย เพียงคัดลอกไฟล์ .py ไปวางในโฟลเดอร์โปรเจกต์ใหม่แล้ว import ใช้งานทันที นอกจากนี้ยังสามารถใส่ docstring หรือคำอธิบายไว้หน้าฟังก์ชันเพื่อให้เข้าใจวิธีใช้งานเมื่อย้อนกลับมาอ่านในภายหลัง หรือเมื่อแบ่งปันให้ผู้อื่นในทีมใช้งานร่วมกัน ในโปรเจกต์ที่มีฟังก์ชันหลากหลายควรแยกกลุ่มฟังก์ชันเป็นโมดูลต่าง ๆ ตามหน้าที่ เช่น led_utils.py สำหรับควบคุม LED, relay_control.py สำหรับควบคุมรีเลย์, wifi_utils.py สำหรับเชื่อมต่อ Wi-Fi วิธีนี้ช่วยให้โค้ดดูเป็นระบบและแก้ไขหรือขยายงานในอนาคตได้ง่าย เมื่อปรับปรุงฟังก์ชันหรือเพิ่มฟังก์ชันใหม่ในโมดูล เพียงแก้ไขไฟล์โมดูลนั้นและบันทึก โปรแกรมหลักที่ import โมดูลนี้จะสามารถใช้งานฟังก์ชันใหม่ได้ทันทีโดยไม่ต้องแก้ไขโครงสร้างโปรแกรมหลัก นอกจากนี้ยังช่วยลดโอกาสการเกิดข้อผิดพลาดจากการคัดลอกโค้ดซ้ำหลายจุด การจัดการโค้ดแบบนี้จึงเหมาะสมที่สุดสำหรับงานพัฒนา MicroPython ที่มีการเติบโตหรือเปลี่ยนแปลงรายละเอียดบ่อย ๆ

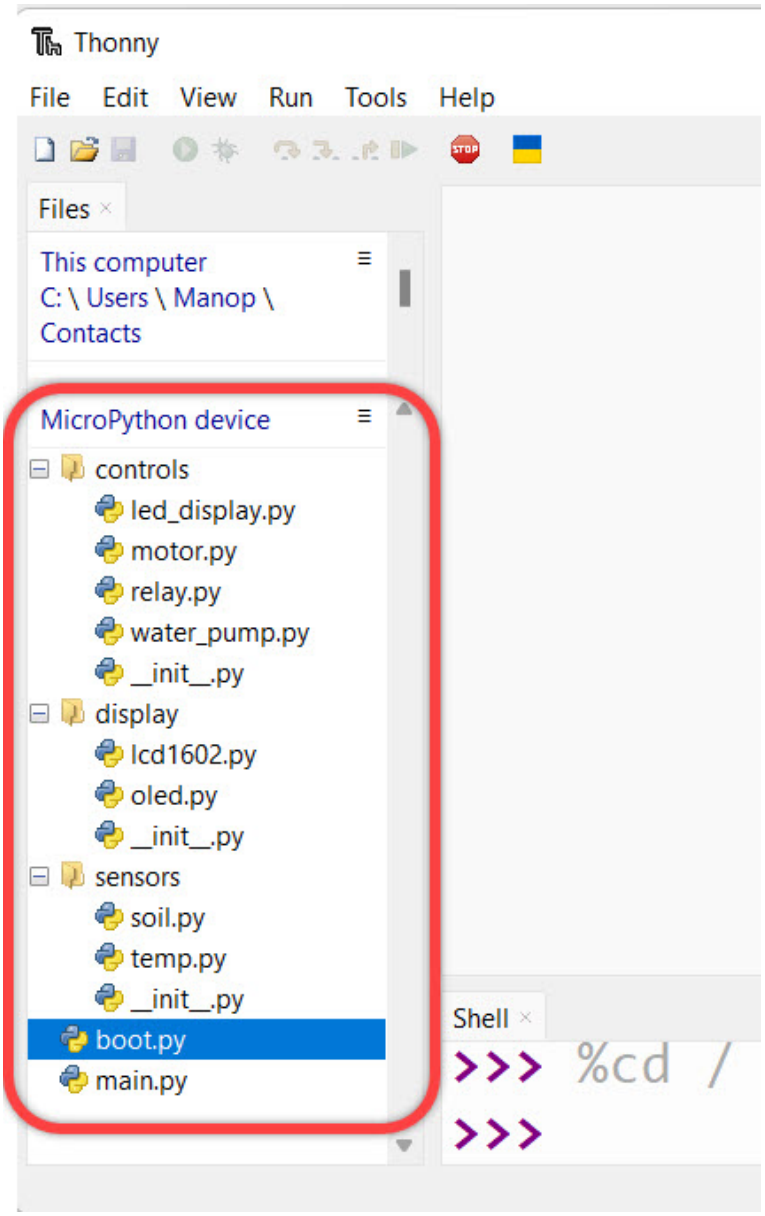
สำหรับมือใหม่ที่เพิ่งเริ่มสร้างโมดูลของตนเอง อาจเจอปัญหาเช่นสะกดชื่อไฟล์ผิด วางไฟล์ผิดโฟลเดอร์ หรือใช้ชื่อซ้ำกับโมดูลมาตรฐานจน import ไม่ได้ หรือโปรแกรมฟ้องว่า ModuleNotFoundError ควรตรวจสอบรายละเอียดเหล่านี้ทุกครั้งก่อนทดลองรันจริง การฝึกสร้างโมดูลตั้งแต่โปรเจกต์แรกจะช่วยให้เขียนโปรแกรมอย่างเป็นระบบและพร้อมขยายงานหรือแก้ไขในระยะยาว

3.6.3 โครงสร้างโฟลเดอร์และโมดูลย่อย

ในโปรเจกต์ MicroPython เมื่อระบบเริ่มเติบโตมากขึ้น โค้ดมีหลายส่วนที่ทำหน้าที่แตกต่างกัน เช่น ส่วนที่ควบคุมเซ็นเซอร์ ส่วนควบคุมมอเตอร์ ส่วนติดต่อจอแสดงผล และส่วนติดต่อระบบเครือข่าย การวางโค้ดทั้งหมดไว้ในไฟล์เดียวหรือโฟลเดอร์เดียวจะเริ่มสร้างปัญหาหลายอย่าง ไม่ว่าจะเป็นความยุ่งเหยิงของไฟล์ การค้นหาฟังก์ชันยากขึ้น หรือความเสี่ยงต่อการเขียนโค้ดซ้ำซ้อนผิดพลาดเมื่อทีมงานหลายคนทำงานร่วมกัน การออกแบบให้แยกโค้ดออกเป็นกลุ่มย่อยตามหน้าที่ และเก็บโค้ดแต่ละกลุ่มไว้ในโฟลเดอร์แยกกัน จึงเป็นแนวทางมาตรฐานที่ใช้ในงานพัฒนาซอฟต์แวร์ทุกระดับ ตั้งแต่โปรเจกต์ทดลองไปจนถึงระบบขนาดใหญ่

แต่การแยกไฟล์ไว้ในโฟลเดอร์เฉย ๆ ยังไม่เพียงพอให้ระบบ MicroPython รู้จักวิธีเข้าไปนำเข้า (import) โค้ดภายในโฟลเดอร์เหล่านั้นมาใช้งานได้โดยตรง เพราะโฟลเดอร์ธรรมดาสำหรับระบบไฟล์ไม่ใช่สิ่งที่ภาษาคอมพิวเตอร์จะมองเห็นเป็น package โดยอัตโนมัติ ต้องมีการบอกตัวภาษา ว่าโฟลเดอร์นี้ตั้งใจจะใช้เป็น package หรือกลุ่มของโมดูลที่ต้องการ import เข้ามาใช้งาน

Python และ MicroPython ใช้ไฟล์พิเศษชื่อ __init__.py เป็นตัวประกาศว่าโฟลเดอร์ใดบ้างควรถือว่าเป็น package ในระบบ import ไฟล์ __init__.py นี้ไม่จำเป็นต้องมีโค้ดใด ๆ ภายในก็ได้ เพราะหน้าที่สำคัญที่สุดของมันในบริบทของ MicroPython คือการเป็นตัวระบุว่าโฟลเดอร์นี้ถูกยกระดับเป็น package ให้ interpreter สามารถเดินเข้าไปหาไฟล์ย่อยภายในได้อย่างถูกต้องตามระบบ import การสร้างโฟลเดอร์เพื่อจัดกลุ่มโค้ดควบคุมอุปกรณ์แต่ละประเภทอาจแบ่งโครงสร้างได้เช่น



แต่ละโฟลเดอร์ย่อย เช่น sensors, displays, controls จะเก็บโมดูลที่เกี่ยวข้องในหมวดนั้น ๆ ไว้ด้วยกัน ไฟล์ **__init__.py** อาจเป็นไฟล์ว่างไท่ต้องเขียนโค้ด (หรือใส่โค้ดตั้งค่าพื้นฐานก็ได้) เพราะไฟล์จะเป็นตัวบอกให้ Python หรือ MicroPython รู้ว่าโฟลเดอร์นี้เป็น package ที่นำเข้าได้ การ import โมดูลจากโฟลเดอร์ย่อยให้ใช้รูปแบบไฟล์ **__init__.py** จะต้องมีอยู่ในทุกโฟลเดอร์ย่อยเช่น sensors, controls และ displays เพื่อประกาศตัวโฟลเดอร์เหล่านี้เป็น package อย่างถูกต้อง

ชื่อไฟล์ **__init__.py** ต้องสะกดอย่างแม่นยำ ไม่มีการยืดหยุ่นในเรื่องรูปแบบชื่อใด ๆ ทั้งสิ้น ชื่อนี้ประกอบด้วย **ขีดล่างสองตัว (underscore)** วางอยู่ด้านหน้าคำว่า **init** และ**ขีดล่างสองตัววางอยู่ด้านหลังคำว่า init** ตามด้วยนามสกุล .py ทั้งหมดนี้ต้องเป็นตัวพิมพ์เล็กเท่านั้น ห้ามใช้ตัวพิมพ์ใหญ่แม้แต่ตัวเดียว ชื่อไฟล์ถูกต้องจะมีหน้าตาเป็นแบบนี้

__init__.py

ที่เห็นเป็นขีดยาว ๆ คือการพิมพ์ขีดล่าง (underscore) เข้าไป 2 ตัวต่อกัน __

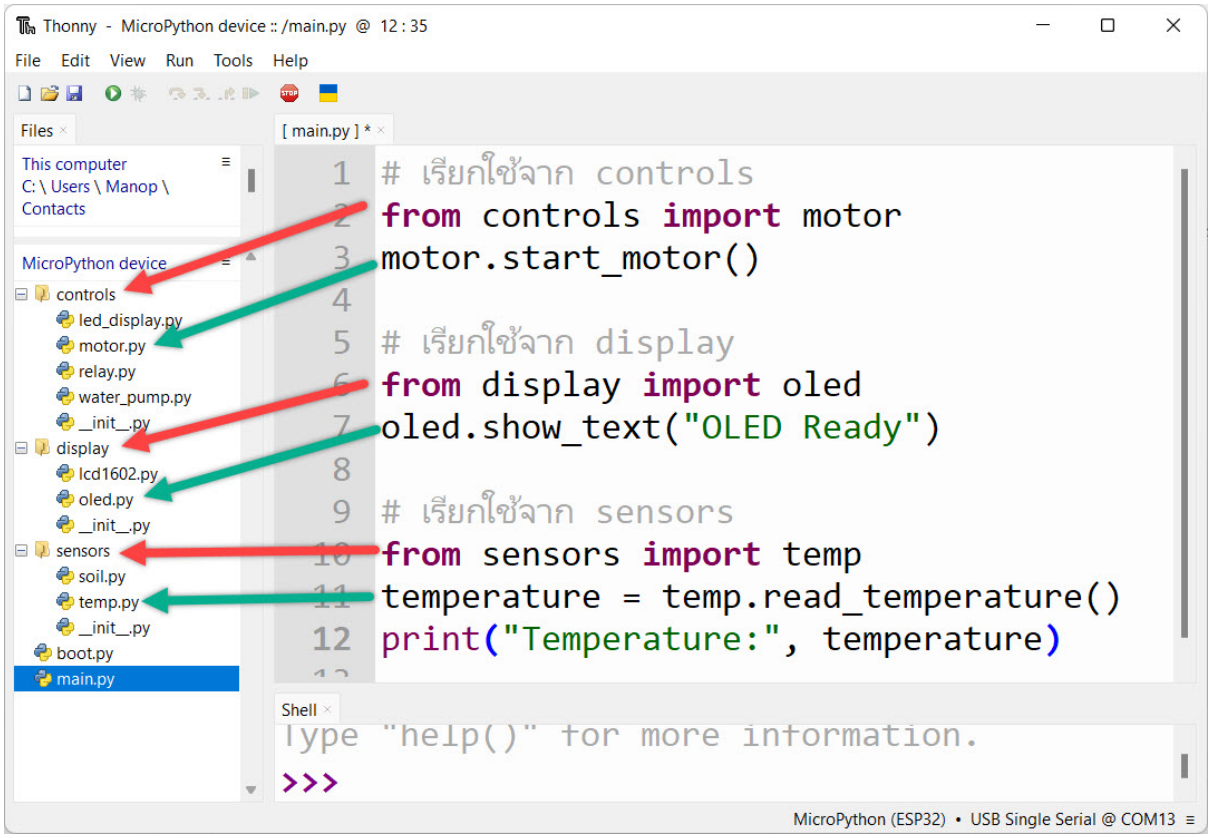
ขีดล่างทั้งสี่ตัวในชื่อนี้เป็น underscore มาตรฐาน หากสะกดผิดเพียงตัวเดียว ไม่ว่าจะพิมพ์ผิดหรือตกหล่น ระบบจะไม่สามารถมองเห็นไฟล์นี้เป็น __init__.py ได้ และโฟลเดอร์นั้นจะไม่ถูกนับเป็น package ส่งผลให้โมดูลทั้งหมดภายในโฟลเดอร์ดังกล่าวไม่สามารถนำเข้า (import) มาใช้งานได้ หากมีการพยายาม import จะเกิดข้อผิดพลาดขณะรันโปรแกรม (runtime error) ทันที ตัวอย่างชื่อผิดที่ทำให้การ import ล้มเหลวแสดงในภาพนี้

init.py	_init_.py	__init_.py	_init__.py	__init__.PY	__Init__.py

ระบบ interpreter จะตรวจสอบชื่อไฟล์อย่างเข้มงวดโดยไม่มีการเดาหรือยืดหยุ่นใด ๆ หากไม่มีไฟล์ __init__.py ที่สะกดถูกต้อง ระบบจะถือว่าโฟลเดอร์นั้นไม่ใช่ package และไม่สามารถ import module ใดภายในโฟลเดอร์นั้นได้

เมื่อมีไฟล์ **__init__.py** อยู่ในโฟลเดอร์อย่างถูกต้องแล้ว การ import module ย่อยภายใน package สามารถเขียนคำสั่งดึงไฟล์ออกจากโฟลเดอร์เหล่านั้นมาใช้งานได้ตามโครงสร้างชื่อโฟลเดอร์ เช่น หากในโฟลเดอร์ sensors มีไฟล์ temp.py อยู่ การ import เข้ามาใช้งานจะเขียนเป็น **from sensors import temp**

เมื่อคำสั่งนี้ถูกรัน ระบบจะเข้าไปยังโฟลเดอร์ sensors ซึ่งมี **__init__.py** ประกาศตัว package ไว้แล้ว จากนั้นจึงนำเข้าไฟล์ temp.py มาเป็น module ตามที่ระบุไว้ เมื่อนำเข้าสำเร็จแล้ว สามารถเรียกใช้งานฟังก์ชันหรือคลาสที่เขียนไว้ในไฟล์ temp.py ได้ เช่น **temperature = temp.read_temperature()**
โครงสร้างคำสั่ง import ในรูปแบบนี้สามารถขยายใช้กับโฟลเดอร์อื่น ๆ ได้เช่นเดียวกัน เช่น controls หรือ displays โดยเปลี่ยนชื่อ package และชื่อไฟล์ module ตามตำแหน่งจริงในระบบไฟล์



ระบบ import ของ MicroPython จะเดินหาไฟล์แบบตรงไปตรงมาตามโครงสร้างไฟล์นี้ทันที โดยจะเริ่มจาก root directory แล้วเดินตามชื่อโฟลเดอร์ที่เขียนใน import statement แต่ละลำดับอย่างเป็นขั้นตอน เมื่อเจอโฟลเดอร์ที่มี **__init__.py** จะอนุญาตให้เดินต่อไปยัง module ย่อยภายในโฟลเดอร์นั้นทันที

ถ้าใช้โครงสร้างนี้บน MicroPython จริง การอัปโหลดไฟล์ไปยังบอร์ดควรสร้างโฟลเดอร์ย่อยให้เหมือนกับที่เตรียมไว้ในคอมพิวเตอร์ หรือใช้เครื่องมืออย่าง Thonny, rshell, ampy เพื่อโอนไฟล์และโฟลเดอร์ทั้งโครงสร้างขึ้นไปในครั้งเดียว

ข้อควรระวังสำหรับมือใหม่ คือชื่อโฟลเดอร์และชื่อไฟล์ควรเป็นภาษาอังกฤษตัวเล็กทั้งหมด ไม่ควรใช้เว้นวรรคหรืออักขระพิเศษเพื่อป้องกันปัญหาเวลา import หรือโอนไฟล์ นอกจากนี้ไฟล์ __init__.py ต้องมีอยู่ในทุกโฟลเดอร์ที่ต้องการใช้เป็น package แม้จะเป็นไฟล์เปล่าก็ตาม เพื่อให้ MicroPython รู้จักและสามารถ import ได้ถูกต้อง

ในโปรเจกต์ขนาดกลางถึงใหญ่ อาจมีโฟลเดอร์อื่นประกอบเพิ่มเติม เช่น configs สำหรับไฟล์ตั้งค่า, utils สำหรับกลุ่มฟังก์ชันช่วยเหลือ, libs สำหรับไลบรารีเสริมจากภายนอก ทุกส่วนควรจัดเรียงแยกกันให้ชัดเจนตามหน้าที่ การจัดโครงสร้างแบบนี้จะช่วยให้ทำงานร่วมกับผู้อื่นได้สะดวก เวลาต้องแก้ไขหรือเพิ่มฟีเจอร์ใหม่ก็เข้าไปที่โฟลเดอร์และไฟล์ที่เกี่ยวข้องได้ทันที ไม่ต้องเปิดทีละไฟล์ทีละบรรทัด

การเข้าใจหลักการจัดโฟลเดอร์และโมดูลย่อยเป็นรากฐานสำคัญที่จะทำให้สามารถต่อยอดโปรเจกต์ได้แบบมืออาชีพ ไม่ว่าจะเป็นงาน IoT, Smart Farm, Automation หรือโค้ดซอฟต์แวร์สำหรับขายหรือเผยแพร่ต่อในอนาคต

3.6.4 โมดูลสำคัญใน MicroPython สำหรับ ESP32

เมื่อเริ่มพัฒนาโปรแกรมบนบอร์ด ESP32 ด้วย MicroPython หนึ่งในสิ่งสำคัญที่สุดที่ต้องเข้าใจและใช้ให้เป็น คือ “โมดูลสำคัญ” ที่มากับระบบ ซึ่งเปรียบเสมือนเครื่องมือหลักของโปรแกรมเมอร์สายฝังตัว โมดูลเหล่านี้ไม่ได้เป็นแค่ฟังก์ชันสำเร็จรูปหรือของแถมในภาษา แต่คือสะพานเชื่อมระหว่างซอฟต์แวร์กับฮาร์ดแวร์จริงของบอร์ด ESP32 ทุกการควบคุมอุปกรณ์ไฟฟ้า อ่านค่าจากเซ็นเซอร์ รับส่งข้อมูล เชื่อมต่อ Wi-Fi หรือแม้แต่การจัดการไฟล์ในระบบ ต่างก็ต้องพึ่งพาโมดูลเหล่านี้ทั้งสิ้น หากไม่เข้าใจภาพรวมและหน้าที่แต่ละโมดูล การเขียนโปรแกรมให้ ESP32 จะติดขัดตั้งแต่ขั้นแรก เพราะแค่สั่งงานพื้นฐานก็อาจ error หรือไม่ได้ผลลัพธ์ที่ต้องการ

machine

machine คือโมดูลหลักที่ทำให้ MicroPython แตกต่างจาก Python ทั่วไป เพราะเป็นกล่องเครื่องมือที่เปิดทางให้โปรแกรมสามารถ “สั่งงานกับฮาร์ดแวร์” ของ ESP32 ได้โดยตรง ทุกอย่างที่เกี่ยวกับการควบคุมขาอินพุต/เอาต์พุต การอ่านเซ็นเซอร์ การสั่งรีเลย์ มอเตอร์ หรือสร้างสัญญาณพัลส์ (PWM) ล้วนต้องพึ่งโมดูล machine ทั้งสิ้น โมดูลนี้ทำให้ ESP32 ไม่ใช่แค่คอมพิวเตอร์ขนาดจิ๋วที่คำนวณเลขหรือประมวลผลข้อมูลได้อย่างเดียว แต่เป็น “สมอง” ที่ควบคุมอุปกรณ์ในโลกจริงได้ ตั้งแต่เปิดปิดไฟบ้าน วัดความชื้นในดิน ตรวจวัดสภาพอากาศ รับสัญญาณจากปุ่มกด หรือทำงานร่วมกับระบบเซ็นเซอร์ขั้นสูงผ่าน I2C/SPI/UART

ในโปรแกรม MicroPython ทุกครั้งที่ต้องการสั่งงานกับขา GPIO เช่น เปิดไฟ LED, รับค่าจากสวิตช์, หรือวัดแรงดันไฟฟ้าจากขาอนาล็อก โปรแกรมจะเริ่มจาก import machine เสมอ ตัวอย่างเช่น machine.Pin ใช้กำหนดโหมดขาเป็นอินพุตหรือเอาต์พุต สั่งให้ขา GPIO ใด ๆ ทำงานตามที่ต้องการ machine.ADC ใช้สำหรับอ่านค่าอนาล็อก เช่น วัดความชื้นดิน หรือรับค่า VR (Potentiometer) ขณะที่ machine.PWM ใช้สร้างความถี่ปรับค่าดิวตี้ไซเคิลควบคุมความสว่าง LED หรือรอบหมุนของมอเตอร์ ส่วน machine.I2C, machine.SPI, machine.UART คือช่องทางเชื่อมต่อกับเซ็นเซอร์หรือจอแสดงผลต่าง ๆ ไม่ว่าจะเป็นจอ OLED, เซ็นเซอร์วัดอุณหภูมิ, โมดูล GPS หรือ RFID จุดแข็งของ machine คือการเข้าถึงขา logic และ register ภายในชิป ESP32 ได้ทันทีโดยไม่ต้องเขียนภาษา C หรือ C++ เหมือนยุคก่อน ฟังก์ชันทุกตัวรองรับการตั้งค่า hardware-level อย่างแท้จริง และทุกคำสั่งที่เรียกใช้จะถูกแปลงเป็นแรงดันไฟฟ้าจริงที่ขาบนบอร์ด ไม่ใช่แค่การจำลองในซอฟต์แวร์ ฟังก์ชันสำคัญที่ใช้บ่อยในงานควบคุมอุปกรณ์ ได้แก่

- **Pin (ขาดิจิทัล)** ใช้สำหรับควบคุมขาอินพุตและเอาต์พุตแบบดิจิทัลของบอร์ด เช่น ตั้งขาให้เป็นเอาต์พุตเพื่อสั่งงานหลอด LED หรือเป็นอินพุตเพื่ออ่านสถานะปุ่มกด โดยรองรับทั้งแบบ pull-up และ pull-down

• **ADC (อ่านค่าอนาล็อก)** อ่านค่าแรงดันไฟฟ้าที่ขาอนาล็อกและแปลงเป็นค่าดิจิทัล ใช้กับเซ็นเซอร์ที่ให้สัญญาณแอนะล็อกเช่น เซ็นเซอร์ความชื้นในดิน โพเทนชิโอมิเตอร์ หรือเซ็นเซอร์วัดแสง

• **PWM (สัญญาณพัลส์)** สร้างสัญญาณพัลส์แบบปรับความกว้างได้ (Pulse Width Modulation) สำหรับควบคุมอุปกรณ์ที่ต้องการปรับความแรง เช่น ความสว่างของ LED ความเร็วพัดลม หรือความเร็วมอเตอร์

• **Timer (ตั้งเวลาทำงานซ้ำ)** ตั้งเวลาสั่งงานซ้ำอัตโนมัติโดยไม่ต้องเขียน loop เอง เช่น ตั้งเวลาให้อ่านค่าจากเซ็นเซอร์ทุก 1 วินาที หรือกระพริบไฟตามจังหวะ

• **I2C/SPI/UART (สื่อสารกับอุปกรณ์อื่น)** ช่องทางสื่อสารข้อมูลระหว่างบอร์ดกับอุปกรณ์อื่น เช่น เซ็นเซอร์, หน้าจอ, โมดูลสื่อสาร เชื่อมต่อแบบบัส (I2C), แบบอนุกรมหลายสาย (SPI) หรือแบบอนุกรมสองสาย (UART)

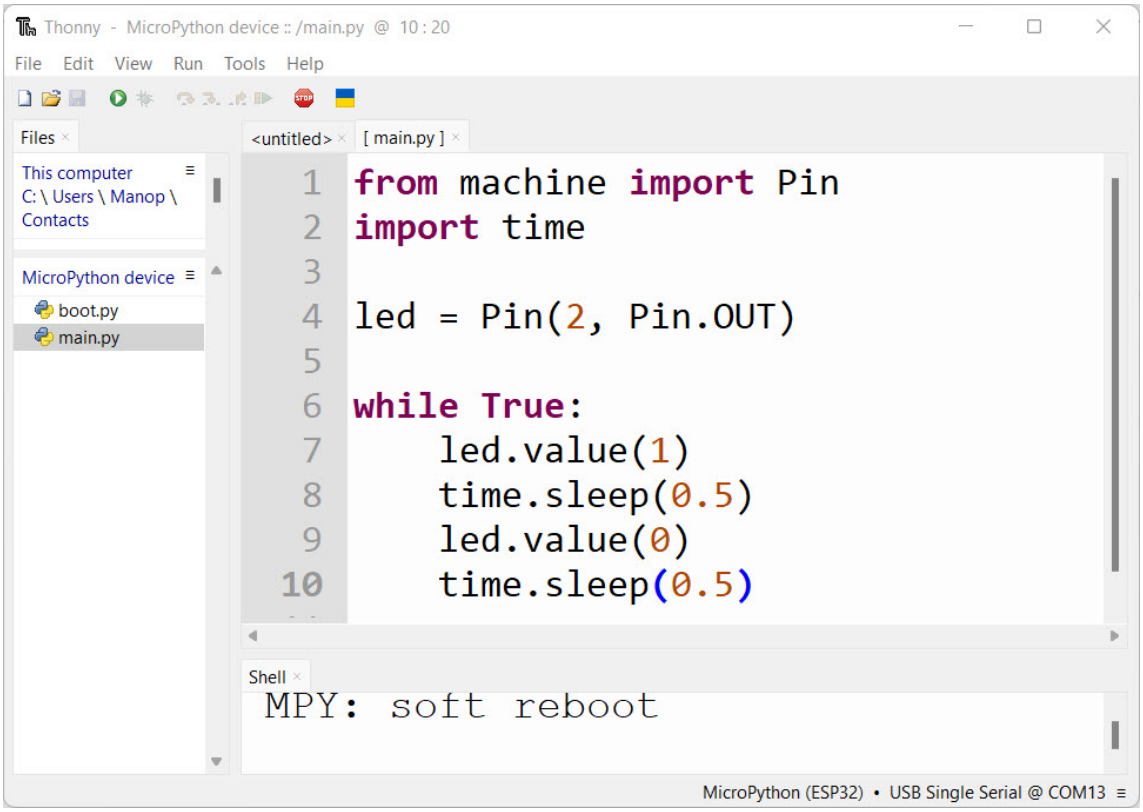
• **SoftI2C/SoftSPI (เลือก software หรือ hardware interface ได้)** เป็นการสื่อสารแบบ I2C หรือ SPI แต่ทำงานผ่านซอฟต์แวร์ จึงเลือกใช้ขาใดก็ได้บนบอร์ด ต่างจาก hardware I2C/SPI ที่ใช้ขาเฉพาะตามฮาร์ดแวร์กำหนด

• **reset, freq (ตั้งค่าการทำงานของชิป)** reset() ใช้รีเซ็ตบอร์ดให้กลับไปเริ่มต้นใหม่เหมือนเพิ่งเปิดเครื่อง ส่วน freq() ใช้กำหนดหรืออ่านค่าความถี่สัญญาณนาฬิกาของบอร์ดที่ใช้กำหนดความเร็วในการประมวลผล

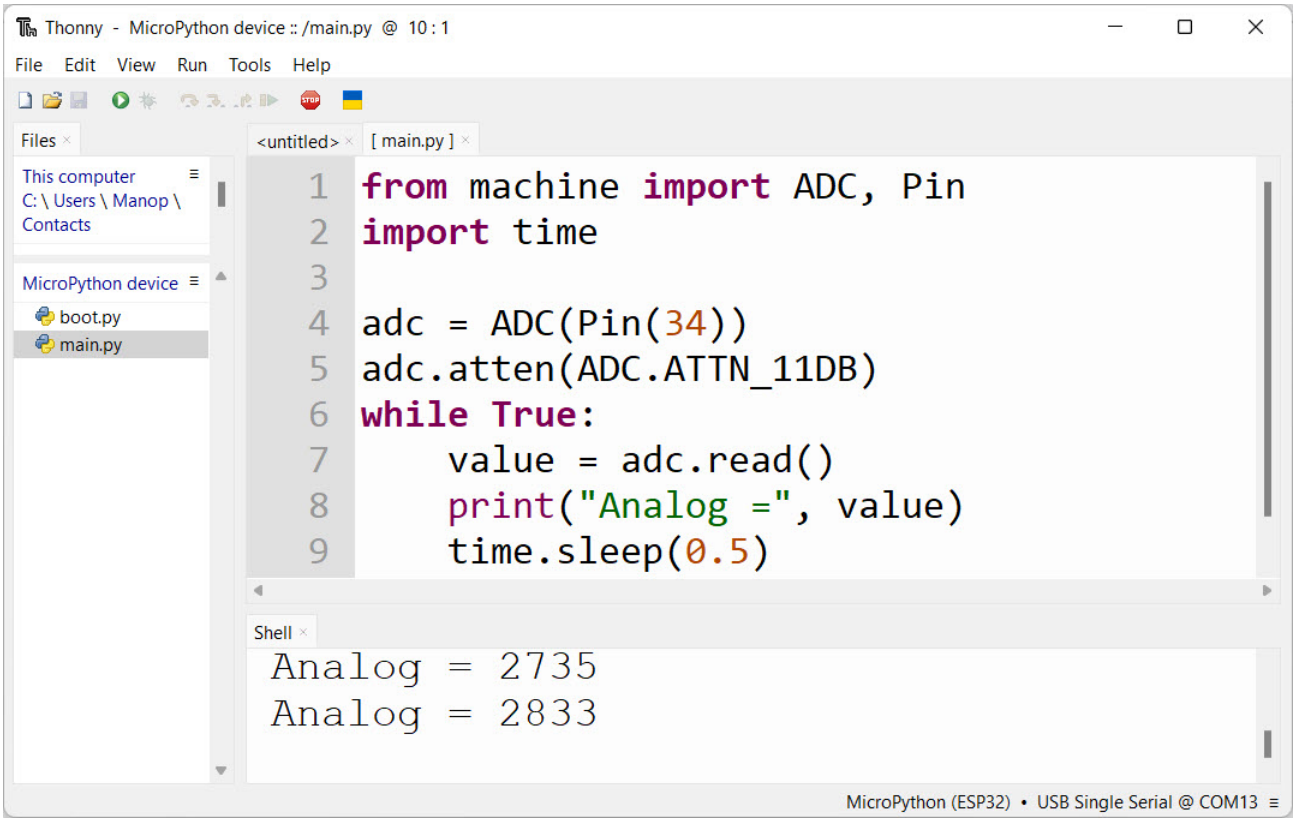
การใช้งานจริงของ **machine** พบได้ในแทบทุกโปรเจกต์ IoT เช่น สั่งเปิด-ปิด LED, วัดค่าเซ็นเซอร์ความชื้นดินและสั่งงานปั๊มน้ำ, อ่านสัญญาณดิจิทัลจาก sensor switch, เชื่อมต่อกับจอแสดงผลหรือเซ็นเซอร์ผ่านบัสต่าง ๆ และควบคุมระบบอัตโนมัติในบ้านหรือโรงงาน

ตัวอย่างใช้งาน machine กับ ESP32

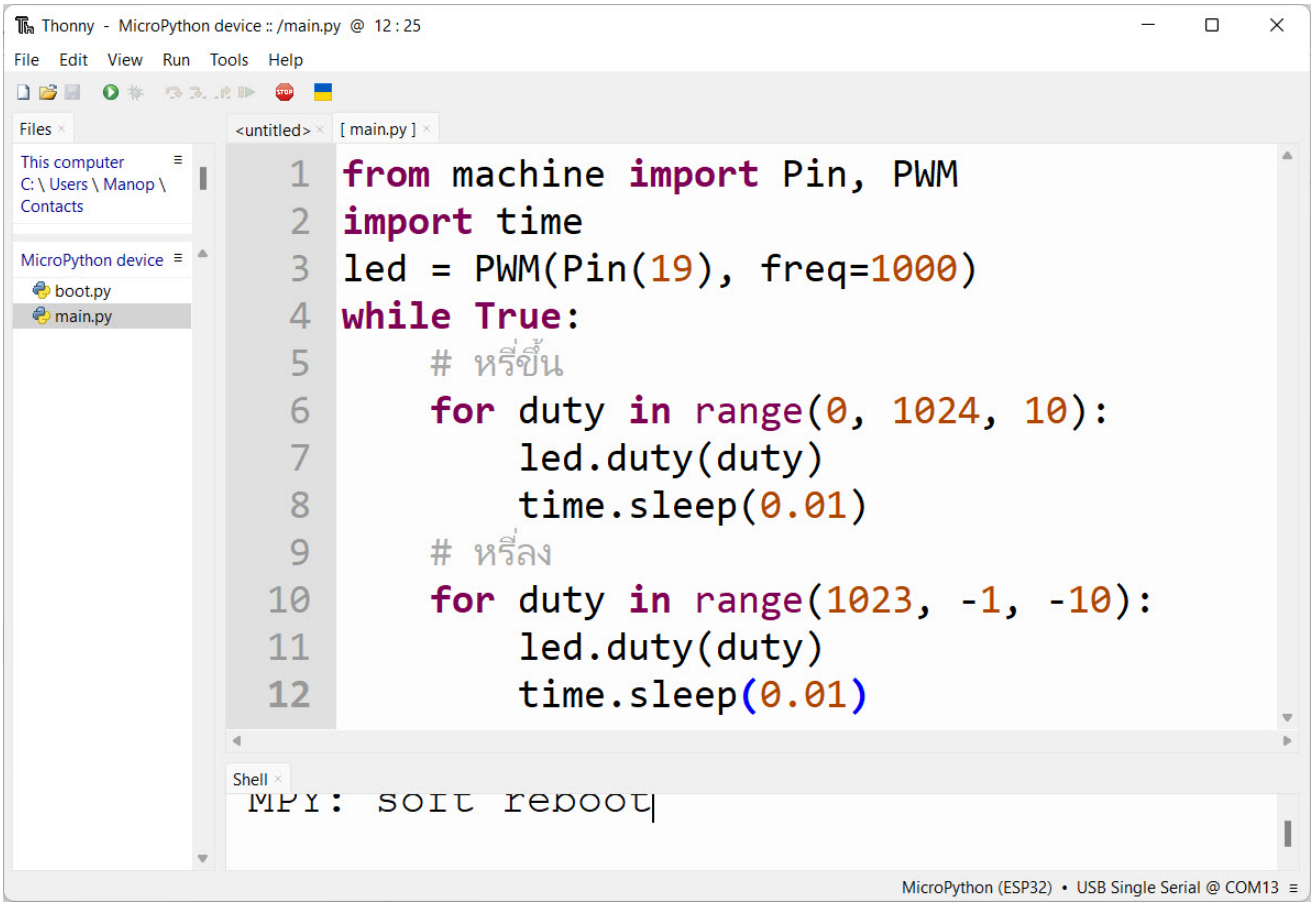
1. เปิด/ปิด LED ที่ขา 2



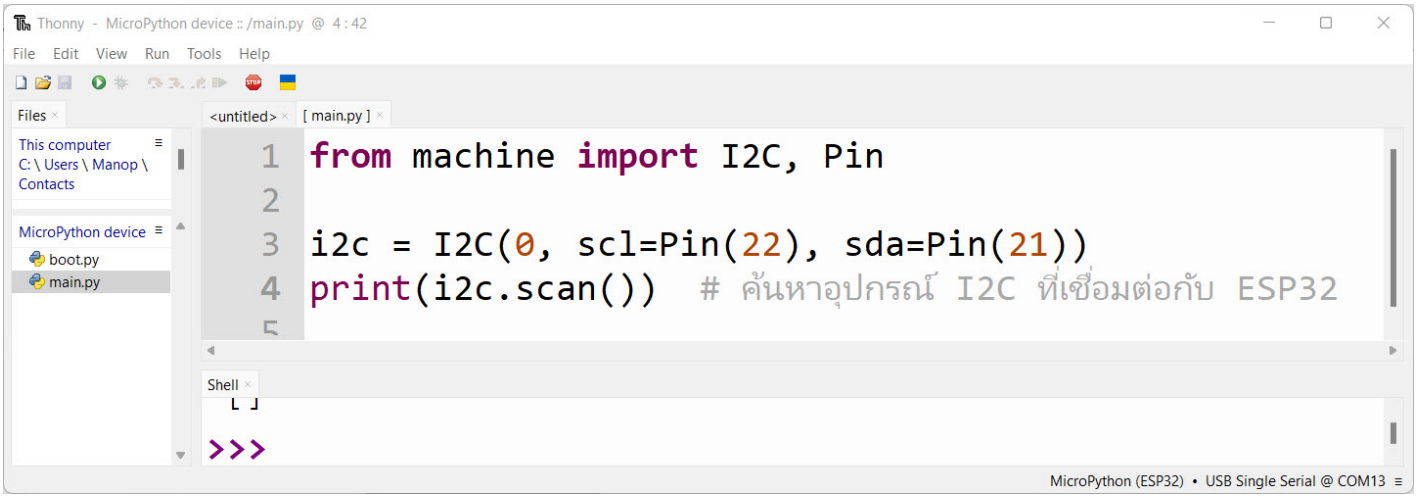
3. อ่านค่าอนาล็อกจากขา 34



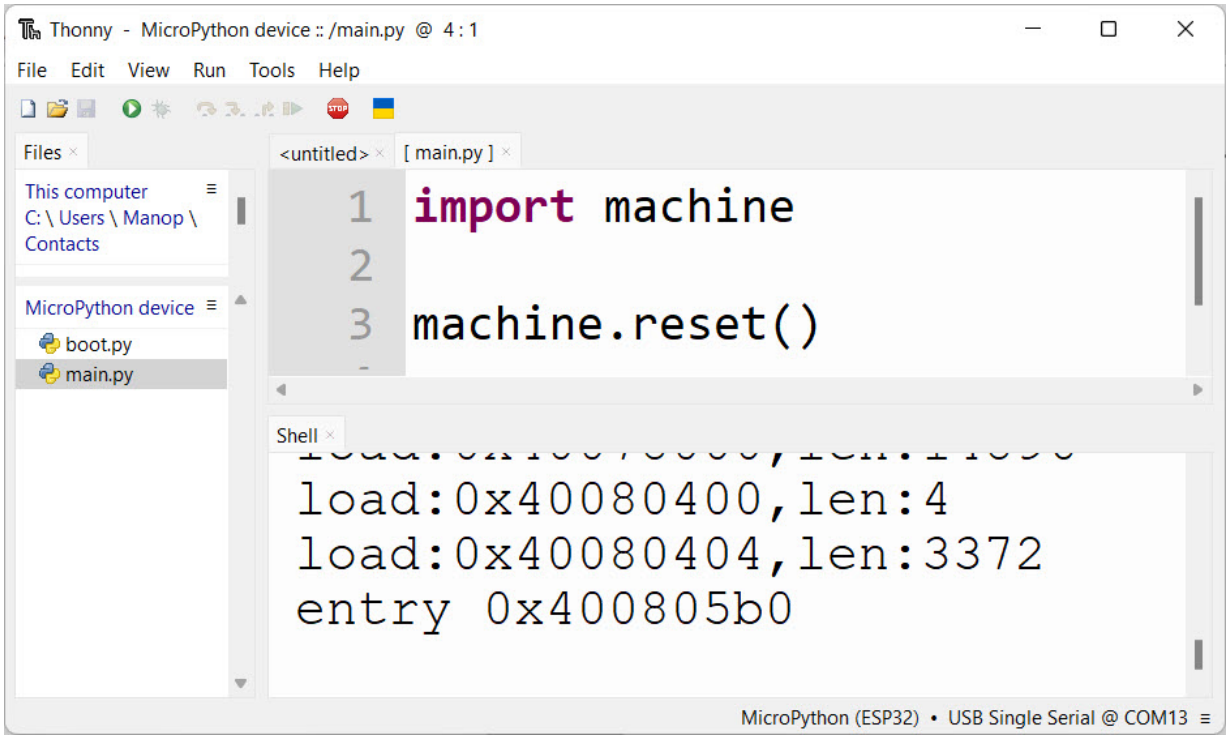
4. สร้าง PWM ควบคุมความสว่าง LED ที่ขา 19



4. สื่อสารกับจอหรือเซ็นเซอร์ผ่าน I2C



5. รีเซ็ตบอร์ด



คำสั่งนี้อย่าเขียนใน main.py เดี๋ยววนรีเซ็ตออกไม่ได้ต้องเฟิร์มแวร์ใหม่นะครับ

ทุกตัวอย่างนี้ทดลองได้กับบอร์ดจริงและเป็นพื้นฐานของงานควบคุมอุปกรณ์ทั้งหมดบน ESP32 หากเข้าใจ **machine** แล้ว จะขยายงานสู่โปรเจกต์ IoT หรือระบบฝังตัวขนาดใหญ่ได้แบบมืออาชีพ

Time

time คือโมดูลพื้นฐานที่ MicroPython เตรียมไว้สำหรับจัดการเรื่อง “เวลา” ในทุกโปรแกรมบน ESP32 ไม่ว่าจะเป็นการหน่วงรอการทำงาน การตั้งจังหวะ การวัดระยะเวลาหรือจับเวลาระหว่างเหตุการณ์ และการซิงโครไนซ์กระบวนการต่าง ๆ ในงานควบคุมอุปกรณ์อิเล็กทรอนิกส์ การเขียนโปรแกรมฝังตัวมักต้องควบคุมเหตุการณ์ให้เกิดขึ้นตามจังหวะที่กำหนด เช่น กระพริบไฟ LED ทุก 0.5 วินาที อ่านค่าจากเซ็นเซอร์ทุก 1 วินาที ตรวจสอบสัญญาณอินพุตเป็นระยะ หรือหน่วงรอให้เซ็นเซอร์เตรียมพร้อมก่อนใช้งาน หากขาดฟังก์ชันเหล่านี้ โปรแกรมจะไม่สามารถควบคุมอุปกรณ์ในโลกจริงให้ทำงานได้แม่นยำตรงตามที่เรากำหนด

โมดูล time ใน **MicroPython** มีฟังก์ชันหลักที่ใช้บ่อย เช่น **time.sleep()** สำหรับหน่วงเวลาตามจำนวนวินาทีที่กำหนด, **time.sleep_ms()** สำหรับหน่วงตามหน่วยมิลลิวินาที, **time.sleep_us()** สำหรับหน่วยไมโครวินาที เหมาะกับงานที่ต้องการความละเอียดสูง นอกจากนี้ยังมี **time.ticks_ms()** และ **time.ticks_us()** ซึ่งใช้สำหรับจับเวลาแบบแม่นยำสูงและไม่มีปัญหาเรื่อง integer overflow ในระยะยาว เหมาะกับงานจับเวลา

แบบเรียลไทม์ เช่น จับระยะเวลาระหว่างอินพุตหรือวัดช่วงเวลาตอบสนองของเซ็นเซอร์ การเลือกใช้ sleep หรือ ticks ขึ้นกับว่าต้องการแค่หน่วงรอ หรือจะใช้วัดระยะเวลาระหว่างเหตุการณ์จริง

ความสำคัญของ time สำหรับ ESP32 คือช่วยให้ควบคุมการทำงานซ้ำ ๆ อย่างมีระบบ เช่น สั่งให้วัดเซ็นเซอร์ทุก 2 วินาที เปิดปิดอุปกรณ์ตามตารางเวลา ทำระบบ scheduler หรือป้องกันการค้างของโปรแกรมในกรณีที่ต้องรออุปกรณ์ภายนอก โปรแกรมจำนวนมากจะใช้ time.sleep() ภายใน loop เพื่อให้วงจรอิเล็กทรอนิกส์มีเวลาทำงานหรือให้ผลลัพธ์ทันก่อนจะวนรอบใหม่ ข้อควรระวังคือการใช้ sleep() จะหยุดโปรแกรมทั้งหมดในช่วงเวลานั้น ถ้าต้องควบคุมหลายอุปกรณ์พร้อมกันควรใช้ ticks จับเวลาร่วมกับโค้ดตรวจสอบสถานะจะยืดหยุ่นกว่า

ตัวอย่างการใช้งาน time บน ESP32 จากตัวอย่างที่ได้ทดลองผ่านมาแล้ว เช่น

- กระพริบไฟ LED ทุก 1 วินาที
- อ่านค่าเซ็นเซอร์ทุก 0.5 วินาที

โปรแกรมเหล่านี้สามารถทดลองได้กับ ESP32 จริงทันที หากเข้าใจหลักการของ time แล้วจะสามารถพัฒนาโปรแกรมที่ต้องการควบคุมจังหวะ ทำ Scheduler หรือลูปแบบเรียลไทม์ได้อย่างแม่นยำในทุกโปรเจกต์

network

network คือโมดูลสำคัญที่ทำให้ ESP32 สามารถเชื่อมต่อกับเครือข่าย Wi-Fi ได้เหมือนคอมพิวเตอร์หรือสมาร์ทโฟน ซึ่งเป็นหัวใจของงาน IoT ทุกรูปแบบบนบอร์ด ESP32 ไม่ว่าคุณจะสร้างบ้านอัจฉริยะ, ส่งข้อมูลเซ็นเซอร์ขึ้น cloud, เชื่อมต่อกับแอปพลิเคชันมือถือ, ควบคุมอุปกรณ์จากระยะไกล หรือใช้งาน MQTT/HTTP/OTA โมดูล network จะเป็นตัวกลางสำคัญที่เชื่อมบอร์ดกับโลกอินเทอร์เน็ตหรือเครือข่ายท้องถิ่น

การใช้งาน network กับ ESP32 สามารถแบ่งเป็น 2 โหมดหลัก คือ Station (STA) ที่ใช้เชื่อมต่อกับ Wi-Fi router เหมือนอุปกรณ์ทั่วไป และ Access Point (AP) ที่เปลี่ยน ESP32 ให้กลายเป็นจุดปล่อย Wi-Fi ให้มือถือหรือคอมพิวเตอร์อื่นเชื่อมต่อเข้ามาโดยตรง โหมด STA เหมาะกับโปรเจกต์ที่ต้องส่งข้อมูลไปยัง server, MQTT broker หรือ cloud services ต่าง ๆ ส่วนโหมด AP เหมาะกับโปรเจกต์ที่ต้องการให้ผู้ใช้เข้ามาควบคุมบอร์ดจากมือถือหรือคอมพิวเตอร์โดยตรงโดยไม่ต้องผ่าน router กลาง

ในการใช้งานจริง การเชื่อมต่อ Wi-Fi แบบ Station จะเริ่มด้วยการสร้างออบเจกต์ network.WLAN(network.STA_IF) จากนั้นตั้งค่าให้ active(True) แล้ว connect(SSID, PASSWORD) รอจนกว่าจะเชื่อมต่อสำเร็จ สามารถตรวจสอบสถานะได้ด้วย isconnected() และดูหมายเลข IP address ได้ด้วย

ifconfig() สิ่งที่ต้องระวัง คือหาก Wi-Fi หลุดหรือเปลี่ยน hotspot อาจต้องรันคำสั่ง reconnect ใหม่เสมอ โดยเฉพาะงาน IoT ที่ต้องทำงานต่อเนื่องตลอดเวลา

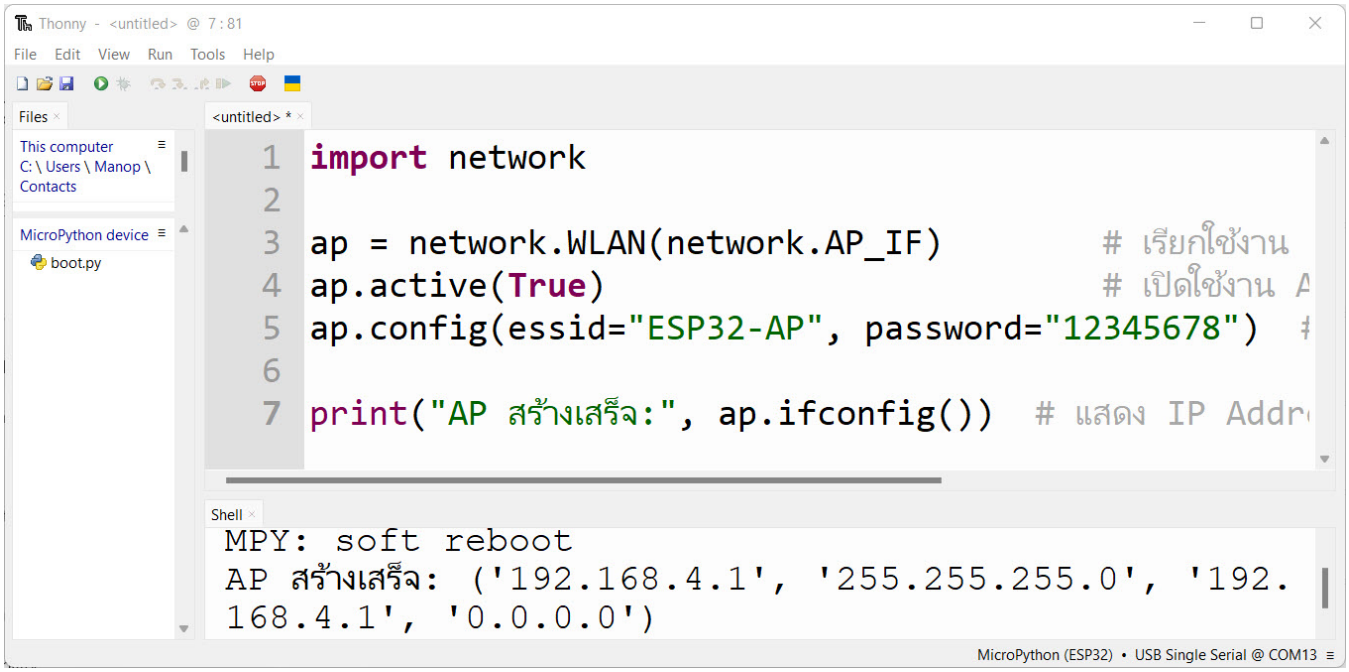
นอกจากเชื่อมต่อ Wi-Fi ธรรมดา network ยังใช้สำหรับสื่อสารข้อมูลระดับล่างกับโปรโตคอล TCP/UDP, ทำเว็บเซิร์ฟเวอร์ (HTTP) หรือเชื่อมต่อ MQTT โดยตรงได้ โมดูลนี้ยังช่วยให้ ESP32 อัปเดตเฟิร์มแวร์แบบ OTA - (Over-the-Air) ผ่าน Wi-Fi ได้โดยไม่ต้องเสียบสาย USB ซึ่งสำคัญกับการดูแลอุปกรณ์จำนวนมากในสถานที่จริง

ตัวอย่างการเชื่อมต่อ Wi-Fi (Station Mode) แบบละเอียด:

ตั้งค่า ESP32 ให้เป็น Access Point (โหมด AP)

เมื่อกำหนดให้ ESP32 ทำงานในโหมด Access Point (AP Mode) จะทำให้บอร์ดกระจายสัญญาณ WiFi ของตัวเองออกมา โดยอุปกรณ์อื่น ๆ เช่น คอมพิวเตอร์, สมาร์ทโฟน หรืออุปกรณ์ IoT ตัวอื่น สามารถเชื่อมต่อเข้าเครือข่ายที่ ESP32 สร้างขึ้นมาได้โดยตรง ไม่ต้องพึ่ง Router โค้ดตัวอย่างการตั้งค่า AP มีดังนี้

(ดูภาพหน้าถัดไป)



คำอธิบายแต่ละคำสั่ง:

- network.WLAN(network.AP_IF)

  สร้างอินสแตนซ์ของ WiFi interface โดยเลือกโหมด Access Point (AP_IF)

- ap.active(True)

  เปิดใช้งานอินเทอร์เฟซ AP เพื่อเริ่มปล่อยสัญญาณ WiFi

- ap.config(essid="ESP32-AP", password="12345678")

  กำหนดชื่อเครือข่าย (SSID) เป็น "ESP32-AP" และตั้งรหัสผ่านตามที่กำหนด โดยรหัสผ่านต้องมีความยาวอย่างน้อย 8 ตัวอักษรขึ้นไป ตามมาตรฐานความปลอดภัยของระบบ WiFi

- ap.ifconfig()

  ดึงข้อมูลการตั้งค่า IP ปัจจุบันของ Access Point ซึ่งโดยค่าเริ่มต้น ESP32 จะกำหนด IP address ของตัวเองเป็น 192.168.4.1 พร้อมระบบ DHCP ในตัว

เพิ่มเติมสำหรับโหมด Access Point:

- ค่า default IP address: 192.168.4.1/24
- หากไม่มีการตั้งค่า DHCP เพิ่มเติม ESP32 จะปล่อย DHCP server อัตโนมัติให้ client ที่เชื่อมต่อเข้ามารับ IP ไปใช้งานได้ทันที
- สามารถรองรับอุปกรณ์ client หลายเครื่องเชื่อมต่อเข้ามาพร้อมกันได้

เมื่อตั้งค่าในโหมด Access Point แล้ว ESP32 จะทำหน้าที่คล้ายเราเตอร์ขนาดเล็ก กระจายสัญญาณ WiFi สำหรับการเชื่อมต่อโดยตรงจากอุปกรณ์อื่น เช่น โทรศัพท์มือถือ, คอมพิวเตอร์ หรือบอร์ด IoT ตัวอื่น ๆ โดยไม่

ต้องพึ่งพาอุปกรณ์เน็ตเวิร์กภายนอก เหมาะสำหรับใช้ทำระบบควบคุมอัตโนมัติแบบปิดระบบ (local network) การทดสอบระบบ IoT หรือสร้าง Web Server บน ESP32 ที่ไม่ต้องออกสู่อินเทอร์เน็ต

ข้อควรระวังคือ หากบอร์ดรีบูตหรือเกิด error ระหว่างเชื่อมต่อ ควรเพิ่มโค้ดตรวจสอบสถานะและ reconnect ใหม่ได้เสมอ รวมถึงควรหลีกเลี่ยงการใส่รหัสผ่านผิดซ้ำ ๆ เพราะ ESP32 อาจเข้าโหมด boot loop ในบางเฟิร์มแวร์

สำหรับงาน IoT ที่ต้องการเชื่อมต่อ MQTT, HTTP, WebSocket หรือรับส่งข้อมูลแบบ API, network คือจุดตั้งต้นของทุกโปรเจกต์ หากฝึกใช้ให้คล่อง สามารถประยุกต์โปรแกรมควบคุมจากระยะไกลหรือพัฒนาแอปควบคุมอุปกรณ์ IoT ขนาดใหญ่ได้อย่างมืออาชีพ

OS

os คือโมดูลที่ทำหน้าที่จัดการไฟล์และโฟลเดอร์ในระบบแฟลชของบอร์ด ESP32 เมื่อใช้งาน MicroPython โมดูลนี้จะทำให้บอร์ด ESP32 ทำตัวเหมือน “คอมพิวเตอร์ขนาดจิ๋ว” ที่มีระบบแฟ้มข้อมูลในตัวเอง ทุกการสร้าง ลบ เปลี่ยนชื่อไฟล์ หรือการตรวจสอบรายชื่อไฟล์ภายใน flash memory จะต้องใช้ os เข้ามาช่วยควบคุมโดยตรง

โมดูล os ยังเป็นหัวใจสำคัญของโปรเจกต์ที่มีการเก็บ log, โหลดหรือเซฟไฟล์ตั้งค่า (config), ทำ OTA

(อัปเดตเฟิร์มแวร์ผ่าน Wi-Fi) หรือโอน/อ่านไฟล์ข้อมูลทุกชนิด

โดยปกติ เมื่อนำโค้ดหรือไฟล์ .py, .txt, .json, หรือไฟล์ภาพขึ้นไปไว้ในบอร์ด ESP32 ไม่ว่าจะผ่าน Thonny, ampy, rshell หรือเครื่องมืออื่น ข้อมูลเหล่านี้จะถูกจัดเก็บไว้ในพื้นที่แฟลชของบอร์ดเหมือนเป็นไดรฟ์ย่อยบนคอมพิวเตอร์ การใช้ os.listdir() จะช่วยให้ตรวจสอบได้ว่าใน flash มีไฟล์อะไรอยู่บ้าง ซึ่งสำคัญมากสำหรับการ debug, การติดตั้งไลบรารี, หรือการแบ่งไฟล์หลายโปรเจกต์ในบอร์ดเดียว

ขณะที่ os.remove(), os.rename(), os.mkdir() จะช่วยให้เขียนโปรแกรมที่จัดการไฟล์หรือโฟลเดอร์อัตโนมัติ เช่น เมื่ออ่านข้อมูลจากเซ็นเซอร์เก็บลงไฟล์ log สามารถตั้งเงื่อนไขให้ลบไฟล์เก่า สร้างไฟล์ใหม่ หรือจัดเก็บข้อมูลแยกตามวันที่

โมดูล os ยังมีฟังก์ชันสำหรับตรวจสอบขนาดไฟล์ (os.stat()), เปลี่ยน working directory (os.chdir()), และจัดการกับระบบไฟล์ของบอร์ด ในกรณีที่ต้องอัปเดตเฟิร์มแวร์ หรือแก้ปัญหาบอร์ดค้างจากไฟล์เสียหาย การจัดการไฟล์อย่างถูกต้องและปลอดภัยมีความสำคัญกับงาน IoT ที่ต้องเก็บข้อมูลระยะยาวหรือรับ–ส่งไฟล์กับ cloud/server เป็นประจำ

ตัวอย่างการใช้งาน os กับ ESP32

1. แสดงรายชื่อไฟล์ทั้งหมดในบอร์ด

แสดงรายชื่อไฟล์และโฟลเดอร์ที่อยู่ในระบบแฟ้มปัจจุบัน
import os print(os.listdir())

2. สร้างโฟลเดอร์ใหม่สำหรับเก็บไฟล์ log

ตัวอย่างการตรวจสอบว่ามีโฟลเดอร์ชื่อ log แล้วหรือยัง หากยังไม่มีจะสร้างใหม่
import os **if 'log' not in os.listdir():** **os.mkdir('log')**

3. ลบไฟล์เก่าออกจากบอร์ด

ตัวอย่างการลบไฟล์ oldfile.txt ถ้ามีอยู่ในระบบ
import os **if 'oldfile.txt' in os.listdir():** **os.remove('oldfile.txt')**

4. เปลี่ยนชื่อไฟล์

ตัวอย่างการเปลี่ยนชื่อไฟล์จาก data.txt เป็น data_bak.txt
import os **if 'data.txt' in os.listdir():** **os.rename('data.txt', 'data_bak.txt')**

5. เขียนข้อมูลลงไฟล์และอ่านกลับ

ตัวอย่างการเขียนข้อความลงไฟล์ test.txt แล้วอ่านกลับมาแสดงผล

```
# ไม่ต้อง import อะไร
with open('test.txt', 'w') as f:
    f.write('Hello ESP32!\n')
with open('test.txt', 'r') as f:
    print(f.read())
```

6. ตรวจสอบขนาดไฟล์

ตัวอย่างการอ่านขนาดของไฟล์ test.txt จากตำแหน่ง index ที่ 6 ในผลลัพธ์ของ os.stat()

```
import os
stat = os.stat('test.txt')
print("ขนาดไฟล์ (byte):", stat[6])
```

os จึงเป็นเครื่องมือสำคัญในการควบคุมและจัดการข้อมูลระดับระบบแฟ้มใน ESP32 ทุกโปรเจกต์ที่ต้องเก็บ log, โหลดค่า config, อัปเดตเฟิร์มแวร์, หรือทำงานกับไฟล์จำนวนมาก จะต้องใช้ os เป็นประจำ ฝึกใช้ os ให้คล่องตั้งแต่ต้นจะช่วยให้ดูแลระบบและแก้ปัญหาหน้างานได้อย่างมืออาชีพ

sys

sys เป็นโมดูลมาตรฐานที่ใช้สำหรับเข้าถึงและจัดการข้อมูลเกี่ยวกับตัวระบบ MicroPython และสภาพแวดล้อมของ ESP32 โดยตรง แม้จะไม่ได้เกี่ยวข้องกับการควบคุมฮาร์ดแวร์เช่น machine หรือการจัดการไฟล์เช่น os แต่ sys มีประโยชน์อย่างมากในงานตรวจสอบสถานะเวอร์ชันของเฟิร์มแวร์ การ debug โปรแกรม การตรวจจับ platform ที่กำลังใช้งาน และการจัดการพฤติกรรมบางอย่างของระบบฝังตัว เช่น การหยุดหรือรีเซ็ตโปรแกรม การรับ argument หรือการตั้งค่า path ในการ import ไลบรารี

สิ่งที่ใช้บ่อยที่สุดใน sys คือการตรวจสอบข้อมูลระบบ เช่น sys.platform ซึ่งจะบอกว่าโค้ดกำลังรันบน ESP32 จริงหรือไม่ (กรณีต้องการพัฒนาโค้ดเดียวกันให้รองรับหลาย platform) หรือ sys.version สำหรับดูเวอร์ชัน MicroPython ที่ใช้งานอยู่จริง ซึ่งสำคัญสำหรับงานที่ต้องอาศัยฟีเจอร์ใหม่ ๆ หรือตรวจสอบความเข้ากันได้กับไลบรารีภายนอก นอกจากนี้ sys.path ยังใช้สำหรับตรวจสอบหรือเพิ่ม path สำหรับ import โมดูลเสริม หากต้องอัปโหลดไลบรารีหรือไฟล์ที่ไม่อยู่ในโฟลเดอร์หลัก สามารถเพิ่ม path ลงไปได้เองใน runtime

ในงานบางประเภทที่ต้องการให้โปรแกรมหยุดทำงานหรือออกจากลูปแบบฉุกเฉิน สามารถใช้ sys.exit() เพื่อจบโปรแกรมได้ทันที (คล้ายกับ exit() ในภาษาอื่น ๆ) และยังสามารถตรวจสอบหรือจัดการ resource อื่น ๆ ในระบบผ่าน sys ได้ในระดับลึก เช่น การตั้งค่า recursion depth, การจัดการ exception handler ฯลฯ

ตัวอย่างการใช้งาน sys บน ESP32

1. ตรวจสอบ platform และเวอร์ชัน

แสดงชื่อระบบและเวอร์ชันของ MicroPython ที่ ESP32 ใช้งานอยู่
import sys **print("Platform:", sys.platform)** **print("MicroPython Version:", sys.version)**

2. ตรวจสอบ path สำหรับ import ไลบรารี

ใช้ดูว่าระบบจะค้นหาไฟล์ .py จากโฟลเดอร์ใดบ้างเมื่อนำเข้าไลบรารี
import sys **print("PATH สำหรับ import:", sys.path)**

3. เพิ่ม path สำหรับ import ไลบรารีที่โฟลเดอร์ libs

ทำให้สามารถ import ไลบรารีในโฟลเดอร์ /libs ได้ (เช่น from mylib import foo)
import sys **sys.path.append('/libs')** **print(sys.path)**

4. สั่งหยุดโปรแกรมทันที

⚠ คำสั่ง sys.exit() จะ **หยุดการทำงานทันที** และไม่รันโค้ดบรรทัดถัดไป บางเฟิร์มแวร์อาจไม่แสดงข้อความเพิ่มเติมหลังคำสั่งนี้ (เพราะหยุดทำงานไปแล้ว)
import sys **print("หยุดโปรแกรม")** **sys.exit()**

5. ตรวจสอบขนาด integer สูงสุดที่รองรับ

แสดงค่าจำนวนเต็มสูงสุดที่สามารถจัดเก็บได้ในระบบ (ใน MicroPython บางรุ่นอาจไม่รองรับ sys.maxsize และเกิด AttributeError ได้)
import sys **print("int max size:", sys.maxsize)**

แม้ sys จะไม่ใช่โมดูลที่ต้องเรียกใช้ทุกโปรแกรม แต่การเข้าใจและใช้ sys อย่างถูกจุด จะช่วยให้สามารถพัฒนาโค้ดที่ตรวจสอบสถานะระบบ ติดตามปัญหา และย้ายข้าม platform ได้อย่างยืดหยุ่น ยิ่งโปรเจกต์ที่มีขนาดใหญ่หรือทำงานกับไลบรารีจำนวนมาก การใช้ sys ร่วมกับเทคนิคการตรวจสอบ version และ path จะช่วยลดปัญหา incompatibility และ debug ได้ง่ายขึ้นมาก

ujson

ujson (MicroPython JSON) คือโมดูลที่ใช้สำหรับแปลงข้อมูลระหว่างรูปแบบ JSON กับโครงสร้างข้อมูลของ Python (dict, list, ฯลฯ) โดยเฉพาะอย่างยิ่งในงาน IoT ที่ต้องรับส่งข้อมูลกับ Cloud, Server หรือ REST API การจัดเก็บหรือรับส่งข้อมูลรูปแบบ JSON ถือเป็นมาตรฐานสากล เนื่องจาก JSON อ่านง่าย ขนาดเล็ก และใช้งานข้ามระบบได้สะดวก ujson ถูกออกแบบให้ทำงานได้รวดเร็วและกินทรัพยากรน้อย เหมาะกับบอร์ดไมโครคอนโทรลเลอร์ที่มีแรมและแฟลชจำกัด

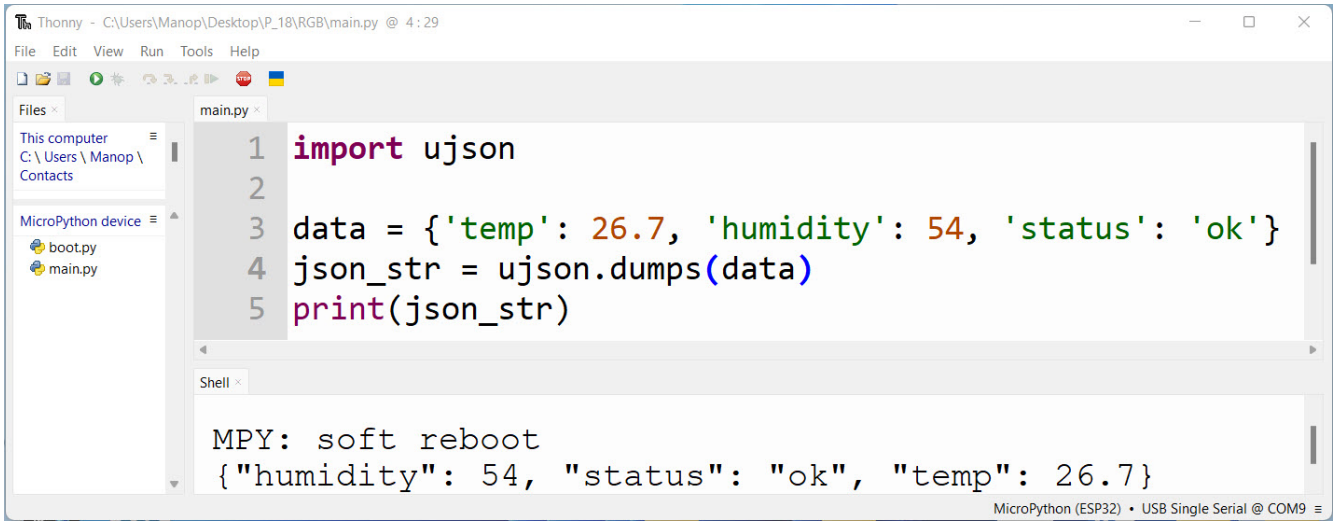
ประโยชน์หลักของ ujson คือใช้แปลงข้อมูลจาก dictionary หรือ list ให้เป็น string รูปแบบ JSON (serialize) เพื่อส่งออกผ่าน Wi-Fi, MQTT, HTTP หรือบันทึกลงไฟล์ และแปลงข้อมูลที่ได้รับจาก server (deserialize) ให้กลับเป็นโครงสร้างข้อมูลของ MicroPython เพื่อนำไปประมวลผลต่อ เช่น รับค่าจาก Cloud API เป็น JSON แล้วดึงเฉพาะ field ที่ต้องการใช้งาน หรือจัดเก็บ log อุปกรณ์ในรูปแบบไฟล์ JSON เพื่อเรียกใช้ในอนาคต

ฟังก์ชันสำคัญของ ujson มี 2 ตัวคือ

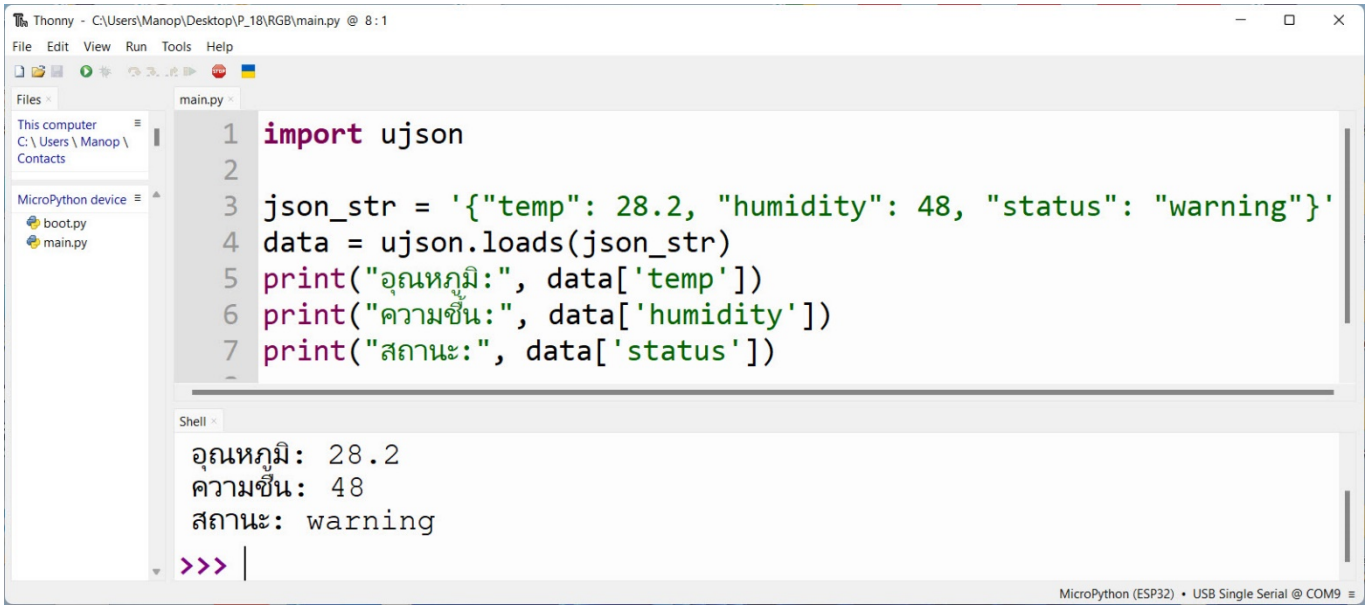
- ujson.dumps(obj) สำหรับแปลง Python dict, list, ฯลฯ เป็น JSON string
- ujson.loads(str) สำหรับแปลง JSON string กลับเป็น Python dict, list, ฯลฯ

ตัวอย่างการใช้งาน ujson บน ESP32

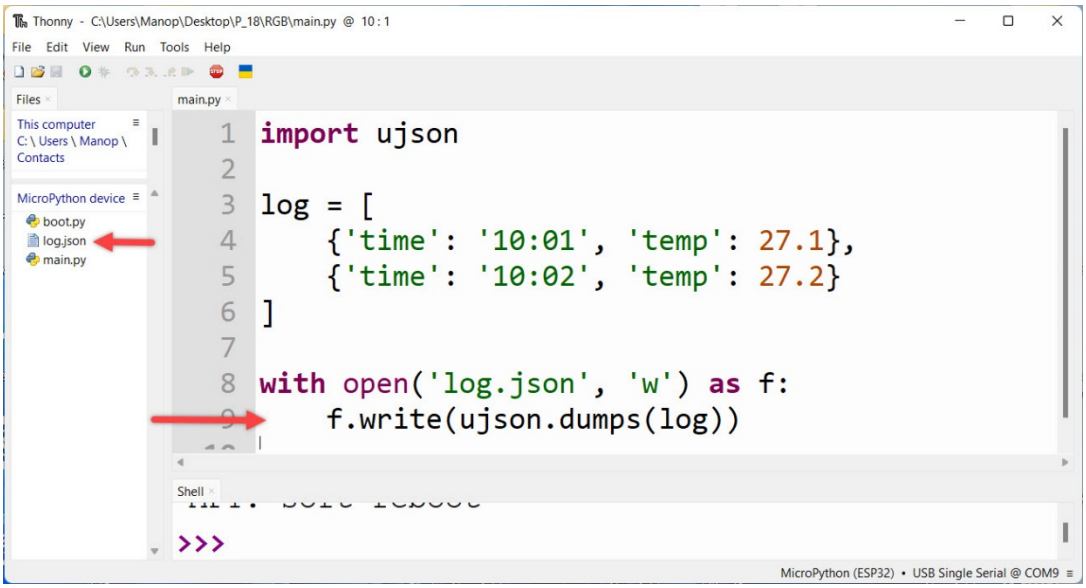
1. แปลงข้อมูล dict เป็น JSON string เพื่อส่งออก



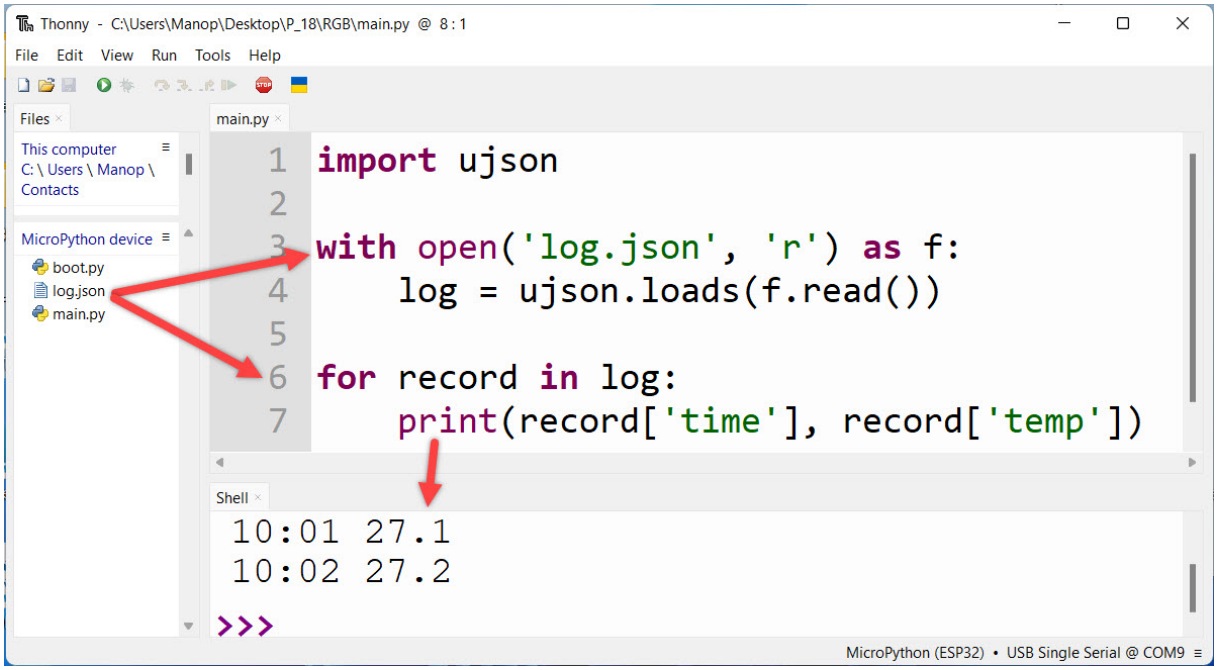
2. แปลง JSON string ที่ได้รับจาก server เป็น dict



3. บันทึกข้อมูลลงไฟล์ log.json



4. โหลดข้อมูล log จากไฟล์



ข้อควรระวังในการใช้ ujson คือชนิดข้อมูลที่นำไป serialize ต้องเป็นข้อมูลพื้นฐานที่ JSON รองรับ (dict, list, int, float, string, bool, None) ไม่รองรับ object หรือชนิดข้อมูลเฉพาะแบบ Python บางอย่างโดยตรง หากต้องการบันทึก object ซับซ้อนควรแปลงเป็น dict ให้เรียบร้อยก่อน

การฝึกใช้ ujson ให้คล่องจะช่วยให้การสื่อสารข้อมูลระหว่าง ESP32 กับระบบภายนอก (เช่น Cloud, Server, MQTT, Web API) เป็นไปอย่างเป็นระบบ ยืดหยุ่น และขยายงานต่อได้ง่ายในทุกโปรเจกต์ IoT

random

ในระบบฝังตัวและ IoT แม้จะเป็นงานที่เน้นการควบคุมฮาร์ดแวร์แบบแม่นยำ แต่ความสามารถในการสร้างข้อมูลสุ่มก็มีบทบาทสำคัญอย่างมากในหลายสถานการณ์ เพราะโลกของการเชื่อมต่อและควบคุมอุปกรณ์จริงไม่ได้มีแต่เรื่องของการสั่งงานตรง ๆ เท่านั้น แต่ยังมีความจำเป็นต้องสุ่มข้อมูลเพื่อทดสอบระบบ, ลดปัญหาการชนกันของเครือข่าย, กระจายภาระโหลด, สร้างรหัสผ่าน, หรือลดความเสี่ยงจากการโจมตีซ้ำแบบเดิม การสร้างตัวเลขสุ่มจึงเป็นส่วนหนึ่งของการทำให้ระบบมีความเสถียร ปลอดภัย และพร้อมรับมือกับโลกจริงที่มีความไม่แน่นอนอยู่เสมอ

ใน MicroPython บน ESP32 มีโมดูล random ติดตั้งมาพร้อมในระบบมาตั้งแต่ต้น โดยถูกออกแบบให้ทำงานได้อย่างเบา (lightweight) ไม่กินทรัพยากรหน่วยความจำมากเกินไป จึงเหมาะกับบอร์ดไมโครคอนโทรลเลอร์ที่มีข้อจำกัดเรื่องแรมและพื้นที่แฟลช random ช่วยให้สามารถสร้างข้อมูลสุ่มได้ทั้งแบบจำนวนเต็ม (integer) แบบเลขทศนิยม (float) แบบสุ่มหยิบจากรายการ (list choice) และแบบสุ่มกระจายช่วง (uniform distribution)

ความสำคัญของ random ในโปรแกรมฝังตัวจริง

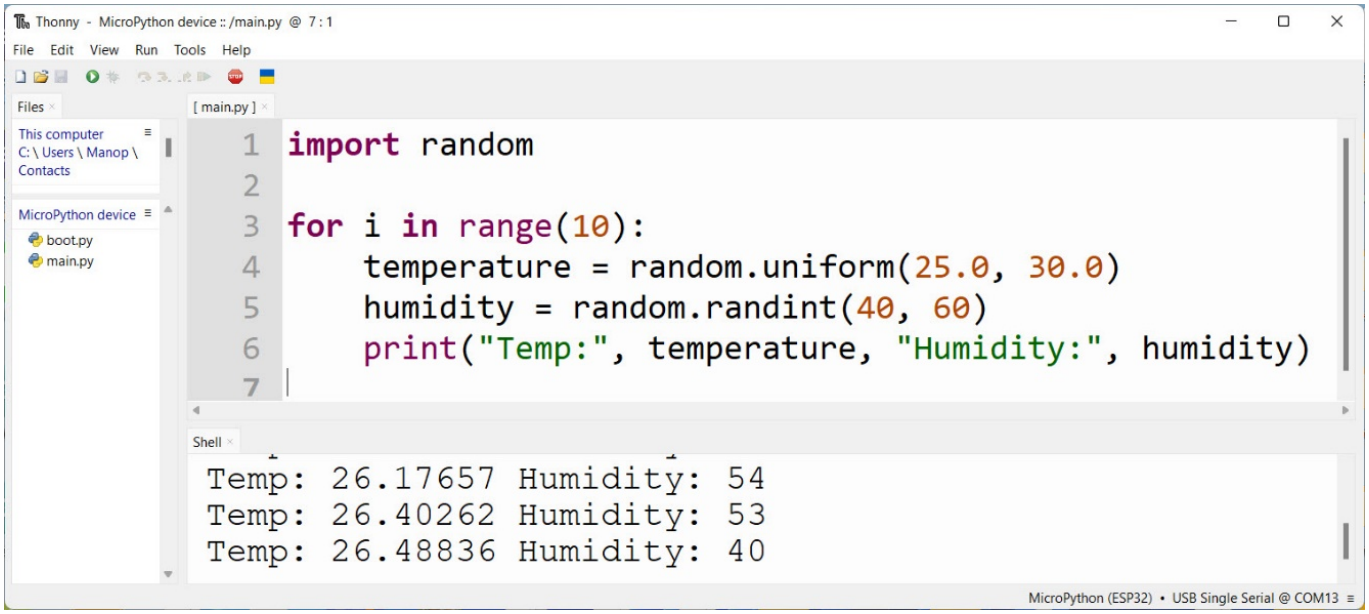
1. **ใช้ในการทดสอบระบบ (Testing & Simulation):**
   ระหว่างที่ยังไม่มีเซ็นเซอร์จริงหรือฮาร์ดแวร์เชื่อมต่อครบ เราสามารถใช้ random สร้างข้อมูลสุ่มจำลอง เช่น อุณหภูมิ, ความชื้น, ความดัน เพื่อทดสอบการประมวลผล การแสดงผล หรือการเชื่อมต่อกับ cloud ให้แน่ใจก่อนนำของจริงมาต่อ
2. **ลดการชนสัญญาณในระบบ IoT (Collision Avoidance):**
   เมื่อมี ESP32 หลายตัวเชื่อมต่อ server พร้อมกัน หากทุกตัว reconnect หรือส่งข้อมูลในช่วงเวลาเดียวกันอาจทำให้ network หนาแน่นจนล่ม การสุ่มเวลาหน่วงแตกต่างกันก่อนเชื่อมต่อจะช่วยลดโอกาสชนกันได้อย่างมีประสิทธิภาพ
3. **ใช้ในระบบความปลอดภัย (Security):**
   เช่น สุ่มรหัส OTP (One Time Password) ในระบบยืนยันตัวตนชั่วคราว สุ่ม session key ในระบบเข้ารหัส หรือสุ่ม salt เพื่อเพิ่มความปลอดภัยในการจัดเก็บรหัสผ่าน
4. **ใช้ในระบบควบคุมที่ต้องการพฤติกรรมหลากหลาย:**
   ตัวอย่างเช่น สุ่มลำดับการเปิดไฟตกแต่ง, สุ่มเวลาในการเปิดปิดเครื่องจักรทดสอบความเสถียร, หรือสุ่มเส้นทางวิ่งของหุ่นยนต์จำลอง
5. **ใช้กระจายโหลดในระบบที่มีหลาย ESP32 ทำงานร่วมกัน:**
   เช่น ระบบเซ็นเซอร์จำนวนมากที่เชื่อม MQTT หากสุ่มช่วงเวลาส่งข้อมูลคนละเวลา จะช่วยให้ broker ไม่รับโหลดกระจุกตัว

ฟังก์ชันหลักของ random

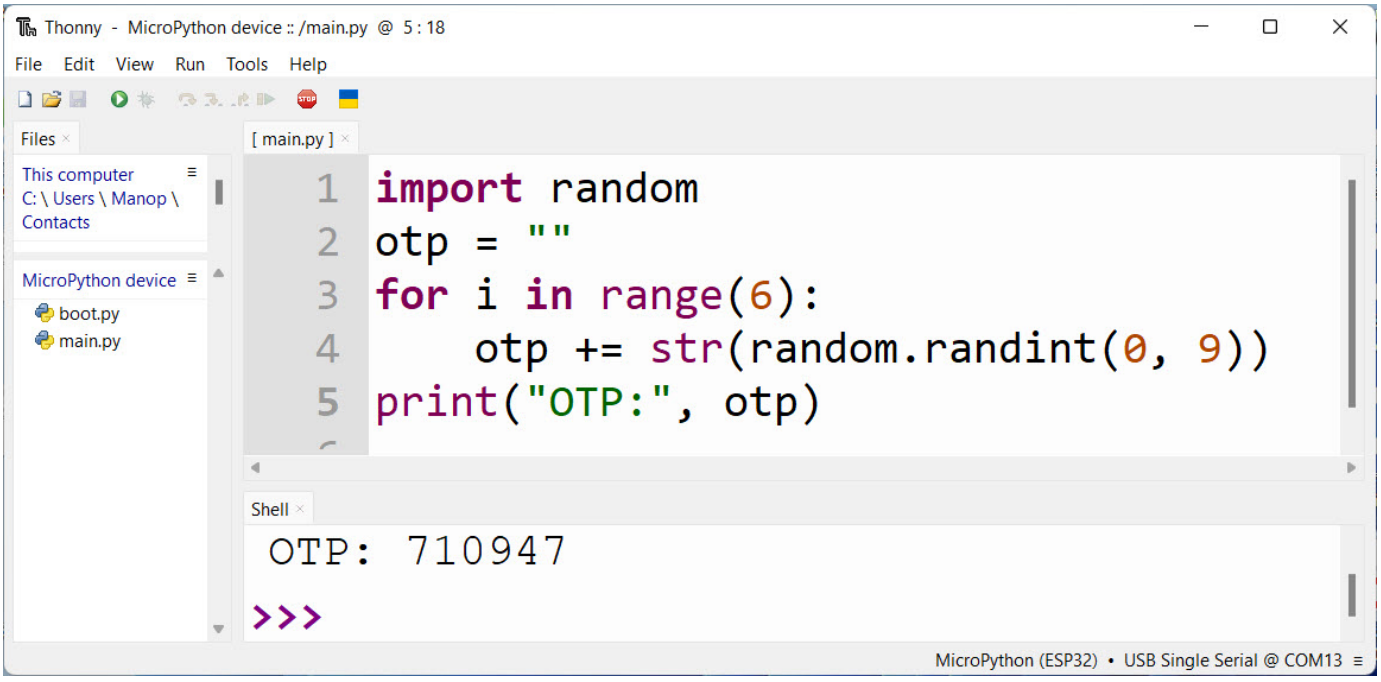
- random.randint(a, b) สุ่มจำนวนเต็มระหว่าง a ถึง b (รวมปลายทั้งสองด้าน)
- random.uniform(a, b) สุ่มเลขทศนิยมช่วง a ถึง b
- random.random() สุ่มทศนิยมช่วง 0.0 ถึง 1.0
- random.choice(seq) สุ่มหยิบ 1 สมาชิกจาก list หรือ tuple
- random.seed() ตั้งค่าจุดเริ่มต้นของชุดสุ่ม (เพื่อทดสอบซ้ำให้ได้ค่าชุดเดิม)

ตัวอย่างการใช้งาน random บน ESP32

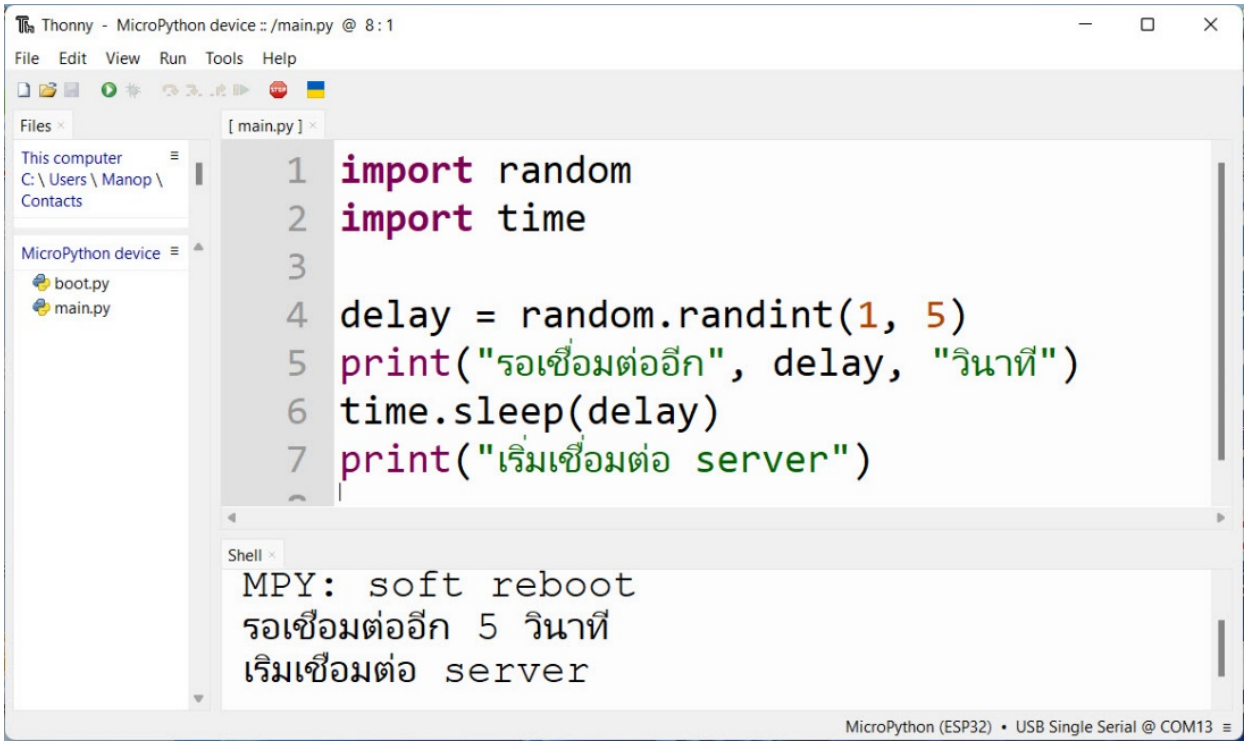
1. สุ่มข้อมูลเซ็นเซอร์จำลองระหว่างพัฒนา



2. สุ่ม OTP สำหรับระบบยืนยันตัวตน



3. สุ่มเวลา delay ป้องกันการชนสัญญาณ



4. สุ่มเลือกรายการ Wi-Fi กรณีมีหลาย hotspot

ข้อควรระวังการใช้ random บน ESP32

แม้ random ใน MicroPython จะเหมาะกับงานทั่วไป แต่มันยังเป็น “pseudo random” หรือการสุ่มที่อาศัยการคำนวณ ไม่ใช่ true random จากฮาร์ดแวร์ จึงไม่ควรนำไปใช้ในระบบความปลอดภัยระดับสูงเช่นงานเข้ารหัสจริง หากต้องการ random ที่ปลอดภัยควรใช้โมดูล hardware RNG เพิ่มเติม (ซึ่ง ESP32 มี RNG ในตัวด้วยสำหรับระดับระบบปฏิบัติการ) การฝึกใช้ random ตั้งแต่ต้นช่วยให้เขียนโค้ดที่ทดสอบง่าย กระจายโหลดดีขึ้น และสร้างระบบ IoT ที่ยืดหยุ่นพร้อมทำงานในสถานการณ์จริงได้อย่างมั่นคง

สรุปการใช้งานโมดูลสำคัญ เมื่อพัฒนาโปรแกรมควบคุม ESP32 ด้วย MicroPython การเข้าใจการใช้งานของแต่ละโมดูลสำคัญไม่ใช่เพียงแค่การจำชื่อฟังก์ชันหรือคำสั่ง แต่ต้องมองให้ออกว่าบทบาทของแต่ละโมดูลนั้นเข้ามาเกี่ยวข้องในขั้นตอนไหนของระบบและตอบโจทย์ปัญหาแบบไหนในโลกของงานฝังตัว การเชื่อมโยงหน้าที่ของแต่ละโมดูลกับสถานการณ์การใช้งานจริงจึงเป็นสิ่งสำคัญที่สุด เพราะระบบควบคุมฮาร์ดแวร์จริงย่อมไม่ได้มีเพียงคำสั่งเปิดไฟหรืออ่านค่าเซ็นเซอร์เท่านั้น แต่ยังเกี่ยวพันไปถึงการเชื่อมต่อเครือข่าย การจัดเก็บข้อมูล การตรวจสอบเวอร์ชันซอฟต์แวร์ การเตรียมข้อมูลสำหรับแลกเปลี่ยนกับระบบภายนอก รวมถึงการสร้างพฤติกรรมสุ่มเพื่อเพิ่มความยืดหยุ่นของระบบ

Machine

ทุกโปรแกรม MicroPython ที่ต้องควบคุมโลกจริงต้องเริ่มต้นที่ machine เสมอ โมดูลนี้เป็นเสมือนสะพานตรงเชื่อมระหว่างคำสั่งโปรแกรมกับขาฮาร์ดแวร์ทุกขาใน ESP32 ตั้งแต่การสั่งเปิดปิดไฟ LED บนขา 2 ด้วย machine.Pin, การอ่านค่าเซ็นเซอร์แรงดันด้วย machine.ADC, การสร้างสัญญาณ PWM ปรับความเร็วรอบมอเตอร์ด้วย machine.PWM, ไปจนถึงการสื่อสารกับจอ OLED หรือเซ็นเซอร์ความชื้นผ่าน bus I2C, SPI หรือ UART ทั้งหมดนี้ไม่มีทางทำได้เลยถ้าไม่มี machine เข้ามาควบคุมโดยตรง ความสามารถของ machine ยังครอบคลุมไปถึงการควบคุมระบบเวลาในตัวชิปผ่าน Timer รวมถึงการสั่งรีเซ็ตหรือปรับความถี่การทำงานของ CPU ได้ด้วย

time

ระบบควบคุมทั้งหมดไม่สามารถปล่อยให้คำสั่งรันต่อเนื่องแบบไม่มีจังหวะได้ จังหวะคือหัวใจของระบบควบคุมอัตโนมัติ time เข้ามาจัดการตรงนี้โดยตรง ไม่ว่าจะเป็นการหน่วงเวลาสั่งเปิดปิดอุปกรณ์ตามกำหนด การรอเวลาให้เซ็นเซอร์พร้อมอ่านค่า การวนลูปตรวจสอบสถานะทุก ๆ 1 วินาที หรือแม้แต่การวัดเวลาตอบสนองของระบบที่ต้องแม่นยำในระดับมิลลิวินาทีหรือไมโครวินาที time.sleep() ทำให้หน่วงเวลาได้แบบหยุดโปรแกรมทั้งระบบ ส่วน time.ticks_ms() และ time.ticks_us() ทำให้วัดเวลาได้อย่างต่อเนื่องแม้เวลาจะทบลูปแล้วก็ตาม จึงเหมาะกับระบบฝังตัวที่ต้องควบคุมจังหวะละเอียดในงานจริง

network

เมื่อก้าวเข้าสู่โลกของ IoT คำสั่งควบคุมไฟฟ้าธรรมดาไม่เพียงพออีกต่อไป network เข้ามามีบทบาทในการเชื่อมต่อ ESP32 เข้าสู่เครือข่ายไร้สาย ไม่ว่าจะเป็นการเชื่อมต่อ Wi-Fi แบบ station เพื่อส่งข้อมูลไปยัง server หรือ cloud หรือแม้แต่ตั้งตัวเองเป็น access point ให้มือถือหรืออุปกรณ์ภายนอกเข้ามาสั่งงาน network.WLAN() ทำให้กำหนดโหมด เชื่อมต่อ ตรวจสอบสถานะเครือข่าย และดึงหมายเลข IP Address ได้ง่าย ๆ ภายในไม่กี่บรรทัด ซึ่งกลายเป็นพื้นฐานของทุกโปรเจกต์ IoT ยุคใหม่

os

เมื่อเริ่มมีการเก็บข้อมูลในระบบฝังตัว ความสามารถในการจัดการไฟล์ก็จะถูกดึงเข้ามาใช้งานทันที os ช่วยให้ ESP32 ทำตัวเป็นระบบแฟ้มข้อมูลขนาดเล็กภายในตัวเองได้อย่างมีประสิทธิภาพ ทั้งการสร้างไฟล์บันทึก log รายวัน การลบไฟล์เก่าที่หมดอายุ การสำรองข้อมูลตั้งค่าแบบ config ไปจนถึงการเตรียมระบบอัปเดตเฟิร์มแวร์ OTA os.listdir(), os.remove(), os.rename() และ os.mkdir() คือคำสั่งสำคัญที่ทำให้ระบบฝังตัวสามารถดูแลข้อมูลภายใน flash ได้อย่างปลอดภัยและเป็นระบบ

sys

เมื่อระบบเติบโตมากขึ้น sys เข้ามาช่วยให้การควบคุมและตรวจสอบสภาพแวดล้อมของระบบมีความยืดหยุ่นมากขึ้น sys.platform ใช้ตรวจสอบว่าโค้ดกำลังรันบนบอร์ดประเภทใด sys.version บอกเวอร์ชันของ MicroPython ที่กำลังใช้งาน sys.path เปิดทางให้จัดการ path สำหรับการ import ไลบรารีภายนอก และ sys.exit() ช่วยหยุดโปรแกรมทันทีเมื่อจำเป็นต้องรีเซ็ตการทำงาน ระบบที่ต้องรองรับไลบรารีหลากหลายหรือทำงานข้าม platform จำเป็นต้องใช้ sys ควบคู่กับระบบ import เสมอ

ujson

เมื่อระบบ IoT ต้องรับส่งข้อมูลกับระบบ cloud หรือ server ภายนอก JSON กลายเป็นภาษากลางที่ทุกระบบเข้าใจร่วมกัน ujson จึงเป็นตัวกลางที่แปลงข้อมูลในระบบ MicroPython ให้อยู่ในรูปแบบที่ส่งออกไปยัง server ได้สะดวก ujson.dumps() แปลง dictionary ให้กลายเป็น JSON string ส่วน ujson.loads() ดึงข้อมูล JSON string ที่ได้รับกลับมาให้เป็น dictionary ที่โปรแกรมสามารถใช้งานต่อได้ทันที ระบบ IoT เกือบทุกประเภท ตั้งแต่การส่งข้อมูลเซ็นเซอร์ขึ้น cloud, ควบคุมอุปกรณ์จากมือถือ หรือแลกเปลี่ยนคำสั่งกับระบบ backend ล้วนต้องพึ่งพา ujson เป็นแกนหลักในการแลกเปลี่ยนข้อมูลเสมอ

Random

ในโลกของการควบคุมจริง ความสุ่มไม่ได้มีไว้แค่ใช้ในเกม แต่กลับกลายเป็นเครื่องมือสำคัญที่ช่วยกระจายโหลด ลดโอกาสชนสัญญาณ ลดปัญหา timing collision บนเครือข่าย IoT random.randint(), random.uniform(), random.choice() ถูกใช้ตั้งแต่การสร้างข้อมูลจำลองระหว่างทดสอบ การสุ่ม delay เชื่อมต่อ Wi-Fi หลายอุปกรณ์พร้อมกัน การสุ่มรหัส OTP ไปจนถึงการกระจายตารางเวลาส่งข้อมูล MQTT เมื่อระบบต้องขยายตัวจากการควบคุมเดี่ยว ๆ ไปสู่ระบบ IoT ที่มีอุปกรณ์จำนวนมาก สุ่มถูกที่ ถูกเวลา จึงช่วยเพิ่มเสถียรภาพระบบได้

ทุกโมดูลเหล่านี้คือชุดเครื่องมือจริงที่นักพัฒนา MicroPython บน ESP32 ทุกคนต้องฝึกใช้ให้ชำนาญ เพราะงานควบคุมอุปกรณ์ฝังตัวไม่ได้มีแค่สั่งงานขาไฟฟ้าเพียงอย่างเดียว แต่ต้องมองเห็นทั้งภาพของฮาร์ดแวร์ ระบบเวลา เครือข่าย การจัดเก็บข้อมูล ความเข้ากันได้ของซอฟต์แวร์ การแลกเปลี่ยนข้อมูลข้ามระบบ และการ

รักษาเสถียรภาพของระบบในโลก IoT จริงทั้งหมดพร้อมกัน การเข้าใจความสัมพันธ์ของโมดูลเหล่านี้ตั้งแต่ต้นจะทำให้สามารถออกแบบระบบควบคุมที่เติบโตไปสู่ระบบขนาดใหญ่ได้อย่างมั่นคงและเป็นมาตรฐานสากล

3.6.5 เทคนิคมืออาชีพ

เมื่อเข้าใจการใช้งานโมดูลหลักใน MicroPython บน ESP32 แล้ว ขั้นตอนถัดไปคือการเรียนรู้วิธีประยุกต์และเขียนโปรแกรมในระดับมืออาชีพ ซึ่งไม่ใช่แค่เพียงรู้จักคำสั่งแต่ละตัว แต่ต้องสามารถออกแบบระบบควบคุมที่มีความเสถียร ยืดหยุ่น และขยายตัวได้ง่ายในระบบ IoT จริง เทคนิคต่อไปนี้จะช่วยยกระดับการเขียนโค้ด MicroPython ให้ก้าวพ้นจากโค้ดทดลอง ไปสู่โค้ดระบบฝังตัวที่ใช้งานจริงในงานวิศวกรรมและระบบผลิตภัณฑ์

1. จัดระเบียบโค้ดด้วยโมดูลย่อยตั้งแต่เริ่มต้น

การเขียนฟังก์ชันทุกอย่างไว้ในไฟล์เดียวทำให้โค้ดเริ่มต้นดูง่าย แต่เมื่อระบบเติบโตจะกลายเป็นจุดอันตราย เพราะยากต่อการแก้ไข บำรุงรักษา หรือขยายความสามารถใหม่ การสร้างไฟล์โมดูลแยกย่อยสำหรับแต่ละหน้าที่ เช่น sensor_utils.py, relay_control.py, wifi_setup.py แล้วนำมานำเข้า (import) เฉพาะส่วนที่ใช้งาน จะทำให้โค้ดมีความเป็นระบบ ดูแลรักษาง่าย ย้ายโปรเจกต์ได้สะดวก และใช้ซ้ำได้ในอนาคต เทคนิคนี้ช่วยลดความซ้ำซ้อนของโค้ดได้มหาศาลในงานจริง

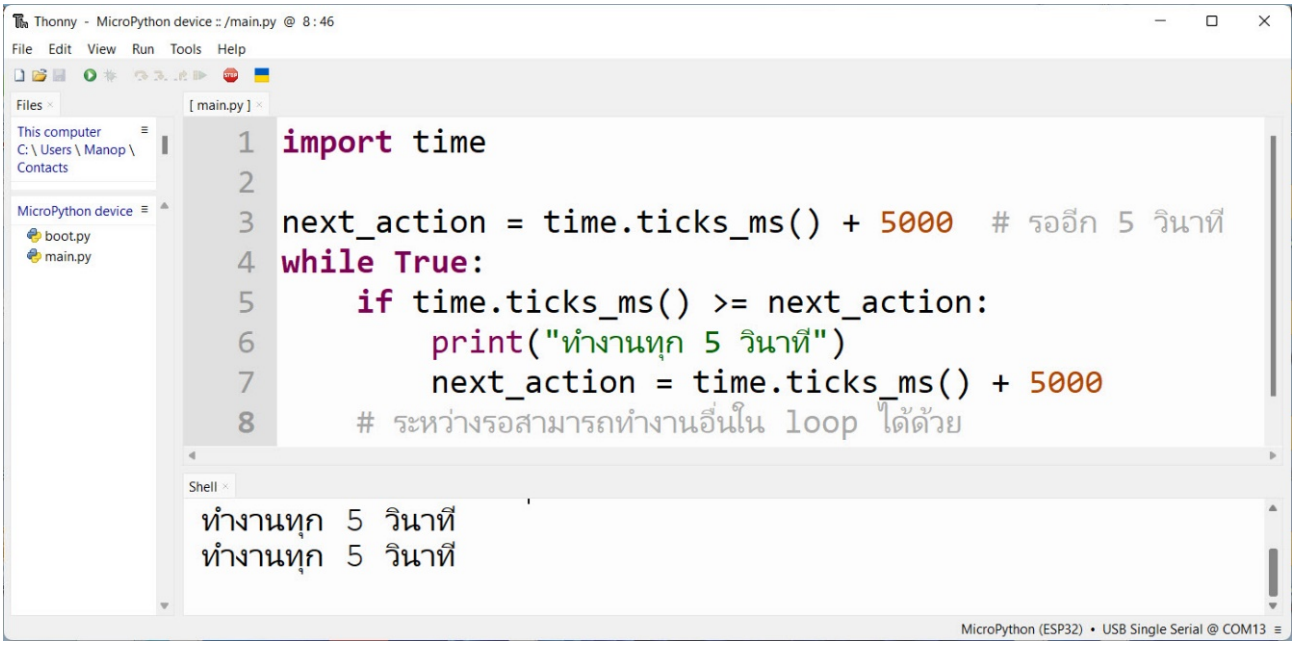
2. แยกฟังก์ชันควบคุมกับฟังก์ชันติดต่อสื่อสารออกจากกัน

ในงาน IoT ส่วนใหญ่มักมีทั้งส่วนที่ควบคุมฮาร์ดแวร์ (sensor, actuator) และส่วนที่รับส่งข้อมูล (Wi-Fi, MQTT, HTTP, Cloud) ถ้าเขียนโค้ดรวมสองอย่างนี้ไว้ปนกันจะทำให้ debug ยาก ควรเขียนฟังก์ชันควบคุมอุปกรณ์ให้ทำงานได้อิสระ แล้วให้ส่วนสื่อสารเป็นเพียงตัวส่งค่าที่อ่านได้ขึ้น cloud หรือ server เท่านั้น วิธีนี้ทำให้สามารถทดสอบระบบควบคุมพื้นฐานได้โดยไม่ต้องเชื่อมต่อเน็ต และสามารถแยกพัฒนาแต่ละส่วนได้อย่างยืดหยุ่น

3. ใช้ระบบจับเวลาแบบไม่หยุดโปรแกรม (Non-blocking Timing)

การใช้ time.sleep() ในการหน่วงเวลาทำให้โค้ดหยุดทั้งระบบขณะรอ ซึ่งในระบบฝังตัวจริงมักมีอุปกรณ์หลายตัวทำงานพร้อมกัน เทคนิคมืออาชีพคือการใช้ time.ticks_ms() หรือ time.ticks_us() เพื่อจับเวลาล่วงหน้า แล้วเช็คเวลาปัจจุบันทุกครั้งใน loop แทนการ sleep ทั้งระบบ เช่น:

(ดูภาพประกอบหน้าถัดไป) ⏩



เทคนิคนี้ช่วยให้ ESP32 สามารถควบคุมหลายอุปกรณ์พร้อมกันใน loop เดียวโดยไม่หยุดรอการทำงานใด ๆ

4. ตรวจสอบสถานะการเชื่อมต่อเครือข่ายอย่างต่อเนื่อง

ในงาน IoT จริงการเชื่อมต่อ Wi-Fi ไม่ได้เสถียรเหมือนในห้องแลบ อาจเกิดหลุดสัญญาณเป็นระยะ ESP32 ควรตรวจสอบสถานะ Wi-Fi เป็นระยะ ๆ ด้วย network.WLAN().isconnected() และหากหลุดควร reconnect อัตโนมัติ โดยไม่ต้องรีเซ็ตบอร์ดทั้งตัวเสมอไป

5. จัดเก็บ log หรือ config ไว้ในแฟ้มอย่างมีระบบ

ทุกระบบจริงควรมีระบบ log เพื่อตรวจสอบเหตุการณ์ย้อนหลัง ควรใช้ os บันทึกข้อมูลทุกครั้งที่เกิดเหตุการณ์สำคัญ เช่น สถานะเซ็นเซอร์ผิดปกติ Wi-Fi หลุด ระบบรีเซ็ต เป็นต้น เช่น:

นอกจากนี้ไฟล์ config ก็ควรจัดเก็บในรูปแบบ JSON เพื่อให้ง่ายต่อการอ่าน-เขียนค่าตั้งต้นที่อัปเดตผ่านระบบ OTA ได้ในอนาคต

6. ใช้ random กระจายโหลดระบบ IoT

ในกรณีที่มี ESP32 หลายตัวเชื่อมต่อ broker MQTT หรือ cloud server พร้อมกัน ควรใช้ random.randint() สุ่มเวลาหน่วงก่อนเชื่อมต่อหรือส่งข้อมูลแต่ละครั้ง เพื่อป้องกันการโหลดระบบพร้อมกันแบบ burst ที่อาจทำให้ server ล่มหรือรับข้อมูลไม่ทัน

7. ระวังการใช้ memory และ resource ของบอร์ด

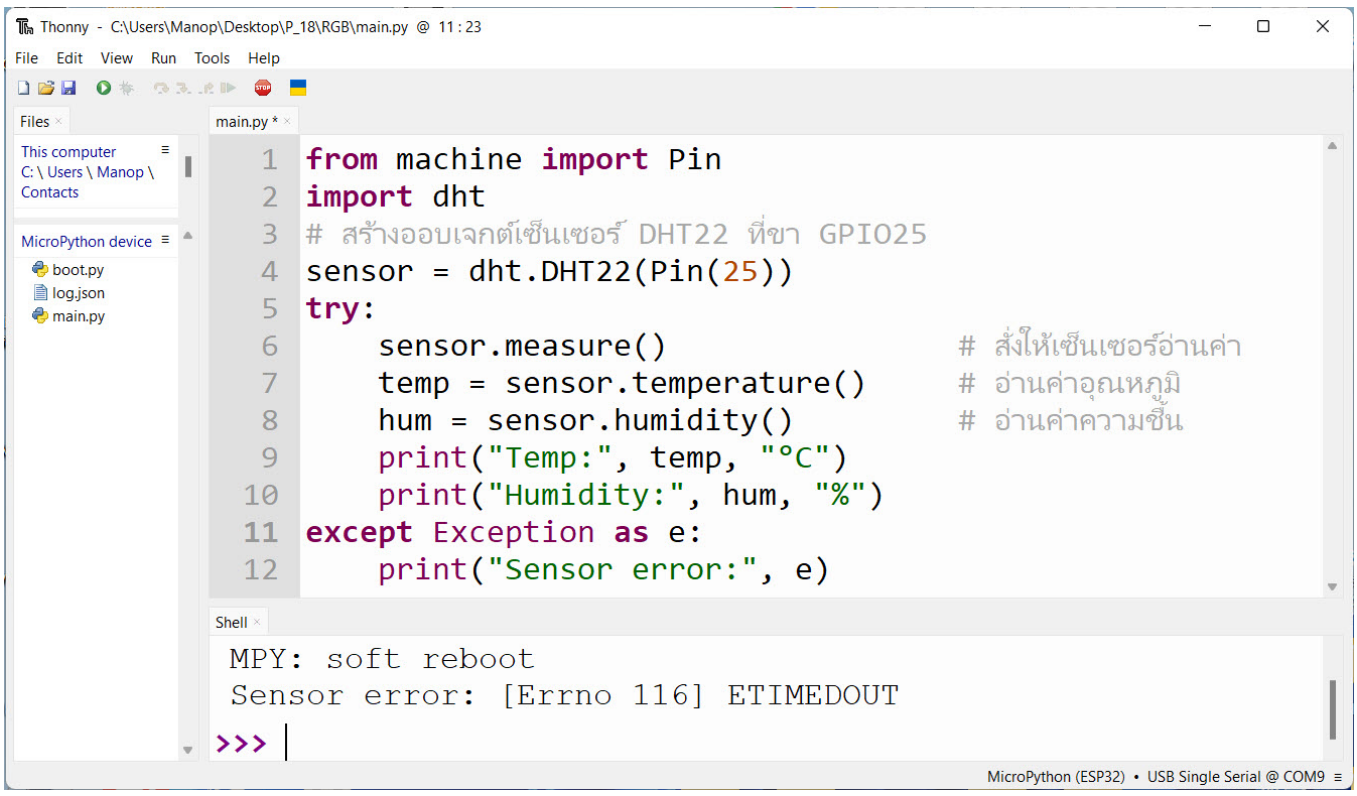
MicroPython บน ESP32 แม้จะมีแรมมากกว่าไมโครคอนโทรลเลอร์ยุคเก่า แต่ก็ยังมีข้อจำกัดเมื่อรันโค้ดขนาดใหญ่ หรือเมื่อเก็บ log ต่อเนื่องนานหลายชั่วโมง ควรออกแบบโค้ดให้ปล่อย resource คืนเสมอเมื่อไม่ใช้งาน เช่น ปิด connection เมื่อเสร็จ, ปิดไฟล์หลังอ่าน-เขียนเสร็จ, เคลียร์ list หรือ dictionary ขนาดใหญ่ที่ไม่ได้ใช้แล้ว รวมถึงออกแบบระบบ log ให้มีการวนทับไฟล์เก่าแบบ cyclic log เพื่อลดปัญหา flash เต็ม

8. คิดเผื่อการ OTA (Over-the-Air Update) ตั้งแต่แรก

เมื่อระบบเติบโต อุปกรณ์จำนวนมากจะไม่สามารถเสียบสายเพื่ออัปเดตเฟิร์มแวร์ได้เสมอ OTA จึงกลายเป็นเทคนิคสำคัญระดับอุตสาหกรรม การแยกโค้ดเป็นโมดูลเล็ก ๆ, มีไฟล์ config แยกต่างหาก, และใช้ระบบ log บันทึกสถานะ จะช่วยให้การออกแบบระบบ OTA ในอนาคตง่ายกว่าการเขียนโค้ดรวมทุกอย่างปนกันตั้งแต่แรก

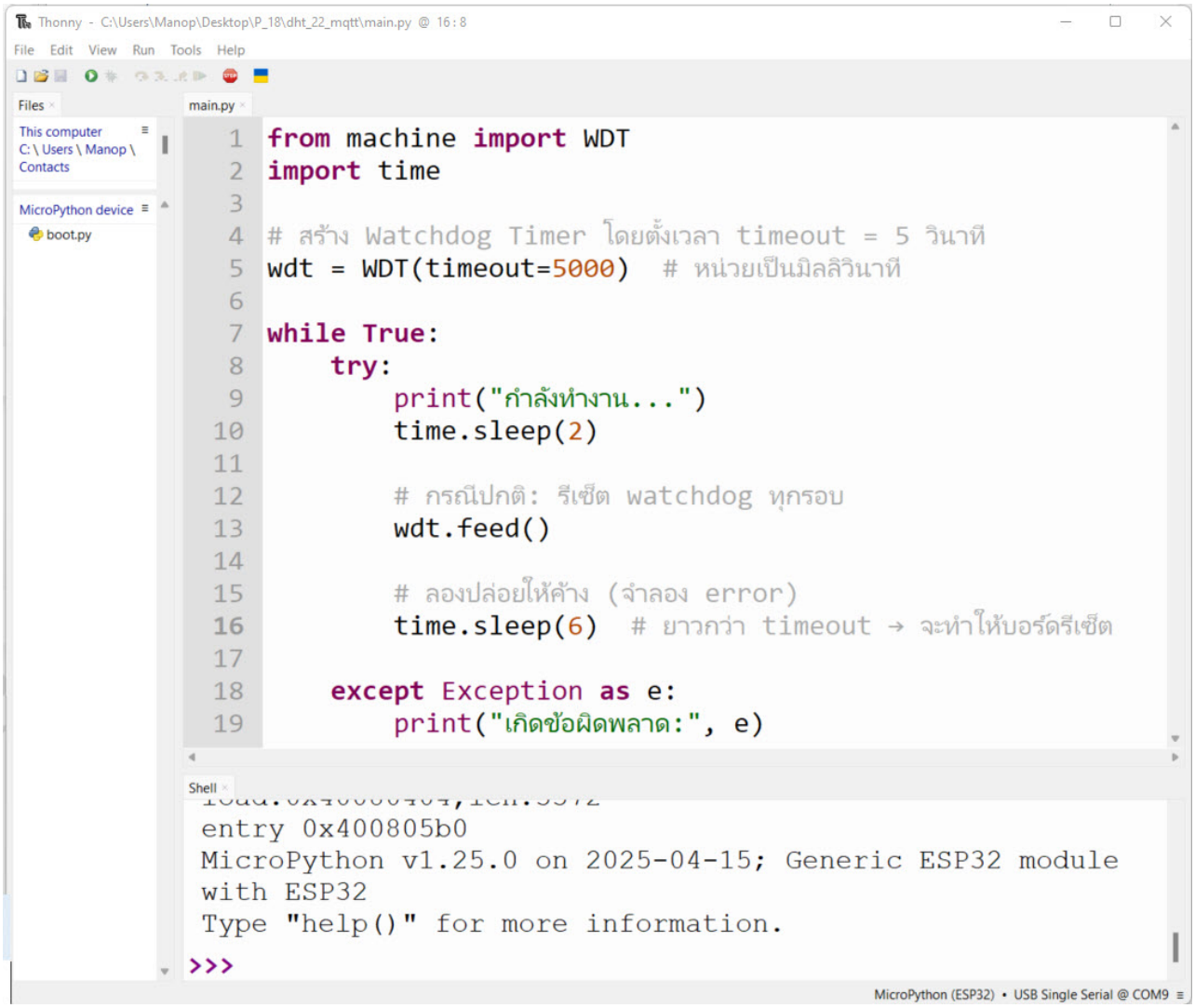
9. ใช้ try-except ครอบระบบป้องกันบอร์ด crash

โค้ดที่นำไปใช้งานจริงนั้นมีโอกาสเกิด error ที่ไม่คาดคิดเสมอ เช่น Wi-Fi หลุด, MQTT ล่ม, Sensor อ่านค่าไม่ได้ หากไม่มีการดัก error ไว้ อาจทำให้โปรแกรมหยุดทั้งระบบโดยไม่สามารถกลับมาทำงานได้เอง ควรใช้ try-except ครอบการเชื่อมต่อหรือการอ่านค่าจากอุปกรณ์ทุกตัว เช่น:



10. ทำ Watchdog ตรวจสอบโปรแกรมค้าง

MicroPython บน ESP32 รองรับ hardware watchdog ในการตรวจสอบว่าระบบยังทำงานอยู่หรือไม่ หากเกิดการค้างหรือ hang โปรแกรม Watchdog จะสั่ง reset บอร์ดอัตโนมัติ ช่วยให้ระบบทำงานต่อเนื่องแบบอัตโนมัติในงานจริงได้



การเขียน MicroPython บน ESP32 ในระดับมืออาชีพต้องไม่หยุดแค่การรู้คำสั่ง แต่ต้องคำนึงถึงโครงสร้างโค้ด ความยืดหยุ่น การตรวจสอบปัญหา การจัดการ resource และการเตรียมระบบให้พร้อมสำหรับงานภาคสนามจริง การฝึกใช้เทคนิคเหล่านี้จะทำให้โค้ด MicroPython จากเพียงโค้ดทดลอง กลายเป็นระบบ IoT ฝังตัวระดับอุตสาหกรรมได้อย่างสมบูรณ์

บทที่ 4: สั่งงานขา GPIO: จากไฟกะพริบสู่การอ่านค่าเซ็นเซอร์

หลังจากที่ได้ทบทวนพื้นฐานภาษา Python ที่จำเป็นสำหรับการเขียนโปรแกรม MicroPython ในบทที่ 3 ไปแล้ว ในบทนี้ เราจะนำความรู้เหล่านั้นมาประยุกต์ใช้กับการควบคุมฮาร์ดแวร์ที่สำคัญที่สุดส่วนหนึ่งของไมโครคอนโทรลเลอร์ นั่นคือ ขา **GPIO (General Purpose Input/Output)** ขาเหล่านี้เปรียบเสมือนแขนขา

ของ ESP32 ที่ช่วยให้สามารถรับรู้สภาวะจากโลกภายนอก (Input) และส่งคำสั่งหรือสัญญาณออกไปควบคุมอุปกรณ์ต่างๆ (Output) ได้

บทนี้จะเริ่มต้นด้วยการทำความเข้าใจแนวคิดพื้นฐานของ GPIO, การใช้งานโมดูล machine.Pin ใน MicroPython, การควบคุมอุปกรณ์ Output อย่างง่ายเช่น LED, การอ่านค่าจากอุปกรณ์ Input เช่น สวิตช์หรือปุ่มกด, และปิดท้ายด้วยการเรียนรู้เกี่ยวกับ PWM (Pulse Width Modulation) เพื่อควบคุมความสว่างของ LED หรือการทำงานของ Servo motor เบื้องต้น

4.1 บทนำสู่ GPIO (Introduction to GPIO)

GPIO คืออะไร? GPIO ย่อมาจาก General Purpose Input/Output หรือแปลเป็นไทยได้ว่า "ขาอินพุต/เอาต์พุตอเนกประสงค์" เป็นขาบนตัวชิปไมโครคอนโทรลเลอร์ที่สามารถกำหนดค่าให้ทำงานได้สองโหมดหลักๆ คือ:

1. **โหมดอินพุต (Input Mode):** ใช้สำหรับอ่านค่าสัญญาณดิจิทัล (Digital Signal) จากอุปกรณ์ภายนอก เช่น สวิตช์, ปุ่มกด, หรือเซ็นเซอร์บางชนิดที่ให้ผลลัพธ์เป็นสัญญาณดิจิทัล (มีหรือไม่มีสัญญาณ, เปิดหรือปิด)
2. **โหมดเอาต์พุต (Output Mode):** ใช้สำหรับส่งสัญญาณดิจิทัลออกไปควบคุมอุปกรณ์ภายนอก เช่น สั่งให้ LED ติดหรือดับ, ควบคุมการทำงานของรีเลย์, หรือส่งสัญญาณไปยังไมโครคอนโทรลเลอร์ตัวอื่น

ระดับแรงดันไฟฟ้า (Logic Levels) ใน ESP32 ไมโครคอนโทรลเลอร์ ESP32 ทำงานด้วยระดับแรงดันไฟฟ้า 3.3 โวลต์ (3.3V) ซึ่งหมายความว่า:

- **สัญญาณดิจิทัลลอจิก "สูง" (Logic HIGH หรือ 1):** จะแทนด้วยแรงดันไฟฟ้าประมาณ 3.3V
- **สัญญาณดิจิทัลลอจิก "ต่ำ" (Logic LOW หรือ 0):** จะแทนด้วยแรงดันไฟฟ้าประมาณ 0V (หรือ Ground)

การทำความเข้าใจเรื่องระดับแรงดันไฟฟ้านี้สำคัญมากเมื่อต้องต่อ ESP32 เข้ากับอุปกรณ์หรือเซ็นเซอร์อื่นๆ เพื่อให้แน่ใจว่าระดับแรงดันไฟฟ้าเข้ากันได้และไม่ทำให้เกิดความเสียหาย

ข้อควรระวังในการใช้งาน GPIO:

- **กระแสไฟฟ้า (Current):** ขา GPIO แต่ละขาสามารถจ่ายหรือรับกระแสไฟฟ้าได้ในปริมาณที่จำกัด (โดยทั่วไปสำหรับ ESP32 อยู่ที่ประมาณ 12mA ถึง 40mA ต่อขา ขึ้นอยู่กับการกำหนดค่าและสภาวะ) หากต่ออุปกรณ์ที่กินกระแสไฟฟ้ามากกว่าที่ขาสามารถจ่ายได้ อาจทำให้ขา GPIO หรือตัวชิป ESP32 เสียหายได้ ในกรณีที่ต้องการควบคุมอุปกรณ์ที่กินกระแสสูง เช่น มอเตอร์ขนาดใหญ่ หรือแถบ LED

จำนวนมาก จำเป็นต้องใช้วงจรขับ (Driver Circuit) เช่น ทรานซิสเตอร์, MOSFET, หรือโมดูลรีเลย์ มาช่วย

- **แรงดันไฟฟ้าเกิน (Overvoltage):** ห้ามจ่ายแรงดันไฟฟ้าที่สูงกว่า 3.3V (บางตำราอาจระบุว่าไม่ควรเกิน 3.6V) เข้าไปยังขา GPIO ของ ESP32 โดยตรง เพราะอาจทำให้ชิปเสียหายถาวรได้ หากต้องการเชื่อมต่อกับอุปกรณ์ที่ทำงานด้วยระดับแรงดันไฟฟ้า 5V จำเป็นต้องใช้วงจรแปลงระดับแรงดันไฟฟ้า (Logic Level Converter)
- **การลัดวงจร (Short Circuit):** ตรวจสอบการต่อวงจรให้ถูกต้องเสมอ การต่อขา Output ที่มีสถานะเป็น HIGH (3.3V) เข้ากับ Ground โดยตรง หรือต่อขา Output สองขาที่สถานะต่างกันเข้าด้วยกัน อาจทำให้เกิดการลัดวงจรและสร้างความเสียหายได้

4.2 การใช้งานโมดูล machine.Pin อย่างมืออาชีพ

การควบคุมขา GPIO ใน MicroPython จำเป็นต้องใช้ **คลาส Pin** ซึ่งอยู่ภายใต้ **โมดูล machine** โดย Pin เป็นออบเจกต์ที่ใช้แทนขาต่าง ๆ บนบอร์ด ESP32 และสามารถกำหนดได้ว่าจะใช้เป็น **Input** หรือ **Output** ซึ่งในส่วนนี้เราจะเรียนรู้วิธีใช้งานในมุมต่าง ๆ อย่างละเอียด รวมถึงการป้องกันข้อผิดพลาดที่พบบ่อย

การนำเข้าโมดูล machine และการกำหนด GPIO เป็น Output

ก่อนใช้งานขา GPIO ต้อง **import** โมดูล machine เข้ามาก่อน แล้วจึงสร้างออบเจกต์ Pin เพื่อใช้งานขานั้น ๆ จากนั้นจึงกำหนดให้ขาใดขาหนึ่งเป็นขาเอาต์พุต เช่น GPIO2:

ตัวอย่างที่ใช้งานได้จริง: สร้างวงจรไฟกระพริบ (LED Blinking)

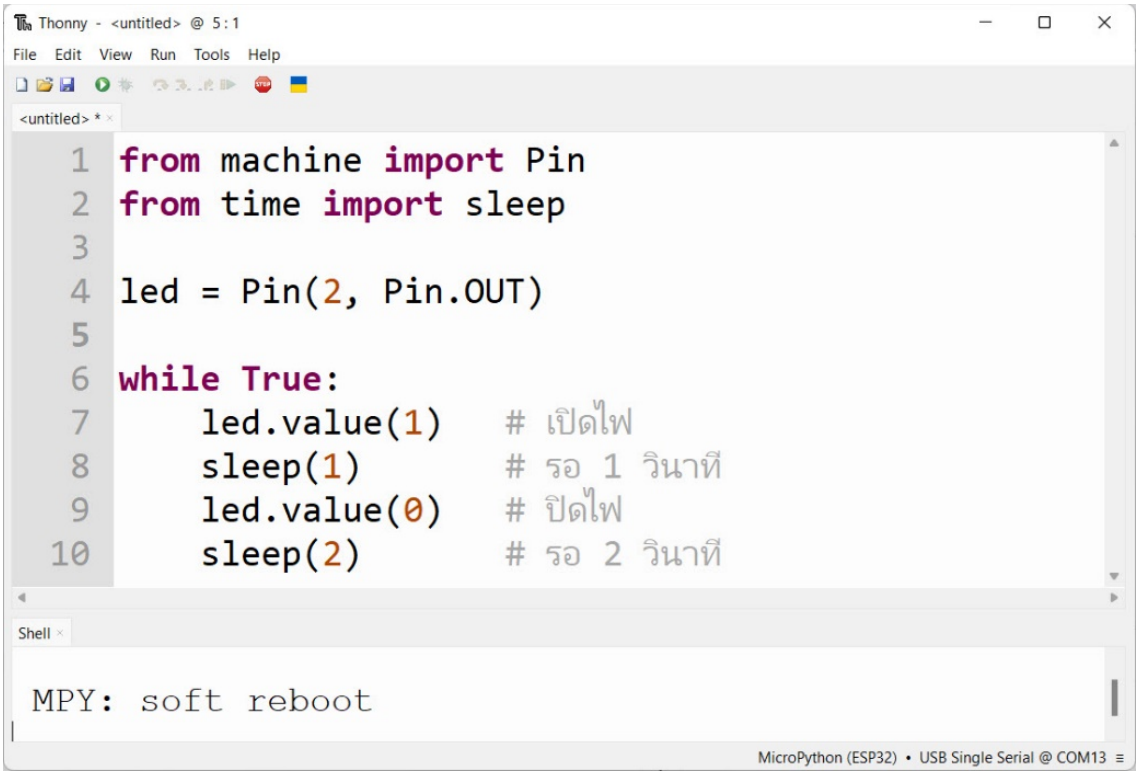
นี่คือรูปแบบที่ควรใช้สำหรับทดสอบ GPIO เบื้องต้น:

โปรแกรมนี้จะทำให้ไฟ LED ติดและดับสลับกันทุก 0.5 วินาที ซึ่งมองเห็นได้อย่างชัดเจนด้วยตาเปล่า

อธิบายคำสั่งอย่างละเอียด

คำสั่ง	ความหมาย
Pin(pin_number, mode)	สร้างออบเจกต์ขา GPIO โดยระบุหมายเลขขาและโหมดการทำงาน
Pin.OUT	ตั้งให้ขาทำหน้าที่เป็นขาเอาต์พุต (Output)
Pin.IN	ตั้งให้ขาทำหน้าที่เป็นขาอินพุต (Input)
led.on()	ส่งค่า HIGH (3.3V) ออกไปที่ขานั้น
led.off()	ส่งค่า LOW (0V) ออกไปที่ขานั้น
led.value()	อ่านค่าสถานะปัจจุบันของขานั้น (0 หรือ 1)
led.value(1)	สั่งให้ขานั้นส่งค่า HIGH
led.value(0)	สั่งให้ขานั้นส่งค่า LOW

ตัวอย่าง: เปิด LED 1 วินาที แล้วปิด 2 วินาที



สรุปประเด็นสำคัญที่ควรรู้

- ขา GPIO ต้องกำหนดหน้าที่การทำงาน (Input/Output) ให้ตรงกับการใช้งาน
- การใช้งาน GPIO อย่างมืออาชีพเริ่มต้นที่การควบคุม LED แต่จะนำไปสู่การควบคุมรีเลย์, มอเตอร์, หรือแม้แต่สั่งงานหุ่นยนต์ในขั้นต่อไป

• ห้ามจ่ายแรงดันไฟฟ้าเข้าที่ขา GPIO เกินกว่าระดับที่ชิป ESP32 รองรับ (โดยทั่วไปไม่เกิน 3.3V) และห้ามต่อโหลดที่ดึงกระแสมากเกินกว่าความสามารถในการจ่ายกระแสของขา GPIO ซึ่งโดยทั่วไปรองรับกระแสได้ประมาณ 12 mA ต่อขา และไม่ควรเกิน 40 mA เด็ดขาด หากฝืนต่อโหลดที่กินกระแสมาก เช่น รีเลย์, มอเตอร์, หรือ LED ความสว่างสูง โดยไม่ใช้วงจรขยาย เช่น ทรานซิสเตอร์หรือไดรเวอร์ จะทำให้ชิปเสียหายได้

การสร้างออบเจกต์ Pin: การสร้างออบเจกต์ Pin ทำได้โดยการเรียก Constructor ของคลาส Pin พร้อมระบุพารามิเตอร์ที่จำเป็น:

pin_obj = machine.Pin(pin_number, mode, pull=None, value=None)

- **pin_number**: หมายเลขขา GPIO ที่ต้องการใช้งาน ตัวเลขนี้จะอ้างอิงตามหมายเลขที่พิมพ์กำกับไว้บนบอร์ด ESP32 (เช่น 2, 4, 12, 13, GPIO2, GPIO4 เป็นต้น) ควรตรวจสอบ Pinout Diagram ของบอร์ดที่ใช้งานเพื่อความถูกต้อง
- **mode**: กำหนดโหมดการทำงานของขา GPIO มีค่าหลักๆ คือ:
    - machine.Pin.OUT: กำหนดเป็นขาเอาต์พุต
    - machine.Pin.IN: กำหนดเป็นขาอินพุต
    - machine.Pin.OPEN_DRAIN: (โหมดพิเศษ จะไม่กล่าวถึงในระดับเบื้องต้นนี้)
- **pull** (สำหรับโหมดอินพุตเป็นหลัก): กำหนดการใช้งานตัวต้านทาน Pull-up หรือ Pull-down ภายในของ ESP32 มีค่าที่เป็นไปได้คือ:
    - machine.Pin.PULL_UP: เปิดใช้งานตัวต้านทาน Pull-up ภายใน (ดึงสัญญาณให้เป็น HIGH เมื่อไม่มีสัญญาณภายนอก)
    - machine.Pin.PULL_DOWN: เปิดใช้งานตัวต้านทาน Pull-down ภายใน (ดึงสัญญาณให้เป็น LOW เมื่อไม่มีสัญญาณภายนอก)
    - None: ไม่ใช้งานตัวต้านทาน Pull-up/Pull-down ภายใน (ค่าปริยาย)
- **value** (สำหรับโหมดเอาต์พุต): กำหนดสถานะเริ่มต้นของขา Output เมื่อสร้างออบเจกต์ (0 หรือ 1)

ตัวอย่าง:

```
# กำหนด GPIO2 เป็นขา Output (สำหรับ LED ออนบอร์ด)
led = machine.Pin(2, machine.Pin.OUT)
```

```
# กำหนด GPIO24 เป็นขา Input พร้อมเปิดใช้งาน Pull-up ภายใน (สำหรับปุ่มกด)
button = machine.Pin(24, machine.Pin.IN, machine.Pin.PULL_UP)
```

4.3 Digital Output: การควบคุม LED และอุปกรณ์อื่นๆ

การควบคุม Digital Output คือการสั่งให้ขา GPIO มีสถานะเป็น HIGH (3.3V) หรือ LOW (0V) เพื่อควบคุมอุปกรณ์ต่างๆ อุปกรณ์ที่ง่ายที่สุดสำหรับการทดลองคือ LED (Light Emitting Diode)

4.3.1 การทำให้ LED ออนบอร์ดกะพริบ (ทบทวนและขยายความ)

ในบทที่ 2 เราได้ทดลองทำให้ LED ออนบอร์ด (มักจะอยู่ที่ GPIO2) กะพริบแล้ว โค้ดพื้นฐานมีดังนี้:

หากเป็นโมดูล ESP32 แบบ 38PIN จะไม่มี LED บนบอร์ดมาให้ ต้องต่อ LED กับ GPIO อื่น ๆ

```
import machine
import time

# สมมติ LED ออนบอร์ดอยู่ที่ GPIO2
onboard_led = machine.Pin(2, machine.Pin.OUT)

while True:
    onboard_led.value(1)  # หรือ onboard_led.on() - ทำให้ LED ติด
    time.sleep_ms(500)    # หน่วงเวลา 0.5 วินาที
    onboard_led.value(0)  # หรือ onboard_led.off() - ทำให้ LED ดับ
    time.sleep_ms(500)    # หน่วงเวลา 0.5 วินาที
```

- **pin_obj.value(x)**: ใช้สำหรับตั้งค่าหรืออ่านค่าสถานะของขา GPIO
    - เมื่อใช้กับขา Output: pin_obj.value(1) จะตั้งสถานะเป็น HIGH, pin_obj.value(0) จะตั้งสถานะเป็น LOW
    - เมื่อใช้กับขา Input: pin_obj.value() จะคืนค่าสถานะปัจจุบันของขา (1 หรือ 0)
- **pin_obj.on()**: เป็นคำสั่งย่อสำหรับตั้งค่าขา Output เป็นสถานะที่ทำให้ LED "ติด" (สำหรับ ESP32 ส่วนใหญ่คือการตั้งค่าเป็น LOW หาก LED ต่อแบบ Active Low หรือตั้งค่าเป็น HIGH หากต่อแบบ Active High)
- **pin_obj.off()**: เป็นคำสั่งย่อสำหรับตั้งค่าขา Output เป็นสถานะที่ทำให้ LED "ดับ"

หมายเหตุ: LED ออนบอร์ดของ ESP32 DevKitC ส่วนใหญ่มักจะต่อแบบ Active Low หมายความว่า LED จะสว่างเมื่อขา GPIO2 มีสถานะเป็น LOW (value(0) หรือ on()) และจะดับเมื่อมีสถานะเป็น HIGH (value(1) หรือ off()) หากโค้ดข้างต้นให้ผลตรงข้าม ให้สลับการทำงานของ value(1) และ value(0) หรือ on() และ off()

4.3.2 การต่อ LED ภายนอกและเขียนโปรแกรมควบคุม

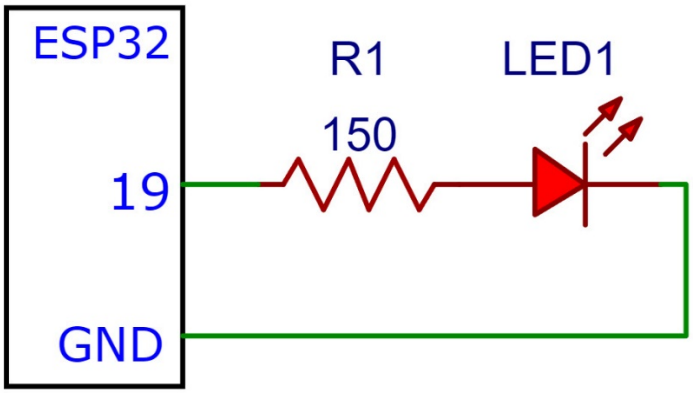
การทดลองกับ LED ภายนอกจะช่วยให้เข้าใจการทำงานของ GPIO ได้ชัดเจนยิ่งขึ้น

อุปกรณ์ที่ต้องใช้:

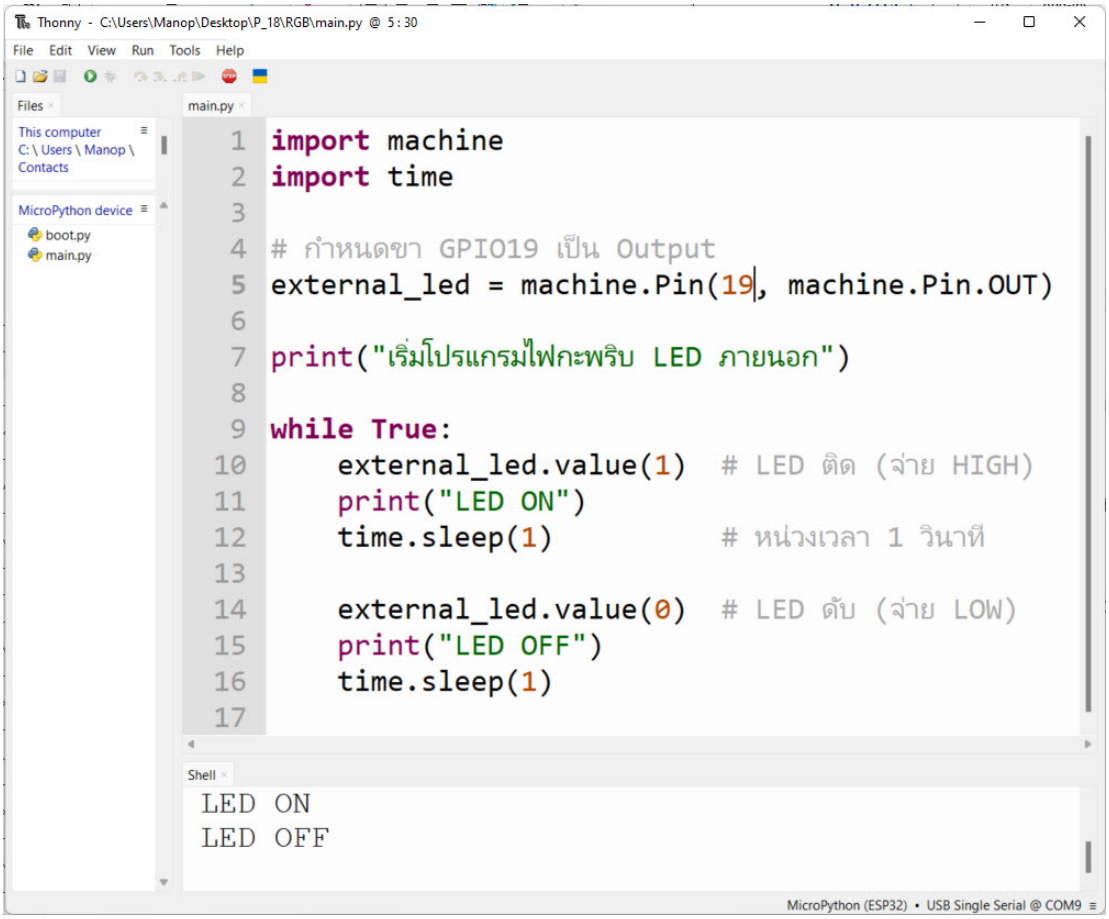
- LED 1 ดวง (สีใดก็ได้)
- ตัวต้านทาน (Resistor) ค่าประมาณ 150 โอห์ม (Ω) ถึง 330 Ω (เพื่อจำกัดกระแสไม่ให้ LED เสียหาย)
- Breadboard (แผงต่อวงจร)
- สายไฟเชื่อมต่อ (Jumper wires)

การต่อวงจร:

1. LED มีสองขา คือ ขาแอโนด (Anode - ขาบวก, มักจะเป็นขาที่ยาวกว่า) และขาแคโทด (Cathode - ขาลบ, มักจะเป็นขาที่สั้นกว่า หรือมีรอยบากที่ขอบตัวถัง)
2. ต่อขาแอโนด (ขาบวก) ของ LED เข้ากับขา GPIO ของ ESP32 ที่ต้องการ (เช่น GPIO19) **ผ่านตัวต้านทาน**
3. ต่อขาแคโทด (ขาลบ) ของ LED เข้ากับขา GND (Ground) ของ ESP32



โค้ดควบคุม LED ภายนอก (สมมติว่าต่อกับ GPIO19):



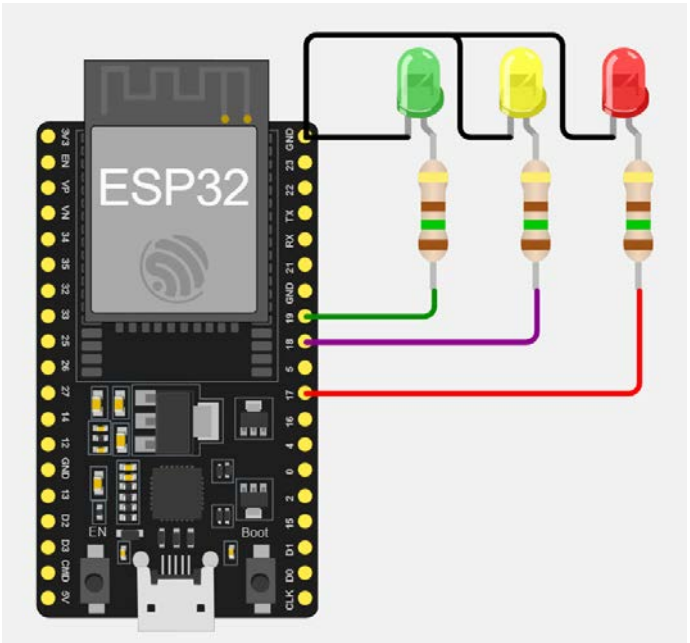
ในกรณีนี้ LED จะติดเมื่อ GPIO19 จ่ายสัญญาณ HIGH (3.3V) และดับเมื่อจ่าย LOW (0V)

4.3.3 ตัวอย่าง: ควบคุม LED 3 ดวงคล้ายไฟจราจรเบื้องต้น

เราสามารถใช้ LED หลายดวงเพื่อจำลองการทำงานของไฟจราจรได้

อุปกรณ์เพิ่มเติม:

- LED สีแดง 1 ดวง, สีเหลือง 1 ดวง, สีเขียว 1 ดวง
- ตัวต้านทาน 150-330 Ω จำนวน 3 ตัว

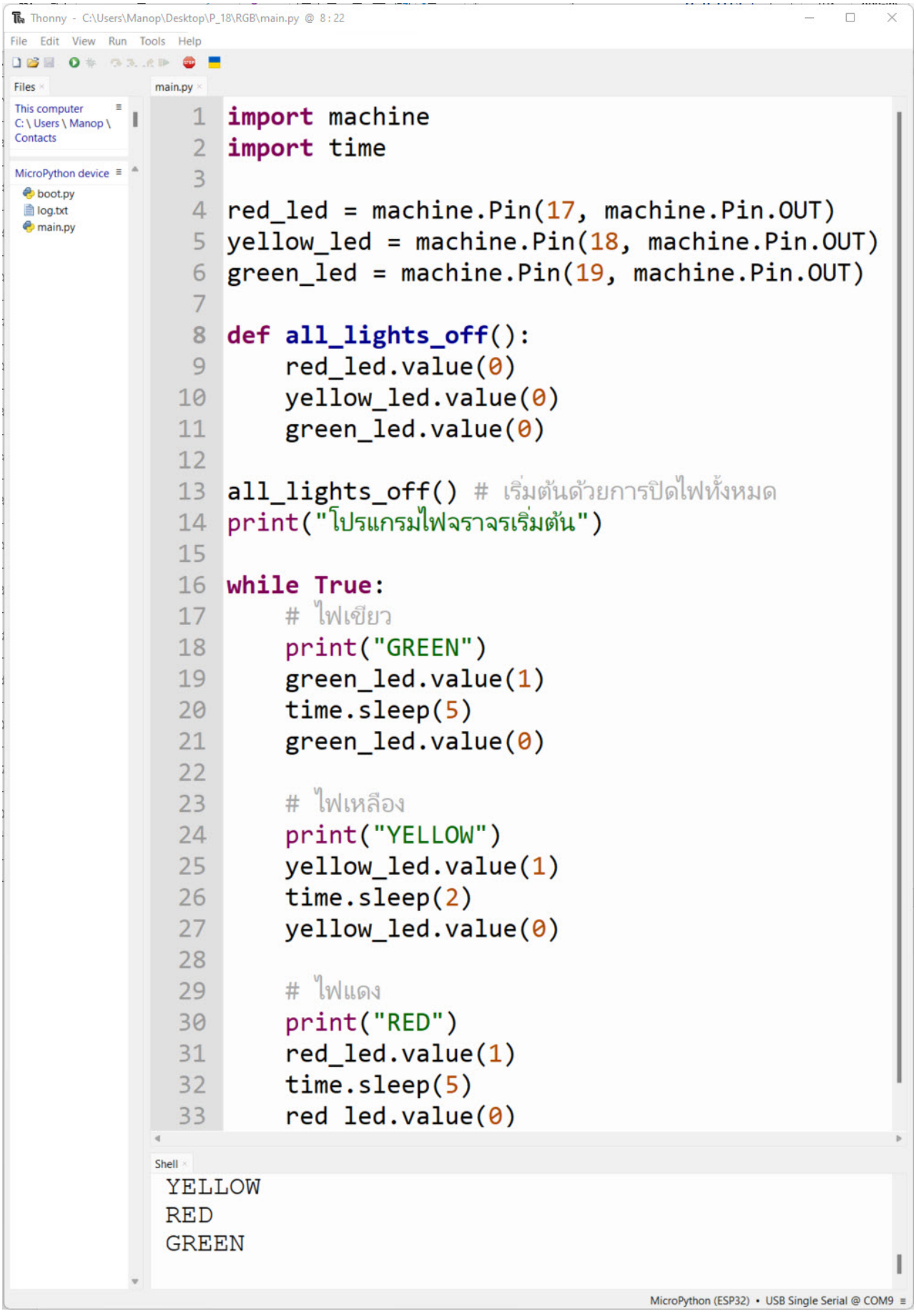


การต่อวงจร: ต่อ LED แต่ละดวง (พร้อมตัวต้านทาน)

เข้ากับขา GPIO ที่แตกต่างกัน เช่น:

- LED สีแดง -> GPIO17
- LED สีเหลือง -> GPIO18
- LED สีเขียว -> GPIO19
- (ขาแคโทดของ LED ทุกดวงต่อลง GND)

โค้ดสั่งงานให้ LED ติดและดับไปทีละดวง คล้าย ๆ ไฟจราจร



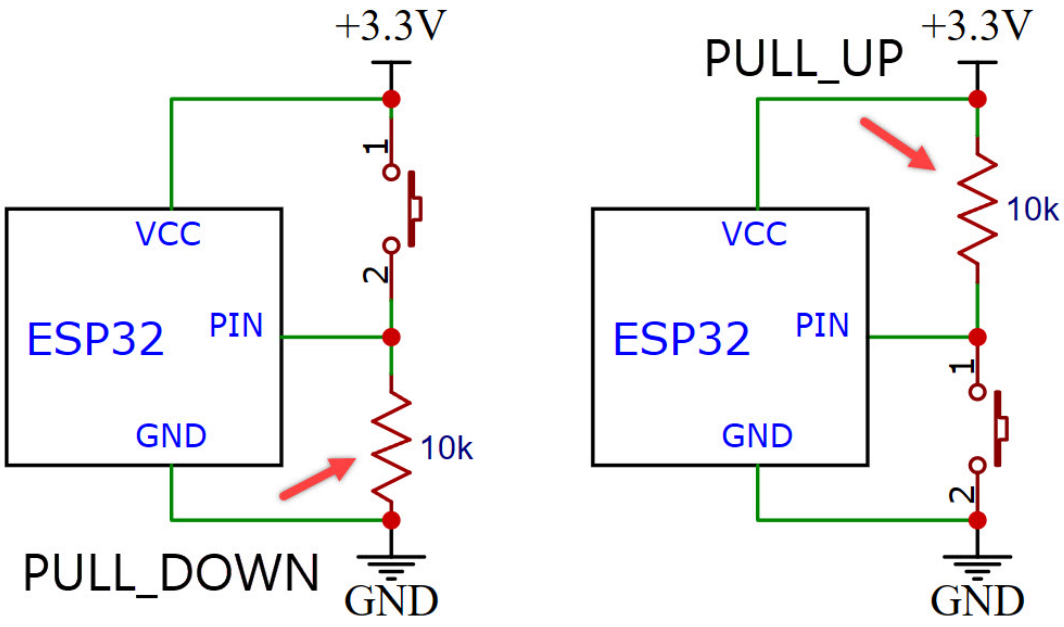
4.4 Digital Input: การอ่านค่าจากสวิตช์และปุ่มกด

การอ่านค่า Digital Input คือการตรวจสอบว่ามีสัญญาณ HIGH หรือ LOW เข้ามาที่ขา GPIO หรือไม่ ซึ่งมักใช้กับการอ่านสถานะของสวิตช์หรือปุ่มกด

4.4.1 การต่อสวิตช์ (Push button) กับ ESP32

ปุ่มกด (Push button) ทั่วไปจะมี 2 หรือ 4 ขา เมื่อกดปุ่ม ขาต่างๆ จะเชื่อมต่อกันทางไฟฟ้า เมื่อปล่อยปุ่มขาก็จะแยกออกจากกัน การต่อปุ่มกดเข้ากับขา Input ของไมโครคอนโทรลเลอร์จำเป็นต้องมี **ตัวต้านทาน Pull-up หรือ Pull-down** เพื่อให้ขา Input มีสถานะที่แน่นอนเมื่อยังไม่ได้กดปุ่ม (ป้องกันสภาวะ Floating ที่ขา Input ไม่รู้ว่าเป็น HIGH หรือ LOW)

- **Pull-up Resistor Configuration:**
  - ต่อตัวต้านทาน (เช่น 10kΩ) ระหว่างขา Input กับ VCC (3.3V)
  - ต่อสวิตช์ระหว่างขา Input กับ GND
  - เมื่อไม่ได้กดสวิตช์: ขา Input จะถูกดึงให้เป็น HIGH (อ่านค่าได้ 1)
  - เมื่อกดสวิตช์: ขา Input จะต่อลง GND ทำให้เป็น LOW (อ่านค่าได้ 0)
- **Pull-down Resistor Configuration:**
  - ต่อตัวต้านทาน (เช่น 10kΩ) ระหว่างขา Input กับ GND
  - ต่อสวิตช์ระหว่างขา Input กับ VCC (3.3V)
  - เมื่อไม่ได้กดสวิตช์: ขา Input จะถูกดึงให้เป็น LOW (อ่านค่าได้ 0)
  - เมื่อกดสวิตช์: ขา Input จะต่อเข้ากับ VCC ทำให้เป็น HIGH (อ่านค่าได้ 1)



การใช้ Pull-up/Pull-down ภายในของ ESP32: ESP32 มีความสามารถในการเปิดใช้งานตัวต้านทาน Pull-up หรือ Pull-down ภายในชิปสำหรับขา GPIO ที่กำหนดเป็น Input ได้ ทำให้ไม่จำเป็นต้องต่อตัวต้านทานภายนอก ซึ่งสะดวกมาก

ใช้ Pull-up ภายใน: ขา GPIO15 จะเป็น HIGH เมื่อไม่ได้กด และเป็น LOW เมื่อกด (ถ้าสวิตช์ต่อลง GND)

button_pull_up = machine.Pin(15, machine.Pin.IN, machine.Pin.PULL_UP)

ใช้ Pull-down ภายใน: ขา GPIO15 จะเป็น LOW เมื่อไม่ได้กด และเป็น HIGH เมื่อกด (ถ้าสวิตช์ต่อเข้า 3.3V)

 # button_pull_down = machine.Pin(15, machine.Pin.IN, machine.Pin.PULL_DOWN)

ในทางปฏิบัติ การใช้ Pull-up ภายในแล้วต่อสวิตช์ลง GND มักเป็นที่นิยมมากกว่า

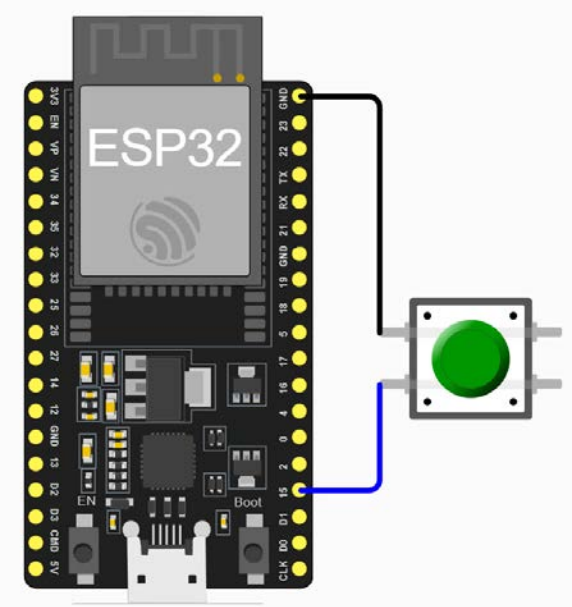
4.4.2 การอ่านค่าจากสวิตช์ด้วย pin_obj.value() เมื่อกำหนดขา GPIO เป็น Input แล้ว สามารถอ่านสถานะของขาได้ด้วยเมธอด value()

main.py *

```
import machine
import time

# สมมติใช้ GPIO15 เป็น Input พร้อม Pull-up ภายใน, สวิตช์ต่อระหว่าง GPIO15 กับ GND
button = machine.Pin(15, machine.Pin.IN, machine.Pin.PULL_UP)

while True:
    button_state = button.value() # อ่านสถานะของปุ่ม
    if button_state == 0: # ถ้าเป็น 0 แสดงว่าปุ่มถูกกด (เพราะใช้ Pull-up)
        print("ปุ่มถูกกด!")
    else:
        print("ปุ่มไม่ได้ถูกกด")
    time.sleep_ms(100) # หน่วงเวลาเล็กน้อยเพื่อลดการทำงานของ loop
```

4.4.3 ปัญหาการสั่นของสวิตช์ (Switch Bouncing) และแนวทางการแก้ไขเบื้องต้น

เมื่อกดหรือปล่อยสวิตช์ทางกลไก หน้าสัมผัสภายในสวิตช์อาจมีการกระทบกันหลายครั้งในช่วงเวลาสั้น ๆ (มิลลิวินาที) ก่อนจะสัมผัสกันสนิทหรือแยกจากกันสนิท ปรากฏการณ์นี้เรียกว่า "Switch Bouncing" หรือ "การสั่นของสวิตช์" ซึ่งทำให้ไมโครคอนโทรลเลอร์อาจอ่านค่าสถานะที่เปลี่ยนแปลงไปมาหลายครั้งจากการกดปุ่มเพียงครั้งเดียว



(ภาพประกอบจากอินเตอร์เน็ต)

แนวทางการแก้ไข (Software Debouncing แบบง่าย): วิธีที่ง่ายที่สุดคือการหน่วงเวลา (Delay) เล็กน้อยหลังจากตรวจพบการเปลี่ยนแปลงสถานะของปุ่มครั้งแรก และอ่านค่าอีกครั้งเพื่อยืนยัน หรือกล่าวได้ว่าหน่วงเวลาเพื่อทำให้การตรวจสอบการกดสวิตซ์นั้น ไม่สนใจการเปลี่ยนแปลงที่เกิดขึ้นในช่วงเวลานั้น

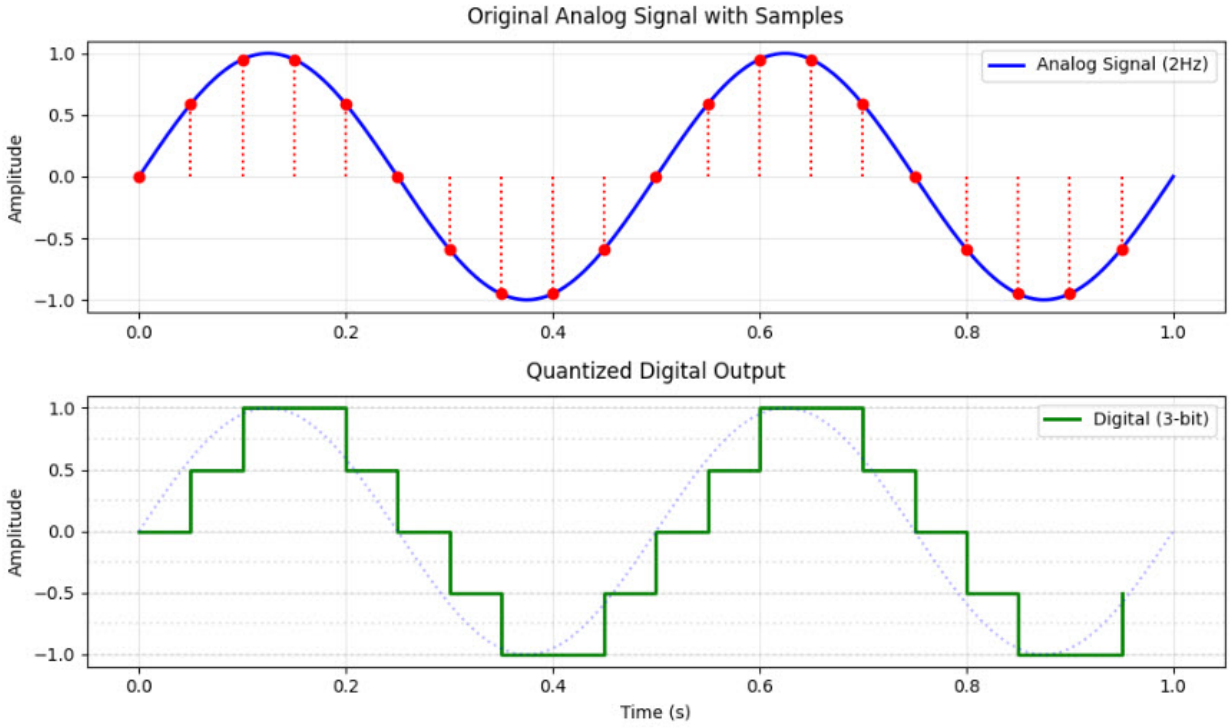
main.py *

```python
import machine
import time

button = machine.Pin(15, machine.Pin.IN, machine.Pin.PULL_UP)
led = machine.Pin(2, machine.Pin.OUT) # LED ออนบอร์ด

last_button_state = 1 # สถานะปุ่มก่อนหน้า (1 คือไม่ได้กด)
led_state = 0         # สถานะ LED (0 คือดับ)

print("โปรแกรมควบคุม LED ด้วยปุ่มกด (Debounced)")

while True:
    current_button_state = button.value()

    if current_button_state != last_button_state: # ตรวจสอบว่าสถานะปุ่มเปลี่ยนไปหรือไม่
        time.sleep_ms(20) # หน่วงเวลาสั้นๆ เพื่อรอให้การสั่นของสวิตช์หมดไป (Debounce delay)
        current_button_state = button.value() # อ่านค่าอีกครั้งหลังหน่วงเวลา

        if current_button_state == 0 and last_button_state == 1: # ตรวจสอบว่าเป็นขาลง (กดปุ่ม)
            print("ปุ่มถูกกด!")
            led_state = not led_state # สลับสถานะ LED
            led.value(led_state)
            print(f"LED State: {'ON' if led_state else 'OFF'}")

    last_button_state = current_button_state # อัปเดตสถานะปุ่มล่าสุด
    # time.sleep_ms(10) # อาจใส่ delay เล็กน้อยใน loop หลักเพื่อลดภาระ CPU
```

มีเทคนิค Software Debouncing ที่ซับซ้อนกว่านี้ แต่สำหรับผู้เริ่มต้น วิธีการหน่วงเวลาก็เพียงพอสำหรับการทดลองเบื้องต้น

4.5 Analog Input (ADC)

ESP32 มีความสามารถในการอ่านค่าสัญญาณแอนะล็อก (Analog Signal) ซึ่งเป็นสัญญาณที่มีค่าต่อเนื่อง ไม่ใช่แค่ HIGH หรือ LOW เหมือนสัญญาณดิจิทัล การอ่านค่าแอนะล็อกทำได้โดยใช้ ADC (Analog-to-Digital Converter) ที่มีอยู่ภายในชิป ESP32 ซึ่งมีประโยชน์มากสำหรับการอ่านค่าจากเซ็นเซอร์หลายชนิด เช่น ตัวต้านทานปรับค่าได้ (Potentiometer), เซ็นเซอร์วัดแสง (LDR), หรือเซ็นเซอร์อุณหภูมิบางรุ่น



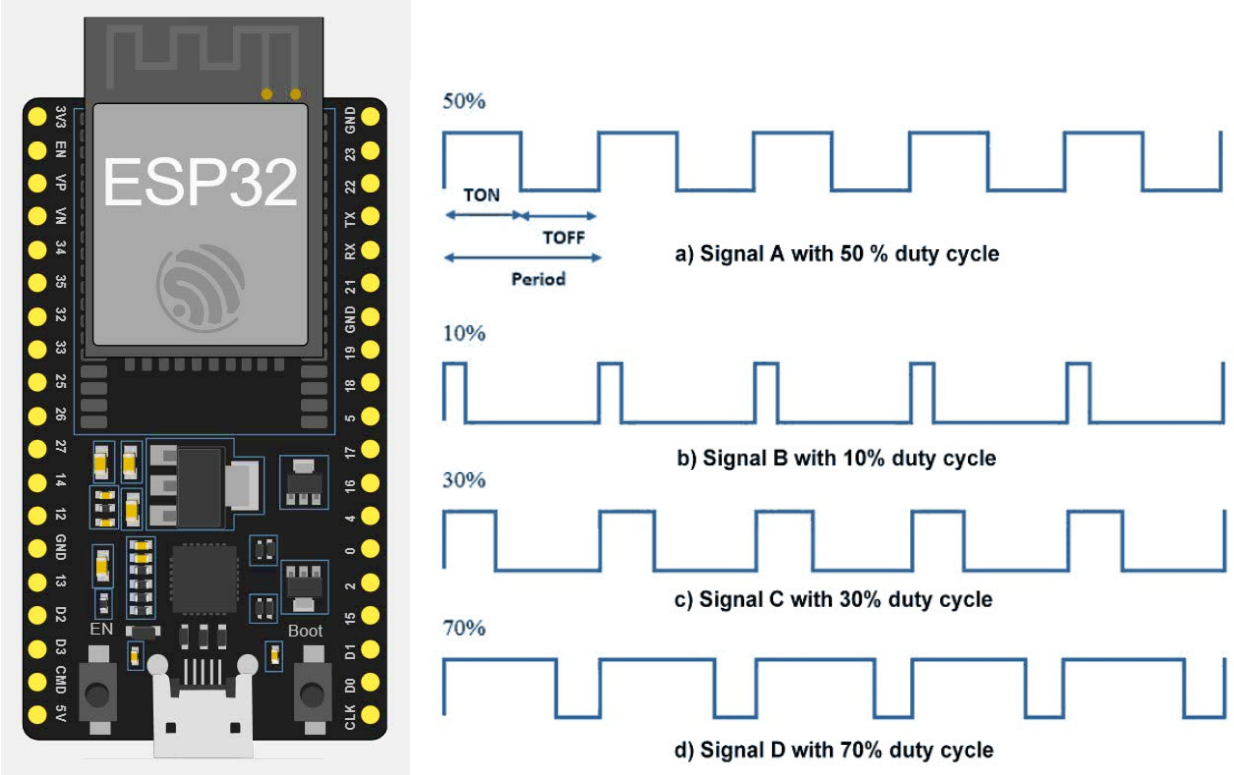
(ภาพตัวอย่างจากอินเตอร์เน็ต)

รายละเอียดเกี่ยวกับการใช้งาน ADC และการอ่านค่าจากเซ็นเซอร์แอนะล็อก จะอธิบายอย่างละเอียดในบทที่ 6 ซึ่งจะเน้นเรื่องการทำงานกับเซ็นเซอร์โดยเฉพาะ

4.6 PWM (Pulse Width Modulation) การควบคุมความสว่างของ LED และการขับ Motor เบื้องต้น

PWM (Pulse Width Modulation) คืออะไร

PWM (Pulse Width Modulation) คือเทคนิคการสร้างสัญญาณดิจิทัลแบบคลื่นสี่เหลี่ยมที่สามารถปรับสัดส่วนเวลาของสถานะ HIGH (เปิด) และ LOW (ปิด) ภายในหนึ่งคาบเวลาได้ตามต้องการ สัดส่วนนี้เรียกว่า **Duty Cycle** ซึ่งมีผลต่อปริมาณพลังงานไฟฟ้าเฉลี่ยที่ส่งไปยังอุปกรณ์ เช่น หลอด LED หรือมอเตอร์ DC



(ภาพตัวอย่างจากอินเตอร์เน็ต)

- **Duty Cycle 0%** = สัญญาณเป็น LOW ตลอดเวลา อุปกรณ์ไม่ทำงาน
- **Duty Cycle 50%** = สัญญาณเป็น HIGH ครึ่งหนึ่งของคาบ และ LOW อีกครึ่งหนึ่ง อุปกรณ์ได้รับแรงดันเฉลี่ยครึ่งหนึ่ง
- **Duty Cycle 100%** = สัญญาณเป็น HIGH ตลอดเวลา อุปกรณ์ได้รับแรงดันเต็มที่

การประยุกต์ใช้ PWM ได้แก่ การปรับความสว่าง LED, การควบคุมความเร็วรอบมอเตอร์, หรือการสร้างแรงดันอนาล็อกเทียมในวงจรต่าง ๆ

PWM บน ESP32 ใน MicroPython: ใช้ได้แค่ 10 บิต กับ 16 บิต

แม้ว่า ESP32 จะรองรับความละเอียด PWM ได้สูงสุด 20 บิตในระดับฮาร์ดแวร์ แต่ **MicroPython บน ESP32 ใช้ได้จริงแค่ 2 แบบเท่านั้น คือ**

- **10 บิต** (0–1023) ผ่านเมธอด duty() (สำหรับ MicroPython รุ่นเก่า ก่อน v1.13)
- **16 บิต** (0–65535) ผ่านเมธอด duty_u16() (มาตรฐาน MicroPython รุ่นใหม่ v1.13 ขึ้นไป)

ผู้อ่านไม่สามารถเลือกความละเอียดอื่นเองได้ ทุกคำสั่ง PWM ใน MicroPython จะเป็นไปตามนี้เท่านั้น

ความถี่ PWM ที่เลือกได้จริง

ใน MicroPython จำนวนบิตและความถี่ที่เลือกได้สัมพันธ์กันดังนี้:

- ถ้าใช้ **10 บิต** (duty()) ⟶ กำหนดความถี่ได้สูงสุดประมาณ **78 kHz**
- ถ้าใช้ **16 บิต** (duty_u16()) ⟶ จะกำหนดความถี่ได้สูงสุดแค่ประมาณ **1.2 kHz** เท่านั้น

เหตุผล: ยิ่งความละเอียด (จำนวนบิต) มาก ความถี่สูงสุดที่ตั้งได้จะลดลง เพราะสัญญาณนาฬิกาของ PWM hardware บน ESP32 นั้นมีค่าคงที่

ข้อควรจำ: สำหรับงานควบคุม LED ใช้ความถี่ 500–2000 Hz ก็เพียงพอ สำหรับงานขับมอเตอร์ DC ใช้ 5–20 kHz หากต้องการเสียงรบกวนน้อย แต่ใน MicroPython ความละเอียด 16 บิต จะไปได้ไม่เกิน 1.2 kHz เท่านั้น จึงอาจพิจารณาใช้เพียง 10 บิตเพื่อให้ใช้งานความถี่สูงได้ ถ้าบอร์ดนั้นรองรับ

ตารางสรุปความละเอียดและความถี่ PWM ที่ใช้งานได้จริงใน MicroPython บน ESP32

เมธอด	ความละเอียด	ช่วงค่า Duty	ความถี่สูงสุดที่ใช้งานได้ (โดยประมาณ)
duty()	10 บิต	0–1023	78 kHz
duty_u16()	16 บิต	0–65535	1.2 kHz

การแปลงเปอร์เซ็นต์ Duty Cycle เป็นค่าที่ใช้ในโค้ด

แม้จะพูดว่า Duty Cycle เป็น “เปอร์เซ็นต์” (0–100%) แต่ในโปรแกรม MicroPython ต้องใส่ค่า **ตัวเลขเต็ม** ตามช่วงค่าในตารางด้านบน แต่หากต้องการกำหนดค่าเป็นเปอร์เซ็นสามารถแปลงค่าตัวเลขได้ตามสูตร

สูตรแปลงสำหรับแต่ละแบบ

- **10 บิต**: duty = (เปอร์เซ็นต์ / 100) * 1023
- **16 บิต**: duty_u16 = (เปอร์เซ็นต์ / 100) * 65535

ตารางเทียบตัวอย่าง

Duty Cycle (%)	duty (10 บิต)	duty_u16 (16 บิต)
0	0	0
25	255	16384
50	512	32768
75	767	49151
100	1023	65535

ตรวจสอบเวอร์ชัน MicroPython ก่อนเขียนโค้ด

ก่อนเขียนโค้ด PWM ควรตรวจสอบเวอร์ชัน MicroPython ที่ใช้งาน เพื่อเลือกใช้เมธอดให้ถูกต้อง

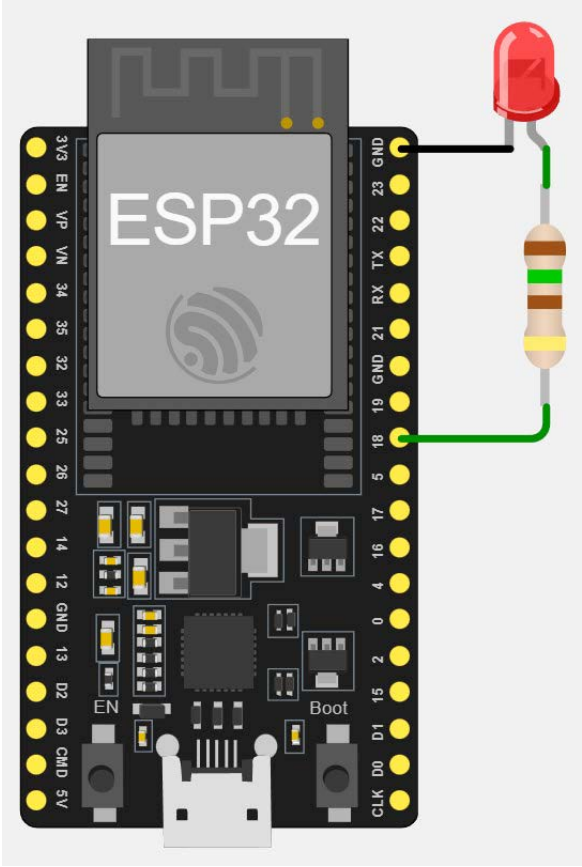
```
#โค้ดสำหรับตรวจสอบเวอร์ชัน
import sys
print(sys.implementation)
```

- ถ้าเป็นรุ่นเก่า (< v1.13) ให้ใช้ duty()
- ถ้าเป็นรุ่นใหม่ (≥ v1.13) ให้ใช้ duty_u16()

4.6.1 ตัวอย่าง: การหรี่ไฟ LED (Fading LED)

ตัวอย่างโค้ด: ปรับความสว่าง LED แบบ 16 บิต (duty_u16)

```
import machine
import time
pwm = machine.PWM(machine.Pin(18))
pwm.freq(1000)
while True:
    # เพิ่มความสว่าง
    for duty in range(0, 65536, 4096):
        pwm.duty_u16(duty)
        time.sleep(0.05)
    # ลดความสว่าง
    for duty in range(65535, -1, -4096):
        pwm.duty_u16(duty)
        time.sleep(0.05)
```



โค้ดชุดนี้เริ่มต้นด้วยการนำเข้าไลบรารี machine ซึ่งเป็นโมดูลหลักสำหรับควบคุมขาและฟังก์ชันฮาร์ดแวร์ของบอร์ดใน MicroPython และไลบรารี time สำหรับใช้หน่วงเวลาระหว่างขั้นตอนการปรับค่า PWM เพื่อให้เกิดเอฟเฟกต์การเปลี่ยนแปลงความสว่างอย่างค่อยเป็นค่อยไป จากนั้นมีการกำหนดขา GPIO หมายเลข 12 ให้เป็นขา PWM ด้วยคำสั่ง pwm = machine.PWM(machine.Pin(12)) ซึ่งหมายความว่าอุปกรณ์ที่เชื่อมต่อกับขานี้ (โดยทั่วไปคือ LED) จะสามารถควบคุมความสว่างได้ผ่านการปรับค่า Duty Cycle ของสัญญาณ PWM

ถัดมาโค้ดจะกำหนดความถี่ของสัญญาณ PWM ให้เป็น 1000 Hz ด้วย pwm.freq(1000) โดยความถี่ระดับนี้ถือว่าเหมาะสมสำหรับงานควบคุมความสว่าง LED เพราะสูงพอที่จะทำให้แสงไม่กระพริบจนตาเห็น อีกทั้งยังไม่สูงเกินไปจนสูญเสียประสิทธิภาพในการควบคุม

เมื่อเข้าสู่ส่วนของลูปหลักซึ่งใช้ while True: เพื่อให้โค้ดทำงานซ้ำไปเรื่อย ๆ ภายในลูปจะประกอบด้วยสองช่วงคือช่วงเพิ่มความสว่างและช่วงลดความสว่าง โดยในช่วงแรกใช้ for duty in range(0, 65536, 4096): เพื่อสร้างลูปที่ค่า duty ไล่จาก 0 (LED ดับสนิท) ไปจนถึง 65535 (LED สว่างเต็มที่) โดยเพิ่มครั้งละ 4096 ซึ่งค่าดังกล่าวแสดงถึงระดับความสว่างแบบละเอียดในระบบ 16 บิต เมื่อถึงแต่ละค่าของ duty ก็จะสั่ง pwm.duty_u16(duty) เพื่อกำหนดค่า duty cycle ให้ขา PWM แล้วหน่วงเวลา 0.05 วินาทีด้วย time.sleep(0.05) เพื่อให้ความสว่างเปลี่ยนอย่างต่อเนื่อง ไม่กระชากหรือเปลี่ยนเร็วเกินไปจนมองไม่เห็นเอฟเฟกต์

จากนั้นจะเข้าสู่ช่วงลดความสว่างโดยใช้ for duty in range(65535, -1, -4096): ซึ่งเป็นการวนค่าจาก 65535 กลับลงมาถึง 0 ทีละ -4096 โดยใช้ค่าลบใน step เพื่อให้ค่า duty ลดลงในแต่ละรอบ การใส่ค่า stop เป็น -1 ก็เพื่อให้ลูปนี้รวมค่าถึง 0 ด้วย (เนื่องจากฟังก์ชัน range จะสิ้นสุดก่อนถึงค่า stop ที่กำหนด) โครงสร้างในแต่ละรอบเหมือนเดิม คือสั่งปรับ duty แล้วหน่วงเวลา 0.05 วินาทีต่อรอบ ผลลัพธ์ที่ได้คือ LED ค่อย ๆ มืดลงจากสว่างเต็มที่จนดับสนิท

การวนลูปแบบนี้จะทำให้เกิดเอฟเฟกต์ LED ค่อย ๆ สว่างขึ้นแล้วค่อย ๆ มืดลงต่อเนื่องไปเรื่อย ๆ สามารถนำหลักการนี้ไปประยุกต์ใช้กับมอเตอร์ เซอร์โว หรือโหลดที่รองรับ PWM อื่น ๆ ได้อีกด้วย จุดสำคัญของการใช้ค่า step เป็นบวกสำหรับขาขึ้นและลบสำหรับขาลงก็เพื่อควบคุมทิศทางของการเปลี่ยนแปลงความสว่างในแต่ละช่วงอย่างถูกต้อง

ข้อควรระวัง

- **MicroPython มีเพียง 2 ความละเอียดเท่านั้น คือ:** 10 บิต (duty()) และ 16 บิต (duty_u16())
- **ความถี่สูงสุดจะลดลงเมื่อใช้ความละเอียดสูงขึ้น:** หากต้องการทั้งความถี่สูงและความละเอียดสูง ต้องพัฒนาโดยใช้ ESP-IDF หรือ Arduino Core แทน
- **ควบคุมมอเตอร์ต้องใช้วงจรขับเสมอ:** ห้ามต่อมอเตอร์เข้ากับขา GPIO โดยตรง

สรุปการใช้งาน machine.PWM (MicroPython)

คลาส PWM ใช้สำหรับสร้างสัญญาณ PWM บนขาที่กำหนด

- **สร้างอ็อบเจกต์ PWM**

  pwm = machine.PWM(machine.Pin(12))

- **ตั้งค่า Duty Cycle**

  pwm.duty(value)

  ใช้บนบางบอร์ด (เช่น ESP8266, ESP32 บางเวอร์ชัน) ค่าที่ใช้: 0–1023 (10 บิต) หรือ 0–255 (8 บิต) ขึ้นอยู่กับบอร์ด

  - ESP8266: 0–1023
  - ESP32: บางเวอร์ชันรองรับ duty 10 บิต, แต่ถ้าเฟิร์มแวร์ใหม่รองรับ duty_u16() จะตั้งค่าได้ 0–65535 (16 บิต)

  pwm.duty_u16(value)

  (ในบอร์ดใหม่ ๆ หรือ RP2040/ESP32 เฟิร์มแวร์ล่าสุด) ค่าที่ใช้: 0–65535 (16 บิต)

  - ถ้าสั่ง **pwm.duty(512)** มันจะไปใช้ความละเอียด 10 บิต และดิวตี้ไซเคิล 50% ถ้าเฟิร์มแวร์รองรับ
  - ถ้าต้องการความละเอียด 16 บิต และดิวตี้ไซเคิล 50% ต้องสั่ง **pwm.duty_u16(32768)**

- **การตั้งค่าความถี่**

  pwm.freq(value)

  ใช้กำหนดความถี่ของ PWM หน่วยเป็น Hz เช่น pwm.freq(1000)

- **หยุดการใช้งาน PWM**

  pwm.deinit()

  ปิดการทำงาน PWM บนขานั้น ทำให้ขากลับสู่ Digital I/O ปกติ คืนทรัพยากร Timer/Peripheral ให้ระบบ

หากต้องการตั้งค่าความถี่ และดิวตี้ไซเคิลคงที่เขียนแค่นี้ก็ทำงานได้แล้ว

```
import machine
import time
pwm = machine.PWM(machine.Pin(18))
pwm.freq(1000)
pwm.duty_u16(32768)
```

โค้ดตัวอย่างการแปลงค่าเปอร์เซ็น ให้เป็นเลข 16 บิต เพื่อคนสั่งงานจะได้ไม่ต้องคำนวณเอง การใช้งานเราแค่ใส่ค่าเข้าไปว่าต้องการกี่ % เช่น 50% ฟังก์ชันจะเอาไปคำนวณเองว่า 50% ของ 65535

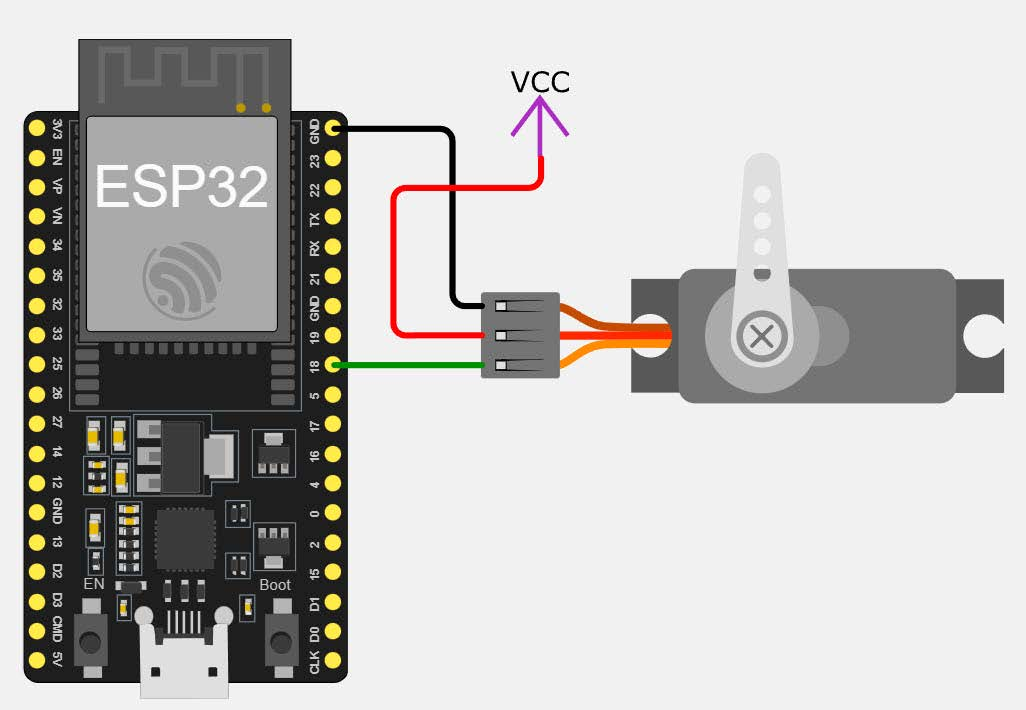
```
import machine
import time

# ฟังก์ชันแปลงเปอร์เซ็นต์ (0-100%) เป็น duty cycle 16 บิต (0-65535)
def percent_to_duty16(percent):
    if percent < 0:
        percent = 0
    if percent > 100:
        percent = 100
    return int((percent / 100) * 65535)

duty_percent = 50  # ต้องการให้ LED ติดที่ 50%
duty = percent_to_duty16(duty_percent)

led_pwm = machine.PWM(machine.Pin(18))
led_pwm.freq(1000)
led_pwm.duty_u16(duty)
```

ในการปรับค่าดิวตี้ไซเคิลนั้นสามารถใช้ค่าที่เซ็นเซอร์ต่าง ๆ อ่านค่าจากอินพุต แล้วเอามาเข้าสูตรคำนวณแปลงให้เป็นค่าดิวตี้ไซเคิลเพื่อปรับความสว่างของ LED หรือปรับรอบการหมุนของมอเตอร์ก็ได้

4.6.2 ตัวอย่าง: การควบคุมตำแหน่งของ Servo Motor (SG90) เบื้องต้น

Servo Motor เป็นมอเตอร์ชนิดพิเศษที่สามารถควบคุมให้หมุนไปยังตำแหน่ง (มุม) ที่ต้องการได้ภายในช่วงที่กำหนด (โดยทั่วไปคือ 0-180 องศา) Servo motor รุ่นเล็กที่นิยมใช้เช่น SG90 จะถูกควบคุมด้วยสัญญาณ PWM

- สายสีน้ำตาล (หรือสีดำ): ต่อเข้ากับ GND ของ ESP32
- **สายสีแดง:** ต่อเข้ากับแหล่งจ่ายไฟ 5V (ESP32 จ่าย 3.3V อาจไม่เพียงพอสำหรับ Servo บางตัว ควรใช้แหล่งจ่าย 5V ภายนอก และหากใช้แหล่งจ่ายภายนอก ต้องต่อ GND ของแหล่งจ่ายภายนอกเข้ากับ GND ของ ESP32 ด้วย)
- **สายสีส้ม (หรือสีเหลือง):** ต่อเข้ากับขา GPIO ของ ESP32 ที่จะใช้ส่งสัญญาณ PWM (เช่น GPIO18)



- **สัญญาณ PWM สำหรับ Servo:**
  - โดยทั่วไป Servo motor จะใช้ความถี่สัญญาณ PWM ประมาณ 50Hz (คาบเวลา 20ms)
  - ตำแหน่งของ Servo จะถูกควบคุมโดยความกว้างของพัลส์ (Pulse Width) ในช่วงที่เป็น HIGH:
    - ประมาณ 1ms (Duty Cycle 5% ของ 20ms) -> ตำแหน่ง 0 องศา
    - ประมาณ 1.5ms (Duty Cycle 7.5% ของ 20ms) -> ตำแหน่ง 90 องศา (ตรงกลาง)
    - ประมาณ 2ms (Duty Cycle 10% ของ 20ms) -> ตำแหน่ง 180 องศา

ค่า Duty Cycle สำหรับ machine.PWM (0-1023) ที่สอดคล้องกับ 50Hz:

- 1ms pulse = (1ms / 20ms) * 1023 ≈ 51
- 1.5ms pulse = (1.5ms / 20ms) * 1023 ≈ 77
- 2ms pulse = (2ms / 20ms) * 1023 ≈ 102

ตัวอย่าง โค้ดควบคุม Servo Motor

```
import machine
import time


# กำหนดขาสำหรับ Servo (เช่น GPIO18)
servo_pin = machine.Pin(13)
servo_pwm = machine.PWM(servo_pin, freq=50) # Servo ใช้ความถี่ 50Hz
def set_servo_angle(angle):
    """แปลงมุม (0-180) เป็นค่า duty cycle (ประมาณ 26-128 สำหรับ SG90 ที่ 50Hz)
       ค่า duty ที่แม่นยำอาจต้องปรับเทียบสำหรับ Servo แต่ละตัว"""
    if angle < 0: angle = 0
    if angle > 180: angle = 180
    # ค่า duty สำหรับ SG90 ที่ 50Hz มักจะอยู่ในช่วงประมาณ 25 (0.5ms) ถึง 125 (2.5ms)
    # ซึ่งสอดคล้องกับ 0-180 องศา (อาจต้องปรับค่า min_duty, max_duty)
    min_duty = 26  # ค่าโดยประมาณสำหรับ 0 องศา
    max_duty = 128 # ค่าโดยประมาณสำหรับ 180 องศา
    duty_val = int(min_duty + (max_duty - min_duty) * (angle / 180))
    servo_pwm.duty(duty_val)
print("โปรแกรมควบคุม Servo Motor")
try:
    while True:
        print("ตั้ง Servo ไปที่ 0 องศา")
        set_servo_angle(0)
        time.sleep(2)
        print("ตั้ง Servo ไปที่ 90 องศา")
        set_servo_angle(90)
        time.sleep(2)
        print("ตั้ง Servo ไปที่ 180 องศา")
        set_servo_angle(180)
```

```
            time.sleep(2)
    except KeyboardInterrupt:
        print("หยุดโปรแกรม")
    finally:
        servo_pwm.deinit() # ปิด PWM
        print("Servo PWM deinitialized")
```

หมายเหตุ: ค่า Duty Cycle ที่แม่นยำสำหรับมุมต่างๆ ของ Servo อาจแตกต่างกันไปใน Servo แต่ละตัวและแต่ละยี่ห้อ อาจจำเป็นต้องมีการปรับเทียบ (Calibration) เล็กน้อย

ขอขยายความสำหรับคนที่อยากเข้าใจลึกขึ้น (และสำหรับ MG90S ด้วย)

1. **MG90S กับ SG90 ใช้หลักการเหมือนกัน!**

- min_duty และ max_duty อาจต้อง “ลองปรับจูน” ตามแต่ละตัว (ของจีนบางล็อตต้องจูนเพิ่มหรือลด)

2. ค่า duty สำหรับเซอร์โวที่ใช้ servo_pwm.duty(x)

- ใน MicroPython (esp8266/esp32):
    - คำสั่ง .duty() (ไม่ใช่ .duty_u16()) จะรับค่าช่วง 0–1023 (10 บิต)
    - ที่ 50Hz, ค่า “ประมาณ 25–128” สอดคล้องกับ 0.5–2.5ms

3. **ค่า min/max duty ที่แนบในโค้ดนี้ (26, 128) มาตรฐานแล้วสำหรับบอร์ด ESP8266/ESP32 ทั่วไป**

ถ้าเอาไปใช้จริงกับ MG90S

- **MG90S:**
    - ทดลอง min_duty = 26 กับ max_duty = 128 ก่อน
    - ถ้ามุมยังไม่สุด หรือเซอร์โวมีเสียงดังผิดปกติ ให้ขยับค่า min/max ทีละน้อย
    - อัตราส่วนและการแปลง angle เป็น duty เหมือนตัวอย่าง

- ถ้าต้องการใช้กับ PWM แบบ 16 บิต (duty_u16()), สูตร duty จะต่างออกไปนิดหน่อย (ต้องแมปค่าช่วง 0.5ms–2.5ms ไปยัง 0–65535)

ตัวอย่างผลลัพธ์: เซอร์โวจะหมุนไปที่ 0°, 90°, 180° แล้ววนซ้ำ

4.7 สรุปและการประยุกต์ใช้งาน GPIO

ในบทนี้ ผู้อ่านได้เรียนรู้หลักการพื้นฐานและการใช้งานขา GPIO ของ ESP32 ด้วย MicroPython อย่างครอบคลุม ตั้งแต่การกำหนดค่าเป็น Digital Output เพื่อควบคุม LED, การกำหนดค่าเป็น Digital Input เพื่ออ่านสถานะจากสวิตช์พร้อมทั้งแนวทางการจัดการปัญหาสัญญาณสั่น (Debouncing), และการใช้เทคนิค PWM เพื่อควบคุมความสว่างของ LED รวมถึงการควบคุมตำแหน่งของ Servo motor เบื้องต้น

ความสามารถในการควบคุมขา GPIO เหล่านี้เป็นทักษะพื้นฐานที่สำคัญอย่างยิ่ง เพราะเป็นประตูสู่การสร้างโครงงานที่สามารถโต้ตอบกับสภาพแวดล้อมและอุปกรณ์ภายนอกได้หลากหลายรูปแบบ เช่น:

- **ระบบอัตโนมัติ:** เปิด/ปิดไฟตามการกดสวิตช์, ควบคุมปั๊มน้ำตามระดับน้ำ (โดยใช้เซ็นเซอร์ที่ให้สัญญาณดิจิทัล)
- **หุ่นยนต์:** ควบคุมการเคลื่อนที่ของล้อด้วยมอเตอร์ (ผ่านวงจรขับ), ควบคุมแขนกลด้วย Servo motor
- **ระบบแสดงผลและแจ้งเตือน:** แสดงสถานะต่างๆ ด้วย LED หลายดวง, สร้างเสียงแจ้งเตือนด้วย Buzzer (ซึ่งก็ควบคุมด้วยสัญญาณดิจิทัลหรือ PWM ได้)
- **การเชื่อมต่อกับเซ็นเซอร์:** อ่านค่าจากเซ็นเซอร์ดิจิทัลหลายชนิด (เช่น PIR sensor ตรวจจับความเคลื่อนไหว, Magnetic reed switch ตรวจจับการเปิด/ปิดประตู)

ในบทต่อไป เราจะก้าวไปอีกขั้นโดยการเรียนรู้วิธีการให้ ESP32 เชื่อมต่อกับเครือข่าย Wi-Fi ซึ่งจะเปิดประตูสู่โลกของ Internet of Things (IoT) อย่างแท้จริง เตรียมพบกับการทำให้ ESP32 ของเราออนไลน์ได้ในบทที่ 5

ส่วนที่ 3: ESP32 เชื่อมต่อโลกภายนอก

บทที่ 5: การเชื่อมต่อ Wi-Fi และเครือข่ายเบื้องต้น

ในบทที่ผ่านมา เราได้เรียนรู้วิธีการควบคุมขา GPIO ของ ESP32 เพื่อโต้ตอบกับอุปกรณ์ภายนอก เช่น LED, สวิตช์, และ Servo motor ซึ่งเป็นการสร้างปฏิสัมพันธ์ในระดับฮาร์ดแวร์ ในบทนี้ เราจะยกระดับความสามารถของ ESP32 ขึ้นไปอีกขั้น ด้วยการทำให้มันสามารถเชื่อมต่อกับเครือข่าย Wi-Fi และสื่อสารผ่านอินเทอร์เน็ตได้ นี่คือหัวใจสำคัญของการสร้างโครงงาน Internet of Things (IoT) อย่างแท้จริง

บทนี้จะครอบคลุมตั้งแต่ความสำคัญของการเชื่อมต่อ Wi-Fi, การใช้งานโมดูล network ใน MicroPython เพื่อเชื่อมต่อ ESP32 เข้ากับเครือข่าย Wi-Fi ในโหมด Station, ความรู้พื้นฐานเกี่ยวกับเครือข่ายคอมพิวเตอร์ที่จำเป็น,

การดึงข้อมูลจากอินเทอร์เน็ตโดยใช้ API สาธารณะ, ไปจนถึงการสร้าง Web Server ง่ายๆ บน ESP32 เพื่อให้สามารถเข้าถึงและควบคุม ESP32 ผ่านเว็บเบราว์เซอร์ได้

5.1 ทำไมต้องเชื่อมต่อ Wi-Fi?

การเชื่อมต่อ Wi-Fi เป็นหนึ่งในคุณสมบัติเด่นที่สุดของชิป ESP32 เมื่อ ESP32 สามารถเชื่อมต่ออินเทอร์เน็ตได้ ความเป็นไปได้ในการสร้างโครงงานจะขยายออกไปอย่างมหาศาล ตัวอย่างเช่น:

- **การรับส่งข้อมูลระยะไกล:** ESP32 สามารถส่งข้อมูลที่อ่านได้จากเซ็นเซอร์ (เช่น อุณหภูมิ, ความชื้น, คุณภาพอากาศ) ไปยัง Cloud server เพื่อจัดเก็บ, วิเคราะห์, หรือแสดงผลบนแดชบอร์ดออนไลน์
- **การควบคุมระยะไกล:** สามารถสั่งงาน ESP32 (เช่น เปิด/ปิดไฟ, ควบคุมเครื่องใช้ไฟฟ้า) จากที่ใดก็ได้ในโลกผ่านแอปพลิเคชันบนสมาร์ตโฟนหรือหน้าเว็บไซต์
- **การอัปเดตข้อมูลอัตโนมัติ:** ESP32 สามารถดึงข้อมูลล่าสุดจากอินเทอร์เน็ต เช่น เวลามาตรฐาน, ข้อมูลพยากรณ์อากาศ, หรือข่าวสาร มาแสดงผลหรือใช้ในการตัดสินใจ
- **การแจ้งเตือน:** ส่งการแจ้งเตือนไปยังผู้ใช้ผ่านอีเมล, LINE, หรือแพลตฟอร์มอื่นๆ เมื่อมีเหตุการณ์สำคัญเกิดขึ้น (เช่น ตรวจพบผู้บุกรุก, ระดับน้ำสูงเกินกำหนด)
- **การทำงานร่วมกับบริการออนไลน์:** เชื่อมต่อกับบริการ API (Application Programming Interface) ต่างๆ เพื่อเพิ่มความสามารถให้กับโครงงาน เช่น บริการรู้จำเสียง, บริการแผนที่, หรือบริการฐานข้อมูลออนไลน์

การเรียนรู้วิธีการเชื่อมต่อ Wi-Fi จึงเป็นทักษะพื้นฐานที่สำคัญสำหรับนักพัฒนา IoT ทุกคน

5.2 โมดูล network: กุญแจสู่โลกออนไลน์

ใน MicroPython การจัดการเกี่ยวกับการเชื่อมต่อเครือข่ายทั้งหมดจะทำผ่านโมดูล network โมดูลนี้มีคลาสและฟังก์ชันที่จำเป็นสำหรับการตั้งค่าและควบคุมอินเทอร์เฟซเครือข่ายต่างๆ ที่ ESP32 รองรับ โดยเฉพาะอย่างยิ่ง Wi-Fi

โหมดการทำงานหลักของ Wi-Fi บน ESP32:

ESP32 รองรับการทำงานของ Wi-Fi ได้สองโหมดหลัก:

1. **โหมด Station (STA_IF):**
    - ในโหมดนี้ ESP32 จะทำหน้าที่เป็น **Client** (ลูกข่าย) เชื่อมต่อเข้ากับ Access Point (AP) ที่มีอยู่แล้ว เช่น Router Wi-Fi ที่บ้านหรือที่ทำงาน เพื่อเข้าถึงเครือข่ายท้องถิ่น (LAN) และอินเทอร์เน็ต
    - นี่เป็นโหมดที่ใช้บ่อยที่สุดสำหรับการสร้างอุปกรณ์ IoT ที่ต้องการเชื่อมต่ออินเทอร์เน็ต
2. **โหมด Access Point (AP_IF):**

- ในโหมดนี้ ESP32 จะทำหน้าที่เป็น **Access Point** เอง สร้างเครือข่าย Wi-Fi ขนาดเล็กขึ้นมา เพื่อให้ Client อื่นๆ (เช่น สมาร์ตโฟน, แล็ปท็อป) สามารถเชื่อมต่อเข้ามายัง ESP32 ได้โดยตรง
- โหมดนี้มีประโยชน์สำหรับการตั้งค่า ESP32 ในครั้งแรก (Provisioning), การสื่อสารแบบ Peer-to-Peer ระหว่างอุปกรณ์โดยไม่ต้องมี Router, หรือเมื่อต้องการสร้างเครือข่ายเฉพาะกิจขนาดเล็ก

ในบทนี้ เราจะมุ่งเน้นไปที่การใช้งาน **โหมด Station (network.STA_IF)** เพื่อเชื่อมต่อ ESP32 เข้ากับเครือข่าย Wi-Fi ที่มีอยู่

5.3 การเชื่อมต่อ ESP32 เข้ากับเครือข่าย Wi-Fi (โหมด Station)

ขั้นตอนการเชื่อมต่อ ESP32 เข้ากับเครือข่าย Wi-Fi ในโหมด Station มีดังนี้:

5.3.1 ขั้นตอนการเชื่อมต่อ

1. **Import โมดูล network**:

```
import network
import time      # สำหรับการหน่วงเวลาตรวจสอบ
```

2. **สร้างออบเจกต์ WLAN สำหรับ Station interface**: WLAN (Wireless Local Area Network) คือคลาสหลักสำหรับจัดการการเชื่อมต่อ Wi-Fi

```
sta_if = network.WLAN(network.STA_IF)
```

3. **เปิดใช้งาน (Activate) interface**: ก่อนจะใช้งาน ต้องเปิดการทำงานของ Wi-Fi interface ก่อน

```
if not sta_if.active():
    print("Activating Wi-Fi station interface...")
    sta_if.active(True)
```

4. **สแกนหาเครือข่าย Wi-Fi ที่พร้อมใช้งาน (Optional)**: สามารถสแกนหาเครือข่าย Wi-Fi ในบริเวณนั้น เพื่อดูว่ามีเครือข่ายใดบ้างที่ ESP32 ตรวจพบ ฟังก์ชัน scan() จะคืนค่าเป็นลิสต์ของทูเพิล ซึ่งแต่ละทูเพิลประกอบด้วยข้อมูลของเครือข่ายที่พบ (SSID, BSSID, channel, RSSI, authmode, hidden)

```
# print("Scanning for Wi-Fi networks...")
# available_networks = sta_if.scan()
```

```
# print("Available networks:")
# for net in available_networks:
#     print(f"  SSID: {net[0].decode()}, RSSI: {net[3]} dBm, Channel: {net[2]}")
```

หมายเหตุ: net[0] คือ SSID ซึ่งเป็น bytes object จึงต้องใช้ .decode() เพื่อแปลงเป็นสตริง

5. **เชื่อมต่อกับเครือข่ายที่ต้องการ**: ใช้เมธอด connect() โดยระบุ SSID (ชื่อ Wi-Fi) และรหัสผ่านของเครือข่ายที่เราจะเชื่อมต่อ

```
ssid = 'YOUR_WIFI_SSID'                    # <--- แทนที่ด้วยชื่อ Wi-Fi ที่เราใช้จริง
password = 'YOUR_WIFI_PASSWORD'  # <--- แทนที่ด้วยรหัสผ่าน Wi-Fi ที่เราใช้จริง


if not sta_if.isconnected():
    print(f"Connecting to network '{ssid}'...")
    sta_if.connect(ssid, password)
    # รอจนกว่าจะเชื่อมต่อสำเร็จ
    max_wait_seconds = 15
    wait_counter = 0
    while not sta_if.isconnected() and wait_counter < max_wait_seconds:
        print(f"Connection attempt {wait_counter + 1}/{max_wait_seconds}...")
        time.sleep(1) # รอ 1 วินาที
        wait_counter += 1
```

6. **ตรวจสอบสถานะการเชื่อมต่อ**: ใช้เมธอด isconnected() ซึ่งจะคืนค่า True หากเชื่อมต่อสำเร็จ และ False หากยังไม่ได้เชื่อมต่อหรือการเชื่อมต่อล้มเหลว

```
if sta_if.isconnected():
    print("Wi-Fi connected successfully!")
else:
    print("Failed to connect to Wi-Fi.")
```

7. **การแสดงข้อมูลเครือข่ายหลังเชื่อมต่อสำเร็จ**: เมื่อเชื่อมต่อสำเร็จ สามารถใช้เมธอด ifconfig() เพื่อดูการตั้งค่าเครือข่ายที่ ESP32 ได้รับ เช่น IP Address, Subnet Mask, Gateway, และ DNS Server

```
if sta_if.isconnected():
    network_config = sta_if.ifconfig()
    print("Network configuration:")
    print(f"  IP Address:    {network_config[0]}")
    print(f"  Subnet Mask:   {network_config[1]}")
    print(f"  Gateway:       {network_config[2]}")
    print(f"  DNS Server:    {network_config[3]}")
```

5.3.2 โค้ดตัวอย่างฟังก์ชันสำหรับเชื่อมต่อ Wi-Fi

เพื่อให้ง่ายต่อการนำไปใช้งาน สามารถสร้างเป็นฟังก์ชันได้ดังนี้:

```
import network
import time


def connect_wifi(ssid, password, max_wait=15):
    """เชื่อมต่อ ESP32 กับเครือข่าย Wi-Fi ที่กำหนด"""
    sta_if = network.WLAN(network.STA_IF)
    if not sta_if.active():
        print("Activating Wi-Fi station interface...")
        sta_if.active(True)

    if not sta_if.isconnected():
        print(f"Connecting to network '{ssid}'...")
        sta_if.connect(ssid, password)

        wait_counter = 0
        while not sta_if.isconnected() and wait_counter < max_wait:
            print(f"  Attempt {wait_counter + 1}/{max_wait}...")
            time.sleep(1)
            wait_counter += 1
```

```
    if sta_if.isconnected():
        print("Wi-Fi connected successfully!")
        config = sta_if.ifconfig()
        print(f"  IP Address: {config[0]}")
        return True
    else:
        print("Failed to connect to Wi-Fi.")
        return False

# --- การเรียกใช้งานฟังก์ชัน ---
my_ssid = "YOUR_WIFI_SSID"  # <--- แก้ไข
my_password = "YOUR_WIFI_PASSWORD" # <--- แก้ไข

if connect_wifi(my_ssid, my_password):
    print("Ready to use network features.")
else:
    print("Please check Wi-Fi credentials or network availability.")
```

ข้อควรจำ: ก่อนจะอัปโหลดโค้ดนี้ไปยัง ESP32 ต้องแก้ไข my_ssid และ my_password ให้ถูกต้อง

5.4 พื้นฐานเครือข่ายคอมพิวเตอร์เบื้องต้น (สำหรับ IoT)

เพื่อให้เข้าใจการทำงานของ ESP32 บนเครือข่ายได้ดีขึ้น ควรทำความรู้จักกับคำศัพท์พื้นฐานทางเครือข่ายต่อไปนี้:

- **IP Address (Internet Protocol Address):**
    - เป็นหมายเลขประจำตัวของอุปกรณ์แต่ละชิ้นที่เชื่อมต่ออยู่ในเครือข่ายคอมพิวเตอร์ ทำให้สามารถระบุและสื่อสารกันได้ (เปรียบเหมือนเลขที่บ้าน)
    - **IPv4:** รูปแบบที่ใช้กันแพร่หลาย เช่น 192.168.1.100 ประกอบด้วยตัวเลข 4 ชุด คั่นด้วยจุด แต่ละชุดมีค่าตั้งแต่ 0-255

- **Private IP:** ใช้ภายในเครือข่ายท้องถิ่น (LAN) เช่น เครือข่ายที่บ้านหรือสำนักงาน (เช่น 192.168.x.x, 10.x.x.x) ไม่สามารถเข้าถึงได้โดยตรงจากอินเทอร์เน็ต
- **Public IP:** เป็น IP ที่ Router ได้รับจากผู้ให้บริการอินเทอร์เน็ต (ISP) และใช้ในการสื่อสารกับโลกอินเทอร์เน็ตภายนอก

- **Subnet Mask:**
  - ใช้เพื่อระบุว่าส่วนใดของ IP Address เป็นส่วนของ "เครือข่าย" (Network ID) และส่วนใดเป็นส่วนของ "โฮสต์" (Host ID) ภายในเครือข่ายนั้นๆ เช่น 255.255.255.0
- **Gateway (Default Gateway):**
  - เป็นอุปกรณ์ (โดยทั่วไปคือ Router) ที่ทำหน้าที่เป็นประตูเชื่อมต่อระหว่างเครือข่ายท้องถิ่น (LAN) กับเครือข่ายภายนอก (เช่น อินเทอร์เน็ต) เมื่ออุปกรณ์ใน LAN ต้องการสื่อสารกับอุปกรณ์นอก LAN ข้อมูลจะถูกส่งผ่าน Gateway
- **DNS (Domain Name System) Server:**
  - ทำหน้าที่แปลงชื่อโดเมนที่มนุษย์เข้าใจง่าย (เช่น www.google.com) ให้เป็น IP Address ที่คอมพิวเตอร์ใช้ในการสื่อสาร (เช่น 142.250.201.163)
- **Client และ Server:**
  - **Client (ลูกข่าย):** เป็นฝ่ายที่ร้องขอบริการหรือข้อมูล เช่น เว็บเบราว์เซอร์ของเราร้องขอหน้าเว็บ
  - **Server (แม่ข่าย):** เป็นฝ่ายที่ให้บริการหรือข้อมูล เช่น เว็บเซิร์ฟเวอร์ที่เก็บหน้าเว็บและส่งให้เมื่อมีการร้องขอ
  - ในบริบทของ ESP32:
    - เมื่อ ESP32 ดึงข้อมูลจากอินเทอร์เน็ต มันจะทำหน้าที่เป็น **Client**
    - เมื่อ ESP32 สร้าง Web Server เพื่อให้เราเข้าไปดูข้อมูล มันจะทำหน้าที่เป็น **Server**
- **HTTP (Hypertext Transfer Protocol):**
  - เป็นโปรโตคอล (ชุดของกฎเกณฑ์และข้อตกลง) ที่ใช้ในการสื่อสารและแลกเปลี่ยนข้อมูลบน World Wide Web ส่วนใหญ่ใช้สำหรับส่งและรับหน้าเว็บ (HTML) และข้อมูลอื่นๆ
  - **GET request:** เป็นการร้องขอข้อมูลจาก Server เช่น เมื่อเราพิมพ์ URL ในเว็บเบราว์เซอร์แล้วกด Enter
  - **POST request:** เป็นการส่งข้อมูลจาก Client ไปยัง Server เช่น เมื่อเรากรอกฟอร์มบนหน้าเว็บแล้วกด Submit

5.5 การดึงข้อมูลจากอินเทอร์เน็ตด้วยโมดูล urequests

เมื่อ ESP32 เชื่อมต่อ Wi-Fi ได้แล้ว ก็สามารถทำหน้าที่เป็น HTTP Client เพื่อดึงข้อมูลจากเว็บไซต์หรือ API ต่างๆ บนอินเทอร์เน็ตได้ MicroPython มีไลบรารีขนาดเล็กชื่อ urequests (หรือ requests ในบาง build ที่ใหม่กว่า) สำหรับการทำ HTTP requests

urequests คืออะไร? urequests เป็นโมดูลที่เลียนแบบการทำงานของไลบรารี requests ที่ได้รับความนิยมใน Python เวอร์ชันปกติ แต่ถูกปรับให้มีขนาดเล็กและใช้ทรัพยากรน้อย เหมาะสำหรับไมโครคอนโทรลเลอร์

การติดตั้ง urequests (ถ้ายังไม่มีในเฟิร์มแวร์): เฟิร์มแวร์ MicroPython สำหรับ ESP32 บางเวอร์ชันอาจมีโมดูล urequests ติดตั้งมาให้แล้ว หากไม่มี สามารถติดตั้งได้ผ่าน Thonny IDE:

1. ใน Thonny ไปที่ Tools > Manage packages...
2. ในช่องค้นหา พิมพ์ micropython-urequests แล้วคลิก Search on PyPI
3. เลือก micropython-urequests จากผลการค้นหาแล้วคลิก Install

การใช้ urequests.get() เพื่อดึงข้อมูลจาก API สาธารณะ: API (Application Programming Interface) คือช่องทางที่ซอฟต์แวร์หนึ่งเปิดให้ซอฟต์แวร์อื่นเข้ามาเรียกใช้งานฟังก์ชันหรือเข้าถึงข้อมูลได้ มี API สาธารณะมากมายที่ให้บริการข้อมูลที่เป็นประโยชน์ เช่น เวลา, สภาพอากาศ, ข้อมูลข่าวสาร

ตัวอย่าง: ดึงเวลาปัจจุบันจาก WorldTimeAPI WorldTimeAPI (http://worldtimeapi.org/) เป็น API สาธารณะที่ไม่ต้องใช้ Key สำหรับการเข้าถึง ให้ข้อมูลเวลาตามโซนต่างๆ ในรูปแบบ JSON

```
import urequests # หรือ import requests หากใช้ชื่อนี้
import json      # สำหรับจัดการข้อมูล JSON


# ตรวจสอบให้แน่ใจว่าเชื่อมต่อ Wi-Fi แล้ว (ใช้ฟังก์ชัน connect_wifi จากตัวอย่างก่อนหน้า)
# if not connect_wifi(my_ssid, my_password):
#     raise RuntimeError("Wi-Fi connection failed")


# URL ของ API สำหรับเวลาในกรุงเทพฯ
time_api_url = "http://worldtimeapi.org/api/timezone/Asia/Bangkok"


print(f"Fetching time from {time_api_url}...")
try:
```

```
    response = urequests.get(time_api_url)

    if response.status_code == 200: # 200 OK หมายถึงการร้องขอสำเร็จ
        print("Successfully fetched data!")
        # ข้อมูลที่ได้จาก API จะอยู่ในรูปแบบ JSON
        data = response.json() # แปลง JSON response เป็น Python dictionary

        # แสดงข้อมูลบางส่วน
        print(f"  Datetime: {data['datetime']}")
        print(f"  Day of week: {data['day_of_week']}")
        print(f"  Timezone: {data['timezone']}")

        # ปิดการเชื่อมต่อ (urequests ส่วนใหญ่จัดการให้ แต่การ close() เป็น good practice)
        response.close()
    else:
        print(f"Error fetching data: Status code {response.status_code}")
        response.close() # อย่าลืม close แม้จะ error

except Exception as e:
    print(f"An error occurred: {e}")
```

- **response.status_code**: บอกสถานะของการร้องขอ HTTP (เช่น 200 คือสำเร็จ, 404 คือไม่พบหน้า, 500 คือ Server error)
- **response.json()**: เมธอดนี้จะพยายามแปลงเนื้อหาของ Response (ซึ่งควรจะเป็น JSON string) ให้เป็น Python dictionary หรือ list ทำให้ง่ายต่อการเข้าถึงข้อมูล
- **response.text**: หากต้องการเข้าถึงเนื้อหา Response เป็นสตริงธรรมดา
- **response.close()**: เป็นการดีที่จะปิดการเชื่อมต่อหลังจากใช้งานเสร็จ เพื่อคืนทรัพยากร

5.6 การสร้าง Web Server ง่ายๆ บน ESP32 (เบื้องต้น)

นอกจากการเป็น Client แล้ว ESP32 ยังสามารถทำหน้าที่เป็น Web Server ขนาดเล็กได้ เพื่อให้ Client อื่นๆ (เช่น คอมพิวเตอร์หรือสมาร์ตโฟนในเครือข่าย Wi-Fi เดียวกัน) สามารถเชื่อมต่อเข้ามาดูข้อมูลหรือส่งคำสั่งเบื้องต้นได้ผ่านเว็บเบราว์เซอร์ การสร้าง Web Server ใน MicroPython จะใช้โมดูล socket

ขั้นตอนการสร้าง Web Server พื้นฐาน:

1. **Import โมดูล socket และ network**
2. **เชื่อมต่อ Wi-Fi** (เพื่อให้ ESP32 มี IP Address)
3. **สร้าง Socket Object:** กำหนดประเภทของ Socket (AF_INET สำหรับ IPv4, SOCK_STREAM สำหรับ TCP)
4. **Bind Socket:** ผูก Socket เข้ากับ IP Address ของ ESP32 และ Port ที่ต้องการ (Port 80 เป็น Port มาตรฐานสำหรับ HTTP)
5. **Listen:** ให้ Socket เริ่มรอรับการเชื่อมต่อจาก Client
6. **Loop รอ Accept การเชื่อมต่อ:** เมื่อมี Client เชื่อมต่อเข้ามา เมธอด accept() จะคืนค่าเป็น Connection object และ Address ของ Client
7. **รับ HTTP Request:** อ่านข้อมูลคำร้องขอที่ Client ส่งมา
8. **สร้างและส่ง HTTP Response:** สร้างเนื้อหา HTML หรือข้อมูลอื่นๆ ที่จะส่งกลับไปให้ Client พร้อมทั้ง HTTP Header ที่ถูกต้อง
9. **ปิด Connection กับ Client:** หลังจากส่ง Response แล้ว ให้ปิดการเชื่อมต่อกับ Client นั้นๆ
10. **กลับไปรอการเชื่อมต่อใหม่** (หรือปิด Server หากไม่ต้องการรับอีก)

โค้ดตัวอย่าง: Web Server แสดงข้อความ "Hello from ESP32"

```
import network
import socket
import time


# ---------- ฟังก์ชันเชื่อมต่อ Wi-Fi ----------
def connect_wifi(ssid, password):
    sta_if = network.WLAN(network.STA_IF)
    if not sta_if.active():
```

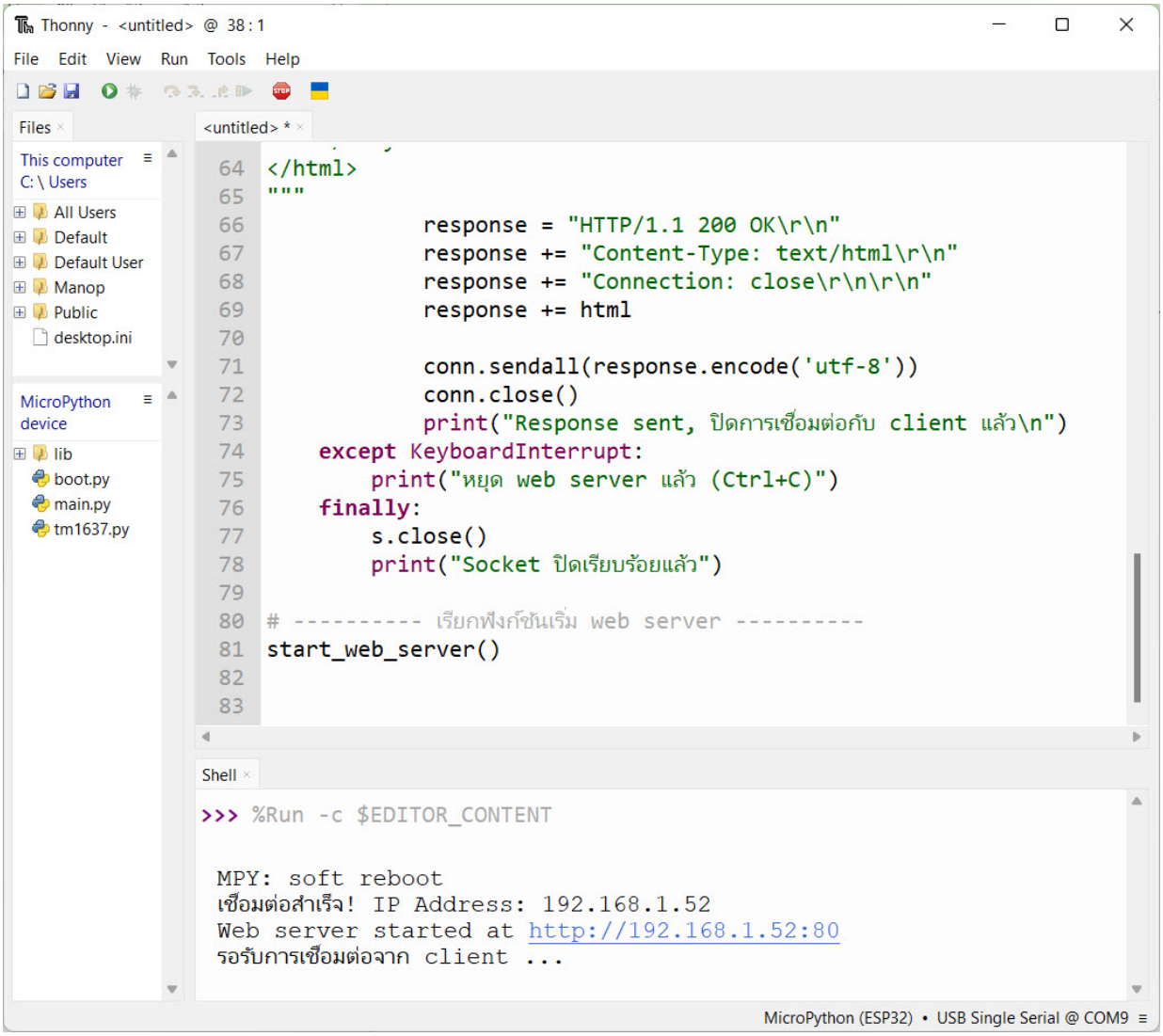
```
        sta_if.active(True)
    if not sta_if.isconnected():
        print("กำลังเชื่อมต่อ Wi-Fi ...")
        sta_if.connect(ssid, password)
        timeout = 15  # หน่วงรอสูงสุด 15 วินาที
        while not sta_if.isconnected() and timeout > 0:
            print(".", end="")
            time.sleep(1)
            timeout -= 1
        print()
    if sta_if.isconnected():
        print("เชื่อมต่อสำเร็จ! IP Address:", sta_if.ifconfig()[0])
        return True
    else:
        print("เชื่อมต่อ Wi-Fi ไม่สำเร็จ")
        return False

# ---------- เริ่มเชื่อมต่อ Wi-Fi ----------
SSID = "ใส่ชื่อไวไฟที่จะใช้เชื่อมต่อจริง"
PASSWORD = "ใส่รหัสผ่านจริงที่จะเชื่อมต่อ"

if not connect_wifi(SSID, PASSWORD):
    raise RuntimeError("Wi-Fi connection failed, ไม่สามารถเปิด web server ได้")

# ---------- เริ่มต้น Web Server ----------
def start_web_server():
    sta_if = network.WLAN(network.STA_IF)
    ip_address = sta_if.ifconfig()[0]
    port = 80
```

```
    # 1. สร้าง socket
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind((ip_address, port))
    s.listen(1)

    print(f"Web server started at http://{ip_address}:{port}")

    try:
        while True:
            print("รอรับการเชื่อมต่อจาก client ...")
            conn, addr = s.accept()
            print("Client เชื่อมต่อจาก:", addr)
            request = conn.recv(1024)
            request_str = request.decode('utf-8')
            print("Request:", request_str.splitlines()[0])

            # สร้าง HTTP response
            html = """<!DOCTYPE html>
<html>
    <head><title>ESP32 Web Server</title></head>
    <body>
        <h1>Hello from ESP32!</h1>
        <p>This page is served by a MicroPython web server on an ESP32.</p>
    </body>
</html>
"""
            response = "HTTP/1.1 200 OK\r\n"
```

```
            response += "Content-Type: text/html\r\n"
            response += "Connection: close\r\n\r\n"
            response += html

            conn.sendall(response.encode('utf-8'))
            conn.close()
            print("Response sent, ปิดการเชื่อมต่อกับ client แล้ว\n")
    except KeyboardInterrupt:
        print("หยุด web server แล้ว (Ctrl+C)")
    finally:
        s.close()
        print("Socket ปิดเรียบร้อยแล้ว")

# ---------- เรียกฟังก์ชันเริ่ม web server ----------
start_web_server()
```

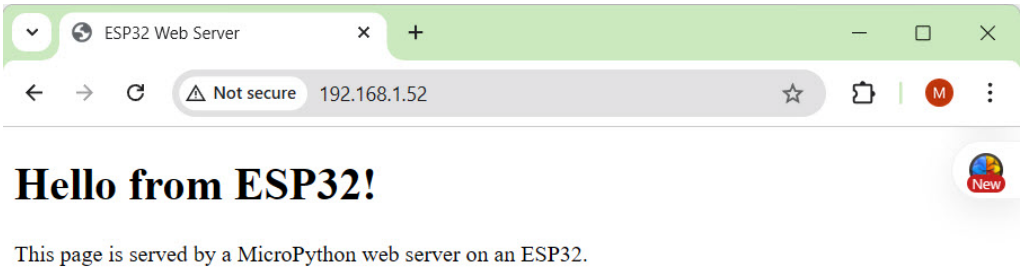
การทดสอบ:

1. อัปโหลดโค้ดนี้ (พร้อมส่วนเชื่อมต่อ Wi-Fi ที่ถูกต้อง) ไปยัง ESP32
2. เปิดโปรแกรม Thonny ดู Shell เพื่อดู IP Address ที่ ESP32 ได้รับ และข้อความว่า Server เริ่มทำงานแล้ว



ภาพตัวอย่างจากผู้เขียนที่ทดลองให้ดู

3. บนคอมพิวเตอร์หรือสมาร์ตโฟนที่อยู่ในเครือข่าย Wi-Fi เดียวกัน เปิดเว็บเบราว์เซอร์ (เช่น Chrome, Firefox)
4. พิมพ์ IP Address ของ ESP32 ลงในแถบที่อยู่ของเบราว์เซอร์ (เช่น http://192.168.1.100) แล้วกด Enter
5. หน้าเว็บที่มีข้อความ "Hello from ESP32!" ควรจะปรากฏขึ้น



นี่เป็น Web Server ที่ง่ายที่สุด สามารถพัฒนาต่อยอดเพื่อแสดงข้อมูลจากเซ็นเซอร์ หรือสร้างปุ่มสำหรับควบคุม LED บน ESP32 ได้ (ซึ่งอาจต้องเรียนรู้การจัดการ HTTP request ที่ซับซ้อนขึ้น)

5.7 สรุปและก้าวต่อไปสู่โลก IoT

ในบทนี้ ผู้อ่านได้เรียนรู้หัวใจสำคัญของการทำให้ ESP32 สามารถ "ออนไลน์" ได้สำเร็จ ตั้งแต่การเชื่อมต่อเข้ากับเครือข่าย Wi-Fi ในโหมด Station, ทำความเข้าใจพื้นฐานเครือข่ายที่จำเป็น, ทดลองดึงข้อมูลจาก API ภายนอกด้วย urequests เพื่อให้ ESP32 เป็น Client, และสร้าง Web Server อย่างง่ายด้วยโมดูล socket เพื่อให้ ESP32 สามารถทำหน้าที่เป็น Server ได้

ความสามารถในการเชื่อมต่อเครือข่ายนี้เปิดโลกแห่งความเป็นไปได้ใหม่ ๆ ให้กับโครงงาน ESP32 อย่างแท้จริง ไม่ว่าจะเป็นการเก็บข้อมูลขึ้น Cloud, การควบคุมอุปกรณ์จากระยะไกล, หรือการทำให้อุปกรณ์สามารถโต้ตอบกับบริการออนไลน์ต่างๆ ได้

ในบทต่อไป (บทที่ 6) เราจะกลับไปสำรวจโลกของฮาร์ดแวร์อีกครั้ง โดยจะเจาะลึกเรื่องการทำงานกับเซ็นเซอร์ชนิดต่างๆ ทั้งแบบดิจิทัลและแอนะล็อก เพื่อให้ ESP32 ของเราสามารถรับรู้ข้อมูลจากสภาพแวดล้อมรอบตัวได้ละเอียดยิ่งขึ้น ยกตัวอย่างเช่น การอ่านค่าอุณหภูมิ, ความชื้น, ระยะทาง, และอื่น ๆ

บทที่ 6: รู้จักกับเซ็นเซอร์และอุปกรณ์เสริมยอดนิยม

ในบทที่ผ่านมา เราได้เรียนรู้วิธีการทำให้ ESP32 เชื่อมต่อกับเครือข่าย Wi-Fi และสื่อสารผ่านอินเทอร์เน็ต ซึ่งเป็นการเปิดประตูสู่โลกของ Internet of Things (IoT) อย่างไรก็ตาม หัวใจสำคัญอีกอย่างหนึ่งของอุปกรณ์ IoT คือความสามารถในการรับรู้ข้อมูลจากสภาพแวดล้อมรอบตัว อุปกรณ์ที่ทำหน้าที่นี้คือ **เซ็นเซอร์ (Sensor)** และความสามารถในการโต้ตอบกับผู้ใช้หรือสภาพแวดล้อมผ่านอุปกรณ์แสดงผลและอุปกรณ์เอาต์พุตอื่น ๆ

บทนี้จะนำเสนอภาพรวมของเซ็นเซอร์ประเภทต่าง ๆ ที่นิยมใช้งานร่วมกับ ESP32 และ MicroPython, วิธีการค้นหาและติดตั้งไลบรารีที่จำเป็น, การอ่านค่าจากเซ็นเซอร์ดิจิทัลและแอนะล็อก, การสื่อสารกับเซ็นเซอร์ผ่านโปรโตคอล I2C, ภาพรวมของ SPI, การแสดงผลข้อมูลไปยังหน้าจอ OLED, 7-Segment Display และ LED Matrix รวมถึงการควบคุมอุปกรณ์เอาต์พุตเพิ่มเติมเช่น Buzzer และ DC Motor ขนาดเล็ก การทำความเข้าใจวิธีการทำงานและการเชื่อมต่อกับอุปกรณ์เหล่านี้จะช่วยให้ผู้อ่านสามารถสร้างโครงงาน IoT ที่มีความสามารถหลากหลายและตอบสนองต่อสภาวะแวดล้อมได้อย่างชาญฉลาด

6.1 การรับรู้โลกภายนอกด้วยเซ็นเซอร์และปฏิสัมพันธ์กับอุปกรณ์

เซ็นเซอร์คืออะไร? เซ็นเซอร์ คืออุปกรณ์อิเล็กทรอนิกส์ที่ทำหน้าที่ตรวจจับหรือวัดคุณสมบัติทางกายภาพบางอย่างของสภาพแวดล้อม (เช่น แสง, อุณหภูมิ, ความชื้น, การเคลื่อนไหว, ระยะทาง, ความดัน) แล้วแปลงคุณสมบัตินั้นให้อยู่ในรูปของสัญญาณไฟฟ้าที่ไมโครคอนโทรลเลอร์สามารถอ่านและประมวลผลได้

ประเภทของสัญญาณจากเซ็นเซอร์: สัญญาณไฟฟ้าที่ได้จากเซ็นเซอร์สามารถแบ่งออกเป็นประเภทหลักๆ ได้ดังนี้:

1. **สัญญาณดิจิทัล (Digital Signal):** ให้ผลลัพธ์เป็นสถานะเพียงสองอย่าง คือ HIGH (1) หรือ LOW (0)
2. **สัญญาณแอนะล็อก (Analog Signal):** ให้ผลลัพธ์เป็นค่าแรงดันไฟฟ้าที่เปลี่ยนแปลงอย่างต่อเนื่อง ไมโครคอนโทรลเลอร์จำเป็นต้องใช้ ADC (Analog-to-Digital Converter) เพื่อแปลงสัญญาณนี้
3. **สัญญาณผ่านโปรโตคอลการสื่อสาร:** เซ็นเซอร์ที่ซับซ้อนมากขึ้นมักจะสื่อสารผ่านโปรโตคอลมาตรฐาน เช่น I2C, SPI, One-Wire, หรือ UART

ความสำคัญของเซ็นเซอร์และอุปกรณ์เสริมในโครงงาน IoT: เซ็นเซอร์เปรียบเสมือนประสาทสัมผัสของอุปกรณ์ IoT ในขณะที่อุปกรณ์แสดงผลและอุปกรณ์เอาต์พุตอื่นๆ ทำหน้าที่เป็นช่องทางในการโต้ตอบ ทำให้ระบบสามารถ:

- เก็บข้อมูลสภาพแวดล้อมและตรวจจับเหตุการณ์
- ทำงานอัตโนมัติและตัดสินใจตามข้อมูลที่ได้รับ
- แสดงผลข้อมูลให้ผู้ใช้ทราบ
- ส่งสัญญาณเสียงหรือควบคุมการเคลื่อนไหว

6.1.1 วิธีการค้นหาและติดตั้งไลบรารีสำหรับ MicroPython MicroPython มีไลบรารีมาตรฐานมาให้จำนวนหนึ่ง แต่สำหรับเซ็นเซอร์หรือโมดูลเฉพาะทางหลาย ๆ ตัว เราจำเป็นต้องติดตั้งไลบรารีเพิ่มเติมเพื่อให้สามารถใช้งานได้ง่ายขึ้น ไลบรารีเหล่านี้มักจะเขียนด้วย Python (ไฟล์ .py) หรือคอมไพล์เป็น bytecode (ไฟล์ .mpy เพื่อประหยัดพื้นที่และโหลดเร็วขึ้น)

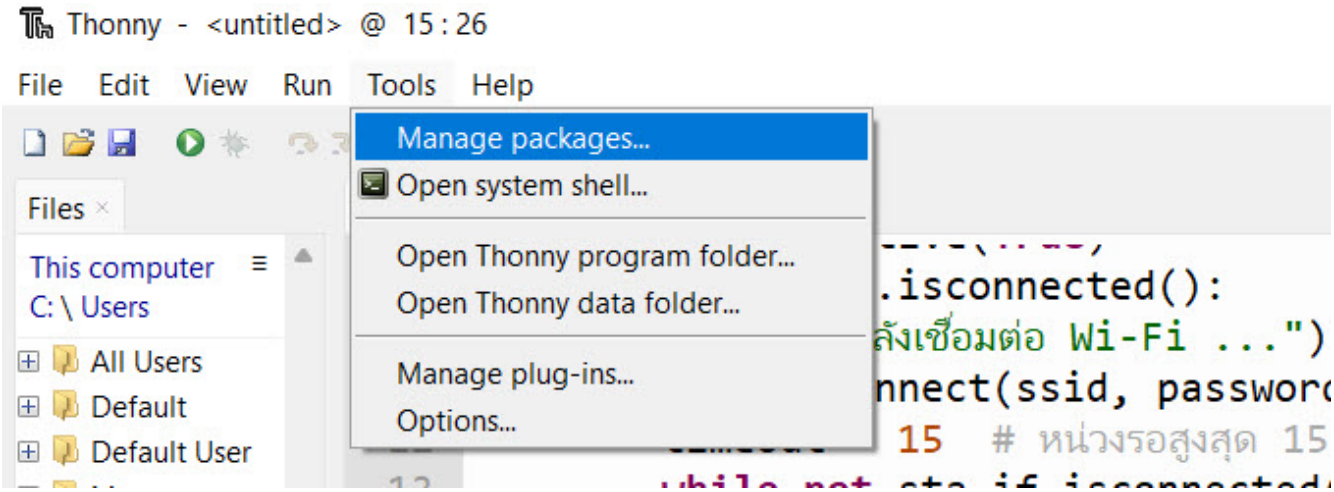
แหล่งค้นหาไลบรารี:

1. **GitHub:** เป็นแหล่งรวมโปรเจกต์ Open Source ขนาดใหญ่ ค้นหาด้วยคำว่า "micropython" ตามด้วยชื่อเซ็นเซอร์หรือชิป เช่น "micropython bme280", "micropython ssd1306"
2. **ฟอรั่ม MicroPython (MicroPython Forum):** ชุมชนผู้ใช้งานมักจะแบ่งปันไลบรารีและโค้ดตัวอย่าง
3. **PyPI (Python Package Index):** แม้ PyPI จะเน้นไลบรารีสำหรับ Python ทั่วไป แต่ก็มีไลบรารี MicroPython จำนวนมากที่เผยแพร่อยู่ที่นี่ (มักจะมีคำว่า micropython- นำหน้าชื่อแพ็กเกจ)
4. **Awesome MicroPython:** เป็นรายการรวบรวมแหล่งข้อมูล, ไลบรารี, และโปรเจกต์ที่เกี่ยวกับ MicroPython (ค้นหา "awesome-micropython" บน GitHub)

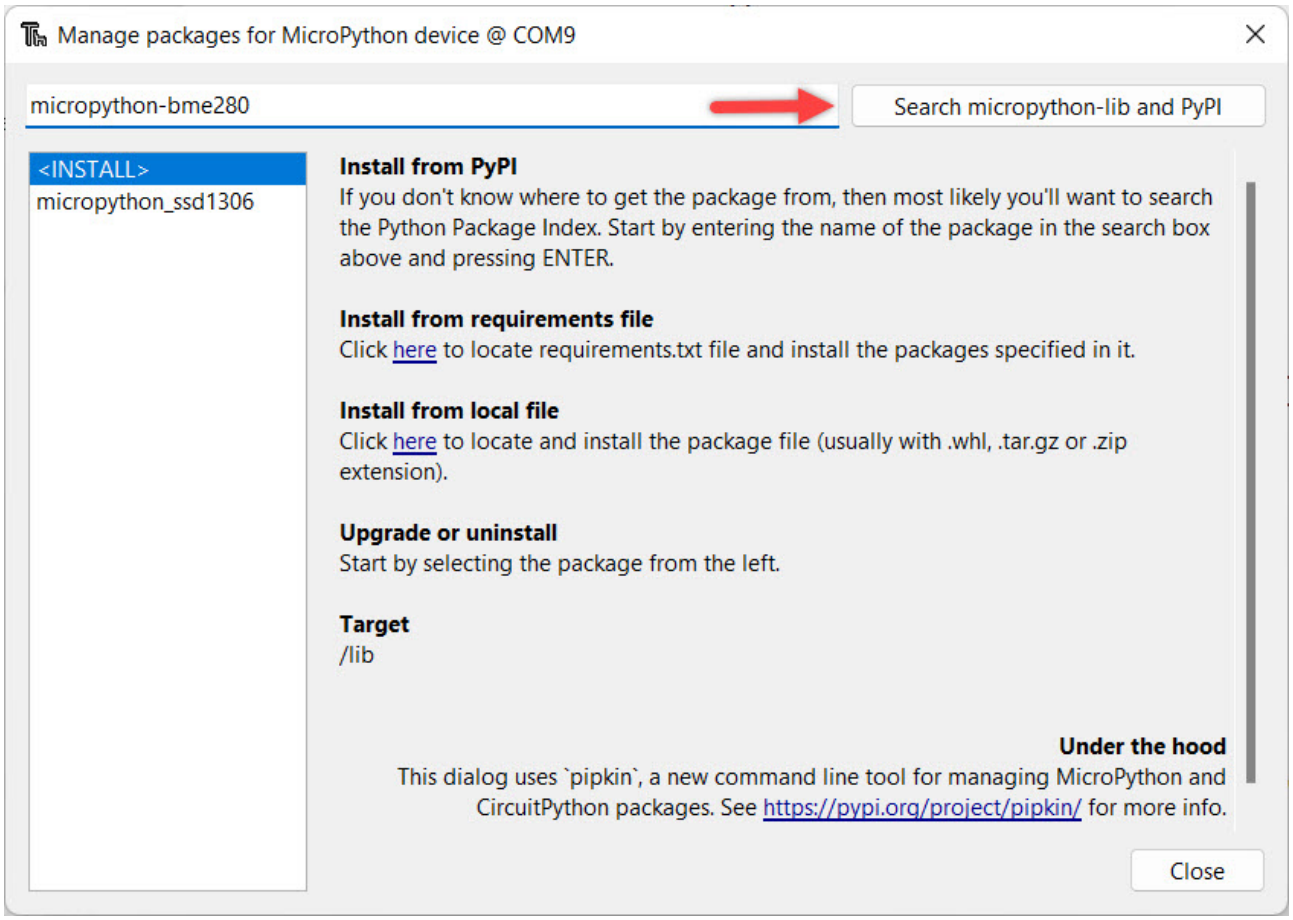
วิธีการติดตั้งไลบรารี

- **วิธีที่ 1: ใช้ Thonny IDE (ง่ายที่สุดสำหรับผู้เริ่มต้น)**

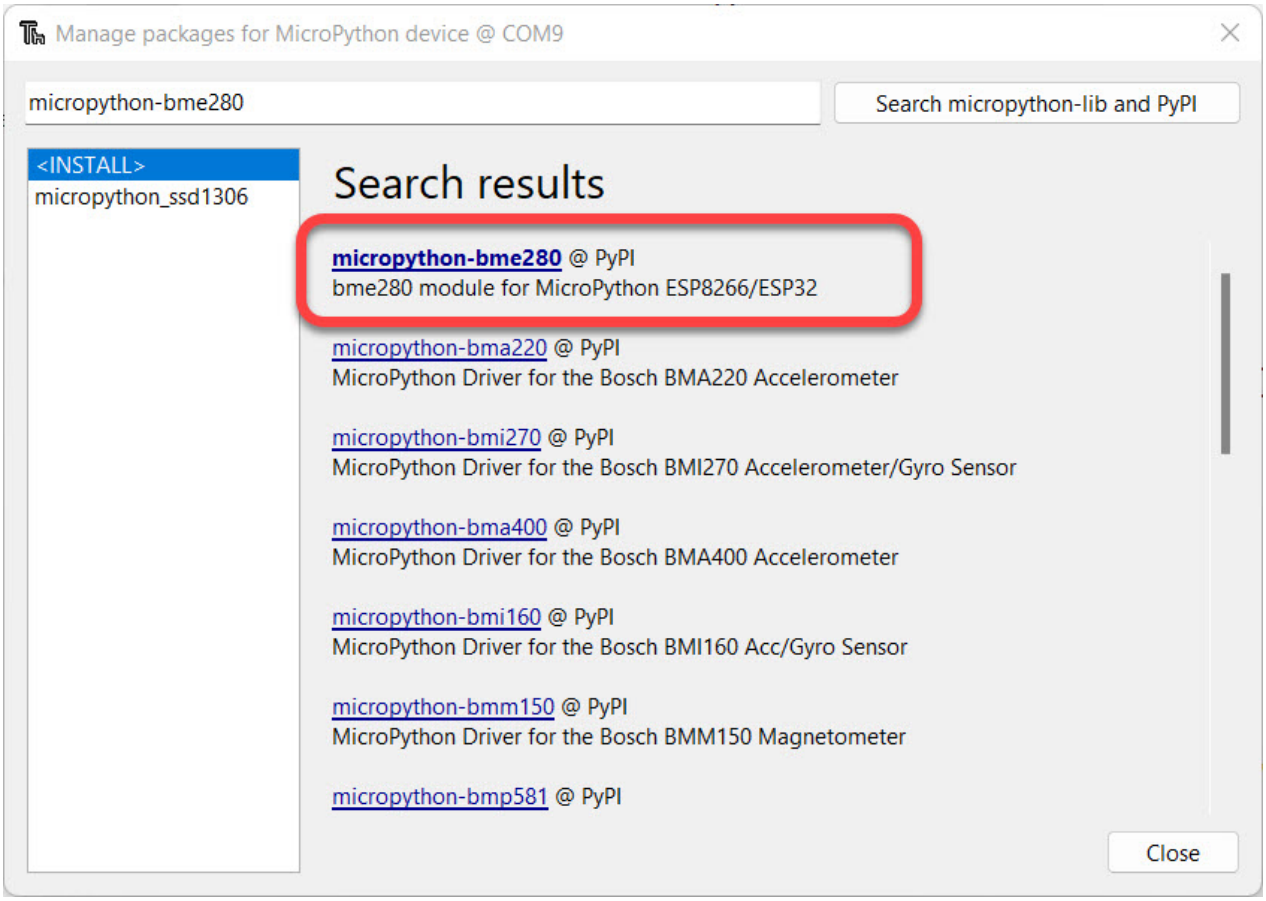
1. ใน Thonny ไปที่เมนู Tools > Manage packages...

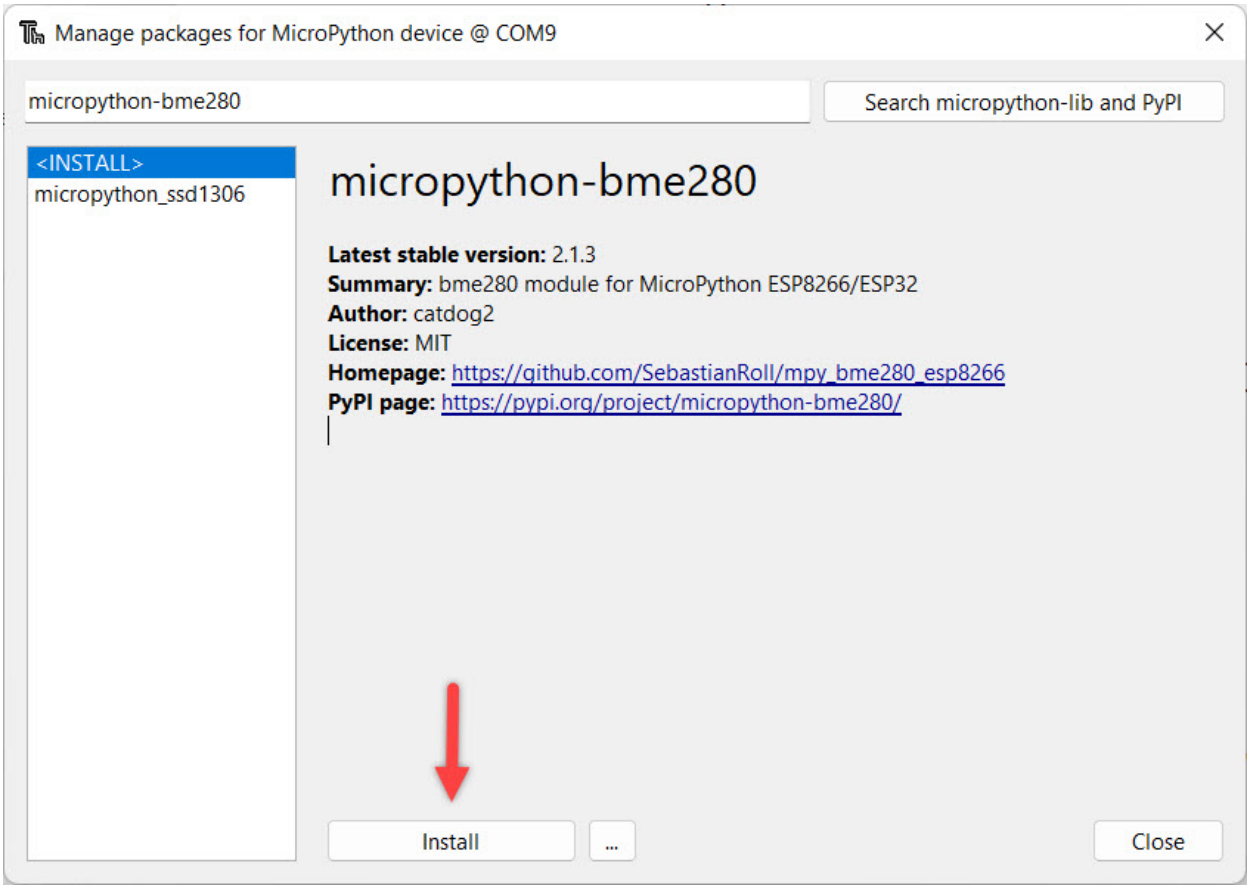


2. ในช่องค้นหา พิมพ์ชื่อไลบรารีที่ต้องการ (เช่น micropython-ssd1306, micropython-bme280)
3. คลิก Search on PyPI

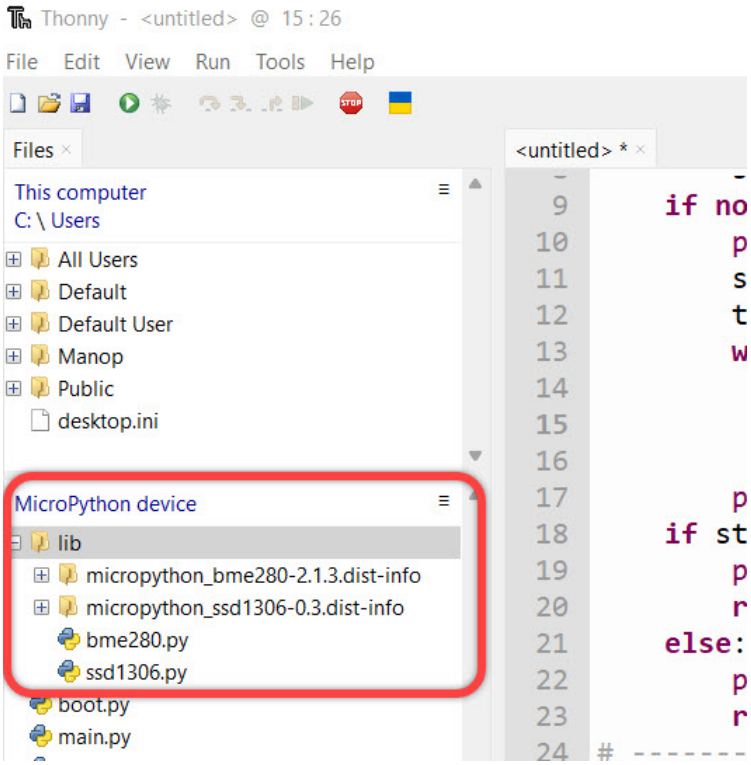


4. เลือกไลบรารีที่ต้องการจากผลการค้นหา แล้วคลิก Install





5. Thonny จะดาวน์โหลดและติดตั้งไลบรารีไปยัง MicroPython device (ESP32) โดยอัตโนมัติ โดยทั่วไปจะอยู่ในโฟลเดอร์ /lib บนอุปกรณ์



- **วิธีที่ 2: คัดลอกไฟล์ไลบรารีไปยังอุปกรณ์ด้วยตนเอง**
    1. ดาวน์โหลดไฟล์ไลบรารี (ที่เป็นไฟล์ .py หรือ .mpy) จากแหล่งต่างๆ เช่น GitHub
    2. ใน Thonny เปิดมุมมองไฟล์ของคอมพิวเตอร์และ MicroPython device (View > Files)
    3. ลากไฟล์ไลบรารีจากคอมพิวเตอร์ไปวางไว้ในโฟลเดอร์ /lib (แนะนำ) หรือโฟลเดอร์ราก (root directory) ของ MicroPython device
    4. หากไลบรารีมีหลายไฟล์หรือมีโครงสร้างโฟลเดอร์ ให้คัดลอกไปทั้งโครงสร้างนั้น
- **วิธีที่ 3: ใช้ upip (สำหรับผู้ใช้งาน Command Line)** upip (MicroPython Package Installer) เป็นเครื่องมือสำหรับติดตั้งแพ็กเกจจาก PyPI โดยตรงบนอุปกรณ์ MicroPython ผ่าน REPL (หากเฟิร์มแวร์มี upip ติดตั้งมาด้วย)

  # >>> import upip

  # >>> upip.install('micropython-ssd1306')

  วิธีนี้อาจจะสะดวกน้อยกว่าสำหรับผู้เริ่มต้นที่ใช้ Thonny เป็นหลัก

  เมื่อติดตั้งไลบรารีแล้ว สามารถ import มาใช้งานในโค้ดได้ตามปกติ

6.2 การอ่านค่าจากเซ็นเซอร์ดิจิทัล

- **PIR Motion Sensor:** หลักการทำงาน, การต่อวงจร, โค้ดตัวอย่าง

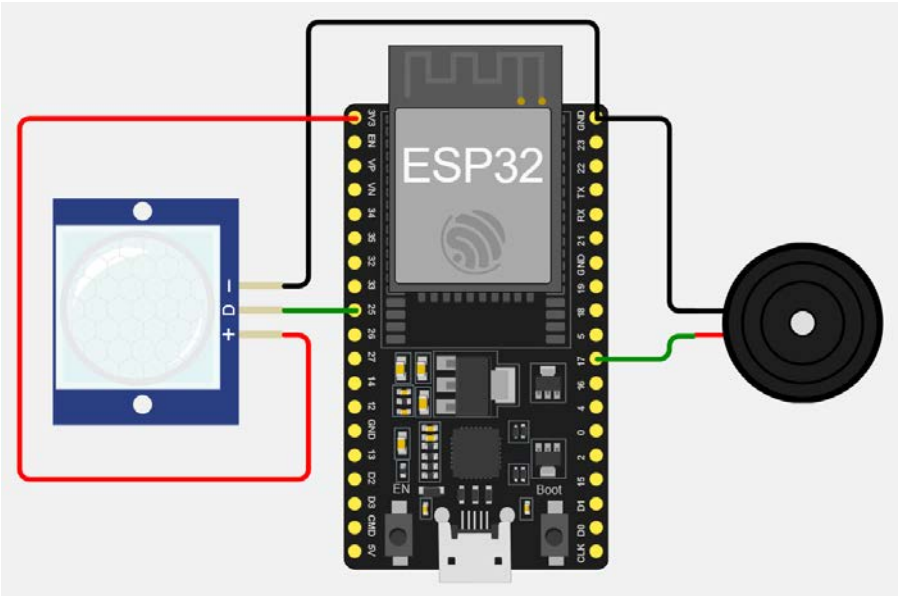
เซ็นเซอร์ตรวจจับความเคลื่อนไหวแบบ PIR (Passive Infrared Sensor) เป็นอุปกรณ์ที่ออกแบบมาเพื่อจับรังสีอินฟราเรดที่ปล่อยออกมาจากร่างกายมนุษย์หรือสิ่งมีชีวิต เมื่อมีสิ่งมีชีวิตเคลื่อนผ่านบริเวณหน้าตัวเซ็นเซอร์ สัญญาณอินฟราเรดที่เซ็นเซอร์รับได้จะเปลี่ยนแปลงทันที ทำให้วงจรอิเล็กทรอนิกส์ภายใน PIR Sensor สร้างสัญญาณดิจิทัลเอาต์พุต (HIGH/LOW) ส่งออกไปยังขา GPIO ของไมโครคอนโทรลเลอร์ เช่น ESP32 สัญญาณนี้สามารถนำไปใช้กระตุ้นให้เกิดการตอบสนองทันที เช่นการสั่งเสียงเตือนเปิดไฟ หรือแจ้งเตือนระบบอัตโนมัติอื่น ๆ

หลักการทำงาน

PIR Motion Sensor จะตรวจจับความแตกต่างของรังสีอินฟราเรดที่ตกกระทบเซ็นเซอร์ เมื่อตรวจพบการเคลื่อนไหว สัญญาณเอาต์พุต OUT จะเปลี่ยนจาก LOW เป็น HIGH ในทันที ซึ่ง ESP32 สามารถอ่านค่านี้ได้จากขาอินพุตที่ต่ออยู่

การต่อวงจร (ให้ตรวจสอบขาใช้งานของ PIR ที่ใช้ ให้แน่ใจ ตำแหน่งขาจริง อาจไม่ตรงกับภาพตัวอย่าง)

- **PIR Sensor:**
  - VCC ⟶ 3.3V ของ ESP32
  - GND ⟶ GND
  - OUT ⟶ GPIO25 (หรือขาอินพุตที่สะดวก)
- **Active Buzzer:**
  - ขั้ว + ต่อ GPIO17 (หรือขา Output ที่สะดวก)
  - ขั้ว – ต่อ GND



ตัวอย่างโค้ด MicroPython สั่งงาน Buzzer เมื่อพบการเคลื่อนไหว

```
from machine import Pin
import time
pir = Pin(25, Pin.IN)         # ขาอินพุตจาก PIR Sensor
buzzer = Pin(17, Pin.OUT)     # ขาเอาต์พุตสำหรับ Buzzer
while True:
    if pir.value() == 1:    # ตรวจจับการเคลื่อนไหว
        buzzer.value(1)      # เปิดเสียงเตือน
        time.sleep(0.5)      # ส่งเสียง 0.5 วินาที
        buzzer.value(0)      # ปิดเสียงเตือน
        time.sleep(0.5)      # หน่วงก่อนอ่านรอบถัดไป
    else:
        buzzer.value(0)      # ปิดเสียงหากไม่มีการเคลื่อนไหว
        time.sleep(0.1)
```

คำอธิบาย

- สร้างออบเจกต์ pir สำหรับอ่านสถานะจากเซ็นเซอร์ตรวจจับความเคลื่อนไหว
- สร้างออบเจกต์ buzzer สำหรับควบคุมเสียงเตือน
- ทุกครั้งที่ PIR ตรวจพบการเคลื่อนไหว (pir.value() == 1) จะสั่งให้ buzzer ดัง 0.5 วินาที แล้วหยุด 0.5 วินาที
- ถ้าไม่พบการเคลื่อนไหว buzzer จะเงียบ

ข้อควรระวัง

- ใช้ Active Buzzer สำหรับโค้ดนี้ เพราะควบคุมเปิด/ปิดด้วยลอจิก 0 หรือ 1 ได้โดยตรง
- ตรวจสอบขา GPIO ที่ใช้กับบอร์ดจริง และเปลี่ยนเลขขาในโค้ดให้ตรงกับวงจรที่ต่อใช้งานจริง
- หากใช้งานร่วมกับระบบแจ้งเตือนหรือไฟ อาจต่ออุปกรณ์ควบคุมอื่นเพิ่มตามต้องการ

การเชื่อมต่อ PIR Sensor กับเสียงเตือนในลักษณะนี้ถือเป็นตัวอย่างสำคัญที่แสดงถึงการทำงานของระบบอัตโนมัติในงาน IoT และระบบฝังตัว ซึ่งสามารถต่อยอดไปสู่โปรเจกต์รักษาความปลอดภัย สมาร์ทโฮม หรือระบบตอบสนองอัตโนมัติอื่น ๆ ได้อย่างมีประสิทธิภาพ

6.3 การอ่านค่าจากเซ็นเซอร์แอนะล็อกด้วย ADC

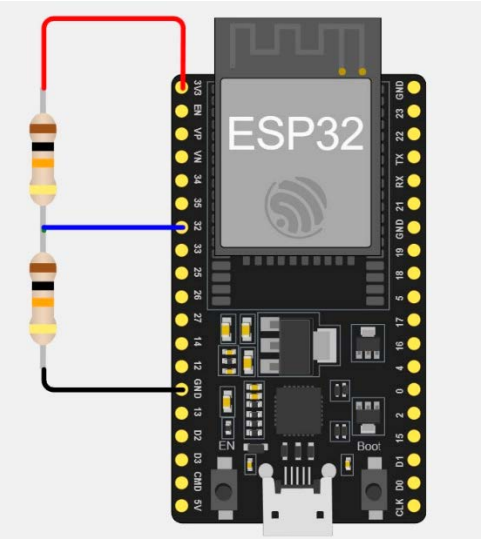
ในระบบไมโครคอนโทรลเลอร์อย่าง ESP32 การรับข้อมูลจากเซ็นเซอร์ประเภทแอนะล็อก เช่น โพเทนชิโอมิเตอร์ (Potentiometer), ตัวต้านทานแสง (LDR), เซ็นเซอร์อุณหภูมิแบบแอนะล็อก ฯลฯ จำเป็นต้องใช้วงจร “แปลงสัญญาณแอนะล็อกเป็นดิจิทัล” หรือที่เรียกว่า **ADC (Analog-to-Digital Converter)**

ADC จะทำหน้าที่แปลงแรงดันไฟฟ้าต่อเนื่อง (Analog Voltage) ที่ได้จากเซ็นเซอร์ ให้เป็น “ค่าตัวเลขดิจิทัล” ที่ไมโครคอนโทรลเลอร์สามารถประมวลผลได้

พื้นฐานของ ADC ใน ESP32

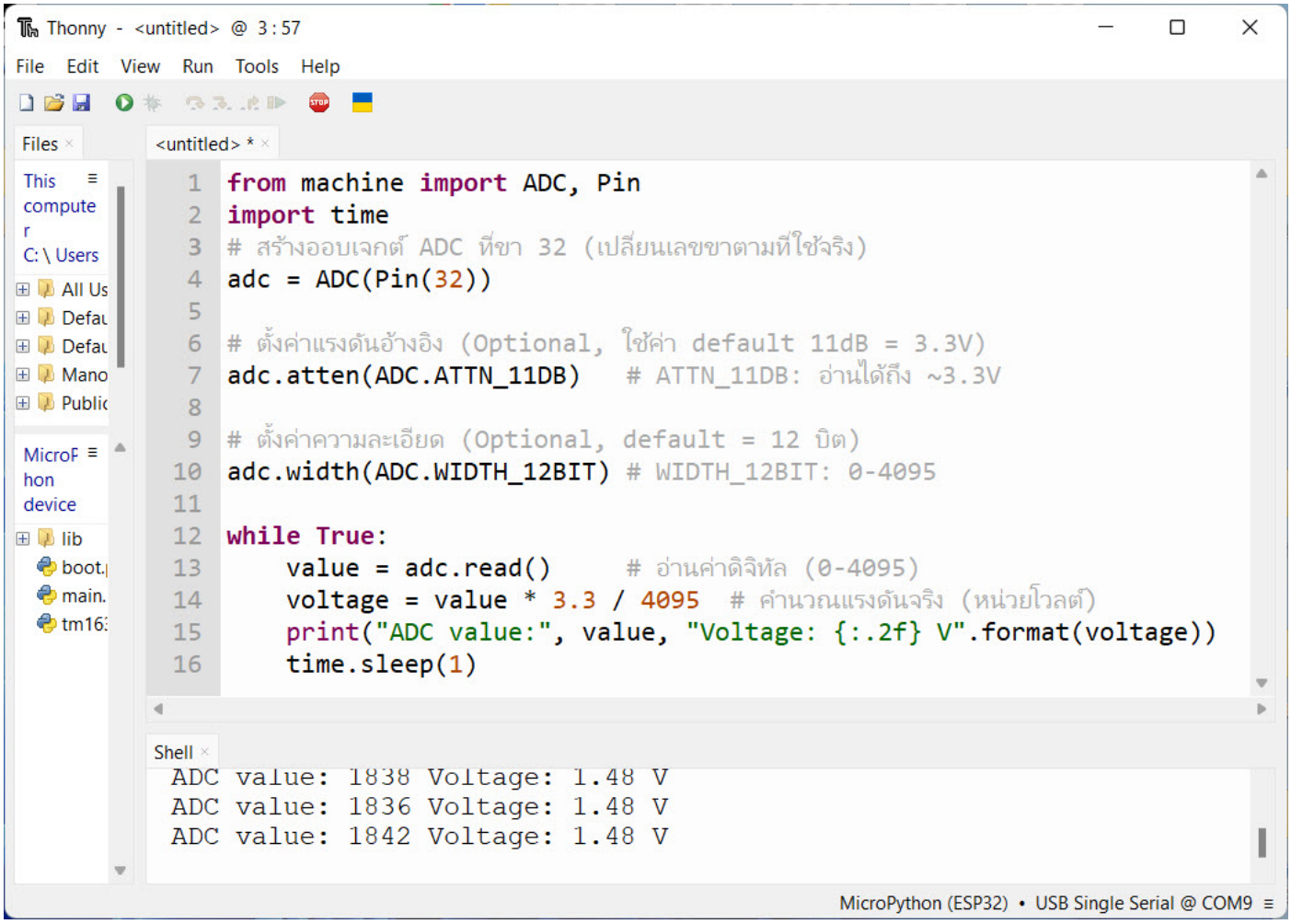
- ESP32 มี ADC อยู่ในตัว สามารถอ่านแรงดันไฟฟ้าจากขาต่าง ๆ ได้โดยตรง
- รองรับ ADC 12 บิต (0–4095) แต่ในบางเฟิร์มแวร์อาจถูกลดเหลือ 9 หรือ 10 บิต ขึ้นกับการกำหนดค่า
- แรงดันไฟฟ้าสูงสุดที่อ่านได้ (Vref) ปกติคือ 3.3V (บางบอร์ดอาจต่างกัน)
- ขา ADC ที่รองรับ: GPIO32, GPIO33, GPIO34, GPIO35, GPIO36, GPIO39 ฯลฯ (ขึ้นกับรุ่นและโมดูล)

- **การใช้งาน machine.ADC (MicroPython)**



วงจรทดสอบเบื้องต้น หรืออาจต่อแรงดัน 3.3V เข้าที่ขาอินพุตได้โดยตรงก็จะอ่านค่าแรงดันไฟออกมาได้

โมดูล machine ใน MicroPython มีฟังก์ชันสำหรับอ่านค่า ADC ได้ง่าย ตัวอย่างการอ่านค่าจากขา ADC



คำอธิบาย

- ADC(Pin(32)) = เลือกขา ADC ที่จะอ่านค่า (เลือก GPIO32 ในตัวอย่าง)
- adc.atten(ADC.ATTN_11DB) = ตั้งค่าAttenuation(attenuate) เป็น 11dB เพื่ออ่านแรงดันได้ถึง 3.3V
  - **ATTN_0DB**: 1.0V
  - **ATTN_2_5DB**: 1.34V
  - **ATTN_6DB**: 2.0V
  - **ATTN_11DB**: 3.3V
- adc.width(ADC.WIDTH_12BIT) = กำหนดความละเอียด 12 บิต (0–4095)
- adc.read() = อ่านค่าดิจิทัลที่ได้จากเซ็นเซอร์ (0–4095)
- สามารถแปลงค่าเป็นแรงดันไฟฟ้าจริงได้ ด้วยสูตร: voltage = value * 3.3 / 4095

ข้อควรระวัง

- **ขา ADC บางขา** อาจใช้เป็นอินพุตดิจิทัลไม่ได้ หรือใช้กับวงจรอื่นร่วมไม่ได้ (เช็ค datasheet ของบอร์ด)
- **สัญญาณรบกวน**: การอ่านค่าสัญญาณแอนะล็อก ควรเดินสายสั้น ใช้กราวด์ดี ไม่ผ่านสัญญาณรบกวนแรง

- ถ้าแรงดันสูงเกิน 3.3V อาจทำให้ขา ADC หรือบอร์ดเสียหาย (ควรใช้วงจรแบ่งแรงดัน/ตัวต้านทานถ้าจำเป็น)

ตัวอย่างการใช้งานจริง: การอ่านค่าโพเทนชิโอมิเตอร์

ต่อขากลางของโพเทนชิโอมิเตอร์เข้าขา ADC (เช่น GPIO32) ขาอีกสองขาเข้ากับ 3.3V และ GND โปรแกรมจะอ่านค่าแอนะล็อกได้ตั้งแต่ 0–4095 เมื่อหมุนโพเทนชิโอมิเตอร์

- **ตัวอย่าง: Potentiometer**

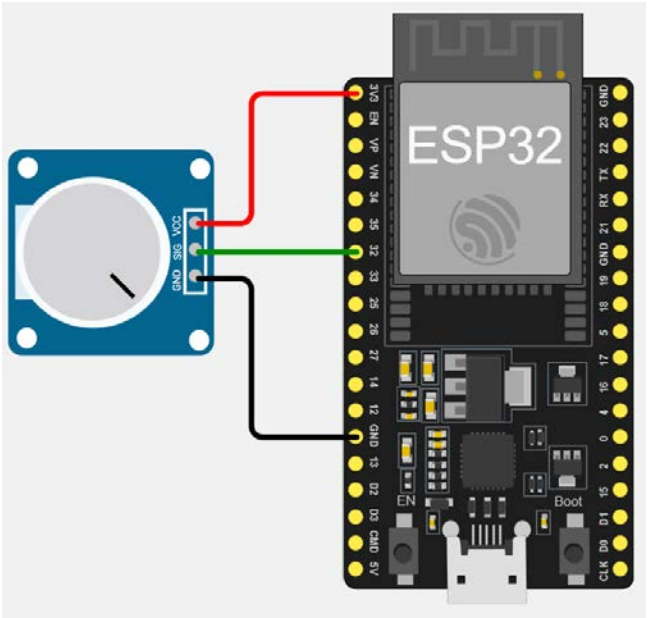
  ตัวอย่าง: การอ่านค่าโพเทนชิโอมิเตอร์ (Potentiometer) ด้วย ESP32 และ MicroPython

  โพเทนชิโอมิเตอร์ (Potentiometer) เป็นอุปกรณ์ปรับค่าแรงดันแอนะล็อกที่นิยมใช้มากที่สุดในการทดลองไมโครคอนโทรลเลอร์ ไม่ว่าจะเป็นการทดลองปรับความสว่าง LED, ควบคุมความเร็วมอเตอร์, หรือรับค่าป้อนกลับ (Feedback) จากผู้ใช้ หลักการของโพเทนชิโอมิเตอร์คือ “ตัวต้านทานปรับค่าได้” โดยมีขา 3 ขา ได้แก่ ขา 1 ต่อกับแรงดันไฟฟ้า (VCC), ขา 2 เป็นขากลาง (Wiper) ซึ่งจะจ่ายแรงดันไฟฟ้าแปรผันตามการหมุน และขา 3 ต่อกับกราวด์ (GND)

การเชื่อมต่อกับ ESP32

1. ขา 1 (VCC) ⟶ 3.3V บนบอร์ด ESP32 (ไม่ควรต่อ 5V)
2. ขา 2 (Wiper) ⟶ ขา ADC ของ ESP32 (เช่น GPIO32)
3. ขา 3 (GND) ⟶ GND ของบอร์ด ESP32

ผังวงจรตัวอย่าง:



โค้ดตัวอย่าง:

```
from machine import ADC, Pin
import time
adc = ADC(Pin(32))
```

```
adc.atten(ADC.ATTN_11DB)      # อ่านแรงดันได้ถึง 3.3V
adc.width(ADC.WIDTH_12BIT)    # ตั้งค่าความละเอียด 12 บิต (0-4095)
while True:
    value = adc.read()        # อ่านค่าตัวเลขดิจิทัล (0-4095)
    print("ADC Value:", value)
    time.sleep(0.2)
```

คำอธิบายโค้ด:

• สร้างออบเจกต์ ADC ที่ขา 32 สำหรับรับสัญญาณแอนะล็อก

• กำหนด attenuation ให้สามารถอ่านแรงดันขาอินพุตได้สูงสุดประมาณ 3.3V

• ตั้งค่าความละเอียดของ ADC เป็น 12 บิต ทำให้ค่าที่อ่านได้อยู่ในช่วง 0–4095

• ในลูปหลักของโปรแกรม จะอ่านค่าตัวเลข ADC จากโพเทนชิโอมิเตอร์โดยตรง

• ค่าที่ได้จะเปลี่ยนแปลงตามการหมุนโพเทนชิโอมิเตอร์ โดยปลายข้างหนึ่งจะได้ค่าใกล้ 0 และปลายอีกข้างจะได้ค่าใกล้ 4095

ข้อควรระวังและเทคนิคเสริม:

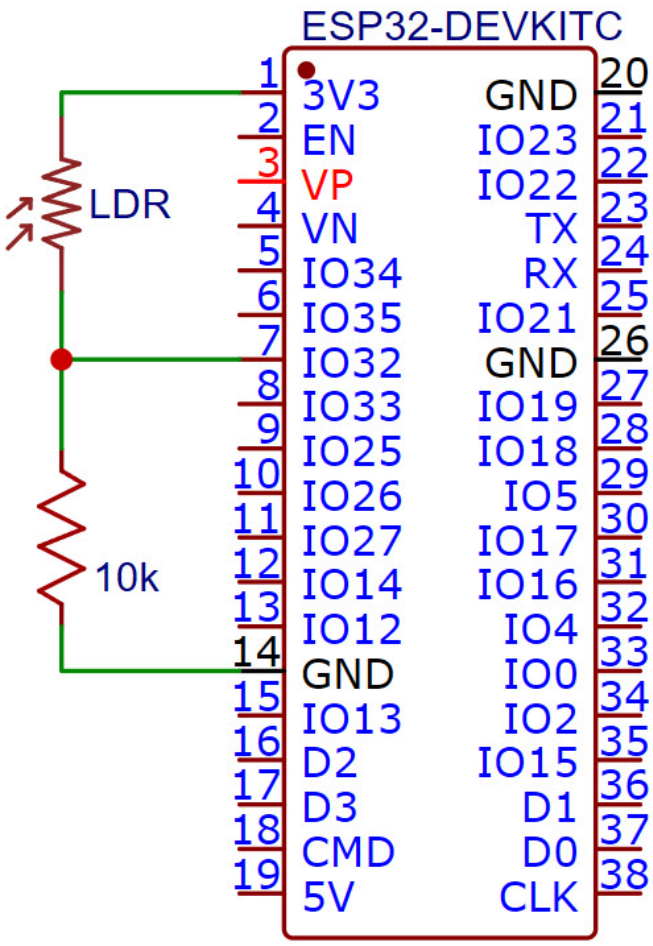
- ควรต่อขากลาง (Wiper) ตรงไปยังขา ESP32 ที่รองรับการทำงาน ADC เท่านั้น
- หลีกเลี่ยงการต่อขา VCC เข้ากับ 5V เพราะจะทำให้ ADC เสียหาย
- สามารถนำค่าที่อ่านได้ไปใช้งานต่อ เช่น ควบคุมความสว่าง LED, ส่งข้อมูลไปยัง Web Server ฯลฯ

ประโยชน์ของการทดลองนี้

ผู้อ่านจะเข้าใจหลักการรับสัญญาณแอนะล็อกและการแปลงค่าเป็นดิจิทัลอย่างลึกซึ้ง โดยเห็นชัดเจนจากการหมุนโพเทนชิโอมิเตอร์แล้วค่า ADC ของ ESP32 เปลี่ยนแปลงตามทันที สิ่งนี้สามารถประยุกต์ใช้ได้ตั้งแต่ระดับพื้นฐาน เช่น การควบคุมความสว่าง LED การปรับความเร็วมอเตอร์ หรือการทำวอลุ่มเสียง ไปจนถึงระดับขั้นสูงที่ใช้โพเทนชิโอมิเตอร์เป็นส่วนหนึ่งของระบบควบคุมฟีดแบ็ก เช่น การกำหนดตำแหน่งอ้างอิงในเซอร์โวมอเตอร์ การปรับค่าพารามิเตอร์ในระบบควบคุมอัตโนมัติ หรือการตั้งค่าตัวแปรสำคัญในระบบ IoT ที่ส่งข้อมูลขึ้นคลาวด์เพื่อใช้ร่วมกับแดชบอร์ดควบคุมระยะไกล การทดลองนี้จึงทำให้เห็นทั้งมิติของการเรียนรู้พื้นฐานและศักยภาพในการนำไปต่อยอดสู่การสร้างผลิตภัณฑ์อัจฉริยะที่ซับซ้อนได้จริง

- **ตัวอย่าง: Photoresistor (LDR)**

การเชื่อมต่อ LDR กับ ESP32



ในการใช้งาน LDR (Light Dependent Resistor) เพื่อวัดความเข้มแสงด้วย ESP32 จำเป็นต้องนำ LDR มาต่ออนุกรมกับตัวต้านทานมาตรฐาน (เช่น 10kΩ) เพื่อสร้างวงจรแบ่งแรงดัน (Voltage Divider) โดยนำขาหนึ่งของ LDR ต่อเข้ากับแรงดันไฟฟ้า 3.3V และนำขาอีกข้างต่อกับขาของตัวต้านทาน จากนั้นปลายอีกข้างของตัวต้านทานต่อเข้ากับกราวด์ (GND) จุดสำคัญคือ ให้นำขาจุดเชื่อมระหว่าง LDR กับตัวต้านทานต่อเข้ากับขา ADC ของ ESP32 เมื่อต่อวงจรในลักษณะนี้ จะทำให้แรงดันไฟฟ้าที่ขา ADC เปลี่ยนแปลงไปตามปริมาณแสงที่ตกกระทบ LDR กล่าวคือ เมื่อมีแสงมาก LDR จะมีความต้านทานต่ำ ทำให้แรงดันที่ขา ADC สูงขึ้น ในขณะที่เมื่อแสงน้อย LDR จะมีความต้านทานสูง ส่งผลให้แรงดันที่ขา ADC ต่ำลง วิธีการเชื่อมต่อแบบนี้จึงเป็นมาตรฐานในการนำ LDR มาใช้งานกับวงจรไมโครคอนโทรลเลอร์ทุกประเภท

6.4 การสื่อสารกับเซ็นเซอร์ผ่าน I2C

- **I2C คืออะไร?**

I2C (อ่านว่า “ไอ-ทู-ซี” หรือ “ไอ-สแควร์-ซี”) เป็นชื่อย่อจากคำว่า **Inter-Integrated Circuit** เป็นมาตรฐานการสื่อสารแบบอนุกรม (Serial Communication Protocol) ที่ออกแบบโดยบริษัท Philips (ปัจจุบันคือ NXP Semiconductors) สำหรับการเชื่อมต่อและแลกเปลี่ยนข้อมูลระหว่างอุปกรณ์อิเล็กทรอนิกส์หลายตัวบนแผงวงจรเดียวกัน เช่น ไมโครคอนโทรลเลอร์ เซ็นเซอร์ หน่วยความจำ EEPROM ชิป RTC หน้าจอ LCD ฯลฯ

ลักษณะเด่นของ I2C

I2C เป็นมาตรฐานการสื่อสารที่ใช้สายสัญญาณหลักเพียงสองเส้น ได้แก่สายข้อมูล (SDA: Serial Data) และสายสัญญาณนาฬิกา (SCL: Serial Clock) ซึ่งช่วยให้สามารถเชื่อมต่ออุปกรณ์อิเล็กทรอนิกส์หลายตัวเข้าด้วยกัน

บนบัสเดียวได้อย่างสะดวก อุปกรณ์แต่ละชิ้นที่อยู่บนบัส I2C จะถูกกำหนดหมายเลขที่อยู่ (Address) เฉพาะตัว ทำให้สามารถติดต่อกับอุปกรณ์แต่ละตัวได้โดยไม่เกิดความซ้ำซ้อน แม้จะเชื่อมต่ออุปกรณ์จำนวนมากในระบบเดียวกันก็ตาม ในทางปฏิบัติ บัส I2C สามารถรองรับอุปกรณ์ได้หลายสิบตัวพร้อมกัน ขึ้นอยู่กับข้อจำกัดด้านไฟฟ้าและจำนวน Address ที่สามารถใช้ได้ภายในระบบ การสื่อสารในบัส I2C อุปกรณ์แต่ละตัวจะทำหน้าที่แตกต่างกัน โดยมีอุปกรณ์ที่ทำหน้าที่ควบคุมการสื่อสาร เรียกว่า Master ซึ่งมักเป็นไมโครคอนโทรลเลอร์ เช่น ESP32 หรือ Arduino ส่วนอุปกรณ์ที่รอรับคำสั่งและตอบสนองเมื่อถูกเรียกจะทำหน้าที่เป็น Slave ได้แก่ เซ็นเซอร์ หน่วยความจำ EEPROM หรือโมดูลนาฬิกา RTC

มาตรฐาน I2C อนุญาตให้มี Master ได้มากกว่าหนึ่งตัวในระบบเดียวกัน (multi-master) แต่ในการใช้งานทั่วไปจะกำหนดให้มี Master เพียงตัวเดียวเพื่อความสะดวกในการควบคุมข้อมูล ข้อมูลที่ส่งผ่านบัส I2C จะถูกแบ่งออกเป็นข้อมูลทีละ 1 ไบต์ โดยการรับส่งข้อมูลจะอาศัยสัญญาณนาฬิกาจาก Master เป็นตัวกำหนดจังหวะในการสื่อสารเสมอ

ข้อดีของ I2C

- ใช้สายสัญญาณน้อย ประหยัดพื้นที่และสะดวกในการเดินสาย
- สามารถต่ออุปกรณ์หลายตัวได้โดยใช้สายชุดเดียว
- เหมาะกับงานที่ต้องติดต่อกับเซ็นเซอร์หรือชิปต่อพ่วงจำนวนมาก
- ไลบรารีซอฟต์แวร์สำหรับควบคุม I2C มีให้เลือกใช้งานครบทุกภาษาและแพลตฟอร์ม

ข้อจำกัดของ I2C

- ความเร็วสูงสุดถูกจำกัด (มาตรฐานทั่วไปคือ 100kHz, Fast Mode 400kHz, บางรุ่นได้ถึง 1MHz)
- ไม่เหมาะกับการส่งข้อมูลระยะไกล (สายสัญญาณควรสั้นเพื่อป้องกันสัญญาณรบกวน)
- ต้องใช้ตัวต้านทาน Pull-up ที่สาย SDA และ SCL เพื่อให้สัญญาณทำงานได้ถูกต้อง

การนำ I2C มาใช้งานกับไมโครคอนโทรลเลอร์ได้รับความนิยมอย่างแพร่หลาย โดยเฉพาะในการเชื่อมต่อกับเซ็นเซอร์และอุปกรณ์เสริมต่าง ๆ เช่น เซ็นเซอร์วัดอุณหภูมิและความชื้น BME280, เซ็นเซอร์วัดความเร่ง MPU6050, หน้าจอแสดงผล OLED, โมดูลนาฬิกา RTC DS3231 และชิปหน่วยความจำ EEPROM เป็นต้น สำหรับ ESP32 ที่ใช้งานร่วมกับ MicroPython ผู้ใช้สามารถเลือกกำหนดขา SDA และ SCL ได้อย่างยืดหยุ่นตามความเหมาะสมของวงจร ทั้งนี้ยังสามารถใช้ฟังก์ชัน Software I2C เพื่อกำหนดขาใดก็ได้ให้ทำหน้าที่เป็น

สายสัญญาณ I2C ตามต้องการ ทำให้การต่ออุปกรณ์เสริมและการออกแบบวงจรมีความสะดวกและเหมาะสมกับทุกโปรเจกต์

I2C คือโปรโตคอลสื่อสารมาตรฐานที่ใช้เพียงสองสายสัญญาณหลักในการเชื่อมต่ออุปกรณ์อิเล็กทรอนิกส์หลายตัวเข้ากับไมโครคอนโทรลเลอร์บนบัสเดียวกันได้อย่างสะดวกและมีประสิทธิภาพ เหมาะสำหรับการใช้งานในระบบ IoT และงานระบบอัตโนมัติหรืออุปกรณ์อัจฉริยะหลากหลายประเภท

“เพื่อไม่ให้เนื้อหาในหนังสือเล่มนี้ซับซ้อนจนเกินไป จะไม่ขยายไปถึงรายละเอียดเชิงเทคนิคของโปรโตคอล I²C เช่น การอธิบายบิต Start และ Stop การตอบสนอง ACK/NACK การวิเคราะห์ Timing ของบัส โครงสร้างเฟรมข้อมูล วิธีการ Arbitration การจัดการความเร็วบัส (Standard, Fast, High-Speed) หรือการออกแบบฮาร์ดแวร์ให้รองรับหลายอุปกรณ์ เพราะสิ่งเหล่านี้เป็นหัวข้อเชิงลึกที่เหมาะสมกับการศึกษาเฉพาะทางในด้านโปรโตคอลสื่อสารโดยตรง ดังนั้น ในบทนี้จะมุ่งเน้นเพียงการใช้งาน I²C บน ESP32 ด้วย MicroPython เพื่อเชื่อมต่อและสื่อสารกับอุปกรณ์เสริม เช่น จอ OLED หรือเซ็นเซอร์เท่านั้น และหากผู้อ่านสนใจสามารถศึกษาเพิ่มเติมได้จากหนังสือที่ว่าด้วยโปรโตคอลการสื่อสารโดยเฉพาะ ซึ่งจะอธิบายกลไกเหล่านี้อย่างเป็นระบบและครบถ้วน”

ในการใช้งาน I2C บนบอร์ด ESP32 ร่วมกับ MicroPython ผู้ใช้งานสามารถกำหนดขา SDA และ SCL ได้อย่างยืดหยุ่นตามความต้องการ เนื่องจาก ESP32 รองรับ I2C ได้แทบทุกขา GPIO แต่เพื่อความสะดวกในการเริ่มต้นและลดปัญหาที่อาจเกิดขึ้นจากการชนกับฟังก์ชันอื่น ขาที่นิยมใช้สำหรับตัวอย่างและงานทั่วไปคือ GPIO21 สำหรับสาย SDA และ GPIO22 สำหรับสาย SCL ซึ่งเป็นขาค่าเริ่มต้น (default) ที่ถูกกำหนดไว้ในบอร์ดส่วนใหญ่ อย่างไรก็ตาม หากต้องการใช้งานร่วมกับอุปกรณ์เสริมหรือโปรเจกต์ที่มีการจองขาเหล่านี้ไปแล้ว MicroPython ยังอนุญาตให้ผู้ใช้กำหนดขาอื่นใดให้เป็น SDA หรือ SCL ได้ตามสะดวก โดยกำหนดไว้ขณะสร้างออบเจกต์ I2C ในโค้ด ซึ่งเป็นข้อดีสำคัญของ ESP32 และช่วยให้การออกแบบวงจรและการต่อโมดูลต่าง ๆ มีความยืดหยุ่นสูงสุด

ตัวอย่างการกำหนดขา SDA และ SCL สำหรับใช้งาน I2C ใน MicroPython

```
from machine import Pin, I2C
i2c = I2C(0, scl=Pin(22), sda=Pin(21), freq=400000)  # SCL = GPIO22, SDA = GPIO21
```

การเลือกใช้ขา SDA / SCL ดังกล่าว จะทำให้ผู้อ่านสามารถต่อวงจรและทดสอบตัวอย่างในหนังสือได้ตรงตามคำอธิบายทุกขั้นตอน และยังสามารถปรับเปลี่ยนขาได้ตามเงื่อนไขของแต่ละโปรเจกต์ในอนาคต

- **การใช้งาน machine.I2C และการสแกนอุปกรณ์**

การใช้งาน I2C บน MicroPython สำหรับ ESP32 สามารถทำได้โดยใช้คลาส machine.I2C ซึ่งช่วยให้การเชื่อมต่อและสื่อสารกับอุปกรณ์ I2C เป็นไปอย่างสะดวกและยืดหยุ่น ในการเริ่มต้นใช้งาน ผู้ใช้จะต้องสร้างออบเจกต์ I2C ขึ้นมาก่อน โดยกำหนดขา SDA และ SCL ตามที่ต้องการ จากนั้นสามารถใช้ฟังก์ชัน scan() เพื่อตรวจสอบว่าในระบบมีอุปกรณ์ I2C เชื่อมต่ออยู่กี่ตัว และแต่ละตัวใช้หมายเลข Address อะไรบ้าง

ตัวอย่างการสร้างออบเจกต์ I2C และสแกนอุปกรณ์บน ESP32 ด้วย MicroPython

```
from machine import Pin, I2C

# สร้างออบเจกต์ I2C โดยใช้ขา SCL = GPIO22, SDA = GPIO21
i2c = I2C(0, scl=Pin(22), sda=Pin(21))

# สแกนหาอุปกรณ์ I2C ที่เชื่อมต่ออยู่ในบัส
devices = i2c.scan()

if len(devices) == 0:
    print("ไม่พบอุปกรณ์ I2C ใด ๆ")
else:
    print("พบอุปกรณ์ I2C ทั้งหมด", len(devices), "ตัว")
    for d in devices:
        print("Address: 0x{:02X}".format(d))
```

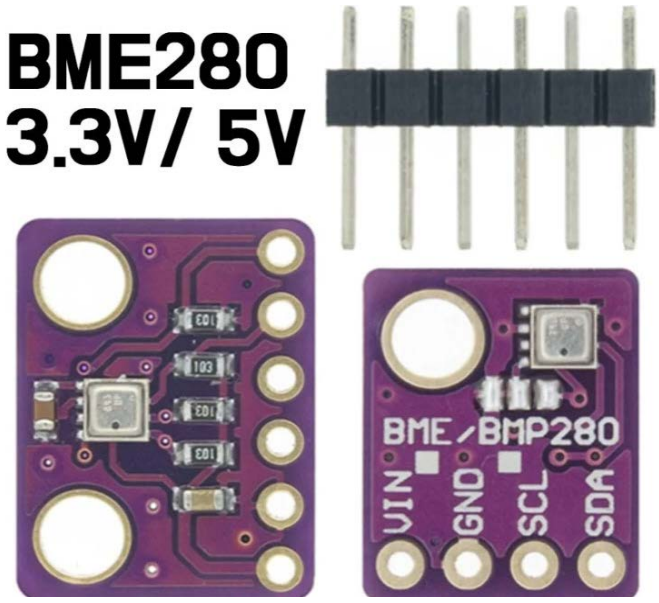
คำอธิบายโค้ด:

- สร้างออบเจกต์ I2C โดยกำหนดขา SDA เป็น GPIO21 และ SCL เป็น GPIO22
- เรียกใช้ฟังก์ชัน scan() เพื่อตรวจสอบว่าในบัส I2C มีอุปกรณ์เชื่อมต่ออยู่กี่ตัว และมี Address อะไรบ้าง
- Address ของอุปกรณ์ที่ตรวจพบจะแสดงผลในรูปแบบเลขฐานสิบหก (Hexadecimal) ซึ่งสามารถนำไปใช้ในการติดต่อกับอุปกรณ์แต่ละตัวได้

การสแกนอุปกรณ์ I2C เป็นขั้นตอนสำคัญในการทดสอบระบบก่อนเขียนโปรแกรมควบคุมจริง เพราะจะช่วยให้มั่นใจว่าอุปกรณ์ I2C ถูกเชื่อมต่อถูกต้องและสามารถสื่อสารกับ ESP32 ได้ หากพบ Address ตรงกับข้อมูลในเอกสารของเซ็นเซอร์หรือโมดูลที่ใช้งาน แสดงว่าระบบพร้อมสำหรับการเขียนโปรแกรมขั้นต่อไป

- ตัวอย่าง: เซ็นเซอร์อุณหภูมิและความชื้น BME280 (ใช้ไลบรารี)

BME280 เป็นเซ็นเซอร์ดิจิทัลที่ใช้วัดอุณหภูมิ ความชื้นสัมพัทธ์ และความกดอากาศในอุปกรณ์เดียวกัน รองรับการเชื่อมต่อด้วยโปรโตคอล I2C ซึ่งต่อใช้งานกับ ESP32 ได้โดยตรง และมีไลบรารี MicroPython ให้ใช้งานอย่างสะดวก



การต่อวงจร (I2C):

- VCC ต่อกับ 3.3V ของ ESP32
- GND ต่อกับ GND ของ ESP32
- SDA ต่อกับ GPIO21 ของ ESP32
- SCL ต่อกับ GPIO22 ของ ESP32

ขั้นตอนใช้งานใน MicroPython:

1. ดาวน์โหลดไฟล์ไลบรารี bme280.py จาก
   https://raw.githubusercontent.com/robert-hh/BME280/master/bme280.py
   แล้วอัปโหลดไฟล์นี้ลงในบอร์ด ESP32 (ผ่าน Thonny หรือ esptool) หรือตามวิธีในหนังสือนี้

2. เขียนโปรแกรมตัวอย่างเพื่ออ่านค่าอุณหภูมิ ความชื้น และความกดอากาศ

```
from machine import Pin, I2C
import bme280
import time
# สร้างออบเจกต์ I2C โดยใช้ SDA = GPIO21, SCL = GPIO22
i2c = I2C(0, scl=Pin(22), sda=Pin(21))
# สร้างออบเจกต์ BME280
sensor = bme280.BME280(i2c=i2c)
while True:
    temp, pressure, humidity = sensor.read_compensated_data()
    print("Temp: {:.2f} °C, Pressure: {:.2f} hPa, Humidity: {:.2f} %".format(
        temp/100, pressure/100, humidity/1024))
    time.sleep(1)
```

หมายเหตุ:

- ค่า temp ที่อ่านได้ต้องหาร 100 เพื่อแปลงเป็น °C
- ค่า pressure ต้องหาร 100 เพื่อแปลงเป็น hPa
- ค่า humidity ต้องหาร 1024 เพื่อแปลงเป็น %RH

ข้อควรระวัง

หากไม่ได้ติดตั้งไฟล์ bme280.py ลงในบอร์ด ESP32 จะไม่สามารถ import และใช้งานเซ็นเซอร์นี้ได้

6.5 การสื่อสารกับเซ็นเซอร์ผ่าน SPI (ภาพรวมเบื้องต้น)

SPI (Serial Peripheral Interface) คือโปรโตคอลสื่อสารแบบอนุกรมอีกชนิดหนึ่งที่นิยมใช้สำหรับเชื่อมต่อระหว่างไมโครคอนโทรลเลอร์กับอุปกรณ์ภายนอก เช่น หน่วยความจำแฟลช (Flash Memory), หน้าจอแสดงผลความละเอียดสูง (TFT/LCD), โมดูล SD Card, เซ็นเซอร์บางประเภท รวมถึงชิปควบคุม LED Matrix หรืออุปกรณ์ที่ต้องการความเร็วในการส่งข้อมูลสูง

โครงสร้างและหลักการของ SPI

SPI แตกต่างจาก I2C ตรงที่เป็นการสื่อสารแบบ Full Duplex (ส่งและรับข้อมูลพร้อมกันได้) และใช้งานสายสัญญาณมากกว่า โดยมีสายหลัก 4 เส้น ได้แก่

- **MOSI (Master Out Slave In):** ส่งข้อมูลจาก Master ไปยัง Slave
- **MISO (Master In Slave Out):** ส่งข้อมูลจาก Slave กลับไปยัง Master
- **SCK (Serial Clock):** สัญญาณนาฬิกาที่ Master สร้างให้ทุกการส่งข้อมูล
- **CS (Chip Select) หรือ SS (Slave Select):** ใช้สำหรับเลือกอุปกรณ์ Slave ที่ต้องการสื่อสาร (ในระบบที่มี Slave หลายตัว จะต้องมี CS แยกแต่ละอุปกรณ์)

บน ESP32 จะมีขาที่รองรับ SPI โดยตรงอยู่หลายชุด โดยขา SPI หลัก (VSPI) ที่นิยมใช้มีดังนี้

- SCK = GPIO18
- MOSI = GPIO23
- MISO = GPIO19
- CS (Chip Select) สามารถเลือกขา GPIO ใดก็ได้ที่ว่าง

ข้อดีของ SPI คือรองรับการส่งข้อมูลความเร็วสูงกว่า I2C, ไม่มีการชน Address เพราะใช้ขา CS ในการเลือกอุปกรณ์, และโค้ดที่ควบคุมอุปกรณ์มักจะเขียนง่ายและเสถียร

ข้อเสีย คือใช้สายสัญญาณมากกว่า I2C, แต่ละอุปกรณ์ต้องใช้ขา CS แยกกัน

ตัวอย่างอุปกรณ์ที่ใช้ SPI:

- จอ OLED/LED ความละเอียดสูง
- SD Card
- โมดูลเซ็นเซอร์บางรุ่น (เช่น MAX6675, MAX7219, MCP3008)

การใช้งานใน MicroPython

MicroPython มีคลาส machine.SPI สำหรับสร้างออบเจกต์ SPI และใช้งานได้คล้ายกับ I2C เช่นกัน ตัวอย่างโค้ดสั้น ๆ สำหรับสร้างออบเจกต์ SPI ใน ESP32

```
from machine import Pin, SPI
spi = SPI(1, baudrate=10000000, polarity=0, phase=0,
          sck=Pin(18), mosi=Pin(23), miso=Pin(19))
```

เมื่อใช้งานร่วมกับอุปกรณ์แต่ละชนิด (เช่น SD Card หรือจอ LED Matrix) จะต้องกำหนดขา Chip Select (CS) เพิ่มเติม และใช้ไลบรารีเฉพาะสำหรับอุปกรณ์นั้น

สรุป

SPI เป็นโปรโตคอลสื่อสารที่สำคัญและใช้งานอย่างกว้างขวางในงานอิเล็กทรอนิกส์ยุคใหม่ โดยเหมาะกับอุปกรณ์ที่ต้องการส่งข้อมูลปริมาณมากหรือความเร็วสูง ในบทต่อไปจะมีตัวอย่างการใช้งานจริงกับอุปกรณ์ที่รองรับ SPI

6.6 การแสดงผลข้อมูล

6.6.1 หน้าจอ OLED (SSD1306)

หน้าจอ OLED รุ่น SSD1306 เป็นโมดูลแสดงผลขนาดเล็กที่ได้รับความนิยมสูงสุดสำหรับใช้งานร่วมกับบอร์ด ESP32 และบอร์ดไมโครคอนโทรลเลอร์อื่น ๆ ในงาน IoT หรือโครงงานอิเล็กทรอนิกส์ที่ต้องการแสดงผลข้อมูลแบบง่าย ๆ โดยจุดเด่นของหน้าจอชนิดนี้คือ ความคมชัดสูง ประหยัดพลังงาน ใช้สายสัญญาณน้อย และราคาย่อมเยา

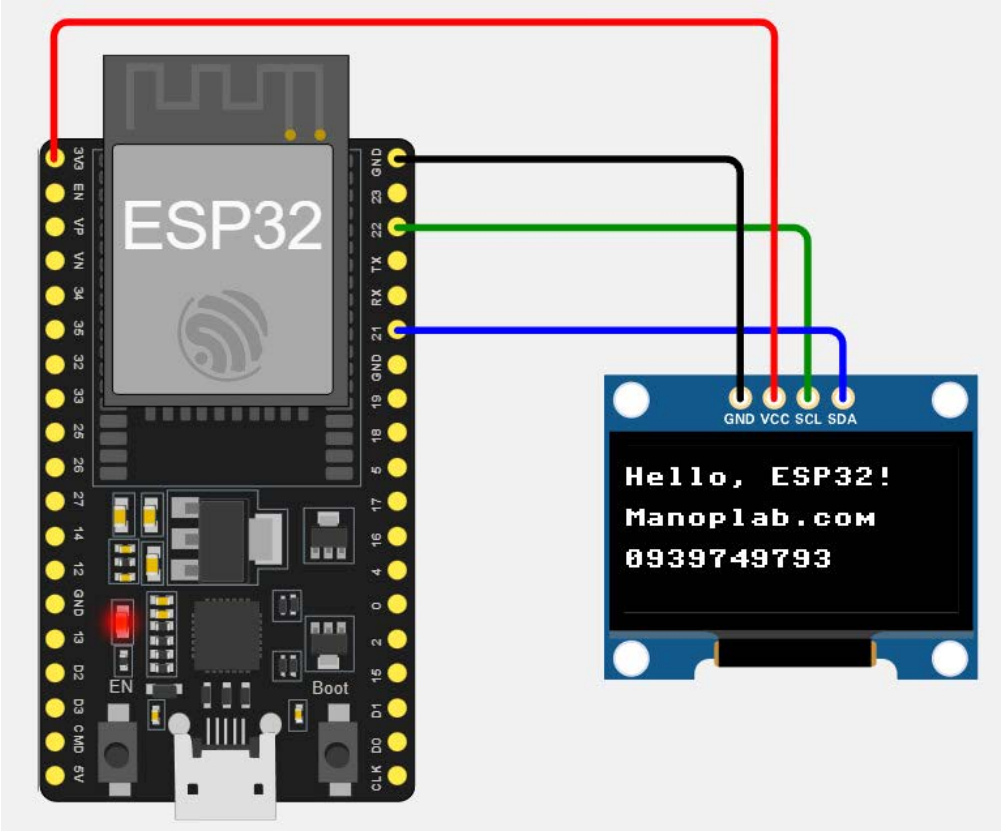
จอ OLED SSD1306 ที่ใช้งานในหนังสือเล่มนี้เป็นแบบขาว-ดำ (Monochrome) ขนาด 0.96 นิ้ว ความละเอียด 128x64 พิกเซล (หรือ 128x32 ในบางรุ่น) แผงจอจะแสดงผลด้วยสีขาว, ฟ้า หรือเหลืองบนพื้นหลังสีดำ แล้วแต่รุ่นที่เลือกใช้ แม้จะเห็นสีแตกต่างกัน แต่การแสดงผลข้อมูลจะเป็นแบบ 1-bit คือ “เปิด” หรือ “ปิด” เท่านั้น

ข้อดีของจอ OLED SSD1306 คือรองรับการเชื่อมต่อแบบ I2C ใช้สายสัญญาณเพียงสองเส้น (SDA และ SCL) ทำให้เดินสายง่ายและเหลือขา GPIO สำหรับอุปกรณ์อื่นอีกมาก นอกจากนี้ MicroPython ยังมีไลบรารีสำเร็จรูปสำหรับ SSD1306 ทำให้การแสดงผลข้อความ ตัวเลข หรือกราฟิกบนจอเป็นเรื่องง่ายสำหรับมือใหม่

การเลือกใช้จอชนิดนี้ในหนังสือ เพราะต้องการให้ผู้อ่านหาซื้ออุปกรณ์ได้ง่าย ติดตั้งและทดลองใช้งานได้จริงทุกขั้นตอน อีกทั้งยังเหมาะกับโครงงานที่ใช้พลังงานจากแบตเตอรี่ เนื่องจากกินไฟต่ำและมีขนาดกะทัดรัด

การเชื่อมต่อขา (I2C)

- VCC ต่อกับ 3.3V ของ ESP32
- GND ต่อกับ GND ของ ESP32
- SDA ต่อกับ GPIO21 ของ ESP32
- SCL ต่อกับ GPIO22 ของ ESP32



ตัวอย่างโค้ดแสดงข้อความบนหน้าจอ OLED SSD1306 อย่างง่าย

```
from machine import Pin, I2C
import ssd1306

i2c = I2C(0, scl=Pin(22), sda=Pin(21))
oled = ssd1306.SSD1306_I2C(128, 64, i2c)

oled.fill(0)
oled.text("Hello, ESP32!", 0, 8)        # บรรทัดที่ 1
oled.text("Manoplab.com", 0, 24)    # บรรทัดที่ 2
oled.text("0939749793", 0, 40)         # บรรทัดที่ 3
oled.show()
```

หมายเหตุ: ก่อนใช้งานต้องอัปโหลดไฟล์ ssd1306.py ลงในบอร์ดด้วย (ดาวน์โหลดได้จาก GitHub)

6.6.2 7-Segment Display (เช่น โมดูล TM1637) 7-Segment Display เป็นอุปกรณ์แสดงผลตัวเลข (และตัวอักษรบางตัว) ที่นิยมใช้กันมานาน ประกอบด้วย LED 7 ส่วน (Segment) ที่เรียงกันเป็นเลข 8 และอีก

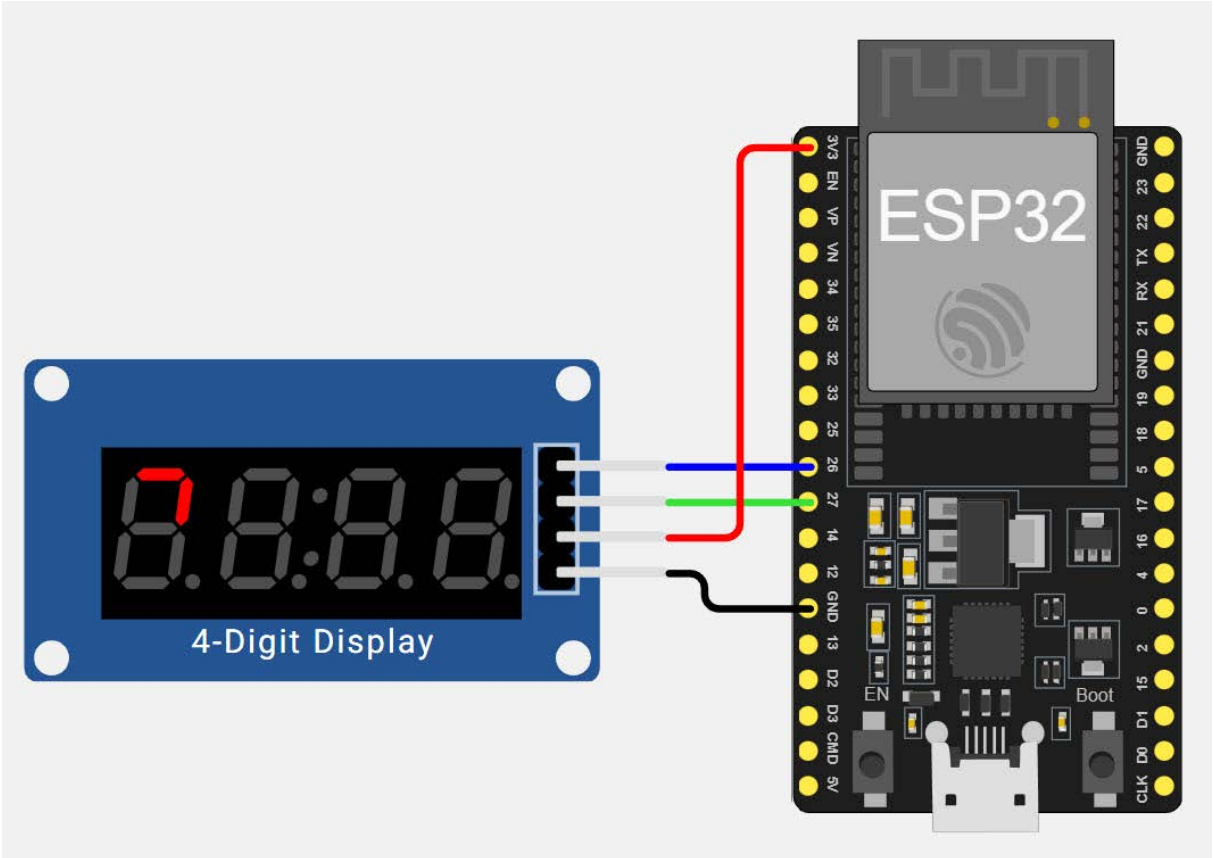
ส่วนหนึ่งสำหรับจุดทศนิยม (Decimal Point) การขับ 7-Segment โดยตรงหลายๆ หลักอาจต้องใช้ขา GPIO จำนวนมากและมีการทำ Multiplexing ที่ซับซ้อน โมดูลที่ใช้ชิปขับจอเช่น TM1637 จึงเป็นที่นิยมเพราะใช้งานง่าย ควบคุมผ่านสายสัญญาณเพียง 2 เส้น (คล้าย I2C แต่ไม่ใช่ I2C มาตรฐาน)

- **โมดูล TM1637:** มักจะมี 4 ขา คือ VCC, GND, CLK (Clock), DIO (Data I/O)
- **การติดตั้งไลบรารี:** ค้นหาและติดตั้งไลบรารี micropython-tm1637 ผ่าน Thonny หรือโหลดได้ที่ท้ายเล่ม

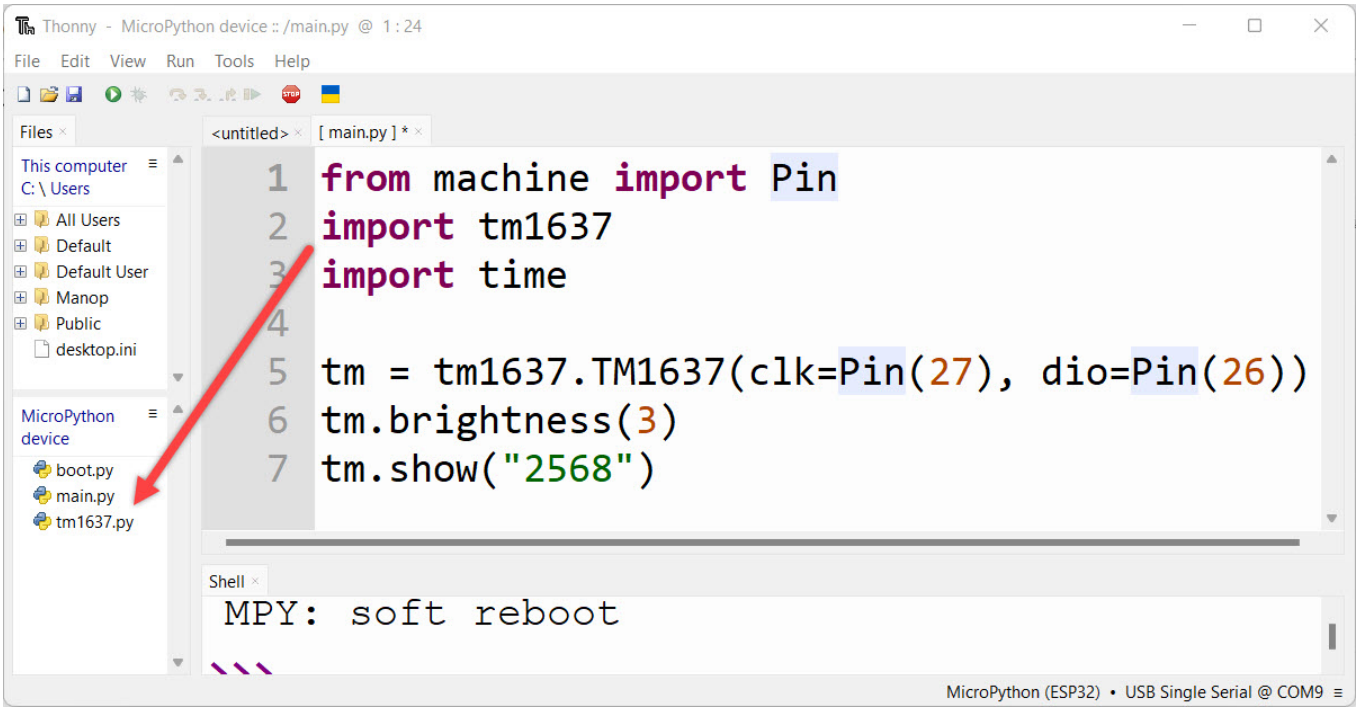
หรือ 📥 URL ดาวน์โหลดไฟล์ไลบรารี TM1637 สำหรับ MicroPython:

https://raw.githubusercontent.com/mcauser/micropython-tm1637/master/tm1637.py

- **การต่อวงจร:**
    - VCC -> 3.3V หรือ 5V ของ ESP32 (ขึ้นอยู่กับโมดูล)
    - GND -> GND ของ ESP32
    - CLK -> ขา GPIO ของ ESP32 (เช่น GPI26) สามารถเปลี่ยนใช้ขาอื่นได้
    - DIO -> ขา GPIO ของ ESP32 (เช่น GPI27) สามารถเปลี่ยนใช้ขาอื่นได้

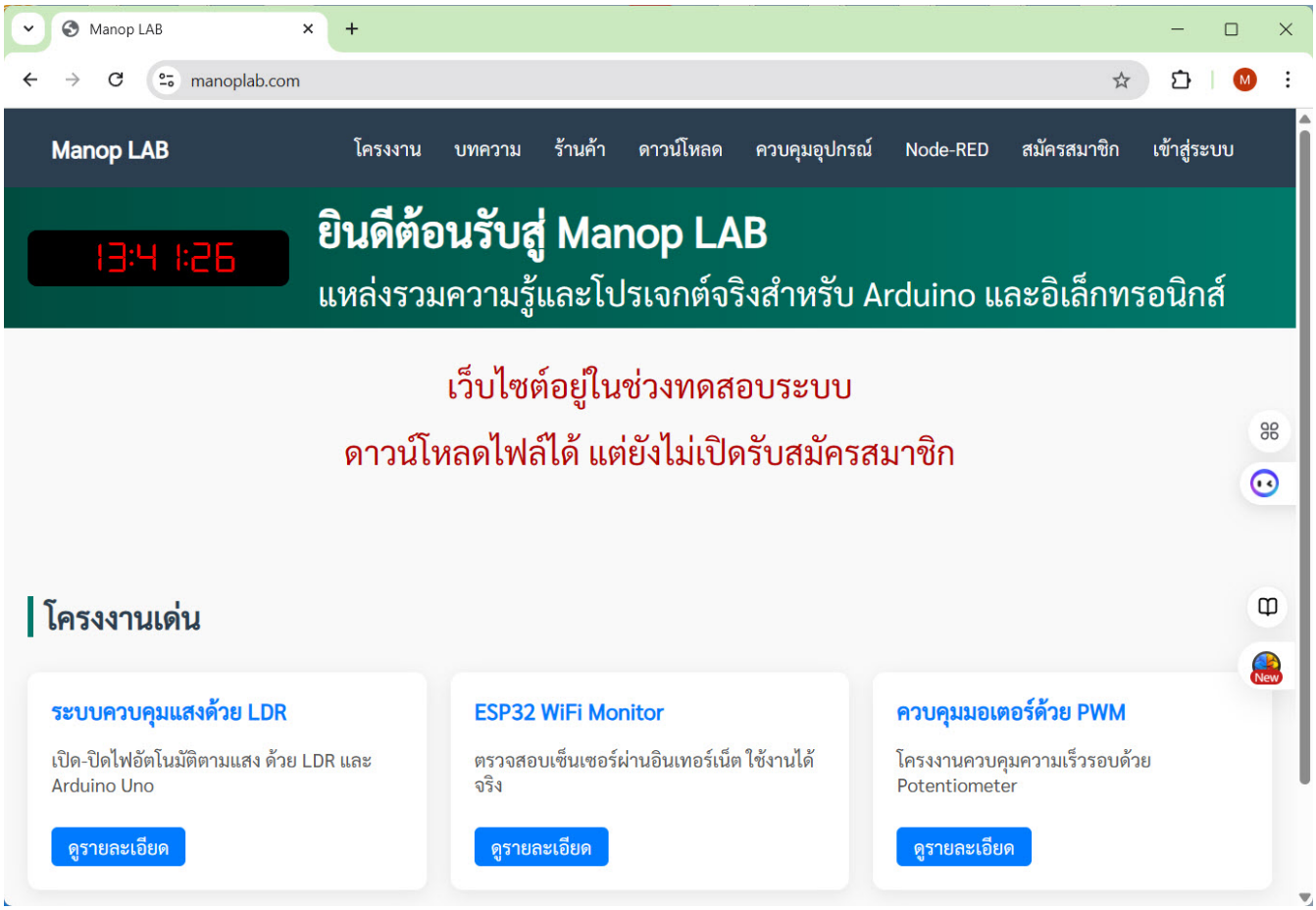


โค้ดตัวอย่าง:



ตัวอย่างโค้ดการแสดงผลแบบง่าย

โปรเจกต์และโค้ดต่าง ๆ ในหนังสือ ให้ดาวน์โหลดได้จากเว็บไซต์ manoplab.com



<untitled> *

```python
import machine, time, tm1637
clk = machine.Pin(27)
dio = machine.Pin(26)
display = tm1637.TM1637(clk=clk, dio=dio)
display.brightness(6)
# แสดงไฟวิ่งทีละหลัก
def sweep():
    seg = [0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F]
    for i in range(4):
        d = [0] * 4
        for s in seg:
            d[i] = s
            display.write(d)
            time.sleep(0.2)
# งูเลื้อยไป-กลับ
def snake():
    # Segment bitmask
    A = 0x01  # ด้านบน
    B = 0x02  # ขวาบน
    C = 0x04  # ขวาล่าง
    D = 0x08  # ล่าง
    E = 0x10  # ซ้ายล่าง
    F = 0x20  # ซ้ายบน
    # งูเลื้อยวนตามขอบจอแบบตามเข็มนาฬิกา
    path = [
        (0, A), (1, A), (2, A), (3, A),  # ขอบบน (ซ้าย→ขวา)
        (3, B), (3, C), (3, D),          # ขอบขวา (บน→ล่าง)
        (2, D), (1, D), (0, D),          # ขอบล่าง (ขวา→ซ้าย)
        (0, E), (0, F)                   # ขอบซ้าย (ล่าง→บน)
    ]

    while True:
        for digit, seg in path:
            frame = [0x00] * 4
            frame[digit] = seg
            display.write(frame)
            time.sleep(0.15)
# คำกระพริบ
def flash():
    for w in ["HELO", "OPEN", "WAIT", "LOVE"]:
        for _ in range(2):
            display.show(w)
            time.sleep(0.5)
            display.show("    ")
            time.sleep(0.2)
# แถบโหลดด้วย L
def bar():
    for s in ["    ", "L   ", "LL  ", "LLL ", "LLLL", "LLL ", "LL  ", "L   "]:
        display.show(s)
        time.sleep(0.15)
# ตัวเลขเคลื่อนที่
def scroll():
    for i in range(10):
        display.show(" {}  ".format(i))
        time.sleep(0.1)
    for i in range(9, -1, -1):
        display.show("  {} ".format(i))
        time.sleep(0.1)
# หัวใจเต้น
def beat():
    for _ in range(3):
        display.show("LOVE")
        time.sleep(0.2)
        display.show("    ")
        time.sleep(0.1)
while True:
    sweep()
    snake()
    flash()
    bar()
    scroll()
    beat()
```

6.6.3 LED Matrix (เช่น 8x8 กับ MAX7219)

โปรเจกต์ LED Matrix ดูและดาวน์โหลดโค้ดได้ที่เว็บ manoplab.com

6.7 การควบคุมอุปกรณ์เอาต์พุตเพิ่มเติม

นอกเหนือจาก LED แล้ว ESP32 ยังสามารถควบคุมอุปกรณ์เอาต์พุตอื่นๆ เพื่อสร้างเสียงหรือการเคลื่อนไหวได้

6.7.1 Buzzer (ตัวกำเนิดเสียง) Buzzer เป็นอุปกรณ์ที่ใช้สร้างเสียง มีสองประเภทหลัก:

- **Active Buzzer:**
  - มีวงจรกำเนิดเสียง (Oscillator) อยู่ภายในตัว
  - เพียงแค่จ่ายไฟ (เช่น 3.3V หรือ 5V) ก็จะส่งเสียงบี๊บต่อเนื่องทันที
  - การควบคุม: ต่อขาหนึ่งเข้ากับ GND และอีกขาหนึ่งเข้ากับขา GPIO ของ ESP32 ที่ตั้งเป็น Output เมื่อสั่งให้ GPIO เป็น HIGH (หรือ LOW ขึ้นอยู่กับการต่อ) Buzzer ก็จะส่งเสียง
  - เหมาะสำหรับการแจ้งเตือนง่ายๆ ที่ไม่ต้องการเปลี่ยนโทนเสียง

```
import machine
import time
active_buzzer_pin = machine.Pin(26, machine.Pin.OUT)
print("Active Buzzer Test (ON for 2 seconds)")
active_buzzer_pin.on() # หรือ active_buzzer_pin.value(1)
time.sleep(2)
active_buzzer_pin.off() # หรือ active_buzzer_pin.value(0)
```

- **Passive Buzzer:**
  - ไม่มีวงจรกำเนิดเสียงภายในตัว เปรียบเสมือนลำโพงขนาดเล็ก
  - ต้องใช้สัญญาณ PWM (Pulse Width Modulation) ที่มีความถี่ต่างๆ ป้อนเข้าไปเพื่อสร้างเสียงโทนต่างๆ
  - การควบคุม: ต่อขาหนึ่งเข้ากับ GND และอีกขาหนึ่งเข้ากับขา GPIO ของ ESP32 ที่สามารถสร้าง PWM ได้

```
import machine
import time

passive_buzzer_pin_num = 25
# สร้างออบเจกต์ PWM สำหรับ Passive Buzzer
# ความถี่ (freq) จะกำหนดโทนเสียง, Duty Cycle (duty) จะกำหนดความดัง (โดยทั่วไปตั้งไว้ที่ 50% หรือ 512)
buzzer_pwm = machine.PWM(machine.Pin(passive_buzzer_pin_num), freq=440, duty=0) # เริ่มด้วยเสียงเงียบ

def play_tone(pwm_obj, frequency, duration_ms, duty_cycle=512):
    if frequency > 0:
        pwm_obj.freq(frequency)
        pwm_obj.duty(duty_cycle) # 50% duty cycle (0-1023)
    time.sleep_ms(duration_ms)
    pwm_obj.duty(0) # ปิดเสียง

print("Passive Buzzer Test - Playing tones")
try:
    # โน้ต C4 (ประมาณ 261 Hz), E4 (ประมาณ 329 Hz), G4 (ประมาณ 392 Hz)
    play_tone(buzzer_pwm, 261, 300) # C4
    time.sleep_ms(50)
    play_tone(buzzer_pwm, 329, 300) # E4
    time.sleep_ms(50)
    play_tone(buzzer_pwm, 392, 300) # G4
    time.sleep_ms(500)

    # Simple melody
    melody = [(262,200), (294,200), (330,200), (349,200), (392,400)] # freq, duration
```

```
    for note in melody:
        play_tone(buzzer_pwm, note[0], note[1])
        time.sleep_ms(50)

except KeyboardInterrupt:
    print("Program stopped by user.")
finally:
    buzzer_pwm.deinit() # ปิด PWM
    print("Passive Buzzer test finished.")
```

6.8 การเลือกใช้เซ็นเซอร์ อุปกรณ์แสดงผล และอุปกรณ์เอาต์พุต

ในบทนี้ ผู้อ่านได้เรียนรู้วิธีการค้นหาและติดตั้งไลบรารีสำหรับ MicroPython, การทำงานกับเซ็นเซอร์ประเภทต่างๆ (ดิจิทัล, แอนะล็อก, I2C), การแสดงผลข้อมูลด้วยหน้าจอ OLED, 7-Segment Display, และ LED Matrix, รวมถึงการควบคุมอุปกรณ์เอาต์พุตเพิ่มเติมเช่น Buzzer และ DC Motor ขนาดเล็ก

การมีความเข้าใจในอุปกรณ์เหล่านี้และวิธีการเชื่อมต่อและควบคุมด้วย ESP32 และ MicroPython เป็นทักษะที่สำคัญอย่างยิ่งในการสร้างโครงงาน IoT ที่มีความสามารถรอบด้าน สามารถรับรู้ข้อมูลจากสภาพแวดล้อม, ประมวลผล, แสดงผล, และโต้ตอบกับโลกทางกายภาพได้

ปัจจัยในการเลือกอุปกรณ์:

- **ฟังก์ชันการทำงาน:** อุปกรณ์นั้นสามารถทำงานได้ตรงตามความต้องการของโครงงานหรือไม่
- **อินเทอร์เฟซการสื่อสาร:** Digital, Analog, I2C, SPI, PWM ฯลฯ (ต้องเข้ากันได้กับ ESP32)
- **ความซับซ้อนในการใช้งานและไลบรารี:** มีไลบรารี MicroPython รองรับหรือไม่ หรือต้องเขียนโค้ดควบคุมเองในระดับต่ำ
- **การใช้พลังงาน, ขนาด, และราคา:** เหมาะสมกับข้อจำกัดของโครงงานหรือไม่

ในบทสุดท้าย (บทที่ 7) เราจะนำความรู้ทั้งหมดที่ได้เรียนมาตั้งแต่บทที่ 1 จนถึงบทนี้ มาสร้างเป็นโครงงาน IoT ที่สมบูรณ์ยิ่งขึ้นหลายๆ แบบ เพื่อให้เห็นภาพรวมของการประยุกต์ใช้ MicroPython และ ESP32 ในการแก้ปัญหาหรือสร้างสรรค์สิ่งประดิษฐ์จริง

บทที่ 7: โปรเจกต์ตัวอย่างสำหรับผู้เริ่มต้น (ทำตามได้จริง)

รวมพลังความรู้สู่โครงงานจริง

จากเนื้อหาที่ผ่านมาทั้งหมด ผู้อ่านได้เรียนรู้ตั้งแต่พื้นฐานของ MicroPython และ ESP32, การควบคุมขา GPIO, การเชื่อมต่อ Wi-Fi และการสื่อสารผ่านเครือข่าย, ไปจนถึงการทำงานร่วมกับเซ็นเซอร์และอุปกรณ์เสริมยอดนิยมต่างๆ ในบทนี้ ถึงเวลาแล้วที่เราจะนำองค์ความรู้เหล่านั้นมาบูรณาการเข้าด้วยกันเพื่อสร้างสรรค์โครงงาน (Project) ที่สามารถจับต้องได้และนำไปประยุกต์ใช้งานจริงได้

การลงมือทำโปรเจกต์ถือเป็นขั้นตอนที่สำคัญอย่างยิ่งในการเรียนรู้ เพราะจะช่วยให้:

- **ทบทวนความรู้:** ได้นำทฤษฎีและคำสั่งต่างๆ ที่เรียนมากลับมาใช้งานจริง
- **เข้าใจการประยุกต์ใช้:** เห็นภาพว่าแต่ละส่วนประกอบทำงานร่วมกันได้อย่างไรเพื่อบรรลุเป้าหมายที่ใหญ่ขึ้น
- **ฝึกฝนการแก้ปัญหา:** การทำโปรเจกต์มักจะพบเจอปัญหาหรือข้อผิดพลาดที่ไม่คาดคิด ซึ่งการแก้ไขปัญหาเหล่านั้นจะเสริมสร้างทักษะและความเข้าใจให้ลึกซึ้งยิ่งขึ้น
- **สร้างแรงบันดาลใจ:** การเห็นผลงานที่สร้างขึ้นด้วยตนเองเป็นรูปธรรมจะสร้างความภาคภูมิใจและกระตุ้นให้อยากเรียนรู้และพัฒนาต่อไป

ในบทนี้ ในตอนแรกนั้นผู้เขียนจะนำเสนอโปรเจกต์ตัวอย่าง 5 โครงงาน ที่เริ่มจากง่ายไปสู่ซับซ้อนขึ้นตามลำดับ โดยแต่ละโปรเจกต์จะมีคำอธิบายวัตถุประสงค์, รายการอุปกรณ์ที่ต้องใช้, ขั้นตอนการต่อวงจร, การเขียนโค้ด MicroPython โดยละเอียด, และแนวทางการทดสอบผลลัพธ์ และเน้นขั้นตอนที่ชัดเจน, โค้ดที่เข้าใจง่าย, และมีส่วนกระตุ้นให้ผู้อ่านได้ทดลองต่อยอด

แต่เนื่องจากข้อจำกัดในเรื่องของรูปเล่มที่หนาแล้ว ด้วยเนื้อหามากทำให้ต้นทุนการพิมพ์เพิ่มขึ้น ผู้เขียนจึงขอนำโปรเจกต์ไปเขียนไว้ในเว็บไซต์ของผู้เขียนเอง และแจ้งช่องทางการดาวน์โหลดไฟล์ไว้ในหนังสือเล่มนี้แทน โดยจะเอาไว้ที่หน้าสุดท้ายของหนังสือ การทำเช่นนี้ผู้เขียนคิดว่ามีความยืดหยุ่นมากกว่าเพราะสามารถเพิ่มเติมรายละเอียดและเพิ่มจำนวนของโปรเจกต์ได้ในอนาคต

สามารถเข้าชมและดาวน์โหลดไฟล์โปรเจกต์ต่าง ๆ ได้ที่ manoplab.com

---

บทที่ 8: สิ่งที่เรียนรู้ต่อไปและแหล่งข้อมูลเพิ่มเติม

8.1 การเดินทางที่ยังไม่สิ้นสุด

ขอแสดงความยินดีด้วยครับ! หากผู้อ่านได้เดินทางมาถึงบทนี้ นั่นหมายความว่าท่านได้ผ่านการเรียนรู้เนื้อหาสำคัญทั้งหมดเกี่ยวกับการใช้งาน ESP32 ร่วมกับ MicroPython ตั้งแต่พื้นฐานการเขียนโปรแกรม Python, การควบคุมขา GPIO, การทำงานกับเซ็นเซอร์และอุปกรณ์เสริมต่างๆ, การเชื่อมต่อเครือข่าย Wi-Fi, การสร้าง Web Server, ไปจนถึงการลงมือทำโปรเจกต์จริงหลายโครงงาน คุณได้เห็นแล้วว่า ESP32 และ MicroPython เป็นเครื่องมือที่ทรงพลังและยืดหยุ่นเพียงใดในการสร้างสรรค์สิ่งประดิษฐ์และโครงงาน Internet of Things (IoT) อย่างไรก็ตาม โลกของเทคโนโลยีและอิเล็กทรอนิกส์นั้นกว้างใหญ่และมีการพัฒนาอยู่เสมอ ความรู้ที่ได้จากหนังสือเล่มนี้เปรียบเสมือนก้าวแรกที่มั่นคง และเป็นประตูสู่การเรียนรู้ในระดับที่สูงขึ้นและซับซ้อนยิ่งขึ้น บทสุดท้ายนี้มีจุดมุ่งหมายเพื่อชี้แนะแนวทางให้ผู้อ่านสามารถศึกษาค้นคว้าด้วยตนเองต่อไป รวมถึงแนะนำแหล่งข้อมูลที่เป็นประโยชน์ และให้กำลังใจในการสร้างสรรค์โครงงานใหม่ๆ อย่างไม่หยุดยั้ง

8.2 ก้าวสู่หัวข้อขั้นสูง (ภาพรวมเบื้องต้น)

มีหัวข้อและเทคโนโลยีอีกมากมายที่เกี่ยวข้องกับ ESP32 และ MicroPython ซึ่งน่าสนใจและสามารถนำไปต่อยอดความสามารถของโครงงานได้อย่างมาก ในส่วนนี้จะแนะนำบางหัวข้อในภาพรวมเพื่อให้ผู้อ่านเห็นแนวทาง:

8.2.1 การสื่อสารไร้สาย Bluetooth Low Energy (BLE) บน ESP32

นอกเหนือจาก Wi-Fi แล้ว ESP32 ยังรองรับการสื่อสารไร้สายแบบ Bluetooth Low Energy (BLE) ซึ่งเป็นเทคโนโลยีที่ออกแบบมาเพื่อการสื่อสารระยะใกล้ที่ประหยัดพลังงาน เหมาะสำหรับอุปกรณ์สวมใส่ (Wearables), iBeacons, การส่งข้อมูลเซ็นเซอร์ขนาดเล็กไปยังสมาร์ตโฟนหรือคอมพิวเตอร์, หรือการควบคุมอุปกรณ์ที่ไม่ต้องการ Bandwidth สูง

- **BLE คืออะไร?** เป็นมาตรฐานย่อยของ Bluetooth ที่เน้นการใช้พลังงานต่ำ ทำให้เหมาะกับอุปกรณ์ที่ทำงานด้วยแบตเตอรี่

BLE หรือ **Bluetooth Low Energy** เป็นมาตรฐานย่อยของเทคโนโลยี Bluetooth ที่ถูกออกแบบมาให้เน้นการใช้พลังงานต่ำเป็นพิเศษ แตกต่างจาก Bluetooth แบบคลาสสิกที่ใช้พลังงานสูงกว่าและมุ่งเน้นไปที่การสตรีมข้อมูล เช่น เสียงหรือไฟล์ขนาดใหญ่ BLE ถูกออกแบบมาเพื่อการส่งข้อมูลขนาดเล็กในช่วงเวลาสั้น ๆ แต่ทำได้อย่างต่อเนื่อง จึงทำให้อุปกรณ์ที่ใช้แบตเตอรี่ขนาดเล็กสามารถทำงานได้ยาวนานเป็นเดือนหรือเป็นปีโดยไม่ต้องชาร์จใหม่

คุณสมบัติเด่นของ BLE อยู่ที่การเปิด–ปิดการสื่อสารได้อย่างรวดเร็ว ใช้พลังงานในโหมดสแตนด์บายต่ำมาก และรองรับการเชื่อมต่อแบบหลายอุปกรณ์ (multi-device) เหมาะสำหรับการนำไปใช้กับอุปกรณ์ประเภท **IoT (Internet of Things)** เช่น เซ็นเซอร์ไร้สาย สมาร์ตวอทช์ เครื่องวัดสุขภาพ อุปกรณ์ติดตาม รวมถึงอุปกรณ์ที่ต้องการการสื่อสารแบบเรียลไทม์แต่ไม่ต้องการส่งข้อมูลปริมาณมาก

ด้วยคุณสมบัตินี้ BLE จึงกลายเป็นมาตรฐานหลักสำหรับการสื่อสารไร้สายระยะสั้นในงานที่ต้องการทั้งความสะดวกและการประหยัดพลังงาน โดยเฉพาะอย่างยิ่งอุปกรณ์ที่ขับเคลื่อนด้วยแบตเตอรี่ ซึ่งต้องการยืดอายุการใช้งานให้ยาวนานที่สุด

- **แนวคิดหลัก:** BLE แนวคิดการทำงานหลักของ BLE อยู่บนพื้นฐานของ **โปรไฟล์ GATT (Generic Attribute Profile)** ซึ่งเป็นมาตรฐานในการกำหนดรูปแบบข้อมูลและวิธีการสื่อสารระหว่างอุปกรณ์ BLE ทั้งสองฝั่ง GATT แบ่งโครงสร้างข้อมูลออกเป็นสองระดับหลักคือ **Services** และ **Characteristics**
  - **Service (บริการ)** คือกลุ่มของฟังก์ชันที่เกี่ยวข้องกัน โดยทำหน้าที่เหมือนเป็น “หมวดหมู่” ของข้อมูล ตัวอย่างเช่น Service สำหรับ Heart Rate, Service สำหรับ Battery Level หรือ Service ที่เราสร้างขึ้นเองสำหรับอ่านค่าอุณหภูมิจากเซ็นเซอร์
  - **Characteristic (คุณลักษณะ)** คือหน่วยข้อมูลย่อยภายใน Service ซึ่งจะเก็บค่าหรือสถานะจริง ๆ ที่ต้องการแลกเปลี่ยน เช่น ค่าอัตราการเต้นของหัวใจในหน่วย bpm, ค่าเปอร์เซ็นต์แบตเตอรี่, หรือค่าที่วัดได้จากเซ็นเซอร์อุณหภูมิ โดย Characteristic สามารถกำหนดสิทธิ์การเข้าถึงได้หลายแบบ เช่น **Read**, **Write**, หรือ **Notify**

นอกจากโครงสร้างของข้อมูลแล้ว BLE ยังมีการแบ่งบทบาทของอุปกรณ์ออกเป็นสองประเภทที่ชัดเจน:

- **Central (Client)** คืออุปกรณ์ที่ทำหน้าที่เริ่มการเชื่อมต่อ ค้นหา และดึงข้อมูล เช่น สมาร์ตโฟน แท็บเล็ต หรือคอมพิวเตอร์
- **Peripheral (Server)** คืออุปกรณ์ที่ประกาศตัวเองและให้บริการข้อมูล เช่น ESP32 ที่ทำหน้าที่เป็นเซ็นเซอร์วัดอุณหภูมิแล้วส่งข้อมูลออกมาให้ Central อ่าน

การที่ BLE ออกแบบโครงสร้างเช่นนี้ทำให้นักพัฒนาสามารถสร้างโครงงานที่ซับซ้อน แต่ยังคงเป็นมาตรฐานและเข้ากันได้กับอุปกรณ์ BLE อื่น ๆ ทั่วโลกได้อย่างง่ายดาย

- **MicroPython กับ BLE:** MicroPython มีโมดูล ubluetooth (หรือ bluetooth ในบาง build) ที่ช่วยให้สามารถเขียนโปรแกรมควบคุมการทำงานของ BLE บน ESP32 ได้ เช่น การสร้าง ESP32 ให้เป็น BLE

Peripheral เพื่อ Advertise (ประกาศ) ตัวเอง หรือสร้าง Service และ Characteristic ให้ Central device (เช่น สมาร์ตโฟน) เข้ามาอ่านหรือเขียนค่าได้

- **ตัวอย่างการใช้งาน:** สร้างเซ็นเซอร์วัดอุณหภูมิที่ส่งค่าผ่าน BLE ไปยังแอปพลิเคชันบนมือถือ, สร้างรีโมทคอนโทรล BLE,

การเรียนรู้ BLE จะเปิดมิติใหม่ให้กับการสร้างโครงงานที่ต้องการการเชื่อมต่อไร้สายแบบประหยัดพลังงาน

การโฆษณาชื่ออุปกรณ์ (Advertising)

ให้ ESP32 โฆษณาชื่อ เช่น ESP32-BLE แล้วเห็นทันทีในแอป BLE Scanner / nRF Connect

```
# MicroPython (ESP32): BLE Advertising แสดงชื่ออุปกรณ์
import bluetooth
import time


def adv_payload(name: str) -> bytes:
    b = name.encode()
    # 0x09 = Complete Local Name
    return bytes([len(b) + 1, 0x09]) + b


ble = bluetooth.BLE()
ble.active(True)


NAME = "Manop-LAB"   #ตั้งชื่ออุปกรณ์ที่ต้องการให้แสดง
ble.gap_advertise(100_000, adv_payload(NAME))  # 100_000 µs = 100 ms


while True:
    time.sleep(1)
```

ทดสอบ: เปิดแอปสแกน BLE บนมือถือ ⟶ เห็น ESP32-BLE ในรายการสแกน

แนะนำให้ใช้แอพพลิเคชัน **nRF Connect for Mobile ดาวน์โหลดได้จาก play store**

GATT Server (Write ⟶ ควบคุม LED)

แนะนำให้ใช้แอพพลิเคชัน **nRF Connect for Mobile ดาวน์โหลดได้จาก play store**

สร้าง Service/Characteristic ที่รับคำสั่ง ON / OFF จากมือถือเพื่อควบคุม LED

```
# MicroPython (ESP32): BLE GATT Server ควบคุม LED ด้วยคำสั่งที่เขียนเข้ามา (Write)
import bluetooth
from machine import Pin
from micropython import const
import time

# IRQ events
_IRQ_CENTRAL_CONNECT    = const(1)
_IRQ_CENTRAL_DISCONNECT = const(2)
_IRQ_GATTS_WRITE        = const(3)

led = Pin(2, Pin.OUT)   # เปลี่ยนขาตามบอร์ดที่ใช้งานจริง (เช่น 17 หรือ 18)
                        #เพราะบอร์ด ESP32 38PIN จะไม่มี LED มาให้
def adv_payload(name: str) -> bytes:
    b = name.encode()
    return bytes([len(b) + 1, 0x09]) + b  # 0x09 = Complete Local Name

ble = bluetooth.BLE()
ble.active(True)

# UUID กำหนดเอง (เปลี่ยนได้)
SERVICE_UUID = bluetooth.UUID("12345678-1234-5678-1234-56789abcdef0")
CHAR_UUID    = bluetooth.UUID("12345678-1234-5678-1234-56789abcdef1")

# Characteristic: READ (อ่านสถานะล่าสุด), WRITE (รับคำสั่งจากมือถือ)
CHAR_CMD = (CHAR_UUID, bluetooth.FLAG_READ | bluetooth.FLAG_WRITE)
```

```
SERVICE  = (SERVICE_UUID, (CHAR_CMD,))
handles  = ble.gatts_register_services([SERVICE])
handle_cmd = handles[0][1][0]

ble.gatts_write(handle_cmd, b"READY")  # ค่าเริ่มต้นเมื่อกด Read

def on_ble(event, data):
    if event == _IRQ_CENTRAL_CONNECT:
        # ต่อสำเร็จ
        pass
    elif event == _IRQ_CENTRAL_DISCONNECT:
        # กลับมาโฆษณาใหม่เมื่อหลุดการเชื่อมต่อ
        ble.gap_advertise(100_000, adv_payload("ESP32-LED"))
    elif event == _IRQ_GATTS_WRITE:
        conn_handle, attr_handle = data
        if attr_handle == handle_cmd:
            cmd = ble.gatts_read(handle_cmd).strip().upper()
            if cmd == b'ON':
                led.value(1)
                ble.gatts_write(handle_cmd, b"OK:ON")
            elif cmd == b'OFF':
                led.value(0)
                ble.gatts_write(handle_cmd, b"OK:OFF")
            else:
                ble.gatts_write(handle_cmd, b"ERR")  # คำสั่งอื่น

ble.irq(on_ble)

# เริ่มโฆษณา
```

```
ble.gap_advertise(100_000, adv_payload("ESP32-LED"))


# วนรอ
while True:
    time.sleep(1)
```

วิธีทดสอบย่อ: Scan ⟶ Connect ⟶ เปิด Service/Characteristic ⟶ **Write** ON หรือ OFF (ASCII) ⟶ LED ติด/ดับ และ Read จะเห็น OK:ON/OK:OFF

8.2.2 การทำงานกับไฟล์บน ESP32 (File System)

หน่วยความจำ Flash ภายในของ ESP32 ไม่เพียงแต่ใช้เก็บเฟิร์มแวร์ MicroPython และโค้ดโปรแกรมเท่านั้น แต่ยังสามารถแบ่งส่วนหนึ่งมาใช้เป็น **ระบบไฟล์ (File System)** ขนาดเล็กได้ (โดยทั่วไปคือ FAT file system) ทำให้เราสามารถสร้าง, อ่าน, เขียน, และลบไฟล์ได้เหมือนกับการทำงานกับไฟล์บนคอมพิวเตอร์ทั่วไป

- **การใช้งาน:** MicroPython มีโมดูล os ที่มีฟังก์ชันสำหรับการจัดการไฟล์และไดเรกทอรี (เช่น os.listdir(), os.mkdir(), os.remove(), os.rename()) รวมถึงฟังก์ชันในตัวของ Python เช่น open(), read(), write(), close() สำหรับการทำงานกับเนื้อหาไฟล์โดยตรง
- **ประโยชน์:**
    - **การบันทึกข้อมูล (Data Logging):** บันทึกค่าที่อ่านได้จากเซ็นเซอร์ลงในไฟล์ CSV หรือ Text file เป็นระยะๆ เพื่อนำไปวิเคราะห์ภายหลัง
    - **การเก็บค่าการตั้งค่า (Configuration Storage):** เก็บค่าการตั้งค่าต่างๆ ของโปรแกรม (เช่น SSID/Password ของ Wi-Fi, ค่า Threshold ของเซ็นเซอร์) ไว้ในไฟล์ เพื่อให้โปรแกรมสามารถโหลดค่าเหล่านี้เมื่อเริ่มต้นทำงาน หรือเพื่อให้ผู้ใช้สามารถแก้ไขค่าเหล่านี้ได้โดยไม่ต้องแก้ไขโค้ดหลัก
    - **การให้บริการไฟล์ผ่าน Web Server:** เก็บไฟล์ HTML, CSS, JavaScript, หรือรูปภาพขนาดเล็กไว้ในระบบไฟล์ของ ESP32 แล้วให้ Web Server ที่สร้างขึ้นอ่านไฟล์เหล่านี้ไปแสดงผลบนเบราว์เซอร์ ทำให้สามารถสร้างหน้าเว็บที่ซับซ้อนและสวยงามขึ้นได้

ตัวอย่างโค้ดการเขียนและอ่านไฟล์อย่างง่าย:

main.py *

```
ตัวอย่างโค้ดการเขียนและอ่านไฟล์อย่างง่าย:
Python
# เขียนข้อมูลลงไฟล์
try:
    with open('sensor_log.txt', 'a') as f: # 'a' for append mode
        f.write("Temperature: 25.5 C, Humidity: 60%\n")
    print("Data written to sensor_log.txt")
except Exception as e:
    print(f"Error writing to file: {e}")

# อ่านข้อมูลจากไฟล์
try:
    with open('sensor_log.txt', 'r') as f: # 'r' for read mode
        content = f.read()
        print("File content:")
        print(content)
except Exception as e:
    print(f"Error reading file: {e}")
```

8.2.3 การประหยัดพลังงานด้วยโหมด Deep Sleep

สำหรับโครงงานที่ต้องการทำงานด้วยพลังงานจากแบตเตอรี่เป็นระยะเวลานาน (เช่น เซ็นเซอร์ไร้สายที่ติดตั้งในพื้นที่ห่างไกล) การจัดการพลังงานเป็นสิ่งสำคัญอย่างยิ่ง ESP32 มีโหมดการทำงานที่ประหยัดพลังงานหลายระดับ และโหมดที่ประหยัดพลังงานได้มากที่สุดคือ **Deep Sleep**

- **Deep Sleep คืออะไร?** ในโหมดนี้ CPU หลักและส่วนประกอบส่วนใหญ่ของ ESP32 จะถูกปิดการทำงาน เหลือเพียง RTC (Real-Time Clock) controller และ ULP (Ultra Low Power) co-processor (ถ้ามีการใช้งาน) ที่ยังคงทำงานอยู่ ทำให้กินกระแสไฟฟ้าต่ำมาก (ในระดับไมโครแอมป์)
- **การใช้งานใน MicroPython:** ใช้ฟังก์ชัน machine.deepsleep(time_ms) โดย time_ms คือระยะเวลา (มิลลิวินาที) ที่ต้องการให้ ESP32 อยู่ในโหมด Deep Sleep หากไม่ระบุ time_ms ESP32 จะอยู่ใน Deep Sleep จนกว่าจะมีสัญญาณปลุกจากภายนอก
- **วิธีการปลุก (Wake-up Sources):**
    - **Timer:** ตั้งเวลาให้ RTC ปลุก ESP32 ขึ้นมาทำงานเมื่อครบกำหนดเวลาที่ตั้งไว้
    - **External Wake-up:** ใช้สัญญาณไฟฟ้าจากภายนอก (เช่น การกดปุ่ม, สัญญาณจากเซ็นเซอร์) ที่ต่อเข้ากับขา RTC GPIO ที่กำหนดไว้ เพื่อปลุก ESP32
    - **Touch Pad Wake-up:** ใช้การสัมผัสที่ Touch Pad ของ ESP32 (ถ้ามี)

- **ULP Co-processor Wake-up:** เขียนโปรแกรมให้ ULP co-processor ตรวจสอบเงื่อนไขบางอย่าง แล้วปลุก CPU หลักเมื่อเงื่อนไขนั้นเป็นจริง

ตัวอย่างโค้ด Deep Sleep และปลุกด้วย Timer:

main.py *

```python
import machine
import time

print("Woke up from sleep (or first boot)!")
# ทำงานที่ต้องการ เช่น อ่านเซ็นเซอร์, ส่งข้อมูล
print("Doing some work...")
time.sleep(5) # สมมติว่าทำงาน 5 วินาที

sleep_duration_ms = 20 * 1000 # ต้องการให้ Sleep เป็นเวลา 20 วินาที
print(f"Going to deep sleep for {sleep_duration_ms / 1000} seconds...")
machine.deepsleep(sleep_duration_ms)
```

หมายเหตุ: หลังจาก Deep Sleep แล้ว ESP32 จะรีสตาร์ทโปรแกรมใหม่ทั้งหมด สถานะของตัวแปรต่างๆ จะไม่ถูกเก็บไว้ ยกเว้นข้อมูลที่เก็บไว้ใน RTC Memory (ต้องใช้เทคนิคเฉพาะในการเข้าถึง)

8.2.4 หัวข้ออื่นๆ ที่น่าสนใจ

- **Interrupts (การขัดจังหวะ):** ช่วยให้ ESP32 สามารถตอบสนองต่อเหตุการณ์ภายนอก (เช่น การเปลี่ยนแปลงสถานะของ Pin) ได้ทันทีโดยไม่ต้องคอยตรวจสอบใน Loop ตลอดเวลา (machine.Pin.irq())
- **Timers (ตัวตั้งเวลา):** สร้าง Timer เพื่อให้ฟังก์ชันบางอย่างทำงานเป็นระยะๆ หรือหลังจากหน่วงเวลาที่กำหนด โดยไม่กระทบกับการทำงานหลักของโปรแกรม (machine.Timer)
- **ULP Co-processor Programming:** (ขั้นสูง) เขียนโปรแกรมด้วย Assembly สำหรับ ULP co-processor เพื่อให้ทำงานบางอย่างในขณะที่ CPU หลักอยู่ใน Deep Sleep เช่น การอ่านค่าจากเซ็นเซอร์อย่างช้าๆ แล้วปลุก CPU หลักเมื่อค่าเกิน Threshold

8.3 เคล็ดลับในการแก้ไขปัญหาที่พบบ่อย (Troubleshooting Tips)

การพัฒนาโครงงานอิเล็กทรอนิกส์และซอฟต์แวร์มักจะพบเจอปัญหาต่างๆ การมีแนวทางในการแก้ไขปัญหา (Troubleshooting) ที่ดีจะช่วยให้สามารถค้นหาสาเหตุและแก้ไขได้อย่างรวดเร็ว

- **เริ่มต้นจากง่ายไปซับซ้อน:** หากโปรเจกต์ไม่ทำงาน ให้ลองแยกส่วนประกอบออกมาทดสอบทีละส่วน เช่น ทดสอบการทำงานของเซ็นเซอร์เดี่ยวๆ, ทดสอบการเชื่อมต่อ Wi-Fi, ทดสอบการแสดงผลบนจอ ก่อนจะนำมารวมกัน

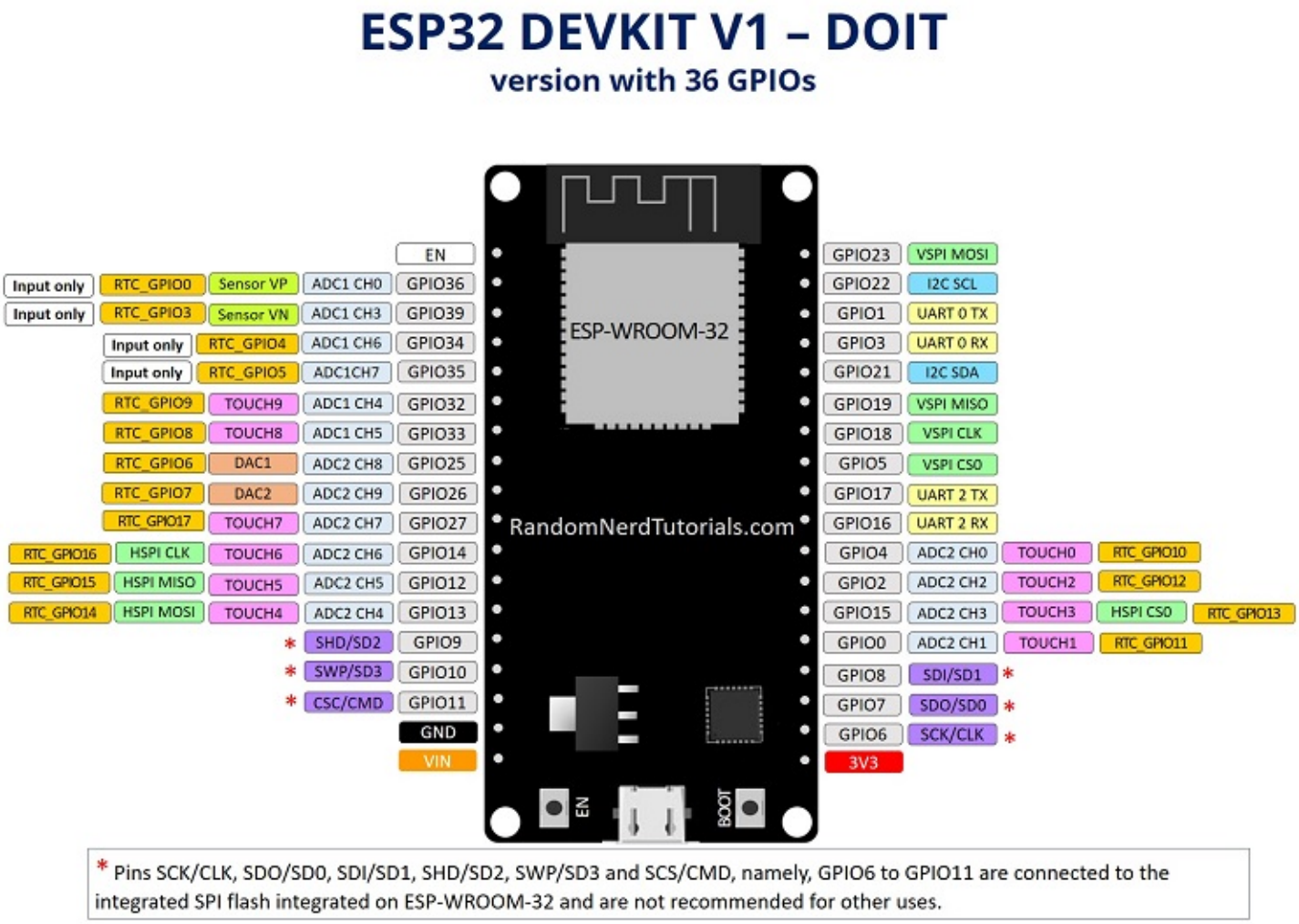
- **ใช้ print() อย่างชาญฉลาด:** คำสั่ง print() เป็นเพื่อนที่ดีที่สุดในการ Debugging แทรก print() เข้าไปในจุดต่างๆ ของโค้ดเพื่อดูค่าของตัวแปร หรือดูว่าโปรแกรมทำงานมาถึงจุดนั้นหรือไม่
- **ตรวจสอบการเชื่อมต่อฮาร์ดแวร์ซ้ำๆ:** ปัญหาหลายครั้งเกิดจากสายไฟหลุด/หลวม, ต่อผิดขา, ขาดการเชื่อมต่อ GND ร่วมกัน, หรือปัญหาเรื่องแรงดันไฟฟ้า ตรวจสอบวงจรให้ละเอียดอีกครั้ง เทียบกับ Diagram ที่ถูกต้อง
- **อ่านข้อความ Error อย่างละเอียด:** เมื่อ MicroPython เกิด Error มันจะแสดงข้อความ (Traceback) บอกว่า Error เกิดขึ้นที่ไฟล์ไหน บรรทัดใด และเป็น Error ประเภทใด พยายามทำความเข้าใจข้อความนั้น มันมักจะชี้เบาะแสไปที่สาเหตุของปัญหา
- **ปัญหาการเชื่อมต่อ Wi-Fi:**
    - ตรวจสอบ SSID และ Password ให้ถูกต้อง (Case-sensitive)
    - ตรวจสอบความแรงของสัญญาณ Wi-Fi ESP32 อาจจะอยู่ไกลจาก Router เกินไป
    - ตรวจสอบการตั้งค่าของ Router (เช่น MAC filtering, จำนวน Client สูงสุด)
- **MemoryError:** เกิดขึ้นเมื่อ ESP32 ใช้หน่วยความจำ RAM จนหมด
    - พยายามลดการใช้ตัวแปรขนาดใหญ่ หรือ Buffer ที่ไม่จำเป็น
    - เรียกใช้ import gc; gc.collect() เพื่อให้ Garbage Collector ทำงานเคลียร์หน่วยความจำที่ไม่ได้ใช้แล้ว
    - สำหรับโปรแกรมขนาดใหญ่ ลองพิจารณาการใช้ Frozen Modules หรือคอมไพล์โค้ดเป็น .mpy file
- **ImportError: no module named 'xxx':** ไม่พบไลบรารีที่ต้องการ import
    - ตรวจสอบว่าได้ติดตั้งไลบรารีนั้นลงใน ESP32 ถูกต้องหรือไม่ (อยู่ใน /lib หรือ root)
    - ตรวจสอบการสะกดชื่อโมดูลให้ถูกต้อง (Case-sensitive)
- **ปัญหาจาก Power Supply:** หาก ESP32 ทำงานผิดปกติหรือรีสตาร์ทเองบ่อยๆ อาจเกิดจากแหล่งจ่ายไฟไม่เพียงพอ (โดยเฉพาะเมื่อมีการใช้งาน Wi-Fi หรืออุปกรณ์ที่กินกระแสสูง) ลองใช้แหล่งจ่ายไฟที่มีคุณภาพดีและจ่ายกระแสได้เพียงพอ
- **ใจเย็นและ methodical:** เมื่อเจอปัญหา อย่าเพิ่งท้อแท้ ลองแยกแยะปัญหาเป็นส่วนเล็กๆ และทดสอบทีละอย่าง ค่อยๆ ไล่หาสาเหตุไปทีละขั้นตอน

ภาคผนวก ก

แผนผังขา (Pinout Diagram) ของ ESP32 รุ่นยอดนิยม

การทราบตำแหน่งและหน้าที่ของขาต่างๆ บนบอร์ด ESP32 เป็นสิ่งสำคัญอย่างยิ่งในการต่อวงจรและเขียนโปรแกรมควบคุมฮาร์ดแวร์ บอร์ด ESP32 ที่นิยมใช้กันอย่างแพร่หลายคือรุ่น ESP32-DevKitC ซึ่งมีหลายเวอร์ชันย่อย แต่โดยทั่วไปจะมีชุดขาหลักๆ ที่คล้ายคลึงกัน

ESP32-DevKitC Pinout (ตัวอย่างทั่วไปสำหรับบอร์ด 30 หรือ 38 ขา)



ด้านล่างนี้เป็นคำอธิบายขาที่สำคัญบางส่วน โดยอ้างอิงจากบอร์ด ESP32-DevKitC ที่ใช้ชิป ESP32-WROOM-32 หรือ ESP32-WROVER (หมายเลขขาอาจแตกต่างกันเล็กน้อยในบอร์ดจากผู้ผลิตรายอื่น ควรตรวจสอบกับเอกสารของบอร์ดที่ใช้งานจริงเสมอ):

ขาไฟเลี้ยง (Power Pins):

- **3V3 (หรือ 3.3V):** ขาจ่ายแรงดันไฟฟ้า 3.3V ออกจากตัวควบคุมแรงดันไฟฟ้า (Voltage Regulator) บนบอร์ด ใช้สำหรับจ่ายไฟให้กับเซ็นเซอร์และโมดูลอื่นๆ ที่ทำงานด้วยแรงดัน 3.3V
- **GND:** ขากราวด์ (Ground) ของวงจร เป็นจุดอ้างอิงแรงดัน 0V ต้องต่อ GND ของ ESP32 เข้ากับ GND ของอุปกรณ์อื่นๆ ที่เชื่อมต่อด้วยเสมอ
- **VIN (หรือ 5V, VUSB):** ขารับแรงดันไฟฟ้าเข้า หรือจ่ายแรงดันไฟฟ้าออกจากแหล่งจ่ายไฟ USB (ประมาณ 5V) สามารถใช้จ่ายไฟให้กับ ESP32 (ถ้าไม่ได้จ่ายผ่านพอร์ต USB) หรือใช้เป็นแหล่งจ่าย 5V ให้กับอุปกรณ์อื่นที่ต้องการ (เช่น โมดูล Relay, Servo motor บางรุ่น) *ข้อควรระวัง: หากจ่ายไฟเข้าที่ขานี้ ควรมีวงจรป้องกันที่เหมาะสม*
- **EN (Enable) / CH_PD (Chip Power Down):** ขาสำหรับเปิด/ปิดการทำงานของชิป ESP32 โดยทั่วไปจะถูกดึงขึ้น (Pull-up) ด้วยตัวต้านทานเพื่อให้ชิปทำงาน หากดึงลง GND ชิปจะหยุดทำงาน (เป็นขา Reset ชนิดหนึ่ง) ปุ่ม RESET หรือ EN บนบอร์ดมักจะต่อกับขานี้

ขาควบคุมและโปรแกรม (Control and Programming Pins):

- **GPIO0 (BOOT):** เป็นขา Strapping Pin ที่มีความสำคัญในการเข้าโหมดโปรแกรม (Flashing Mode หรือ Download Mode) โดยปกติเมื่อต้องการอัปโหลดเฟิร์มแวร์หรือโปรแกรมใหม่ จะต้องดึงขานี้ลง GND (โดยการกดปุ่ม BOOT ค้างไว้) ขณะที่ทำการรีเซ็ตหรือจ่ายไฟให้บอร์ด ในโหมดการทำงานปกติ ขานี้สามารถใช้เป็น GPIO ทั่วไปได้
- **GPIO2:** เป็นขา Strapping Pin อีกขาหนึ่ง มักจะมี LED สีฟ้าออนบอร์ดต่ออยู่กับขานี้ (สำหรับ ESP32-DevKitC) และเกี่ยวข้องกับการเลือกโหมดบูตเริ่มต้น

ขา GPIO (General Purpose Input/Output) ทั่วไป: ESP32 มีขา GPIO จำนวนมากที่สามารถกำหนดค่าให้เป็น Input หรือ Output เพื่อเชื่อมต่อกับเซ็นเซอร์, LED, สวิตช์, และอุปกรณ์อื่นๆ ได้ ขา GPIO ส่วนใหญ่สามารถทำหน้าที่ได้หลายอย่าง (Multiplexed) เช่น:

- **Digital Input/Output:** อ่านหรือส่งสัญญาณดิจิทัล (HIGH/LOW)
- **Analog Input (ADC):** ขาที่สามารถอ่านค่าแรงดันไฟฟ้าแอนะล็อกได้ เช่น:
    - ADC1_CH0 (GPIO36/SVP), ADC1_CH3 (GPIO39/SVN), ADC1_CH6 (GPIO34), ADC1_CH7 (GPIO35), ADC1_CH4 (GPIO32), ADC1_CH5 (GPIO33)
    - ADC2_CH0 (GPIO4), ADC2_CH1 (GPIO0 - *ใช้งานด้วยความระมัดระวัง*), ADC2_CH2 (GPIO2 - *ใช้งานด้วยความระมัดระวัง*), ADC2_CH3 (GPIO15), ADC2_CH4 (GPIO13), ADC2_CH5

(GPIO12), ADC2_CH6 (GPIO14), ADC2_CH7 (GPIO27), ADC2_CH8 (GPIO25), ADC2_CH9 (GPIO26)

- **ข้อควรจำ:** ADC2 ไม่สามารถใช้งานได้เมื่อ Wi-Fi ทำงานอยู่ ควรใช้ ADC1 หากมีการใช้งาน Wi-Fi พร้อมกัน ขา GPIO36 (SVP) และ GPIO39 (SVN) เป็นขา Input only และไม่มี Pull-up/Pull-down ภายใน

- **Touch Sensor Pins:** ขาที่สามารถใช้เป็นเซ็นเซอร์สัมผัสแบบ Capacitive ได้ เช่น:
  - TOUCH0 (GPIO4), TOUCH1 (GPIO0), TOUCH2 (GPIO2), TOUCH3 (GPIO15), TOUCH4 (GPIO13), TOUCH5 (GPIO12), TOUCH6 (GPIO14), TOUCH7 (GPIO27), TOUCH8 (GPIO33), TOUCH9 (GPIO32)
- **PWM (Pulse Width Modulation):** ขา GPIO ส่วนใหญ่สามารถสร้างสัญญาณ PWM ได้ (ดูรายละเอียดในบทที่ 4)

ขาสำหรับการสื่อสารแบบอนุกรม (Serial Communication Pins):

- **I2C (Inter-Integrated Circuit):**
  - **SDA (Serial Data):** ขาข้อมูลสำหรับ I2C โดยทั่วไปคือ **GPIO21**
  - **SCL (Serial Clock):** ขาสัญญาณนาฬิกาสำหรับ I2C โดยทั่วไปคือ **GPIO22**
  - ESP32 มี I2C controller 2 ชุด (I2C0 และ I2C1) ทำให้สามารถกำหนดขาอื่นๆ ให้เป็น I2C ได้หากจำเป็น
- **SPI (Serial Peripheral Interface):** ESP32 มี SPI controller หลายชุด (SPI0, SPI1 หรือ HSPI, SPI2 หรือ VSPI)
  - **VSPI (ทั่วไปใช้สำหรับ User):**
    - **MOSI (Master Out Slave In):** GPIO23
    - **MISO (Master In Slave Out):** GPIO19
    - **SCLK (Serial Clock):** GPIO18
    - **CS (Chip Select):** GPIO5 (หรือขา GPIO อื่นๆ ที่ผู้ใช้กำหนด)
  - **HSPI (ทั่วไปใช้สำหรับ User):**
    - **MOSI:** GPIO13
    - **MISO:** GPIO12
    - **SCLK:** GPIO14

        - **CS:** GPIO15

- **UART (Universal Asynchronous Receiver/Transmitter):** ESP32 มี UART controller 3 ชุด (UART0, UART1, UART2)
    - **UART0:**
        - **TXD0 (Transmit Data):** GPIO1
        - **RXD0 (Receive Data):** GPIO3
        - (UART0 มักใช้สำหรับการสื่อสารผ่าน USB (Serial Monitor) และการอัปโหลดโปรแกรม)
    - **UART1:** (ขาอาจจะถูกใช้โดย Flash memory ในบางโมดูล ต้องตรวจสอบ Datasheet)
        - **TXD1:** GPIO10 (มักใช้กับ SD_DATA2)
        - **RXD1:** GPIO9 (มักใช้กับ SD_DATA3)
    - **UART2:**
        - **TXD2:** GPIO17
        - **RXD2:** GPIO16

ขา RTC GPIOs: เป็นขา GPIO ที่สามารถทำงานได้แม้ในขณะที่ ESP32 อยู่ในโหมด Deep Sleep และสามารถใช้เป็นแหล่งปลุก (Wake-up source) ได้ เช่น GPIO0, GPIO2, GPIO4, GPIO12-15, GPIO25-27, GPIO32-39

ตาราง Pinout (ตัวอย่างสำหรับ ESP32-DevKitC ทั่วไป) ดูหน้าถัดไป

GPIO	Input	Output	Notes
0	pulled up	OK	outputs PWM signal at boot, must be LOW to enter flashing mode
1	TX pin	OK	debug output at boot
2	OK	OK	connected to on-board LED, must be left floating or LOW to enter flashing mode
3	OK	RX pin	HIGH at boot
4	OK	OK	
5	OK	OK	outputs PWM signal at boot, strapping pin
6	x	x	connected to the integrated SPI flash
7	x	x	connected to the integrated SPI flash
8	x	x	connected to the integrated SPI flash
9	x	x	connected to the integrated SPI flash
10	x	x	connected to the integrated SPI flash
11	x	x	connected to the integrated SPI flash
12	OK	OK	boot fails if pulled high, strapping pin
13	OK	OK	
14	OK	OK	outputs PWM signal at boot
15	OK	OK	outputs PWM signal at boot, strapping pin
16	OK	OK	
17	OK	OK	
18	OK	OK	
19	OK	OK	
21	OK	OK	
22	OK	OK	
23	OK	OK	
25	OK	OK	
26	OK	OK	
27	OK	OK	
32	OK	OK	
33	OK	OK	
34	OK		input only
35	OK		input only
36	OK		input only
39	OK		input only

หมายเหตุสำคัญ:

- **Strapping Pins:** ขา GPIO0, GPIO2, GPIO4, GPIO5, GPIO12, GPIO15 มีบทบาทในการกำหนดค่าเริ่มต้นของชิป ESP32 ขณะบูต (Boot-up configuration) การต่อวงจรกับขาเหล่านี้ควรทำด้วยความระมัดระวัง หากสถานะของขาเหล่านี้ขณะบูตไม่ถูกต้อง อาจทำให้ ESP32 ไม่ทำงานตามที่คาดหวัง หรือไม่สามารถอัปโหลดโปรแกรมได้
- **ADC2 และ Wi-Fi:** ขอย้ำอีกครั้งว่า ขาที่ใช้ ADC2 จะไม่สามารถใช้งานได้เมื่อ Wi-Fi เปิดใช้งานอยู่ หากโครงงานจำเป็นต้องใช้ Wi-Fi และ ADC พร้อมกัน ควรเลือกใช้ขา ADC1
- **ตรวจสอบเอกสารของบอร์ดเสมอ:** แผนผังขาข้างต้นเป็นตัวอย่างทั่วไปสำหรับ ESP32-DevKitC แต่บอร์ดจากผู้ผลิตรายอื่น หรือบอร์ดที่มีคุณสมบัติพิเศษ (เช่น ESP32-S2, S3, C3) อาจมี Pinout ที่แตกต่างกัน ควรตรวจสอบจาก Datasheet หรือ Pinout Diagram ที่ผู้ผลิตบอร์ดนั้นๆ ให้มาเสมอ เพื่อความถูกต้องแม่นยำในการต่อวงจร

การทำความเข้าใจ Pinout ของบอร์ด ESP32 ที่ใช้งาน จะช่วยให้สามารถเลือกใช้ขาต่างๆ ได้อย่างถูกต้องและเต็มประสิทธิภาพ

ภาคผนวก ข

สรุปคำสั่งและโมดูล MicroPython ที่ใช้บ่อย

ส่วนนี้เป็นการสรุปคำสั่งและโมดูลที่สำคัญเพื่อให้ผู้อ่านสามารถทบทวนและใช้อ้างอิงได้

1. กลุ่มคำสั่งโครงสร้างภาษา Python (Core Language)

- **if, elif, else, for, while, break, continue**
- **try, except, finally, raise**
- **def (สร้างฟังก์ชัน), return**
- **class (สร้างคลาส), self**
- **import, from ... import ..., as, pass, global, nonlocal**

---

2. กลุ่มตัวแปรและชนิดข้อมูล

- **int(), float(), str(), bool(), list(), dict(), tuple(), set()**
- **len(), range(), type(), isinstance(), enumerate(), zip(), min(), max(), sum()**

---

3. การทำงานกับสตริง

- **.split(), .join(), .replace(), .find(), .format(), .strip(), .lower(), .upper()**

---

4. คณิตศาสตร์และการสุ่ม

- **import math**
  - **math.sqrt(), math.sin(), math.cos(), math.tan(), math.pi**
- **import random**
  - **random.randint(a, b), random.choice(list)**

---

5. การจัดการไฟล์

- open(), .read(), .write(), .close()
- with open() as f:

---

6. หน่วยความจำและระบบ

- import gc, gc.collect()
- import sys, sys.platform, sys.version

---

7. การหน่วงเวลาและการจับเวลา

- import time
- time.sleep(), time.sleep_ms(), time.sleep_us()
- time.ticks_ms(), time.ticks_us(), time.ticks_diff()

---

8. การจัดการระบบแฟ้ม (Filesystem)

- import os
- os.listdir(), os.remove(), os.rename(), os.mkdir(), os.stat()

---

9. การเชื่อมต่อ Wi-Fi

- import network
- WLAN(), active(), connect(), disconnect(), isconnected(), ifconfig()

---

10. การใช้งาน MQTT

- from umqtt.simple import MQTTClient
- connect(), disconnect(), publish(), subscribe(), check_msg(), wait_msg()

---

11. โมดูลควบคุมขา GPIO (Digital I/O)

- from machine import Pin

- Pin.OUT, Pin.IN, Pin.value(), Pin.on(), Pin.off()

---

12. การอ่านสัญญาณ Analog (ADC)

- from machine import ADC
- ADC.read(), ADC.atten(), ADC.width()

---

13. การสร้างสัญญาณ PWM

- from machine import PWM
- PWM.freq(), PWM.duty(), PWM.deinit()

---

14. การสื่อสาร I2C

- from machine import I2C
- I2C.scan(), I2C.readfrom(), I2C.writeto(), I2C.readfrom_mem(), I2C.writeto_mem()

---

15. การสื่อสาร UART (Serial)

- from machine import UART
- UART.read(), UART.write(), UART.init()

---

16. การสื่อสาร SPI

- from machine import SPI
- SPI.read(), SPI.write(), SPI.init()

---

17. การใช้งาน Timer

- from machine import Timer
- Timer.init(), Timer.deinit()

---

18. Watchdog Timer (WDT)

- **from machine import WDT**
- **WDT.feed()**

---

19. การควบคุมระบบและพลังงาน

- **from machine import deepsleep()**
- **from machine import reset(), reset_cause()**
- **from machine import freq()**

คำส่งท้าย

จงสร้างสรรค์อย่างไม่สิ้นสุด

การเดินทางผ่านหนังสือเล่มนี้ได้สิ้นสุดลงแล้ว แต่การเดินทางในโลกแห่งการสร้างสรรค์ด้วย ESP32 และ MicroPython เพิ่งจะเริ่มต้นขึ้น ความรู้และทักษะที่ได้รับจากหนังสือเล่มนี้เป็นเพียงเครื่องมือพื้นฐานเท่านั้น สิ่งสำคัญที่สุดคือ ความคิดสร้างสรรค์, ความอยากรู้อยากเห็น, และความมุ่งมั่นที่จะเรียนรู้และทดลองสิ่งใหม่ ๆ อย่ากลัวที่จะผิดพลาด เพราะทุกความผิดพลาดคือบทเรียนที่มีค่า จงตั้งคำถาม และอย่าหยุดที่จะค้นหาคำตอบ โลกของเทคโนโลยีเปิดกว้างรอให้เราไปสำรวจและสร้างสรรค์สิ่งประดิษฐ์ที่น่าทึ่งในแบบของเราเอง

ขอให้ทุกท่านสนุกกับการเป็น "Maker" สร้างสรรค์โครงงานที่แก้ปัญหา, อำนวยความสะดวก, หรือเพียงแค่สร้างความสุขให้กับตัวเองและผู้อื่น และหากเป็นไปได้ ลองแบ่งปันความรู้และประสบการณ์ กลับสู่ชุมชน เพื่อช่วยให้ ผู้เริ่มต้นคนอื่นๆ ได้เติบโตและเรียนรู้เช่นเดียวกัน ข้าพเจ้าในฐานะผู้เขียน ขออำนวยพรให้ทุกโครงงานของทุกท่าน เต็มเปี่ยมไปด้วยความสำเร็จในทุกขั้นตอน และเปี่ยมด้วยความสุขตลอดเส้นทางแห่งการเรียนรู้และสร้างสรรค์

มานพ ปักษี

เดือน มิถุนายน พุทธศักราช ๒๕๖๘

ผู้เขียนขอย้ำอีกครั้งว่าไฟล์โปรเจกต์และโค้ดโปรแกรมต่าง ๆ ในหนังสือเล่มนี้สามารถดาวน์โหลดได้ที่เว็บไซต์

manoplab.com

การดาวน์โหลดให้เข้าไปที่เมนู “ไฟล์หนังสือ MicroPython”

ใส่รหัสผ่าน micropythonESP32